Backend Engineering Fundamentals

The Building Blocks of Every Backend, from the Request to Production

Study notes on a backend engineering video course
27 chapters · each with a one-page overview, review questions, a glossary and further topics

Contents

Page numbers point to each chapter's one-page overview; the paper starts on the next page. "Video" is the number in the original playlist.

#	Chapter	Video	Page
1	The HTTP Protocol	5	3
2	Routing	6	15
3	Serialization & Deserialization	7	22
4	Authentication & Authorization	8	29
5	Validation & Transformation	9	41
6	Handlers, Services, Repositories & Middleware	10	49
7	REST API Design	11	59
8	Databases with PostgreSQL	12	71
9	Caching	13	84
10	Task Queues & Background Jobs	14	94
11	Full-Text Search with Elasticsearch	15	104
12	Error Handling & Fault Tolerance	16	112
13	Configuration Management	17	124
14	Logging, Monitoring & Observability	18	133
15	Graceful Shutdown	19	142
16	Backend Security	20	150
17	Scaling & Performance, Part 1	21.1	162
18	Scaling & Performance, Part 2	21.2	174
19	Concurrency & Parallelism	22	187
20	Object Storage, Part 1	23	199
21	Object Storage, Part 2	24	212
22	Real-Time Backends	25	222
23	Testing for Backend Engineers	26	232
24	The Twelve-Factor App	27	245
25	OpenAPI	28	259
26	Webhooks	29	270
27	DevOps for Backend Engineers	30	281
	References		*295*

The HTTP Protocol — One-Page Overview

Chapter 1 · every section and sub-section of the paper at a glance

1 Introduction: Two Ideas at the Heart of HTTP

HTTP keeps no memory between requests, and the client always speaks first.

- **1.1 Statelessness** — Self-contained requests; simple, scalable, crash-tolerant servers.
- **1.2 The client-server model** — Client initiates and supplies everything; server answers. HTTPS = same rules.

2 Transport and Protocol Versions

HTTP needs a reliable transport: TCP up to HTTP/2, QUIC over UDP for HTTP/3.

- **2.1 HTTP over TCP** — Reliability required, not connections; OSI layer 7 is the backend's home.
- **2.2 From HTTP/1.0 to HTTP/3** — Persistent connections, then multiplexing, then QUIC without HOL blocking.

3 Messages and Headers

Start line, headers, blank line, body; headers are metadata written on the parcel.

- **3.1 Anatomy of a message** — Method, URL, version; status line with code and reason phrase.
- **3.2 Why headers exist** — The parcel analogy: metadata readable without opening the body.
- **3.3 Four categories of headers** — Request, general, representation and security headers.
- **3.4 Extensibility and remote control** — New headers without changing the protocol; client steers the server.

4 HTTP Methods

Methods declare the intent of a request and decide whether repeating it is harmless.

- **4.1 Methods express intent** — `GET`, `POST`, `PATCH`, `PUT`, `DELETE`.
- **4.2 PUT versus PATCH** — Complete replacement versus selective update; prefer PATCH.
- **4.3 Idempotency** — GET, PUT, DELETE idempotent; POST creates a new note each time.

5 CORS and the OPTIONS Method

Browsers enforce the same-origin policy; servers relax it with CORS headers.

- **5.1 Same-origin policy and CORS** — Origin = scheme, host and port; cross-origin needs permission.
- **5.2 The simple request flow** — `Origin` in, `Access-Control-Allow-Origin` out, or a CORS error.
- **5.3 Preflighted requests** — Non-simple method, header or content type triggers an OPTIONS inquiry.
- **5.4 Demo in Burp Suite** — Header removed: blocked; PUT with JSON and token: OPTIONS, 204, then PUT.

6 Status Codes

Three-digit codes tell every client the outcome in one standard vocabulary.

- **6.1 Why status codes exist** — No guessing from bodies; client reacts to 401 or 400 directly.
- **6.2 Informational and success** — 100, 101; 200, 201 and 204.
- **6.3 Redirection** — 301 permanent, 302 temporary, 304 not modified.
- **6.4 Client errors** — 400, 401, 403, 404, 405, 409, 429.
- **6.5 Server errors** — 500, 501; 502, 503, 504 from proxies and overload.
- **6.6 Demo** — Each code fired after its preflight; 500 reveals nothing.

7 HTTP Caching

Responses are reused while fresh and revalidated cheaply with validators.

- **7.1 Freshness and validators** — `Cache-Control: max-age=10`, `ETag`, `Last-Modified`.
- **7.2 Conditional requests and 304** — `If-None-Match` 3141 gives 304; after an update, 200 with 2943.
- **7.3 Limits in production** — Stale ETags serve old data; client-side caches as an alternative.

8 Content Negotiation and Compression

Accept headers let the client choose format, language and encoding.

- **8.1 Three kinds of negotiation** — Media type, language and encoding.
- **8.2 Demo: language and format** — English JSON, then Spanish, then Spanish XML.
- **8.3 HTTP compression** — 11,000 entries: 3.8 MB with gzip, 26 MB without.

9 Persistent Connections

Since HTTP/1.1 one TCP connection carries many requests by default.

- **9.1 One connection, many requests** — Avoids repeated TCP setup and teardown.
- **9.2 Connection and Keep-Alive headers** — `timeout`, `max`, and `Connection: close`.

10 Large Requests and Responses

Files go up as multipart bodies; large responses come down as streams.

- **10.1 Uploads with multipart/form-data** — A boundary string delimits the parts of the body.
- **10.2 Streaming responses** — `text/event-stream` and keep-alive deliver chunks.

11 SSL, TLS and HTTPS

HTTPS is HTTP over TLS, the secure successor of SSL.

- **11.1 From SSL to TLS** — Encryption and server authentication with certificates; TLS 1.3.
- **11.2 HTTPS** — Encrypts the browser-server exchange, e.g. login credentials.

HTTP at a glance — a stateless, client-initiated exchange of messages whose methods, headers and status codes carry intent, permissions, cache state and format, with CORS enforced by browsers on top. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Understanding HTTP: The Protocol Every Backend Speaks

Chapter 1 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — This chapter presents HTTP as a backend engineer meets it in daily work. It starts from the protocol's two founding ideas, statelessness and the client-server model, places HTTP above TCP and QUIC, and traces the versions from HTTP/1.0 to HTTP/3. It then dissects request and response messages, the four categories of headers, the methods and their idempotency, and the simple and preflighted CORS flows, demonstrated with an intercepting proxy. The status-code classes are explained code by code, followed by conditional caching with ETag, Last-Modified and 304 responses. The chapter closes with content negotiation and compression, persistent connections, multipart uploads, streamed responses and the roles of SSL, TLS and HTTPS.

Keywords — HTTP, statelessness, HTTP headers, HTTP methods, idempotency, CORS preflight, status codes, conditional caching, content negotiation

1 INTRODUCTION: TWO IDEAS AT THE HEART OF HTTP

HTTP is the protocol through which browsers and other clients send data to servers and receive data from them. Clients and servers can communicate over many protocols, but HTTP is the one found in the great majority of backend code bases, and it rests on two ideas: statelessness and the client–server model.

1.1 Statelessness

HTTP is *stateless*: the server keeps no memory of past interactions. Each request carries everything the server needs to process it — the method, the URL, the headers — and once the server has responded, it forgets the request. The next request from the same client is treated as a new and unrelated event. Requests are therefore *self-contained*: whatever identifies the caller, such as an authentication token or session information, must travel with every request. To return a user's profile, for example, the server needs the client's cookie or token on each request; otherwise it cannot know which user is asking.

The stateless model has three benefits. **Simplicity**: the server does not store per-client session information, which would cost resources and add complexity. **Scalability**: because no single server has to keep track of a session, requests can be distributed freely across many servers. **Resilience**: if a server crashes, no client interaction is damaged, since there is no session or memory of a request that must be restored. Applications still need continuity — a logged-in user, a shopping cart — so developers implement state management on top of HTTP with cookies, sessions or tokens (see Chapter 4).

1.2 The client-server model

Every HTTP exchange has a client and a server. The client, typically a web browser or an application, initiates the communication by sending a request and is responsible for supplying all the information the server needs: the URL of the resource, the headers and, where relevant, a body. The server hosts resources such as websites, APIs or other content and waits for incoming requests. When one arrives, it processes it and sends back an appropriate response: a web page, JSON data, a text file, an error message or any other kind of content. The protocol fixes the direction: communication is always initiated by the client, and the server responds.

In this chapter HTTP and HTTPS are interchangeable. HTTPS is, simplified, a more secure version of HTTP that adds encryption through TLS and certificates; the principles of the protocol are the same. Section 11 summarises what TLS adds.

2 TRANSPORT AND PROTOCOL VERSIONS

2.1 HTTP over TCP

Before any request or response can be sent, client and server must establish a connection, a medium over which messages travel. HTTP does not require its transport protocol to be connection-based; it requires only that the transport be reliable — that it does not lose messages or, at minimum, reports an error when it does. Of the two most common transport protocols on the internet, TCP and UDP, TCP is the reliable one, so HTTP has traditionally relied on TCP, which is connection-based: a connection is opened with TCP's *three-way handshake* before any HTTP bytes are exchanged.

Sending and receiving data over a network is usually described with the OSI model. Backend engineers mostly work in its top layer, layer 7, the *application layer*, where HTTP lives. TCP handshakes, connection establishment and TLS encryption reach into the lower layers and belong mainly to network engineering; they are good to know, but for application work the essential model is short: client and server establish a network connection, and messages are sent and received over it.

2.2 From HTTP/1.0 to HTTP/3

Successive versions of HTTP redefined how clients and servers move messages over connections (Table 1). In **HTTP/1.0** each request opened a new connection, which

was closed after the response; every exchange paid the full cost of establishing and closing a connection, which slowed performance. **HTTP/1.1** introduced *persistent connections*: multiple requests and responses reuse the same TCP connection that was established before the first request, which improved performance significantly (Section 9). It also added chunked transfer encoding and better caching mechanisms.

HTTP/2 introduced *multiplexing*: several requests and responses travel concurrently over a single connection. It uses *binary framing* instead of text, compresses headers with HPACK, and added *server push*, which lets a server send resources before the client requests them (major browsers have since removed support for push). **HTTP/3** is built on *QUIC*, a transport-layer protocol that runs over UDP instead of TCP and provides reliability itself. It establishes connections faster, reduces latency and handles packet loss better. HTTP/3 keeps multiplexing but without *head-of-line blocking*, which HTTP/2 still suffers at the TCP level: one lost TCP packet stalls every stream on the connection until it is retransmitted, whereas QUIC delivers each stream independently.

Version	Transport	Main change
HTTP/1.0	TCP	New connection per request
HTTP/1.1	TCP	Persistent connections, chunked transfer, better caching
HTTP/2	TCP	Binary framing, multiplexing, HPACK, server push
HTTP/3	QUIC over UDP	Faster setup, no head-of-line blocking

Table 1: HTTP versions as summarised in the lecture

3 MESSAGES AND HEADERS

3.1 Anatomy of a message

A *request message* is sent by the client to the server; a *response message* is what the client receives back. Figure 1 shows one of each. The first line of the request holds the **request method** (here `PUT`), the **resource URL** being requested from the server (Chapter 2 shows how a server maps it to code) and the **HTTP version**, here HTTP/1.1. Then come the **headers**, among them `Host`, the domain of the server addressed. A blank line follows the last header to signal that the headers are over and the body begins. The **request body** is the information the client wants to send to the server. The response starts with a status line: the HTTP version, the **status code** and its reason phrase — `200 OK` means the request succeeded. Response headers follow, then a blank line and the **response body**.

```
PUT /api/users/42 HTTP/1.1     <- method, URL, version
Host: api.example.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json
Content-Length: 26

{"name": "Ada", "age": 36}

HTTP/1.1 200 OK                <- version, status, reason
Date: Sun, 01 Nov 2026 10:00:00 GMT
Content-Type: application/json
Cache-Control: no-cache

{"id": 42, "name": "Ada", "age": 36}
```

Figure 1: A request (top) and its response (bottom), each a start line, headers, an empty line and a body (illustrative)

3.2 Why headers exist

Headers are *key–value pairs* of parameters sent with a request or received with a response. Why give this metadata its own section instead of putting the values in the URL or the body? The lecturer answers with a parcel. When we send a parcel, the recipient's name, address, phone number, state and postal code are written on the outside, not packed inside. Everyone who carries the parcel from sender to receiver, through different modes of transport, needs that information; if it were inside, each of them would have to open the parcel to see where it goes, again and again. Keeping the metadata on top gives a quick way to check it. HTTP headers work the same way: proxies, load balancers, caches and the server's own framework can read a message's metadata without parsing its body.

3.3 Four categories of headers

The many headers can be grouped into four categories (Table 2).

Request headers are sent by the client to give the server information about the request. `User-Agent` identifies the kind of client: a browser, Postman, another server or a mobile app. `Authorization` carries credentials such as a bearer token that identify the user. `Accept` states what kind of content the client expects: JSON, plain text or HTML. Request headers help the server understand the client's environment, preferences and capabilities.

General headers appear in both requests and responses and carry metadata about the message itself: the `Date` of the message, caching directives such as `no-cache` or `max-age` in `Cache-Control`, and `Connection`, which says whether to keep the connection alive or close it.

Representation headers describe the representation of the resource being transmitted, whether in a request body or a response. `Content-Type` gives the media type, such as JSON or HTML (Chapter 3 covers serialization); `Content-Length` the size in bytes; `Content-Encoding` any encoding applied, such as gzip or deflate; `ETag` a unique identifier of a version of the resource, used mostly for caching. These headers ensure that client and server know how to interpret and process the body.

Security headers strengthen requests and responses by controlling browser behaviour such as content loading, cookies and encryption. `Strict-Transport-Security` (HSTS) ensures that the client communicates with the server only over HTTPS, preventing protocol-downgrade attacks. `Content-Security-Policy` restricts the sources from which content such as JavaScript, CSS and images can be loaded, which helps prevent cross-site scripting. `X-Frame-Options` prevents the page from being embedded in an iframe, mitigating clickjacking. `X-Content-Type-Options` ensures that the browser does not guess the MIME type of content, preventing MIME-sniffing attacks. `Set-Cookie` with the `HttpOnly` and `Secure` flags makes a cookie inaccessible to JavaScript and ensures it is sent only over HTTPS. Chapter 16 returns to these defences.

Category	Examples	Describes
Request	User-Agent, Authorization, Accept	The client and its wishes
General	Date, Cache-Control, Connection	The message as a whole
Representation	Content-Type, Content-Length, Content-Encoding, ETag	The body
Security	HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Set-Cookie	Browser behaviour

Table 2: The four categories of headers with the lecture's examples

3.4 Extensibility and remote control

Two ideas follow from the header mechanism. The first is **extensibility**. Headers can be added or customised without altering the underlying protocol: adding one piece of metadata can change the whole flow of an interaction. This has made HTTP adaptable to new technologies and use cases. Security enhancements arrived as headers such as `Strict-Transport-Security`, which forces secure connections. Developers create custom headers for their own applications, traditionally named like `X-Custom-Header` (RFC 6648 has since deprecated the `X-` prefix for new headers, though it remains common). Content negotiation arrived through `Accept`, `Accept-Language` and `Accept-Encoding`, which let a server serve different versions of the same content according to the client's preference.

The second idea is that headers act as a **remote control** for the server: they let the client send instructions or preferences that influence how the server processes the request and shapes the response. In content negotiation the client asks for a format with `Accept` and the server answers with HTML if the client asked for HTML, JSON if it asked for JSON. For caching and expiration, the server uses `Cache-Control` or `Expires` to control how long the client may cache a resource. For authentication, the client identifies itself through the `Authorization` header, which drives access-control decisions. Headers thus add capabilities on top of the bare messages.

4 HTTP METHODS

4.1 Methods express intent

HTTP methods exist to represent the different kinds of action that a client — a browser or any API consumer — can request on a server. Instead of every request doing the same thing, the method defines the *intent* of the interaction, which gives each type of action a clear semantic meaning (Table 3). `GET` fetches data from the server and should not modify anything there. `POST` creates data on the server and has a request body, which is the natural way to send the user's data. `PATCH` updates data — for example, a profile page where users change their name — and also carries a body. `PUT` updates data too, but whatever data arrives in its body completely replaces the previous instance of the resource. `DELETE`, as the name says, removes a resource. `HEAD` appears among the simple CORS methods and `OPTIONS` in the preflight (Section 5). Chapter 7 builds REST API design on these semantics.

Method	Intent	Body	Idempotent
GET	Read a resource	No	Yes (safe)
POST	Create a resource	Yes	No
PUT	Replace a resource	Yes	Yes
PATCH	Update selected fields	Yes	Not guaranteed
DELETE	Remove a resource	Usually no	Yes
OPTIONS	Ask for capabilities	No	Yes (safe)

Table 3: The methods discussed in the lecture

4.2 PUT versus PATCH

The difference is the scope of the replacement. PATCH can be thought of as a selective replacement, or an append: the fields in the body change and everything else stays as it was. PUT is a complete replacement: the body becomes the new resource. Developers often use PUT where they should use PATCH, against the semantics; the lecturer's rule of thumb is to use PATCH unless there is a specific use case for PUT.

> **Warning.** A handler with true PUT semantics that receives a partial body — only `{"name": "Ada"}` for a user who also has an email and an avatar — must treat the omitted fields as absent. A client that "updates the name" with PUT then erases everything else. Fix one semantics per endpoint and document it.

4.3 Idempotency

Methods are classified as *idempotent* or *non-idempotent*. An idempotent method can be called multiple times with the same result. GET is idempotent: however many times the same data is fetched, it should be the same, because GET must not modify anything on the server. PUT is idempotent: it completely replaces the resource, so replacing the old data with the same new data once or many times leaves the same result. DELETE is idempotent: a resource can be deleted only once; after that it is already gone, and repeating the action cannot produce a different result.

POST is non-idempotent. If a user of a note-taking app submits a POST to create a note, the first request succeeds and creates a note; a second identical submission creates a second note. Two different results for the same request make POST non-idempotent.

Idempotent and safe methods. Idempotency concerns the state of the server, not the response: a repeated DELETE may return 404 instead of 204, yet the resource is just as deleted. RFC 9110 also defines *safe* methods — GET, HEAD, OPTIONS — which are read-only; every safe method is idempotent, but PUT and DELETE are idempotent without being safe. PATCH is not guaranteed to be idempotent: a patch saying "add 1 to the counter" changes the result each time it is applied. Idempotency is what allows a client or proxy to retry a request after a lost response.

5 CORS AND THE OPTIONS METHOD

5.1 Same-origin policy and CORS

The `OPTIONS` method fetches the capabilities of a server for a cross-origin request. A developer rarely sends it directly, but it appears regularly in the browser's network tab as the *preflight* request of the CORS flow. Browsers follow the *same-origin policy* by default: a web page may not make requests to an origin different from the one that served it. *Cross-Origin Resource Sharing* (CORS) is a security mechanism, enforced by browsers, that controls how web applications interact with resources hosted on other origins. Without CORS, a browser blocks requests from an application running on one origin, such as `example.com`, to a different origin, such as `api.example.com`. With CORS, the server specifies who may access its resources and how. A cross-origin request follows one of two flows: the *simple request* flow or the *preflighted request* flow. In a backend, CORS is usually configured once in middleware (see Chapter 6).

Origin. An origin is the triple scheme, host and port, as in `https://example.com:443`. Two URLs have the same origin only if all three match: `http://localhost:5173` and `http://localhost:3000` are cross-origin because their ports differ, and `example.com` and `api.example.com` because their hosts differ.

5.2 The simple request flow

Let the frontend live at `example.com` and the server at `api.example.com`, and let the page send a GET. The browser automatically adds an `Origin` header naming the origin of the request. The request uses a simple method — GET, POST or HEAD — so it goes straight to the server. The server checks `Origin` against its CORS policy; if the origin is allowed, it includes `Access-Control-Allow-Origin` in the response along with the resource. While processing the response, the browser sees that the page's origin, `example.com`, differs from the host it called, and checks for that header. If the header names the client's origin, or is `*`, meaning that all origins are allowed, the browser passes the response to the JavaScript that requested it.

If the server has no CORS handling, or does not allow this origin, it leaves the header out. When the browser parses such a response and finds the header absent, it blocks the response from reaching the page: an error appears in the console and the network tab marks the request as a CORS error (Figure 2).

```
Page at https://example.com calls fetch(...)

GET /api/data HTTP/1.1
Host: api.example.com
Origin: https://example.com      <- added by browser

(a) origin allowed by the server's CORS policy
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://example.com
-> browser passes the body to the page's JavaScript

(b) origin not allowed: header left out
HTTP/1.1 200 OK
-> browser withholds the body; CORS error in console
```

Figure 2: The simple CORS flow with an allowed and a refused origin (illustrative)

A subtle point. In the simple flow the request does reach the server and is processed in full; the browser only withholds the response from the page. A cross-origin simple POST therefore still performs its side effects even when the console reports a CORS error. CORS relaxes a browser restriction on *reading* responses; it does not protect the server, and non-browser clients ignore it entirely. Protection against unwanted state changes requires authentication and CSRF defences (see Chapter 16).

5.3 Preflighted requests

For other requests the browser first sends a *preflight*: a request before the original one that inquires about the server's capabilities and tells the server what the original request will contain. A request is preflighted when it is cross-origin and at least one of three conditions holds:

- the method is not GET, POST or HEAD — for example PUT or DELETE;
- the request includes a non-simple header, such as `Authorization` or a custom header;
- the `Content-Type` is other than `application/x-www-form-urlencoded`, `multipart/form-data` or `text/plain`, the simple content types — for example `application/json`.

Since frontends and backends mostly exchange JSON, most cross-origin API calls are preflighted. The lecturer describes non-simple headers as anything apart from general and request headers; more precisely, the Fetch standard keeps a short safelist — essentially `Accept`, `Accept-Language`, `Content-Language` and `Content-Type` with the three values above — and any other header, including the request header `Authorization`, triggers a preflight.

The preflight is where the OPTIONS method is finally used (Figure 3). It has the method `OPTIONS`, the resource URL of the original request, the HTTP version, a `Host` header naming the API and an `Origin` header naming the frontend. `Access-Control-Request-Method` asks whether a meth-

od, say PUT, is supported for this URL — in effect: "I am making a cross-origin request; do you support PUT on this route?" — and `Access-Control-Request-Headers` asks whether non-simple headers such as `Authorization` are accepted. The preflight carries no body; it is only an inquiry. A server that does not handle CORS will not answer with the expected headers, and the browser automatically blocks the original request. A server that does handle CORS answers with `204 No Content` — success with no body, all the information being in headers — and four important headers, which the browser checks off one by one:

- `Access-Control-Allow-Origin`: the client's origin, `example.com`, is a valid cross-origin caller (or `*`: all clients are allowed);
- `Access-Control-Allow-Methods`: PUT and DELETE, for instance, are allowed on this route;
- `Access-Control-Allow-Headers`: the `Authorization` header is allowed;
- `Access-Control-Max-Age`: the server's answer stays valid for, say, 24 hours, so the browser need not send another preflight before each request, which saves bandwidth for both servers and clients.

When all checks pass, the browser sends the original request, and the server responds to it with whatever the operation requires. The simple flow and the preflighted flow together make up CORS. The preflight cache is kept per origin and URL, and browsers cap `Max-Age` whatever the server sends (Chromium at two hours).

```
OPTIONS /api/items/7 HTTP/1.1
Host: api.example.com
Origin: https://example.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: authorization

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: PUT, DELETE
Access-Control-Allow-Headers: Authorization
Access-Control-Max-Age: 86400          <- 24 hours

-> all checks pass: browser now sends PUT /api/items/7
```

Figure 3: A preflight and the server's answer, with the values of the lecture's example (illustrative)

5.4 Demo in Burp Suite

The demos of the course show no code: the server could be written in any language or framework, and only its observable behaviour matters. Traffic is inspected with *Burp Suite*, a tool used by ethical hackers, used here to intercept and visualise HTTP traffic. The frontend runs at `localhost:5173` and the API at `localhost:3000`. The hosts are the same, but the ports differ, so the browser treats every call as cross-origin; under the same-origin policy the page could only call `localhost:5173`.

Simple request. The request shows the two headers that matter for CORS: `Origin`, the frontend's origin, and `Host`, the API it connects to. The response carries `Access-Control-Allow-Origin` with the frontend's origin (it could also be `*`), so the browser treats the response as expected, lets it through, and the page renders it. The server is then changed so that it no longer returns the header. With the history cleared and caching disabled in the browser, so that a fresh request is made, the same request is fired again. The browser blocks the response and reports a CORS error: the request was cross-origin and the response lacked `Access-Control-Allow-Origin`.

Preflighted request. With CORS enabled again, a request is fired, and the log shows two entries: first the OPTIONS preflight, then the original request. The original request qualifies for a preflight on all three counts: PUT is not a simple method, it carries an `Authorization` header, and its body is JSON with `Content-Type: application/json` — any one would have sufficed. The preflight carries `Access-Control-Request-Method: PUT` and asks about the `authorization` and `content-type` headers. The server answers `204 No Content` with `Content-Length: 0`, since preflights have neither request nor response bodies, and with `Access-Control-Allow-Origin` naming the frontend; `Access-Control-Allow-Methods: GET, POST, PUT, DELETE`, which tells the client the server's capabilities; `Access-Control-Allow-Headers: Content-Type, Authorization`, the two non-simple headers the client asked about; and `Access-Control-Max-Age: 0`, set to zero for testing so that every request triggers a fresh preflight — with a cache of five minutes, repeated requests would not be preflighted. Since the preflight succeeded with a 2xx code, the browser sends the original PUT with its resource URL, origin, `Authorization` header, host and request body, and the server answers with a response body.

6 STATUS CODES

6.1 Why status codes exist

Response codes communicate the result of a request in a standardised way. From the code alone the client can tell whether the request succeeded and what state the server is in, without inspecting the body and reasoning: "had the request succeeded I would expect this structure, had it failed that one, had the server crashed a null object". Codes inform the client at once whether a request succeeded, failed or requires further action, and they help it handle errors through specific codes. On `401`, for unauthorized access, the client can log the user out and ask them to log in again. On `400`, a bad request caused by an invalid value in a form submission, it can ask the user to correct the form and resubmit.

Status codes are also **standardised** across all web services, so servers communicate consistently with every client regardless of platform or language. Whether the server is written in Python, Go, Rust, JavaScript or Ruby, a successful request returns 200 and a creation returns 201. Before status codes, clients had to guess the outcome from the content of the response, with inconsistent and inefficient results; the codes are a universal language that all

clients and servers understand. Each code has three digits, and the first digit gives its class (Table 4).

Class	Meaning	Codes in the lecture
1xx	Informational	100, 101
2xx	Success	200, 201, 204
3xx	Redirection	301, 302, 304
4xx	Client error	400, 401, 403, 404, 405, 409, 429
5xx	Server error	500, 501, 502, 503, 504

Table 4: Status-code classes and the codes the lecture discusses

6.2 Informational and success codes

`100 Continue` tells the client that the server has received the headers and the client may proceed to send the body. It is used for large uploads: the client sends the headers first (announcing this with `Expect: 100-continue`), and only if the server accepts the request does it send the rest of the body. `101 Switching Protocols` indicates that the server is switching to a protocol the client requested, such as an upgrade from HTTP to WebSocket (see Chapter 22). The 1xx codes are rare in daily work; classes 2, 4 and 5 are the ones met constantly.

Three success codes are used most. `200 OK` is the most common: the request succeeded and the server is returning the requested resource or has performed the requested action, as in a GET that retrieves a resource. `201 Created` means the request has been fulfilled and resulted in a new resource, typically after a POST or a form submission. `204 No Content` also means success, with nothing in the body: the preflight answer of Section 5, whose information is carried in headers, or a DELETE, after which the server has deleted the resource and has nothing to return.

6.3 Redirection codes

`301 Moved Permanently` means the resource has moved permanently to a new URL and future requests should use that URL. If a route `/user` is renamed to `/person`, answering requests to `/user` with a 301 redirect to `/person` keeps backward compatibility, so old users and applications that still call the old route do not break. `302` (reason phrase *Found*) is a temporary redirect: the resource is temporarily at a different URL, but the client should keep using the original URL for future requests. A typical use is a campaign of a few hours during which a route is redirected to a new one, to catch new traffic or show a different UI, with the change reverted afterwards. `304 Not Modified` says that the resource has not changed since the client last requested it. It is used with conditional GET requests and ETags for efficient caching: the client should keep using its cached response instead of downloading a new one (Section 7).

6.4 Client errors

The 4xx codes are errors triggered by something the client did, and they are the class a backend engineer deals with most.

- `400 Bad Request`: the client sent invalid or illogical data — an array or a string where a number is expected, a phone number where an email is expected. It tells the client that its request format is wrong and must be fixed before a new request (see Chapter 5).
- `401 Unauthorized`: the request requires authentication and the client either failed to provide valid credentials or is not authenticated at all — the expected JWT has expired or was never sent. Despite its name, 401 means *unauthenticated*.
- `403 Forbidden`: the server understood the request but refuses to authorise it, which can happen even when the client is authenticated. If user A tries to delete a resource of user B, the server answers that A lacks the necessary permission.
- `404 Not Found`, probably the most famous code: the requested resource is unavailable, because the URL is incorrect or the resource has been deleted.
- `405 Method Not Allowed`: an invalid method was used, such as PUT on a resource that accepts only GET or POST. It often comes from frontend typos — PATCH instead of PUT, PUT instead of POST.
- `409 Conflict`: the request conflicts with the current state, with many use cases. If an app lets users create folders whose names must be unique, the server checks on each POST whether the folder exists and, if so, answers 409; the client understands that a folder with that name exists and tries another name.
- `429 Too Many Requests`: used for rate limiting. If the server allows at most 60 requests per client per second and a client exceeds that, its further requests receive 429.

6.5 Server errors

- `500 Internal Server Error`, the most famous of the class: an unexpected condition on the server — a process broke or an exception was raised and not handled. Answering 500, instead of returning an empty response or breaking or hanging the request, tells the client that something went wrong on the server (see Chapter 12).
- `501 Not Implemented`: the server does not support the requested method or functionality. The lecturer uses it for something planned but not yet built — "not supported now, perhaps in the future"; RFC 9110 defines it only as missing support, typically for a method the server does not recognise for any resource.
- `502 Bad Gateway`: a server acting as a proxy, as in a load-balanced system or behind a reverse proxy such as nginx, received an invalid response from the upstream server. Application code does not return it intentionally; proxies and load balancers do.
- `503 Service Unavailable`: the service is down or temporarily unable to handle the request, for example under high traffic or during maintenance; the client should try again later.
- `504 Gateway Timeout`: similar to 502, but specifically the upstream server failed to respond within the timeout. If

nginx gets no response from the application server running behind it in time, nginx answers 504.

In the lecturer's estimate, these codes cover about 95% of practical use cases.

6.6 Demo

A frontend fires a series of requests at a server that answers each with a chosen code. Every original request in the log is preceded by an OPTIONS preflight, because the requests are cross-origin. The responses are: `200 OK` for a successful request; a POST answered with `201 Created` and a body saying the resource was created successfully — mock data, but the usual shape of such a response; `400 Bad Request` with the message "missing required data"; `401 Unauthorized`, meaning that the JWT, cookie or other credential is missing, expired or no longer valid; `403 Forbidden`, "you do not have access", for an action the user is not authorised to perform; `404 Not Found`, "the requested resource could not be found"; `409 Conflict`, "resource already exists", as with the duplicate folder; `500 Internal Server Error`, whose body says only "internal server error", deliberately telling the client nothing more for security reasons; and `503 Service Unavailable`, "please try again later".

7 HTTP CACHING

HTTP caching stores copies of responses for reuse, reducing repeated requests to the server. It improves load time, reduces bandwidth and decreases server load: if the data has not changed, the client does not download it again and the server does not send it again. Chapter 9 treats caching in general; this section follows the HTTP mechanism through the lecturer's demo.

7.1 Freshness and validators

On page load the frontend issues a GET to `/api/resource` with no special request headers. The server answers 200 with the resource in its body and three headers that drive caching:

- `Cache-Control: max-age=10`: keep this response cached for at most 10 seconds;
- `ETag`: an identifier of this version of the resource. The demo uses a random number, but ETags are usually hashes computed from the response, so that a different body yields a different tag;
- `Last-Modified`: the date and time the resource last changed, from which the client can judge whether to use its cached copy or request a new one.

The `max-age` defines *freshness*: within those 10 seconds the browser may reuse the stored copy without contacting the server at all. Once the copy is stale, the browser does not discard it but *revalidates* it with the server, which is what `ETag` and `Last-Modified` are for.

7.2 Conditional requests and 304

A second fetch of the same resource is again a GET to the same endpoint, now with two request headers. `If-None-Match` carries the ETag the browser holds; `If-Modified-Since` carries the stored `Last-Modified` date. Together they say: if the current ETag differs from mine, or the resource has been modified since this date, my version is outdated and you should send the updated resource; otherwise I will use the cached version in my browser cache. The server checks them, finds the resource unchanged, and answers `304 Not Modified` without a body: the resource has not been modified since the client last fetched it, so the client goes on using its cached version.

The demo then updates the resource. The update is sent as a POST for simplicity (a PATCH or PUT would be the semantically correct choice). The server answers 200 and sends a new ETag, `2943`. The client then issues a GET that still carries the old ETag, `3141`, and the old date. Because the resource was modified after that, the server answers 200 instead of 304, with the new body, the new ETag `2943` and a new `Last-Modified`. A further fetch sends `2943` with the new date; this is the latest version, so the server answers 304 again (Figure 4).

```
1  GET /api/resource
   <- 200 OK  Cache-Control: max-age=10
              ETag: "3141"   Last-Modified: T1
2  GET /api/resource
   If-None-Match: "3141"   If-Modified-Since: T1
   <- 304 Not Modified       (no body: use the cache)
3  POST (update the resource)
   <- 200 OK  ETag: "2943"
4  GET /api/resource
   If-None-Match: "3141"   If-Modified-Since: T1
   <- 200 OK  ETag: "2943"   Last-Modified: T2
5  GET /api/resource
   If-None-Match: "2943"   If-Modified-Since: T2
   <- 304 Not Modified
```

Figure 4: The caching demo as a sequence; ETags 3141 and 2943 are the demo's values, T1 and T2 stand for dates (illustrative layout)

A subtle point. The server does not answer 304 when *either* validator matches, as the demo's narration suggests. RFC 9110 gives `If-None-Match` precedence: when it is present, `If-Modified-Since` is ignored and the ETag comparison alone decides. ETags are preferred because `Last-Modified` has a resolution of one second and cannot distinguish two changes made within the same second.

7.3 Limits in production

In production this gets considerably more complicated. The server has to implement and manage all the ETags, and if it forgets to update an ETag when the resource changes, clients keep using a cached, outdated resource. For this reason the lecturer prefers client-side caching libraries such as React Query, which give the client full control over when to use a cached resource and when, and at what interval, to refetch; HTTP caching remains a good option when the use case is simple. The two work at different levels: React Query caches data inside one JavaScript application, whereas HTTP caching also serves CDNs, proxies and non-browser clients, and many frameworks can derive ETags from the response body automatically, which removes the manual bookkeeping.

8 CONTENT NEGOTIATION AND COMPRESSION

8.1 Three kinds of negotiation

Content negotiation is the mechanism by which client and server agree on the best format in which to exchange data — how they exchange information about the type, encoding and representation of the content. The client indicates its preferred format, such as JSON, XML or HTML, and the server responds in a compatible format or, if none is available, in a fallback format. Three kinds are common (Table 5). In **media-type** negotiation the client names the desired format in `Accept`, for example `application/json` or XML. In **language** negotiation it requests a language, such as English or Spanish, in `Accept-Language`. In **encoding** negotiation it lists the encodings it supports, such as gzip or deflate, in `Accept-Encoding`, and the server responds in one of those compression formats.

Kind	Request header	Example	Response header
Media type	Accept	application/json	Content-Type
Language	Accept-Language	en, es	Content-Language
Encoding	Accept-Encoding	gzip, deflate	Content-Encoding

Table 5: The three kinds of content negotiation

8.2 Demo: language and format

A frontend lets the user choose the language, the format and the preferred encoding, and a server answers accordingly. The default choice — English, JSON, gzip — produces a GET whose `Accept-Language` prefers English and whose `Accept` is `application/json`; the browser itself sends `Accept-Encoding: gzip, deflate, br, zstd`, the encodings it supports. The server answers with JSON in English. Switching the language to Spanish changes a single header, `Accept-Language`, and the server returns the same data in Spanish. Switching the format to XML as well changes `Accept`, and the response arrives as XML in Spanish. The client tells the server its preferences for data format and language, and the server makes the client's work easier by keeping to them.

8.3 HTTP compression

HTTP compression, with gzip, deflate or other formats, falls under the same umbrella. In the demo the JSON response is replaced by a very large file of 11,000 entries. With compression on, the response carries `Content-Encoding: gzip`, because the client listed gzip among the encodings it accepts; the transferred size is 3.8 MB, and the intercepted body looks like unreadable binary because it is compressed. Compression is then disabled on the server and the same request fired again: the same file of 11,000 entries now transfers as 26 MB, almost seven times as much, and every client would have to download it. That is the case for compression: the server compresses a large response, the browser decompresses it, and the client receives exactly the same data while far less crosses the network.

9 PERSISTENT CONNECTIONS

9.1 One connection, many requests

In the early days of HTTP, specifically HTTP/1.0, each request–response cycle required a separate connection to the server. Establishing and closing TCP connections is resource-intensive and slow, so this was inefficient. HTTP/1.1 introduced *persistent connections*: a single TCP connection is reused for multiple requests and responses, avoiding the overhead of opening and closing a connection for every interaction. The mechanism that enables this is called *keep-alive*: client and server reuse the same connection until one of them decides to close it. In HTTP/1.1 connections are persistent **by default** — nothing has to be done explicitly — and remain open for further requests unless closed explicitly. Because fewer connections must be established, latency falls and resources are saved.

9.2 Connection and Keep-Alive headers

Although persistence is the default in HTTP/1.1, the header `Connection: keep-alive` is still sometimes sent to ask the server explicitly to keep the connection open. A `Keep-Alive` header can add options: `timeout`, how long the idle connection should remain open, and `max`, how many requests may be sent before the connection is closed. With `Connection: close` the connection is closed after the response is sent; this was the default behaviour of HTTP/1.0 and can still be enforced explicitly in HTTP/1.1. These headers are worth understanding, but the defaults work well and they are rarely touched directly. HTTP/2 and HTTP/3 forbid such connection-specific headers: their multiplexed connections are long-lived by design.

10 LARGE REQUESTS AND RESPONSES

10.1 Uploads with multipart/form-data

Files — video, images, audio — are very large compared with typical JSON bodies, and are transferred differently. To send them from client to server a *multipart* request is used. In the demo a picture is selected and uploaded. The request is a POST with a `Content-Length` and `Content-Type: multipart/form-data`, and the important part is the `boundary` parameter of that header: a string that delimits the parts of the body, so that the server knows where one part ends and the next begins. Searching the body for the boundary finds it at the start of the file's binary data and again at its end, where the binary data finishes (Figure 5). The server reads the file and responds with some of its details to confirm that the upload succeeded. Whenever large files must be sent to the server, multipart requests are the tool; Chapter 20 discusses where such files should be stored.

```
POST /api/upload HTTP/1.1
Content-Type: multipart/form-data; boundary=XyZ123
Content-Length: 284731

--XyZ123
Content-Disposition: form-data; name="file"; filename="a.jpg"
Content-Type: image/jpeg

...binary data of the image...
--XyZ123--
```

Figure 5: A multipart upload; the boundary opens the part and, with a trailing `--`, closes the body (illustrative)

10.2 Streaming responses

The opposite direction is a large response. In the demo a large text file on the server is streamed to the client in chunks. Clicking "stream data" sends an ordinary GET; the first chunk appears, and further chunks keep arriving continuously from the server. The response has two important headers. `Content-Type: text/event-stream` says that the body is not a single text document but will be streamed to the client as a series of events. `Connection: keep-alive` keeps the connection open until all the data has been sent. Until the file is completely transferred, the server keeps sending chunks, and the client keeps appending what it receives, reconstructing the whole text file (Figure 6). The lecturer calls this transfer chunked transfer or a text event stream.

```
GET /api/stream HTTP/1.1

HTTP/1.1 200 OK
Content-Type: text/event-stream
Connection: keep-alive

data: first chunk of the text file ...

data: next chunk ...

... events keep arriving on the same response
until the whole file has been sent
```

Figure 6: A streamed response delivering a large file as events (illustrative)

A subtle point. Two descriptions in these demos are loose. A multipart body is not a file cut into pieces: each *part* is one form field or one file, which is why a single uploaded image sits between one pair of boundary lines. And a stream arrives as one long response, not as separate requests: `text/event-stream` is the Server-Sent Events format (see Chapter 22), and in HTTP/1.1 a body of unknown length like this is framed with `Transfer-Encoding: chunked`, while HTTP/2 and HTTP/3 use their own frames.

11 SSL, TLS AND HTTPS

11.1 From SSL to TLS

SSL was the original protocol for securing communication between a client, such as a web browser, and a server. It encrypts data so that sensitive information such as passwords or credit-card numbers cannot be intercepted by attackers. SSL is now outdated because of security vulnerabilities and has been replaced by **TLS**, its modern and more secure successor, which clients and servers use today. TLS encrypts data in transit, so that anything sent between client and server is protected from interception and tampering. It uses certificates to authenticate the server and to establish an encrypted connection, which prevents eavesdropping and data breaches. TLS is updated continuously, newer versions offering better security; the currently recommended version is TLS 1.3.

11.2 HTTPS

HTTPS is HTTP with the security features provided by SSL, originally, and by TLS today: the underlying mechanism is TLS, and HTTPS is HTTP running over it. When a user visits a website over HTTPS, TLS encrypts the communication between the browser and the server, which protects sensitive data such as login credentials from being intercepted. For application-level backend work this is the level of detail needed; the inner workings of TLS, like those of TCP, belong to network engineering.

12 CONCLUSION

- HTTP is stateless and client-initiated: each request carries its own context and credentials, which keeps servers simple, scalable and tolerant of crashes.
- HTTP needs only a reliable transport: TCP up to HTTP/2, QUIC over UDP for HTTP/3.
- A message is a start line, headers, an empty line and an optional body; headers are metadata that every intermediary can read without opening the body.
- Methods declare intent: GET, PUT and DELETE are idempotent, POST is not, and PUT replaces a resource while PATCH updates selected fields.
- CORS is enforced by browsers: simple requests are checked on the response, other cross-origin requests are preceded by an OPTIONS preflight answered with `Access-Control-Allow-*` headers.
- Status codes are a shared vocabulary: 2xx success, 3xx redirection, 4xx client mistakes, 5xx server or proxy failures.
- HTTP caching combines freshness (`max-age`) with validation (`ETag`, `Last-Modified`, `304 Not Modified`).
- Accept headers negotiate format, language and encoding; gzip cut the demo's 26 MB response to 3.8 MB.
- Connections persist by default since HTTP/1.1; files go up as multipart bodies, large responses come down as streams, and HTTPS is HTTP over TLS.

APPENDIX A REVIEW QUESTIONS

1. A single-page app served from `https://app.shop.io` calls `https://api.shop.io` with `PUT /cart/items/9`, a JSON body and an `Authorization: Bearer` token. The API answers OPTIONS with 204, `Access-Control-Allow-Origin: https://app.shop.io`, `Access-Control-Allow-Methods: GET, POST` and `Access-Control-Allow-Headers: Content-Type`. Trace which requests reach the server, what the browser reports and what the page's JavaS-

cript receives, and give the smallest server change that makes the call work.

2. A product API returns `Cache-Control: max-age=3600` and an ETag that a developer increments in the update handler; a nightly import job writes price changes directly to the database. After the import, some customers see old prices for up to an hour, and some still see them later while the server log shows only 304 responses for their requests. Explain both symptoms from the caching mechanics and propose a design that removes them.
3. A mobile app on an unreliable network retries every request whose response has not arrived within five seconds. It calls `POST /orders`, `PUT /users/42/avatar`, `PATCH /carts/7` with `{"addQuantity": 1}` and `DELETE /sessions/abc`. For each, describe the server state after a request that succeeded but whose response was lost and which was retried once, say which endpoints are safe to retry as they stand, and design protection for the others.
4. You design `POST /folders`, which creates a folder inside a parent folder. Choose and justify a status code for each case: the name field is missing; the bearer token expired five minutes ago; the user is authenticated but has only read access to the parent; the parent belongs to another tenant; a sibling with the same name exists; the database is unreachable; the user exceeded 60 requests per minute. Discuss where 403 and 404 trade clarity against leaking the existence of resources.
5. Behind nginx, an API shows two symptoms: a report endpoint that needs about 70 seconds fails with 504 after exactly 60 seconds, and short bursts of 502 appear during every deployment, while the application logs nothing unusual for either. Explain which component produces each code and why the application logs are silent, and propose changes for both; relate the deployment symptom to Chapter 15.
6. A JSON export of 11,000 entries is 26 MB uncompressed and 3.8 MB with gzip, and 2,000 mobile clients download it every hour. Compare compressing on every request, storing a pre-compressed copy and paginating the endpoint, in terms of bandwidth, server CPU and latency. Then explain what goes wrong if a shared cache stores the gzip version and serves it to a client that sent no `Accept-Encoding`, and how the server should prevent it.
7. Users must upload videos of up to 2 GB through your API. Compare sending the file as base64 inside a JSON body with sending it as `multipart/form-data`, considering body size, server memory, parsing and timeouts, and explain how `Expect: 100-continue` could reject an unwanted upload early. Then argue whether the bytes should pass through the API servers at all (see Chapter 20).
8. A service keeps logged-in users' sessions in the memory of its process. After scaling to three instances behind a round-robin load balancer, users are logged out at random, and a crash of one instance logs out a third of all users. Explain both symptoms in terms of HTTP's statelessness and compare sticky sessions, a shared session store and self-contained signed tokens for this workload (see Chapter 4).

APPENDIX B GLOSSARY

Term	Meaning
Statelessness	The server keeps no memory between requests; each request carries all needed context
Idempotent method	Repeating it leaves the server as one call would (GET, PUT, DELETE)
Safe method	Read-only method that changes no server state (GET, HEAD, OPTIONS)
Origin	Scheme, host and port of a URL; the unit of the same-origin policy
Same-origin policy	Browser rule that stops a page reading responses from other origins
CORS	Browser-enforced mechanism by which a server allows origins, methods and headers
Preflight request	OPTIONS request sent before a non-simple cross-origin request
Status code	Three-digit result of a request; the first digit gives the class
ETag	Identifier, usually a hash, of one version of a resource
Conditional request	Request with `If-None-Match` or `If-Modified-Since`; may get 304
Content negotiation	Choosing format, language and encoding through Accept headers
Content-Encoding	Compression applied to the body, such as gzip
Persistent connection	TCP connection reused for many requests; default since HTTP/1.1
multipart/form-data	Body format for forms and files; parts separated by a boundary
text/event-stream	Server-Sent Events format for a response that streams events
TLS	Successor of SSL; encrypts and authenticates the connection under HTTPS

APPENDIX C FURTHER TOPICS

Credentialed CORS requests

When a cross-origin request carries cookies (`fetch` with `credentials: "include"`), the server must answer with `Access-Control-Allow-Credentials: true` and must name the exact origin, because the wildcard `*` is rejected for credentialed requests. Such responses should also carry `Vary: Origin`, so that caches do not hand one origin's answer to another. This applies whenever a frontend on one subdomain uses cookie-based sessions with an API on another. Reflecting any incoming `Origin` together with credentials is a common and serious misconfiguration.

The Vary header

`Vary` lists the request headers a response depends on, such as `Accept-Encoding`, `Accept-Language` or `Origin`. Browser caches, CDNs and reverse proxies add those headers to the cache key, so a gzip body is not served to a client that can-

not decompress it and a Spanish page is not served to an English reader. Every cacheable endpoint that uses content negotiation needs it, and bugs caused by its absence typically appear only once a CDN is placed in front of the API.

Cache-Control beyond max-age

`no-store` forbids storing a response at all, while `no-cache` allows storing but requires revalidation before every reuse. `private` restricts storage to the user's own browser, `public` permits shared caches, and `s-maxage` sets a separate lifetime for shared caches; `stale-while-revalidate` (RFC 5861) lets a cache serve a stale copy while refreshing it in the background. These directives are how a backend keeps personal data out of CDNs while letting versioned static assets be cached for a year.

Redirects 307 and 308

Historically, clients following a 301 or 302 often turned a POST into a GET. `307 Temporary Redirect` and `308 Permanent Redirect` forbid that change: the client must repeat the same method and body at the new URL. Backends that move or proxy endpoints accepting POST, PUT or DELETE should use them instead of 302 and 301, which remain fine for ordinary GET links.

Problem Details for HTTP APIs

RFC 9457 standardises a JSON error body, with media type `application/problem+json` and fields such as `type`, `title`, `status`, `detail` and `instance`. It complements the status code with machine-readable detail, such as which field failed validation, without each API inventing its own error format. ASP.NET Core and other frameworks can emit it directly, and it suits every 4xx and 5xx response of a public API (see Chapter 12).

Optimistic concurrency with If-Match

The ETags used for caching also protect writes. A client sends PUT or PATCH with `If-Match` carrying the ETag it last read; if someone changed the resource in the meantime, the server answers `412 Precondition Failed` instead of silently overwriting the other change, the classic lost update. This matters for any resource edited concurrently by several users or devices, such as documents, settings or stock levels.

Range requests and 206 Partial Content

A client can request part of a representation with `Range: bytes=0-1048575`; a server that advertises `Accept-Ranges: bytes` answers `206 Partial Content` with that slice. This is how video players seek, how interrupted downloads resume and how large files are fetched in parallel pieces. It applies whenever a backend serves large files or media, and object stores support it natively (see Chapter 20).

Idempotency keys for POST

Because POST is not idempotent, a retried payment or order request can create duplicates. The client sends a unique `Idempotency-Key` header with each logical operation; the server stores the key with the first result and returns that same result for any retry carrying the key. Payment APIs such as Stripe popularised the pattern, and the IETF HTTPAPI working group has drafted a standard header for it. It applies to every non-idempotent endpoint called over unreliable networks.

Routing — One-Page Overview

Chapter 2 · every section and sub-section of the paper at a glance

1 Introduction: The Where of a Request

The method states what the client wants, the route states where, and routing maps the pair to exactly one handler.

- **1.1 The what and the where** — `GET /users` combines the intent "fetch" with the address `/users`; the server maps both to a handler that runs the business logic and returns the users.
- **1.2 Method and route as one key** — `GET` and `POST` on `/api/books` share one route but form two unique method–path keys, so they reach different handlers and never clash.

2 Static and Dynamic Routes

A route is either a constant string or a pattern whose named slots capture path parameters.

- **2.1 Static routes** — `/api/books` has no variable part: the client always sends the same string and gets the same kind of data.
- **2.2 Dynamic routes** — `/api/users/:id` accepts any ID in its last segment; for `/api/users/123` the router puts `123` into the slot `id`. Colon syntax is common; ASP.NET Core writes `{id}`.
- **2.3 Path parameters** — Captured values are always strings, to be converted and validated; the route reads like a sentence, the core idea of REST paths.

3 Query Parameters

Key–value pairs after `?` carry metadata about a request, mostly on GET requests, which have no usable body.

- **3.1 Anatomy of a query string** — `/api/search?query=some+value`: path and method select the handler; the query string is read after the match.
- **3.2 Why not the path or the body** — Path parameters are reserved for identifying resources; free search text in the path is possible but hard to maintain and loses the semantics.
- **3.3 Pagination** — Response metadata (total 100, limit 20, 5 pages, current page 1) lets the client request `?page=2`; page 1 is the server's default.
- **3.4 Filtering and sorting** — Filters, the sort field and ascending or descending order travel as further key–value pairs.

4 Nested Routes

Nesting resources in the path expresses their relationships, and each level is a route of its own.

- **4.1 Nesting resources** — `/api/users/123/posts/456`: a static part and two path parameters, read as "post 456 of user 123".
- **4.2 Every level is a route** — `/api/users`, `/api/users/123`, `…/posts` and `…/posts/456` each match a different handler with a different meaning.

5 Route Versioning and Deprecation

A version segment lets an API change its response format while existing clients get time to migrate.

- **5.1 Two versions of one resource** — `/api/v1/products` returns `id`, `name`, `price`; `/api/v2/products` returns `id`, `title`, `price`.
- **5.2 Why version the route** — A new client such as a mobile app needs a new format; a version says so clearly, without inventing `/api/new-products`.
- **5.3 Deprecating the old version** — Announce v1 as deprecated, give clients a migration window, then remove it: a stable workflow for breaking changes.

6 Catch-All Routes

A final wildcard route answers every request that no other route matched.

- **6.1 Requests no handler serves** — `/api/v3/products` has no handler, yet the client still needs a clear answer.
- **6.2 The fallback handler** — `/*`, registered last, returns "route not found"; it should keep status 404 and the API's own error format.

Routing at a glance — the method says what to do and the path says where; the router turns the pair into one handler, reading path parameters for identity and query parameters for options, with version segments for change and a catch-all for everything unmatched. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Routing: Mapping Requests to Handlers

Chapter 2 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Routing is the second half of every request: the HTTP method states what the client wants, and the route states where. The chapter shows how a server combines method and path into a unique key that selects one handler, and distinguishes static routes from dynamic routes with path parameters. It then explains query parameters and their use for pagination, filtering and sorting on GET requests, and nested routes as the expression of relationships between resources. Route versioning with an orderly deprecation of old versions is presented as the workflow for breaking changes, and the catch-all route as the answer to requests that no handler serves.

Keywords — routing, route, handler, path parameter, query parameter, pagination, nested route, API versioning, catch-all route

1 INTRODUCTION: THE WHERE OF A REQUEST

Chapter 1 described the HTTP method as the *what* of a request: it expresses the client's intent, whether that is fetching, adding, updating or deleting data. Routing expresses the *where*: the resource on which that intent should act. The server needs both halves to decide which code to run.

1.1 The what and the where

In a request `GET /users` the method says "fetch", and the route path `/users` says what to fetch and from where. The server takes the combination of intent and address, maps it to a particular *handler* — the set of instructions that performs the business logic, the database operations and whatever else is needed — and returns the result, here an array of users.

> **Routing.** Routing is the mapping of a request's method and URL path to the server-side logic, the handler, that serves it. The method is the action, the path is the address, and together they select exactly one handler.

1.2 Method and route as one key

The lecture's demo captured, in the Burp Suite intercepting proxy, the requests sent by a small React app. The first was `GET /api/books`. In its request line `GET` is the action and `/api/books` is the route, the address the client wants to reach inside the server; `books` is a *resource*, a notion treated with REST API design in Chapter 7. The server answered with data (fillers in the demo; a real server would typically authenticate the request first, see Chapter 4). The second request was `POST /api/books`: a different method on the same route. The server created a book and again returned all the resources.

Both requests carry the identical route, yet they reach different code. The server checks the method, then the route, and concatenates the two into a unique routing key that maps to one handler (Figure 1). `GET /api/books` and `POST /api/books` are therefore two distinct paths through the server that never clash: the method differentiates them.

```
Routing table (method + path pattern -> handler)
GET    /api/books          listBooks
POST   /api/books          createBook
GET    /api/users/:id      getUser

Incoming request    lookup result
GET  /api/books     row 1 -> listBooks
POST /api/books     row 2 -> createBook
PUT  /api/books     no row -> no handler
```

Figure 1: Method and path pattern together form the key that selects a handler (illustrative).

2 STATIC AND DYNAMIC ROUTES

2.1 Static routes

The two book routes are *static routes*: they contain no variable part. The string `/api/books` is a constant; the client sends exactly that string on every call, nothing in it changes, and it always returns the same kind of data.

2.2 Dynamic routes

The next demo request was `GET /api/users/123`, where `123` is the ID of a user. Such an endpoint fetches the details of one particular user: the server extracts the ID from the route path and uses it, for instance to load the user from the database. The demo server only echoed the ID in its response, to show that it had received it.

On the server, the matching side looks like `r.get("/api/users/:id", handler)`. The `get` part matches the method, the string matches the path, and the colon marks `:id` as a dynamic parameter: any string in that position is accepted, so every GET request of the form `/api/users/`*something* is routed to this handler. For the demo request the router matched the method GET, then the segment `api`, then `users`, and finally inserted `123` into the slot named `id` (Figure 2). The lecturer presents the colon as the convention he has seen across the industry, in Java, Python, Node.js, Go and Rust servers alike.

```
r.get("/api/users/:id", getUser)   // route definition

route     GET   /api   /users   /:id
request   GET   /api   /users   /123
result    match match  match    id = "123"

handler receives params = { id: "123" }   (a string)
```

Figure 2: Matching `GET /api/users/123` against a route with one path parameter (route definition as in the lecture; matching steps illustrative).

A subtle point. The colon is the style of Express, Gin and Rails, but it is not universal. ASP.NET Core, Spring, FastAPI and Go's standard `ServeMux` (since Go 1.22) write the slot as `{id}`; Flask and Django write `<id>` or `<int:id>`. The concept, a named slot inside the path, is the same everywhere.

2.3 Path parameters

Although `123` looks like a number, in route paths numbers, special characters and everything else arrive as strings: the slot receives the string `"123"`. Converting it to an integer, and rejecting a value such as `abc`, is left to the handler or to a validation layer (Chapter 5). A slot normally spans a single segment, so `/api/users/123/posts` does not match `/api/users/:id`.

A dynamic route also reads naturally: "get me some data (`GET`), located under `/api/users`, belonging to the user with ID 123". Providing such a human-readable construct for addressing data is the central idea of REST APIs. A route with a variable part is a *dynamic route*, and the variable part is called a *route parameter* or *path parameter*, because it comes right after a forward slash and is part of the path itself. This distinguishes it from the second kind of request parameter, the query parameter, with which it is easily confused.

3 QUERY PARAMETERS

3.1 Anatomy of a query string

For the query demo a value was typed into a search box, producing `GET /api/search?query=some+value`. The part `/api/search` is the route in the sense used so far: together with the method, it is what the server matches against its handlers. The remainder — the question mark followed by a key and a value — is the *query string*, and each key–value pair in it is a *query parameter* (Figure 3). The query string plays no part in selecting the handler; the handler reads it after the match.

```
GET /api/search?query=some+value HTTP/1.1
    |---------| |--------------|
       path       query string

/api/search       with GET, selects the handler
?                 starts the query string
query=some+value  one query parameter: key=value
+                 an encoded space: value is "some value"
&                 joins pairs: ?page=2&limit=20
```

Figure 3: Anatomy of a request with a query parameter (request from the lecture; annotations illustrative).

3.2 Why not the path or the body

POST and PUT requests have a body, a container in which the client sends user-defined values to the server. In REST APIs a GET request has no body, so another channel is needed. (HTTP does not forbid content in a GET, but RFC 9110 gives it no defined meaning and some servers and proxies reject or drop it, so it cannot be relied on.)

Path parameters are not the answer either, because they serve a specific purpose: semantic expression. `/api/users/123` means "the details of the user whose ID is 123". A search term has no such meaning; it is whatever the user typed. Sending it as `/api/search/some value` is technically possible, but hard to maintain, and it defeats the purpose of giving endpoints semantic expressions. Query parameters fill the gap: a set of key–value pairs, typically sent with GET requests, that carry metadata about the request.

3.3 Pagination

Pagination is the typical example. An endpoint `/api/books` that returns books page by page answers with a JSON object: the books under `data`, plus metadata about the response such as the total number of items, the current page, the total number of pages and the limit (Figure 4); the exact fields depend on the implementation. With a default limit of 20 and 100 books in total, the first response holds books 1–20, reports page 1, and gives 100 / 20 = 5 total pages.

The client uses this metadata to shape its subsequent requests according to its own needs. The first request needed no query parameters, because the server defaults to page 1 and its default limit. For the next chunk the client sends `GET /api/books?page=2`: the same route, with the page number travelling as a query parameter.

```
GET /api/books                 (no query: page 1, limit 20)

{"data": [ ...20 books... ],
 "total": 100,
 "currentPage": 1,
 "totalPages": 5,
 "limit": 20}

GET /api/books?page=2              -> books 21-40
GET /api/books?page=2&limit=50     -> books 51-100
GET /api/books?author=herbert&sort=title&order=asc
```

Figure 4: A paginated response and follow-up requests built from its metadata (illustrative).

3.4 Filtering and sorting

The same mechanism carries the other options of a request: filtering by a user-defined parameter, the field to sort by, and the sort order, ascending or descending. All of them travel as key–value pairs in the query string. Table 1 contrasts the two kinds of parameters.

Aspect	Path parameter	Query parameter
Position	a path segment, after `/`	after `?`, pairs joined by `&`
Form	value in a named slot (`:id`)	`key=value` pair
Role	identifies the resource	metadata: page, filter, sort, search
Routing	part of the matched route	read after the match
Example	`/api/users/123`	`/api/books?page=2`

Table 1: Path parameters and query parameters compared.

4 NESTED ROUTES

4.1 Nesting resources

A *nested route* is not a separate type of routing but a practice found almost everywhere. To express meaning, REST APIs often nest one type of resource inside another, and nested routes are the result. The demo request was `GET /api/users/123/posts/456`. Its static part is `/api/users`; `123` is a first path parameter and `456` a second. Read semantically, the first part concerns data related to user 123, the second says that the data wanted is that user's posts, and the last level narrows it to the particular post with ID 456.

4.2 Every level is a route

The path can stop at any level, and each stopping point is a unique route with its own handler (Table 2). `/api/users` is the static route from before and returns the list of all users. Adding a dynamic parameter gives `/api/users/123`, which the server matches, together with GET, to a different handler that returns that one user. Adding `/posts` gives a route whose handler returns all posts of user 123 — a different meaning altogether — and adding `/456` returns only the post with that ID. Each level of nesting expresses a different semantic meaning and produces a different response. According to the lecturer, nested routes appear in practically every API of even medium complexity.

GET route	Parameters	Handler returns
`/api/users`	none (static)	all users
`/api/users/123`	user ID	user 123
`/api/users/123/posts`	user ID	all posts of user 123
`/api/users/123/posts/456`	user ID, post ID	post 456 of user 123

Table 2: The levels of a nested route; each is a separate route with its own handler.

5 ROUTE VERSIONING AND DEPRECATION

5.1 Two versions of one resource

The versioning demo sent two requests that differ from the earlier ones only by a version segment: `GET /api/v1/products` and `GET /api/v2/products`. Both responses contain a `data` array of product objects, but the v1 objects have the fields `id`, `name` and `price`, while the v2 objects have `id`, `title` and `price` (Figure 5). This is *route versioning*, a very common practice for servers with a REST interface.

```
GET /api/v1/products
{"data": [{"id": 1, "name": "Desk lamp", "price": 30},
          {"id": 2, "name": "Bookshelf", "price": 85}]}

GET /api/v2/products
{"data": [{"id": 1, "title": "Desk lamp", "price": 30},
          {"id": 2, "title": "Bookshelf", "price": 85}]}
```

Figure 5: One resource in two versions (field names as in the lecture; values illustrative).

5.2 Why version the route

The example is trivial; the situation behind it is common. An endpoint returns data in a particular format, and later new requirements arrive: a server that first served only a web app may also have to serve a React Native, Android or Flutter app, and for that client the response must change. One option is to change the whole route, for example to `/api/new-products`. Versioning is the better option, for two reasons. It states the intention clearly: version 1 of the API serves responses in one format, version 2 in another. And the route itself is not renamed; only the version segment changes.

> **Breaking change.** A breaking change alters an API's contract so that existing clients stop working, for example by renaming or removing a field, changing its type or moving a route. Renaming `name` to `title` is one: every client that reads `name` breaks. Adding a new field usually is not.

5.3 Deprecating the old version

Versioning also makes it possible to retire v1 in an orderly way. When v2 is released, the backend team notifies the frontend engineers that v1 is deprecated. They then have a window in which to migrate their request endpoints to the v2 format; once it closes, v1 is removed entirely and v2 becomes the version in use. The result is a stable workflow for introducing new structures and breaking changes into an API, in which every client has the opportunity to migrate before the old structure disappears.

6 CATCH-ALL ROUTES

6.1 Requests no handler serves

The last demo sent `GET /api/v3/products`. The server has handlers for v1 and v2 but none for v3: it does not serve requests at this endpoint, yet it must still send some response.

6.2 The fallback handler

The usual solution is a *catch-all route*. After all the routes, and all the methods associated with them, have been registered, the server registers one final route, `/*`, whose star matches any path. A request that has gone through all the previous route matching without success reaches this entry and is mapped to its handler, which sends a user-friendly message saying that the requested route does not exist (Figure 6). The lecture contrasts this with a null response, which it describes as the default when no catch-all

is defined; with the catch-all, the client is told explicitly that the endpoint is not served.

```
r.get("/api/books", listBooks)
r.post("/api/books", createBook)
r.get("/api/users/:id", getUser)
r.get("/api/v1/products", listProductsV1)
r.get("/api/v2/products", listProductsV2)
r.all("/*", notFound)                // registered last

GET /api/v3/products -> no earlier match -> notFound

HTTP/1.1 404 Not Found
Content-Type: application/json

{"error": "Route GET /api/v3/products does not exist"}
```

Figure 6: Routes registered in order, with the catch-all last (illustrative).

A subtle point. Most frameworks do not send an empty or null response for an unmatched route: they already return `404 Not Found` with a short default body (Express sends an HTML page reading `Cannot GET /api/v3/products`; ASP.NET Core sends a 404 with an empty body). The catch-all's real value is a body in the API's own format, such as a JSON error object. It must keep the 404 status: a friendly message sent with `200 OK` looks like success to clients, caches and monitoring.

Warning. In routers that try routes in registration order, such as Express, a catch-all registered before other routes swallows every request meant for them, so it must come last. ASP.NET Core instead ranks routes by specificity: a catch-all parameter such as `{*path}` and the `MapFallback` endpoint get the lowest priority wherever they are declared.

7 CONCLUSION

- The method is the what of a request and the route the where; routing maps the pair to one handler.
- Method plus path forms a unique key, so `GET /api/books` and `POST /api/books` never clash.
- Static routes are constant strings; dynamic routes contain named slots such as `:id` that capture path parameters.
- Everything captured from a path is a string; conversion and validation belong to the handler or a validation layer.
- Path parameters identify resources; query parameters carry request metadata as key–value pairs, typically on GET.
- Pagination, filtering and sorting (`page`, `limit`, `sort`, `order`) are the classic uses of query parameters.
- Nested routes express relationships between resources, and every level is a separate route with its own handler.
- Version segments such as `/api/v1` allow breaking changes, with an announced deprecation window for clients to migrate.
- A catch-all route registered last answers unmatched requests with a clear, consistently formatted 404.

APPENDIX A REVIEW QUESTIONS

1. An Express-style router tries routes in registration order and contains, in this order: `GET /api/users/:id` → `getUser`, `GET /api/users/me` → `getMe`, and a catch-all for all methods on `/api/*` → `notFound`. Predict which handler runs, with which parameter values, for `GET /api/users/me`, `GET /api/users/123/posts` and `DELETE /api/users/123`. Then reorder or extend the table so that each request reaches the handler its author intended, and justify each change.
2. A shop's search box builds its request as `GET /api/search/` followed by the raw text the user typed. Support tickets report that searching for `tcp/ip`, `c#` and `50%` returns a 404, wrong results or a 400, while `redis` works. Explain, for each of the three inputs, what the server's router actually receives and matches. Then redesign the endpoint with a query parameter and state how the client must encode these three values and the value `c++`.
3. A client lists books with `GET /api/books?page=N` (default limit 20). The first response reports `total: 100` and `totalPages: 5`, and the client loops over pages 1 to 5. Between its requests for page 2 and page 3, three new books are inserted at the front of the server's default ordering. Determine which books the client receives twice and which it never sees, then propose a different set of query parameters that removes the anomaly and state what it costs the client.
4. Your mobile app reads the field `name` from `GET /api/v1/products`; a new tablet app needs it renamed to `title`. App-store data shows that six months after any release, 25% of active users still run the previous app version. Compare renaming the field inside v1, adding `/api/newproducts`, and introducing `/api/v2/products`, and design a deprecation timeline, including the evidence you would require before removing v1.
5. After a release, the error-rate dashboard shows 0% client errors, yet users of an old client report empty product lists. The investigation finds that the old client still calls `GET /api/v1/products`, which the release removed, and that the catch-all handler answers `200 OK` with `{"message": "route not found"}`. Explain how each observation follows from this handler, and specify the response the catch-all should send instead and what that changes for the client and for monitoring.
6. The handler for `GET /api/users/:userId/posts/:postId` runs `SELECT * FROM posts WHERE id = $1` with `postId` and never reads `userId`; post 456 belongs to user 123. Predict the response to `GET /api/users/999/posts/456`, explain why this matters if the API authorises access by checking that the caller is the user named in the path (see Chapter 4), and compare fixing the handler with replacing the route by a flat `/api/posts/456`.
7. A handler for `GET /api/users/:id` passes the captured value straight into a query on an integer column. Pre-

dict the outcome for `/api/users/123`, `/api/users/0123`, `/api/users/12abc` and `/api/users/-1` in a stack that coerces strings to numbers loosely and in one whose database driver rejects non-numeric input. Decide where conversion and validation should happen (see Chapter 5) and which status code each invalid case should receive.

8. A single-page app on `https://app.example.com` sends `PUT https://api.example.com/api/books/7` with a JSON body. The API registers `GET` and `POST /api/books`, `GET /api/books/:id` and, last, a catch-all for all methods that returns 404 with a JSON message; it has no CORS middleware. Trace every HTTP exchange the browser performs (see Chapter 1) and state what the application code finally observes. Then explain whether answering unsupported methods with `405 Method Not Allowed` and an `Allow` header would change that outcome.

APPENDIX B GLOSSARY

Term	Meaning
Routing	Mapping a request's method and path to the handler that serves it
Route	The path or path pattern a server matches requests against
Handler	The server-side function that runs the logic for one matched route
Routing key	The method–path combination that identifies exactly one handler
Static route	A route with no variable part, e.g. `/api/books`
Dynamic route	A route with named slots, e.g. `/api/users/:id`
Path (route) parameter	A value captured from a slot in the path; always a string
Query string	The part of a URL after `?`: `key=value` pairs joined by `&`
Query parameter	One key–value pair in the query string, carrying request metadata
Resource	The thing a route addresses, such as books or users (Chapter 7)
Pagination	Returning a collection in chunks, steered by `page` and `limit`
Nested route	A path that nests resources, e.g. `/api/users/123/posts/456`
Route versioning	A version segment such as `/api/v2` that separates response formats
Breaking change	A contract change that makes existing clients fail
Deprecation	Announcing that a version will be removed after a migration window
Catch-all route	A final wildcard route (`/*`) for requests nothing else matched

APPENDIX C FURTHER TOPICS

Router data structures

Routers differ in how they find the matching route. Express compiles every route pattern to a regular expression and tests them in registration order, so lookup cost grows with the number of routes and the order decides conflicts. Routers such as Go's httprouter (underlying Gin) and Fastify's find-my-way store routes in a radix tree and match segment by segment, in time proportional to the path length; ASP.NET Core compiles its endpoints into a deterministic finite automaton. The data structure explains a framework's rules about which routes may coexist and which one wins, and it matters for performance in services with hundreds of routes.

Route constraints and typed parameters

Many frameworks let a route declare the type or shape of a parameter, as in ASP.NET Core's `{id:int}` or `{id:guid}` and Flask's `<int:id>`. The router then matches only values of that form and hands the handler a converted value, which also disambiguates routes such as `/users/{id:int}` and `/users/me`. The ASP.NET Core documentation warns against using constraints for input validation, because a non-matching value produces `404 Not Found` instead of a more informative `400 Bad Request`. Constraints serve routing; validating the value remains the job of the validation layer (Chapter 5).

405 Method Not Allowed and the Allow header

When a path exists but not for the requested method, HTTP prescribes `405 Method Not Allowed`, and RFC 9110 requires the response to carry an `Allow` header listing the supported methods. A catch-all that answers such requests with 404 hides the fact that the resource exists and makes client bugs harder to diagnose. ASP.NET Core endpoint routing and Go's `ServeMux` since Go 1.22 produce 405 automatically, whereas Express by default falls through to 404. Related details are the automatic handling of `HEAD` by the GET handler and the answering of `OPTIONS` requests, including CORS preflights.

Percent-encoding of path and query values

Characters with a syntactic role in a URL — `/`, `?`, `#`, `&`, `=`, `%` and others — must be percent-encoded when they occur inside a value (RFC 3986). The rules depend on the position: `+` means a space only in form-encoded query strings, so a literal plus is sent as `%2B`, and an encoded slash `%2F` inside a path parameter is decoded, rejected or kept as is by different servers. Clients should build URLs with an encoder for the right component, such as JavaScript's `encodeURIComponent`, rather than by string concatenation. Inconsistent decoding between a proxy and the application is a known source of routing and security bugs.

Other versioning schemes

A path segment is only one place to put the version. Azure REST APIs take an `api-version` query parameter, GitHub's REST API reads an `X-GitHub-Api-Version` header, and Stripe pins each account to a dated version that a `Stripe-Version` header can override; media-type versioning puts the version into the `Accept` header. Header-based schemes keep URLs stable and allow fine-grained versions, while path versions are visible, easy to route at a proxy and easy to try

in a browser. The choice also decides whether caches must vary their entries on a header (the `Vary` header).

Deprecation and Sunset headers

Besides announcing a deprecation to client teams, a server can signal it on every response. The `Sunset` header (RFC 8594) gives the date after which a resource is expected to stop responding, and the `Deprecation` header, standardised in RFC 9745, marks a resource as deprecated, optionally together with a link to migration documentation. Clients, SDKs and API gateways can log or alert on these headers automatically, which reaches consumers who never read release notes. Request metrics per version are the complementary tool for deciding when the old version can actually be removed.

Cursor (keyset) pagination

The page-number scheme of the lecture is *offset pagination*: page 3 with limit 20 skips 40 rows. Offsets become slow for deep pages, because the database still reads the skipped rows, and they shift when rows are inserted or deleted between requests, causing duplicates or gaps. *Keyset* or *cursor* pagination instead sends the position of the last item seen (`?after=...&limit=20`) and queries `WHERE id > :last ORDER BY id LIMIT 20`, which uses an index and stays stable under concurrent writes. Its cost is that clients can no longer jump to an arbitrary page number (see Chapter 8 for indexes).

Trailing slashes and case sensitivity

Whether `/api/books`, `/api/books/` and `/API/Books` are the same route is a framework decision. RFC 3986 treats the path as case-sensitive, but Express and ASP.NET Core match case-insensitively by default; Express also ignores a trailing slash unless strict routing is enabled, while Django, with its default `APPEND_SLASH` setting, redirects to the slashed form. Inconsistent handling produces duplicate cache entries, broken links, and routes that work locally but return 404 behind a stricter proxy. Choosing one canonical form and redirecting the others (301 or 308) keeps URLs unambiguous.

Path-based routing at the edge

In larger systems routing also happens before the application: a reverse proxy or API gateway such as nginx matches a path prefix (`location /api/orders/`) and forwards the request to the service that owns it. The proxy may strip or rewrite the prefix, so the path the application's router sees can differ from the one the client sent. This is how one public domain fronts many services, and how versions can be served by separate deployments (`/api/v1` to the old one, `/api/v2` to the new). Debugging a 404 therefore often starts by finding which layer failed to match.

Serialization & Deserialization — One-Page Overview

Chapter 3 · every section and sub-section of the paper at a glance

1 Introduction: Data Across Language Boundaries

Client and server seldom share a language or a type system, yet each must understand the data the other sends.

- **1.1 Client, server and channel** — A browser JavaScript app (React, Angular, Vue) talks to a server on `localhost` or in AWS, GCP or Azure over HTTP, gRPC or WebSockets; an HTTP request carries a method and URL, headers and a body.
- **1.2 Two type systems** — A dynamically typed JavaScript client and a compiled, strictly typed Rust server have no data types in common, yet a value must mean the same thing on both sides.

2 A Common Format Both Sides Agree On

Client and server agree on one standard and convert to and from it at each end of the exchange.

- **2.1 Designing the solution** — A common standard is a set of rules that both sides agree to send and receive data in, independent of the language either is written in.
- **2.2 The round trip** — Client serializes, server deserializes into a Rust struct, server serializes its response, client parses it; the `Content-Type` header declares the format.
- **2.3 Definition** — Converting data to and from a common format for transmission or storage, so that it is language- and domain-agnostic.

3 Serialization and the Network Stack

Serialization happens at the application layer; everything below it is outside the backend engineer's concern.

- **3.1 The OSI model at a high level** — Application layer on top, physical layer at the bottom, the same stack on both machines; a high-level picture of the layers is enough.
- **3.2 Below the application layer** — Data becomes segments, IP packets, frames and finally bits carried as voltage or light; the server's application layer receives the same JSON the client produced.

4 Serialization Standards and the Series' Choices

The series uses the most widely used option in each area, and JSON for serialization.

- **4.1 Most-used technology first** — HTTP rather than WebSockets or gRPC, PostgreSQL among relational and non-relational databases, JSON for roughly 80% of client–server traffic.
- **4.2 Text-based and binary formats** — JSON, YAML and XML are readable text; Protocol Buffers, the most popular binary format, trades readability for compactness and speed.

5 JSON: Structure and Rules

JavaScript Object Notation is human-readable, language-independent and built from a few basic types.

- **5.1 Name and uses** — It looks like a JavaScript object but is used everywhere: API request and response bodies, configuration files and runtime logs.
- **5.2 Syntax rules** — Braces, double-quoted string keys, values that are strings, numbers, booleans, null, arrays or nested objects; stricter than JavaScript literals; identifiers belong in strings.

6 Demo: A JSON Round Trip

Burp Suite shows a books API exchanging JSON in both directions.

- **6.1 The request** — `POST /api/books` carries the keys `id`, `title` and `author`: one number and two strings, transmitted as text.
- **6.2 The response** — An object with one key holding an array of book objects, which the client parses and renders; the client could equally be written in PHP.

Serialization at a glance — client and server agree on a common format; each side serializes its own data into that format before sending and deserializes it into its own types on receipt, and for HTTP APIs the format is usually JSON. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Serialization and Deserialization: How Clients and Servers Share Data

Chapter 3 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — A client and a server are usually written in different languages with unrelated type systems, such as a JavaScript front end and a Rust back end, yet each must understand the other's data. The solution is a common format that both agree on: serialization converts each side's data into it before transmission or storage, and deserialization converts it back into the receiver's own types. In the layered network model this conversion belongs to the application layer, and everything below it is outside the backend engineer's concern. Serialization standards are either text-based, such as JSON, YAML and XML, or binary, such as Protocol Buffers, and JSON dominates HTTP communication between clients and servers. The chapter sets out JSON's structure and rules and follows a JSON request and response through a demo of a books API.

Keywords — serialization, deserialization, JSON, data interchange format, language-agnostic, text-based formats, binary formats, Protocol Buffers, OSI model

1 INTRODUCTION: DATA ACROSS LANGUAGE BOUNDARIES

A client and a server that exchange data are seldom written in the same language, and their type systems may have nothing in common. Serialization and deserialization are the conversions that let them exchange data regardless: each side translates its own values into an agreed common format before sending, and translates the format back into its own values on receipt. The concept involves few rules, but every request and response a backend handles depends on it.

1.1 Client, server and channel

The setting is a client, typically a browser such as Chrome running a front-end application, and a server running either on the developer's machine (`localhost`) or remotely in a cloud such as AWS, GCP or Azure. They communicate over the network by one of several means: HTTP with conventional REST endpoints, gRPC, or WebSockets. The series concentrates on HTTP, and the client here is a JavaScript application built with a framework or library such as React, Angular or Vue, as most highly interactive client-side applications are.

To send information, the client makes an HTTP request (see Chapter 1): a method such as `GET` or `POST` with the URL, then the headers, then the request body carrying the data the client wants to transmit (Figure 1). The server answers with a response that the client must in turn be able to parse.

```
CLIENT                          SERVER (Rust)
Chrome, JavaScript app          localhost, or a cloud:
(React, Angular, Vue)           AWS, GCP, Azure
--------- HTTP request --------->
<-------- HTTP response ---------

POST /api/books HTTP/1.1         <- request line
Host: some-domain.com            <- headers
Content-Type: application/json
                                 <- empty line
{"name": "some string"}          <- body
```

Figure 1: The setting of the lecture: a JavaScript client and a Rust server exchanging an HTTP request and response (illustrative).

1.2 Two type systems

Suppose the client sends a JavaScript object such as `{ name: "some string" }` in the request body, and the web server is written in Rust. JavaScript is dynamically typed and has no ahead-of-time compilation; Rust is compiled and strict about types, and its data types are entirely different from JavaScript's. The data must still leave one machine, cross the Internet, arrive at another machine elsewhere, and be understood there in the server's own types so that business logic can run on it. The response then travels back, and the client must understand it well enough to render the UI or run its own logic. The problem is how one representation, transmitted over a network, comes to mean the same thing on both sides.

2 A COMMON FORMAT BOTH SIDES AGREE ON

2.1 Designing the solution

Imagine being asked to solve this problem from scratch: two machines in different locations, connected over the Internet, must send each other data that the receiver can parse and understand, and the method must not depend on the language either machine's software is written in. The obvious solution is a *common standard*, which is simply a set of rules that client and server both agree to: the client sends its data in this format, and the server sends its responses in the same format. Neither side then

needs to know anything about the other's language, only about the standard. Such an agreement is a *serialization standard*.

2.2 The round trip

One exchange involves four conversions (Figure 2). When sending, the client converts whatever it holds in JavaScript into the standard format and sends it over the network. The server takes the data in that format and converts it into its own representation, in Rust a struct, after which it understands the data and does what it has to do. To respond, the server converts its data from the Rust struct back into the common format and sends it, and the client reads the response and parses it into JavaScript values.

```
CLIENT (JavaScript)            SERVER (Rust)

{ name: "Dune" }               Book { name: "Dune" }
  |                              ^
  | serialize                    | deserialize
  v                              |
{"name":"Dune"} -- network ---> {"name":"Dune"}

Response, the same trip in reverse:
struct -> serialize -> JSON text -> network
  -> deserialize -> JavaScript object
```

Figure 2: The four conversions of one exchange through a common format; on the server a library such as serde_json would perform them (illustrative).

In code each conversion is usually one call: in the browser `JSON.stringify` turns an object into JSON text and `JSON.parse` turns text into objects, and in Rust a library such as `serde_json` plays the same role for structs. The receiver must also know which format a body is in; in HTTP the sender declares it in the `Content-Type` header, for JSON `application/json`.

2.3 Definition

Serialization and deserialization. *Serialization* converts data from a program's own representation into a common, agreed format for transmission or storage; *deserialization* converts data from that format back into the receiving program's own representation. Because both sides depend only on the format, programs written in different languages and running in different environments can exchange data and respond to it.

The emphasis is on being *language-agnostic*, or more generally domain-agnostic: clients and servers from different domains, languages and environments can all make sense of the same data. The definition also covers storage, which the lecture names but does not develop: the same conversion takes place whenever data is written to a file, a cache or a database column and read back later, possibly by a different program.

3 SERIALIZATION AND THE NETWORK STACK

3.1 The OSI model at a high level

Data transmission over the Internet is usually explained with the OSI model, which divides network communication into layers through which data passes on its way to and from the hardware. A backend engineer need not master every aspect of it, such as the details of IP packets or data frames, but should have a high-level picture of how the layers cooperate to carry data across the network. At the top of the stack is the *application layer*, at the bottom the *physical layer*, with the other layers between them; which end is called the top does not matter, only that the two are at opposite ends. The server has the same stack (Figure 3).

```
          CLIENT                          SERVER
 +----------------------+        +----------------------+
 | 7 Application  JSON  |        | 7 Application  JSON  |
 | 6 Presentation       |        | 6 Presentation       |
 | 5 Session            |        | 5 Session            |
 | 4 Transport segments |        | 4 Transport segments |
 | 3 Network   packets  |        | 3 Network   packets  |
 | 2 Data link frames   |        | 2 Data link frames   |
 | 1 Physical  bits     | -----> | 1 Physical  bits     |
 +----------------------+        +----------------------+
   bits travel as voltage levels or light in optical fibre
   backend engineer's concern: the JSON at layer 7
```

Figure 3: The OSI layers on both machines; only the application-layer payload concerns the backend engineer (illustrative).

3.2 Below the application layer

On its way down the client's stack the data is transformed repeatedly: the transport layer splits it into segments, the network layer wraps them in IP packets, the data link layer puts those into frames, and the physical layer finally transmits bits, zeros and ones carried as voltage signals or as light in optical fibre. At the server the bits are read, the network-related work is undone layer by layer, and what reaches the server's application layer is exactly what the client's application layer handed down.

The backend engineer's mental model. The client adheres to an agreed standard, here JSON, and the server understands that standard. At the application layer the data is JSON on the way out and JSON again on arrival, and the server only ever reads JSON. Which formats the data passes through in between is not the backend engineer's responsibility.

4 SERIALIZATION STANDARDS AND THE SERIES' CHOICES

4.1 Most-used technology first

Backend engineering spans hundreds of technologies, and nobody can learn the concepts behind all of them. The series therefore uses the most widely used option in each area, enough to start working and gain experience, and leaves the rest to be learned over a career (Table 1). For communication it uses HTTP, because REST APIs are still

the most common means of communication between clients and servers on the Internet, and an engineer experienced in HTTP can pick up WebSockets or gRPC along the way. For databases it uses PostgreSQL, one of the most popular relational databases and today often the first choice of startups and enterprises alike. For serialization it uses JSON, which by the lecturer's rough estimate is used about 80% of the time in client–server communication.

Area	Options named	Series uses
Communication	HTTP (REST), WebSockets, gRPC	HTTP
Databases	PostgreSQL, MySQL, SQLite (relational); MongoDB, DynamoDB (non-relational)	PostgreSQL
Serialization	JSON, YAML, XML (text); Protocol Buffers (binary)	JSON

Table 1: Technology choices of the series and the alternatives named in the lecture

4.2 Text-based and binary formats

Serialization standards fall into two families (Table 2). *Text-based* formats encode data as readable characters: JSON, YAML and XML. YAML and XML are not necessarily used for HTTP communication between clients and servers, but they are serialization standards all the same. *Binary* formats encode data as compact bytes that are not meant to be read by people; the most popular is Protocol Buffers (Protobuf), and there are others, such as Avro. Binary formats generally give up human readability in exchange for smaller payloads and faster parsing, and usually rely on a schema that both sides share.

Family	Formats	Readable	Typically met in
Text-based	JSON	yes	HTTP APIs, configuration, logs
Text-based	YAML, XML	yes	configuration, documents
Binary	Protocol Buffers, Avro	no	gRPC, internal services

Table 2: The two families of serialization standards

5 JSON: STRUCTURE AND RULES

5.1 Name and uses

JSON stands for *JavaScript Object Notation*. In appearance and behaviour it closely resembles a JavaScript object, which explains the name, but it is not limited to JavaScript and is used everywhere. Besides the bodies of HTTP REST API requests and responses, the use that matters here, it appears in configuration files and in log files, where it is a popular choice for recording application or server data at runtime (see Chapter 14). Its strongest point is that it was designed to be human-readable, and its data types are basic enough to be familiar to anyone with a little programming experience.

5.2 Syntax rules

A JSON object starts with an opening curly brace and ends with a closing one. Between them are key–value pairs separated by commas, with a colon between each key and its value. Keys obey a strict rule: every key is a string inside double quotes, and no other data type can be a key. A value can be a string, a number, a boolean (`true` or `false`), an array, or another object, so objects can be nested. In the lecturer's example an `address` field holds a nested object with a `country` string, `"India"`, and a phone number given as a number (Figure 4). The nested object follows exactly the same rules as the outer one: braces around it, double-quoted string keys, and values of the same kinds, nested to any depth.

```
{                                  <- opening brace
  "name": "Asha",                  <- key and string value
  "age": 25,                       <- number
  "isStudent": false,              <- boolean
  "skills": ["Go", "Rust"],        <- array
  "address": {                     <- nested object
    "country": "India",
    "phone": 3456
  }
}                                  <- closing brace
```

Figure 4: A JSON object with a nested address, after the lecturer's example; fields other than address, country and phone are illustrative.

These rules are stricter than JavaScript's own object literals, which is easy to forget because the two look alike. `{name: 'Dune'}` is a valid JavaScript object but not valid JSON, since the key is unquoted and the string uses single quotes; a strict parser on the server rejects such a body before any business logic runs (see Chapter 5).

A subtle point. The lecture lists five kinds of value; JSON has a sixth, `null`, so the full set is string, number, boolean, null, array and object (RFC 8259). Nor must a JSON text be an object: any value may stand at the top level, so `[1, 2, 3]` or `"ok"` is a valid body. Objects are the usual choice for API bodies because fields can be added to them later without breaking readers.

Identifiers are not numbers. The lecturer's sketch gives the phone number as a JSON number, which illustrates the type but is wrong for a real API. Phone numbers, postal codes and account numbers should be strings: JSON's grammar does not allow a number to begin with zero, a client that parses `"0412"` into an integer silently drops the zero, and a number cannot hold `+`, spaces or dashes. JSON numbers are for quantities on which arithmetic makes sense.

6 DEMO: A JSON ROUND TRIP

6.1 The request

The demo returns to the books application used in the earlier demos, this time watching the data format rather than the route. The traffic is inspected in Burp Suite: the lecturer calls the `POST` books endpoint, opens the call in the HTTP history and expands the request. It is a `POST` to the path `/api/books`, and its body is JSON exactly as described above: an opening brace, the double-quoted string keys `id`, `title` and `author`, and as values one number and two

strings (Figure 5). This text is what is transmitted over the network.

```
POST /api/books HTTP/1.1
Content-Type: application/json

{"id": 3, "title": "Dune", "author": "Frank Herbert"}

--- response body ---
{
  "books": [
    {"id": 1, "title": "Emma", "author": "Jane Austen"},
    {"id": 3, "title": "Dune", "author": "Frank Herbert"}
  ]
}
```

Figure 5: The demo exchange inspected in Burp Suite; the keys id, title and author are from the lecture, the values and the name of the array key are illustrative.

6.2 The response

The server understands the body, takes the `id` and the other fields, adds the book to its collection and responds with another JSON document. It again opens and closes with braces and has a single key whose value is an array; each element of the array is itself a JSON object with double-quoted keys and number and string values. The client receives this response, parses it and renders the list on the page in its own way. Here the client is JavaScript, but it could as well be written in PHP or anything else, since the format is all the two sides share. As the lecturer stresses, serialization involves few concepts; what matters is knowing that this conversion takes place on both sides of every exchange.

7 CONCLUSION

- A JavaScript client and a Rust server share no data types, so data must cross between them in a common format.
- Serialization converts a program's data into the agreed format; deserialization converts it back into the receiver's own types, for transmission or storage.
- One HTTP exchange involves four conversions: client out, server in, server out, client in; the `Content-Type` header names the format.
- Below the application layer data becomes segments, packets, frames and bits; the backend engineer deals only with the JSON at the application layer.
- Serialization standards are text-based (JSON, YAML, XML) or binary (Protocol Buffers, Avro); JSON dominates HTTP client–server traffic.
- A JSON object has braces, double-quoted string keys, and values that are strings, numbers, booleans, null, arrays or nested objects.
- JSON is stricter than JavaScript literals, and identifiers such as phone numbers belong in strings, not numbers.
- In the demo, `POST /api/books` sends `id`, `title` and `author` as JSON and receives an object holding an array of books, which the client renders.

APPENDIX A REVIEW QUESTIONS

1. A React front end calls `fetch("/api/books", { method: "POST", body: book })`, where `book` is a plain JavaScript object, and sets no headers. The Rust server, which expects `application/json`, rejects every such request, and its logs show the received body as the text `[object Object]`. Explain which conversion was skipped and on which side, what exactly travelled over the network and with which declared media type, and why the server's deserializer could not recover the data.
2. An orders API returns `{"orderId": 9007199254740993, "total": 19.99}`. The browser client then shows order 9007199254740992 and receives 404 when it calls `GET /api/orders/9007199254740992`, while a Go client of the same API works correctly. Trace the value through each serialization and deserialization step, identify where and why it changes, and propose a contract change that fixes the browser without breaking the Go client.
3. A Java service streams 40,000 telemetry records per second, each with 25 numeric fields, to a Go service; the same data is also shown on a browser dashboard. Compare JSON and Protocol Buffers separately for the service-to-service link and for the browser link, weighing payload size, parsing cost, debuggability with tools such as Burp Suite, and how each side learns about a newly added field, and justify the design you would choose.
4. A sign-up API defines `phone` as a JSON number. Users whose numbers start with 0 report two different failures: some apps receive 400 Bad Request, while others succeed but the stored number has lost its first digit. Explain both symptoms from JSON's number grammar and the behaviour of the clients' serializers, and design the schema change together with a migration for stored data and for old clients that cannot be updated immediately.
5. A server deserializes request bodies into `struct Signup { email: String, age: u8 }`. Predict the outcome for the bodies `{"email": "a@b.co", "age": 30}`, `{"email": "a@b.co", "age": "30"}` and `{"email": "a@b.co", "age": 300}`, stating the stage at which each is accepted or rejected and the status code the client should see. Then argue which checks belong to deserialization and which to the validation layer of Chapter 5.
6. A team serializes user preferences to JSON, stores the text in a PostgreSQL column and caches it in Redis. A release renames the struct field `darkMode` to `colorScheme` and adds a required field `theme`; afterwards some users' preferences silently reset to defaults while others cause errors. Diagnose both symptoms as problems of serialization for storage rather than for transmission, and design a change process that would have avoided them.
7. One endpoint returns JSON responses of about 3 MB, and clients intermittently report "unexpected end of JSON input". One engineer proposes capturing IP pack-

ets to look for corrupted bits; another proposes inspecting the application-layer response in Burp Suite. Using the layered mental model, argue which investigation should come first and what each can reveal, and name two application-level causes that produce exactly this symptom.

8. `GET /api/books` currently returns a top-level JSON array of book objects, and the team must now add a total count and a cursor for the next page without breaking existing clients. Explain why the top-level array makes this hard, compare at least two migration options, and state which you would choose when some mobile clients cannot be forced to upgrade.

APPENDIX B GLOSSARY

Term	Meaning
Serialization	Converting a program's own data into a common format for transmission or storage
Deserialization	Converting data in the common format back into the receiving program's own types
Serialization standard	The format, a set of rules, that both sides agree to send and receive data in
Language-agnostic	Usable by programs regardless of the language they are written in
JSON	JavaScript Object Notation; human-readable text format of objects, arrays and simple values
JSON object	Key–value pairs in curly braces, every key a double-quoted string
Nested object	An object used as the value of a key inside another object
Text-based format	A format encoded as readable characters, e.g. JSON, YAML, XML
Binary format	A format encoded as compact bytes not meant to be read by people
Protocol Buffers	Google's schema-based binary format, the most popular binary standard
YAML	Text-based serialization format common in configuration files
XML	Tag-based text format for structured documents and data
OSI model	Seven-layer reference model of how data travels across a network
Application layer	Top OSI layer, where the payload such as JSON is produced and read
Physical layer	Bottom OSI layer, sending bits as electrical or optical signals
Content-Type	HTTP header naming the media type of the body, e.g. application/json

APPENDIX C FURTHER TOPICS

Number precision across languages

JSON numbers have no defined size or precision, and RFC 8259 warns that interoperability is good only for values that fit IEEE 754 double precision. JavaScript stores every number as such a double, so integers above $2^{53} - 1$ (9,007,199,254,740,991) are silently rounded by `JSON.parse`, while a Rust or Go server reading the same text into a 64-bit integer keeps them exact. Large 64-bit identifiers are therefore commonly serialized as strings. Money deserves the same caution and is better sent as an integer count of minor units or as a decimal string than as a binary floating-point number.

Dates and times

JSON has no date or time type, so every API must choose a textual convention that each side deserializes into its own date type. The common choice is an RFC 3339 (ISO 8601) timestamp with an explicit offset, such as `2026-11-01T09:30:00.000Z`, the form JavaScript's `Date.prototype.toJSON` produces; Unix timestamps in seconds or milliseconds are the main alternative. A timestamp without an offset is read in whatever time zone the receiver assumes, causing errors of whole hours that appear only for some users. Calendar dates such as birthdays are better sent as `2026-11-01` with no time at all.

Character encoding

JSON is text, so its bytes must be decoded with the right character encoding before parsing. RFC 8259 requires UTF-8 for JSON exchanged between systems that are not part of a closed ecosystem and forbids adding a byte order mark to networked JSON. Strings may contain any Unicode character, with quotation marks, backslashes and control characters escaped. A backend that decodes request bytes with another encoding, or truncates them in the middle of a multi-byte character, corrupts names and addresses in ways that pass every ASCII-only test.

Schemas and API contracts

The agreement between client and server stays informal until it is written down as a schema. JSON Schema describes which fields a JSON document must contain, with their types and constraints, and OpenAPI uses it to describe whole HTTP APIs (see Chapter 25). A written contract lets the server validate bodies systematically, lets tools generate matching client and server types in many languages, and lets teams detect breaking changes before deployment. It matters most when several teams or external consumers depend on the same API.

Schema evolution and compatibility

Serialized data outlives the code that wrote it: old clients keep calling new servers, and JSON stored by last year's code is read by this year's. A change is backward compatible when new code can read old data, and forward compatible when old code can read new data. The usual rules are to add fields as optional with defaults, to let readers ignore unknown fields, and never to rename a field or change its type in place; Protocol Buffers builds these rules into the schema through numbered fields. Kleppmann's chapter on encoding and evolution treats the subject in depth.

Protocol Buffers and gRPC

Protocol Buffers, the binary format the lecture names, defines messages in a `.proto` schema in which every field has a name, a type and a field number. The `protoc` compiler generates serialization code for many languages, and on

the wire each field is identified by its number rather than its name, which makes messages much smaller than the equivalent JSON. gRPC uses Protocol Buffers over HTTP/2 for calls between services. Because payloads are unreadable without the schema, Protobuf suits internal service-to-service traffic better than public APIs consumed from browsers.

Insecure deserialization

Some language-native serialization mechanisms, such as Java object serialization, Python's `pickle` and PHP's `unserialize`, can reconstruct arbitrary object types and run code while doing so. Deserializing untrusted input with them can give an attacker remote code execution, and OWASP lists the problem among software and data integrity failures. Data-only formats such as JSON avoid this when they are deserialized into explicitly declared types, combined with limits on body size and nesting depth (see Chapter 16).

Serialization for storage

The lecture's definition includes storage, but its examples concern transmission. A backend serializes data whenever it writes to a cache, a message queue, a file or a database column, and the reader may be a different program or a later version of the same one. PostgreSQL offers two column types for JSON: `json` keeps the exact input text, whereas `jsonb` stores a decomposed binary form that is slower to write but faster to query and can be indexed. The format of stored data deserves the same care as an API's, because stored data stays long after the code that wrote it has changed.

Authentication & Authorization — One-Page Overview

Chapter 4 · every section and sub-section of the paper at a glance

1 Introduction: who you are and what you can do

Two questions, asked in a fixed order.

- **1.1 Two questions** — authentication: *who are you?* Authorization: *what can you do?*
- **1.2 Plan** — history, building blocks, four types of authentication, RBAC, two defences.

2 A brief history of authentication

Each era answers the weakness of the one before.

- **2.1–2.3 Before computers** — trust and handshakes → seals (have) → passphrases (know).
- **2.4 Passwords** — CTSS stored them in plain text; hashing followed; today slow, salted hashes.
- **2.5 Keys and tickets** — Diffie–Hellman (1976) and PKI; Kerberos tickets (1980s).
- **2.6 MFA** — know + have + are; biometrics bring false matches.
- **2.7 Today and next** — OAuth, JWT, zero trust, WebAuthn; post-quantum cryptography.

3 Building blocks: sessions, JWTs and cookies

Three mechanisms recur in every flow.

- **3.1 Stateless HTTP** — carts and logins need memory between requests.
- **3.2 Sessions** — random ID → user data in Redis or a database, with expiry.
- **3.3 JWT** — header, claims, signature; scales, but is hard to revoke.
- **3.4 Cookies** — returned automatically; HttpOnly, Secure, SameSite.

4 Stateful versus stateless authentication

Where the truth about a login lives.

- **4.1 Stateful** — session ID in an HttpOnly cookie, looked up on each request.
- **4.2 Stateless** — JWT in the Authorization header, verified by its signature.
- **4.3 Trade-off** — control and easy revocation versus scale.
- **4.4 Hybrid** — sessions for the web app, tokens for mobile apps and partners.

5 API keys

Programmatic access without a login form.

- **5.1 Mechanism** — a random key generated in a dashboard, sent in a header.
- **5.2 Example** — a server calls an AI provider; key → plan, quota, permissions.
- **5.3 Machine to machine** — no human trigger; key kept in an environment variable.

6 OAuth 2.0 and OpenID Connect

Delegated access without sharing passwords.

- **6.1–6.2 Delegation** — shared passwords gave full, unrevocable access.
- **6.3 OAuth** — resource owner, client, resource server, authorization server.
- **6.4 OAuth 2.0** — RFC 6749 (2012): bearer tokens, flows; code + PKCE today.
- **6.5–6.6 OIDC** — an ID token (a JWT) adds identity: 'Sign in with Google'.

7 Choosing an authentication method

Rules of thumb.

- **Map** — web app → stateful; API → stateless; third party → OAuth; machine → key.

8 Authorization and role-based access control

Permissions belong to roles, roles to users.

- **8.1–8.2 Need** — an admin-only dead zone; a secret god-mode string fails.
- **8.3 RBAC** — roles hold permissions per resource.
- **8.4 Request cycle** — middleware resolves the role early; handlers check it.
- **8.5 401 vs 403** — no valid identity → 401; known but not permitted → 403.

9 Security practices

Fail identically in message, status and time.

- **9.1 Generic errors** — 'Authentication failed' for every case.
- **9.2 Timing attack** — an unknown user fails before the slow hash.
- **9.3–9.4 Defence** — hash a dummy on every path, or pad failures to a fixed total.

Auth at a glance — prove identity (password, session, token, key, OAuth/OIDC) → carry it across stateless requests (session cookie or JWT) → resolve the role → allow, or answer 401 (no identity) or 403 (no permission); never reveal which check failed, in words or in time. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Authentication and Authorization: Sessions, Tokens, Delegation and Access Control

Chapter 4 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — This paper distinguishes authentication, which establishes who a subject is, from authorization, which decides what that subject may do. It traces authentication from implicit trust, wax seals and passphrases to passwords, hashing, public-key cryptography, Kerberos tickets and multi-factor schemes, and states how passwords must be stored today. It then introduces the three building blocks of modern flows, namely sessions, JSON Web Tokens and cookies, and compares four types of authentication: stateful, stateless, API-key and OAuth 2.0 with OpenID Connect, including the authorization code flow with PKCE that current practice recommends. The second part covers role-based access control, its place in the request cycle and the difference between the status codes 401 and 403. The paper closes with two defences: generic error messages, and the removal of timing differences between failed logins.

Keywords — authentication, authorization, session, JSON Web Token, cookie, OAuth 2.0, OpenID Connect, role-based access control, timing attack

1 INTRODUCTION: WHO YOU ARE AND WHAT YOU CAN DO

Signing up and logging in are among the most familiar interactions on the web. They are the visible part of two distinct processes, which the lecturer first summarizes in two sentences.

1.1 Two questions

> **Definition.** Authentication assigns an identity to a subject in a given context; it answers the question *who are you?* Authorization determines the capabilities, or permissions, of that subject in the same context; it answers the question *what can you do?*

The context may be a platform, an operating system or a phone, and the subject may be a person or, as Section 5 shows, another program. Permissions can only be decided for a known identity, so authorization depends on authentication, and the paper treats authentication first.

1.2 Plan of the paper

Section 2 traces the history of authentication. Section 3 introduces three building blocks of modern flows: sessions, JSON Web Tokens and cookies. Sections 4 to 6 describe four types of authentication, namely stateful, stateless, API-key and OAuth 2.0 with OpenID Connect, and Section 7 gives rules for choosing among them. Section 8 covers authorization and role-based access control, and Section 9 two security practices: generic error messages and protection against timing attacks.

2 A BRIEF HISTORY OF AUTHENTICATION

Modern mechanisms are easier to understand against the problems they were invented to solve. In the lecturer's short history, each era answers the weakness of the one before.

2.1 Implicit trust

In pre-industrial societies authentication was implied: a person's identity was the community's recognition of that person. A respected village elder could vouch for someone, and deals were sealed with a handshake, a gesture of mutual recognition. This human, contextual trust failed as populations grew and interactions went beyond familiar circles, because an elder is not known, let alone trusted, in another town or country. Society needed proofs of identity that worked without personal acquaintance, which marks the beginning of explicit authentication.

2.2 Seals: something you have

By the medieval period the answer was the wax seal, a unique pattern pressed onto letters and agreements as a signature. Seals were the first widely adopted authentication tokens, physical representations of identity based on possession: *something you have*. They could be forged, which is the first recorded *authentication bypass attack*, that is, defeating authentication with malicious intent. The response, watermarks and encrypted codes in trade documents, began cryptographic thinking.

2.3 Passphrases: something you know

During the Industrial Revolution the telegraph became critical infrastructure, and its messages needed validation. Operators used pre-agreed passphrases: static *shared secrets*, fixed strings agreed in advance and used repeatedly. The principle moved from something you possess to *something you know*, and the passphrase is the direct ancestor of the password.

2.4 Passwords and hashing

Authentication became digital with the mainframes of the mid-twentieth century. In 1961 researchers at MIT working on the Compatible Time-Sharing System (CTSS) introduced passwords, so that several users could share one machine without seeing one another's data. The passwords were stored in plain text, and the weakness became visible

when the password file was printed. The lecturer presents this incident as the origin of secure password storage. The response was hashing: an algorithm that turns a string of any length into an irreversible output of fixed length, always the same for the same input. The server stores only the hash; at login it hashes the submitted password and compares the two values. Authentication thereby began to align with confidentiality, integrity and availability, the core tenets of information security.

A subtle point. The lecture compresses the chronology. CTSS was built at the MIT Computation Center under Fernando Corbató and first demonstrated in 1961; Project MAC, which the lecturer names, began in 1963. The printing of the password file is usually dated to 1962. Hashed storage came later: one-way protection of passwords is credited to Roger Needham at Cambridge in the late 1960s, and Unix adopted salted, deliberately slowed hashing, described by Morris and Thompson in 1979.

Two refinements, beyond the lecture, are essential today (see Chapter 16). Because hashing is deterministic, attackers precompute the hashes of common passwords; each password is therefore first combined with a *salt*, a random value unique to each user and stored beside the hash. Salting alone is not enough. General-purpose hashes such as SHA-256 are designed to be fast, and one graphics card computes billions per second, so stolen salted hashes can still be guessed offline. Passwords must be hashed with a deliberately slow, tunable algorithm whose cost is raised as hardware improves: bcrypt, scrypt or Argon2id, or PBKDF2 with a high iteration count where FIPS approval is required. These libraries generate the salt themselves and embed it in their output.

Warning. Never store passwords in plain text, under reversible encryption, or under a fast hash such as MD5 or SHA-256, salted or not.

2.5 Public-key cryptography and tickets

The 1970s brought an explosion of cryptographic research. In 1976 Whitfield Diffie and Martin Hellman published their key exchange, which lets two parties establish a shared secret over an untrusted medium and introduced asymmetric, or public-key, cryptography. It became the backbone of modern authentication protocols and of the public key infrastructure (PKI) behind HTTPS (see Chapter 1). The lecturer places Kerberos in the same era: a trusted third party issues tickets that verify both the user and the service, a precursor of today's token-based systems.

A subtle point. Kerberos does not belong to the 1970s. It was developed at MIT for Project Athena in the 1980s and described publicly in 1988, building on the Needham–Schroeder protocol of 1978. It also relies on symmetric, not public-key, cryptography: its key distribution center shares a secret key with every user and service. It remains the default protocol of Windows domains.

2.6 Multi-factor authentication

In the 1990s the growing internet needed authentication at scale, and passwords alone were too weak against *brute-force* attacks, which try every combination, and *dictionary* attacks, which try likely passwords. Multi-factor authentication (MFA) combines independent principles: **something you know**, such as a password or PIN; **something you have**, such as a smart card or one-time-password generator; and **something you are**, such as a fingerprint or retina scan. Biometric authentication, which identifies users by unique physical traits through pattern recognition and statistical models, was a major development, but it brought its own problems: false positives, which accept the wrong person; false negatives, which reject the right one; and the security of stored templates, since a fingerprint, unlike a password, cannot be changed once it has leaked.

2.7 The twenty-first century and beyond

Cloud computing, mobile devices and API-based architectures demanded more robust frameworks: OAuth and OAuth 2.0 (Section 6), JWTs (Section 3.3), *zero-trust architecture*, in which every request is authenticated whatever its origin in the network, and passwordless authentication such as WebAuthn, which relies on a key pair held in a hardware device. For the future the lecturer names decentralized identity, built on technologies such as blockchain; behavioural biometrics; and post-quantum cryptography, designed to survive quantum computers that would break algorithms such as RSA.

A subtle point. Quantum computers are not simply much faster machines, as the lecture suggests; the threat lies in specific algorithms. Shor's algorithm would break RSA, Diffie–Hellman and elliptic-curve cryptography, whereas symmetric ciphers and hashes are only weakened. NIST published its first post-quantum standards (ML-KEM, ML-DSA and SLH-DSA) in August 2024.

Era	Mechanism	Principle
Pre-industrial	Vouching, handshake	Human, contextual trust
Medieval	Wax seals	Something you have
Industrial	Telegraph passphrases	Something you know
1960s	Passwords (CTSS), then hashing	Stored secrets
1970s–1980s	Public-key cryptography, Kerberos	Keys and tickets
1990s	MFA, biometrics	Know + have + are
21st century	OAuth, JWT, zero trust, WebAuthn	Tokens and key pairs

Table 1: The evolution of authentication

3 BUILDING BLOCKS: SESSIONS, JWTS AND COOKIES

Three components recur in every flow described later. The lecturer introduces them first, so that they are familiar by the time the flows are assembled from them.

3.1 Stateless HTTP and the need for memory

HTTP is stateless by design (see Chapter 1): each request is an isolated interaction that carries everything the server needs, and the protocol remembers nothing of earlier exchanges. That suited the early web of static pages, but dynamic sites need memory: a cart must persist, and a user must stay logged in from page to page. These needs started the search for stateful interaction on top of a stateless protocol.

3.2 Sessions

> **Definition.** A session is temporary, server-side context about one user, identified by a unique session ID that lets the server recognize the user's later requests.

At login the server creates a session ID and stores it with the data it needs repeatedly (identity, role, cart) in a persistent store: a database, or Redis, which keeps its data in memory. The ID goes to the browser in a cookie (Section 3.4) and comes back with every request, so the server can fetch the data. Sessions are short-lived, expiring for example after fifteen minutes. Their storage evolved from files on the server, which did not scale, to databases, which survived restarts, to in-memory stores such as Redis and Memcached, which are faster than database lookups (see Chapter 9).

> **A subtle point.** The lecture says that on expiry the server simply creates a new session. For authentication that happens only after the user logs in again; otherwise expiry would protect nothing. Servers commonly extend a session while the user is active and impose an absolute limit, after which a fresh login is required.

3.3 JSON Web Tokens

By the mid-2000s web applications had become globally distributed, and sessions became a bottleneck in two ways: storing them for millions of users was costly, and synchronizing them across regions added latency and consistency problems. Developers sought to take the state off the server without losing security and integrity.

> **Definition.** A JSON Web Token (JWT) is a compact, self-contained token that carries *claims* about a subject, such as its ID and role, together with a signature that lets the receiver verify the issuer and detect any alteration.

JWTs were formalized in 2015. A JWT consists of three Base64url-encoded parts separated by dots, which the site jwt.io decodes for inspection: a **header** naming the signing algorithm; a **payload** of claims, by convention `sub` (subject) for the user ID and `iat` (issued at) for the time of issue, with optional fields such as name and role; and a **signature** computed with a secret key that only the server holds. If anyone changes the token, verification with the secret fails.

```
Header     {"alg": "HS256", "typ": "JWT"}
Payload    {"sub": "42", "name": "Ada", "role": "admin",
            "iat": 1761300000, "exp": 1761300900}
Signature  HMAC-SHA256(base64url(header) + "." +
                       base64url(payload), secret)

Token      eyJhbGciOi... . eyJzdWIiOi... . SflKxwRJ...
           header          payload         signature
```

Figure 1: The three parts of a JWT, decoded (illustrative values)

Verification now happens on every request without any storage lookup, which makes it lightweight. The lecturer names three advantages. **Statelessness** removes server-side storage and its cost. **Scalability** follows because every instance that holds the secret can verify a token, so a service can be scaled to many instances, or split into microservices, without any of them coordinating on authentication; a client can also present the same token to servers on other domains, where a cookie would not be sent. **Portability** comes from the compact, URL-friendly format, which fits in headers, cookies, URLs and other storage of limited size, including the browser's local storage, where, the lecturer stresses, it should not be kept.

Two disadvantages follow from the same statelessness. A stolen token lets anyone impersonate the user until it expires, and the server cannot revoke one user's access; the only built-in remedy, changing the secret, logs out every user of the platform. A blacklist in Redis or a database restores revocation, but, as the lecturer observes, if every request needs a lookup anyway, the stateful approach could have been used from the start.

> **A subtle point.** Three details go beyond the lecture. Base64url is an encoding, not encryption: anyone can read a JWT's payload, so it must hold no secrets. Theft and revocation are usually limited by a short `exp` (expiry) claim combined with a refresh token that is stored, and can be revoked, on the server. And with an asymmetric algorithm such as RS256, services verify tokens with a public key and need not hold the signing secret.

The lecturer's practical advice is to use an authentication provider, such as Auth0 or Clerk, for medium and large production systems unless the team is very confident in its own workflows: choosing and maintaining secure techniques, from the hashing algorithm to the salt, is then the provider's job. When learning, by contrast, implementing authentication by hand is the best way to understand its parts and trade-offs.

3.4 Cookies

> **Definition.** A cookie is a small value that a server stores in the user's browser and that the browser sends back automatically with every later request to that server.

Cookies are a browser feature built into the HTTP request workflow. A server cannot read other servers' cookies, an important security property of browsers, and the browser attaches a cookie without any action by the user or by application code. After a successful login the server sets a

cookie containing a session ID or a JWT; the browser returns it with every request, and the server validates it to identify and authorize the user. The cookie thus automates the round trip of the token, controlled entirely from the server side.

```
HTTP/1.1 200 OK              (response to the login)
Set-Cookie: sid=7f3a9c...; HttpOnly; Secure;
            SameSite=Lax; Path=/; Max-Age=900

GET /notes HTTP/1.1          (every later request)
Cookie: sid=7f3a9c...
```

Figure 2: A server setting a session cookie and the browser returning it

An authentication cookie should be `HttpOnly`, so that injected JavaScript cannot read it; `Secure`, so that it travels only over HTTPS; and `SameSite`, set to `Lax` or `Strict`, so that it is restricted on requests initiated by other sites. Automatic sending is also what makes cross-site request forgery (CSRF) possible: another site can make the browser send a request that carries the victim's cookie (see Chapter 16).

4 STATEFUL VERSUS STATELESS AUTHENTICATION

The lecturer now turns to the types of authentication used in daily work: stateful and stateless authentication, compared in this section; API keys (Section 5); and OAuth 2.0 with OpenID Connect (Section 6).

4.1 Stateful authentication

The client, typically a browser, sends a username or e-mail address and a password. The server verifies them, generates a session ID, bundles it with the user's data and stores both, usually in Redis because of its fast access, and returns the ID in an `HttpOnly` cookie that JavaScript cannot read. Every later request carries the cookie; the server checks that the session exists and has not expired, loads the user's data, and lets the call proceed. The ID may be any cryptographically random string, or even a JWT: what makes the flow stateful is that the server's store holds the truth about the login.

```
POST /login  {email, password}        client -> server
             credentials verified
             SET session:7f3a9c... (Redis, TTL 15 min)
                 = {user: 42, role: user}
200 OK       Set-Cookie: sid=7f3a9c...; HttpOnly

GET /notes   Cookie: sid=7f3a9c...    client -> server
             GET session:7f3a9c... -> user 42 -> serve
```

Figure 3: The stateful flow from login to an authenticated request

4.2 Stateless authentication

The server verifies the credentials and returns a JWT with the user's ID and role, signed with a secret key that must itself be stored securely. The client sends the token with every request, by convention in the `Authorization` header. The server verifies the signature, reads the user ID, and either serves the request or returns an error. No store is consulted: everything the server needs is in the self-contained token.

```
GET /notes HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

server: verify the signature with the secret key
  valid    -> sub = 42, role = user -> handle request
  invalid  -> 401 Unauthorized
```

Figure 4: A stateless request carrying a JWT

The lecturer leaves the error status open ('unauthorized or forbidden or whatever the convention is'); Section 8.5 shows that a missing or invalid token calls for 401.

4.3 Trade-offs

Stateful authentication gives centralized control and real-time knowledge of every active session, so access is easy to revoke, whether one session or all of a user's. It suits applications with moderate traffic and strict session requirements, and the lecturer recommends it for most applications. Its costs are limited scalability and greater operational complexity when servers in several regions share a store. **Stateless** authentication scales, needs no session store, suits distributed architectures and fits mobile apps, in which browser cookies are not the natural mechanism. Its weakness is revocation: if a user reports a compromised account, the only built-in remedy is changing the secret, which logs out everyone.

	Stateful	**Stateless**
State lives in	Server store (Redis, database)	The signed token itself
Client sends	Session ID in a cookie	JWT in the Authorization header
Per-request check	Store lookup	Signature verification
Revocation	Immediate	Hard until expiry
Scaling	Needs a shared store	Any instance can verify
Best for	Web applications	APIs, mobile, distributed systems

Table 2: Stateful and stateless authentication compared

4.4 A hybrid

The lecturer's resolution is to choose per client: stateful sessions for the main web application in the browser, where cookies work naturally and revocation matters most, and stateless tokens for mobile apps and for third-party integrations between servers, where scalability and simplicity matter more. Each kind of client then receives the mechanism that suits it, and the product as a whole is not forced to accept the weaknesses of either approach everywhere.

5 API KEYS

API keys serve a different need from the flows above: programmatic access to a server rather than to its user interface.

5.1 How API keys work

In a platform's dashboard the user clicks a button such as 'Generate API key' and receives a cryptographically secure

random string, created with a set of permissions and possibly an expiry date. Attached to requests, usually in a header, it grants direct access to the server behind the interface, and with it to the operations that the interface itself performs.

5.2 An example

The ChatGPT interface sends questions to OpenAI's servers and renders the answers. OpenAI also issues API keys to people who want the underlying models programmatically, either for an interface of their own or inside their own server. When a request arrives with a key, the provider identifies the customer and looks up the plan, the remaining quota and the permissions, all of which are kept on the provider's side.

5.3 Machine-to-machine communication

The usual interaction on the web is human: a person uses a mouse and keyboard on an interface, the interface sends programmatic requests to the server, and the responses are rendered for the person to read. In *machine-to-machine* communication no human and no interface take part: a platform's server, asked by a user to summarize a paragraph, calls OpenAI's API with the platform's key and returns the summary (Figure 5). Stateful and stateless flows need a human to fill in a login form; a machine simply keeps its key in an environment variable or a secret store (see Chapter 13) and sends it with each request. API keys are easy to generate and ideal for this kind of traffic.

```
1. user -> our server     POST /summarize  (cookie)
2. our server -> provider POST /v1/...
                          Authorization: Bearer sk-...
                          provider: key -> account, plan,
                                    quota, permissions
3. provider -> our server the summary
4. our server -> user     the summary, shown in our UI
```

Figure 5: A human request that triggers a machine-to-machine call

A subtle point. Some practices go beyond the lecture. A key is handled like a password: the provider shows it once and stores only a hash of it (a fast hash is acceptable here, because a long random key, unlike a human-chosen password, cannot be guessed). It carries the narrowest permissions the client needs, is rotated periodically, is revoked at once if it leaks, and is never shipped in front-end or mobile code, from which anyone can extract it.

6 OAUTH 2.0 AND OPENID CONNECT

The last type of authentication arose from a problem the others cannot solve: one application acting for a user on another application.

6.1 Too many passwords

Every platform meant another username and password, and the more sites a person used, the more credentials had to be created, remembered or kept in a password manager. Two problems followed. The first was a **security risk**: in the early internet era users reused passwords such as `12345` or `password` across sites, so a single breach compromised many accounts. The second was **fatigue**: the sheer number of accounts overwhelmed users.

6.2 The delegation problem

Websites also began to need resources held by other websites: a travel app scanning Gmail for flight tickets, a social app importing a user's contacts. This *delegation problem* was first solved by users handing over their passwords, with disastrous results. A password gives full access, with no way to limit permissions, and revoking it means changing the password on every site where it is reused.

6.3 OAuth: tokens instead of passwords

OAuth, short for open authorization, was born in 2007, when engineers from companies such as Google and Twitter standardized delegation through tokens. A password grants everything: with someone's Google password one can view the person's photos and maps history and read, add or delete contacts and calendar entries. A token grants a specific permission on a specific part of the account, such as reading the contacts, and nothing else.

Definition. Delegated authorization lets a resource owner grant a client limited, scoped access to resources held on another server, without sharing the owner's credentials.

Four roles take part: the **resource owner**, the user who owns the data; the **client**, the application requesting access; the **resource server**, which holds the data; and the **authorization server**, which authenticates the user and, after consent, issues the token. Google may operate both servers, but the roles are distinct. The client redirects the user to the authorization server; the user logs in there and grants the requested permissions; the authorization server sends a token to the client; and the client uses the token to access the resource, without ever seeing the password.

6.4 OAuth 2.0

OAuth 1.0 was complex to implement and relied on error-prone cryptographic signatures. OAuth 2.0 introduced **bearer tokens**, usable by whoever holds them: simpler, but dangerous if stolen, so they may travel only over HTTPS. It also offered **flows** for different kinds of application (Table 3). The client-credentials flow serves calls between servers with no user involved; in the device-code flow a device without a keyboard, such as a smart TV, shows a code that the user approves on a phone.

A subtle point. The lecturer dates OAuth 2.0 to 'around 2010 or so'. Drafts began then, but OAuth 2.0 was finalized in October 2012 as RFC 6749, with bearer tokens specified in RFC 6750. OAuth 1.0 was completed in December 2007 (RFC 5849, 2010) and spoke of the user, the consumer and the service provider; the four role names above are those of OAuth 2.0.

Flow	Intended for (lecture)	Status today
Authorization code	Server-side web apps	Recommended for every client, with PKCE
Implicit	Browser-based apps	Deprecated
Client credentials	Machine to machine	Current
Device code	Limited-input devices, such as smart TVs	Current (RFC 8628)

Table 3: The OAuth 2.0 flows named in the lecture

A subtle point. The lecture calls the implicit flow discouraged; it is deprecated. It returned the access token in the redirect URL, from which it leaks through browser history, logs and referrers. The OAuth 2.0 Security Best Current Practice (RFC 9700, January 2025) recommends the authorization code flow with PKCE (RFC 7636) for all client types, and requires PKCE for public clients such as single-page and mobile apps. The client sends the hash of a random *code verifier* with the authorization request and the verifier itself when it redeems the code, so an intercepted code is useless. The password grant, in which the client collects the user's password, must not be used.

6.5 OpenID Connect

OAuth solved authorization through delegation, but not authentication: it tells a client what it may access, not who the user is. OpenID Connect (OIDC), built on OAuth 2.0's security mechanisms and finalized in 2014, adds the missing identity layer. It is the reason the lecture introduced JWTs early: they reappear here.

Definition. OpenID Connect extends OAuth 2.0 with an *ID token*, a JWT in which the authorization server states who the user is.

The ID token carries the user ID, the time of issue, the issuer and the profile data the client requested, such as name and e-mail address. It powers 'Sign in with Google', Facebook or Discord: the platform takes the user's identity from the provider and stores or uses it instead of running a login of its own.

6.6 The flow in practice

In the lecturer's example a note-taking app offers 'Sign in with Google' and also wants the user's Google Keep notes. It redirects the user to Google's authorization server, where the user logs in and grants the requested permissions. The client receives an authorization code (in some variants an ID token as well), exchanges the code for an access token and an ID token, reads the user's identity from the ID token, and uses the access token to fetch the approved notes and nothing else (Figure 6).

```
browser -> client       clicks 'Sign in with Google'
client -> browser       302 to auth.example/authorize
                          ?client_id, redirect_uri, state,
                          scope=openid email profile ...,
                          code_challenge (PKCE)
browser -> auth server  user logs in, approves the scopes
auth server -> browser  302 to redirect_uri
                          ?code=...&state=...
client -> auth server   POST /token  code + code_verifier
                          (+ client secret if server-side)
auth server -> client   access_token, id_token (a JWT)
client -> resource srv  GET /notes
                          Authorization: Bearer ...
```

Figure 6: The authorization code flow with OpenID Connect and PKCE

A subtle point. The lecture has the client exchange the code with the resource server. The exchange takes place at the authorization server's token endpoint; the resource server never sees the code and accepts only the access token. The ID token is meant for the client, which must validate its signature, issuer, audience and expiry; the access token is meant for the resource server and is not proof of identity.

The lecturer calls the two protocols the key makers of the digital age: they ensure that no user or platform obtains more access than was granted, and together they turned password-sharing chaos into secure integration between platforms.

7 CHOOSING AN AUTHENTICATION METHOD

The lecturer closes the discussion of authentication with rules of thumb, summarized in Table 4.

Type	Ideal for	Example
Stateful	Web applications, much of SaaS; session data kept on the server	A browser dashboard
Stateless	APIs and distributed systems; tokens carry the user's data	A mobile app's API
OAuth 2.0 / OIDC	Third-party integrations; login through external providers	'Sign in with Google'
API key	Machine-to-machine traffic; single-purpose API access	A backend calling an AI provider

Table 4: When to use which type of authentication

In the lecturer's experience stateful and stateless authentication cover most of the APIs one builds. The types also combine: one product may use sessions for its web app, tokens for its mobile app, OpenID Connect for social login and API keys for its integrations, as the hybrid of Section 4.4 already suggested. Whatever the combination, the advice of Section 3.3 still holds: a production system may delegate much of this work to an authentication provider, provided its engineers understand what the provider does on their behalf.

8 AUTHORIZATION AND ROLE-BASED ACCESS CONTROL

Authorization is a smaller topic than authentication, but every non-trivial backend needs it: it decides what an authenticated user may do.

8.1 The need: a dead zone for deleted notes

In the lecturer's note-taking platform, a deleted note moves to a trash bin and, after 30 days, to a *dead zone*: it disappears from the user's view but remains in the database. The platform's creator wants to manage these notes through an admin interface that ordinary users must not reach.

8.2 A naive solution

A secret 'god-mode' string sent with each request could unlock the special capabilities. Anyone who intercepts it, however, can wipe the database or tamper with other users' data, and giving the same access to maintainers means sharing the string or adding more strings, which soon becomes unmanageable. The general requirement is that users differ in their capabilities; in a multi-tenant product, for instance, an organization's admin grants read and write access to some members and read-only access to others. Providing specific permissions to specific users is what authorization means.

8.3 Role-based access control

Definition. Role-based access control (RBAC) assigns each user a *role*, such as user, moderator or admin, and grants *permissions*, that is, operations on resources, to roles rather than to individual users.

In the lecturer's first example the user role may only read, while the admin and moderator roles may read and write. Custom roles, each with its own permissions, and permissions that differ per resource make the scheme as granular as the product needs: on notes, users may read, write and delete; on dead-zone notes, only admins have any access (Table 5).

Resource	user	moderator	admin
Own notes	read, write, delete	read, write, delete	read, write, delete
Shared notes	read	read, write	read, write
Dead-zone notes	none	none	read, restore, purge

Table 5: An illustrative RBAC matrix for the note-taking platform

8.4 RBAC in the request cycle

At sign-up the server assigns a role, usually user. On every request the client presents its session ID or JWT. Early in the request cycle the server determines the role, from a token claim or a database lookup, and attaches it to the request for the middleware and handlers that follow (see Chapter 6). The dead-zone handler serves an admin and answers a user with 403 Forbidden: the user lacks the permission for this resource.

```
// 1. authentication middleware: who is calling?
user = verifySession(req) or verifyJwt(req)
if user is null:  return 401 Unauthorized
req.ctx.user = user;  req.ctx.role = user.role

// 2. authorization, in the dead-zone handler
if not can(req.ctx.role, "read", "deadZoneNotes"):
    return 403 Forbidden
return deadZoneNotes.list()
```

Figure 7: Authentication and role check in the request cycle (pseudocode)

8.5 401 versus 403

The two status codes correspond to the two questions of this paper.

- **401 Unauthorized**: *no valid identity*. The credential is missing, malformed or expired, its signature is wrong, or the session does not exist. The client should authenticate and retry, and the response should carry a `WWW-Authenticate` header naming the expected scheme.
- **403 Forbidden**: *identity known, permission missing*. The user is authenticated, but the role does not allow the operation; logging in again will not help.

Despite its name, 401 signals a failure of authentication. A missing or invalid token is therefore a 401, and the 403 of the RBAC example belongs to an authenticated user without the admin role. Some systems answer 404 instead of 403 when even the existence of a resource must stay hidden (see Chapter 16).

A subtle point. A role read from a JWT claim is a snapshot taken when the token was issued: if an admin's role is withdrawn, the old token keeps granting admin rights until it expires, the revocation problem of Section 3.3 again. A role looked up in the database is always current, so systems that put roles in tokens keep them short-lived or check sensitive permissions against the database.

9 SECURITY PRACTICES

The lecture ends with two practices for implementing authentication. Both concern information that leaks not through the data a server returns but through the way it fails.

9.1 Generic error messages

Messages such as 'user not found', 'incorrect password' or 'account locked due to too many failed attempts' help legitimate users, but they also help attackers (Table 6). An attacker with a list of target accounts skips the names that do not exist and concentrates brute-force or dictionary attacks on those confirmed to exist. Every failure in the authentication workflow should therefore return the same generic message, such as 'Authentication failed'. Friendly, specific messages remain appropriate in other workflows, such as input validation (see Chapter 5).

Message	What an attacker learns
'User not found'	No account here; try the next e-mail on the list
'Incorrect password'	The account exists; attack its password
'Account locked'	The account exists, and when to try again
'Authentication failed'	Nothing about which check failed

Table 6: Helpful messages and what they reveal

A subtle point. The login form is not the only leak. A sign-up form that answers 'e-mail already registered' and a reset form that answers 'no such account' reveal the same information; the usual remedy is a neutral reply, such as 'if an account exists, we have sent an e-mail'. The status code and the response time must not differ either.

9.2 Timing attacks

A timing attack, rare but real, extracts information from how long a server takes to respond. The lecturer's typical login handler performs three checks in sequence: it finds the user by username or e-mail; it checks whether the account is locked, as servers often lock an account for some minutes or a day after too many failed attempts; and it hashes the submitted password in the same way as at sign-up and compares the result with the hash stored in the database. An unknown user fails at the first step and is answered quickly. A wrong password fails only after the hash, which takes noticeably longer. The difference, perhaps 200 ms, tells an attacker which usernames exist, even when every message reads 'Authentication failed'.

```
unknown user    lookup: none          -> 401 after   5 ms
locked account  lookup: ok, locked    -> 401 after   6 ms
wrong password  lookup: ok, unlocked,
                hash + compare: no    -> 401 after 250 ms
```

Figure 8: Three failures, one message, three response times (illustrative)

A subtle point. The lecturer speaks of the server encrypting the submitted password 'using the same key'. Password hashing is not encryption and uses no key; nothing is ever decrypted. What is reused at login is the salt and cost stored with the hash, which bcrypt and Argon2 embed in the stored string and apply in their verify function. The deliberate slowness of these algorithms (Section 2.4) is exactly what makes the timing gap large.

9.3 The lecturer's defences

The lecturer's principle is that response times must be equalized, and he names two methods. The first is **constant-time comparison**: cryptographically secure functions for comparing password hashes, whose execution time does not depend on how similar the inputs are. The second is a **simulated delay**: when the username does not match, the server waits, with `setTimeout` in Node.js or `time.Sleep` in Go, for about 200 ms before answering, so that the two failures look alike.

9.4 What actually closes the gap

A subtle point. Neither method, as stated, removes the gap of Figure 8. Constant-time comparison protects only the comparison of two secrets: a naive comparison stops at the first differing byte and so reveals how much of a guessed value matched. That matters for API keys, tokens and signatures, but does nothing about the hash that the unknown-user path skips, which is the real source of the difference. A fixed delay added to both paths keeps the difference intact. A delay added only to the fast path must equal the hashing time, which varies with load, hardware and cost factor, so an estimate such as 200 ms leaves a residual gap that an attacker can measure by averaging many requests.

Two fixes do work, and they can be combined.

- **Do the same work on every path.** When the user does not exist, or the account is locked, verify the submitted password against a *dummy hash*, computed once at start-up with the same algorithm and cost, and discard the result. Every failure then includes one full hash computation.
- **Pad every failure to the same total duration.** Record when the request started and, before answering, sleep until a fixed deadline above the slowest path, for example 500 ms. The padding is computed per request, so the total duration, not the added delay, is constant.

```
DUMMY = argon2id_hash(random_bytes(32))  # at start-up

def login(email, password):
    start = now()
    user = users.find(email)
    stored = user.password_hash if user else DUMMY
    ok = argon2id_verify(stored, password)  # always runs
    if user is None or user.locked or not ok:
        sleep_until(start + 500 ms)   # pad failures
        return 401, "Authentication failed"
    return 200, create_session(user)
```

Figure 9: A login handler without a timing side channel (pseudo-code)

Constant-time comparison remains the right tool wherever a secret is compared directly, such as an API key or a webhook signature (see Chapter 26). Standard libraries provide it: `crypto.timingSafeEqual` in Node.js, `subtle.ConstantTimeCompare` in Go and `CryptographicOperations.FixedTimeEquals` in .NET; password-hashing libraries already use it inside their verify functions.

10 CONCLUSION

- Authentication answers *who are you?*; authorization answers *what can you do?*, and depends on the identity that authentication establishes.
- Proof of identity moved from trust to seals, passphrases and passwords, then to hashing, public-key cryptography, Kerberos tickets and multi-factor authentication.
- Passwords are stored only as salted hashes from a slow, tunable algorithm such as bcrypt, scrypt or Argon2id, never in plain text or under a fast hash.

- Sessions give stateless HTTP a memory; JWTs move the state into a signed, readable token; cookies carry either automatically and should be HttpOnly, Secure and SameSite.
- Stateful authentication offers control and revocation, stateless authentication scale; a hybrid gives each client the mechanism that suits it.
- API keys serve machine-to-machine access. OAuth 2.0 delegates scoped access without passwords, OpenID Connect adds identity, and every client should now use the authorization code flow with PKCE.
- RBAC grants permissions to roles. A missing identity is a 401, a missing permission a 403.
- Failed logins must look identical in message, status and time: one generic message, and the same work or the same total duration on every path.

APPENDIX A REVIEW QUESTIONS

1. A breach exposes a users table of two million rows in which each password is stored as the SHA-256 hash of the password followed by one salt shared by the whole application. Explain in what order an attacker recovers passwords and why the shared salt helps so little. Then design a migration to Argon2id that protects every stored account immediately, without asking two million users to reset their passwords.
2. A web application keeps its sessions in Redis. After a second region with its own Redis instance is added and a global load balancer starts sending each request to the region with the lowest latency, users of the web application report being logged out at random, sometimes several times an hour, while the mobile app, which uses JWTs, is unaffected. Explain the symptom, and compare two possible fixes in terms of latency, consistency and how quickly a session can be revoked.
3. A login handler answers every failure with 'Authentication failed'. To hide its timing, a developer added `sleep(200 ms)` on the unknown-user path when bcrypt at cost 10 took about 200 ms on the team's servers; a locked account is still rejected directly after the lookup. A year later the cost factor is raised to 12. Predict the response times an attacker measures for an unknown, a locked and an existing account, and explain what the outcome shows about the original fix.
4. A platform issues JWTs valid for 24 hours that carry a `role` claim and are verified with an RS256 public key by five microservices. An admin demotes a moderator who has been deleting other users' posts. Describe what the demoted moderator can still do, for how long and why, then design a change that bounds this window to five minutes without a database query in every service on every request, and state what the change costs.
5. A travel app wants to find flight confirmations in its users' Gmail. Its developer requests a scope with full read and write access to the mailbox, keeps the access token in the browser's local storage, and identifies the user by having the browser send that access token to the backend, which calls a Google API with it and takes the e-mail address from the response. Evaluate each of the three decisions and describe the flow you would use instead, stating which token serves which purpose and where each is kept.
6. To protect its tokens from cross-site scripting, a team moves its JWT from local storage, from where it was sent in the `Authorization` header, into a cookie marked `HttpOnly` and `Secure`. Because the frontend runs on `app.example.com` and the API on `api.example.net`, the cookie also needs `SameSite=None`. Which attack becomes harder, which becomes possible that was not before, and what must the API now add? Relate the answer to the defences of Chapter 16.
7. A multi-tenant notes product stores one `role` column on its users table, copies it into the JWT and checks it in middleware. A new requirement lets one person be an admin in organization A and a read-only member of organization B. Explain what the current design gets wrong, redesign the data model and the per-request check, and decide which status code a member of A should receive when requesting a note of B by its ID, justifying the choice between 403 and 404.
8. A partner's server exports about 50,000 records from your API every night. Compare a long-lived API key, the OAuth 2.0 client-credentials flow and a JWT with a one-year expiry that you issue by hand, in terms of what your server stores, how access is scoped, how credentials are rotated and what happens after a leak. Then explain what changes if the partner may export only the records of customers who have individually agreed to it.

APPENDIX B GLOSSARY

Term	Meaning
Authentication	Establishing who a subject is
Authorization	Deciding what a subject may do
Session	Server-side state about a user, keyed by a session ID
Cookie	A value stored in the browser and sent back automatically
JWT	A signed, self-contained token of claims
Hashing	A one-way, deterministic transformation; passwords need a slow, tunable one (bcrypt, scrypt, Argon2id)
Salt	A random per-user value combined with a password before hashing
API key	A random secret for programmatic, machine-to-machine access
OAuth 2.0	A framework for delegated, scoped access (RFC 6749, 2012)
Bearer token	A token that grants access to whoever presents it
PKCE	A one-time verifier that binds an authorization code to its client
OpenID Connect	An identity layer on OAuth 2.0; its ID token is a JWT stating who the user is
RBAC	Access control that grants permissions to roles
401 / 403	No valid identity / identity known but permission missing
Timing attack	Inferring secrets from differences in response time
Constant-time comparison	A comparison whose duration does not depend on the inputs

APPENDIX C FURTHER TOPICS

Broken object level authorization

A role check answers whether a user may read notes at all, not whether the note with ID 9172 belongs to that user. When a handler loads an object by an ID taken from the request without checking its owner or tenant, any authenticated user can read or change other users' data by changing the number in the URL; the flaw is also called an insecure direct object reference (IDOR). The OWASP API Security Top 10 (2023) ranks it first among API risks. The robust defence is to scope every query by the caller's identity, for example `WHERE id = $1 AND owner_id = $2`, instead of checking ownership in scattered handlers.

Attribute- and relationship-based access control

RBAC becomes awkward when a permission depends on more than a role: on the resource's department, the time of day, or whether a document was shared with the caller. Attribute-based access control (ABAC, described in NIST SP 800-162) evaluates policies over attributes of the subject, the resource, the action and the environment, often in a policy engine such as Open Policy Agent or Cedar. Relationship-based access control (ReBAC) derives permissions from a graph of relations, such as user, member of team, editor of folder; Google described its global system of this kind, Zanzibar, in 2019, and OpenFGA and SpiceDB follow its model. These models suit sharing features and multi-tenant products that RBAC could express only with hundreds of roles.

Session fixation

In a session fixation attack the attacker obtains a valid but unauthenticated session ID and makes the victim's browser use it, for instance through a crafted link or a sibling subdomain that can set cookies. If the server keeps the same ID when the victim logs in, the attacker's copy of that ID now opens an authenticated session. The remedy is to issue a fresh session ID, and invalidate the old one, at login and at every change of privilege, such as completing a second factor or entering an admin area. Frameworks provide a function for this, but hand-written session code often omits it.

Refresh token rotation and reuse detection

Short-lived access tokens limit the damage of theft only if the long-lived refresh token is protected as well. With rotation, every use of a refresh token returns a new one and invalidates the old; if an already-used refresh token appears again, either the attacker or the legitimate client holds a stolen copy, and the authorization server revokes the whole chain, forcing a new login. RFC 9700 requires refresh tokens of public clients, such as single-page and mobile apps, to be either rotated in this way or sender-constrained. The cost is server-side state for each chain and care with concurrent refreshes from the same client.

JWT validation pitfalls

Verifying 'the signature' leaves room for well-known mistakes. A token whose header declares `"alg": "none"` carries no signature, and some early libraries accepted it; in an algorithm-confusion attack a server that expects RS256 receives an HS256 token signed with the server's own public key used as the HMAC secret. A verifier must therefore fix the accepted algorithms and keys in its configuration, never take them from the token, and must check `exp`, `iss` and `aud`, so that a token issued for another service is refused. Issuers publish their public keys as a JSON Web Key Set (JWKS) and name the key in each token's `kid` header, which lets keys rotate without downtime; RFC 8725 collects these practices.

Sender-constrained tokens

A bearer token works for whoever holds it, so a token copied from a log, a proxy or a compromised device is as good as the original. Sender-constrained tokens bind the token to a key that only the legitimate client holds: with mutual TLS (RFC 8705) the token is tied to the client's certificate, and with DPoP (RFC 9449) the client signs a short proof for every request with a private key whose public part is recorded in the token. A stolen token is then useless without the private key. They are used for high-value APIs such as open banking, and RFC 9700 recommends them wherever plain bearer tokens are too risky.

Passkeys

Passkeys are FIDO credentials, used through the W3C WebAuthn API, that replace the password with a key pair:

the server stores the public key and, at each login, sends a random challenge that the user's device signs once the user unlocks it with a fingerprint, face or PIN. Each credential is bound to the domain that registered it, so a look-alike phishing site cannot obtain a usable signature, and a breach of the server exposes no secret that works anywhere. Unlike the hardware keys of earlier deployments, passkeys can be synchronized across a user's devices by the platform's credential manager. For a backend, adopting them means storing credential IDs and public keys per user, verifying signed assertions with a maintained library, and offering account recovery that is no weaker than the passkey itself.

Time-based one-time passwords

The six-digit codes of authenticator apps follow TOTP (RFC 6238): server and app share a secret, usually exchanged through a QR code at enrolment, and each computes an HMAC of the current 30-second time step, truncated to a few digits. The server accepts the code of the current step and usually one step on either side to tolerate clock drift, must refuse a code that has already been used, and must rate-limit attempts, since six digits allow only a million values. Unlike a password, the shared secret cannot be stored as a hash, because the server needs it to compute the code; it is encrypted at rest instead. TOTP raises the cost of stolen passwords considerably but, unlike passkeys, can still be phished in real time by a proxy site.

Credential stuffing and password policy

Credential stuffing replays e-mail and password pairs leaked from other sites against a login endpoint, exploiting the password reuse described in Section 6.1; it succeeds without any weakness in the target's own hashing. Defences combine rate limits per account and per source address, step-up checks such as CAPTCHA or a second factor when traffic looks automated, and lockouts that are temporary, since a permanent lockout lets anyone who knows a user's e-mail address deny that user service. NIST SP 800-63B also reshaped password policy: it favours length and long passphrases over composition rules and periodic forced changes, and asks that new passwords be checked against lists of breached and common values, which Have I Been Pwned offers through a k-anonymity API that never receives the full password hash.

Validation & Transformation — One-Page Overview

Chapter 5 · every section and sub-section of the paper at a glance

1 Introduction: validation in a layered backend

Input is checked at one precise point of the request cycle.

- **1.1 Three layers** — repository (storage), service (business logic), controller (HTTP concerns: status codes, response format and the validation of incoming data).
- **1.2 The entry point** — after the route is matched, before the controller calls a service or anything reaches the database.

2 Why validate at the entry point

Unchecked input fails late and obscurely; checked input fails early and clearly.

- **2.1 Expected shape** — body, query, path parameters and headers; `name` must be present, a string, 5–100 characters, checked in that order.
- **2.2 Without validation** — `{"name": 0}` travels to PostgreSQL; the client gets a 500 it cannot act on, or a bad row is stored silently.
- **2.3 With validation** — 400 Bad Request with an error per field before any work is done; demo `POST /api/valid/syntactic`.

3 Kinds of validation

Structure, meaning, type and the relations between fields.

- **3.1 Syntactic** — e-mail, phone and date formats; 'Invalid email format'. Form is not existence.
- **3.2 Semantic** — a birth date cannot lie in the future; an age must be from 1 to 120.
- **3.3 Type** — string, number, boolean, array, object; rules reach each array element.
- **3.4 Complex** — the confirmation must equal the password; `partner` is required only when `married` is true.
- **3.5 Strictness** — as strict as the endpoint needs; rules that need stored state belong to the service.

4 Transformation

Change input into the form the server needs, before or after the checks.

- **4.1 Strings** — `?page=2&limit=20` arrives as `"2"` and `"20"` and fails a number rule through no fault of the client.
- **4.2 Casting** — convert strictly, then check page 1–499 and limit 1–9,999; a failed cast is a 400.
- **4.3 Normalization** — lowercase the e-mail, add '+' to the phone, reformat the date: one form per value.

5 One validation and transformation pipeline

A reusable middleware driven by a schema per endpoint.

- **5.1 Schema** — declarative rules per endpoint; Zod, Pydantic, FluentValidation and others.
- **5.2 One place** — every input rule together; the controller receives clean, typed data; undeclared fields are rejected or stripped.
- **5.3 Errors as documentation** — field messages tell a client what to send, never how the server works.

6 Frontend versus backend validation

The same checks serve different purposes.

- **6.1 Two purposes** — frontend: user experience and instant feedback; backend: security and data integrity.
- **6.2 No frontend** — Postman and Insomnia call the API directly, skipping every form check.
- **6.3 Together** — the form blocks a bad e-mail without a request; the valid request returns 200.

Validation at a glance — after route matching, one pipeline casts and normalizes the input and checks its types, syntax, meaning and cross-field rules; a failure returns 400 with an error per field before any business logic runs, and frontend checks only improve the experience. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Validation and Transformation: Checking and Shaping Input at the Entry Point of an API

Chapter 5 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — This paper places validation and transformation at the entry point of an API, after route matching and before the controller calls any service or repository. It explains why unchecked input fails late, deep inside the system and as an unhelpful 500, while a validation pipeline answers early with 400 Bad Request and a message for each field. It classifies validation as syntactic, semantic, type-based and complex, each illustrated by the lecturer's demo endpoints. It then describes transformation, from casting string query parameters to numbers to normalizing e-mail addresses and phone numbers, and the single schema-driven pipeline that holds every input rule. It closes by separating frontend validation, which serves user experience, from backend validation, which serves security and data integrity and can never be skipped.

Keywords — input validation, syntactic validation, semantic validation, type validation, transformation, type casting, normalization, 400 Bad Request, client-side validation

1 INTRODUCTION: VALIDATION IN A LAYERED BACKEND

Validation and transformation are not a large or complex topic but a set of rules to keep in mind while designing an API. Both serve the integrity of the data a backend stores and the security of the backend itself. This paper places them in the request cycle, explains why they belong at the entry point (Section 2), classifies the kinds of validation (Section 3), describes transformation (Section 4) and the pipeline that combines the two (Section 5), and separates frontend from backend validation (Section 6).

> **Definition.** *Validation* checks that incoming data has the structure, types and values an endpoint expects, and rejects the request if it does not. *Transformation* changes incoming data into the form the server needs, for example by casting a string to a number or lowercasing an e-mail address.

1.1 Repository, service and controller

A typical backend executes a request in layers (see Chapter 6). The *repository* layer deals with persistent storage, whether a relational database, Redis or another store: connections, queries, inserts and deletes. The *service* layer executes the business logic of an API call; it calls one or more repository methods and may also send notifications and e-mails or make webhook calls. The *controller* layer calls the service method that belongs to the route and returns what that method produces to the client. Controller and service are kept apart so that everything concerning HTTP stays in one layer: which success or error code to return, in what format the data goes out, and the validation of the data that comes in.

1.2 The entry point

When a request arrives, the router matches its path and calls the controller method assigned to that route (see Chapter 2). Validation and transformation run exactly there: after the route is matched and before the controller executes any significant logic, calls a service or reaches a database (Figure 1).

```
client: JSON body, query, path parameters, headers
  -> route matching
  -> validation & transformation pipeline
       fails:  400 Bad Request with field errors
       passes: controller -> service -> repository
                 -> database
```

Figure 1: Where the validation and transformation pipeline runs

2 WHY VALIDATE AT THE ENTRY POINT

2.1 Every input has an expected shape

Clients all over the world send requests to the same server, and every part of a request is data they control: the JSON body, the query parameters, the path parameters and the headers. Before any of it enters the server's logic, the server confirms that it has exactly the format the endpoint needs. Suppose an endpoint expects a JSON body with a field `name` holding a string of 5 to 100 characters. The pipeline checks this one level at a time (Figure 2): first whether the field is present, then whether its value is a string rather than, say, an array or a boolean, then whether its length is within bounds, so that neither a whole paragraph nor a two-character string passes. Each failure stops the request with its own message, and the rules can be as specific as the endpoint requires.

```
rule  name: required, string, 5 to 100 characters

{}                        -> name: Required
{"name": 0}               -> name: Expected string
{"name": "Al"}            -> name: Too short (min 5)
{"name": "<paragraph>"}   -> name: Too long (max 100)
{"name": "Dune Messiah"} -> passes
```

Figure 2: Checks on one field, applied in order (illustrative messages)

2.2 Without validation: a failure deep inside

Suppose the endpoint creates a book and a client sends `{"name": 0}`. With no validation, the value travels through

the controller and the service to a repository method that inserts a new row into a `books` table, whose column was declared in PostgreSQL as `name TEXT NOT NULL`. A relational database enforces the types and constraints of its columns, so, in the lecturer's account, the insert fails and, depending on the server's error handling, the client receives 500 Internal Server Error. That status misplaces the fault: a 5xx code says the server failed, a 4xx code that the request must change (see Chapter 1). The client's data was at fault, yet the client learns only that something unexpected happened and has nothing to correct, a very poor experience for a basic form.

A subtle point. This particular insert would not fail. PostgreSQL converts other types to text on assignment, so the row is stored with the name `'0'`, and drivers that send parameters as text produce the same result. A silent bad row is worse than an error. The database does reject a missing name (`NOT NULL`) or text sent to an integer column, as described. Either way, its constraints are a last line of defence, not a substitute for validation.

2.3 With validation: an early 400

Checked at the entry point, the same request is rejected with 400 Bad Request and a message stating which constraint failed, so the client can retry with correct data. Nothing significant has run: no service method, no query, no half-finished operation such as an e-mail sent for a record that was never created, and the system never reaches an unexpected state. The first demo, sent with the API client Insomnia to a local server, shows the behaviour (Figure 3). A `POST` to `/api/valid/syntactic` with an empty JSON body returned an array of errors: the endpoint expects `email`, `phone` and `date`, and all three are required. With a random string as e-mail, a number as phone and a valid date, two errors remained. A well-formed address and a phone number sent as a string then produced 200 OK. Some APIs answer 422 Unprocessable Content when well-formed JSON fails validation and keep 400 for malformed requests; the choice is a convention of the API (see Chapter 7).

```
POST /api/valid/syntactic
{}
400  email: Required
     phone: Required
     date:  Required

{"email": "abcd", "phone": 98765, "date": "2025-11-05"}
400  email: Invalid email format
     phone: Expected string, received number

{"email": "test@test.com", "phone": "98765",
 "date": "2025-11-05"}
200  OK
```

Figure 3: Three attempts at the syntactic endpoint (illustrative values)

3 KINDS OF VALIDATION

The lecturer names three kinds of validation that cover most situations; a fourth, rules that relate several fields, appears in the demos (Table 1). The categories are not strict. They build on one another, as the `name` example showed: a value must be present and of the right type before its structure can be parsed, and well formed before its meaning or its relation to other fields can be judged, so each later rule runs only once the earlier ones hold.

Kind	Question	Examples from the lecture
Syntactic	Is the structure right?	E-mail, phone number, date format
Semantic	Does the value make sense?	Birth date not in the future; age 1–120
Type	Is the data type right?	String, number, boolean, array elements
Complex	Do the fields agree?	Passwords match; partner if married

Table 1: Four kinds of validation

3.1 Syntactic validation

Syntactic validation checks that a value follows a required structure. An e-mail address has a local part, the character @ and a domain: a name followed by one of thousands of top-level domains such as `.org`, `.co` or `.in`. A phone number has a country code followed by the number of digits that country uses, for example ten. A date must follow the format the API expects, such as year, month and day. Whatever the language or library, the validation algorithm compares the string with the structure and raises an error on a mismatch, as with 'Invalid email format' in Figure 3. Two limits apply. Syntax proves only form: `test@test.com` is valid although nobody may read it, and only a verification message proves that an address exists. And in the demo the phone rule required no more than a string, so any quoted digits passed; a real syntactic rule, such as the E.164 format (a plus sign, a country code and at most 15 digits), is stricter.

3.2 Semantic validation

Semantic validation checks whether a well-formed value makes sense. A date of birth entered in a form cannot lie in the future, and an age of 365 is impossible, at least for now. The semantic demo endpoint expected a date of birth and an age. An empty body returned an error for each: the date of birth is required, and the age must be a number. With 1995-06-12 and 43 the request succeeded and echoed the values. Moving the year of birth to 2026 produced 'Date of birth cannot be in the future', and an age of 430 produced 'Number must be less than or equal to 120'. The endpoint accepts ages from 1 to 120, a logical range for a person even though one-year-olds rarely call APIs.

3.3 Type validation

Type validation is the basic check that each field has the expected data type: string, number, boolean, array, or a nested JSON object whose fields have types of their own. The type demo expected four fields: a string, a number, an array and a boolean. With a string in every field, the server answered 'Expected number, received string', 'Expected array, received string' and likewise for the boolean. With 10 as the number, `[1, 2]` as the array and `false` as the

boolean, a new error appeared inside the array: element 0 was expected to be a string but was a number (Figure 4). Rules apply to every element of an array, not only to the array itself. With `["1", "2"]` each field matched its type and the request succeeded.

```
{"stringField": "text", "numberField": 10,
 "arrayField": [1, 2], "booleanField": false}
400  arrayField[0]: Expected string, received number

{..., "arrayField": ["1", "2"], ...}
200  all four fields echoed
```

Figure 4: A type rule that reaches inside an array (illustrative field names)

3.4 Complex validation

The last endpoint expected `password`, a password confirmation and `married`. Platforms ask for a password twice to make sure the user typed what they meant, so the rule is that the two values must be equal. With two different strings and `married` set to `false`, the server returned 'Passwords don't match' on the confirmation field and 'String must contain at least 8 character(s)' on the password. Once both held the same string of eight characters, the request succeeded. Setting `married` to `true` then produced 'Partner name is required when married is true': the field `partner` is optional while `married` is false and mandatory when it is true. With a partner's name added, the server returned 200 and echoed every field. The lecturer calls such rules complex because they relate several fields and follow directly from the requirements of the service layer. Each error is attached to the field that must change, so a client can display it beside the right input.

3.5 How strict to be

None of these categories needs to be memorized. They are questions to ask while designing an endpoint: how strict and how specific to be about each field, from very loose to very tight, as the requirements demand. One boundary is worth adding. The input pipeline decides what can be decided from the request alone. Rules that depend on stored state, such as whether an e-mail address is already registered or a referenced record exists, need the database and belong in the service layer (see Chapter 6).

4 TRANSFORMATION

Transformation executes operations on the data provided by the client to bring it into the form the server expects. It may run before validation, so that the checks can work at all, or after it, to give the service layer a convenient structure; it can also depend on the outcome of validation or supply default values, such as a page size when the client sends none.

4.1 Query parameters arrive as strings

A paginated endpoint such as `GET /bookmarks?page=2&limit=20` must validate its query parameters like any other input. Its requirements are that `page` is a number greater than 0 and less than 500, and that `limit` is a number greater than 0 and less than 10,000, so that a single request never returns more than 10,000 records. Query values, however, always reach the server as strings, whether they come from a browser or any other client: they may look like numbers, but `page` arrives as `"2"` and `limit` as `"20"`. The first requirement, being a number, therefore fails, and the client receives an error such as 'page should be a number, received string', although the client made no mistake. On the wire every value is text; a parser may also turn a repeated key, as in `?page=1&page=2`, into an array, one more reason to check types.

4.2 Casting before validation

Making these values usable is the server's responsibility. *Casting* means forcing a value of one data type into another; here the pipeline casts the strings to numbers and only then checks the ranges (Figure 5). Path parameters and headers are text as well and need the same treatment. Frameworks with typed binding perform the cast themselves: in ASP.NET Core an action parameter declared as `int page` is converted from the query string, and a value that cannot be converted is reported as a validation error.

```
GET /bookmarks?page=2&limit=20

raw query       page = "2"       limit = "20"
validate only   page: Expected number, received string
cast first      page = 2         limit = 20
then validate   page 1..499      limit 1..9999   passes
```

Figure 5: Casting query values before checking their ranges (illustrative messages)

> **Note.** Casting must be strict, or it turns invalid input into valid-looking values. In JavaScript, `Number("")` is 0, `parseInt("2abc")` is 2 and `Boolean("false")` is `true`. A safe pipeline accepts only text that is entirely a number, or exactly `true` or `false`, reports a failed cast as a validation error (400), never as a 500, and applies the remaining checks to the converted value.

4.3 Normalizing values

Transformation also reshapes valid data for the service layer. The transformation demo expected the same three fields as the syntactic one. The client sent an e-mail address in mixed upper and lower case, a phone number without the leading plus sign and a date in one particular format; the response showed the address in lower case, a plus sign added to the phone number and the date in another format (Figure 6). The pipeline had changed every value before the business logic ran. One canonical form per value means, for instance, that `Test@Mail.com` and `test@mail.com` cannot become two accounts, and that lookups match however the user typed the address.

```
sent       {"email": "TeSt@Test.COM",
            "phone": "919876543210",
            "date":  "2025-11-05"}

returned   {"email": "test@test.com",
            "phone": "+919876543210",
            "date":  "2025-11-05T00:00:00.000Z"}
```

Figure 6: The transformation demo, before and after the pipeline (illustrative values)

5 ONE VALIDATION AND TRANSFORMATION PIPELINE

5.1 Middleware and schema

The pipeline is implemented as a middleware function or a utility function that every endpoint reuses (see Chapter 6). Each endpoint provides a *schema*, a declarative description of the input it expects; the pipeline walks through the fields of the input, applies the transformations and checks every rule (Figure 7). Schema libraries exist for most stacks, for example Zod in TypeScript, Pydantic in Python, and data annotations or FluentValidation in ASP.NET Core, whose API controllers answer 400 automatically when a model fails validation.

```
schema SemanticBody
  dateOfBirth           date     required, not after today
  age                   number   required, 1..120

schema ComplexBody
  password              string   at least 8 characters
  passwordConfirmation  string   must equal password
  married               boolean
  partner               string   required if married

schema BookmarksQuery   (transform first, then check)
  page                  "2" -> 2       1..499
  limit                 "20" -> 20     1..9999
```

Figure 7: The demo endpoints' rules as declarative schemas (illustrative pseudo-code)

5.2 One place for every input rule

Validation and transformation are paired in a single pipeline so that all the logic of the input layer stays in one place. Whoever needs to know what an endpoint requires, or what is done to its data before the business logic runs, reads one schema instead of searching controllers and services. The controller then receives data that is valid and already in the form it needs, never the raw request. A schema should also settle what happens to fields it does not declare: rejecting or stripping them stops a client from slipping in a property such as `role` that the service never meant to accept (see Chapter 16).

5.3 Errors as documentation

Errors that name every failing field and the reason let the client correct everything in one round trip. The demos showed a further benefit: a client that sends an empty body learns which fields the endpoint expects and what values they accept, which helps when API documentation is missing or out of date. The messages should describe the client's input and nothing else; database errors and stack traces never belong in them (see Chapter 12).

6 FRONTEND VERSUS BACKEND VALIDATION

6.1 Two purposes

A common mistake is to treat frontend validation as a replacement for backend validation. The confusion arises because forms validate their own inputs: a field `name` is checked for its minimum and maximum length or its type, errors appear in the form itself, and the API is called only when the user submits valid data. Frontend validation exists for user experience: immediate feedback that the data does not match what is expected. It contributes nothing to security or data integrity, which are the purposes of backend validation. Every endpoint needs both (Table 2).

	Frontend validation	**Backend validation**
Purpose	User experience, instant feedback	Security, data integrity
Runs	In the form, before a request	At the entry point, before logic
Bypassed by	Any API client, e.g. Insomnia	Nothing
Status	Recommended	Mandatory

Table 2: Frontend and backend validation compared

6.2 Clients without a frontend

A server has many kinds of clients. A web application may validate thoroughly, but an API client such as Postman or Insomnia calls the endpoints directly, with no user interface acting as a proxy and therefore no frontend validation at all; every demo in this lecture was sent that way. A backend that relies on frontend checks for integrity breaks as soon as the client changes. Server-side rules must therefore be as strict and specific as possible, written without regard to what any client validates.

> **Warning.** Any check that runs in a client can be skipped by sending the request another way. Validation also narrows what reaches the server without replacing defences such as parameterized queries and output encoding (see Chapter 16).

6.3 The two layers together

The lecturer's demo frontend offers a form for each endpoint. With an invalid e-mail address, the phone number 12345 and a date chosen from a date picker, pressing Submit sent no request at all: the form's own check had rejected the e-mail field. After the address was corrected to one at gmail.com, the form sent the request, and the browser's developer tools showed the payload, the response preview and the status 200. The client-side check gave the user instant feedback; the server-side check is the one that protects the data. Because both enforce the same requirements, they are best derived from one specification, or from one shared schema when client and server use the same language.

7 CONCLUSION

- Validation and transformation run after route matching and before the controller calls any service, so invalid input costs no work and leaves no unexpected state.
- Without them, bad input fails deep inside, as a 500 the client cannot act on, or is stored silently; with them, the client receives 400 and a message for each field.
- Syntactic rules check structure, semantic rules meaning, type rules data types down to array elements, and complex rules the relations between fields.
- Rules that need stored state, such as uniqueness of an e-mail address, belong in the service layer.
- Query, path and header values arrive as text; the server casts them, strictly, before applying numeric rules such as page 1–499 and limit 1–9,999.
- Normalization gives each value one canonical form, such as a lowercased e-mail address or a phone number with its plus sign.
- One schema-driven pipeline keeps every input rule in one place, and its field errors double as documentation.
- Frontend validation serves user experience; backend validation serves security and data integrity and is mandatory, because API clients bypass the frontend.

APPENDIX A REVIEW QUESTIONS

1. A bookmarks endpoint casts every query value with JavaScript's `Number()`, then checks `page` 1–499 and `limit` 1–9,999, and its query parser turns a repeated key into an array. Predict the outcome of `?page=&limit=20`, `?page=2&limit=1e3`, `?page=2&page=3` and `?page=0x10`, and decide for each whether a strict pipeline should behave differently, and why.
2. Users created through the web form can log in, but some users imported through a partner's bulk endpoint cannot, although their addresses look identical in the admin panel; a few imported addresses even exist twice, once in capitals. The login handler lowercases the address before its lookup. Diagnose the cause from these symptoms, and describe both the code change and the data repair needed so that the problem cannot recur.
3. The users table contains rows with an age of `-3` and a date of birth in 2031, although the web form checks both fields and the backend schema declares `age` as a number and `dateOfBirth` as a date. Explain how these rows were most likely written, and evaluate a colleague's proposal to add PostgreSQL `CHECK` constraints instead of extending the pipeline, considering the status code and message the client would receive in each design.
4. Design the input schema for `POST /transfers`, whose body holds `fromAccount`, `toAccount`, `amount`, `currency`, an optional `scheduledFor` and a free-text `note`. For each rule you introduce, state its kind (type, syntactic, semantic or complex) or explain why it belongs in the service layer instead, and say which values the pipeline should transform before the service receives them.
5. Two teams disagree about error responses. One returns 400 with the first failing rule as a plain string; the other returns 422 with an array of every failing field and its message. A mobile app must highlight every invalid field of a twelve-field form over a slow network, and a partner's server logs failures for later analysis. Compare the two designs for both clients, including what each status code tells a client under HTTP semantics.
6. A sign-up endpoint reports 'email: already registered' as a field error in a 400 response with the same shape as its format errors. Using the attacks of Chapter 4, explain what an attacker gains from this response, and redesign the endpoint so that clients still receive precise messages for malformed input while nothing reveals whether an account exists. State in which layer each of your checks runs.
7. A `PATCH /users/me` handler binds the JSON body directly onto the user entity and saves it. A client sends `{"name": "Ada", "role": "admin", "emailVerified": true}`. Predict the outcome with no schema, with a schema that strips undeclared fields and with one that rejects them, and argue which policy suits a public API whose mobile clients are updated months after the server.
8. A pipeline checks that `name` has 5 to 100 characters and also trims surrounding whitespace. Predict the stored value and the response for `" Al "` and for a 100-character name followed by three spaces, first when the trim runs after the length check and then when it runs before. From the outcomes, derive a general rule for ordering transformations and checks in a pipeline.

APPENDIX B GLOSSARY

Term	Meaning
Validation	Checking input against the rules an endpoint expects
Transformation	Changing input into the form the server needs
Syntactic validation	Checking structure, such as an e-mail format
Semantic validation	Checking that a value makes sense, such as a past birth date
Type validation	Checking data types, including array elements
Complex validation	Rules that relate several fields
Casting	Converting a value from one data type to another
Normalization	Reducing variants of a value to one canonical form
Schema	A declarative description of an endpoint's expected input
Validation pipeline	Reusable middleware that transforms and validates input
Query parameter	A key–value pair in the URL, always received as text
400 Bad Request	The request is invalid and must be changed by the client
500 Internal Server Error	An unexpected failure on the server
Frontend validation	Checks in the client, for user experience
Backend validation	Checks on the server, for security and data integrity
Data integrity	Accuracy and consistency of stored data

APPENDIX C FURTHER TOPICS

JSON Schema and OpenAPI

JSON Schema is a standard, language-independent vocabulary for describing JSON documents: types, required properties, string patterns and formats, numeric ranges, array items, and whether additional properties are allowed. The schemas of Section 5 can be written in it instead of in one library's syntax, and OpenAPI 3.1, the usual format for describing HTTP APIs (see Chapter 25), uses JSON Schema 2020-12 for request and response bodies. One description can then drive server-side validation, generated client code and the published documentation, so that the three cannot drift apart. Validators exist for most languages, but they differ in which optional formats, such as `email` or `date-time`, they actually enforce.

Parse, don't validate

The phrase, from a 2019 essay by Alexis King, names a design in which input is checked once at the boundary and converted into a type that can hold only valid values, instead of being checked and then passed on as the same raw string or dictionary. A function that takes an `EmailAddress` or a `PageNumber` cannot be called with unchecked data, so later layers neither repeat the checks nor forget them. Zod's `parse`, Pydantic models and C# types with validating constructors work in this spirit. The type system then enforces the boundary between the pipeline and the service layer.

Allow-list validation

An allow-list accepts only values known to be good, such as an enumeration of sortable fields, a pattern of permitted characters or the ISO 4217 currency codes, and rejects everything else. A deny-list, which rejects known-bad input such as `<script>` or `DROP TABLE`, fails as soon as an attacker finds an encoding, a case variant or a construct the list did not foresee. OWASP therefore recommends allow-lists for every field whose valid values can be stated. They matter most where input is placed into code or queries that cannot be parameterized, such as a column name in an `ORDER BY` clause.

Mass assignment

Mass assignment happens when a framework binds every field of a request onto a model and saves it, so a client can set properties the form never showed, such as `isAdmin` or `balance`. In 2012 a researcher used it against GitHub, a Ruby on Rails application, to obtain write access to the Rails repository itself, after which Rails adopted strong parameters; ASP.NET Core calls the same flaw over-posting. The robust defence is a dedicated input model (a DTO) per endpoint that declares only the fields a client may set and is mapped explicitly onto the entity, rather than binding request bodies to database entities. The OWASP API Security Top 10 (2023) lists it under broken object property level authorization.

Problem Details for HTTP APIs

RFC 9457 (2023, replacing RFC 7807) defines a standard JSON shape for error responses, served as `application/problem+json`, with the members `type`, `title`, `status`, `detail` and `instance`, and allows extension members such as a list of field errors. A common shape lets clients handle failures from every endpoint, and from different APIs, with one piece of code instead of parsing ad hoc messages. API controllers in ASP.NET Core answer validation failures with it by default, as a `ValidationProblemDetails` object whose `errors` member maps each field to its messages. The format applies to every 4xx and 5xx response, not only to validation (see Chapter 12).

Regular-expression denial of service

The regular-expression engines of JavaScript, Python, Java and .NET backtrack: on a pattern with nested or overlapping quantifiers, such as `^(a+)+$`, a crafted input that almost matches makes the engine try exponentially many paths, and a request of a few dozen characters can occupy a CPU core for seconds or longer. Validation is where such patterns typically live, for e-mail addresses, URLs and phone numbers, and in a single-threaded runtime such as Node.js one such request blocks every other. The defences are to cap the input length before matching, to prefer simple, anchored patterns or well-tested library validators, to set a match timeout where the engine offers one, as .NET does, or to use a linear-time engine such as RE2.

ReDoS is a reminder that every validation rule is code running on hostile input.

Unicode normalization

The same visible text can be encoded by different sequences of code points: *é* may be one precomposed character or an *e* followed by a combining accent, and full-width or ligature forms look like ordinary letters. Unicode Standard Annex #15 defines normalization forms that map such variants to one representation: NFC and NFD for canonical equivalence, NFKC and NFKD also for compatibility characters. Without normalization two usernames that look identical can both be registered, and uniqueness checks and lookups miss each other, the Unicode counterpart of the lowercasing in Section 4.3. Identifiers are therefore normalized, usually to NFC or NFKC, and case-folded before validation and storage; look-alike letters from different scripts, such as Cyrillic and Latin *a*, need a separate confusables check.

Dates, times and time zones

ISO 8601 and its internet profile, RFC 3339, define unambiguous text formats such as `2025-11-05` and `2025-11-05T14:30:00+05:30`, the safest forms for an API to accept and return. The harder problem is meaning: a timestamp is an instant and needs an offset or `Z`, whereas a date of birth is a calendar date that belongs to no time zone. Converting such a date to midnight UTC, as in Figure 6, can make it appear as the previous day to a client west of Greenwich, and a semantic rule such as 'not in the future' depends on whose today is meant. Backends therefore store instants in UTC, keep pure dates in date types, and reject timestamps without an offset rather than guess one.

Limits before parsing

Some checks must run before the body is parsed at all, because parsing itself consumes memory and CPU. A server caps the request body size and answers 413 Content Too Large beyond it, rejects media types it does not accept with 415 Unsupported Media Type, and bounds the nesting depth of JSON, the length of arrays and the number of fields, which a schema cannot do once the parser has already built the whole document. Servers and frameworks provide these limits, for example Kestrel's `MaxRequestBodySize` in ASP.NET Core and the `limit` option of Express's JSON parser. Without them a single oversized or deeply nested request can exhaust a server before any validation rule runs.

Handlers, Services, Repositories & Middleware — One-Page Overview

Chapter 6 · every section and sub-section of the paper at a glance

1 Introduction: the request life cycle inside the server

What happens between the port and the response, in three cooperating layers.

- **1.1 From the entry point to the handler** — the OS forwards the request to the listening port (3000, 4000); routing maps `/users/123` to a handler function.
- **1.2 Why three layers** — not a requirement but a design pattern: code that scales, is maintainable, easy to extend and easy to debug.

2 The handler (controller)

Owns everything about HTTP: data in, status code and data out.

- **2.1 Request and response objects** — supplied by the runtime to every handler, in Go, Express, Python or ASP.NET Core.
- **2.2 Extracting and binding** — query for GET, body for POST, PUT, PATCH; JSON deserialized into a struct or class; failure is 400.
- **2.3 Validation and transformation** — validate every input, then set defaults such as `sort = date` when none is sent.
- **2.4 Calling the service** — clean data plus the user ID and permissions from authentication.
- **2.5 Sending the response** — map the outcome to 200, 201, 204, 4xx or 500, with data or an error.

3 Services and repositories

Business logic without HTTP above, one database operation per method below.

- **3.1 The service** — a plain function: reading it, one cannot tell it serves an API; it may only send an e-mail.
- **3.2 Orchestration** — several repository methods, merged results, external APIs, notifications.
- **3.3 The repository** — builds the query from insert, filter and sort data, runs it, returns the result.
- **3.4 One kind of data per method** — all books and one book by ID are two methods, not one with an optional parameter.

4 Middleware

Optional functions at the boundaries of the life cycle.

- **4.1 Functions in the middle** — before routing, between routing and handler, until the response is sent.
- **4.2 Request, response and next** — `next` passes control on, or the middleware answers and ends the request.
- **4.3 Why middleware** — common work written once for thousands of endpoints; bad requests stop early.
- **4.4 Order matters** — the registration order is the processing order.

5 Common middleware

Security, observability, error handling and payload processing.

- **5.1 CORS** — compare `Origin` with the allowed origins and add headers; the browser blocks the rest.
- **5.2 Security headers and authentication** — CSP on every response; bad token: 401; valid: user into the context.
- **5.3 Rate limiting** — e.g. 30 calls in 2 seconds per IP; above it, 429 Too Many Requests.
- **5.4 Logging and monitoring** — method, path, query and body of every request.
- **5.5 Global error handling** — any error becomes a structured 4xx or 5xx; it must enclose what it guards.
- **5.6 Compression and parsing** — gzip for large JSON; parsing, binding and validation can move here.
- **5.7 A typical order** — CORS, logging, authentication, …, error handling last.

6 Request context

Key–value state scoped to one request and shared across its boundaries.

- **6.1 State scoped to one request** — set by one middleware, read by later ones without explicit passing.
- **6.2 The authenticated user** — `POST /books` takes the owner's ID from the context, never from the body.
- **6.3 Request ID and tracing** — a UUID in every log line and in `X-Request-ID` to other services.
- **6.4 Cancellation and deadlines** — signals that stop downstream work when the client leaves or time runs out.

The request inside the server at a glance — middleware handles what every request needs, the handler translates between HTTP and plain data, the service does the work and the repository talks to the database, while the request context carries the user, the request ID and cancellation through all of them. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Handlers, Services, Repositories and Middleware: The Life of a Request Inside the Server

Chapter 6 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — This paper follows a request inside the server, from the port on which the operating system delivers it, through routing, to the function that handles it. It describes the handler, or controller, which binds the request data into native types, validates and transforms it, calls a service and turns the outcome into an HTTP response. It then separates the service layer, which holds the business logic and orchestrates work without any knowledge of HTTP, from the repository layer, in which each method performs one kind of database operation. It explains middleware, functions that receive the request, the response and a next function and run between these stages, and why their order matters, using CORS, security headers, authentication, rate limiting, logging, global error handling and compression as examples. It closes with the request context, state scoped to one request that carries the authenticated user, a request ID and cancellation signals across middleware and handlers.

Keywords — request life cycle, handler, controller, service layer, repository pattern, middleware, next function, request context, separation of concerns

1 INTRODUCTION: THE REQUEST LIFE CYCLE INSIDE THE SERVER

The lecture treats three topics together: handlers, services and repositories; middleware; and the request context. Each describes part of what happens to a request inside the server, and explaining them separately would repeat the same picture three times. Earlier lectures followed a request from the client to the server and back (see Chapter 1); this one follows it from the moment it reaches the server until the response leaves. The paper describes the handler (Section 2), services and repositories (Section 3), middleware (Section 4) and its common kinds (Section 5), and the request context (Section 6).

1.1 From the entry point to the handler

A server process listens on a port, such as 3000 or 4000. When an HTTP request arrives for that port, the operating system forwards it to the process: this is the *entry point* of the request, and everything that happens from here until the response is sent is the request life cycle inside the server. The request first reaches the routing mechanism (see Chapter 2), which matches its method and path, static such as `/users` or dynamic such as `/users/123`, to a *handler*: a function defined in advance to handle requests to that API. From the handler the work may pass to a service and from there to a repository, which talks to the database; the result travels back the same way and the handler sends the response (Figure 1).

```
client --HTTP--> operating system --port 3000--> server
server entry point (process listening on port 3000)
  -> routing      GET /books  ->  ListBooks
  -> handler      HTTP in, HTTP out
     -> service       business logic
        -> repository     database queries
           -> database
        <- rows
     <- result
  <- response     status code, headers, body
```

Figure 1: The path of a request inside the server (illustrative)

1.2 Why three layers

The handler could do all the work itself. Nothing requires the code to be split into a handler, a service and a repository; the split is a design pattern, adopted so that the code base scales as it grows, stays maintainable, makes new features easy to add and makes faults easy to locate. Each layer has one kind of responsibility. Express and Go speak of handlers, MVC frameworks such as ASP.NET Core of controllers and their actions; this paper uses the two terms interchangeably.

> **Definition.** The *handler* or *controller* is the function a route maps to; it owns everything about HTTP: reading the input, validating it, choosing the status code and the response format. The *service* executes the business logic of an operation, independently of HTTP. The *repository* performs database operations: it constructs a query, runs it and returns the result.

2 THE HANDLER (CONTROLLER)

2.1 The request and response objects

In every server framework the lecturer has used, in Go, Node.js and Python, a handler receives at least two objects: a *request object*, from which it reads the values of the HTTP request, and a *response object*, through which it sets the status code, modifies the response headers and sends the body. The programmer does not create them; the language runtime or the framework supplies them as parameters for every request (Figure 2). Some frameworks vary the shape: a Django view receives the request and returns a response object, and an ASP.NET Core action receives bound parameters and returns a result, with both objects available through `HttpContext`.

```
// Go, net/http
func ListBooks(w http.ResponseWriter, r *http.Request)

// Node.js, Express
app.get("/books", (req, res) => { ... });

// Python, Django: request in, response returned
def list_books(request): ...

// ASP.NET Core, minimal API
app.MapGet("/books", (HttpContext ctx) => ...);
```

Figure 2: Request and response objects supplied by four frameworks (illustrative)

When the handler starts, it already knows which operation the client wants, because routing chose this handler. What it needs is the client's data, so that the operation can run: database operations, other logic, external API calls, e-mails, notifications. Depending on the request it then sends content back or not; the lecturer's example is a successful DELETE, which by REST convention returns 204 No Content with an empty body.

2.2 Extracting and binding the data

The first responsibility of a handler is to take the data out of the request object, and where it looks depends on the method: a GET request carries its input in the query parameters, a POST, PUT or PATCH request in the body, and path parameters and headers may matter for any method. The client serialized its data, usually as JSON, the most popular format across the frontend–backend boundary, so that it could travel over the network (see Chapter 3). The handler therefore deserializes the JSON into the native data structures of its language: a struct in Go or Rust, a dictionary or an instance of a class in Python. Most frameworks call this step *binding*: the request body is bound to a native type. ASP.NET Core calls it model binding and fills action parameters from the body, the route, the query string and the headers.

In Node.js the handler usually skips this step, because a middleware registered before routing, such as `express.json()` (formerly the separate body-parser package), has already turned the JSON into JavaScript objects. In Go or Python the handler binds explicitly. If binding fails, because the body is not valid JSON or a value cannot be converted to the declared type, the handler responds with 400 Bad Request to tell the client that the payload is invalid, and the request ends there: nothing further runs.

> **A subtle point.** The lecture mentions reading a body on DELETE requests and says REST requires 204 for a successful DELETE. HTTP defines no meaning for content in a DELETE (or GET) request, and some implementations reject such requests, so identifiers belong in the path and filters in the query. And 204 is the usual choice when nothing needs to be returned, not a rule: RFC 9110 also allows 200 with a body describing the result, and 202 when the deletion has only been accepted.

2.3 Validation and transformation

Once bound, the data is in a form the language understands. JavaScript handles JSON smoothly because JSON was modelled on JavaScript object syntax; other languages integrate less smoothly and need this explicit conversion before the data means anything to them. The second step is validation (see Chapter 5). Some implementations have no validation layer, but in a production-grade backend everything that comes from an external client is validated, path parameters, query parameters, body and headers, as specifically as possible: mandatory values are present, every value has the expected format, nothing is missing and nothing malicious slips through. A failure again ends the request with 400.

After validation the handler may, optionally, transform the data: modify it so that the downstream layers can process it conveniently. The lecturer's example is a GET endpoint that lists books and accepts a query parameter `sort` with the value `name` or `date`, the date the book was created; the books are sorted by that field. A good pattern to establish early in API design is to make query parameters optional wherever possible (see Chapter 7). A client that calls `GET /books` with no parameters therefore passes validation without a 400, and the transformation step sets the default: `sort = date`, or `name` if the requirements say so. Without the default, the service would have to check whether `sort` was sent and branch on the answer; with it, every request follows one expected path through the code, which is more readable. Transformations that the checks themselves need, such as casting query strings to numbers, run before validation (see Chapter 5).

2.4 Calling the service

With data that is bound, validated and transformed, the handler calls the service method for the operation. It passes that data together with everything else the service needs that is not part of the client's input, such as the authenticated user's ID and permissions (Section 6). The service then does the actual processing.

2.5 Sending the response

When the service returns, the handler completes its fourth and last step: it turns the outcome into a response (Figure 3). Depending on whether the service succeeded, it chooses the status code: 200, 201 or 204 on success, according to the operation; a 400-series code when the client was at fault; 500 when the server was. It sends an appropriate error message on failure, or the success data, here an array of books with 200 OK. The handler thus controls the flow of data in both directions, from client to server and back, which is why it is also called a controller. To tell client faults from server faults, it needs the service to report failures in a form it can classify, for example as distinct error types such as 'not found' or 'conflict' that it maps to 404 or 409.

```
handler ListBooks(request, response):
  // 1. bind: query string -> native struct
  query, err = bind(request.query, ListBooksQuery)
  if err: return response.status(400).json(err)

  // 2. validate, then transform
  if query.sort not in (none, "name", "date"):
    return response.status(400).json(fieldErrors)
  if query.sort is none: query.sort = "date"

  // 3. call the service with clean data
  books, err = bookService.listBooks(query.sort)

  // 4. respond
  if err: return response.status(500).json(error)
  return response.status(200).json(books)
```

Figure 3: The four steps of a handler for GET /books (illustrative pseudo-code)

3 SERVICES AND REPOSITORIES

Below the handler the work splits again: the service decides what must happen, the repository how data is read or written. Table 1 summarises the three layers with the books example.

Layer	Responsibility	Knows HTTP?	Books example
Handler	Bind, validate, transform, call the service, respond	Yes	Defaults `sort`, returns 200 and the books
Service	Business logic and orchestration	No	Asks the repository for books sorted by `sort`
Repository	One database operation per method	No	Builds and runs the `SELECT`

Table 1: The three layers and their responsibilities

3.1 The service: business logic without HTTP

The service layer ideally contains nothing related to HTTP; all of that stays in the handler. A service method is just a function: it receives data, processes it and returns a result, without knowing which status code will be sent, in what format the response goes out or how the input was validated. The lecturer's test is that a reader looking at a service method should not be able to tell that it is used in an API. This is where the line of responsibility is drawn. The lecture does not spell out the reasons; the main ones are that such a function can be called unchanged from a background job, a command-line tool or another service (see Chapter 10), and tested without constructing HTTP requests (see Chapter 23).

Not every service needs the database. A service whose only job is to send an e-mail receives the address, executes the logic that sends the message and returns something like `{success: true}` to the handler, without calling any repository method. A service that must persist data or fetch it from the database uses the repository layer for that.

3.2 Orchestration

A service method may do many things. It can call several repository methods, merge the data they return, keep the fields each result needs and return the combination to the handler. It can also call external APIs, send notifications and e-mails, and do anything else the operation requires: all the actual processing of an API call happens in the service. Handlers deal with data formats, input and output; repositories deal with database calls; services orchestrate. That is how the three layers are isolated from one another.

3.3 The repository: building and running queries

The repository layer, also called the database layer depending on the terminology of the team, has a single responsibility. It takes the data an operation needs, whether the values to insert or the filtering and sorting parameters of a read, constructs the database query, executes it and returns whatever the database returned. In the books example the `sort` parameter travels from the handler through the service to the repository, which turns it into the ordering clause of the query (Figure 4). The repository returns the rows to the service, the service returns them to the handler, and the handler sends the response. Because `sort` comes from the client, the repository maps the validated value to a fixed column name rather than pasting it into the SQL text (see Chapter 16).

```
GET /books                    (no query parameters)
handler     bind              sort = none
            validate          passes: sort is optional
            transform         sort = "date"
service     listBooks(sort = "date")
repository  findAllBooks(sort = "date")
            SELECT * FROM books ORDER BY created_at
service     returns the books to the handler
handler     200 OK  [{"id": 1, ...}, {"id": 2, ...}]
```

Figure 4: The books example through the three layers (illustrative)

3.4 One kind of data per method

Each repository method should return one kind of data. With two operations, fetching all books and fetching one book, the repository has two methods: `findAllBooks`, which returns all books, and `findBookById`, which takes the ID of a book and returns that book. It should not have a single method with an optional parameter that returns either the list or one book depending on whether the parameter is set. A repository method does one thing: fetch, insert or delete some data, as the requirement states. A service method, by contrast, may combine many such operations, as Section 3.2 described.

4 MIDDLEWARE

4.1 Functions in the middle

Between the entry point and the moment the response is sent, the request crosses boundaries between functions. Before routing there may be functions that run on every request; after routing there may be more before the handler; and after the handler, service and repository, still more may run until the response reaches the client. These functions are called *middleware* because they execute in the middle: between the entry point and routing, between routing and the handler, between the handler and the re-

sponse, at the boundaries of different execution contexts (Figure 5).

> **Definition.** *Middleware* is a function that runs in the request life cycle between the entry point, routing and the handler. It receives the request, the response and a `next` function, and either passes execution on by calling `next` or ends the request by sending a response itself.

```
request arrives at the entry point
  [CORS]            check Origin, add headers   -> next()
  [logging]         log method and path         -> next()
  routing           GET /books -> ListBooks
  [authentication]  bad token: 401, stop
                    valid token: user to context -> next()
  [permissions]     role may list books?        -> next()
  handler -> service -> repository
  [error handling]  any error -> structured 4xx/5xx
response sent to the client
```

Figure 5: Middleware at the boundaries of the life cycle, in a typical order (illustrative)

4.2 Request, response and next

A middleware is a kind of handler with one additional capability. Like a handler, it receives the request and response objects from the runtime; in addition it receives a function conventionally called `next`. Calling `next` passes execution across the next boundary: to the next middleware or, if none follows, to routing or to the handler. It is therefore more accurate to say that `next` passes execution to the next processing context than to the next middleware. Every middleware is optional: an application uses the ones its requirements call for, and a request can go straight from routing to the handler.

Because it holds the request object, a middleware can read values from the request and modify them; because it holds the response object, it can send a response to the client itself. In that case the request never reaches routing or the handlers further along the life cycle. Reading, modifying and responding are the capabilities that make middleware useful.

4.3 Why middleware

Middleware exists for the same reason functions exist: so that code needed in many places is written once and reused. A backend receives thousands of requests a day, or millions and billions on a popular platform, and some operations apply to all of them: security checks, logging, authentication, parsing of the data. A large application also has hundreds or thousands of endpoints, each with its own handler, which cannot be avoided. Without middleware each handler would repeat the same code. Moving that code into a shared function and calling it from every handler still duplicates the call in every handler, and one forgotten call leaves an endpoint unprotected. The common operations, security, logging, data parsing, compression, are therefore delegated to a separate component of the life cycle placed before the handler. Reducing code duplication is the first reason. The second follows from the response object: a request that fails a check is answered before it reaches the handler, so precious server resources are not spent on it.

4.4 Order matters

From the entry point, each middleware passes execution to the next with `next`, through routing and further middleware, until a response is sent. The order in which middleware is registered is therefore the order in which a request is processed, and it matters. A check placed too late lets work run that it should have prevented; a middleware placed before the one that prepares its input finds nothing there; and, as Section 5.5 shows, an error handler in the wrong place never sees the errors it should handle.

5 COMMON MIDDLEWARE

The lecturer's examples fall into four groups: security (CORS, security headers, authentication, rate limiting), observability (logging and monitoring), error handling, and processing of the payload (compression and parsing). Each meets the two conditions that make code a middleware rather than a function called from handlers: it must run for every request, and it needs the request object, the response object or both (Table 2).

Middleware	Uses	Ends the request with	Otherwise
CORS	`Origin` header	Nothing; the browser blocks	Adds CORS headers
Security headers	Response headers	—	Adds headers
Authentication	Token in a header	401 Unauthorized	Puts the user in the context
Rate limiting	Client IP, counters	429 Too Many Requests	Counts the request
Logging	Method, path, query, body	—	Writes a log entry
Error handling	An error from any stage	Structured 4xx or 5xx	—
Compression	Response body	—	Compresses, e.g. gzip

Table 2: Common middleware; each passes on with `next` unless it ends the request

5.1 CORS

Cross-Origin Resource Sharing (see Chapter 1) relaxes the same-origin policy, a security and privacy mechanism of browsers: an application running in the browser cannot access resources outside its own origin, so a page from `example.com` can read responses only from `example.com`. The restriction is lifted when the remote server sends headers stating that the client's origin is allowed; the browser then does not block the response. CORS thus lets a server limit which clients, at least which web applications, can communicate with it.

On the server, the CORS middleware reads the origin of the request, which the runtime provides from the `Origin` header, and compares it with the origins the server allows, typically the domain of the application's own frontend. If it matches, the middleware adds the appropriate response

headers, such as `Access-Control-Allow-Origin`, and calls `next`. If it does not, it adds no headers, and the browser blocks the response when it arrives. CORS is a middleware because both conditions hold: it must run for every API call, and it uses the request (the origin) and the response (the headers).

A subtle point. The browser, not the server, enforces CORS. A cross-origin request from a disallowed page may still reach the server and run; the browser only withholds the response from the page. Clients outside a browser, such as curl or Postman, send any `Origin` or none, so CORS limits which web pages can read responses, while authentication limits who can use the API. Placing CORS first matters for two other reasons: a browser's preflight `OPTIONS` request carries no credentials and must be answered before authentication rejects it, and error responses need CORS headers too, or the page cannot read them.

5.2 Security headers and authentication

A security-headers middleware sets response headers that instruct the browser to apply protections, for example `Content-Security-Policy`; `Strict-Transport-Security` and `X-Content-Type-Options` are other common ones. They belong on every response, so the middleware adds them and calls `next`.

Authentication is a middleware so that the logic of extracting and verifying credentials is not duplicated (see Chapter 4). The middleware takes the token, a JWT or a session ID, from the request, from a header or wherever the implementation carries it, and verifies it (Figure 6). On failure it sends 401 Unauthorized immediately and terminates the request without calling `next`. On success it does not respond: it extracts what downstream code needs, such as the user ID and the user's role (whether the user is an admin, for example), stores it in the request context (Section 6) and passes execution on. An authenticated user who lacks permission for a route is refused later, by a permission check, with 403 Forbidden.

```
// Express: (req, res, next) supplied by the runtime
function authenticate(req, res, next) {
  const token = req.headers.authorization;
  const user = verifyToken(token);    // JWT or session ID
  if (!user) {
    return res.status(401).json({ error: "unauthorized" });
  }
  req.user = { id: user.id, role: user.role };  // context
  next();                             // pass control on
}
app.use(authenticate);
```

Figure 6: An authentication middleware (illustrative)

5.3 Rate limiting

A rate limiter prevents one client from sending a large number of requests in a short time. The team decides in advance what is reasonable, both the time window and the number of calls, for example no more than 30 calls in the last 2 seconds, or 500 calls in another window, and a middleware counts the requests, typically per IP address. A client that exceeds the limit receives 429 Too Many Requests, the code for too many requests in a short period; otherwise the middleware calls `next`. Counting by IP address has limits: behind a load balancer the server sees the balancer's address unless it reads a trusted `X-Forwarded-For` header, and many users behind one NAT share an address, so authenticated APIs often limit per user or per API key instead.

5.4 Logging and monitoring

A logging middleware records every HTTP request, in the terminal or in a log file: the path, such as `/users`, the method, such as GET, the query parameters, the request body and other parameters, with secrets such as passwords and tokens left out. The records make debugging easier, feed reports that track parameters across requests and support audits (see Chapter 14).

5.5 Global error handling

The lecturer considers global error handling very important: every backend should have it. An error can occur anywhere, in a handler, in a service or in any middleware executing between the function boundaries. The global error-handling middleware receives the error, whatever its form, and determines whether it is a client error or a server error; for this the application defines fields on its errors that mark the kind. It then sends a 400-series or 500-series status with a properly structured error body, usually a message and a code (sometimes called an ID) from which the frontend can render a user-friendly message (see Chapter 12). It thus turns unstructured errors from anywhere in the application into one consistent format.

Because errors may arise at any stage, this middleware is placed last, after everything else, just before the response is sent. Its position matters in particular: placed in the middle of the chain, it would never see an error raised later in a handler, since in the lecturer's model the request flows one way and the error cannot travel back to an earlier middleware.

A subtle point. The one-way picture fits Express, where an error-handling middleware, recognised by its four parameters `(err, req, res, next)`, is registered after the routes and receives errors passed to `next(err)`. Go's handler wrappers, Koa and ASP.NET Core nest middleware like the layers of an onion instead: each one calls the next and regains control when it returns, so code after the call runs on the way back, with the response. There the exception handler is registered first, as the outermost layer, and every error propagating back passes through it. The rule is the same in both models: the error handler must enclose everything whose errors it handles.

5.6 Compression and data parsing

A compression middleware shrinks large responses, for example JSON with thousands of fields, using an algorithm such as gzip. The response travels compressed, and modern browsers decompress it into a form the client can use, such as a JavaScript object. The client announces the en-

codings it accepts in the `Accept-Encoding` header, and the server names the one it used in `Content-Encoding`. Data parsing can be delegated to middleware too: serialization and deserialization, binding, and validation and transformation, so that individual handlers do not repeat them. Whether to do so depends on the requirements of the backend and the conventions of the code base.

5.7 A typical order

In a typical life cycle CORS runs first, so that a request from an origin that should not reach the server is identified at the earliest stage; logging follows, then authentication and checks such as permissions, and global error handling comes at the end (Figure 5). No order fits every application. Time spent deciding, from the requirements, in what order the request should be examined and modified is well spent, because the order determines what the client experiences.

6 REQUEST CONTEXT

6.1 State scoped to one request

The authentication middleware of Section 5.2 stored the user's ID and role somewhere and made them available to the next middleware and the handler. That storage is the *request context*. Every framework, language or library, whether Node.js, Go or Python, provides some implementation of it. The concept is abstract, but each implementation is concrete, and while the implementations differ, the concept stays the same across languages.

> **Definition.** The *request context* is storage, usually key–value pairs, scoped to a single request: it is attached to that request alone, readable and writable by every middleware and handler that processes it, and discarded when the request ends.

A server has clients across the world, hundreds of thousands of them, each sending many requests, and each request has its own context attached by the HTTP server. Between the entry point and the response the request crosses many function boundaries: CORS and logging middleware, routing, authentication, application-specific middleware such as permission checks, the handler and global error handling. A value that one of them sets in the context is available to every later one, without being passed explicitly from function to function and therefore without coupling the components closely together (Figure 7). The context is shared, modifiable state kept for convenience and limited to one request.

```
context = {}                          one per request
[request ID]   ctx.requestId = "9f1c..."
[logging]      log(ctx.requestId, "POST /books")
[auth]         ctx.userId = 42, ctx.role = "user"
[permissions]  may role "user" create books?  -> next()
handler        book.user_id = ctx.userId   (not the body)
service        call search service, X-Request-ID: 9f1c...
               stop if ctx is cancelled or past deadline
response sent  context discarded
```

Figure 7: One request's context, written and read along the chain (illustrative)

6.2 The authenticated user

In the lecturer's example the authentication middleware verifies the user's credentials: the e-mail and password on a login attempt, the token on later requests. For a known user it takes out the user ID, the role (user, admin, sales or any other role of the application's RBAC scheme, see Chapter 4) and permissions, such as read or write access to a resource. This metadata concerns only this HTTP request, so it is stored in the request's context. Downstream, a handler for `POST /books` must insert a book, and the `books` table has a `user_id` column. The handler takes the user ID from the context, the result of successful authentication, not from the request payload. Later middleware, the database layer or an RBAC layer use the same values to check whether this user may insert a book, for instance whether the user is an admin.

> **Warning.** Identity comes from the context, never from the payload. A client with malicious intent can put another user's ID in the body or the query and perform operations in that user's name. Every value that decides who the user is and what the user may do must come from verified credentials (see Chapter 16).

6.3 Request ID and tracing

A middleware early in the chain can generate a unique ID for the request, such as a UUID, and save it in the context. Everything that runs during the request then takes it from there: every log line includes it, and in a microservice architecture each call to another service carries it in a header such as `X-Request-ID`. When logs are reviewed for an audit or while debugging, the ID traces the request: where it started, which service received it first and which services it called (see Chapter 14).

6.4 Cancellation and deadlines

The request context also carries cancellation signals, abort signals and deadlines to downstream and external services, so that the server does not hang forever waiting for them. When the client disconnects or the deadline passes, the signal tells every operation still running for that request, such as a database query or an outgoing HTTP call, to stop. Table 3 shows how three frameworks provide the uses described in this section. Go's documentation adds a useful limit: context values are for request-scoped data, not a way to pass optional parameters to functions; the same restraint keeps the shared state of any framework small and predictable.

Use	Go	Express	ASP.NET Core
Key–value state	`context.WithValue`	`res.locals`, `req` fields	`HttpContext.Items`
Authenticated user	A context value	`req.user`	`HttpContext.User`
Request ID	A context value	A `req` field	`TraceIdentifier`
Cancellation, deadline	`ctx.Done()`, `WithTimeout`	An `AbortSignal` tied to `req.on("close")`	`RequestAborted`

Table 3: The request context in three frameworks (common mechanisms, not from the lecture)

7 CONCLUSION

- Inside the server a request passes from the port to routing to a handler; splitting the work into handler, service and repository is a design pattern for scalable, maintainable code, not a requirement.
- The handler owns HTTP: it binds the request data to native types (400 on failure), validates and transforms it, calls the service and maps the outcome to a status code and a body.
- Transformation such as a default `sort` value gives the service one predictable path instead of branches on missing input.
- The service holds the business logic and orchestration and knows nothing about HTTP; each repository method builds and runs one kind of query and returns one kind of data.
- Middleware receives the request, the response and `next`; it either passes control on or answers itself, so common work is written once and bad requests stop early.
- CORS, security headers, authentication (401), rate limiting (429), logging, global error handling, compression and parsing are typical middleware, and their order determines the behaviour.
- The global error handler must enclose everything whose errors it handles: last in Express, first in onion-style frameworks such as ASP.NET Core.
- The request context is state scoped to one request that carries the authenticated user, a request ID and cancellation signals across middleware and handlers; identity is taken from it, never from the payload.

APPENDIX A REVIEW QUESTIONS

1. A handler for `POST /orders` validates the body, computes prices, inserts the order and its items with three SQL statements, sends a confirmation e-mail and returns 201. A nightly job must now create orders from a CSV file. Show how you would split this code into handler, service and repository methods, which parts the job reuses, and where the transaction boundary and the e-mail belong.
2. An Express API registers, in this order: CORS, a JSON error-handling middleware, authentication, the routes. When a route passes a database error to `next(err)`, clients receive the framework's default HTML error page instead of the API's JSON error format. Explain why from the flow of execution, give the correct order, and say where the equivalent exception handler is registered in ASP.NET Core and why.
3. A handler for `POST /books` reads `userId` from the JSON body and stores it as the book's owner, although an authentication middleware runs before it and puts the verified user in the request context. User 7 sends `{"title": "Dune", "userId": 12}` with a valid token. Predict what is stored, what an attacker gains, and how you would change the handler and the service signature so that the flaw cannot recur.
4. After the API is moved behind a load balancer, a rate-limiting middleware set to 30 requests per 2 seconds per IP address answers 429 to most users at peak hours, although each user sends only a few requests. Explain the cause, what the middleware must read instead and under which condition that value can be trusted, and propose a better limit key for authenticated endpoints.
5. A product page needs a book, its author and the book's five latest reviews. A colleague proposes one repository method `getBook(id, includeAuthor?, includeReviews?)` whose result changes shape with the flags. Evaluate this against the rule that a repository method returns one kind of data, propose the repository and service methods instead, and say what you would change if three separate queries proved too slow.
6. A browser application on `https://app.example.com` calls `PUT /profile` on `https://api.example.com` with an `Authorization` header. The API's middleware runs in the order logging, authentication, CORS, routes. Predict step by step what the browser sends and what each middleware does, explain what the user sees, and fix the order.
7. `GET /books?sort=price` returns 500, and `GET /books` without parameters returns books in a different order on each call. The handler binds the query and passes `sort` to the service unchanged; the repository inserts it into `ORDER BY`. Explain both symptoms, which layer should have handled each, and what else this code is exposed to.
8. A service calls a slow reporting database and then an external pricing API, with no timeout on either. Clients give up after 10 seconds, yet during an incident the server's open connections and goroutines keep growing until it stops responding. Explain the mechanism, and how a deadline and cancellation carried in the request context change the behaviour from the handler down to the repository.

APPENDIX B GLOSSARY

Term	Meaning
Request life cycle	Everything a request passes through from the server's entry point to the response
Entry point	Where the operating system hands a request on a port to the server process
Handler (controller)	The function a route maps to; owns HTTP input and output
Binding	Deserializing request data into the native types of the language
Transformation	Changing validated data for convenience, e.g. setting defaults
Service layer	Business logic and orchestration, independent of HTTP
Orchestration	Combining repository calls, external APIs and notifications in one operation
Repository	Layer whose methods each perform one kind of database operation
Middleware	A function between life-cycle stages that receives request, response and `next`
`next`	The function that passes execution to the next processing context
CORS	Browser protocol by which a server lets other origins read its responses
Rate limiting	Capping the requests a client may send in a time window (429)
Global error handler	Middleware that turns any error into a structured 4xx or 5xx response
Request context	Key–value state scoped to one request, shared by middleware and handlers
Request ID	A unique ID per request, logged and forwarded to trace it across services
Cancellation signal	A signal in the context that stops downstream work for a request

APPENDIX C FURTHER TOPICS

Dependency injection

Dependency injection supplies a component with the objects it depends on instead of letting it create them: the handler receives a service, the service receives its repositories. It is how the three layers are wired together in practice, and it makes a service testable with a fake repository. ASP.NET Core has a built-in container with three lifetimes, transient, scoped and singleton, where scoped means one instance per request, the natural lifetime of a database context. In Go and Express the same wiring is usually written by hand with constructors or factory functions.

Unit of Work and transactions

When a service calls several repository methods that write data, the writes usually must succeed or fail together. The Unit of Work pattern tracks the changes made during one business operation and commits them in a single transaction; in EF Core the `DbContext` plays this role and `SaveChanges` commits. The transaction boundary belongs in the service, which knows the whole operation, not in individual repository methods, each of which sees only one step (see Chapter 8 on databases).

DTOs and mapping between layers

A data transfer object is a type that describes exactly what crosses a boundary, such as the body a handler accepts or the JSON it returns, kept separate from the entities the repository maps to tables. Separate types stop internal fields such as password hashes from leaking into responses and stop clients from setting fields they should not (over-posting). The cost is mapping code between the types, written by hand or generated by a library. A common rule is that handlers speak DTOs, repositories speak entities, and services translate.

Hexagonal and clean architecture

Hexagonal architecture (ports and adapters), introduced by Alistair Cockburn, and Robert C. Martin's clean architecture generalize the service–repository split. Business logic sits at the centre and defines interfaces, the ports; HTTP handlers, databases and message queues are adapters around it. Dependencies point inwards, so a service depends on a repository interface rather than on a database library, and storage can be replaced or a second entry point, such as a queue consumer, added without touching the business logic. For small services the extra interfaces can cost more than they return.

Route-scoped middleware and filters

Not every middleware has to run for every request: frameworks can attach middleware to one route or a group of routes. Express accepts middleware functions in a route definition and on routers; ASP.NET Core applies conventions such as `RequireAuthorization` to route groups, and its MVC filters and minimal-API endpoint filters run around a specific action with access to its bound arguments. This lets an admin area require a role, or a public endpoint skip authentication, without branches inside global middleware.

Rate-limiting algorithms

The lecture describes a limit as a number of calls per time window; implementations differ in how they count. A fixed window resets a counter at each interval and allows bursts at the window boundaries, a sliding window smooths them out, and a token bucket refills tokens at a steady rate while permitting short bursts up to the bucket size. With several server instances the counters must live in shared storage such as Redis, or each instance enforces only its own share. A 429 response may carry a `Retry-After` header, and ASP.NET Core ships these algorithms in its rate-limiting middleware.

W3C Trace Context and OpenTelemetry

`X-Request-ID` is a convention; the W3C Trace Context recommendation standardizes the idea with the `traceparent` and `tracestate` headers, which carry a trace ID and the ID of the calling operation from service to service. OpenTelemetry, the common open-source instrumentation framework, propagates these headers automatically and links

the logs, metrics and spans of one request across services. A backend that adopts it gets the tracing of Section 6.3 without hand-written propagation (see Chapter 14).

Implicit context propagation

Passing the context explicitly, as Go does with a `context.Context` first parameter, makes the dependency visible but touches every function signature. Other runtimes propagate request-scoped state implicitly across asynchronous calls: Node.js provides `AsyncLocalStorage`, and .NET provides `AsyncLocal<T>`, on which `IHttpContextAccessor` is built. A logger or repository can then read the current request ID without extra parameters, at the cost of hidden dependencies and of request data leaking into background work started from the request.

REST API Design — One-Page Overview

Chapter 7 · every section and sub-section of the paper at a glance

1 Introduction: why REST API design needs a standard

Settle recurring questions once, in one consistent style.

- **1.1 Recurring questions** — plural or singular paths, PATCH or PUT, custom actions, status codes.
- **1.2 Origin and aim** — rules made for multi-page apps, applied to SPAs; extract guidelines, not new standards.

2 The origins of REST

REST is Fielding's account of how the web was made to scale.

- **2.1 The web and its crisis** — URI, HTTP, HTML, server, browser, editor (1990); then exponential growth.
- **2.2 Six constraints** — client–server, uniform interface, layers, cache, stateless, code on demand.
- **2.3 HTTP/1.1 and REST** — HTTP/1.1, then the 2000 dissertation that named the style.

3 What "representational state transfer" means

Representations of a resource's state move between client and server.

- **3.1 Representation** — one resource, several formats: JSON, HTML, XML.
- **3.2 State** — current attributes, e.g. a cart's items and total.
- **3.3 Transfer** — representations move through HTTP methods.

4 Designing URLs

A URL names a version and plural resources in a hierarchy.

- **4.1 Anatomy** — scheme, domain, path, query, fragment.
- **4.2 API URLs** — `api.example.com/v1/books`; plural for one item too: `/books/{id}`.
- **4.3 Slugs and hierarchy** — `harry-potter`, no spaces or underscores; each slash a level.

5 HTTP methods and idempotency

Whether a repeat changes the server again guides the method.

- **5.1 Idempotency** — the same effect once or a thousand times.
- **5.2 GET, PUT and PATCH** — GET changes nothing; PATCH updates fields, PUT replaces.
- **5.3 DELETE** — the first call deletes; repeats get 404 and change nothing.
- **5.4 POST** — each call creates another book; also carries custom actions.

6 From requirements to an interface

Design the interface first, from the wireframes.

- **6.1 Why standards** — no reading code or guessing; fewer bugs and calls.
- **6.2 Wireframes and resources** — nouns: organizations, projects, users, tasks, tags.
- **6.3 Schema, actions, tool** — three tables, CRUD actions, Insomnia.

7 Listing and creating organizations

List and create share a route; lists are paginated, sorted and filtered.

- **7.1 List and create** — `GET`, `POST /organizations`; 201 returns the entity.
- **7.2 Why paginate** — serializing thousands of rows is slow; users see 10–20.
- **7.3 Response, page, limit** — `data`, `total`, `page`, `totalPages`; 5 items, limit 2: 3 pages.
- **7.4 Sorting** — `sortBy`, `sortOrder`; default `createdAt` descending.
- **7.5 Filtering** — by field: `?status=archived`.

8 Single-resource endpoints and custom actions

The ID selects the resource, the method the operation.

- **8.1 Update** — `PATCH /organizations/{id}`; 200 with the entity.
- **8.2 Get and delete** — GET 200; DELETE 204, empty body.
- **8.3 404 or empty list** — 404 for a missing resource; no matches: 200, `[]`.
- **8.4 Archive** — `POST /organizations/{id}/archive`, 200.

9 The same patterns for projects

A second resource repeats the same patterns.

- **9.1 Create and list** — `POST /projects` with `organizationId`; camelCase.
- **9.2 Consistency** — `description`, never `desc`; plural routes everywhere.
- **9.3 Single project, clone** — `POST /projects/{id}/clone` creates: 201.

10 Guidelines for a good interface

Make the API predictable for its integrators.

- **10.1 Documentation** — interactive OpenAPI/Swagger from the start.
- **10.2 Consistency** — one pattern everywhere.
- **10.3 Sane defaults** — page 1, limit 10–20, newest first; new organizations active.
- **10.4 No abbreviations** — integrators know less than the designer.
- **10.5 Design first** — a design session before any code.

REST API design at a glance — name plural resources in hierarchical routes, choose methods by meaning and idempotency, paginate lists, return status codes that describe outcomes, use POST for custom actions, and design it all consistently before any code. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

REST API Design: From Fielding's Constraints to Consistent, Predictable Endpoints

Chapter 7 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — This paper sets out conventions for designing REST APIs so that routes, methods, payloads and status codes stop being matters of debate. It traces REST to the scalability crisis of the early web, Roy Fielding's six architectural constraints and his 2000 dissertation, and explains what representation, state and transfer mean. It then derives rules for API URLs (an api subdomain, a version, plural lower-case resources, slugs and hierarchical paths) and relates the HTTP methods to idempotency, with POST as the method for custom actions. Following the lecturer's demo, it designs the endpoints of a project-management platform from its wireframes and schema: list, create, get, update and delete, pagination, sorting and filtering, the correct use of 201, 204 and 404, and custom actions such as archiving and cloning. It closes with guidelines on consistency, sane defaults, readable names and interactive documentation, and with the principle that an API is designed before it is coded.

Keywords — REST, Fielding's constraints, resource naming, idempotency, HTTP methods, custom actions, pagination, sorting and filtering, status codes

1 INTRODUCTION: WHY REST API DESIGN NEEDS A STANDARD

API design takes up much of a backend engineer's time, and this lecture treats it in full for REST, the most widely used style of web API; other technologies, such as RPC and GraphQL, are outside its scope. The paper follows the lecture: the history of REST (Section 2) and the meaning of its name (Section 3), the design of URLs (Section 4), HTTP methods and idempotency (Section 5), the workflow from requirements to an interface (Section 6), a demo that designs the endpoints of a project-management platform (Sections 7 to 9), and the guidelines that make an interface easy to integrate (Section 10).

1.1 Recurring questions

The REST style has years of research and the experience of many backend engineers behind it, yet people early in their backend work still stumble over the same questions. Should the resource name in a URI path segment be plural or singular? Is an update a PATCH or a PUT? Which method suits an operation that is neither a fetch, a create, an update nor a delete, a custom action (or action call) that asks the server to do something, and that may sound like an update, and so like PATCH or PUT, or like a creation, and so like POST? And which HTTP status code fits which scenario?

1.2 Origin of the confusion, and the aim

The lecturer traces the confusion to the time when the widespread HTTP API standards were developed. The internet, the web, its clients and its servers were very different: sites were multi-page applications (MPAs), in which the server sends a new page for each navigation. Today the frontend relies on single-page applications (SPAs): in the first request the browser downloads all the JavaScript the application needs, which then performs all routing on the client side from the browser's path and URL, in a client-heavy application. Rules written for the first world leave room for interpretation in the second. The aim is not to create new standards but to extract from the existing ones rules and guidelines that everyone can follow, so that APIs, payloads and documentation share one consistent style. With these questions settled once, an engineer can concentrate on business logic instead of worrying whether an API is RESTful or follows the latest industry practice. The lecture therefore covers API design end to end: designing resources and routes, returning success and error responses, choosing status codes and deciding what data to accept.

2 THE ORIGINS OF REST

2.1 The web and its crisis

In 1990 Tim Berners-Lee started a project called the World Wide Web to share knowledge and information with the whole world. Within about a year he had created six things for it: the URI (uniform resource identifier); HTTP, the protocol over which clients and servers communicate (see Chapter 1); HTML, the markup language that forms the skeleton of a web page; the first web server; the first web browser; and the first what-you-see-is-what-you-get HTML editor, built directly into that browser. All six are still in use in developed forms: URIs remain, HTTP has grown through HTTP/1.1 to HTTP/2 and HTTP/3, pages are still written in HTML, and there are many kinds of web servers and browsers.

The project soon ran into trouble. Its user base grew exponentially within a short time, a scale Berners-Lee had not accounted for, and the web headed towards breakdown. The planning and technologies behind the original design could not absorb the thousands of people joining every day; new techniques, standards and components were needed. Around 1993 Roy Fielding, who later co-founded

the Apache HTTP Server project, became concerned with this scalability problem and proposed a set of constraints to make the web scale (Table 1).

2.2 Fielding's six constraints

The *client–server* constraint separates concerns: the client handles the user interface and the user experience, while the server manages data storage and business logic, the division now called frontend and backend. Each side can evolve independently, which improves scalability. The *uniform interface* simplifies the overall architecture by giving the components of the web one standardized way to communicate, so that every service presents a consistent interface. It comprises four sub-constraints: identification of resources, manipulation of resources through representations, self-descriptive messages, and hypermedia as the engine of application state (HATEOAS).

A *layered system* is composed of hierarchical layers, each of which sees and interacts only with the layer immediately below it. Intermediate components such as load balancers and proxy servers, which today let applications serve millions of users, can therefore be added for scalability and security without affecting the system's core functionality. The *cache* constraint requires the server to label its responses explicitly as cacheable or non-cacheable; clients may then cache and reuse responses when they need them, which reduces server load, improves network efficiency and gives users faster responses (see Chapter 9).

Statelessness means that every request from the client carries all the information needed to understand and process it (see Chapter 1). The server does not remember earlier requests and stores no client context between them, so with each request the client sends whatever identifies it and the data to process. This improves reliability, scalability and visibility, because any server can handle any request: when two servers are added behind a load balancer that distributes traffic by an algorithm such as round robin, all of them can serve the same client. The last constraint, *code on demand*, is optional: a server may temporarily extend a client's functionality by transferring executable code such as JavaScript. It adds flexibility on the client side while the other constraints still hold, though the lecturer notes that it is not heavily used.

Constraint	Requirement	Benefit
Client–server	UI on the client; data and logic on the server	Each side evolves independently
Uniform interface	One standard way for components to communicate	Simple, consistent architecture
Layered system	Each layer sees only the adjacent one	Proxies and load balancers fit in
Cache	Responses labelled cacheable or not	Less load, faster responses
Stateless	Each request carries all it needs	Any server can serve any request
Code on demand	Optional: the server may send code, e.g. JavaScript	Extensible clients

Table 1: Fielding's constraints for a scalable web

2.3 HTTP/1.1 and the name REST

Fielding then worked with Berners-Lee to make the web scale and to standardize its design, and together, with co-authors, they wrote the specification of HTTP/1.1, the first version of the protocol on the IETF standards track. In 2000, after the crisis had been averted, Fielding named and described the web's architectural style in his PhD dissertation: representational state transfer, or REST. Today's REST APIs take their name and principles from that description. The lecturer recommends the dissertation, easy to find by searching for Fielding's REST paper, as a must-read for backend engineers, because it gives the context in which today's technologies, patterns and standards arose.

3 WHAT "REPRESENTATIONAL STATE TRANSFER" MEANS

Definition. REST is an architectural style, not a protocol or a specification: a set of constraints under which clients and servers exchange representations of the state of resources. An HTTP API designed under those constraints and their conventions is called RESTful, or a REST API.

3.1 Representation

Resources on the web, whether data or objects, are exchanged in a specific format, their representation, which depends on the server, the client and the context. JSON is the most popular format today; XML and HTML are also used. Communication between servers typically relies on JSON, while a server answering a browser may send HTML. The same resource can thus have several representations, chosen by the client's needs. A `user` resource in a database, with fields such as `id`, `name` and `createdAt`, can be sent as JSON when another server requests it through an API, and as an HTML document when a browser must display it.

3.2 State

State is the current condition of a resource: the values of its attributes at a given moment. Each resource has a state that can be transferred between client and server, carried by its representation. On an e-commerce site such as Amazon, the state of a shopping cart consists of its items, their quantities and the total price, and that state travels between client and server with each API call.

3.3 Transfer, and the style as a whole

Transfer is the movement of resource representations between client and server, the very purpose of the client–server model; client and server may exchange different representations of the same resource. It happens over a common standard, HTTP, through its methods: GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD and others. When a browser sends a GET request for a web page, for instance, a representation is transferred from server to client. Put together, the name describes a style in which resources are represented in different formats, such as JSON, HTML or XML; the state of those resources is transferred between client and server; clients and servers communicate by ex-

changing these representations; and the whole system follows the constraints of Section 2.2 so that it scales. None of this history needs to be memorized: it explains where today's conventions come from.

4 DESIGNING URLS

4.1 Anatomy of a URL

A typical website URL has five parts (Figure 1). The *scheme* is `http` or its secure, encrypted version `https`. The *authority* is the domain, possibly with a subdomain. The *path* names the resource being accessed, and each forward slash in it expresses a hierarchical relationship between resources. The *query parameters*, used mostly in GET requests, are key–value pairs that give the server more information, such as filters. The *fragment* points into the page: when a URL with a fragment is opened, the browser scrolls to that section.

```
https://example.com/articles/rest?lang=en#history

scheme     https           http, or https (encrypted)
authority  example.com     domain, may have a subdomain
path       /articles/rest  resource; "/" = hierarchy
query      ?lang=en        key-value pairs, e.g. filters
fragment   #history        browser scrolls to that section

API URL    https://api.example.com/v1/books
           scheme, api subdomain, version, plural resource
```

Figure 1: The parts of a website URL and of an API URL (illustrative)

4.2 API URLs and plural resource names

An API URL is built from the same parts according to conventions that most companies follow when building their backends, industry practice rather than a rule. The scheme is `https`; the host is usually an `api` subdomain of the main domain, as in `api.example.com`; most APIs are versioned, usually in the route, so the path begins with `/v1` or `/v2`; the resource comes last. For an API like that of Goodreads, a community platform of readers, books, authors and reviews, the endpoint that lists all books is `https://api.example.com/v1/books`.

The first rule of route design is that every resource in a path segment is a plural noun: `/books`, not `/book`. The common mistake is to switch to the singular when fetching a single item, as in `/book/{id}`. The request concerns one book, but the path segment names the resource, the collection of books, so the route stays plural: `/v1/books/{id}`. Everything up to the resource (scheme, host and version) remains constant across the endpoints of one version.

4.3 Slugs and hierarchy

URLs travel through many environments, servers, clients and operating systems, so they must be readable and safe everywhere. A URL should contain no spaces and no underscores, whether in a slug, an ID or any other segment. A *slug* is a human-readable form of some property of a resource, made fit for a URL. To fetch a book by its name, Harry Potter, the name is first converted to lower case, since capitals are allowed but risk case mismatches between environments; every space is then replaced by a hyphen. The slug `harry-potter` gives the route `/v1/books/harry-potter` (Figure 2).

Each slash in a path adds a level of hierarchy. `/books` denotes the collection of book resources in the server's database or other storage, the first level; `/books/harry-potter` selects one resource from that collection, the second. Reading paths as hierarchical relationships guides every route in the demo, including custom actions (Section 8.4).

```
"Harry Potter"
  -> lower case          "harry potter"
  -> spaces to hyphens   "harry-potter"

/v1/books                 the collection        level 1
/v1/books/harry-potter    one book, by slug     level 2
/v1/books/{id}            one book, by ID       level 2

avoid  /v1/book/harry-potter     singular resource
       /v1/books/Harry_Potter    capitals, underscore
```

Figure 2: From a name to a slug, and the hierarchy of a path (illustrative)

5 HTTP METHODS AND IDEMPOTENCY

Five methods carry most data operations: GET, POST, PUT, PATCH and DELETE (see Chapter 1). Others exist: HEAD fetches only the headers, and OPTIONS is used in the CORS flow to find out whether an origin is allowed. Relating each method to idempotency, a theoretical concept with practical consequences, shows which method belongs in which context (Table 2).

5.1 Idempotency

> **Definition.** An operation is *idempotent* if performing it several times has the same effect as performing it once. For an API: whether a client sends a request once or a thousand times, the outcome in the server environment is the same. The responses may differ; the side effects may not.

5.2 GET, PUT and PATCH

GET retrieves data from a server and is idempotent: however many times a client fetches the list of books, it receives a list of books. If someone else creates a book between two calls, the response changes, but that is not what idempotency measures. It concerns the side effect that the client's own request causes on the server, and whether that effect differs between the first and the thousandth call; a GET is only a fetch and changes nothing.

PATCH and PUT both update a resource, with different meanings. PATCH updates part of a resource, one field or a few. PUT completely replaces the representation of the resource on the server with the client's payload. For a user with `id`, `name` and `createdAt`, a PATCH that renames the user sends only `name` with its new value, and the server updates that field; a PUT sends all the fields, and the server replaces its current instance of the user with the payload, although fields it manages itself, such as `id` and `createdAt`, normally stay under its control. The two are often used interchangeably, which is acceptable to some extent for an

API used only internally. A public API, however, should follow the standard as closely as it can, because the engineers who integrate it assume that it does: PUT where PATCH is meant creates a wrong assumption and confusion, though little real harm, since the behaviour soon shows which operation takes place. The rule is to use PATCH for a partial update and PUT to replace the whole representation.

The lecture classifies both as idempotent. If a user's name is A and a client sends `{"name": "B"}`, the first call changes A to B, the second changes B to B, and after a thousand or a million calls the name is still B (Figure 3). Every call leaves the same state, so repeating an update with the same payload causes no new side effect.

A subtle point. This holds for PUT and for a PATCH that sets fields to given values, but HTTP does not promise it for PATCH in general. RFC 9110 defines PUT, DELETE and the safe methods (GET, HEAD, OPTIONS, TRACE) as idempotent, while RFC 5789 states that PATCH is neither safe nor idempotent, though a particular request can be designed to be idempotent. A patch such as "append this tag" or "add 10 to the balance" changes the state again on every call, so a retried request applies twice. POST is thus not the only non-idempotent method, only the usual one. GET, finally, is also *safe*: it is not meant to change state at all, a stronger property than idempotency.

5.3 DELETE

A DELETE request for the user with ID 1 removes that user: the effect of the first call is that the user no longer exists. When the same request is sent again, the server checks whether the user exists, finds that it does not and answers 404 Not Found, because the action targets an entity that does not exist. The second call caused no side effect: nothing happened beyond the check, and no entity changed state. A client can send the request a million times and receive the same error a million times; only the first call changed anything. DELETE is therefore idempotent, although its first and later responses differ.

```
PATCH  /users/1  {"name": "B"}     A -> B     changed
PATCH  /users/1  {"name": "B"}     B -> B     no change
DELETE /users/1                    204        deleted
DELETE /users/1                    404        no change
POST   /books    {"name": "Dune"}  201, id 1  created
POST   /books    {"name": "Dune"}  201, id 2  created again
```

Figure 3: Repeated requests with idempotent and non-idempotent methods (illustrative)

5.4 POST and custom actions

POST is the method treated as non-idempotent. It usually creates a new resource or entity on the server. To add a book to an inventory, a client puts the book's name, description, weight, dimensions and other properties in the body of a POST, and the server inserts a new book into the database. Suppose the same request is sent again, perhaps by copying it as a cURL command and running it once more. If the server or the database requires unique names, the second call fails with an error that the name cannot be duplicated. Commonly, though, names may repeat, since several books can share a title, and books are told apart by their IDs, which the database generates as UUIDs or as serial values 1, 2, 3, 4. The second call then inserts a second book with the same data and a different ID, without any error, and a thousand calls create a thousand books. The side effect differs with every call, which is exactly what idempotency excludes.

POST is also open-ended. Some operations fall under none of the methods defined for fetching, creating, updating or deleting. An endpoint that sends an e-mail, for instance, takes a body such as `{"target": "user@example.com"}`, extracts the address and sends the message. It is not a fetch, nor a create, an update or a delete; it tells the server to perform an action and is therefore called a *custom action*. Whenever an operation cannot be placed under any of the CRUD methods, it is made a POST. HTTP itself defines POST as processing the enclosed data according to the target resource's own semantics, which is what lets it carry such actions.

Method	Use	Idempotent	Usual success
GET	Fetch one resource or a list	Yes, and safe	200
POST	Create; custom actions	No	201 or 200
PUT	Replace a whole resource	Yes	200
PATCH	Update some fields	Only if the patch sets values	200
DELETE	Remove a resource	Yes	204

Table 2: The five main methods (idempotency as defined in RFC 9110 and RFC 5789)

6 FROM REQUIREMENTS TO AN INTERFACE

6.1 Why standards matter

Before writing any code or business logic, a backend engineer designs the interface of the API: the surface that clients, whether servers or browsers, will interact with. It should be intuitive, pleasant to use and never vague, and it should follow the REST conventions as far as possible, because their purpose is to remove confusion, assumptions and human error from the workflow. Suppose an API deliberately ignored them, using POST where PUT belongs and DELETE to fetch data. An engineer integrating it would have only two ways to learn its behaviour, successful and failing, the structure of its payloads and the method of each call: read its code, which is possible only if it is open source or within the same organization, or try methods and judge from the results whether they behave as expected. Both leave much room for error. When the conventions are followed, even only about 80% of the time, most behaviour is known in advance, and integration takes far less time and effort, with fewer bugs, fewer misunderstandings and fewer sync-up calls with the integrators, especially when documentation and an interactive API play-

ground are in place. Creator and consumer then share the same expectations, and nothing is left to guesswork.

6.2 Wireframes and resources

Design starts from the user-interface design: the wireframes in Figma, or whatever tool the designers and the product team use, and the user stories. They show not how the frontend engineer will consume the API but how the end users of the platform will interact with data. Users interact with the platform, the frontend engineer builds the platform on the API, and the API in turn works with the databases (Figure 4). The wireframes thus connect the first level of consumption, the user, with the lowest, the database, leaving aside the intermediaries of a SaaS platform; once it is clear how users relate to the data, interface design can begin.

Its first concept is the resource. As a rule of thumb, admittedly simplified but mostly reliable, the resources are the nouns that appear in the requirements, whether these come from the wireframes or from conversations with clients and product people. For a project-management SaaS similar to Jira or Linear, the obvious nouns are projects; users; organizations, to which users belong; tasks, which belong to projects; and tags, which organize tasks. Noted down, they are the top-level entities of the backend.

```
users -> frontend -> API (backend) -> database

requirements: wireframes, user stories, conversations
  -> resources        the nouns: organizations, projects
  -> database schema  tables and fields (Chapter 8)
  -> actions          list, get, create, update, delete,
                      custom actions
  -> interface        methods, routes, payloads, codes
  -> code             language, framework, business logic
```

Figure 4: The chain from users to database, and the design workflow (illustrative)

6.3 Schema, actions and tool

Next comes the database schema, which this lecture leaves to the next one (see Chapter 8). For the demo the schema is assumed done, with three tables: `organizations`, the organizations that exist on the platform; `projects`, the projects within an organization; and `tasks`, the tasks created inside a project. The following step is to find the actions a typical client must perform on each resource. The CRUD operations, create, read, update and delete, provide a template: list all, get one, create, update and delete; for organizations these give five actions. Only then does the interface design begin.

The demo designs the interface in Insomnia, an API client that is an alternative to Postman, lighter and with fewer features. What is designed is how clients will call the API and what they will receive, not how it executes: the programming language, the business logic and the database interactions are out of scope. The demo server runs at `http://localhost:3000` without versioning; in production the scheme becomes `https`, the domain changes and the path usually starts with a version such as `/v1`.

7 LISTING AND CREATING ORGANIZATIONS

7.1 List and create

The organization endpoints come first, starting with the list. Fetching data makes it a GET, and its path is the resource name in lower case and in the plural, as Section 4.2 requires: `GET /organizations`, saved in Insomnia as "list organizations". On an empty database it returned an empty list. Creating an organization is a POST to the same URL. The two routes are identical because the server tells the calls apart by their method: a POST on `/organizations` reaches the controller that creates an organization, a GET the one that lists organizations (see Chapter 2).

A POST carries a payload. Of the six fields of the `organizations` table, `id`, `createdAt` and `updatedAt` are handled on the server side and are excluded; the client sends the other three, `name`, `status` and `description`, as a JSON body in which every key and string value is wrapped in double quotes (Figure 5). The server answered 201 Created, the code for a newly created resource, and returned the new entity as it is, including the `id` and `createdAt` generated on the server; HTTP also lets a 201 response give the new resource's URL in a `Location` header. Listing again returned the organization with 200 OK, the code for a success that creates nothing, inside a wrapper with four fields: `data`, `total`, `page` and `totalPages`.

```
organizations table
  id, createdAt, updatedAt     set by server and database
  name, status, description    sent by the client

POST /organizations
{"name": "org 1", "status": "active",
 "description": "some description"}

201 Created
{"id": "6f2a...", "name": "org 1", "status": "active",
 "description": "some description",
 "createdAt": "...", "updatedAt": "..."}
```

Figure 5: Creating an organization (illustrative values)

7.2 Why paginate

The wrapper reflects *pagination*, a server-side technique for endpoints that return a list of resources: instead of every matching row in the database, the response carries one portion of the data. There are two reasons. First, JSON serialization and deserialization are heavy operations (see Chapter 3); serializing a thousand organizations into a payload that can travel across the internet consumes resources, adds delay and hurts the API's performance. Second, a slow response becomes a delay the end user perceives in the frontend, perhaps three or four seconds, although at first glance a user sees only ten or twenty organizations and has to scroll to see more. So the first call returns the first portion, sorted by a field such as the creation time, for instance the latest ten organizations. When the user clicks on page 2, or scrolls down in an infinite scroll, the client asks for the next portion, which it does not yet have, and likewise for page 3, fetching perhaps 20 or 30 items at once. The user is not overwhelmed, nor is

the network, and client and server both perform better. This is why list endpoints are usually paginated.

7.3 The paginated response, page and limit

A paginated response has four fields. `data` holds the portion being returned. `total` is the count of all the organizations in the database, independent of the portion, so that the interface can show metadata such as "10 of 50". `page` identifies the portion this response belongs to; the server numbers the portions, which is why they are called pages. `totalPages` is the number of pages; it lets the frontend build pagination controls and, in an infinite scroll, stop requesting once `page` equals `totalPages`, because the end has been reached.

A GET has no body, so the client controls pagination through query parameters: `limit`, the number of items wanted in each response, and `page`, the portion wanted. When they are missing, the server applies defaults, page 1 and a limit of 10 or 20, rather than failing. With five organizations created in turn, `?limit=2` returned organizations 5 and 4, the latest first, with `total` 5, `page` 1 and `totalPages` 3, since five items in pages of two take three pages, of two, two and one items. `?page=2&limit=2` returned organizations 3 and 2, and `page=3` organization 1 alone, with `page` and `totalPages` both 3 (Figure 6). Without parameters all five came back, because the default limit is larger. A list API should accept both parameters so that clients can reach every portion of the data conditionally and programmatically.

```
newest first:  org 5  org 4 | org 3  org 2 | org 1
               page 1       | page 2       | page 3

GET /organizations?limit=2
{"data": [org 5, org 4],
 "total": 5, "page": 1, "totalPages": 3}

GET /organizations?page=2&limit=2  -> [org 3, org 2]
GET /organizations?page=3&limit=2  -> [org 1]
    "total": 5, "page": 3, "totalPages": 3
```

Figure 6: Five organizations in pages of two (illustrative response bodies)

7.4 Sorting

A list endpoint should also support sorting, through two query parameters. `sortBy` names the field to sort by, which may be any suitable field of the resource, such as `name`, `status` or `id`, depending on the implementation; `sortOrder` gives the direction. In the demo, `?sortBy=name` returned organizations 5, 4, 3, 2, 1, no visible change, because the default order is descending; adding `sortOrder=asc` returned them as 1, 2, 3, 4, 5.

The server must sort even when the client asks for nothing, because a database does not store or return rows in any guaranteed sequence: without an explicit sort, each call could return the same items in a different order. The natural default is `createdAt` in descending order, the latest entries first, which is what a client would assume of a list. The default order also applies on its own: when only `sortBy` is sent, `sortOrder` falls back to descending (Table 3). For stable pages the sort should end with a unique field such as `id`, so that items with equal names or timestamps cannot swap places between pages. These defaults illustrate a broader rule, taken up in Section 10.3: the server should take sane defaults wherever it can and not depend on the client for obvious values. The requests above succeeded without `page`, `limit` or any sort parameter, with no validation error for a missing field, and the API was the more pleasant to use for it.

Parameter	Meaning	Default
`page`	Which portion of the list	1
`limit`	Items per page	10 or 20
`sortBy`	Field to sort by	`createdAt`
`sortOrder`	Ascending or descending	Descending
A field, e.g. `status`	Keep only matching items	No filter

Table 3: Query parameters of a list endpoint

7.5 Filtering

The last feature of a typical list endpoint is filtering by field values, for example when the user flips a switch to see only active or only archived organizations. A sixth organization was created with `"status": "archived"` (201) and appeared at the top of the list. The client filters by passing the field name as a query parameter: `?status=archived` returned only the archived organization, and `?status=active` only the active ones. Other fields work alike: a `name` filter with the name of organization 4 returned that one organization. Pagination, sorting and filtering together make up the features a list endpoint should offer.

8 SINGLE-RESOURCE ENDPOINTS AND CUSTOM ACTIONS

8.1 Update with PATCH

The update endpoint uses PATCH. The lecturer has rarely implemented updates with PUT: endpoints usually take some fields of an entity and update those, and almost never replace the whole entity with a client payload. Submitting whole entities was the practice of multi-page applications; in single-page applications, where data is mostly JSON, clients send partial fields, which PATCH conveys better than PUT, so PATCH is the best practice for update endpoints even though many developers use the two interchangeably. The endpoint must know which organization to update, so its route ends in a dynamic path parameter holding the ID (see Chapter 2): `PATCH /organizations/{id}`, the plural resource followed by the ID. To reactivate organization 6, its ID was copied from the list and the body `{"status": "active"}` sent. The server answered 200 OK, since nothing was created, and returned the updated organization, the usual success response for an update; the list then showed organization 6 as active as well.

8.2 Get and delete

Fetching a single organization is a GET on the same route, `GET /organizations/{id}`, with no body; it returned organization 6 with 200, since it only fetched information. The routes that get, update and delete a single resource look alike, the server address, the plural resource, a slash and the ID in place of the dynamic parameter, and only the method distinguishes them, just as list and create share `/organizations` (Table 4). `DELETE /organizations/{id}`, again without a body, returned an empty response with 204 No Content. The 2xx class means that the operation succeeded; 204 adds that there is nothing to send back, because the organization no longer exists. An empty body with 204 is the usual answer to a delete. The list now began with organization 5.

Action	Request	Success
List	`GET /organizations`	200, paginated list
Create	`POST /organizations`	201, new entity
Get one	`GET /organizations/{id}`	200, entity
Update	`PATCH /organizations/{id}`	200, updated entity
Delete	`DELETE /organizations/{id}`	204, empty body
Archive	`POST /organizations/{id}/archive`	200, archived entity

Table 4: The organization endpoints designed in the demo

8.3 404 or an empty list

Requesting the deleted organization with `GET /organizations/{id}` returned the error message "organization not found" with status 404. This is the proper use of 404: the client asked for a particular resource, identified by its ID, and the server does not have it. A list endpoint does not answer 404 even when it has nothing to return, because it asks not for a particular resource but for a list. With a filter that matches nothing, `status` set to a random value, the response was 200 with `data` as an empty array, `total` 0, `page` 1 and `totalPages` 0 (Figure 7). The rule of thumb: an empty list is an empty array with 200, never "resource not found"; 404 is reserved for requests that name a single resource, or a specific set of resources, that does not exist. With list, create, get, update and delete, all the CRUD operations are now designed, read comprising both get and list.

```
GET /organizations/{id of deleted organization 6}
404  {"message": "organization not found"}

GET /organizations?status=xyz
200  {"data": [], "total": 0, "page": 1, "totalPages": 0}
```

Figure 7: A missing resource versus an empty list (illustrative bodies)

8.4 Custom action: archive

Suppose organization 5 must be archived. At first sight this is an update: its status changes from active to archived, so a PATCH of the status field seems enough. It is not, because archiving an organization involves more than one field. The projects of the organization may have to be removed, the tasks of those projects deleted, and the users involved notified or sent e-mails; many operations run on the server. Changing the status field does not archive an organization, so archiving is a custom action and, as Section 5.4 established, a POST. Its route is `POST /organizations/{id}/archive`: the server address, the plural resource, the ID of the organization and finally the name of the action. The request names make the difference visible in Insomnia: create, list, get, update and delete are CRUD operations; archive is not. The route follows the hierarchy of Section 4.3, from all organizations to one particular organization to an action on it. The request, without a body, returned organization 5 with status archived and the code 200. Not every POST returns 201: a custom action that creates no resource answers 200.

9 THE SAME PATTERNS FOR PROJECTS

The project endpoints repeat every pattern, more quickly, so that the patterns become familiar. In Insomnia the organization requests were moved into a folder `org` and the new ones placed in a folder `project`.

9.1 Create and list projects

Creating a project is `POST /projects`, the plural resource in lower case. The body leaves out the server-handled `id`, `createdAt` and `updatedAt` and carries `name`, `organizationId`, the ID of an existing organization (a placeholder in the demo), `status`, for example `planned`, and `description` (Figure 8). The server answered 201 with the new project. Listing is `GET /projects`, the route of the create call with another method: the routes that create and list a resource are alike, as are those that get, update and delete one. After projects 2 and 3 were created, `?limit=1` returned one project and `?limit=1&page=2` the next portion, again a single project; sorting and filtering work as for organizations.

The lecturer stresses that the field names of any JSON data, in payloads the client sends and in responses the server returns, follow camelCase, as in `organizationId` and `createdAt`. He calls this the common standard for JSON: an API that uses JSON as its serialization format should stick to it, because clients then need no guesswork.

> **A subtle point.** camelCase is a convention, not part of JSON: RFC 8259 places no rules on member names. camelCase is widespread because JSON comes from JavaScript, but large public APIs such as those of GitHub and Stripe use snake_case. What spares integrators guesswork is the point of Section 9.2: one convention, applied everywhere.

```
POST /projects
{"name": "project 1", "organizationId": "<org id>",
 "status": "planned", "description": "some description"}
201  the new project, with id, createdAt, updatedAt

POST /projects/{id of a deleted project}/clone
404  project not found
POST /projects/{id of project 2}/clone
201  a copy of project 2 under a new id
```

Figure 8: Creating a project and cloning one (illustrative values)

9.2 Consistency across resources

Within one platform, query parameters and JSON payloads should look alike whatever the resource. Organizations and projects both have a name and a description, and the create-organization endpoint established the key `description`; the create-project endpoint must use the same key, not a variant such as `desc`, even though that also suggests a description. Once a frontend engineer has integrated one endpoint, they make many assumptions about how the others work: having sent `name` and `description` to `/organizations`, they will send the same keys to the project endpoint without opening the documentation, and if it expects `desc` they get a validation error. Routes work the same way: after `/organizations` they expect `/projects`, and a singular `/project` again produces an error and sends them to the specification. Such errors happen during development, but an API should not waste the time and effort of its integrators or make them guess. From the first endpoint on, routes, payloads and responses should follow one style, without arbitrary decisions per resource; the lecturer counts this consistency of style among the most important characteristics of a good backend engineer.

9.3 Single-project endpoints and cloning

The single-project routes add the project's ID as a dynamic parameter and differ only in method. `GET /projects/{id}`, with an ID copied from the list, returned the project with 200. `PATCH /projects/{id}` with the partial body `{"description": "new description"}` returned 200 and the updated project, the typical behaviour of PATCH. `DELETE /projects/{id}`, without a body, returned 204 No Content and an empty response.

The custom action for projects is cloning: the frontend can clone a project, which creates a new project with a different ID and all the values of the original. A client could simulate this by reading a project and passing its values to the create endpoint, but it cannot know what cloning means on the server: perhaps the tasks of the project must be cloned too, or the owner told by e-mail that the project was cloned, among other operations. Cloning therefore gets its own endpoint, `POST /projects/{id}/clone`, built like every custom action: the server address, the plural resource, the particular resource and the name of the action. It needs no payload here, though it could accept fields that the client wants to override in the copy. The first attempt used the ID of the project deleted a moment earlier and returned 404 "project not found"; with the ID of project 2 it returned 201 Created and the new resource, with the same fields as project 2 (Figure 8). The status code cannot be inferred from the method, nor from the fact that the endpoint is a custom action; it depends on what happens on the server. Cloning creates a project, so 201 is right, whereas archiving created nothing and returned 200.

> **Warning.** The status code describes what happened, not which method was used. A POST returns 201 only when it creates a resource, and a custom action that creates nothing returns 200; a delete returns 204 with an empty body; 404 means that a specific resource does not exist, while a list with no matching items is 200 with an empty array.

The endpoints of tasks would follow the same patterns: list, get, create, update and delete, plus their own custom actions. However many resources an API has, the foundations stay the same.

10 GUIDELINES FOR A GOOD INTERFACE

Beyond the patterns of the demo, the lecturer gives a few rules for a good experience for those who integrate an API and those who maintain it.

10.1 Interactive documentation

Provide interactive documentation, for example with Swagger tools built on an OpenAPI description, which offer a playground for trying out the endpoints (see Chapter 25). Plan it from the start: the team can use it to test the API, and integrators use it both as documentation and as a way to test. The lecturer counts it among the most important things that set a backend engineer apart: how consistently and how often they create, maintain and update the OpenAPI or Swagger documentation.

10.2 Intuitive and consistent

Routes, dynamic parameters, custom actions and JSON payloads should all follow one pattern. Following the global REST conventions is best; an API that, for whatever reason, does not should still follow some pattern and keep to it, instead of changing its style across endpoints and resources, which is a major pain point for whoever integrates it. Behaviour, data format and naming should all be consistent and intuitive.

10.3 Sane defaults

Sane defaults apply to reads and writes alike. In a list endpoint the server assumes page 1 when no page is sent, a limit of 10 or 20, sorting by `createdAt` and descending order. In a POST endpoint it requires only the information absolutely needed to create the entity and supplies defaults for the rest. An organization has a `status` that is either active or archived; the business logic and common sense say that a new organization is active, so when the client omits the field the server sets it to `active`, a safe assumption. Since `description` is optional as well, a client can create an organization by sending only a name, and it is created with status active. Each such field gets the value that makes most sense for the use case.

10.4 No abbreviations

Avoid abbreviations such as `desc` for description. The people who integrate an API do not have the information its designer had while creating the interface, so the designer cannot assume they will understand a short form and

supply the right value for it. Field names in payloads and query parameters alike should be readable and intuitive.

10.5 Design first, code second

A REST API is first designed, not coded or programmed right away, and its design involves many decisions. Starting with an OpenAPI playground such as Swagger, or an API client such as Insomnia or Postman, gives insight into how clients and consumers will use the API and leads to better, more intuitive endpoints. Before programming, in Go, Node.js or any other language, a separate session should be dedicated to designing the interface alone, without thinking about languages or frameworks; that is why the lecture contained no language- or framework-specific implementation at all. Making APIs delightful and intuitive, easy to maintain and easy to consume, is the backend engineer's responsibility.

11 CONCLUSION

- REST is the architectural style Fielding described in 2000 to explain how the web scaled: client–server, uniform interface, layered system, cache, statelessness and optional code on demand, under which clients and servers transfer representations of the state of resources.
- An API URL has `https`, an `api` subdomain, a version such as `/v1` and plural, lower-case resource names; slugs use lower case and hyphens, and each slash adds a level of hierarchy.
- GET, PUT and DELETE are idempotent and POST is not; PATCH is idempotent only when it sets values. PATCH updates fields, PUT replaces the whole representation, and anything outside CRUD is a POST custom action.
- Design starts from wireframes and requirements: the nouns become resources, then come the schema, the CRUD and custom actions and the interface, all before any code.
- A collection route such as `/organizations` serves list and create, a member route such as `/organizations/{id}` serves get, update and delete, the method tells them apart, and actions live at `/resources/{id}/action`.
- List endpoints paginate (`data`, `total`, `page`, `totalPages`), sort (`sortBy`, `sortOrder`) and filter by field, with defaults of page 1, a limit of 10 or 20 and `createdAt` descending.
- Status codes follow outcomes: 201 for creation, 200 for other successes including custom actions that create nothing, 204 for a delete, 404 only for a missing specific resource, and 200 with an empty array for an empty list.
- Consistent routes, keys and responses, unabbreviated names, sane defaults and interactive OpenAPI documentation make an API predictable for those who integrate it.

APPENDIX A REVIEW QUESTIONS

1. A book-community platform needs endpoints to list the reviews of one book with the newest first, to add a review to a book, to fetch a book by its title "The Name of the Wind", and to send a reading reminder by e-mail to everyone who shelved a book. Design the method, route and success status code of each endpoint, and justify every choice, including which of them are custom actions and why.
2. A mobile client sends three requests: `POST /books`, `PATCH /users/7` with `{"name": "B"}`, and `PATCH /accounts/7` with `{"op": "increment", "field": "credits", "by": 10}`. Each times out on the client after the server has committed it, and the client retries once automatically. Predict the server state after each retry, decide which of the three are safe to retry, and explain how you would change the API so that the others can be retried without harm.
3. An infinite scroll calls `GET /organizations?page=N&limit=20` with the default sort. Users report that entries sometimes appear twice while colleagues are creating organizations; after the team changed the default to `sortBy=name`, some entries are skipped even when nobody writes. Explain both symptoms from the way page-based pagination and sorting work, and propose changes to the sort and to the pagination scheme.
4. A frontend shows a red "Not found" banner whenever a user filters tasks by a tag that has no tasks, and a request for a deleted project by its ID returns 200 with `{"data": null}`. Explain which design rules each response breaks, what the frontend currently has to do to cope with them, and specify the correct status codes and bodies.
5. A team implemented archiving by letting clients send `PATCH /organizations/{id}` with `{"status": "archived"}`. Months later, archived organizations still have open projects, and their members were never told. Explain the design flaw, design the replacement endpoint and its status code, and decide what the PATCH endpoint should now do when a body contains a `status` field.
6. A public API offers `POST /organizations` accepting `name` and `description`, `POST /project` accepting `project_name` and `desc`, and two list endpoints, one returning a bare JSON array and the other `data`, `total`, `page` and `totalPages`. Estimate what each inconsistency costs a partner who integrates the organization endpoints first, propose a consistent design, and explain how you would introduce it without breaking existing clients.
7. A developer keeps each client's current position in a list in server memory and exposes `GET /organizations/next`, which returns the caller's next 20 organizations. It works on one server, but after a second instance is added behind a round-robin load balancer, users see pages skipped and repeated. Explain the failure in terms of Fielding's constraints, and redesign the endpoint so that any instance can serve any request.
8. An API documents `PUT /organizations/{id}` as full replacement, while the project endpoints use PATCH. A frontend engineer used to the project endpoints sends

`PUT /organizations/42` with only `{"description": "new"}`. Predict the stored organization under strict replacement semantics, explain what the server should do with `id` and `createdAt` in a PUT body, and argue whether this API should offer PUT, PATCH or both.

APPENDIX B GLOSSARY

Term	Meaning
REST	Representational state transfer: Fielding's architectural style of the web
Resource	A noun the API exposes, such as an organization or a project
Representation	A resource's state in a format such as JSON, XML or HTML
Uniform interface	One standard way for components to communicate; four sub-constraints
HATEOAS	Hypermedia as the engine of application state: responses link to next steps
Statelessness	Each request carries all the server needs; no client context is stored
Layered system	Components see only the adjacent layer, so proxies and load balancers fit in
Idempotent	Repeating a request has the same effect on the server as sending it once
Safe method	A method not meant to change server state, such as GET
Custom action	An operation outside CRUD, modelled as `POST /resources/{id}/action`
Slug	A lower-case, hyphenated, readable form of a property used in a URL
Path parameter	A dynamic segment such as `{id}` that selects one resource
Pagination	Returning a list in portions selected by `page` and `limit`
Sane default	A value the server assumes when the client omits an obvious field
201 Created	Success that created a resource; the new entity is returned
204 No Content	Success with an empty body, as after a delete

APPENDIX C FURTHER TOPICS

Cursor (keyset) pagination

The page-and-limit scheme of the lecture is offset pagination: the database skips (page − 1) × limit rows and returns the next ones. Skipping grows more expensive as pages get deeper, and rows inserted or deleted between two requests shift the boundaries, so items appear twice or are missed. Cursor pagination instead returns an opaque cursor that encodes the sort key of the last item, such as its `createdAt` and `id`, and the next request asks for the items after it, which an index serves directly. It suits infinite scroll and large or fast-changing tables, at the cost of not being able to jump to an arbitrary page.

Idempotency keys

Because POST is not idempotent, a client that retries after a timeout may create the same order or payment twice. An idempotency key is a unique value, usually a UUID, that the client generates and sends in a header such as `Idempotency-Key`; the server stores the key with the result of the first request and returns that stored result for any repeat. Payment APIs such as Stripe's rely on this, and an IETF draft standardizes the header. It applies to any non-idempotent operation that clients, proxies or job queues may retry automatically.

Conditional requests and optimistic concurrency

When two clients read the same resource and both send an update, the later write silently overwrites the earlier one: the lost-update problem. HTTP solves it with validators. The server returns an `ETag`, a version tag, with the resource; the client sends it back in `If-Match` with its PUT or PATCH; and the server answers 412 Precondition Failed if the resource has changed in between. The same mechanism with `If-None-Match` lets clients revalidate cached GET responses and receive 304 Not Modified. It matters for any resource that several users or devices edit (RFC 9110).

JSON Merge Patch and JSON Patch

PATCH says that a resource is changed partially, not how the change is described. JSON Merge Patch (RFC 7396, media type `application/merge-patch+json`) sends a partial document whose fields replace those of the resource and in which `null` removes a field, the form the demo used. JSON Patch (RFC 6902, `application/json-patch+json`) sends a list of operations such as `add`, `remove`, `replace` and `test`, which can edit arrays precisely and make an update conditional. Choosing and documenting one format settles questions such as how a client clears an optional field.

HATEOAS and the Richardson Maturity Model

The lecture names hypermedia as the engine of application state as a sub-constraint of the uniform interface without developing it. In a hypermedia API each response includes links to related resources and to the actions currently allowed, for example an `archive` link only on an active organization, so that clients follow links instead of building URLs from documentation. Leonard Richardson's maturity model grades HTTP APIs from level 0 (one endpoint, RPC over HTTP) through resources (level 1) and HTTP methods and status codes (level 2) to hypermedia (level 3). Most practical REST APIs, including the one designed here, stop at level 2, which Fielding has argued does not meet his definition of REST.

Content negotiation

Section 3.1 states that one resource can have several representations; content negotiation is how HTTP chooses between them. The client lists the formats it accepts in the `Accept` header, for instance `application/json` or `text/html`, and the server responds in one of them, naming it in `Content-Type`, or answers 406 Not Acceptable. The same mechanism covers language preferences and versioned vendor media types. A backend that serves browsers and API clients from the same resources relies on it.

API versioning and evolution

The lecture shows a version in the path, `/v1`, but not when to change it. Adding an optional field or a new endpoint is backward compatible, while removing or renaming a field, changing a type or changing a status code breaks existing clients and calls for a new version. Versions can also be carried in a header or a media type instead of the path. Old versions are retired with notice, which the `Sunset` header (RFC 8594) and the newer `Deprecation` header announce to clients. Public APIs in particular live for years alongside the clients that integrated them.

Long-running actions and 202 Accepted

A custom action such as archiving an organization, with its cascade of deletions and e-mails, may take longer than a client should wait. The server can then answer 202 Accepted, queue the work and return the URL of a status resource, such as `/operations/{id}`, which the client polls or about which it is notified through a webhook (see Chapters 10 and 26). The status code then reports acceptance rather than completion, which is what actually happened. This applies to any action whose duration grows with the size of the data.

Problem Details for HTTP APIs

The demo's errors carry a plain message such as "organization not found". RFC 9457 standardizes a JSON error body, media type `application/problem+json`, with the members `type`, `title`, `status`, `detail` and `instance`, and allows extensions such as a list of invalid fields. One error format across all endpoints is the consistency principle of Section 9.2 applied to failures, and ASP.NET Core can produce it automatically. It applies to every API whose errors are read by programs rather than people (see Chapter 12).

GraphQL and gRPC

The lecture names RPC and GraphQL as alternatives and sets them aside. GraphQL exposes one endpoint with a typed schema and lets the client state exactly which fields and related objects it needs, which avoids over-fetching and repeated round trips for complex screens at the cost of harder HTTP caching and of controlling expensive queries. gRPC is an RPC framework over HTTP/2 with Protocol Buffers contracts and generated clients, efficient and strongly typed, and common between internal services. REST remains the usual choice for public, resource-oriented APIs, and many systems combine the styles.

Databases with PostgreSQL — One-Page Overview

Chapter 8 · every section and sub-section of the paper at a glance

1 Introduction: Why Backends Need Databases

Databases persist data across sessions; backends use disk-based ones because disk is cheap and plentiful.

- **1.1 Persistence** — Data must survive the program that wrote it, like a to-do list reopened later.
- **1.2 What counts as a database** — Any structured storage offering CRUD; in backends, a disk-based system.
- **1.3 Disk-based storage** — RAM is fast but scarce: caches live in RAM, databases on disk.

2 Database Management Systems

A DBMS adds efficient access, integrity, security and concurrency control to stored data.

- **2.1 Responsibilities of a DBMS** — Organization, CRUD access, integrity, security; also scaling.
- **2.2 Why plain text files are not enough** — Slow, error-prone parsing; no structure; lost updates.

3 Relational, Non-relational and the Case for PostgreSQL

Strict schemas buy integrity, flexible documents buy speed of change; PostgreSQL with JSON covers both.

- **3.1 Relational databases** — Tables, foreign keys, a predefined schema, SQL.
- **3.2 Non-relational databases** — Collections of documents; integrity moves into application code.
- **3.3 Choosing by use case** — A CRM suits a relational store, CMS content a document store.
- **3.4 Why PostgreSQL** — Open source, standard SQL, extensible, reliable, strong JSON support.

4 PostgreSQL Data Types

The types a backend uses and how to choose between near-equivalents.

- **4.1 Integers and serial types** — `BIGSERIAL` for auto-incrementing keys; integer ranges.
- **4.2 Exact versus floating-point numbers** — `DECIMAL` for prices, floats where speed beats exactness.
- **4.3 Character types** — Avoid `CHAR`; prefer `TEXT` to `VARCHAR(255)`.
- **4.4 Dates, UUIDs, JSON and the rest** — `TIMESTAMPTZ`, `UUID`, `JSONB` over `JSON`, arrays.

5 Migrations

Schema changes are versioned SQL files applied in order by a tool.

- **5.1 Why migrations** — History in version control and rollback instead of ad-hoc GUI changes.
- **5.2 Up and down migrations** — Apply and revert; a version table records progress.
- **5.3 dbmate in practice** — `DATABASE_URL`, `dbmate new`, `dbmate up`.

6 Schema Design for a Project Management Platform

Five tables show keys, constraints, referential integrity and the three relationship patterns.

- **6.1 Enum types** — Allowed values enforced by the database and documented in the schema.
- **6.2 The users table: keys and constraints** — UUID primary key, `NOT NULL` by default, unique email.
- **6.3 Naming conventions** — Plural table names, lowercase snake_case.
- **6.4 One-to-one: user profiles** — `user_id` as both primary and foreign key.
- **6.5 One-to-many and referential integrity** — `RESTRICT`, `CASCADE`, `SET NULL`; a `CHECK` on priority.
- **6.6 Many-to-many: the linking table** — `project_members` with a composite primary key.
- **6.7 Down migration and applying it** — Drop in reverse; `schema_migrations` appears.

7 Seeding Development Data

Test data for development, inserted by its own migration file.

- **7.1 Seeding** — Data to test against before real user flows exist.
- **7.2 A seed migration with CTEs** — Chained inserts with `WITH … RETURNING`.

8 Queries Behind the User Endpoints

Each endpoint becomes one SQL statement, with request values bound to placeholders.

- **8.1 Listing users with their profiles** — `LEFT JOIN`, `to_jsonb`, a default sort.
- **8.2 Parameterized queries: fetching one user** — Placeholders against SQL injection.
- **8.3 Filtering, sorting and pagination** — `ILIKE`, an allowlisted sort, `LIMIT/OFFSET`.
- **8.4 Creating and updating** — `INSERT … RETURNING`; a partial `PATCH`.

9 Indexes

Lookup structures that replace sequential scans, at a cost on every write.

- **9.1 The cost of a sequential scan** — The book-index analogy; visiting every row.
- **9.2 How an index helps** — Value-to-location lookup, ascending or descending.
- **9.3 When to create an index** — WHERE, JOIN, ORDER BY plus frequency; foreign keys are not auto-indexed.
- **9.4 The index migration** — Eight indexes and the queries they serve.

10 Triggers

A trigger keeps updated_at current without application code.

- **10.1 Keeping updated_at current** — A manual `SET` versus a database rule.
- **10.2 The trigger migration and its effect** — Trigger function, per-table triggers, verified by an update.

Databases at a glance — persist data on disk through a DBMS, design a strict PostgreSQL schema with migrations, then serve each endpoint with a parameterized query tuned by indexes and helped by triggers. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Databases with PostgreSQL: Schema, Migrations, Queries and Indexes

Chapter 8 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — The chapter explains why backends persist data in disk-based databases and what a database management system adds over plain files: efficient access, integrity, security and concurrency control. It compares relational and document databases, argues for PostgreSQL as the default, and surveys its data types with concrete guidance for numbers, strings, time and JSON. It then builds the schema of a project management platform through versioned migrations, covering enums, constraints, referential integrity and the one-to-one, one-to-many and many-to-many patterns, and seeds it with test data. The queries behind the user endpoints follow: joins with embedded JSON, parameterized queries, filtering, sorting, pagination, inserts and partial updates. Finally, indexes replace sequential scans for frequent filters, joins and sorts, and a trigger keeps updated_at current.

Keywords — PostgreSQL, relational database, data types, migrations, constraints, referential integrity, parameterized queries, indexes, triggers

1 INTRODUCTION: WHY BACKENDS NEED DATABASES

Reading and writing data is among the most frequent operations a backend performs, which makes the concepts around databases central to the job. The lecture works with PostgreSQL throughout and assumes the basics of SQL; its focus is what a backend engineer actually does with a database: design a schema, evolve it with migrations, and write the queries behind API endpoints.

1.1 Persistence

At its core a database is a way to *persist* information across sessions: to store data so that it survives after the program that created it has stopped. A to-do app shows the expectation. The user adds items and checks off finished ones; when they close the app and open it again, hours later or on another device, every item must be in exactly the state they left it. Without persistence each session would start from an empty list and all progress would be lost.

1.2 What counts as a database

The term is surprisingly broad: in the simplest sense any structured storage is a database. The contact list on a phone is one. So is a browser's local storage: the developer tools (Application tab) on a site such as Excalidraw show the key–value entries the site has stored, next to session storage and cookies. Even a text file of notes read back later is a rudimentary database. The common pattern is a persistent system that offers ways to create, read, update and delete data (CRUD). In backend work, however, "database" means something narrower: a *disk-based* database serving the application.

1.3 Disk-based storage

A CPU reads from primary memory (RAM) and from secondary storage, a hard disk or a solid-state drive. RAM is far faster but expensive and scarce: a typical machine has 8, 16 or 32 GB, a high-end one 64 or at most 128 GB, while disks of 512 GB to 2 TB are ordinary because disk capacity is cheap. Disk is slower because of the way data is stored on it and fetched from it. This trade-off decides where each kind of system lives. Caches, whether Redis or an in-memory cache inside the process, keep data in RAM because their purpose is fast retrieval (see Chapter 9). Traditional databases, relational ones such as PostgreSQL and non-relational ones such as MongoDB, store their data on disk, because for a database capacity at a low price matters most and some speed can be traded for it. How an engine lays data out on disk, and the algorithms around it, is a deep field the lecture leaves aside; at the application level the RAM-versus-disk distinction is what a backend engineer needs.

2 DATABASE MANAGEMENT SYSTEMS

2.1 Responsibilities of a DBMS

Putting bytes on a disk is not enough. Clients must retrieve, change and delete those bytes, and with hundreds or thousands of gigabytes every such operation must be efficient.

> **Definition.** A database management system (DBMS) is software whose job is to store data, typically on disk, and to give its clients efficient create, read, update and delete operations on it, while keeping the data organized, valid and protected.

Four responsibilities stand out. **Data organization**: data is laid out so that creating, fetching and updating stay efficient as it grows. **Access**: the DBMS provides the CRUD operations to its clients. **Integrity**: the data stays accurate, valid and consistent. In an e-commerce platform each order carries a payment amount stored as a number; if someone tries to insert the string `'something'` into that column, the DBMS must reject the write, because guarding correctness is its responsibility rather than each client's. **Security**: the DBMS protects data from unauthorized access through its own users and roles. A production DBMS also takes on concerns such as scaling and load balancing.

2.2 Why plain text files are not enough

Before DBMSs existed, data was kept in text files, and anyone who does so today meets the same three problems as the data grows.

Parsing. To find one customer, application code, in Python, Java, JavaScript, Go, Rust or any other language, must read the file, split it into lines and compare fields one by one. This is slow (Rust parses faster than JavaScript, Python or Ruby, but every language pays) and error-prone: a parsing bug can corrupt the data or serve customers wrong information.

No structure. A text file accepts any text in any format. Nothing can enforce that a field holds only numbers, so the file cannot promise consistent data.

Concurrency. When two people modify the same value at the same time, a file has no way to combine the changes correctly. Figure 1 traces the lecturer's example: an `amount` of 40, one user adding 20 and another subtracting 20. Each must read before writing, both read 40, and each writes its own result. Which write lands last depends on CPU scheduling and countless other conditions, so the final value is 60 or 20, never the correct 40. A DBMS contains concurrency mechanisms that turn such interactions into a consistent result; a plain file has none.

```
amount = 40                      (one line in a text file)

user A: add 20                   user B: subtract 20
reads 40                         reads 40
writes 40 + 20 = 60              writes 40 - 20 = 20

final value: 60 or 20, whichever write lands last
value expected after both changes: 40
```

Figure 1: The lost update on a shared text file, the lecturer's example laid out as a timeline (illustrative).

3 RELATIONAL, NON-RELATIONAL AND THE CASE FOR POSTGRESQL

3.1 Relational databases

A relational database organizes data in *tables* of rows and columns and expresses relationships between tables with *foreign keys*. Data enters only through a *predefined schema*: every table, its columns and their data types are declared before any row is inserted, and nothing is added on the fly. The strictness buys **data integrity**: at any moment one can rely on the type of every column, the relationships between tables, and rows that are consistent and accurate. Relational databases are queried with SQL (Structured Query Language); MySQL, PostgreSQL, SQL Server and SQLite are well-known examples.

3.2 Non-relational databases

MongoDB is the best-known non-relational (NoSQL) database. Its vocabulary differs (Table 1): a table is a *collection* and a row a *document*. The substantive difference is that every row of a relational table has the same structure, whereas documents in one collection may each have a different structure. That flexibility is the main advantage, and sometimes the main disadvantage. When prototyping, a team can push and fetch any data without designing, enforcing or maintaining a schema. But lacking strong constraints and relationships, a document store makes it easy to introduce inconsistent data: integrity is enforced not by the database but by application code, which adds complexity and, because application code changes often, invites bugs.

Aspect	Relational	Document (NoSQL)
Container	table	collection
Record	row	document
Schema	predefined, same for every row	flexible, may differ per document
Integrity	enforced by the database	mostly in application code
Examples	PostgreSQL, MySQL, SQL Server	MongoDB
Lecture's use case	CRM	CMS content

Table 1: Relational and document databases side by side.

3.3 Choosing by use case

A CRM (customer relationship management) system must keep accurate, consistent data about customers, their contacts and sales opportunities, and run complex queries over the relationships between them; a relational database such as PostgreSQL, with strong integrity, fits. A CMS (content management system) such as Sanity fits a document store. A blog that pulls its articles from a CMS lets an author log in, write an article in Markdown and save it, and the next visitor sees it, without anyone editing the code base, committing and redeploying. Such content is not really structured: an article may hold images, code blocks, YouTube embeds. When it is not known in advance what will be stored, a store that accepts whatever arrives makes sense.

3.4 Why PostgreSQL

MySQL, PostgreSQL, SQLite, SQL Server and others offer broadly the same features and all scale in production, so a default is needed. The lecturer gives five reasons for PostgreSQL. (1) It is **open source and free**, not proprietary, so companies can run it on their own premises and servers. (2) It **adheres to the SQL standard**: a query behaves as it would on MySQL or SQL Server, and if migrations and queries are written in standard SQL, moving to another database later takes only minor changes. (3) It is **extensible**: its documentation runs to about 1,400 pages, its features cover nearly every case a typical SaaS meets, and an extension system adapts it further. (4) It is known for **reliability and scalability**. (5) The deciding one is **JSON support**. The main reason to pick a document database is to store arbitrary data, which in practice means arbitrary JSON: numbers, strings, nested objects, arrays. PostgreSQL has JSON column types with good indexing and querying, so dynamic data such as CMS content can live in a JSON column of an ordinary table, and there is no need to switch databases for it.

For these reasons PostgreSQL is the first choice of many startups and large companies. Articles claiming performance advantages for MySQL matter only when serving millions of users and optimizing one specific bottleneck; until then, PostgreSQL's features decide. SQL is the query language and PostgreSQL the system that executes it. The lecture skips the basics of both (`CREATE TABLE`, `SELECT`, `ORDER BY`, `GROUP BY`), a prerequisite covered by many free courses, and runs its demos in TablePlus, a graphical database client.

4 POSTGRESQL DATA TYPES

Although it skips SQL basics, the lecture surveys the data types once, because every later table depends on them. The demo creates a table `data_types_demo` with one column of each type and inserts one row with `INSERT INTO table (columns) VALUES (values)`, listing the values in the order of the columns. Table 2 summarizes the guidance.

4.1 Integers and serial types

`SERIAL` is an integer column filled from an automatically incrementing counter: when an `INSERT` omits the column, rows receive 1, 2, 3 and so on. `BIGSERIAL` is the same with a far larger maximum and is the usual production choice when an auto-incrementing `id` is the *primary key*, the unique field that identifies a row. `SMALLINT`, `INTEGER` and `BIGINT` all store integers and differ only in the largest value they hold; in the demo row the big integer is visibly longer than the integer.

4.2 Exact versus floating-point numbers

`DECIMAL` and `NUMERIC` are effectively identical. In `DECIMAL(10,2)` the 2 fixes two digits after the decimal point and the 10 caps the total number of digits: `12345678.90` has ten digits and fits, but one more digit before the point does not. `REAL`, `DOUBLE PRECISION` and `FLOAT` are floating-point types, represented and processed quite differently. The rule of thumb: when accuracy matters, use `DECIMAL`. A `price` enters many calculations, and a value represented slightly differently in different places causes real errors. When small discrepancies do not matter, as for an area of 567.8987 that might come back as 567.899, use a floating-point type. Why not always use decimals? Exactness costs speed: floating-point values are faster to store and compute with, which is why scientific computing prefers them.

4.3 Character types

All three character types store text. `CHAR(10)` is fixed-length: storing `'ab'` pads it with eight spaces so that every value has length 10. `VARCHAR(255)` (variable character) sets only a maximum: `'ab'` is stored with length 2, `'abcd'` with length 4, and a longer value than 255 raises a database error. `TEXT` is `VARCHAR` without a declared length; its only ceiling is PostgreSQL's limit of about 1 GB per value.

The guidance is firm. Never choose `CHAR` unless every value has the same length: for weekday codes such as `'Mo'`, `'Tu'`, `'We'`, a `CHAR(2)` wastes nothing. For general text, such as a customer name, a product name or a description, the choice is between `VARCHAR` and `TEXT`, and the lecturer always takes `TEXT`. PostgreSQL's documentation states that there is no performance difference between them; the beliefs that `VARCHAR(255)` is faster or that a `TEXT` column cannot be indexed come from MySQL, SQL Server and SQLite, whose conventions differ, carried over by people who move between databases. The number 255 is itself a MySQL convention (there a `VARCHAR` of up to 255 bytes needs only a one-byte length prefix); in PostgreSQL it means nothing. Declaring a length also has costs. If a `description` declared `VARCHAR(255)` later needs more room, raising the limit takes a migration over existing data; migrations are routine, but a risky one is best avoided. And a number in a schema suggests a meaning: a new team member who reads `VARCHAR(255)` or a length of 10 will dig through the code for the reason, when the author picked it without thought. Backend engineers, unlike data analysts and data engineers who work in SQL directly, reach the database through a driver with application code in between, so length limits can be enforced there (see Chapter 5), keeping migrations simple to read and maintain. `TEXT` says only that the column holds text.

4.4 Dates, UUIDs, JSON and the rest

`BOOLEAN` stores true or false; `DATE` a date; `TIME` a time of day in hours, minutes and seconds; `TIMESTAMP` a date and time; `TIMESTAMPTZ` a time-zone-aware timestamp; `INTERVAL` a duration such as 10 days or one week. `UUID` is a native type for universally unique identifiers, popular for primary keys because of their unique nature. Of the two JSON types, `JSON` stores the document as plain text, exactly as written, while `JSONB` (JSON binary) serializes it into PostgreSQL's own binary format for efficient processing and querying; in the demo's insert both look identical, yet they are stored differently. `JSONB` is PostgreSQL-specific rather than standard SQL, and because of its performance advantage it should be the default. Any type can form an array: `INTEGER[]`, `DATE[]`, `JSONB[]`. Finally there are types for network addresses (`INET`), MAC addresses (`MACADDR`), geometric points (`POINT`) and `XML`, which a typical backend rarely needs.

Type	Stores	Guidance
SERIAL, BIGSERIAL	auto-incrementing integer	BIGSERIAL for integer keys
SMALLINT, INTEGER, BIGINT	integers, growing range	by the largest value expected
DECIMAL(p,s), NUMERIC	exact numbers	money; accuracy matters
REAL, DOUBLE PRECISION	approximate numbers	measurements; speed first
CHAR(n)	fixed length, space-padded	fixed-length codes only
VARCHAR(n)	text up to n characters	rarely needed in PostgreSQL
TEXT	text of any length	default for strings
BOOLEAN, DATE, TIME	truth value, date, time of day	as named
TIMESTAMPTZ, INTERVAL	instant, duration	created_at, updated_at
UUID	128-bit identifier	primary keys
JSON, JSONB	JSON as text / as binary	JSONB for dynamic data
type[]	array of a type	small lists
INET, MACADDR, POINT, XML	specialized values	rarely used

Table 2: PostgreSQL data types covered in the lecture.

A subtle point. Three statements in the type tour need correcting. The integer types grow in the order `SMALLINT` (2 bytes, up to 32,767) < `INTEGER` (4 bytes, about 2.1 billion) < `BIGINT` (8 bytes, about 9.2 quintillion); the lecture put `INTEGER` first. Floating-point values do not vary "across systems", since IEEE 754 is a standard; the problem is that binary fractions cannot represent most decimal fractions exactly, so `0.1 + 0.2` in double precision is not exactly 0.3, which is why money needs `DECIMAL`. And `TIMESTAMPTZ` does not store a time zone: PostgreSQL converts the input to UTC, stores that instant, and converts it to the session's zone on output.

5 MIGRATIONS

5.1 Why migrations

In production nobody opens a GUI client such as TablePlus and types schema changes. Even if nothing went wrong, there would be no record of which changes were applied to a database over time or by whom, and no way to control or reproduce the schema's state at a given moment. Database *migrations* solve this. A migration is a file of ordinary SQL statements, such as `CREATE TABLE`, `CREATE TYPE`, `CREATE INDEX` or `DROP TABLE`, kept in a folder such as `db/migrations`. File names carry a sequence: `1.sql`, `2.sql`, `3.sql`, or a timestamp prefix, which grows monotonically and orders the files just as well. A command-line tool, dbmate and golang-migrate being well-known ones, walks the folder in that order and executes each file against the database configured in the environment (see Chapter 13).

Migrations give two things. The first is a **history of database changes**: the folder lives beside the code and is committed to version control on GitHub, GitLab or Bitbucket, and each change adds a new file, so the evolution of the schema can be traced. The second is **rollback** to an earlier version when something goes wrong. Because a relational schema is strict, every change, whether a new table, a modified table, an index or a trigger, is made through a new migration. A team that instead saved its ad-hoc statements to files and committed them would end up reinventing a migration system.

5.2 Up and down migrations

An *up* migration applies a change: create the tables, the enum types, the indexes. A *down* migration reverts exactly what the up migration did, so that if a release breaks production the database can return to its previous state. Some modern approaches do without down migrations, but the pair is the standard. Tools differ in layout; dbmate keeps both halves in one file, separated by the comments `-- migrate:up` and `-- migrate:down`.

To know where it stands, the tool keeps a bookkeeping table in the database (dbmate's is `schema_migrations`, with a single `version` column) recording which migrations have been applied. When a new file is added, only the files after the recorded version run. This matters because running the same statements twice fails: PostgreSQL rejects a second `CREATE TABLE` for a table that already exists.

5.3 dbmate in practice

The demo works in VS Code. dbmate reads the connection string from the variable `DATABASE_URL`, placed in a `.env` file. The command `dbmate new create_users_table` creates the folders `db/migrations` and an empty file named by a timestamp followed by the given name. Statements after the up comment and before the down comment form the up migration; those after the down comment form the down migration. `dbmate up` applies every pending file and reports each with its duration in milliseconds (Figure 2).

```
# .env
DATABASE_URL=postgres://user:secret@localhost:5432/pm

$ dbmate new create_users_table
Creating migration:
  db/migrations/20250101120000_create_users_table.sql

-- db/migrations/20250101120000_create_users_table.sql
-- migrate:up
CREATE TYPE ...;
CREATE TABLE ...;

-- migrate:down
DROP TABLE ...;
DROP TYPE ...;

$ dbmate up
Applying: 20250101120000_create_users_table.sql
```

Figure 2: The dbmate workflow: connection string, a new migration file with its two sections, and applying it (names and timestamp illustrative).

6 SCHEMA DESIGN FOR A PROJECT MANAGEMENT PLATFORM

The worked example is the database of the project management platform whose API was modelled in Chapter 7: users with profiles, projects owned by users, tasks within projects, and project membership. The first migration creates the whole schema (Figures 3 and 4).

6.1 Enum types

The migration begins with three *enum* types, each a predefined set of allowed values declared with `CREATE TYPE name AS ENUM (...)`: `project_status` (active, completed, archived), `task_status` (pending, in progress, completed, cancelled) and `member_role` (owner, admin, member). A column then uses the enum as its type: `projects.status` is a `project_status`. The lecturer gives two reasons to prefer this to plain `TEXT`. The first is **data integrity**: the database, not application code, checks validity, so an insert or update that sets a status to an arbitrary string such as `'something'` fails with a database-level error. Application validation narrows the chance of bad data; the database check makes the system robust. The second reason, which the lecturer weighs more heavily because validation could live in the backend anyway, is **documentation**. Someone debugging, or a new team member reading the migrations to learn how the schema evolved over months or years, who finds `status TEXT` cannot know that only three values are allowed without tracing every place in the application code that touches the field. An enum states the allowed values at a glance. Enum columns usually get a default taken from the enum, such as `DEFAULT 'active'`, so the application need not send the value.

6.2 The users table: keys and constraints

Every table carries `id`, `created_at` and `updated_at`. The `id` is not metadata but the *primary key*, the value that identifies one row, for instance for an endpoint that receives a user's UUID as a dynamic route parameter and returns that user. It is declared `UUID PRIMARY KEY DEFAULT gen_random_uuid()`: inserts normally omit it and PostgreSQL generates a random UUID natively.

Relational databases attach *constraints* to columns: conditions the database checks on every write so that a column cannot become corrupt, rejecting any statement that violates them. `PRIMARY KEY` implicitly carries two of them, `NOT NULL` and `UNIQUE`, so a second row with the same id is rejected; these properties, together with how foreign keys refer to it, are why every table conventionally gets an `id` primary key.

Every PostgreSQL column accepts NULL unless declared otherwise. The lecturer's rule of thumb is that more than 70% of a table's columns should be `NOT NULL`, which keeps the database in a consistent state: many writes come from automated scripts, and a buggy script that unintentionally pushes NULLs leaves data that is very hard to work with. Unless a column has a reason to be empty, it is `NOT NULL`. In `users`, `email` is `TEXT NOT NULL UNIQUE`: two users must never share an email, and inserting a duplicate violates the unique constraint. `full_name` is `TEXT NOT NULL`, and so is `password_hash`, because a password is never stored in plain text, only its hash (see Chapter 4). `created_at` records when the row was first created and is never set by hand: a default fills in the current timestamp as a `TIMESTAMPTZ`. `updated_at` records the last change to the row, useful, for instance, when a front-end table lists users by when their name was last updated.

```
-- migrate:up
CREATE TYPE project_status AS ENUM
    ('active', 'completed', 'archived');
CREATE TYPE task_status AS ENUM
    ('pending', 'in_progress', 'completed', 'cancelled');
CREATE TYPE member_role AS ENUM
    ('owner', 'admin', 'member');

CREATE TABLE users (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email         TEXT NOT NULL UNIQUE,
    full_name     TEXT NOT NULL,
    password_hash TEXT NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- one-to-one with users
CREATE TABLE user_profiles (
    user_id    UUID PRIMARY KEY
               REFERENCES users ON DELETE CASCADE,
    avatar_url TEXT,
    bio        TEXT,
    phone      TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Figure 3: First migration, part 1: enum types, users and the one-to-one profile table (reconstructed from the lecture; illustrative).

6.3 Naming conventions

Table names are plural: `users`, `user_profiles`, `projects`. Singular names are also used and the choice belongs to the team or company, but plural is the prevailing industry standard. Every table and column name is lowercase, and multi-word names use snake_case: `full_name`, `password_hash`, `created_at`. Engineers from languages where camelCase is idiomatic may want `fullName`, but PostgreSQL folds unquoted identifiers to lowercase, so `fullName` becomes `fullname`. Keeping the capital requires writing `"fullName"` in double quotes everywhere, including inside application code, which quickly makes the code awkward and ugly.

6.4 One-to-one: user profiles

Profile data, here `avatar_url`, `bio` and `phone`, and in a real product social media accounts, websites, portfolio projects and more, lives in a separate table, `user_profiles`, in a *one-to-one* relationship with `users`. The reason is how the data changes. A profile keeps growing; kept in `users`, it would mean repeated migrations of the core table and constant modification of user rows. Isolated, the profile table can

grow without touching `users`. The same pattern serves user preferences and other metadata.

Because each user has at most one profile row, the profile table needs no id of its own. The draft had an `id` plus `user_id UUID NOT NULL UNIQUE`; the lecturer removes the `id` and declares `user_id UUID PRIMARY KEY` referencing `users`, so the column is both the primary key and the foreign key, and `NOT NULL` and `UNIQUE` become implicit. The profile columns stay nullable: a user may choose to have no bio, no phone number or no custom image.

6.5 One-to-many and referential integrity

In `projects` (Figure 4), `name` is `NOT NULL` while `description` may be empty, and `status` is a `project_status`, `NOT NULL`, defaulting to `'active'`. `owner_id` is a foreign key: `UUID NOT NULL REFERENCES users ON DELETE RESTRICT`. Its full form is `REFERENCES users(id)`; because `id` is the primary key of `users`, the column name can be left out.

The `ON DELETE` clause expresses *referential integrity*: using the relationship between tables to protect the data across them. Suppose user 1 owns a project whose `owner_id` is 1, and someone deletes user 1. The database finds the dependent row in `projects` and applies the declared action. With `RESTRICT` the delete fails: the user cannot be removed until their projects are. Table 3 lists the actions. They interact with column constraints: `SET NULL` on `owner_id` would collide with its `NOT NULL`, and the delete would fail with a database error.

Action	When the parent row is deleted	Used here for
RESTRICT	the delete fails while child rows exist	projects.owner_id
CASCADE	child rows are deleted too	tasks.project_id, project_members
SET NULL	the child's foreign key becomes NULL	tasks.assigned_to
SET DEFAULT	the foreign key gets the column default	not used

Table 3: Referential integrity actions on delete.

In `tasks`, `project_id` is a `UUID` because it stores values of `projects.id`, and it is `NOT NULL`: no orphan task without a project can exist. It references `projects` with `ON DELETE CASCADE`, so deleting project 1 also deletes every task whose `project_id` is 1. `title` is `NOT NULL` and `description` optional. `priority` is `INTEGER NOT NULL DEFAULT 1` with a `CHECK` constraint, a custom condition that must hold for a write to go through: here the value must lie between 1 and 5, so nobody can store a priority of 55. `status` is a `task_status` defaulting to `'pending'`; `due_date` is a `DATE`, with no time or time zone; `assigned_to` references `users` with `ON DELETE SET NULL`, so deleting a user leaves their tasks unassigned instead of deleting them. A foreign key also guards inserts: an `assigned_to` value that is a well-formed UUID but matches no row in `users` makes the insert fail.

This is the *one-to-many* pattern: project 1 can have any number of task rows whose `project_id` is 1. It is implemented by placing the parent's primary key in the child table as a plain foreign key, not as the child's primary key, which is what distinguishes it from one-to-one.

```
CREATE TABLE projects (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name        TEXT NOT NULL,
    description TEXT,
    status      project_status NOT NULL DEFAULT 'active',
    owner_id    UUID NOT NULL
                REFERENCES users ON DELETE RESTRICT
    -- + created_at, updated_at as in users
);

CREATE TABLE tasks (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id  UUID NOT NULL
                REFERENCES projects ON DELETE CASCADE,
    title       TEXT NOT NULL,
    description TEXT,
    priority    INTEGER NOT NULL DEFAULT 1
                CHECK (priority BETWEEN 1 AND 5),
    status      task_status NOT NULL DEFAULT 'pending',
    due_date    DATE,
    assigned_to UUID REFERENCES users ON DELETE SET NULL
    -- + created_at, updated_at as in users
);

-- many-to-many: the linking table
CREATE TABLE project_members (
    project_id UUID REFERENCES projects ON DELETE CASCADE,
    user_id    UUID REFERENCES users ON DELETE CASCADE,
    role       member_role NOT NULL DEFAULT 'member',
    -- + created_at, updated_at as in users
    PRIMARY KEY (project_id, user_id)
);

-- migrate:down
DROP TABLE project_members, tasks, projects,
           user_profiles, users;
DROP TYPE member_role, task_status, project_status;
```

Figure 4: First migration, part 2: one-to-many, many-to-many and the down section (reconstructed; illustrative).

6.6 Many-to-many: the linking table

A user can belong to many projects at once, and a project can have many users. When a relationship makes sense read in both directions like this, it is *many-to-many*, and it is stored in a separate *linking table*, here `project_members`, whose job is to record the relationship. The lecturer edits the draft: the `id` column goes; the table keeps two foreign keys, `project_id` referencing `projects` and `user_id` referencing `users`, both `ON DELETE CASCADE`, so deleting a project or a user deletes its membership rows; and the closing `UNIQUE (project_id, user_id)` becomes `PRIMARY KEY (project_id, user_id)`, a *composite primary key*. Given users 1, 2, 3 and projects 1, 2, 3, adding user 1 to project 2 inserts the row (2, 1). User 1 being part of project 2 is a single fact that cannot occur twice, and the primary key's implicit uniqueness and non-nullness enforce exactly that, while the pair identifies the row. The table also holds `role`, a `member_role` defaulting to `'member'`: a role belongs to a user within a project, so it lives on the membership rather than on the user or the project.

Relationship	Example	Implementation
One-to-one	users, user_profiles	parent's key is the child's primary and foreign key
One-to-many	projects, tasks	parent's key as a plain foreign key in the child
Many-to-many	users, projects	linking table with a composite primary key

Table 4: The three relationship patterns in the schema.

6.7 Down migration and applying it

The down section undoes everything: it drops the tables, dependants before the tables they reference, and then the types in reverse order of creation. `dbmate up` applies the file in a few milliseconds; refreshing TablePlus shows the five tables and their columns, all empty, plus the `schema_migrations` table of Section 5.2. Without it, the tool would try to run the same file again and PostgreSQL would answer that the tables already exist.

7 SEEDING DEVELOPMENT DATA

7.1 Seeding

In production, data enters through the platform's user flows: people sign up through one form and create projects through another. During development there are no such users, yet the engineer needs data to test queries and see how interactions will look before deploying a change. Putting test data into a development database is called *seeding*. Seed statements could sit in an existing migration, but the practice the lecturer follows is a separate file: `dbmate new seed_data` creates one whose timestamp is later than the schema migration's, so it runs after it.

A subtle point. `dbmate up` applies every pending file wherever it runs, so a seed file kept in `db/migrations` also inserts its test users, with known password hashes, into production. Development seeds are safer outside the migration chain, as a separate script that only development environments run.

7.2 A seed migration with CTEs

The seed file is plain SQL made readable with *common table expressions* (CTEs), named intermediate results introduced by `WITH` (Figure 5). The first CTE, `inserted_users`, inserts sample users, with emails, names and password hashes, and returns the generated `id` and the `email` of each. The second, `inserted_profiles`, selects from `inserted_users` to create one profile per user: a static avatar URL, a bio chosen by a `CASE WHEN` on the email, and a phone number. Projects and the remaining tables are filled the same way. After `dbmate up`, TablePlus shows rows in every table: users with ids, emails, names and hashes, one profile per user with its own bio, projects and their members.

```
-- migrate:up
WITH inserted_users AS (
    INSERT INTO users (email, full_name, password_hash)
    VALUES ('john@example.com', 'John Doe', 'hash_1'),
           ('jane@example.com', 'Jane Smith', 'hash_2'),
           ('alice@example.com', 'Alice Brown', 'hash_3')
    RETURNING id, email
),
inserted_profiles AS (
    INSERT INTO user_profiles
           (user_id, avatar_url, bio, phone)
    SELECT id,
           'https://example.com/avatar.png',
           CASE WHEN email = 'john@example.com'
                THEN 'Backend engineer'
                ELSE 'Product designer' END,
           '+1-555-0100'
    FROM inserted_users
    RETURNING user_id
)
INSERT INTO projects (name, description, owner_id)
SELECT 'Website redesign', 'New marketing site', id
FROM inserted_users
WHERE email = 'john@example.com';
```

Figure 5: A seed migration chaining inserts through CTEs (names as in the demo, values illustrative).

8 QUERIES BEHIND THE USER ENDPOINTS

With the schema seeded, the lecture writes the SQL behind the user endpoints designed in Chapter 7. In a real backend these statements are assembled and executed through a driver or ORM; the SQL editor shows what finally reaches the database.

8.1 Listing users with their profiles

For `GET /v1/users`, `SELECT * FROM users` returns every user. The front end, however, should receive each user's profile embedded in the user object, so that one API call and one database call suffice. The lecturer writes a query starting from the `FROM` clause rather than `SELECT`, to settle first where the data is drawn from. `FROM users u` gives the table an *alias*, a one- or two-letter name that makes repeated references convenient. The profile sits in another table, and what connects the two is the foreign key `user_profiles.user_id`, so the tables are joined on `u.id = up.user_id`.

The join is a `LEFT JOIN`. A user who has never edited their profile has no row in `user_profiles`; an `INNER JOIN` returns only rows matched on both sides and would drop such users, whereas a `LEFT JOIN` keeps every user of the left table whether or not a profile exists. Inner and left joins are the two used most of the time. In the select list, `u.*` takes every user column, and `to_jsonb(up.*) AS profile`, a built-in PostgreSQL function, converts the matching profile row into a JSON object in a column named `profile`. The result has `id`, `email`, `full_name`, `password_hash`, `created_at`, `updated_at` and `profile`.

A relational database returns rows in no particular order unless told otherwise, so every query that returns a list should sort by default, usually with `ORDER BY u.created_at DESC` to show the newest first (Figure 6). On the way to the client the rows are deserialized and serialized again: in

Node.js the overhead is small, while in Go the rows are first decoded into a struct and then encoded as JSON (see Chapter 3).

> **A subtle point.** `u.*` is convenient in the editor, but it includes `password_hash`, which must never leave the backend. A real list endpoint names its columns (`u.id, u.email, u.full_name, …`) or drops the field in its response type.

```
-- GET /v1/users
SELECT u.*,
       to_jsonb(up.*) AS profile
FROM users u
LEFT JOIN user_profiles up ON u.id = up.user_id
ORDER BY u.created_at DESC;

-- GET /v1/users/:userId
SELECT u.*, to_jsonb(up.*) AS profile
FROM users u
LEFT JOIN user_profiles up ON u.id = up.user_id
WHERE u.id = :user_id;
```

Figure 6: Queries behind the two read endpoints; `:user_id` is the editor's named placeholder (illustrative reconstruction).

8.2 Parameterized queries: fetching one user

`GET /v1/users/:userId` receives the user's id as a dynamic route parameter (see Chapter 7), and the id must reach the query. A *parameterized query* is the safety mechanism for this: the query is sent with an empty slot, a placeholder, and the value for the slot is supplied separately when the query executes. Whatever arrives in the slot is treated purely as a value: if someone passes `DELETE FROM users`, it is compared against ids as a string and never executed. Building the query by concatenating strings instead exposes the backend to *SQL injection*. Database drivers and ORMs in Node.js, Go, Rust and Python all support placeholders.

The query is the list query plus `WHERE u.id = :user_id`. TablePlus recognizes the named placeholder and opens a dialog for its value, where the UUID is typed as an SQL string literal in single quotes. The result is the single user, still with all user columns and the embedded profile. In the backend the request flows through the layers of Chapter 6: the router hands it to a handler, the handler to a service, the service to a repository, which binds the UUID to the placeholder; the returned row is serialized to JSON for the front end.

> **Warning.** Never splice request data into SQL text. With `"... WHERE id = '" + id + "'"`, an id of `' OR '1'='1` turns the filter into one that matches every row, and a value containing a semicolon may append a statement of its own. A placeholder makes this impossible because the value never becomes part of the SQL text.

8.3 Filtering, sorting and pagination

List endpoints, such as all users, projects or tasks, usually support dynamic filtering, dynamic sorting or both, and should be paginated unless a particular interface requires otherwise. The backend reads optional query parameters and fills in defaults (Table 5). The filter, here the first letter of the user's name, has no default: if `letter` is absent, the backend leaves the condition out of the `WHERE` clause entirely. For sorting, the client cannot name an arbitrary column; it chooses among options the backend allows (`full_name`, `email`, `created_at`), with `created_at` and descending order as defaults. The dynamic construction happens in application code, which the SQL editor cannot show, but the SQL it produces can be examined (Figure 7).

Parameter	Default	Effect
page	1	offset = (page − 1) × limit
limit	10 or 20	LIMIT
letter	none: no condition	name starts with the letter
sort_by	created_at	one of full_name, email, created_at
sort_order	desc	ASC or DESC

Table 5: Query parameters of the list endpoint and their defaults.

The filter is `WHERE u.full_name ILIKE :letter || '%'`. `ILIKE` is the pattern-matching operator `LIKE` made case-insensitive; `||` concatenates the parameter with `%`, the wildcard for any sequence of characters, so the pattern `J%` means "starts with J, anything after". In the demo, `J` returns John Doe and Jane Smith, `X` returns nothing, and `A` returns Alice Brown. Sorting becomes `ORDER BY :sort_by :sort_order`; because the query uses aliases, the backend must translate the client's choice into the aliased form `u.email`. Descending order returns John before Jane, ascending Jane before John. Pagination appends `OFFSET` and `LIMIT`. Users number pages from 1, but the offset counts rows to skip from 0, so the backend converts: with `LIMIT 1`, page 1 is offset 0 and returns Jane Smith, page 2 is offset 1 and returns John Doe. In general the offset is (page − 1) × limit. Real code differs considerably because everything is assembled dynamically, but at the database level this is what happens.

```
-- in the SQL editor
SELECT u.*, to_jsonb(up.*) AS profile
FROM users u
LEFT JOIN user_profiles up ON u.id = up.user_id
WHERE u.full_name ILIKE :letter || '%'
ORDER BY :sort_by :sort_order
OFFSET :page LIMIT :limit;

-- as a backend sends it: sort column and direction
-- taken from the allowlist, values bound as $1..$3
SELECT u.id, u.email, u.full_name, to_jsonb(up.*) AS profile
FROM users u
LEFT JOIN user_profiles up ON u.id = up.user_id
WHERE u.full_name ILIKE $1 || '%'
ORDER BY u.email ASC
LIMIT $2 OFFSET $3;           -- $3 = (page - 1) * limit
```

Figure 7: The list query with filter, sort and pagination: as run in the editor, and as a backend would send it (second form illustrative).

A subtle point. TablePlus fills placeholders by substituting text, which is why `u.email` and `DESC` could be supplied as parameters in the demo. A PostgreSQL driver sends values separately from the SQL text, and a placeholder may stand only where a value is allowed: `ORDER BY $2` binds a constant string and sorts nothing, and `ORDER BY $2 $3` is a syntax error. The column and the direction are therefore picked in code from the allowlist and written into the SQL text; the allowlist, not escaping, is what keeps this safe.

8.4 Creating and updating

`POST /v1/users` is a plain insert, `INSERT INTO users (email, full_name, password_hash) VALUES (...)` with three placeholders; the backend computes the hash before the query. `RETURNING *` makes the statement return the rows it created, so the API can answer with the new user without a second query. Run with `test@gmail.com`, the name "test test" and an arbitrary string as the hash, it returns one row, and the user appears in the table.

The update endpoint is a `PATCH` with a *partial* payload: a profile form lets the user change their bio, phone number or avatar URL, and any subset may be sent. The backend checks which of the allowed fields are present and puts only those into the `SET` clause; absent fields stay untouched. For a payload with `bio` and `phone`, the constructed statement updates those two columns `WHERE user_id = :user_id`, with the id taken from the URL, and `RETURNING *` gives back the row's new state (Figure 8). Run with "updated bio" and a new number, both columns change. Yet `updated_at` still equals `created_at`, because nothing has updated it.

```
-- POST /v1/users
INSERT INTO users (email, full_name, password_hash)
VALUES (:email, :full_name, :password_hash)
RETURNING *;

-- PATCH /v1/users/:userId   (payload had bio and phone)
UPDATE user_profiles
SET bio = :bio, phone = :phone
WHERE user_id = :user_id
RETURNING *;
```

Figure 8: The create and partial-update statements (illustrative reconstruction).

9 INDEXES

9.1 The cost of a sequential scan

The list query contains two conditions worth noticing: the join's equality condition `u.id = up.user_id` and the sort on `u.email`. Both are places where indexes help. The name comes from a book's index, which lists where each chapter starts: chapter 1 on page 1, chapter 2 on page 29, chapter 3 on page 35. In a book of about 100 pages, a reader looking for chapter 4 does not skim 50 pages one by one; they read the index, see page 54 and open the book there.

A client such as TablePlus shows a table's rows in sequence, but the database does not store them that way: each row lives wherever on disk the engine's storage algorithms placed it. Asked for the task with a given id, a database without an index has one option, a *sequential scan*: visit the rows one by one at their locations on disk, compare each id with the requested one, and return the row that matches. With six tasks this is fast; with a thousand, a million or a billion rows, visiting every location makes the query slow.

9.2 How an index helps

An *index* is a separate lookup structure for a column of a table: for each value it records where on disk the row holding that value is stored, task id to row location, entry by entry. The index is kept compactly in one place rather than scattered over the disk, so searching it is fast; once the database finds the value, it jumps straight to the row's location and returns the row. The lecturer leaves the internals, the B-tree, for further reading. A second property is **order**: an index is ascending or descending. Because `GET /v1/users` sorts by `created_at` descending by default, an index on `created_at` in descending order lets the database read the timestamps already in the required order instead of fetching them all and sorting.

9.3 When to create an index

Three signals suggest an index on a column: it appears in a `WHERE` clause, in a join condition, or in a sort. Signals are not a mandate. An index must be accurate at all times, so every insert and update of the table also updates each of its indexes, an overhead that is small per write but grows with the size of the database. Whether an index pays off depends on how often its query runs and what response time the user experience requires. The pragmatic approach: when a field meets one of the three signals and its query is called frequently, create the index and monitor performance; if the query later turns out to be rare, drop the index. Two facts guide the choice. PostgreSQL indexes every primary key automatically, but it does not index foreign-key columns, which are exactly the columns an API's joins use. Fetching all tasks of a project joins `projects.id`, a primary key and already indexed, with `tasks.project_id`, a foreign key and not indexed, so the latter needs an index.

9.4 The index migration

A third migration, created with dbmate, adds the indexes of Table 6 in its up section, each with the generic syntax `CREATE INDEX idx_users_created_at ON users (created_at DESC);`, and drops them in its down section. Each serves an endpoint or query. The `status` index serves a get-all-tasks endpoint whose filter lets the front end pass any status, such as pending, in progress or completed, and expects the matching tasks quickly; the `email` index serves finding a user by email, or a join that matches on it.

Index	Serves
users (email)	look-up or join by email
users (created_at DESC)	default sort of GET /v1/users
tasks (project_id)	all tasks of a project (join)
tasks (assigned_to)	all tasks of a user (join)
tasks (created_at DESC)	default sort of the task list
tasks (status)	status filter of the task list
project_members (project_id)	members of a project
project_members (user_id)	projects of a user

Table 6: Indexes created by the third migration and the queries they serve.

A subtle point. Three refinements. A B-tree index is not searched entry after entry, as the lecture's walkthrough suggests: it is a balanced tree, so finding one key among millions takes a handful of page reads. A B-tree can also be read backwards, so a plain ascending index on `created_at` serves `ORDER BY created_at DESC` just as well; `DESC` in an index matters for multi-column indexes with mixed orders. And `UNIQUE` and `PRIMARY KEY` constraints already create indexes: the unique constraint on `users.email` makes the extra email index a duplicate, and the composite key `(project_id, user_id)` already serves lookups by `project_id`, so of the two `project_members` indexes only the one on `user_id` adds anything.

10 TRIGGERS

10.1 Keeping updated_at current

Every update of a row should also set its `updated_at` to the time of the update. There are two ways. In the manual way, application code adds `updated_at = now()`, or a bound timestamp, to every `UPDATE` it builds: one more parameter in every statement, and easy to forget. The other is a database *trigger*: a rule that performs an action when a given event occurs. Here the rule is that whenever a row of a table is updated, its `updated_at` is set to the current timestamp.

10.2 The trigger migration and its effect

The trigger lives in the same migration as the indexes (Figure 9). First comes a function: PostgreSQL lets one define custom functions, and this one is declared to return a trigger. Its body, between `BEGIN` and `END` (a PL/pgSQL block, not a transaction), takes the new version of the row, `NEW`, sets its `updated_at` to the current timestamp and returns the row. Then comes one trigger per table, for `users`, `user_profiles`, `projects` and `tasks`, each named so that the down migration can drop it, and each saying: before every update, for each affected row, execute the function. The down section drops the triggers, the function and the indexes. After `dbmate up`, the earlier profile update is run again with the bio "updated bio 3", and the row's `updated_at` now shows the date and time of the recording: the trigger works.

```
-- migrate:up
CREATE INDEX idx_users_email ON users (email);
-- ... the other indexes of Table 6

CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_users_updated_at
    BEFORE UPDATE ON users
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
-- likewise for user_profiles, projects and tasks

-- migrate:down
DROP TRIGGER trg_users_updated_at ON users;  -- etc.
DROP FUNCTION set_updated_at();
DROP INDEX idx_users_email;                  -- etc.
```

Figure 9: The third migration: indexes, the trigger function and a per-table trigger (names illustrative).

11 CONCLUSION

- Backends persist data in disk-based databases for cheap capacity, while caches keep hot data in RAM.
- A DBMS adds efficient CRUD, organization, integrity, security and concurrency control that files lack.
- PostgreSQL is a sound default: open source, standard SQL, extensible, reliable, with JSONB for dynamic data.
- Choose types deliberately: DECIMAL for money, TEXT for strings, TIMESTAMPTZ for instants, UUID keys.
- Every schema change is a versioned migration with up and down sections, tracked by a version table.
- Let the database guard integrity: NOT NULL by default, UNIQUE, CHECK, enums, deliberate ON DELETE actions.
- One-to-one shares the parent's key, one-to-many adds a foreign key, many-to-many needs a linking table.
- Most daily work turns an API payload into a dynamically built, parameterized query and returns its rows.
- Index frequent WHERE, JOIN and ORDER BY columns, weighing write cost; triggers automate updated_at.

APPENDIX A REVIEW QUESTIONS

1. A team stores product prices as `DOUBLE PRECISION`, supplier names as `VARCHAR(255)`, and order times as `TIMESTAMP` written by servers in two regions with different time-zone settings. Months later invoice totals are off by a cent, a 300-character supplier name is rejected, and orders from the two regions sort out of sequence. Explain each symptom from the column types, and plan migrations that fix them without losing data.
2. With the schema of this chapter, a script runs `DELETE FROM users WHERE email = 'john@example.com'`. John owns one project, is a member of two others, has a profile and is assigned to three tasks in projects he does not own. Trace what PostgreSQL does for each foreign key and state the outcome of the statement. How would

the outcome change if `projects.owner_id` were declared `ON DELETE CASCADE`, and what data would then be lost?

3. A `GET /v1/tasks` endpoint accepts `sort_by` and `sort_order`. Developer A binds them as `ORDER BY $2 $3`; developer B concatenates the raw query-string values into the SQL text. Predict what each version does for `sort_by=due_date&sort_order=asc` and for `sort_by=due_date;DROP TABLE tasks`, then design an implementation that is correct, safe and still flexible.
4. The `tasks` table has 20 million rows. `GET /v1/projects/:projectId/tasks`, which filters by project and orders by `created_at DESC` with `LIMIT 20`, takes four seconds, while inserts into `tasks` have slowed since a colleague added twelve indexes "to be safe". Explain both symptoms from how indexes work, and decide which indexes to keep, add or drop, justifying each by the queries that would use it.
5. The product team wants per-user notification settings (about 30 fields, expected to grow), labels that can be attached to many tasks and shared across projects, and a list of watchers per task. Propose the tables, keys, nullability, `ON DELETE` actions and enum-versus-text choices, and justify each against the relationship patterns and integrity rules of this chapter.
6. One engineer argues that because the backend already validates task status and priority (see Chapter 5), the `task_status` enum and the priority `CHECK` are redundant; another proposes removing the application validation because the database enforces the rules anyway. Evaluate both positions, considering scripts that write directly to the database, the error a client receives, documentation for new team members, and what it takes to add a new status value later.
7. A startup runs subscription billing and a blog whose articles mix text, images, code blocks and video embeds. One proposal uses PostgreSQL for billing and MongoDB for articles; another keeps everything in PostgreSQL with a `JSONB` column for article bodies. Compare the two for data integrity, querying article content, operational cost and the code the backend must contain, and recommend one.
8. After a release, `PATCH /v1/users/:userId` returns 200 with the new bio, yet in production a list sorted by `updated_at` shows edited users in their old positions, while staging behaves correctly. Give three plausible causes involving migrations, triggers or the query itself, the checks that would distinguish them, and the changes to the team's workflow that prevent schema drift between environments.

APPENDIX B GLOSSARY

Term	Meaning
DBMS	Software that stores data and provides efficient, safe CRUD on it
Constraint	Rule checked on every write: NOT NULL, UNIQUE, CHECK, foreign key
Primary key	Column(s) identifying a row; implies NOT NULL and UNIQUE
Foreign key	Column whose values must exist as a key in another table
Referential integrity	Rules such as ON DELETE CASCADE protecting related rows
Enum type	Type restricted to a fixed list of values
Composite primary key	Primary key made of several columns together
Linking table	Table recording a many-to-many relationship
Migration	Versioned SQL file with up and down sections
Seeding	Inserting test data into a development database
Parameterized query	SQL with placeholders whose values are sent separately
SQL injection	Attack in which input becomes executable SQL
Sequential scan	Reading every row of a table to find matches
Index	Lookup structure from column values to row locations
Trigger	Function run automatically on a table event
JSONB	PostgreSQL's binary, indexable JSON type

APPENDIX C FURTHER TOPICS

Transactions and isolation levels

A transaction groups statements so that they commit or roll back together, and its isolation level (Read Committed by default in PostgreSQL, Repeatable Read, Serializable) defines what concurrent transactions see of each other. They are the database's real answer to the lost update of Figure 1: an atomic `UPDATE ... SET amount = amount + 20`, a row lock with `SELECT ... FOR UPDATE`, or Serializable isolation with retries. A backend needs them whenever one request performs several dependent writes, such as creating a project together with its owner's membership row.

Connection pooling

Each PostgreSQL connection is a separate server process with its own memory, and opening one costs a network handshake and authentication. Backends therefore keep a pool of open connections in the driver, or put an external pooler such as PgBouncer in front of the database, and lend connections to requests. The pool size bounds concurrency against the database: too small and requests queue, too large and many scaled-out instances together exhaust the server's connection limit.

Reading query plans with EXPLAIN

`EXPLAIN` shows the plan PostgreSQL chooses for a query, such as a sequential scan, an index scan or a particular join method, and `EXPLAIN ANALYZE` runs the query and reports actual times and row counts. It is how one verifies that an index is used instead of assuming it. The planner relies on

table statistics gathered by `ANALYZE`, so stale statistics can produce poor plans on fast-growing tables.

Keyset pagination

With `OFFSET`, the database still reads and discards every skipped row, so deep pages get slower, and rows inserted between requests shift the pages, producing duplicates or gaps. Keyset (cursor) pagination remembers the sort key of the last row returned and asks for `WHERE (created_at, id) < ($1, $2) ORDER BY created_at DESC, id DESC LIMIT 20`, which an index serves directly. It suits feeds and infinite scrolling, at the price of losing jumps to an arbitrary page number.

Indexes for pattern matching

An ordinary B-tree index on `full_name` generally cannot serve `ILIKE 'j%'`, and no B-tree serves a pattern that starts with a wildcard. The `pg_trgm` extension with a GIN trigram index supports `LIKE` and `ILIKE` with wildcards anywhere, and an expression index on `lower(full_name)` with the `text_pattern_ops` operator class handles case-insensitive prefixes. This matters once search-as-you-type filters run on large tables; full search engines are the topic of Chapter 11.

GIN indexes on JSONB

`JSONB` supports containment (`@>`) and key-existence operators that a GIN index can accelerate, while a filter on a field inside a JSONB column without such an index scans every row. The `jsonb_path_ops` operator class gives a smaller, faster index for containment queries only. It applies when dynamic data is queried by its content rather than merely stored and returned whole.

Expand-and-contract migrations

Renaming or retyping a column in one migration breaks application instances that still use the old schema during a rolling deploy, and some `ALTER TABLE` forms lock a busy table. The expand-and-contract pattern adds the new column, deploys code that writes both, backfills in batches, switches reads, and drops the old column in a later migration; `CREATE INDEX CONCURRENTLY` similarly avoids blocking writes while an index is built. It applies to every production database that must stay available during deploys (see Chapter 27).

Normalization

The normal forms (first, second, third and beyond) describe how to split data so that each fact is stored once, avoiding anomalies in which an update changes one copy of a value but not another. The chapter's schema follows them implicitly, for example by keeping the member role on the membership row. Teams sometimes denormalize on purpose, storing a counter or a copied value for read speed, and must then keep the copies consistent, often with triggers or background jobs.

The N+1 query problem

Fetching a list and then issuing one extra query per item, such as a profile query for each user, turns one request into 1 + N database round trips. It appears easily with ORMs that load relations lazily. The cure is a join, as in the list query of Section 8.1, a batched `WHERE id = ANY($1)`, or the ORM's eager loading; query logs and tracing reveal it (see Chapter 14).

Caching — One-Page Overview

Chapter 9 · every section and sub-section of the paper at a glance

1 Introduction: What Caching Is

Caching cuts the time and effort of repeated work by keeping a chosen subset of data in a faster place.

- **1.1 Less time, less effort** — The subset is chosen by access frequency and the probability of next use.
- **1.2 Hits, misses and the subset** — A hit is served at once, a miss goes to the source. Fast storage is costly, so never cache everything.

2 Caching in Large Systems: Three Examples

Google, Netflix and X cache to avoid repeating heavy computation or moving heavy data.

- **2.1 Google Search: caching computed results** — Results of repeated queries kept in a worldwide distributed in-memory cache.
- **2.2 Netflix: caching content at the edge** — Titles in several resolutions; regional subsets on CDN edge servers near users.
- **2.3 X: caching trending topics** — Trends recomputed every few minutes, stored in Redis, read by all.
- **2.4 The common pattern** — Heavy computation or heavy data: the two triggers for caching.

3 Levels of Caching and Content Delivery Networks

Backend engineers meet network, hardware and software caches; the CDN is the main network-level one.

- **3.1 Network, hardware and software caching** — Software caches such as Redis run on RAM but are used through libraries.
- **3.2 Edge servers and points of presence** — Edge servers near users, grouped into PoPs, in front of the origin.
- **3.3 The life of a CDN request** — URL, DNS to the nearest PoP, edge hit or fetch from the origin.
- **3.4 TTL: how long the edge keeps content** — After the TTL the edge refetches fresh content.

4 DNS Caching

Resolution walks root, TLD and authoritative servers, so every layer caches the answer.

- **4.1 Recursive resolution step by step** — ISP or public resolver; root and TLD referrals; authoritative answer.
- **4.2 Caches at every layer** — Browser, operating system and resolver caches skip the walk; record TTLs bound them.

5 Hardware Caching and Why Memory Is Fast

The memory hierarchy explains why RAM-based stores are fast and what they give up.

- **5.1 The memory hierarchy** — L1, L2, shared L3, RAM, disk and network: fast and small to slow and large.
- **5.2 Locality: the array example** — Sequential traversal is fast because the CPU prefetches memory.
- **5.3 RAM versus disk** — Direct electrical addressing against a mechanical head.
- **5.4 Speed against capacity and volatility** — RAM is small and volatile; in-memory stores persist to disk behind the scenes.

6 In-Memory Key-Value Stores

Redis, Memcached and ElastiCache are in-memory key-value NoSQL databases with a simple interface.

- **6.1 What the name says** — Data in RAM, keys mapped to values, no schema and no SQL.
- **6.2 Working with a store from code** — A client library: set a key and value, get the value by key.

7 Caching Strategies

Lazy caching fills the cache on reads; write-through updates it on writes.

- **7.1 Lazy caching (cache-aside)** — Check the cache; on a miss load from the database and store.
- **7.2 Write-through caching** — Each POST, PUT or PATCH writes database and cache in one flow.
- **7.3 Comparing the two** — Freshness against write cost and memory use.

8 Eviction Policies

A full cache must decide which key leaves when a new one arrives.

- **8.1 Why eviction is unavoidable** — RAM and managed caches have fixed limits; a cache is a subset.
- **8.2 No eviction** — Inserts fail once memory is full; the Redis default.
- **8.3 Least recently used (LRU)** — Evict the key accessed longest ago.
- **8.4 Least frequently used (LFU)** — Evict the key with the fewest accesses.
- **8.5 TTL-based eviction** — Evict the key closest to expiry.

9 Caching in Backend Use Cases

In-memory stores hold query results, sessions, external API data and rate-limit counters.

- **9.1 Database query caching** — Expensive, frequent, read-heavy queries; product pages and profiles.
- **9.2 Session storage** — Session lookups on every request without database load.
- **9.3 Caching external API responses** — Weather data cached for an hour to cut bills and respect rate limits.
- **9.4 Rate limiting** — Per-IP counters in memory; over the limit the answer is 429.

Caching at a glance — search results, video segments, DNS answers and Redis keys are all the same idea: a chosen subset of data kept closer and faster than its source; the engineering lies in choosing that subset, keeping it fresh and evicting it when memory runs out. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Caching:
From the Edge Network to In-Memory Stores

Chapter 9 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Caching keeps a subset of data where it is faster and cheaper to reach, so that repeated work costs less time and effort. Three large systems, Google Search, Netflix and X, show the two situations that call for it: heavy computation and heavy data. At the network level, content delivery networks serve content from edge servers near the user, and DNS caches answers in the browser, the operating system and the recursive resolver. The hardware memory hierarchy explains why RAM-based stores such as Redis and Memcached are fast and what they give up for that speed. The chapter then covers the cache-aside and write-through strategies, the eviction policies, and four backend uses: query results, sessions, external API responses and rate-limit counters.

Keywords — cache hit and miss, content delivery network, DNS caching, memory hierarchy, Redis, cache-aside, write-through, eviction policy, rate limiting

1 INTRODUCTION: WHAT CACHING IS

In one sentence, caching is a mechanism that decreases the time and the effort needed to perform some work. This single technique is a major factor in the performance of high-performance applications, especially those that track latency in two-digit microseconds or milliseconds.

1.1 Less time, less effort

More technically, caching means keeping a *subset* of some primary data in a location that is faster to access, so retrieval takes less time, and easier to access, so it takes less effort. Which subset is kept depends on how the data is used: how frequently it is accessed, how probable it is to be needed next, and further parameters. The cached item may be stored data or the result of an operation; in both cases, retrieving it from the cache is cheaper than returning to the source and redoing the work.

1.2 Hits, misses and the subset

> **Cache, cache hit, cache miss.** A *cache* is a fast store that holds a subset of data whose primary copy lives elsewhere. A lookup that finds the requested data in the cache is a *cache hit* and is answered at once. A lookup that does not is a *cache miss*: the data is fetched or computed from the primary source and, usually, stored in the cache for the next request.

The emphasis on *subset* is deliberate. Fast storage costs more per byte than disk-based secondary storage, which is relatively cheap and offers large capacity. Putting everything in the cache would cost too much and would defeat its purpose, so deciding which data to keep, where, and for how long is the real design problem. Every example in this chapter is an answer to that question.

2 CACHING IN LARGE SYSTEMS: THREE EXAMPLES

2.1 Google Search: caching computed results

A search query passes through a complex workflow built on crawling and indexing billions of web pages and ranking them. Running it is computationally expensive: it consumes CPU, memory and other resources. Many queries repeat; a query such as "what is the weather today" is searched millions of times every day. Without caching, Google's servers would recompute the results for every one of these queries, going through the index and running all ranking algorithms each time, which means high latency, high server load and slow responses.

Instead, Google uses a *distributed in-memory cache*. Distributed means that the cache servers are spread across the world rather than concentrated in one location; they store the results that the search pipeline returns. When a user searches, the system first checks whether results for that query are in the cache. On a hit they are returned instantly, since reading from a cache is fast (Section 5 explains why). On a miss, for a query not typed before, the normal workflow runs, and its result is cached so that the next request for the same query, from the same user or any other, is served from the cache (Figure 1).

```
search query: "weather in Pune"
    |
    v
look up the query in the distributed cache
    |-- hit  --> return cached results instantly
    |
    '-- miss --> look up the index, run the ranking
                 algorithms (expensive)
                     |
                     v
                 store the results in the cache
                     |
                     v
                 return the results
```

Figure 1: Query-result caching in a search engine (illustrative).

2.2 Netflix: caching content at the edge

Netflix delivers movies, series and anime to millions of users worldwide and streams very large volumes of data. Part of the volume comes from *encoding*: each title, say "movie 1", is prepared in several resolutions, such as 1080p, 720p and 480p, for different devices and network speeds. Depending on the user's device and connection, an optimised version is sent dynamically, so the user does not waste bandwidth and the servers carry less load.

Delivering this data with minimal buffering and without overloading Netflix's servers relies on a *content delivery network* (CDN). Netflix's *origin servers*, the server racks in its own data centres that hold the actual titles, are located in one region, say the United States. In addition, servers are placed at *edge locations* all over the world, chosen by the available infrastructure, network connectivity and other parameters, so that each region has a server close to its users. Without them, users near the US data centres would see little buffering, because they are geographically close to the servers, while users on the other side of the world, in India for example, would suffer high latency. With edge caching, each request is served from a nearby edge server instead of the origin.

The edge holds a subset, not the whole catalogue: replicating all of Netflix's storage to every location would cost far too much. Deciding which subset to place in each region takes considerable computation: trend analysis, real-time data and machine-learning models of what people in that region watch. The technique is not specific to Netflix. Companies of every size serve video, images and static web assets (HTML, CSS, JavaScript) through CDNs, and a platform such as Vercel has its own edge network: a request to a frontend deployed there is answered from the region closest to the user rather than from the origin server.

2.3 X: caching trending topics

X, formerly Twitter, shows a *trending topics* section; Facebook, LinkedIn and YouTube use the same strategy for similar features. Trends are identified by analysing millions of tweets in real time to extract patterns, a computation over terabytes of data that needs machine-learning algorithms, GPUs and substantial infrastructure. If it ran every time a user opened the trending section, even half the user base doing so would bring the servers down within minutes or seconds.

The computation is also unnecessary per request. A trend such as an ongoing election in some country does not change within seconds or minutes; it stays in the section for hours or days, so caching it is safe. The exact algorithm and interval are not public, but the design is roughly this: every few minutes, tweets from each region are run through trend-detection algorithms, the result is stored in an in-memory key-value store such as Redis, and every user's request is served from that store. This is why the section appears instantly on a reasonable connection.

2.4 The common pattern

The three systems cache for one of two reasons: to avoid **repeating heavy computation** (ranking, trend detection) or to avoid **repeatedly sending large amounts of data** over long distances (video). These are the two common situations in which caching comes into play (Table 1).

System	Expensive work	What is cached	Where
Google Search	index lookup, ranking	query results	distributed in-memory cache
Netflix	moving terabytes worldwide	popular titles per region	CDN edge servers
X	trend detection over tweets	trending topics	in-memory store (Redis)

Table 1: The three examples and what each one caches.

3 LEVELS OF CACHING AND CONTENT DELIVERY NETWORKS

3.1 Network, hardware and software caching

Caching is implemented almost everywhere, from hardware to operating systems to browsers. A backend engineer meets three levels more often than the rest: **network-level**, **hardware-level** and **software-level** caching. Software-level caching is not purely software: a store such as Redis gets its speed from hardware, namely main memory. It is called software-level because of the means of interaction: the engineer uses it through software, a server and a client library. Network-level caching has two major uses, CDNs and DNS.

3.2 Edge servers and points of presence

A CDN caches content on servers that are geographically closer to the end users than the origin. This minimises latency and reduces the computation and resources consumed at the origin server. The servers are called *edge servers* or *edge nodes*; generally, the word *edge* (edge network, edge computing) means that the work happens on a server close to the user instead of on a central or origin server. A *point of presence* (PoP) is a location in a region where several edge servers are concentrated; the term denotes that collection of edge servers.

3.3 The life of a CDN request

1. The user requests a resource, such as an image, a video or a web page, by entering its URL in the browser.
2. The browser sends a DNS query to resolve the domain name in the URL into an IP address, since DNS turns names into the addresses of actual servers.
3. The CDN's DNS system routes the request to the nearest PoP. The choice depends on the user's geographic location and on network conditions; a user in New York is directed to a PoP near New York. This routing is why DNS is a central component of a CDN.
4. An edge server in the PoP checks whether the requested content is in its cache. On a hit it returns the content directly, the happy path.

5. On a miss, the edge server fetches the content from the origin server, for example in the US for a user in India, returns it and keeps a copy. The copy is the genuine resource: the origin holds the complete set, and every edge fills itself from it.

```
user: GET https://cdn.example.com/images/poster.jpg
  |
  | 1. DNS query for cdn.example.com
  v
CDN DNS --> address of the nearest PoP (e.g. Mumbai)
  |
  | 2. HTTP request to an edge server in that PoP
  v
edge server: is the object in my cache?
  |-- hit  --> respond immediately
  '-- miss --> fetch from the origin (US), respond,
               keep a copy until its TTL expires
```

Figure 2: A request through a CDN (illustrative).

The lecturer adds, as a rough example, that the DNS may also consider connection quality and send a user with a poor connection to a PoP that holds lower-quality versions of a video.

> **A subtle point.** The quality of a video stream is normally not chosen by DNS. Streaming uses adaptive bitrate formats (HLS, MPEG-DASH): each title is cut into short segments in every resolution, and the player chooses the resolution segment by segment from the bandwidth it measures. DNS steering picks an edge by proximity, latency, load and server health; a resolution the edge lacks is fetched from the origin like any other miss.

3.4 TTL: how long the edge keeps content

CDNs attach a *time to live* (TTL) to cached content, which decides how long a copy is kept. Companies choose a fair duration, for example "cache this content for the next few hours", because after that time there may be a new version or regional changes. The TTL acts as a safe default: when a request arrives after it has expired, the edge does not serve the old content but fetches fresh content from the origin, serves it and caches it with a new TTL. Staleness is thus bounded without the origin having to notify every edge of every change. For HTTP content the TTL is normally set by the origin through the `Cache-Control` response header (RFC 9111).

4 DNS CACHING

DNS is used constantly, mostly unnoticed, and billions of users issue DNS queries at the same time. To keep their latency low, DNS relies heavily on caching; the wider path of a request to a server is described in the course's introductory videos.

4.1 Recursive resolution step by step

A user enters a domain name, say `example.com`, in the browser and presses Enter. Of everything that follows, only name resolution is considered here.

1. The device, a laptop or a smartphone, sends a DNS query to a *recursive resolver*. The resolver is typically provided by the user's internet service provider (in India, for example, Jio, ACT or Airtel) or by a public DNS provider such as Google Public DNS or Cloudflare. Which one is used does not change the procedure.
2. The resolver first checks its local cache, its own storage without contacting any remote server, for an entry for `example.com`. On a hit it immediately returns the corresponding IP address to the device.
3. On a miss it queries a *root server*. Root servers do not know the IP address of `example.com`, but they hold the addresses of the servers for the *top-level domains* (TLDs) such as `.com` and `.in`, and reply with a referral to the `.com` TLD servers.
4. The `.com` TLD server does not know the address either; it replies with the address of the *authoritative name server* for `example.com`.
5. The authoritative name server holds the domain's records and returns the IP address, which the resolver passes back to the device.

The resolver is called *recursive* because it goes deeper and deeper through different servers on the client's behalf until it has found the IP address. The lecturer was unsure of the number of root servers: there are 13 named root server identities, `a.root-servers.net` to `m.root-servers.net`, each served from many sites around the world.

```
browser DNS cache --miss--> OS DNS cache --miss-->
recursive resolver (ISP, or public: 8.8.8.8, 1.1.1.1)
  | own cache hit: answer at once
  | own cache miss:
  |-- 1 --> root server
  |   <---- referral to the .com TLD servers
  |-- 2 --> .com TLD server
  |   <---- referral to example.com's name servers
  |-- 3 --> authoritative name server
  |   <---- A record: example.com = 203.0.113.10, TTL 3600
  v
cache the answer for the record's TTL, reply to the device
```

Figure 3: DNS resolution and the caches in front of it (illustrative; the address is a documentation address).

4.2 Caches at every layer

Walking from root to TLD to authoritative server for every request would be far too much work, so DNS does it once and reuses the answer for a period of time, at several layers:

- **Operating system.** Windows, macOS and Linux keep a local DNS cache. If it holds the domain, the device skips the recursive resolver, connects straight to the IP address and continues with the normal browser flow.
- **Browser.** Modern browsers such as Chrome and Firefox keep their own DNS cache of domains resolved for earlier requests and, on a hit, also skip the resolver.
- **Recursive resolver.** The ISP's or public provider's resolver caches answers, so it need not contact root, TLD and authoritative servers again for the same name.
- **Further servers.** According to the lecturer, some name servers further along the chain also keep caches that spare them further requests.

A subtle point. Two refinements. A browser normally consults its own DNS cache first and asks the operating system only on a miss, so the usual order is browser, operating system, resolver. And no layer chooses freely how long it keeps an answer: every DNS record carries a TTL set by the domain owner on the authoritative server, and each cache drops the record when that TTL runs out (RFC 1034). Authoritative servers are the source of their zone's records rather than caches of them.

5 HARDWARE CACHING AND WHY MEMORY IS FAST

5.1 The memory hierarchy

Caching is also built into hardware to make computation and CPU-level operations faster. The CPU cores do the actual processing. Next to them sit several layers of cache: an L1 and an L2 cache, and an L3 cache that is shared between the cores of the chip. Below these is the main memory, RAM (random access memory), and below that secondary storage, hard disks and SSDs, and the network. The CPU keeps data that is used repeatedly, or that is predicted to be used soon, in L1, L2 and L3, so that repeated accesses do not go to the slower levels (Figure 4).

```
fast, small, costly per byte
    CPU core (registers)
    L1 cache    ~1 ns        per core
    L2 cache    ~4 ns        per core
    L3 cache    ~10-40 ns    shared by the cores
    RAM         ~100 ns      gigabytes, volatile
    SSD         ~10-100 µs   terabytes, persistent
    hard disk   ~5-10 ms     mechanical seek
    network     ~0.5-150 ms  data centre to other continent
slow, large, cheap per byte
```

Figure 4: The memory hierarchy with approximate access times (illustrative orders of magnitude, not measurements).

5.2 Locality: the array example

Arrays are good data structures for sequential access partly because of hardware caching. When a program starts traversing an array in one direction, say elements 1, 2, 3, 4, 5, the processor's predictive logic loads the memory that follows into the L1 or L2 cache before it is needed, so most later element reads are cache hits. More precisely, memory moves between RAM and the caches in fixed-size blocks called *cache lines*, typically 64 bytes, and the hardware *prefetcher* recognises the sequential pattern and fetches the next lines in advance.

5.3 RAM versus disk

The level that matters for backend caching is main memory. Technologies such as Redis and Memcached are called *in-memory* key-value stores because they keep their data in RAM. On a hard disk, reading data is a mechanical operation: a head moves over the rotating disk until it reaches the data. RAM is built from capacitors and transistors that are reached with electrical signals: any memory location is accessed directly through its address. The access time is therefore almost the same whichever location is read and in whatever order, which is what *random access* means, and it is why reading from RAM is very fast.

5.4 Speed against capacity and volatility

RAM is not available in abundance. It has limited capacity, and it is *volatile*: when power is switched off, its contents are lost and the computer starts fresh. Its design trades non-volatility and capacity for speed. Secondary storage makes the opposite trade: data is physically written to the disk and persists whether or not a program is using it and whether or not the computer is on, but access is slower. RAM therefore cannot replace disk-based storage. Primary memory gives fast access; secondary storage gives persistence and large, cheap capacity.

In-memory stores combine the two. Every read and modification is served from RAM, which is why their data access is fast; behind the scenes they write data to secondary storage and, when the program starts, load it from disk back into memory. Implementing this persistence is the responsibility of the store, not of the application that uses it.

A subtle point. This description fits Redis, which offers point-in-time snapshots (RDB) and an append-only log of writes (AOF), both configurable. Memcached, in its standard configuration, keeps data only in memory and loses it on restart, so an application must always be able to rebuild what it stored there. SSDs, unlike hard disks, have no moving parts; they are slower than RAM because flash reads take tens of microseconds and pass through the block-storage I/O path.

6 IN-MEMORY KEY-VALUE STORES

6.1 What the name says

In backend development the software-level cache is a technology such as Redis or Memcached, or a cloud service such as AWS ElastiCache, which runs these engines as a managed service with a fixed memory limit. All of them provide storage based on primary memory, and all are described as *in-memory key-value NoSQL databases*. Each part of the name carries meaning:

- **In-memory.** Unlike traditional databases such as PostgreSQL or MySQL, which store data on disk (see Chapter 8), they keep data in RAM, which is the source of their speed.
- **Key-value.** Relational databases impose a strict schema: tables must be created, rows inserted under fixed columns. Here the data structure is simple: a key maps to a value, and the value can be a string, a number, a list, a JSON document and so on. The available data types differ between technologies.
- **NoSQL database.** They do not enforce the strictness of SQL databases: no schema, no SQL queries, no joins or aggregations.

6.2 Working with a store from code

Using such a store is straightforward. Every language has a compatible client library, for instance node-redis for

Node.js or StackExchange.Redis for .NET. To store data the application provides a key and a value; to retrieve it, it provides the key and gets the value back (Figure 5). There are no SQL queries or aggregations to reason about. What needs understanding is the machinery behind it: memory, persistence, caching strategies and eviction, because that knowledge drives the decisions about what to cache and how. For practice the lecturer suggests starting with Redis or Valkey, its open-source alternative (a fork of Redis maintained under the Linux Foundation since 2024): install the client library for your language and compare data access with a relational database.

```
SET sess:9f2c "user=42" EX 1800   -> OK, expires in 30 min
GET sess:9f2c                     -> "user=42"
TTL sess:9f2c                     -> 1793 (seconds left)
DEL sess:9f2c                     -> 1 (one key removed)
GET sess:9f2c                     -> (nil): a cache miss
INCR rate:203.0.113.7             -> 1, then 2, 3, ...
```

Figure 5: Basic Redis commands and their replies (illustrative).

7 CACHING STRATEGIES

When an application caches data from its own database, two strategies are the most common.

7.1 Lazy caching (cache-aside)

With *lazy caching*, also called *cache-aside*, a client requests a resource and the server first checks the cache. If the resource is there, the server returns it. If not, it fetches the resource from the primary storage, stores it in the cache and then returns it, so the next request for it, from the same client or another, finds it in the cache. The strategy is called lazy because nothing is cached proactively, by predicting client interactions or patterns; data enters the cache only when someone actually requests it. The pattern is widespread in backend development (Figure 6, upper part).

7.2 Write-through caching

Write-through is a cache update strategy. Whenever a resource is created or changed, by a POST, PUT or PATCH call, the same API call, in the same execution flow, writes the change to the database and makes the same change in the cache, so the next read can be served from the cache (Figure 6, lower part). Every write operation therefore carries extra overhead, because two stores are updated. The benefit is freshness: since database and cache are updated together, the cache does not hold expired entries, and in the lecturer's words the application never serves old data.

```
// lazy caching (cache-aside): the read path
function getProduct(id):
    key = "product:" + id
    product = cache.get(key)
    if product != null:
        return product                    // cache hit
    product = db.findProduct(id)          // cache miss
    cache.set(key, product, ttl = 3600)
    return product

// write-through: the write path, in the same request
function updatePrice(id, newPrice):
    product = db.updatePrice(id, newPrice)
    cache.set("product:" + id, product)
    return product
```

Figure 6: Cache-aside reads and write-through updates (illustrative pseudocode).

> **Warning.** Write-through is not a transaction. The database write and the cache write are two separate operations: if the cache write fails after the database commit, or two requests interleave (A writes the database, B writes the database, B writes the cache, A writes the cache), the cache keeps an old value indefinitely. Data changed by another path, such as a script or another service, also bypasses the cache. Production systems therefore still give such entries a TTL, or delete the key on write and let the next read refill it.

7.3 Comparing the two

Aspect	Lazy (cache-aside)	Write-through
Data enters the cache	on the first read (a miss)	on every write
First read after a write	stale until TTL or invalidation	served fresh from the cache
Cost of a write	database only	database and cache
Memory use	only data that is read	all written data, read or not

Table 2: Lazy caching and write-through compared.

The two are often combined: reads follow cache-aside, and writes update or delete the cached entry. This combination is the one used for query caching in Section 9.1.

8 EVICTION POLICIES

8.1 Why eviction is unavoidable

An in-memory cache uses RAM, whose capacity is limited compared with secondary storage, and a managed service such as ElastiCache has a fixed storage limit. Sooner or later the cache runs out of memory. Since a cache is by definition a subset of the primary data, the frequently accessed part, some data must then be let go so that new data of higher priority can be stored. The *eviction policy* decides how old data is removed to make room. Eviction differs from expiry: a key with a TTL is removed when the TTL ends, whether memory is full or not, whereas eviction happens only under memory pressure.

8.2 No eviction

With *no eviction*, nothing is removed: once memory is full, the next attempt to insert data fails with an error saying that memory is full. This is the configuration in effect

when no eviction policy has been chosen; in Redis the default `maxmemory-policy` is `noeviction`. For a pure cache it makes little sense, although it suits data that must never disappear silently.

8.3 Least recently used (LRU)

LRU keeps track of when each key was last accessed and evicts the key whose last access is the oldest. In the lecturer's example the cache is full with four keys; keys 1, 2 and 3 were accessed today and key 4 yesterday. When key 5 must be stored, the cache has to make space, and LRU selects key 4, the least recently used: key 4 goes out and key 5 comes in.

8.4 Least frequently used (LFU)

LFU keeps track of how often each key has been accessed so far, rather than when it was last accessed. In the lecturer's second example, keys 1, 2, 3 and 4 have been accessed 5, 10, 6 and 23 times. When key 5 arrives, LFU selects the key with the lowest access frequency, key 1, which goes out to make room for key 5. Placing both examples on the same four keys (Table 3) shows that the policies can disagree: key 4 is the most popular overall but the least recent, so LRU evicts it and LFU keeps it.

Key	Last access	Accesses so far	Evicted by
1	today	5	LFU
2	today	10	—
3	today	6	—
4	yesterday	23	LRU

Table 3: The lecturer's LRU and LFU examples combined; key 5 arrives at a full cache.

8.5 TTL-based eviction

In Redis a TTL can be configured for each key, after which the key is invalidated automatically. The TTL can also drive eviction: when key 5 arrives at a full cache, the policy selects the key with the lowest remaining TTL, the one that would expire soonest anyway, removes it and inserts the new key.

Policy	Evicts	Redis setting
No eviction	nothing; writes fail when full	`noeviction` (default)
LRU	key used longest ago	`allkeys-lru`, `volatile-lru`
LFU	key used least often	`allkeys-lfu`, `volatile-lfu`
TTL-based	key closest to expiry	`volatile-ttl`

Table 4: Eviction policies and their Redis `maxmemory-policy` values.

The `volatile-` variants consider only keys that have a TTL, and random variants also exist. Redis approximates LRU and LFU by sampling a few keys instead of keeping an exact order, which saves memory.

9 CACHING IN BACKEND USE CASES

9.1 Database query caching

Consider an SQL query that joins many tables, aggregates over millions of rows and finally returns a few rows: a compute-intensive operation. Monitoring shows that the API calling it is hit very frequently, because it backs the landing page or a dashboard, and that it puts a heavy load on the database. The remedy is to cache the query's result with a TTL, for example one hour. Each request checks the cache first and is served from it; on a miss the query runs once and its result is stored. When the underlying data is modified, the application updates or deletes the cached entry, or handles it in another suitable way. The database is relieved, compute is saved, and the latency of the API drops.

Real platforms apply the same idea. E-commerce sites such as Amazon cache product details, prices and inventory data instead of querying the database for every request. During a sale on a MacBook, millions of users open the same product page; without caching, fetching its images and description would send millions of database reads for information that rarely changes, keeping the database from its important work. Social platforms such as X and Facebook cache user profiles, which change perhaps a few times a year. A celebrity's profile may be fetched a thousand times a day, or a million times before a film release; when the user edits it, the entry is invalidated and replaced. The common pattern is **read-heavy access with infrequent writes**, the strongest signal that data should be cached. Values that must be exact at the moment of a transaction, such as stock and price at checkout, are still read from the database at that moment.

9.2 Session storage

In a typical authentication flow (see Chapter 4), a successful login generates a session token for the user, and the session is stored on the server side. Every later request or API call must fetch that session information. If it lived in the main database, each API call would add a database query: extra latency on every request and extra load on the database. Reading from RAM is much faster, so session data is usually stored in Redis or another in-memory store. Because that store then holds the only copy of the sessions, its persistence and replication settings decide whether a restart logs every user out.

9.3 Caching external API responses

A backend may depend on an external API, for example a weather API, whose data it processes to serve its own frontend. Without caching, every frontend request triggers another call to the weather API, and with many users this becomes thousands of external calls. External APIs usually impose a rate limit or charge by the number of calls, so the bill grows quickly or the limit is hit. Weather data does not change within seconds or minutes, so it is safe to cache. The backend fetches it once, caches it with a TTL of, say,

one hour (keyed by location), and serves all frontend requests in that hour from the cache. When the hour has passed, the entry is invalidated automatically; the next request fetches fresh data, caches it and returns it. Whenever a backend interacts with an external API, caching should be considered to lower the bill and stay under the rate limit.

9.4 Rate limiting

Rate limiting is usually implemented as a middleware (see Chapter 6) placed after the server's entry point and before the route handlers or controllers. For every request it determines the client's public IP address, typically from the `X-Forwarded-For` header, which a reverse proxy such as nginx adds. It then applies a rule such as "at most 50 requests per minute per client", to stop abuse by bots and to protect compute-intensive APIs. The counter lives in a key-value store: the key identifies the IP address (and the current window), and the value counts the requests. The first request in the minute sets the counter to 1, each later one increments it, and once the counter exceeds 50 within the minute the middleware blocks the request and responds with status `429 Too Many Requests` (Figure 7; status codes are covered in Chapter 1).

```
rateLimit(request, next):
    ip  = clientIp(request)       // from X-Forwarded-For
    key = "rate:" + ip + ":" + currentMinute()
    count = INCR key              // atomic; new key -> 1
    if count == 1:
        EXPIRE key 60             // window removes itself
    if count > 50:
        return 429 Too Many Requests
    return next(request)          // on to the handler
```

Figure 7: A fixed-window rate limiter on Redis, 50 requests per minute per IP address (illustrative).

The counters could be kept in PostgreSQL or MySQL, which offer persistent storage and easy retrieval. The difference is time: reading from a relational database is slower, and even 20 or 30 milliseconds per call is added to the latency of every API, since each request makes an extra database call. The database load also grows with traffic: a thousand users making 100 requests a minute produce 100,000 counter updates a minute, about 1,700 a second, of pure rate-limiting traffic. Keeping the counters in an in-memory store serves both goals, minimal API latency and a database free for real work.

Warning. Any client can send its own `X-Forwarded-For` header. Each proxy appends the address it received the request from, so the trustworthy entry is the one added by your own outermost proxy, counted from the right, not the first one in the list. A limiter keyed on a client-supplied value can be bypassed by changing the header on every request, or used to exhaust another client's quota.

Use case	Why it is cached	Freshness handled by
Query results	expensive, called often	TTL, invalidation on change
Products, profiles	read-heavy, rarely written	invalidation on edit
Sessions	read on every request	session expiry (TTL)
External API data	rate limits, per-call price	TTL (one hour)
Rate-limit counters	updated on every request	window TTL

Table 5: The backend use cases of in-memory stores.

10 CONCLUSION

- Caching keeps a chosen subset of data in a faster, cheaper-to-reach place; a hit is served at once, a miss goes to the source and usually fills the cache.
- Cache when work is computationally heavy (ranking, trend detection, complex queries) or data is heavy to move (video, static assets).
- A CDN routes users through DNS to the nearest point of presence; edge servers serve hits, fetch misses from the origin and keep copies for a TTL.
- DNS answers are cached in the browser, the operating system and the recursive resolver, so the walk through root, TLD and authoritative servers is rare.
- RAM is fast because every address is reached directly, but it is small and volatile; Redis adds disk persistence, Memcached accepts loss.
- Cache-aside fills the cache lazily on reads; write-through updates database and cache together, at the price of slower writes and without true atomicity.
- A full cache needs an eviction policy: no eviction rejects writes, LRU drops the least recent key, LFU the least used, TTL-based the one expiring soonest.
- Typical backend uses are expensive query results, read-heavy data, sessions, external API responses and rate-limit counters.

APPENDIX A REVIEW QUESTIONS

1. A product catalogue API serves 5,000 reads per second and 50 price updates per minute over 2 million products of about 4 KB each, and the team has a 2 GB Redis instance. Design the caching layer: the strategy for reads and for writes, the TTL and the eviction policy, justifying each choice with numbers. State what a customer may see in the worst case right after a price change.
2. A dashboard endpoint runs a 3-second aggregation query, cached with cache-aside and a one-hour TTL, and receives 400 requests per second. After a routine Redis restart, p99 latency rises from 40 ms to 6 s for about ten minutes while database CPU stays at 100%. Reconstruct what happened during the first seconds after the restart, and propose two changes that would have prevented the incident.

3. Two requests update the same product under write-through caching with no TTL on cache entries: A sets the price to 100 and B to 120, and the operations run in the order A writes the database, B writes the database, B writes the cache, A writes the cache. State what the database and the cache contain afterwards and how long readers see each value. Then change the write path so that this outcome cannot occur.
4. A cache holding three keys sees the accesses A, A, A, A, B, C, C, B, C before key D must be inserted. Determine which key LRU evicts and which key LFU evicts. A nightly job then reads 10,000 distinct product keys exactly once each through the same cache; compare what each policy does to the daytime hot keys and choose one.
5. At 10:00 on Monday you change the A record of `api.example.com`, whose TTL is 86,400 seconds, to a new server, and you shut the old server down at 11:00. Name every cache layer that may still hand out the old address, give the latest time at which a correctly behaving client can still be sent to the old server, and describe how the migration should have been planned.
6. A news site serves `/styles.css` through a CDN with a 24-hour TTL. A release changes the HTML, which is not cached, and the stylesheet, deployed under the same URL; for the rest of the day some readers see a broken layout and others do not, and the split follows countries. Explain the symptoms in terms of points of presence and TTLs, and propose a deployment practice that removes the problem.
7. A login endpoint allows 50 requests per minute per IP address with the fixed-window counter of Figure 7 and runs on four application instances behind a load balancer. Predict what happens when a client sends 50 requests at 10:00:59 and 50 more at 10:01:00, and what would happen if each instance kept its counters in its own process memory instead of Redis. State what an attacker gains if the limiter keys on the first entry of `X-Forwarded-For`.
8. An application (see Chapter 4) stores sessions with a 30-minute TTL in the same Redis instance that caches query results without a TTL, and memory fills up during a sales peak. Predict what logged-in users and new logins experience under `noeviction`, `allkeys-lru` and `volatile-ttl`. Recommend a configuration and justify it.

APPENDIX B GLOSSARY

Term	Meaning
Cache	Fast store holding a subset of data whose primary copy lives elsewhere
Cache hit / miss	Requested data found in the cache / not found, so fetched from the source
CDN	Content delivery network: servers worldwide that cache content near users
Edge server	CDN server close to the users of a region
Point of presence (PoP)	Location where several edge servers of a region are concentrated
Origin server	Server holding the complete, authoritative content behind a CDN
TTL	Time to live: how long a cached entry may be used before it expires
Recursive resolver	DNS server that queries root, TLD and authoritative servers for the client
Authoritative name server	DNS server that holds the records of a domain
Volatile memory	Memory such as RAM that loses its contents without power
In-memory key-value store	NoSQL database keeping key-to-value data in RAM, e.g. Redis, Memcached
Cache-aside (lazy caching)	Fill the cache on a read miss from the primary storage
Write-through	Update database and cache together on every write
Eviction policy	Rule for which key is removed when the cache is full
LRU	Least recently used: evict the key accessed longest ago
LFU	Least frequently used: evict the key with the fewest accesses

APPENDIX C FURTHER TOPICS

HTTP caching headers

HTTP has its own caching model, specified in RFC 9111. The `Cache-Control` response header tells browsers and shared caches such as CDNs whether a response may be stored (`public`, `private`, `no-store`) and for how long (`max-age`, `s-maxage`). Validators such as `ETag` let a cache revalidate a stale copy with a conditional request and receive a short `304 Not Modified` instead of the full body. These headers are how a CDN's TTL is actually set, and a wrong header can leak one user's private response to others through a shared cache.

Cache stampede

When a popular key expires or a cache restarts empty, many concurrent requests miss at once and all run the same expensive query against the database; this is a cache stampede or thundering herd. Remedies are request coalescing, where one request recomputes while the others wait for its result, random jitter on TTLs so that keys do not expire together, and refreshing hot keys before they expire. The risk is greatest exactly where caching was most needed: expensive computations under high traffic.

Write-behind and refresh-ahead

Two further strategies complement cache-aside and write-through. Write-behind (write-back) writes to the cache first and flushes changes to the database asynchronously, often in batches; writes become fast, but data is lost if the cache fails before the flush. Refresh-ahead reloads entries shortly before they expire, so readers of hot keys never meet a miss. The first suits high-volume counters and metrics, the second read-hot data with predictable access.

Negative caching

A cache can also remember that something does not exist. DNS resolvers cache "no such domain" answers for a period set by the zone (RFC 2308), and applications can cache "not found" results for missing IDs. Without it, repeated requests for non-existent keys, accidental or malicious, always miss and reach the database. Negative entries should have short TTLs, because the item may be created soon.

Consistent hashing and distributed caches

One cache node is limited by its memory, so large caches are partitioned across many nodes. With simple modulo hashing, adding or removing a node remaps almost every key and in effect empties the cache; consistent hashing moves only about 1/N of the keys. Redis Cluster uses a related scheme of 16,384 hash slots assigned to nodes, with replicas for failover. These designs matter once a cache outgrows one machine or must survive a node failure.

Rate-limiting algorithms

The fixed-window counter allows bursts of up to twice the limit across a window boundary. Sliding-window counters smooth this by weighting the previous window, and the token bucket refills tokens at a steady rate while permitting controlled bursts. In Redis these algorithms are made atomic with Lua scripts or transactions; in .NET, `System.Threading.RateLimiting` and the ASP.NET Core rate-limiting middleware provide fixed-window, sliding-window, token-bucket and concurrency limiters. The choice matters for login endpoints and public APIs.

In-process and two-level caching in .NET

Not every cache needs a network hop. An in-process cache such as `IMemoryCache` answers in nanoseconds but is private to each application instance, so instances can disagree and the data is lost on restart. `IDistributedCache` abstracts a shared store such as Redis, and `HybridCache`, introduced with .NET 9, combines both levels and adds stampede protection. A two-level design must invalidate both levels, which suits small, hot, read-mostly data such as configuration and reference tables.

Anycast routing

Anycast announces the same IP address from many locations, and Internet routing (BGP) delivers each packet to the topologically nearest one. The DNS root servers, public resolvers such as 1.1.1.1 and 8.8.8.8, and many CDNs work this way, as an alternative or complement to DNS-based steering. It explains how 13 root server names are served from well over a thousand sites, and why users in one region may reach an unexpected edge when routing changes.

Task Queues & Background Jobs — One-Page Overview

Chapter 10 · every section and sub-section of the paper at a glance

1 Introduction: Work Outside the Request–Response Cycle

Work the response does not depend on can run later, in another process.

- **1.1 What a background task is** — Code outside the request–response cycle, handed to a separate process.
- **1.2 Why it matters** — Slow or unreliable steps otherwise set the speed and reliability of the API.

2 A Worked Example: The Verification Email

Sending the sign-up email inside the request ties users to the provider's uptime; a queue removes the tie.

- **2.1 Sign-up and the email provider** — Template, recipient, subject, sender; API call to Resend, Mailgun, Brevo.
- **2.2 Synchronous failure** — Provider down: sign-up fails, or the user is told an unsent email was sent.
- **2.3 Enqueue and return** — Serialize the email data to JSON, push it to a queue, return 200/201 at once.
- **2.4 The consumer side** — A worker dequeues, deserializes and runs the registered handler.
- **2.5 Retries and exponential backoff** — Failed task re-queued; waits 1, 2, 4, 8 minutes; at most five retries.
- **2.6 The two benefits** — Responsive APIs without timeouts; automatic failure detection and retry.

3 Typical Background Workloads

Work that calls external services or is heavy to compute.

- **3.1 Sending emails** — Every transactional email depends on an external provider.
- **3.2 Images and videos** — Resizing and re-encoding for each device and network.
- **3.3 Reports** — Periodic PDFs or emails, run by cron or library schedulers.
- **3.4 Push notifications** — Device tokens in the database; Google's or Apple's service delivers.

4 How a Task Queue Works

Producers enqueue serialized tasks, a broker holds them, consumers dequeue, execute and acknowledge.

- **4.1 Definition** — A system for managing and distributing background jobs: a to-do list for the backend.
- **4.2 The producer** — Builds the task with all the data a consumer needs, then enqueues it.
- **4.3 The broker** — Temporary holding area until a worker is ready.
- **4.4 The consumer** — Separate process or thread; watches the queue, dequeues, runs the handler.
- **4.5 Brokers and libraries** — RabbitMQ, Redis, Amazon SQS; Celery, BullMQ, Asynq on top.
- **4.6 Acknowledgements** — An ack removes a finished task; a missing ack triggers retry logic.
- **4.7 Visibility timeout** — An unacknowledged task becomes available to other workers again.

5 Kinds of Tasks

Tasks differ by trigger and structure: single, periodic, dependent or fanned out.

- **5.1 One-off tasks** — One event, one task: verification, welcome or reset email.
- **5.2 Recurring tasks** — Periodic reports and cleanup of orphaned sessions.
- **5.3 Chain tasks** — Encode a video, then thumbnails and transcript in parallel.
- **5.4 Batch tasks** — Account deletion fanning out; reports for thousands of users.

6 Design Considerations at Scale

Tasks must survive retries, failures, growth and the limits of other services.

- **6.1 Idempotency** — Safe to run twice; a rolled-back transaction is only part of it.
- **6.2 Error handling** — Catch, log and retry: failures in another process are otherwise silent.
- **6.3 Monitoring** — Queue length, successes, failures and causes; Prometheus and Grafana.
- **6.4 Scaling** — Add consumer nodes horizontally as load grows.
- **6.5 Ordering** — Ordered delivery only where library and broker support it.
- **6.6 Rate limiting** — Throttle calls to external services that limit and bill per call.

7 Best Practices

Small, short, observable tasks are easier to retry, scale and debug.

- **7.1 Small, focused tasks** — One processing unit per task; chains so only the failed part reruns.
- **7.2 Avoid long-running tasks** — Split into concurrent tasks or a parent–child chain.
- **7.3 Error handling and logging** — Let failures reach the queue; record enough to find the cause.
- **7.4 Queue length and worker health** — Alert on backlog thresholds and on workers going down.

Background jobs at a glance — the handler validates, stores and enqueues; a broker holds the serialized task; a worker runs it and acknowledges it, and failures are retried with backoff, so slow or unreliable work never blocks the response. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Task Queues and Background Jobs: Moving Work Out of the Request Path

Chapter 10 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — A background task is code that runs outside the request–response life cycle, so the API can answer before slow or failure-prone work is done. Using the sign-up verification email, the paper contrasts a synchronous call to an email provider with a design that serializes the job into a queue, returns at once and lets a separate worker send the email and retry it with exponential backoff. It surveys the usual background workloads and then the mechanics of a task queue: producers that enqueue, a broker that stores, consumers that dequeue and acknowledge, and the visibility timeout that keeps unacknowledged tasks from being lost. Four kinds of tasks are distinguished: one-off, recurring, chain and batch. The paper closes with the design considerations that matter at scale and with best practices for keeping tasks small, short and observable.

Keywords — background jobs, task queue, message broker, producer and consumer, acknowledgement, visibility timeout, exponential backoff, idempotency, Celery and BullMQ

1 INTRODUCTION: WORK OUTSIDE THE REQUEST–RESPONSE CYCLE

A backend exists to answer requests: a client sends a request, the server processes it and returns a response (see the course's introductory videos). Some of the work a request sets in motion, however, does not have to be finished before the response leaves. Separating that work from the request path is one of the main techniques for building backends that are both scalable and responsive.

1.1 What a background task is

> **Background task.** A background task, or background job, is any piece of code, logic or workflow that runs outside the request–response life cycle of the client–server interaction that caused it. It is handed to a separate process, which finishes it as programmed and reports the result to whichever client or process it is meant to inform.

What qualifies work for the background is that it need not happen immediately. The response does not depend on its result, so it is not mission-critical to that response and need not be synchronous. Because nobody is waiting for it, it can be offloaded to another process and completed on that process's schedule, with retries if it fails.

1.2 Why it matters

Work kept inside the request handler sets two limits on the API. The response can be no faster than the slowest step, and some steps are slow by nature: a call to a third-party service, or heavy processing such as resizing images. The response can also be no more reliable than the least reliable step: when an external service is down, the request either fails or has to conceal the failure. Background processing removes both limits. The handler does the essential work, records the remainder as a task and returns; a worker does the remainder later, and the task-queue system detects failures and retries.

2 A WORKED EXAMPLE: THE VERIFICATION EMAIL

2.1 Sign-up and the email provider

A user signs up on a SaaS platform by typing a name, an email address and a password; the front end sends these to the backend in an API call. The backend validates the input — the format of the email, the length and complexity of the password — stores the user in the database and generates a verification code. It must then send a verification email. Nearly every platform does this, because it proves that the address belongs to the person who typed it rather than being someone else's address typed at random. The email contains either a link, which brings the user back to the front end to complete verification (in some flows, to set a password), or a one-time password: a six- or eight-digit code that the user types into the interface.

The backend does not deliver email itself; it uses a third-party email provider, either a modern API-based one such as Resend, Mailgun or Brevo, or an SMTP provider. The backend fills an HTML template with the code or link and the user's details; it supplies the recipient's address, the subject, the sender's address and whatever default parameters the provider expects; and it makes an API call from its server to the provider's server. The provider checks what it needs to — that the API key is valid, that you own the sending address — sends the email through its own infrastructure, and replies to your server with success or failure.

2.2 Failure in the synchronous design

The provider's server is not under your control and may be less responsive than your own: a traffic spike or an outage on its side, among many possible causes, makes it slow or unavailable. Suppose the whole flow runs synchronously in the sign-up request. The backend validates, writes the user to the database, creates the verification code, calls the pro-

vider — and the call fails. There are two outcomes (Figure 1):

- **The provider call has no proper error handling.** The error propagates and the sign-up API itself fails. The user cannot create an account because an email service elsewhere is down. If the user row was already committed, a second attempt may even be rejected as a duplicate address.
- **The error is handled.** Only the email interaction fails; sign-up succeeds and the front end shows "we have sent you a verification email". That message is false. The user has to come back and, if the platform offers it, press "resend email". Resending is a separate API call: if the provider has recovered, the email goes out; if it is still down, the call fails again and the user is again told an email is on its way.

In both cases the user's experience depends on the availability of a third party at the exact moment of the request.

```
SYNCHRONOUS: the email is sent inside the request
client          backend                  provider
  |-POST /signup--->|                        |
  |                 | validate, insert user, |
  |                 | create code            |
  |                 |----- send email ------>|
  |                 |<---- 503 / timeout ----|
  |<-- 500 or 201 --|                        |
  500: the sign-up fails. 201: a false "email sent".

QUEUED: the request only enqueues
client          backend        queue          worker
  |-POST /signup--->|             |              |
  |                 | validate, insert, code     |
  |                 |-- enqueue -->|             |
  |<-- 201 ---------|             |              |
  |                 |             |-- dequeue -->|
  |                 |             |<---- ack ----|
  worker: deserialize, call the provider, then ack;
  on failure the task is retried later with backoff
```

Figure 1: The sign-up request with the email sent synchronously (top) and through a task queue (bottom); illustrative.

2.3 Enqueue and return

The background design changes only the last step. The handler still does the initial processing, generates the verification code and constructs the email, but instead of calling the provider in the same function and the same request–response cycle, it takes all the information needed to send the email, packages it into a format such as JSON (serializes it) and pushes it into a queue as a task. The queue may hold many tasks or none; it does not matter. No API call to the provider happens at this point: the task only records that a particular function must be executed at some point, somewhere — neither is known yet. The handler then returns a success status, 200 or 201 depending on the semantics chosen (see Chapter 1), and the user immediately sees a screen saying that a verification email has been sent and that its link will verify the account. That completes the creation side of the task.

2.4 The consumer side

On the other side of the queue are *consumers*, called *workers* in some languages and frameworks. A consumer is a program running in a process separate from the main backend application. Many configurations are possible, and at scale producers and consumers run on separate nodes, but the flow is the same: a consumer takes the task out of the queue and, since the task was serialized, deserializes it into its language's native form — a dictionary in Python, an object in JavaScript, a struct in Go. It now holds all the data it needs to perform the task.

Three things are configured on the consumer in advance. The first is the queue it reads from: emails might go to an email queue, in-app notifications and mobile push notifications to queues of their own, each served by its own kind of consumer, set up through whatever library or framework is in use. The second is the shape of the data it expects in a task, which belongs to its deserialization layer. The third is a registered *handler*: the code that runs when it is given that data. The handler is the very function that ran synchronously before background jobs were introduced. Here it takes the HTML template, the sender and receiver addresses, the subject and the user's information (ID, first name, email), and calls the API of the provider in use — Resend, Mailgun or Brevo — which finally delivers the email (Figure 2).

The detour through the queue adds little delay. With few tasks queued, the whole path takes milliseconds to a few seconds; with heavy traffic and a long queue, usually no more than five to ten seconds. Verification emails typically expire after 15 or 20 minutes, so latency is not a problem for this kind of task.

```
# producer: the sign-up handler (backend process)
def signup(req):
    user = users.create(req.name, req.email, req.password)
    code = new_code(digits=6)
    queue.enqueue("email", "send_verification_email",
                  {"user_id": user.id, "to": user.email,
                   "first_name": user.name, "code": code})
    return 201, {"message": "Verification email sent"}

# what travels through the broker (serialized JSON)
{"task": "send_verification_email", "queue": "email",
 "args": {"user_id": 4812, "to": "asha@example.com",
          "first_name": "Asha", "code": "482913"}}

# consumer: a separate worker process
@worker.task(queue="email", name="send_verification_email")
def send_verification_email(data):   # deserialized dict
    html = render("verify.html", name=data["first_name"],
                  code=data["code"])
    provider.send(to=data["to"], sender="no-reply@app.io",
                  subject="Verify your email", html=html)
    # raising here marks the task failed -> retried later
```

Figure 2: Producer and consumer of the verification email, with the serialized task between them (illustrative pseudo-code).

2.5 Failure, retries and exponential backoff

Now repeat the outage. The provider is down, the API call fails, the handler registered with the consumer fails, and so the task has failed. In the synchronous design this broke the request–response cycle and returned a 500 Internal Server Error. In the background no response is at stake. The library that manages the tasks — Celery in Python,

BullMQ in Node.js, Asynq in Go — detects the failure and injects the task into the queue again to be retried.

Retries follow an algorithm, and a popular one is *exponential backoff*: after the task fails it is retried after, say, one minute; if it fails again, after two minutes; then four, then eight — every failure increases the wait. A maximum number of retries, for example five, is configured in advance (Table 1). Large providers such as Resend or Mailgun rarely fail for four or eight minutes straight; their outages usually last seconds or less. The task typically fails once, at most twice, and the third attempt succeeds: the external service is back, the email is sent and the user receives it. Although the provider was down when the user signed up, the email arrived, through the retry mechanism alone.

Retry	Wait before it	Time since first failure
1	1 min	1 min
2	2 min	3 min
3	4 min	7 min
4	8 min	15 min
5	16 min	31 min

Table 1: Exponential backoff with a one-minute base, doubling after each failure, at most five retries; after the fifth failure the task is marked as failed.

The schedule and the token lifetime interact: with a 15-minute code, an email that only gets through on the fourth retry, 15 minutes after the first failure, arrives just as its code expires. The two should be chosen together.

2.6 The two benefits

The example shows the two advantages of background processing. The first is responsiveness: the API call is not blocked by an external service or by heavy processing, so the backend answers quickly and does not time out because of a dependency. The second is reliability: tasks that are prone to fail get failure detection and retries from the framework. Together they let a backend offload any time-consuming, non-critical operation, and they significantly improve the overall user experience.

3 TYPICAL BACKGROUND WORKLOADS

In a typical SaaS application four kinds of work are routinely offloaded. Each either depends on an external service or is too heavy to run while a user waits.

3.1 Sending emails

Every transactional email depends on an external provider and is offloaded exactly as in Section 2.

3.2 Processing images and videos

When a user uploads an image, the backend usually has to resize it into several sizes and formats so that delivery can be optimised for the user's device and network conditions: smartphones need smaller images, desktop screens large ones (the XL and 2XL variants and so on). Videos need the same treatment at far greater cost. The uploader does not need to wait for any of this, so it runs in the background.

3.3 Generating reports

Enterprise SaaS applications send periodic reports. A project-management application, for example, sends each user a daily, weekly or monthly report according to the user's settings, with statistics on processed, completed and pending tasks and the pending items of the current sprint. The report is built either directly as HTML email content or as a generated PDF, and it is sent on a timetable: daily at midnight, weekly on Sunday at 12:00. Such work is offloaded to the background with cron jobs. Task-queue libraries such as Celery and BullMQ also provide scheduled tasks: you configure a date, a time or an interval, and the task is executed again and again at those times (Celery through its *beat* scheduler, BullMQ through repeatable jobs).

3.4 Sending push notifications

Push notifications are the messages that appear in a smartphone's notification panel rather than inside an app: a food-delivery app such as Swiggy or Zomato suggesting an order, or reporting that the delivery partner has reached the restaurant. The backend cannot deliver them to the phone itself; delivery is done by a service of the operating system's vendor. When the app is installed, the device is registered with that push notification service — Google's (Firebase Cloud Messaging) or Apple's (the Apple Push Notification service). The backend stores a code identifying the device (a device token) in its database, and to notify that device it calls Google's or Apple's service, depending on the user's operating system, which then delivers the notification. Because this again means calling an external service, it is offloaded to a background task.

4 HOW A TASK QUEUE WORKS

4.1 Definition

> **Task queue.** A task queue is a system for managing and distributing background jobs. Producers create tasks and enqueue them; a broker stores them until a worker is ready; consumers dequeue them, execute them and acknowledge each one when it is done.

The task queue is the mechanism behind every example so far: the engine that lets an application reliably hand off work to a separate process. The lecturer's picture is a to-do list for the backend. Application code adds tasks to the list, and workers pick them off one by one and carry them out.

4.2 The producer

The *producer* is the application code — Node.js, Python, Go or whatever the backend uses — that creates the task and pushes it into the queue; that is the whole of its job. Creating the task means including everything the consumer will need to execute it: the user ID and profile information such as name and email for an email task, the image for an image-processing task, all the data for a report. The producer serializes this into JSON or another serializable format used by the library and pushes the task into the queue.

Adding an item to a queue is called *enqueuing*, as in data structures. Serialization is an essential part of enqueuing, because the task must cross a process boundary and often a network. In practice large payloads such as the image travel by reference, as a storage key or URL, since brokers are designed for small messages.

4.3 The broker

The queue between producers and consumers is also called the *broker*. It stores tasks until a worker is ready to process them: a temporary holding area from which consumers pick tasks up. Because tasks can wait there, producers and consumers need not run at the same pace; a burst of sign-ups lengthens the queue instead of slowing the responses.

4.4 The consumer

The *consumer*, or worker, runs in a separate process or a separate thread. The technical details vary with the language, framework and library, but the fundamentals do not. It may share the backend's codebase or live in a separate one. It constantly monitors the queue for new items; when it finds a task it takes it out — *dequeues* it — and executes the handler registered for that task (Figure 3).

```
PRODUCERS          BROKER (queues)        CONSUMERS
signup handler -> email:  [t3 t2 t1] -> email workers
upload handler -> media:  [t6 t5]    -> media workers
scheduler      -> report: [t9]       -> report workers
           enqueue: serialize    dequeue: deserialize

After a worker dequeues t1:
  t1 is hidden from other workers (visibility timeout)
  handler succeeds -> ack     -> broker deletes t1
  handler fails    -> failure -> retried after backoff
  no ack in time   -> t1 visible again to any worker
```

Figure 3: Producers, broker and consumers with a queue per kind of work, and what happens to a task after it is dequeued (illustrative).

4.5 Broker technologies and libraries

In a real system the abstract queue is a concrete technology: RabbitMQ, Redis, or Amazon SQS, a managed queuing service from AWS. The lecturer names the publish/subscribe module of Redis as a common basis for task queues, and recommends SQS when task processing must scale across nodes spread around the world, since AWS operates it as a managed service in many regions. Strictly, each SQS queue belongs to one region, so a multi-region design runs a queue per region. The libraries in Table 2 sit on top of these brokers and implement the task-level features: serialization, handler registration, retries and scheduling.

Language	Library	Typical brokers
Python	Celery	RabbitMQ, Redis, Amazon SQS
Node.js	BullMQ	Redis
Go	Asynq	Redis

Table 2: The task-queue libraries named in the lecture and the brokers they run on.

A subtle point. Redis Pub/Sub is a poor fit for a task queue. It is fire-and-forget: a published message goes only to the subscribers connected at that instant and is not stored, so a task published while every worker is down or restarting is lost, and there is no acknowledgement to trigger a retry. Redis-based task libraries such as BullMQ and Asynq therefore keep tasks in ordinary Redis data structures (lists, sorted sets), where they persist until a worker takes them; Redis Streams are another durable option. Pub/Sub suits broadcast notifications, not jobs that must run.

4.6 Acknowledgements

When a consumer completes a task, it sends an *acknowledgement* back to the queue. This tells the queue that the task was processed successfully and can be removed. If no acknowledgement arrives, the queue decides from its parameters what happened and how to react: a task that became unresponsive gets one retry treatment, a task that failed gets another. Celery, BullMQ and Asynq already handle these edge cases.

4.7 Visibility timeout

The *visibility timeout* is the period during which a task, once taken by a consumer, is considered in progress and is hidden from other consumers. If the consumer does not send an acknowledgement within the configured timeout — perhaps it crashed, perhaps the external service it called hung — the queue makes the task available to other consumers so that one of them can process it. The reason lies in the shape of the workflow: the producer pushes a task, a consumer takes it out and processes it, and the queue then removes it. A consumer that took a task and never reported success or failure would otherwise take the task down with it. Someone has to acknowledge every task, as a success or as a failure.

At-least-once delivery. The queue cannot tell a dead worker from a slow one. A task that legitimately runs longer than the visibility timeout is handed to a second worker while the first is still executing it, and a worker that finishes a task but crashes before its acknowledgement arrives causes the whole task to run again. Tasks are therefore delivered at least once, not exactly once: the timeout must exceed the slowest normal run, and every handler must tolerate being executed twice (Section 6.1).

5 KINDS OF TASKS

5.1 One-off tasks

A *one-off task* is a single task triggered by something that happens in the request–response cycle: a particular scenario occurs and one function must run in the background. Sending an email is the typical case. A user registers, so a verification email is sent; verification succeeds, so a welcome email is sent; someone asks to reset a password, so an email with a reset link is sent. In a social-media application, someone sends you a message and a notification goes

out. One-off tasks are encountered far more often than the other kinds, but the others have their place.

5.2 Recurring tasks

Recurring tasks execute periodically at specific intervals. One family is reporting: daily, monthly or annual reports giving users their statistics for that day, month, quarter or year. The other is cleanup and maintenance. Consider stateful authentication (see Chapter 4), in which every session is stored in a sessions table. As users log in and out over a long period, the table fills with abandoned, orphaned sessions that are no longer active but were never deleted. A recurring task, running every month (at its start or end) or every two or three months, goes through the sessions that have been inactive for a long time and deletes them, so that they stop occupying storage in the database.

5.3 Chain tasks

Chain tasks have parent–child relationships: a child task can be triggered only after its parent has run and completed successfully. The lecturer's example is a learning management system (LMS) such as Udemy, where instructors create courses by uploading videos. The instructor selects a video and starts the upload; the backend acknowledges at once so that the request is not blocked, while the upload proceeds — for example directly to S3 through a pre-signed URL (see Chapters 20 and 21). When the upload completes, a set of dependent tasks is triggered (Figure 4):

1. **Encode the video** into several resolutions and formats, to cover a wide range of network conditions and devices and so that it can be served efficiently from CDNs.
2. **Generate thumbnails**, only after encoding has finished.
3. **Process the thumbnails** into several resolutions for different devices and networks, only after thumbnail generation.
4. **Generate the audio transcription**, the subtitles shown during playback. It too depends only on encoding, not on the thumbnails, so it runs in parallel with thumbnail generation.

The structure is therefore a hierarchy, not a single line: encoding is the common parent, after which the thumbnail branch and the transcription branch proceed concurrently.

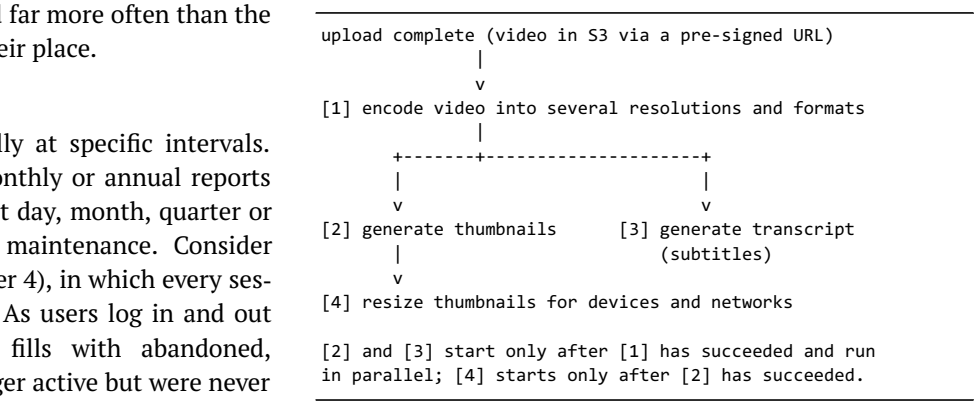


Figure 4: Chain tasks after a video upload to an LMS; each arrow is a parent–child dependency (illustrative).

5.4 Batch tasks

A *batch task* triggers many tasks from a single one. The first example is account deletion, which many SaaS platforms offer as a button. On a large platform a user may have a great deal of data, spread across database shards or regions; deleting all of it inside one request is often impossible, and even when possible it might take 40 to 50 seconds or, in the worst case, more than a minute — far too long to block an API call. So when the delete-account call arrives, the backend creates a delete-account task, pushes it into the queue and immediately returns a success response such as 200, saying that deletion is in progress; the front end logs the user out. Large platforms add a grace period: the user has, say, 3 or 7 days to log back in and cancel, after which the account is deleted permanently. A smaller platform without a grace period treats the account as deleted from the user's point of view at that moment, while the actual deletion runs in the background.

The consumer responsible for the task then works through the user's resources. In a project-management application: every project the user owns, removing the user's entities from it; the user's profile; assets such as logos and cover images; then the account itself; and finally an email confirming the deletion. The delete-account task does not do all of this itself but triggers many tasks — for the user's entities, for the assets and so on — which is what makes it a batch. The second example is scheduled: at midnight or at the end of the week, a large platform triggers thousands of identical tasks at once, each generating and sending one user's report (Figure 5). For an operation accepted but not yet carried out, HTTP also defines the more precise status `202 Accepted` (see Chapter 1).

```
(a) one request, one task that fans out
DELETE /account -> enqueue delete_account(user=4812)
                <- 200 "deletion in progress"; log out
worker runs delete_account(4812), which enqueues:
  +-- remove_user_entities(project=p1)
  +-- remove_user_entities(project=p2)  ... per project
  +-- delete_profile(4812)
  +-- delete_asset("logo.png")
  +-- delete_asset("cover.jpg")         ... per asset
last: delete the account row, email "account deleted"

(b) one schedule tick, thousands of identical tasks
Sunday 00:00  scheduler -> weekly_report(user=1)
                        -> weekly_report(user=2)
                        -> ... one task per user
```

Figure 5: The lecturer's two batch tasks: account deletion fanning out into many tasks, and a scheduled report run (illustrative).

Kind	Triggered by	Lecture's examples
One-off	an event in a request	verification, welcome, reset email; message notification
Recurring	a schedule (cron)	periodic reports; deleting orphaned sessions
Chain	a parent's success	encode -> thumbnails, transcript -> resize
Batch	one task or tick, many tasks	account deletion; reports for thousands

Table 3: The four kinds of background tasks.

6 DESIGN CONSIDERATIONS AT SCALE

6.1 Idempotency

On a large platform with thousands of users, tasks fail and are retried routinely, so each must be *idempotent*: designed so that it can be executed several times safely, without side effects, in case of failure. In a typical delete-account task the worker goes through all the user's resources and entities in the database and removes them, or sets references to null, as the requirements dictate. The lecturer's prescription is to do all of it in one transaction. If something fails part-way — a database error, or an external call that fetches some of the user's data — the work is rolled back manually. When the queue retries the task, it starts from scratch, at 0% completion, on data the failed attempt left untouched, and that is the sense in which the lecturer calls the task idempotent.

A subtle point. A transaction makes the database part of a task atomic; that is necessary but is not the same as idempotent. Idempotency means that running the task twice has the same effect as running it once, and the second run can follow a successful first run, not only a failed one (Section 4.7). A rollback cannot undo effects outside the database — an email already sent, a file deleted from object storage, a call to a payment API — and data spread over several shards or regions, as in the lecturer's own example, cannot normally be covered by one local transaction. Each step must instead be safe to repeat: check whether it has already been done, treat "already absent" as success, and pass an idempotency key to external APIs that accept one (Figure 6).

```
def delete_account(user_id):     # may run more than once
    user = users.get(user_id)
    if user is None:             # user row is deleted last,
        return                   # so absent = all done: ack
    with db.transaction():       # all or nothing in the DB
        projects.remove_member(user_id)  # 0 rows: fine
        profiles.delete(user_id)         # absent: fine
    storage.delete_prefix(f"users/{user_id}/")  # repeatable
    mail.send(user.email, "account_deleted",
              idempotency_key=f"deleted-{user_id}")
    users.delete(user_id)        # last step marks completion
```

Figure 6: A delete-account task written so that a retry at any point is harmless (illustrative).

6.2 Error handling

Error handling in a task system must be robust and extensive, more so than in request handlers, because everything happens in a different process: no user sees an error page, no client retries, and an edge case nobody handled simply disappears. No bottleneck or edge case may be missed; errors must be caught, logged, and turned into retries of the failed task.

6.3 Monitoring

At any moment you need a complete view of the task system: how many tasks are currently in the queue, how many succeeded, how many failed, and the main reasons for failure — an external service or an internal error. The usual technique is metrics: every time something happens, such as a task being created, completed or failed, the application records a metric. A system such as Prometheus collects the metrics and persists them to disk, with replication if needed, and Grafana displays them on dashboards. Metrics, instrumentation, logs, traces and the ELK stack are the subject of Chapter 14.

6.4 Scaling

The system must be designed so that processing capacity can grow later. If the user base spikes and doubles, the number of tasks doubles with it; the answer is to add consumers, scaling them horizontally by adding nodes to the consumer cluster, so that processing stays as responsive as before. This works only if nothing ties a task to a particular worker, which is one more reason for self-contained task payloads.

6.5 Ordering

If tasks must be executed in a particular order, make sure the library and broker support ordered delivery. A plain queue gives no such guarantee once several consumers work in parallel and failed tasks are retried later: a task can overtake an earlier one. Brokers that offer ordering usually do so per key — Amazon SQS FIFO queues, for example, order messages within a message group — at the price of processing each key's tasks one at a time.

6.6 Rate limiting

Tasks that interact with external services need rate limiting so that they do not overload those services. Providers charge per API call and enforce rate limits of their own, and a queue that suddenly releases thousands of tasks

against them is throttled, after which the retries add further load. The limit has to hold across all workers together: ten workers each limited to 10 calls per second still send 100.

7 BEST PRACTICES

7.1 Keep tasks small and focused

The lecturer's first recommendation from experience is that a task should be concerned with a single processing unit. Divide responsibilities among tasks so that the failure of one does not affect the others. Where one piece of processing depends on another, use a chain (Section 5.3): if the child fails, the parent has already succeeded and stays done, and only the child is retried, with exponential backoff or whatever policy is configured. If everything is packed into one task, a failure near the end means repeating the whole task, including the parts that succeeded, which wastes the CPU of the consumer. Small, focused tasks are easier to scale, easier to keep free of bugs and easier to monitor, and they make it easier for the queue to apply retries and similar mechanisms.

7.2 Avoid long-running tasks

A task that takes a long time to complete is a signal that it does too much. Break it into smaller, more manageable chunks: independent tasks that run concurrently, or a parent–child chain, depending on the dependencies. Long tasks also collide with the visibility timeout (Section 4.7): the longer a task runs, the more likely it is to be redelivered while still in progress, or cut off by a worker restart.

7.3 Use proper error handling and logging

The lecturer counts error handling and logging among the most important things in background processing, for two reasons. It gives the queue the chance to retry: the framework can only retry a task it sees fail, so a handler that catches an exception and returns normally is acknowledged as a success and never retried. And it gives you the information needed to debug why a task failed — an external service or an internal error — and to find where the problem lies.

7.4 Monitor queue length and worker health

Monitoring must lead to action, which means alerting. Alert when the queue length exceeds a set limit: the consumers are not keeping up, and the system must be made more scalable. Alert when workers go down: find the error that brings them down. With alerting in place you know at all times that the background task system is running smoothly, rather than learning otherwise from users whose emails never came.

8 CONCLUSION

- A background task is work the response does not depend on; moving it out of the request makes the API fast and independent of slow or failing services.
- The pattern: validate and store synchronously, serialize the remaining work into a task, enqueue it and return at once.
- A task queue has three parts: producers that enqueue, a broker (RabbitMQ, Redis, SQS) that stores, and consumers that dequeue, run a registered handler and acknowledge.
- Failures become retries with exponential backoff and a retry cap, instead of 500s or false "email sent" screens.
- The visibility timeout keeps tasks from being lost, which also means a task can run more than once.
- Tasks come in four kinds — one-off, recurring, chain and batch — chosen by trigger and dependencies.
- Idempotency requires every step to be safe to repeat; a rolled-back transaction covers only the database.
- At scale, plan for error handling, monitoring, horizontal scaling of consumers, ordering and rate limits.
- Keep tasks small, focused and short, let failures reach the queue, and alert on queue length and worker health.

APPENDIX A REVIEW QUESTIONS

1. A sign-up handler generates a verification code valid for 15 minutes and enqueues the email task; the worker retries with exponential backoff (first retry after 1 minute, doubling, at most five retries). The email provider is down from one minute before the sign-up until 12 minutes after it. Determine which attempt delivers the email and how long the code is still valid when it arrives, then propose two changes — to the payload, the retry policy or where the code is generated — that guarantee no user receives an expired code, and weigh what each costs.
2. A video platform's encoding worker reads from a broker with a 5-minute visibility timeout. Since instructors began uploading two-hour lectures, which take about 12 minutes to encode, some videos have three encoded copies in storage, worker CPU is saturated, and the thumbnail and transcript tasks run several times per video. Explain the mechanism step by step, predict how the situation develops as more long videos arrive, and compare at least two fixes.
3. A sign-up handler opens a database transaction, inserts the user, enqueues `send_verification_email(user_id)` and then commits; the worker loads the user by ID before rendering the email. Predict what happens (a) under light load, when a worker dequeues the task within 2 ms, (b) when the commit fails on a duplicate email address, and (c) after the order is changed to commit first, if the broker is unreachable at the moment of enqueueing. Propose a design that avoids all three outcomes.
4. Every Sunday at 00:00 a platform must generate and email a PDF report to each of 50,000 users. One report takes about 3 seconds of CPU, there are 20 workers, and the email provider allows 100 requests per second.

Compare (a) one recurring task that loops over all users with (b) a recurring task that enqueues one task per user, in terms of completion time, the cost of a crash at user 31,000, rate limiting and what monitoring can show; state which you would deploy and what you would add to it.

5. A system with stateful sessions (Chapter 4) keeps 40 million rows in its sessions table and runs a monthly cleanup task that deletes every session inactive for 90 days with a single `DELETE` statement. After the API was scaled to three replicas, each with its own scheduler, the job starts three times at 00:00 and login latency spikes for several minutes. Explain both symptoms and redesign the job so that it is safe to run concurrently or repeatedly and gentle on the database.
6. Design the background processing behind "delete my account" for a platform whose user data lives in three database shards and whose uploaded files sit in object storage, with a 7-day grace period in which the user can log back in and cancel. Specify the tasks and their kinds, what the API returns, which steps can share a transaction and which cannot, and how a worker crash halfway through the shard deletions is recovered without corrupting data or emailing the user twice.
7. An e-commerce backend enqueues a push-notification task for each order status change (paid, shipped, delivered) into one queue served by eight workers. Customers occasionally receive "delivered" before "shipped", and after a provider outage some receive "paid" last. Explain how each violation arises even though the broker hands out tasks first-in, first-out, and compare two designs that guarantee per-order ordering, including their effect on throughput.
8. On Black Friday the email queue grows from about 50 tasks to 400,000 in two hours while worker CPU stays at 5% and the failure counter is flat; the provider's dashboard shows the account pinned at its limit of 50 requests per second. Explain why doubling the workers would not help, estimate how long the backlog takes to drain once traffic falls to 20 emails per second, and propose a queue design that keeps verification and password-reset emails fast during such a spike.

APPENDIX B GLOSSARY

Term	Meaning
Background task	Work run outside the request–response cycle by a separate process.
Task queue	System that manages and distributes background tasks from producers to consumers.
Producer	Application code that creates a task, serializes it and enqueues it.
Broker	The queue infrastructure (RabbitMQ, Redis, SQS) that stores tasks until a worker takes them.
Consumer (worker)	Separate process or thread that dequeues tasks and runs their handlers.
Enqueue / dequeue	Add a task to the queue / take a task out of it.
Serialization	Turning task data into a transferable format such as JSON; deserialization reverses it.
Handler	Function registered for a task type; receives the deserialized data and does the work.
Acknowledgement	Worker's signal that a task is finished, so the broker can remove it.
Visibility timeout	Period a taken task stays hidden; without an ack it becomes available again.
Exponential backoff	Retry policy whose wait doubles after each failure, up to a retry cap.
Idempotency	Running a task twice has the same effect as running it once.
One-off task	A single task triggered by one event, such as a verification email.
Recurring task	A task run on a schedule, such as reports or session cleanup.
Chain task	A child task that starts only after its parent has succeeded.
Batch task	One task or trigger that fans out into many tasks.

APPENDIX C FURTHER TOPICS

Dead-letter queues

A task that still fails after its last retry has to go somewhere. A dead-letter queue holds tasks that exceeded their retry limit or cannot be processed at all (poison messages), so they stop consuming worker time without being lost. Engineers inspect them, fix the cause and replay them; Amazon SQS does this with a redrive policy, RabbitMQ with dead-letter exchanges, and BullMQ keeps exhausted jobs in a failed set. Every queue with a retry cap needs such a destination and an alert on its size.

Jitter in retry backoff

When an outage makes thousands of tasks fail at the same moment, pure exponential backoff retries them all at the same instants, in waves that can knock the recovering service down again. Jitter randomises each delay, for example by drawing it uniformly between zero and the exponential value, which spreads the retries out. It belongs in every retry policy that targets a shared dependency, and is the subject of Brooker's AWS article on backoff.

Transactional outbox

A handler that writes to the database and then enqueues a task is writing to two systems without a shared transac-

tion, so a crash between the two leaves a user without an email or an email without a user. The outbox pattern writes the task as a row in an outbox table inside the same database transaction as the business change; a relay process then publishes outbox rows to the broker and marks them sent. Tasks are published if and only if the change committed, at least once, so consumers must still be idempotent.

Priorities and workload isolation

In a single shared queue, a burst of low-value work such as thousands of weekly reports delays urgent tasks such as password-reset emails. Separate queues with dedicated workers per class of work, or broker-level priorities (RabbitMQ priority queues, BullMQ job priorities), keep latency-sensitive tasks fast. The design must also prevent starvation, so that low-priority work still finishes eventually.

Graceful worker shutdown

Deployments and autoscaling stop workers constantly, and a worker killed mid-task leaves its task to the visibility timeout, adding delay and duplicate work. On a termination signal a worker should stop fetching new tasks, finish or release its in-flight tasks within a deadline, and only then exit (see Chapter 15). Celery's warm shutdown and BullMQ's worker close method implement this behaviour.

Workflows, sagas and orchestration engines

Long chains that span several services need durable state (which steps have finished), timers and compensating actions when a later step fails; the saga pattern pairs each step with such a compensation. Celery's canvas primitives (chain, group, chord) cover simple cases, while workflow engines such as Temporal or AWS Step Functions persist workflow state and manage retries and timeouts per step. They suit processes such as account deletion across shards and services.

Log-based brokers

Apache Kafka stores messages in an append-only, partitioned log; consumers track their position (offset), messages remain after being read, and several consumer groups can read the same stream independently or replay it. Ordering holds within a partition, but there is no per-message acknowledgement or visibility timeout, so one slow message blocks its partition. A log suits event streams consumed by many services; a task queue suits discrete jobs with individual retries.

Autoscaling workers on queue metrics

Instead of adding consumers by hand after an alert, the number of workers can follow the queue: KEDA on Kubernetes, or scaling rules on the SQS metrics ApproximateNumberOfMessagesVisible and ApproximateAgeOfOldestMessage. The age of the oldest message is often the better signal, because it measures the delay users actually experience. Scaling helps only when workers are the bottleneck, not when a downstream rate limit is.

Full-Text Search with Elasticsearch — One-Page Overview

Chapter 11 · every section and sub-section of the paper at a glance

1 Introduction: Why Search Needs Its Own Engine

A pattern query that serves 5,000 products in 50 ms takes 30 s on millions of rows, and it can neither rank results nor forgive typos.

- **1.1 The naive query** — `ILIKE '%laptop%'` on name or description: a case-insensitive substring match, adequate for a small catalogue.
- **1.2 Growth and new requirements** — Millions of products and 30 s latency, plus demands for relevance (a MacBook Pro before a laptop bag) and typo tolerance.

2 Why a Database Scan Cannot Search

A database doing text search behaves like a librarian who must open every book and cannot tell a central match from a passing mention.

- **2.1 The librarian analogy** — Every book on every shelf is checked in turn; hours or days for a library of a billion books.
- **2.2 No sense of relevance** — A book titled on the topic and a last-page mention come back in arbitrary order.
- **2.3 What ILIKE '%term%' does** — No usable B-tree index, a full scan with character-by-character matching, cost linear in the data, unspecified order.
- **2.4 The information explosion** — Google, Amazon and LinkedIn needed millisecond search; information retrieval research since the 1960s supplied the idea.

3 The Inverted Index

Record every word when a document is stored; answer a query by looking up its terms instead of reading the documents.

- **3.1 Flipping the problem** — Terms point to documents and positions; work moves from query time to write time.
- **3.2 A worked index** — `machine` and `learning` mapped to books and pages; a query reads two lists.
- **3.3 Lucene, Elasticsearch and others** — Elasticsearch is built on Apache Lucene; other tools, and PostgreSQL, also offer full-text search.

4 Relevance Scoring

The counts and positions stored in the index let the engine rank matches, by default with BM25.

- **4.1 Ranking the example** — A title match plus the highest frequency puts *Introduction to Machine Learning* first.
- **4.2 BM25 and its signals** — Term frequency, document frequency and field length score each JSON document; boosts come from the query.
- **4.3 Field boosting and the Query DSL** — Title above description above content as a rule, with weights set per query in a JSON query language.

5 Typo Tolerance, Type-Ahead and Choosing a Tool

Search engines forgive misspellings and power suggest-as-you-type boxes; the choice of tool follows the existing stack.

- **5.1 Typo tolerance and type-ahead** — Google corrects 'what is treading today'; in Elasticsearch, fuzzy matching is enabled per query.
- **5.2 PostgreSQL or Elasticsearch** — PostgreSQL full-text search inside a Postgres stack; Elasticsearch where it already runs, as in an ELK logging stack.
- **5.3 How much to learn** — Use Elasticsearch through its documentation and best practice; master databases.

6 Demo: PostgreSQL ILIKE versus Elasticsearch

On the same 50,000 reviews with equivalent criteria, Elasticsearch answered in 0.5–1 s and PostgreSQL in 3–7.5 s.

- **6.1 Setup** — A Next.js UI over Neon PostgreSQL and Elastic Cloud, both in a US-West region.
- **6.2 Loading 50,000 reviews** — Configuration from the environment, 1,000-row batches into Postgres, an index recreated with a text/keyword mapping and bulk-loaded.
- **6.3 The search endpoint** — Both queries are timed and each result is streamed to the UI as soon as it is ready.
- **6.4 Results** — Identical match counts, Elasticsearch several times faster, and caveats on what the timing measures.

Full-text search at a glance — A pattern scan reads every row and cannot rank; an inverted index built at write time turns search into a term lookup, and BM25 with query-time field boosts orders the matches by relevance. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Full-Text Search: Inverted Indexes, Relevance and Elasticsearch

Chapter 11 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — A substring query such as ILIKE '%laptop%' answers quickly on a small catalogue, but it scans every row, cannot rank and cannot forgive typos once the data grows to millions of records. Using the lecturer's librarian analogy, the chapter explains why scanning fails and how the inverted index, built when documents are stored, turns search into a lookup. It then describes how Elasticsearch, built on Apache Lucene, ranks matches with BM25 and query-time field boosting, and where typo tolerance and type-ahead fit in. It compares PostgreSQL full-text search with Elasticsearch and records the lecturer's guidance on how deeply to learn each. A demo over 50,000 reviews, timed against Neon PostgreSQL and Elastic Cloud, closes the chapter together with the caveats needed to read its numbers.

Keywords — full-text search, inverted index, Elasticsearch, Apache Lucene, BM25, relevance scoring, field boosting, fuzzy matching, PostgreSQL

1 INTRODUCTION: WHY SEARCH NEEDS ITS OWN ENGINE

The lecture opens in 2005. An engineer at a fast-growing e-commerce company must build an API that takes a user's search input and returns matching products from a catalogue of about 5,000 items. In a relational database the natural query is a pattern match on the product's name or description (Figure 1).

1.1 The naive query

`ILIKE` is PostgreSQL's case-insensitive `LIKE`; the `%` wildcards match any run of characters before and after the keyword, so a product qualifies when `laptop` occurs anywhere in either column. At 5,000 products the query answers in about 50 ms, and customers searching for "laptop" get a handful of results.

```
-- 5,000 products: answers in about 50 ms
SELECT *
FROM   products
WHERE  name        ILIKE '%laptop%'
   OR  description ILIKE '%laptop%';
```

Figure 1: The lecture's product search, reconstructed from the narration: a case-insensitive substring match on two columns.

1.2 Growth and new requirements

The company grows quickly and the catalogue reaches millions of products. The unchanged query now takes about 30 seconds, and customers and management alike demand a fix. They also want more than speed:

- **Relevance.** A search for "laptop" should show a MacBook Pro first, not a laptop bag that merely contains the word.
- **Typo tolerance.** Customers in a hurry during a sale misspell the query, typing "lpatop" for "laptop", and should still get laptops.

Fast, relevant and robust to typos: these requirements are the reason search engines such as Elasticsearch exist.

2 WHY A DATABASE SCAN CANNOT SEARCH

2.1 The librarian analogy

The lecturer models the database as a librarian who knows exactly where every book is shelved. That suffices for a request by location, but not for a request by topic. Asked for books about machine learning, the librarian must take every book from every shelf and check it in turn: *Harry Potter and the Philosopher's Stone*, no match; *Game of Thrones*, no match; *Introduction to Machine Learning*, a match. The walk does not stop at the first hit, because further matching books may sit on later shelves. Its duration grows with the library: minutes for a small one, hours or even days for a collection of 10 million or a billion books.

2.2 No sense of relevance

The second flaw is independent of speed. Suppose the librarian finds two books: one titled *An Introduction to Machine Learning*, and one on another subject that mentions machine learning once, on its last page. Both match, and the librarian returns them in whatever order it likes, possibly the incidental mention first. A reader who asked for books about machine learning wants the first book at the top and may not want the second at all. Matching answers "does the text contain the words?"; relevance answers "how well does this item answer the request?", and the librarian has no notion of the latter.

2.3 What ILIKE '%term%' does

The query of Figure 1 is this librarian. Because the pattern may begin anywhere in the value, an ordinary B-tree index cannot narrow the search; a B-tree helps a `LIKE` only when the pattern starts with a fixed prefix (see Chapter 8 on indexes). The database therefore scans every row, reads every text field named in the condition and matches the pattern character by character. The method is thorough, since every row containing the substring is returned, but its cost grows linearly with the number of rows and the length of the text, which is how 50 ms becomes 30 s. Nor can it rank:

without an `ORDER BY` the rows arrive in an unspecified order, and the most useful product may sit at position 1,000 or 10,000 of the result.

2.4 The information explosion

In the same period the web grew explosively. Google was crawling and indexing billions of pages, Amazon was cataloguing millions of products and LinkedIn was indexing millions of profiles. None of them could make users wait 30 seconds: by current standards even a 2-second delay is extreme, search results are expected in milliseconds, and for companies earning money every second, latency translates directly into lost conversions and users. The answer came from decades of research rather than a new invention. *Information retrieval*, the study of finding relevant text quickly, had been an active field since the 1960s, and its key idea is to flip the problem around.

3 THE INVERTED INDEX

3.1 Flipping the problem

The librarian searches the books for the words. The inverted approach does the work once, when each book arrives, instead of on every request. On arrival, every word of the book is recorded in an index that maps the word to the books that contain it and to where in each book it occurs. A query then starts from its words and goes straight to the books: terms lead to documents, instead of documents being searched for terms, hence the name *inverted* index. Production implementations involve more engineering and mathematics, but the concept itself is simple.

> **Inverted index.** A mapping from each distinct term of a collection to its *postings list*: the documents containing the term, usually with the frequency and positions of each occurrence. Building it costs work at write time; in exchange, finding the documents for a term costs a lookup instead of a scan of the whole collection.

3.2 A worked index

Figure 2 reproduces the lecturer's index for the two terms of the query "machine learning". The page lists are deliberately short; in a real book titled *Introduction to Machine Learning* both words would appear on hundreds of pages. To answer the query the librarian looks up `machine` and `learning` and reads the two lists; no shelf is walked. The lookup touches only the entries of the query's terms, so its cost depends on how many documents contain those terms, not on the size of the whole library.

```
term        document                          pages
machine  -> Introduction to Machine Learning  1, 15, 23
            The Machine Age                   5, 89
            Coffee Machine Manual             1
learning -> Introduction to Machine Learning  1, 16, 24
            Learning to Cook                  2 places
            Deep Learning Fundamentals        3 places

query "machine learning": read the two lists above;
no book is opened.
```

Figure 2: The lecturer's inverted index for two terms; for the last two books only the number of occurrences was given.

3.3 Lucene, Elasticsearch and other engines

The inverted index is the technology that powers Elasticsearch. Elasticsearch is itself not a from-scratch invention: it is built on **Apache Lucene**, the library that implements the inverted index and its scoring, and adds a server with a JSON API and distribution across machines around it. Nor is Elasticsearch the only full-text search tool. Many others are also built on Lucene, and relational databases have added full-text search of their own, PostgreSQL among them.

4 RELEVANCE SCORING

With the index the librarian is fast. Because the index also records where and how often each term occurs, it brings a second, equally important advantage: matches can be ranked by relevance.

4.1 Ranking the example

Take the matches for `machine` in Figure 2. The term occurs on page 1 of *Introduction to Machine Learning*, in the title, and an occurrence in the title gives a significant boost to the relevance score. Frequency is a second booster: the term appears three times in that book. Title plus highest frequency puts it first. *The Machine Age* also carries the term in its title but uses it less often, so it ranks second; *Coffee Machine Manual* has it in the title but only once and ranks third (Table 1). The same reasoning orders the books for `learning`. The result is what the 2005 company wanted: not merely documents that match, but the most meaningful ones first.

Book	In title	Occurrences	Rank
Introduction to Machine Learning	yes	3	1
The Machine Age	yes	2	2
Coffee Machine Manual	yes	1	3

Table 1: Ranking the matches for `machine` in the lecturer's example.

4.2 BM25 and its signals

The scoring algorithm Elasticsearch uses by default is **BM25** ("Best Matching 25"). It scores each matching *document*, the unit Elasticsearch stores: a JSON document, much like a document in MongoDB; in the example, one book. The lecturer names four signals (Table 2). *Term frequency* is how often the term occurs within one document. *Document frequency* is how common the term is across all documents. *Document length* distinguishes a short document from a long, book-like one. *Field boosting* makes a match in one field count more than in another. The lec-

turer recommends treating the mathematics as optional (Section 5.3); the Elasticsearch documentation covers it for the curious.

Signal	Measures	Effect on score
Term frequency	Occurrences in the document	Higher, with diminishing returns
Document frequency	Documents containing the term	Rarer term weighs more
Field length	Length of the matching field	Shorter field scores higher
Field boost	Weight set by the query	Multiplies the field's score

Table 2: Signals behind a relevance score; the first three are BM25 itself, the boost is applied by the query.

A subtle point. The lecture lists field boosting as a BM25 parameter and leaves the direction of the other signals open. BM25 itself has three ingredients: term frequency with saturation, so the twentieth occurrence adds far less than the second; inverse document frequency, so rare terms count more and near-universal ones almost nothing; and length normalisation, tuned by two constants (k1 = 1.2 and b = 0.75 by default in Elasticsearch). Boosts are multipliers that the query applies to each field's BM25 score. The title advantage in Table 1 therefore has two sources: titles are short, and search queries usually boost them.

4.3 Field boosting and the Query DSL

Field boosting is the signal developers adjust most often. For a book with a title, a description and its content, a match in the title is usually more relevant than one in the description, which in turn outweighs one in the content. These weights are not fixed: each query defines its own boosting, and an application in which the body text matters most can boost `content` instead. Queries are written in the Elasticsearch **Query DSL**, a JSON-based query language with many query types and parameters for extracting different kinds of results; Figure 3 shows a boosted search across three fields.

```
GET /books/_search
{
  "query": {
    "multi_match": {
      "query":  "machine learning",
      "fields": ["title^3", "description^2", "content"]
    }
  }
}
```

Figure 3: Field boosting in the Query DSL: a title match weighs three times, a description match twice as much as a content match (illustrative).

5 TYPO TOLERANCE, TYPE-AHEAD AND CHOOSING A TOOL

5.1 Typo tolerance and type-ahead

The first use case beyond plain matching is typo tolerance, which the lecturer demonstrates on Google search. Which technology Google runs is not public (Elasticsearch, something proprietary built on Lucene, or neither), but it belongs to the same family, and the same experience can be built with Elasticsearch. Google's search box is a *type-ahead* interface: after "what is" has been typed, ranked suggestions are already shown, and shops such as Amazon offer the same pattern. Typing "what is treading today", with a deliberate typo, still yields results for "what is trending today": the engine recognises the most likely intended query. Forgiving misspellings and suggesting while the user types are the experiences that justify a full-text engine in a product.

A subtle point. An engine does not detect a typo "from the context" by itself. In Elasticsearch typo tolerance is requested explicitly: a `match` query with `fuzziness` accepts index terms within a small Levenshtein edit distance of the query term (with `AUTO`: none for terms of one or two characters, one edit for three to five, two for longer terms, a swap of adjacent letters counting as one edit), and suggesters return "did you mean" corrections. Web-scale correction such as Google's also learns from query logs.

```
GET /products/_search
{
  "query": {
    "match": {
      "name": { "query": "lpatop", "fuzziness": "AUTO" }
    }
  }
}
```

Figure 4: A fuzzy query that finds "laptop" for the input "lpatop", one transposition away (illustrative).

5.2 PostgreSQL full-text search or Elasticsearch

A backend that needs a search feature has two practical options. PostgreSQL, as a modern database, has full-text search built in, and in a Postgres-based stack it is the natural candidate. Elasticsearch, however, is used well beyond type-ahead and product search: it is the E of the **ELK stack** (Elasticsearch, Logstash and Kibana), a widely used combination for log management, where its speed at searching logs, deriving statistics and feeding visualisations is the point (see Chapter 14). If the company already runs Elasticsearch as part of an ELK stack, using it for full-text search as well makes more sense than building the feature on PostgreSQL.

5.3 How much to learn

The lecturer is explicit about depth. How the index is built and how the scoring works are worth understanding but rarely needed: unless one is writing a search library or an alternative to Elasticsearch, it is enough to recognise the use cases that call for full-text search and to follow documented best practice. Most search requirements can be met from the examples in the Elasticsearch documentation or from snippets an LLM produces; optimisation requires reading further, and the official documentation is the place to do it. Databases are the opposite case. Indexes, query optimisation and every other part of working with a database must be mastered, because in the lecturer's estimate

they are involved in about 99% of a backend codebase (see Chapter 8).

6 DEMO: POSTGRESQL ILIKE VERSUS ELASTICSEARCH

6.1 Setup

The demo runs the same search against both kinds of system. The application is a Next.js project, chosen only because LLMs generate such prototypes well; its UI sends one search term to a PostgreSQL database and to an Elasticsearch index and shows each result with its timing. To keep the comparison fair, the database is a Neon serverless PostgreSQL project and the index lives in Elastic Cloud, both in a US-West region, so that network distance favours neither side.

6.2 Loading 50,000 reviews

The PostgreSQL table `reviews` has three columns: `id`, `review` (text) and `sentiment` (positive or negative). The data is a CSV file of about 50,000 reviews, each with its text and sentiment label. A Node.js populate script loads it into both stores (Figures 5 and 6):

1. It reads the database URL, the Elasticsearch address and the Elasticsearch API key from environment variables and throws an error if any of them is missing (configuration is the subject of Chapter 13).
2. It creates a Neon client and an Elasticsearch client, and reads the CSV with `readFileSync`.
3. PostgreSQL: `CREATE TABLE IF NOT EXISTS` (a no-op here, since the migration had already run), a reset of the ID numbering to start fresh, and a filter that drops records lacking either the review or the sentiment. The valid records are inserted in batches of 1,000 with plain `INSERT INTO reviews (review, sentiment) VALUES …` statements, because Neon limits how much can be inserted at once.
4. Elasticsearch: if an index named `reviews` exists it is deleted, again to start fresh, and recreated with an explicit mapping: `review` of type `text`, matched word by word, and `sentiment` of type `keyword`, matched only exactly. After the same validity filter, all 50,000 records are sent in one bulk insert.

The script reports 50,000 documents inserted; the Elastic Cloud console shows 50,000 documents in the index, and `SELECT COUNT(*) FROM reviews` in Neon's SQL editor returns 50,000.

> **Text and keyword fields.** A `text` field is *analysed*: at index time its value is split into terms, normalised (typically lower-cased) and entered word by word into the inverted index, so queries can match and score individual words. A `keyword` field is indexed as one exact value, for exact filters, sorting and aggregations: `sentiment` holds exactly `positive` or `negative`.

```
-- PostgreSQL (Neon): created only if absent
CREATE TABLE IF NOT EXISTS reviews (
  id        SERIAL PRIMARY KEY,
  review    TEXT NOT NULL,
  sentiment TEXT NOT NULL   -- 'positive' or 'negative'
);

// Elasticsearch (Elastic Cloud): explicit field mapping
PUT /reviews
{
  "mappings": {
    "properties": {
      "review":    { "type": "text" },
      "sentiment": { "type": "keyword" }
    }
  }
}
```

Figure 5: The demo's table and index mapping (illustrative).

```
// populate.js (outline)
const { DATABASE_URL, ES_URL, ES_API_KEY } = process.env;
if (!DATABASE_URL || !ES_URL || !ES_API_KEY)
  throw new Error("missing environment variable");

const sql = neon(DATABASE_URL);
const es = new Client({ node: ES_URL,
                        auth: { apiKey: ES_API_KEY } });
const valid = parseCsv(readFileSync("reviews.csv", "utf8"))
  .filter(r => r.review && r.sentiment);

// PostgreSQL: start fresh, insert 1,000 rows per batch
await sql`TRUNCATE reviews RESTART IDENTITY`;
for (const batch of chunks(valid, 1000))
  await insertBatch(sql, batch); // INSERT INTO reviews ...

// Elasticsearch: drop, recreate with mapping, bulk-load
if (await es.indices.exists({ index: "reviews" }))
  await es.indices.delete({ index: "reviews" });
await es.indices.create({ index: "reviews", mappings });
await es.helpers.bulk({ datasource: valid,
  onDocument: () => ({ index: { _index: "reviews" } }) });
```

Figure 6: Outline of the populate script (illustrative; helper names invented).

6.3 The search endpoint

The UI is generic; the substance is the API route that the front end calls when the user submits a term (Figure 7). The route reads the search term from the JSON body and answers `400 Bad Request` if it is missing or invalid. It then opens a response *stream*. The two searches finish at different times, and waiting for both would hold the faster result back behind the slower one, so each result is written to the stream as soon as it is available and the front end reads the chunks as they arrive. The UI thus shows directly which side is faster and by how much.

Each search starts a timer before its query. The PostgreSQL side runs `SELECT id, review, sentiment FROM reviews WHERE review ILIKE '%term%'`, case-insensitive to cover more matches and with wildcards so that any characters may precede and follow the term. The Elasticsearch side searches the `reviews` index with a `query_string` query: the term is lower-cased to cover as many matches as possible, wrapped in wildcards for the same "anything before and after" behaviour, and given default fields to search. Both sides pass their result and elapsed time to a helper that writes them to the stream.

```
// app/api/search/route.ts (outline)
export async function POST(req: Request) {
  const { term } = await req.json().catch(() => ({}));
  if (typeof term !== "string" || !term.trim())
    return new Response("Bad Request", { status: 400 });

  const { readable, writable } = new TransformStream();
  const out = writable.getWriter();

  const pg = (async () => {
    const t0 = performance.now();
    const rows = await sql`SELECT id, review, sentiment
      FROM reviews WHERE review ILIKE ${"%" + term + "%"}`;
    await sendResult(out, "postgres", rows, t0);
  })();

  const el = (async () => {
    const t0 = performance.now();
    const res = await es.search({ index: "reviews",
      query: { query_string: {
        query: `*${term.toLowerCase()}*`,
        default_field: "review" } } });
    await sendResult(out, "elasticsearch", res.hits, t0);
  })();

  Promise.all([pg, el]).finally(() => out.close());
  return new Response(readable);
}
```

Figure 7: The search route: two timed queries whose results are streamed as each completes (illustrative).

6.4 Results

For "laptop", Elasticsearch answered in about one second and PostgreSQL in about three, closer to four. For "only", Elasticsearch returned in about 500 ms with roughly 8,000 matches, and PostgreSQL found the same number: the demo deliberately keeps the two criteria equivalent (lower case, arbitrary characters before and after) so that only speed is compared. On a second run Elasticsearch again answered in 500 ms while PostgreSQL was still working; its result arrived after about 7.5 seconds (Table 3). The match counts are identical, the time is many times larger, and the difference is the cost of the scan behind `ILIKE`. The lecturer's conclusion: wherever an application needs search or type-ahead, use full-text search, either PostgreSQL's or a tool such as Elasticsearch.

Query	Elasticsearch	PostgreSQL	Matches
laptop	about 1 s	about 3–4 s	not stated
only (run 1)	about 500 ms	not stated	about 8,000 each
only (run 2)	about 500 ms	about 7.5 s	about 8,000 each

Table 3: Timings reported in the demo over 50,000 reviews.

A subtle point. The demo times substring matching on both sides, once and over the network; it is not full-text search against a database. Unless the code sets `size`, Elasticsearch returns only the 10 best hits plus a total count, while the SQL query ships all of the roughly 8,000 matching reviews, so part of the gap is data volume. The `*term*` pattern is not idiomatic either: a leading wildcard makes Elasticsearch walk its term dictionary (far smaller than the text, which is why it still wins, but its documentation warns that such queries are expensive), and wildcard hits receive a constant score, so BM25 plays no part. A `match` query reads the postings lists directly and ranks the hits. On the PostgreSQL side, a trigram GIN index (`pg_trgm`) can serve `ILIKE '%term%'`, and native full-text search with a GIN index is the like-for-like competitor.

7 CONCLUSION

- `ILIKE '%term%'` scans every row and matches character by character: correct, but its cost grows with the data (50 ms at 5,000 products, 30 s at millions).
- Pattern matching has no notion of relevance; a title match and a passing mention arrive in arbitrary order.
- An inverted index, built when documents are stored, maps each term to the documents and positions where it occurs and turns search into a lookup.
- Elasticsearch is a search server built on Apache Lucene; PostgreSQL also offers full-text search.
- BM25 ranks matches by term frequency, rarity of the term and field length; field boosts in the JSON Query DSL weight title, description and content.
- Typo tolerance and type-ahead are core search features; in Elasticsearch fuzziness is requested explicitly.
- Use PostgreSQL full-text search in a Postgres stack, Elasticsearch where it already runs, for example in an ELK logging stack.
- In the demo over 50,000 reviews, Elasticsearch answered in 0.5–1 s and unindexed `ILIKE` in 3–7.5 s for identical matches.
- Learn Elasticsearch to the level of documented best practice; master databases.

APPENDIX A REVIEW QUESTIONS

1. A classifieds site stores 20 million listings in PostgreSQL and receives about 500 new or edited listings per minute. Product wants search over title and description that ranks good matches first and tolerates one-letter typos; the company runs no Elasticsearch and has a three-person backend team. Decide between PostgreSQL's own search capabilities and introducing Elasticsearch, and justify the choice in terms of latency, relevance quality, operational cost and how soon an edited listing becomes searchable.
2. An index of books has the fields `title` and `content` with default BM25 scoring, and the query is a `multi_match` for "machine learning" over `["title^3", "content"]`. Book

A is titled "Machine Learning Basics" and its 300-word content uses each term twice; book B is titled "Kitchen Appliances" and its 80,000-word content uses "machine" 60 times and "learning" never; book C is titled "The Learning Machine" and its 300-word content uses neither term. Predict the ranking, name the factor that decides each pairwise comparison, and explain how the ranking could change if the boost were removed.

3. A type-ahead endpoint sends every keystroke to Elasticsearch as a `query_string` of the form `*input*`. After launch, p99 latency rises from 40 ms to 2 s, cluster CPU and heap climb at traffic peaks, the slowest requests are those with one- or two-letter inputs, and the suggestions appear in no meaningful order. Explain each symptom from how the index is organised and how such queries are scored, and redesign the endpoint.
4. In a product index, `name` is mapped as `keyword` and `status` as `text`. Users report that searching `macbook` in `name` returns nothing while the exact string `MacBook Pro 14` returns one hit, and that a match on `status` for "in stock" also returns products whose status is "out of stock". Explain both behaviours step by step from what is stored in the inverted index, and state the mapping and query type each field should use.
5. An index holds products named `laptop`, `tv` and `tablet` in a `text` field. For the inputs `lpatop`, `laptopp`, `lptp`, `tb` and `tablte`, each sent as a `match` query with `fuzziness: "AUTO"`, predict which product each input finds and explain the rule behind each case. Then argue whether `AUTO` is a sensible setting for a field of five-character product codes such as `AX310`.
6. An e-commerce backend commits every product change to PostgreSQL and must reflect it in an Elasticsearch index used for search. Compare (a) writing to Elasticsearch in the request handler right after the database commit with (b) enqueuing an indexing job on a task queue (see Chapter 10). For each, describe what customers see if Elasticsearch is unavailable for ten minutes, if the process crashes between the commit and the index write, and if two edits of one product are processed out of order, and specify how the job must be designed to be safe to retry.
7. A colleague reruns the lecture's demo on 2 million reviews and reports Elasticsearch at 0.4 s and PostgreSQL at 41 s, concluding that Elasticsearch is a hundred times faster and should serve all reads. Identify what the measurement actually compared, list at least three factors besides the search method that shaped the two numbers, design a benchmark that compares the approaches fairly, and assess whether the conclusion about serving all reads follows.
8. Your company runs a three-node Elasticsearch cluster as part of an ELK stack; it ingests 200 GB of logs per day with 14-day retention, and its indexing load peaks during incidents. A team proposes placing the 3-million-product search index on the same cluster, citing the lecture's advice to reuse an existing Elasticsearch. Evaluate the proposal: what the reuse saves, which failure modes would couple product search to logging, and under what conditions you would accept, modify or reject it.

APPENDIX B GLOSSARY

Term	Meaning
Full-text search	Search over natural-language text by its words, with ranking, rather than by exact values or substrings
Inverted index	Map from each term to the documents, and positions, that contain it
Postings list	The documents, frequencies and positions stored for one term
Apache Lucene	Java search library implementing inverted indexes and scoring; the core of Elasticsearch
Document	The unit Elasticsearch stores and returns: a JSON object
Index (Elasticsearch)	Named collection of documents sharing one mapping, such as `reviews`
Mapping	Schema of an index: each field's type and how it is indexed
text / keyword	Analysed field searchable word by word / single exact value for filters, sorting and aggregations
BM25	Default relevance function: term frequency, inverse document frequency and length normalisation
Term frequency	How often a term occurs in a document or field
Inverse document frequency	Weight that grows as fewer documents contain the term
Field boosting	Query-time multiplier that makes matches in one field count more
Query DSL	Elasticsearch's JSON-based query language
Fuzziness	Permitted edit distance between a query term and an index term
Type-ahead	Interface that suggests queries or results while the user types
ELK stack	Elasticsearch, Logstash and Kibana, used together for log management

APPENDIX C FURTHER TOPICS

Text analysis: analyzers, tokenizers and stemming

Before a `text` field enters the inverted index, an analyzer turns it into terms: character filters clean the input, a tokenizer splits it into tokens, and token filters lower-case them, remove stop words, reduce words to stems (so that "learning" and "learned" share "learn") or add synonyms. The same analyzer normally runs on the query, which is why "Learning" finds "learning". Analysis decides what can match at all, and a mismatch between index-time and query-time analysis is a common reason why an obviously present word is not found. Language-specific analyzers matter as soon as an application indexes more than plain English.

PostgreSQL full-text search

PostgreSQL turns text into a `tsvector`, a sorted list of normalised lexemes with positions, and queries into a `tsquery`, matched with the `@@` operator and ranked with `ts_rank` or

`ts_rank_cd`. A GIN index on the `tsvector`, often a stored generated column, is PostgreSQL's own inverted index and removes the full scan of the lecture's query. It supports language configurations, per-field weights (labels A to D, comparable to field boosting) and phrase search, and stays transactionally consistent with the data. It is the natural first choice for moderate requirements on data already in PostgreSQL, though its ranking is not BM25 and it offers less fuzzy matching and suggestion support than Elasticsearch.

Trigram indexes (pg_trgm)

The `pg_trgm` extension splits strings into three-character sequences and provides GIN and GiST operator classes that accelerate `LIKE` and `ILIKE` with leading wildcards, as well as similarity search with the `%` operator and `similarity()`. It is the smallest change that fixes a slow substring search without adding a second system, and it gives basic typo tolerance for short strings such as names and product titles. Its costs are index size and poor selectivity for patterns shorter than three characters.

Keeping the search index in sync

Elasticsearch is normally a derived store: the source of truth stays in the database, and every change must be propagated to the index. Dual writes from the request handler are simple but leave the stores inconsistent whenever one write fails; more robust designs publish changes through a transactional outbox or change data capture from the database log (for example with Debezium) and index them asynchronously with retries. Search then becomes eventually consistent, so the design must state how stale results may be and how the index is rebuilt from scratch, typically into a new index behind an alias that is switched when the rebuild completes.

Shards, replicas and near-real-time refresh

An Elasticsearch index is split into primary shards, each a Lucene index, distributed across the nodes of a cluster, with replica shards providing redundancy and extra read capacity. The number of primary shards is fixed when the index is created and changing it requires a split, shrink or reindex, so it must be sized for the expected data. New documents become searchable only after a refresh, by default every second, which is why Elasticsearch is called near real-time: a document searched immediately after indexing may not be found yet.

Autocomplete structures

A type-ahead box cannot afford a wildcard query per keystroke. Elasticsearch offers dedicated structures: the completion suggester stores suggestions in an in-memory finite-state transducer for fast prefix lookup, the `search_as_you_type` field type indexes prefixes and word shingles automatically, and custom edge n-gram analyzers index every prefix of each word at write time. Combined with client-side debouncing, these keep suggestions within tens of milliseconds while preserving relevance ranking.

Paginating search results

Elasticsearch returns the top 10 hits by default, and `from`/`size` paging is capped by `index.max_result_window` (10,000 by default), because every page deep in the result list forces each shard to collect and sort all preceding hits. Deep or exhaustive traversal uses `search_after` with a point-in-time view, which pages by the sort values of the last hit in the manner of keyset pagination in SQL. The reported total is exact only up to 10,000 hits unless `track_total_hits` is set. These limits shape how a search API exposes pages and counts.

Vector and hybrid search

BM25 matches words, not meaning: a query for "notebook computer" scores nothing on a document that only says "laptop" unless synonyms are configured. Semantic search embeds queries and documents as dense vectors with a machine-learning model and retrieves nearest neighbours through approximate k-NN structures such as HNSW graphs, available in Elasticsearch, OpenSearch and PostgreSQL's pgvector extension. Production systems increasingly combine both in hybrid search, for example with reciprocal rank fusion, because lexical scoring is precise for exact terms, codes and names while vectors capture paraphrase.

Error Handling & Fault Tolerance — One-Page Overview

Chapter 12 · every section and sub-section of the paper at a glance

1 Introduction: Errors as a Normal Condition

Failures are certain; a backend needs a fault-tolerant mindset to detect, contain and correct them.

- **1.1 Errors are inevitable** — Failing queries, timing-out APIs, bad user data and unforeseen edge cases.
- **1.2 A mindset, not a toolkit** — Know how bad things can get, what to watch for and how to prevent it.

2 A Taxonomy of Backend Errors

Five error types differ in visibility, in controllability and in where the defence belongs.

- **2.1 Logic errors** — Nothing crashes but results are wrong: a double discount yields negative shipping costs.
- **2.2 Database errors** — Connection and pool failures, unique and foreign-key violations, bad SQL, deadlocks.
- **2.3 External-service errors** — Network faults, provider auth errors, 429 with exponential backoff, outages.
- **2.4 Input validation errors** — Format, range and required-field rules at the entry point; 400 Bad Request.
- **2.5 Configuration errors** — A missing API key should stop start-up under blue-green, not cause a 500 later.

3 Proactive Error Detection

The best error handling starts before errors happen.

- **3.1 Catch errors before they spread** — Detect a failure the moment it occurs, before it does damage.
- **3.2 Health checks** — `/health` returns 200 while up; the status code matters, not the body.
- **3.3 Database checks** — Connectivity, query time (500 ms vs 4–5 s) and integrity via a representative query.
- **3.4 External-service checks** — Test transactions, test emails to internal addresses, test tokens.
- **3.5 Core-functionality checks** — Configuration loaded, caches populated, internal state consistent.

4 Monitoring and Observability

Detect errors quickly and capture enough context to debug them.

- **4.1 Errors and early warning signs** — Degrading performance precedes failure; cover every error source.
- **4.2 Performance and business metrics** — Latency, resources, throughput, plus successful transactions.
- **4.3 Structured logging** — JSON logs with metadata, aggregated in Loki and explored in Grafana.

5 Responding to and Containing Errors

The first reaction, the recovery path and the propagation path decide how far a failure spreads.

- **5.1 Recoverable and unrecoverable errors** — Retry with backoff without adding load; else contain and degrade.
- **5.2 Recovery strategies** — Automatic restarts, cache clean-up and failover; documented, tested runbooks.
- **5.3 Data recovery** — Data is the one irreplaceable asset: backups, restores, transaction-log replication.
- **5.4 Propagation control** — Wrap low-level errors with context and raise them to a level that can act.
- **5.5 Error boundaries** — Separate processes, timeouts and message queues stop failures crossing services.

6 Global Error Handling: The Final Safety Net

One middleware classifies every error and maps it to a status code and a message.

- **6.1 The layered request path** — Router, handler, service, repository; each repository method runs one query.
- **6.2 Two errors on one endpoint** — Creating a book: a 700-character name, then a duplicate name in PostgreSQL.
- **6.3 Propagating to one middleware** — Throw (JavaScript, Python) or return (Go) custom errors up to it.
- **6.4 Mapping errors to responses** — Validation and duplicates 400, no rows and missing references 404, else 500.
- **6.5 Why centralise** — No forgotten check turns into a 500, and no checks repeated in every repository.

7 Security in Error Handling

Error messages and logs must not help attackers.

- **7.1 Do not leak internals** — Never forward table or constraint names; the default branch says only 500.
- **7.2 Authentication messages** — "Invalid email or password" blocks step-by-step account enumeration.
- **7.3 Sensitive data in logs** — No emails, passwords, keys or card numbers; log user ID and correlation ID.

Error handling at a glance — classify the errors a backend meets, detect them before they spread, retry or contain them, route every error through one global handler, and never let messages or logs reveal what an attacker needs. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Error Handling and Fault-Tolerant Systems

Chapter 12 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Errors are a normal condition of backend systems, and handling them well is first a matter of mindset. The chapter classifies backend errors into logic, database, external-service, input-validation and configuration errors, with the lecture's examples and the defence that fits each. It then describes proactive detection through layered health checks and through monitoring of errors, performance and business metrics. Strategies for responding to errors follow: retries with exponential backoff, graceful degradation, automatic and manual recovery, data recovery, controlled propagation and error boundaries. A global error-handling middleware is developed on a book-management API as the final safety net that maps every error to a status code and a message. The chapter closes with the security side of error handling: leaked internals, user enumeration through login messages, and sensitive data in logs.

Keywords — fault tolerance, error taxonomy, health checks, exponential backoff, graceful degradation, error propagation, global error handler, user enumeration, structured logging

1 INTRODUCTION: ERRORS AS A NORMAL CONDITION

In backend development errors are not exceptional events to be eliminated once and for all; they are a normal part of building and running an application. The question is therefore not *whether* errors will happen but how the system behaves when they do: whether it is ready to detect them, to limit their effect and to correct them.

1.1 Errors are inevitable

The lecture opens with four certainties. Queries to the database will sometimes fail. Calls to external APIs will sometimes time out. Users will send erroneous data that breaks an API which does not expect it. Business logic will reach edge cases that nobody anticipated. Each of these becomes one of the error types of Section 2, and none of them can be ruled out by careful coding alone.

1.2 A mindset, not a toolkit

The lecture deliberately contains no frameworks, libraries or code snippets: its subject is a way of thinking. A backend engineer is responsible for executing the core business logic and for making sure that every transaction and every user action completes correctly. That responsibility requires a *fault-tolerant mindset*: being prepared for the worst, knowing how badly things can go wrong and what to watch for, and knowing the detection and prevention practices that startups, large companies and open-source projects typically apply.

> **Fault tolerance.** The ability of a system to keep delivering correct service, possibly in reduced form, when some of its components fail, instead of failing as a whole. In the lecture's sense it covers anticipating failures, detecting them early, limiting their spread and recovering from them.

2 A TAXONOMY OF BACKEND ERRORS

The lecture sorts the errors a backend engineer meets in daily work into five types. They differ in how visible they are, in how much control the team has over them, and in where the defence belongs.

2.1 Logic errors

Logic errors are, in the lecturer's experience, the most common type and the most dangerous, precisely because they do not crash the application. The code runs without exceptions; it simply produces wrong results. The lecturer's example is an e-commerce SaaS platform whose checkout accidentally applies a discount twice, so that customers end up with *negative shipping costs*. Nothing fails and no alert fires, but the platform loses money on every order through this single fault. Unless business outcomes are monitored, or users report the anomaly, such an error can stay unnoticed for weeks or months while the losses grow every week.

Three causes are typical:

- **Misinterpreted requirements.** A sprint usually begins with discussions with the client or the product manager. If a point in those conversations was ambiguous, the engineer may note and implement a requirement different from the intended one; with insufficient testing, the implementation goes straight to production and starts causing problems for users.
- **Incorrectly implemented algorithms.** A discount engine that issues different codes and runs different discount workflows depending on a user's behaviour and purchase history is complex. A slight miscalculation inside it produces losses across the whole platform.
- **Unconsidered edge cases.** A user action that nobody anticipated in a payment or discount flow sends the code down a path that was never reasoned about.

Where logic errors touch payments, money or security, they silently corrupt data and produce wrong business res-

ults over long periods. Because they raise no technical error, they are found by testing, by careful reading of requirements and by watching business metrics (Section 4.2), not by error monitors.

2.2 Database errors

Because almost every backend relies heavily on its database, database errors can bring the entire system down. They range from simple connectivity problems to deadlocks and transaction failures. The lecture distinguishes four kinds.

Connection errors occur when the application cannot communicate with the database at all. The symptoms are 500 responses from the backend and empty screens everywhere in the frontend. Causes include a network outage, an overloaded database server, and an exhausted *connection pool*. A connection pool keeps several TCP connections to the database server open so that a new request does not pay for a fresh TCP handshake and connection set-up each time; it is an optimisation, but when every pooled connection is busy, further requests wait or fail. Whatever the cause, the application effectively cannot work: the backend needs data to answer the frontend, and the frontend needs that answer to display anything (see Chapter 8).

Constraint violations are less obvious: the operation breaks a rule that the database enforces. Creating a user with an email address that already exists violates a *unique constraint*. With a `customers` table and an `orders` table whose non-nullable `customer_id` column is a foreign key to `customers`, inserting an order with a `customer_id` that does not exist in `customers` violates a *foreign-key constraint*, because the database cannot find the referenced row. If such an error is not handled and turned into a properly formatted message for the user or the frontend, it propagates to the main process and the frontend receives a generic 500.

The usual root cause of constraint violations is the validation layer: edge cases not handled in the frontend and the backend let invalid data reach the database, so the first remedy is a more robust validation layer. Some violations, however, cannot be prevented by validation. Only the database knows whether a new email is unique at the moment of insertion. For these cases the focus shifts to the format of the error: the backend must send a user-friendly message that the interface can display, such as "this email already exists, try another one".

Query errors come from malformed SQL. `SELECT * FROM customers` is a correct query; a typo in the table name instead produces an error stating that the table does not exist. A query that is too complex can also run past its time limit and fail.

Deadlocks arise when several database operations wait for each other in a circular dependency: each holds a lock that another needs, so none can proceed. They are particularly hard to reason about and must be expected in concurrent, transactional workloads. PostgreSQL, for example, detects such a cycle and aborts one of the transactions with an error, which the application must be prepared to handle, typically by retrying that transaction.

2.3 External-service errors

Modern SaaS applications depend on many external services: payment processors, email providers, cloud object storage such as S3, and hosted authentication providers such as Auth0 or Clerk. Each of them is a point of failure outside the team's control. Abandoning external dependencies is not a realistic option, since it would mean rebuilding from scratch complex systems into which other companies have invested years of work. The backend must therefore expect every one of these services to fail, and decide in advance what it will do, and how it will compensate its users, when that happens. The lecture lists four sources of failure.

The network. Whether the backend talks to a provider over HTTP, raw TCP or WebSockets, the medium is the network, and the network is not perfect. Connection timeouts, DNS resolution failures and network partitions, in which routing failures leave parts of the network unable to reach each other, must be expected and planned for.

Authentication errors. A provider such as Clerk or Auth0 rejects requests that carry wrong credentials (an incorrect username, email or password), expired tokens or insufficient permissions. The lecturer adds a warning: delegating authentication does not make a system secure on its own. The provider is an integration; the backend is still the team's own and can still create security problems, for example by exposing sensitive user information in its logs (Section 7.3; see Chapter 4).

Rate limiting. Almost every external API, whether an OpenAI key used for an AI feature or Resend used for email, enforces a rate limit. It stops its users, business customers included, from abusing the service and prevents malicious clients from sending an abnormal number of requests in a given period. If some user activity or a logic error makes the platform call the provider abnormally often, the provider's rate-limiting algorithm kicks in and answers with status **429 Too Many Requests**. The platform must expect this and have a strategy ready. The standard one is *exponential backoff*: on a 429 the client waits (one minute in the lecturer's example) and retries; after every further 429 it doubles the wait, from one minute to two, then four, and continues until the requests succeed again (Figure 1).

```
attempt 1 -> 429 Too Many Requests   wait 1 min
attempt 2 -> 429 Too Many Requests   wait 2 min
attempt 3 -> 429 Too Many Requests   wait 4 min
attempt 4 -> 429 Too Many Requests   wait 8 min
attempt 5 -> 200 OK                  continue

delay(n) = 1 min x 2^(n-1)
```

Figure 1: Exponential backoff after repeated 429 responses (delays from the lecture; layout illustrative)

A subtle point. Retrying "until the requests succeed" needs two limits in practice. The delay should be capped and the number of attempts bounded, after which the request fails or is set aside for later; and a random *jitter* should be added to each delay, otherwise many clients that hit the limit together also retry together and hit it again. When a 429 response carries a `Retry-After` header (RFC 6585), the provider's value should take precedence over the computed delay.

Service outages. The least avoidable failure is the provider being down. When a major cloud provider such as GCP, or one AWS service, has an incident, many of its customers go down with it and users flood social media with complaints. Outages also happen during planned maintenance. The backend must handle them gracefully through alternatives. If the Redis service goes down, a second layer, such as an in-memory cache or a second Redis node, keeps the application running, so that important functionality such as payments and order processing is not affected.

2.4 Input validation errors

Input validation errors are caused by the platform's own consumers: a user submits data that does not meet the system's requirements, and the validation layer rejects it. This layer is the first line of defence against incorrect data and malicious input, because it inspects requests at the entry point of the backend and throws an error as soon as something does not satisfy the rules (Chapter 5 treats it in depth). The usual rule types are:

- **Format validation**: does an email have a valid format, does a phone number look like a phone number, is a date valid? Even for custom data, the expected format must be defined exactly and only that format accepted, both for security and for fault tolerance.
- **Range validation**: numeric inputs within a minimum and a maximum, strings neither too short nor too long, an array with at least three and at most 100 elements.
- **Required-field validation**: every field an operation needs must be present.

A failed rule normally produces **400 Bad Request**. These are the easiest errors to predict and handle. External failures are outside the team's control and logic errors are hard to detect, but the requirements of the data are known in advance and simply enforced at the entry point. Every backend, whatever its type, should therefore have a robust validation layer.

2.5 Configuration errors

Configuration errors can prevent the application from starting or make production behave unexpectedly. They typically appear when code moves between the development, testing and production environments. The lecturer's scenario: during development an engineer adds an environment variable, an OpenAI API key, to the local `.env` file, opens a pull request that is merged, and forgets to add the variable to the production configuration, whether that is maintained by hand or loaded from a remote store such as AWS Parameter Store. The deployment succeeds, and two outcomes are possible (Figure 2).

Validated at start-up (best case). If the application checks, before the server starts, that every required configuration value is present, it refuses to start. Most deployments use a blue-green set-up, in which the previous version keeps running until the new one has started successfully, so the failed release never receives traffic and the application stays available.

Not validated (worst case). The server starts normally. Only when a user calls the endpoint that uses the new OpenAI service, for example to generate an image, does the controller try to read the missing key. The service throws, and the user receives a 500. The error surfaces at run time, in front of real users, which is the scenario to avoid.

The rule is to treat every value the application reads from the environment or from a remote store as required unless it is genuinely optional, to validate all of them as the first step before the server starts, and to fail immediately if any is missing or corrupted. Chapter 13 covers configuration management: what configuration a backend needs and where to store it.

```
deploy v2 (OPENAI_API_KEY missing in production)

(a) validated at start-up
    v2: load config -> key missing -> exit(1)
    blue-green: v1 keeps serving, v2 never gets traffic
    result: failed deploy, no user impact

(b) not validated
    v2: starts, health check 200, traffic switched
    user: POST /images -> read key fails -> 500
    result: failure at run time, in front of users
```

Figure 2: The same missing variable with and without start-up validation (illustrative)

Type	Lecture's example	Visibility	Main defence
Logic	Discount applied twice	Silent	Tests, business metrics
Database	Duplicate email, deadlock	Loud (500s)	Validation, error mapping
External service	429 from an email API	Loud	Backoff, fallbacks
Input validation	Missing required field	Expected	Validation layer, 400
Configuration	Missing API key	Start-up or run time	Validate before start

Table 1: The five error types of the lecture

3 PROACTIVE ERROR DETECTION

3.1 Catch errors before they spread

The lecturer names this the key point of the lecture: **the best error handling starts before errors happen**. Whatever the layer, frontend, backend or infrastructure, the strongest strategy is to detect an error at the moment it occurs, before it spreads and causes real damage. Taken together, the checks below form what the lecture calls *pro-*

active error detection: being prepared for worst-case scenarios, having methods in place to prevent them and, if they occur anyway, to correct them.

3.2 Health checks

Health checks monitor a system continuously. An HTTP server typically exposes an endpoint such as `/health` or `/status` that returns a generic body saying that everything is fine. What matters is the status code, not the body: 200 means the server is working, while a 4xx or 5xx code means something is wrong. External services and tools poll this endpoint repeatedly to confirm that the server is up and free of errors.

Such a check, however, only proves that the service is running. A service that runs but cannot do its job still answers 200. The checks must therefore also confirm that each component is actually doing its work.

3.3 Database checks

The first component that decides whether the backend can do its job is the database. A database health check tests **connectivity** (can the application connect at all?), **query performance** (if queries that used to take about 500 ms suddenly take 4 or 5 seconds, something is wrong) and **data integrity**. A simple ping that only shows the server is reachable is not enough: the check should run a representative query and examine its result, how long it took, and how long it takes on average.

3.4 External-service checks

External dependencies need checks of their own, which confirm both that the backend can connect to them and that they behave as they should:

- **Payment processors**: periodic test transactions. When a real user pays, the team knows that the last test, five minutes earlier, succeeded.
- **Email services**: test messages sent to internal addresses, confirming that the provider is reachable and that mail is actually delivered.
- **Authentication services**: test tokens generated and checked against the validation endpoints the provider exposes.

3.5 Core-functionality checks

The last group concerns the application's own core. At start-up all required configuration must be present; if it is not, the application stops there with a meaningful message, so that the infrastructure operator can set the missing variables and restart the service successfully (Section 2.5). Beyond configuration, the checks confirm that the caches needed to run production workloads are populated and that internal data structures, whether derived from configuration or from external services, are consistent.

Layer	What is checked	Lecture's example
Server	Process is up	`GET /health` returns 200
Database	Connectivity, speed, integrity	Representative query; 500 ms vs 4–5 s
Payments	Provider works end to end	Test transaction every few minutes
Email	Delivery	Test message to an internal address
Authentication	Token validation	Test token checked at the provider
Core	Config, caches, internal state	Required variables, populated caches

Table 2: Layers of proactive health checking

4 MONITORING AND OBSERVABILITY

4.1 Errors and early warning signs

Monitoring, logging and observability form a large subject of their own (Chapter 14), but they appear in every discussion of fault-tolerant systems. In this context the core idea of monitoring is to detect errors quickly, as they occur, and to provide enough context to debug them.

The lecturer's first tip, aimed at those already familiar with monitoring practice, is not to track error rates alone. One of the first signs that a system is about to break is a change in its performance: a service whose performance is degrading may be about to fail, so watching performance makes it possible to prevent some failures before they happen. Coverage matters as much as depth. Whatever monitoring service or methodology is used, the set-up should track errors from every part of the application, including HTTP errors, database errors, external-service failures and business-logic errors, and cover as many error sources as possible.

4.2 Performance and business metrics

Performance monitoring tracks response times, resource usage and throughput. On top of these the lecturer insists on **business metrics**. In an e-commerce backend, a sudden drop in the rate of successful transactions, accompanied by many failed ones, points to a technical problem even when every error rate looks normal. Business indicators such as completed transactions and successful authentications therefore belong next to the technical metrics; they are also the main way to notice the silent logic errors of Section 2.1.

4.3 Structured logging

Good logging provides the information needed to understand and debug an error after the fact. Logs should use a structured format such as JSON, which machines can parse easily and to which metadata can be attached. Log aggregators such as Grafana Loki ingest these JSON records, parse them, keep them on external storage and make them searchable, and dashboards in Grafana let engineers explore error rates visually. The choice of tools and their details belong to Chapter 14.

5 RESPONDING TO AND CONTAINING ERRORS

5.1 Recoverable and unrecoverable errors

The lecturer frames the reaction to errors as a few principles that have guided his own work. The first is the immediate response: when an error occurs, the first reaction decides whether it stays a minor problem or becomes a major failure, and the right reaction depends on the type of error and its context.

Recoverable errors are those where waiting and trying again is acceptable. The example is sending an email through an external service such as Resend: if the provider fails, nothing requires the email to leave within milliseconds, so a delay is affordable. For errors of this kind, which include network errors, temporarily unavailable resources and an exhausted database connection pool, retry mechanisms and exponential backoff work well. The caveat is that error handling must not add load to a system that is already under pressure. Retry logic, backoff and any handling of resource over-utilisation must be designed so that they do not deepen the overload they are reacting to.

> **Retrying side effects.** A retry is safe only if repeating the operation does no harm. When a call that charges a card or creates an order times out, the first attempt may have succeeded on the other side although its response was lost; a blind retry then executes it twice. Such operations must be idempotent, or be made idempotent, before a retry policy is applied to them.

Unrecoverable errors cannot be fixed by trying again. For them the strategy is *containment and graceful degradation*: switch to cached data, disable non-essential features, provide alternative functionality or a fallback, and limit how far the damage spreads. The system keeps serving its essential functions in reduced form instead of failing completely.

5.2 Recovery strategies

The second principle concerns recovery. The right strategy depends on the nature of the error and on how critical the affected functionality is. **Automatic recovery** handles many errors without human intervention. If a service has failed and stops answering requests, restarting it automatically, through a tool or a setting in the deployment workflow, almost always works. Clean-up functions such as clearing a corrupted cache, and failing over to backup systems, also work most of the time, but they must be designed carefully, because a badly designed automatic action can make the problem worse. The lecturer recommends a trial-and-error approach: push services to their error thresholds in advance and find out which recovery strategy works for which type of service.

Other failures need **manual recovery**, because they require human judgement and decisions. These procedures must be planned and documented, so that every team member, new hires included, knows the workflow for an incident or a service failure. They must also be tested, so that the team knows they work and can carry them out quickly in a stressful situation, when fast decisions determine what users experience.

5.3 Data recovery

Whatever the recovery path, the data must not be corrupted. Everything else in an application is code and running services that can be redeployed; the one tangible asset is the information it holds: user data, transactions, orders. Data integrity must therefore be a top priority. The data-recovery strategy includes backups taken at key moments together with the ability to restore them, and replication of transaction logs with specialised recovery tools. Replicating the transaction log (the write-ahead log in PostgreSQL) allows a database to be restored to a point just before a corruption, not only to the last backup.

5.4 Propagation control

The third principle is that not every error should be handled at the exact place where it occurs. The code that detects an error often lacks the context to decide what it means, and the error must be passed up to a higher level that has that context; the chain of calls it travels through is what a stack trace records. The propagation must be deliberate. Errors are passed upward to a chosen point, a primary or secondary handling process, along a flow the engineer controls; uncontrolled propagation can spread a failure to other services or bring down the main service.

In languages with exceptions, such as JavaScript or Python, errors are handled with `try/catch` blocks (`try/except` in Python), and exception hierarchies give propagation its structure. The usual pattern is to catch a low-level exception, wrap it with enough context and raise it as a higher-level exception that carries business meaning. At the higher level there is enough information to log the right data, to send a meaningful error to the frontend, or to trigger a recovery strategy. Carrying low-level errors up to a high-level handler with enough context is standard in almost every backend; its end point is the global error handler of Section 6.

5.5 Error boundaries

An *error boundary* is the point at which propagation stops. In a service architecture, boundaries keep an error in one service from affecting others. The means are separate processes, so that a crash stays inside one process; timeouts, so that a slow dependency cannot hold its callers indefinitely; and message queues, which decouple services instead of running them in the same process. With a broker such as RabbitMQ, two services communicate asynchronously: the producer places a message on the queue and moves on, and if the consumer fails, its messages wait in the queue instead of turning into failures of the producer (see Chapter 10).

6 GLOBAL ERROR HANDLING: THE FINAL SAFETY NET

The lecturer calls global error handling the *final safety net* and his favourite error-handling mechanism, one he gives close attention in virtually every backend he builds. It is largely a one-time effort: considerable time goes into it when the platform is first set up and its features, services and dependencies are being learnt, and afterwards it needs only small adjustments and additional conditions. The payoff is immediate rather than distant, which makes it, in his view, the most important error-handling strategy in a backend application.

6.1 The layered request path

The pattern builds on the common backend architecture of earlier lectures (see Chapter 6). The **routing layer** is the entry point and decides which handler serves a request. The **handler** (controller) extracts data from the payload, performs deserialisation, validation and binding, and then calls a **service** method. A service method is an orchestrator for one or more **repository** methods. Repository methods are the leaf nodes: each does one thing, usually executing a single database query, such as `getUserById`. A service that returns a user together with all of that user's posts calls two repository methods, one per query. Keeping repository methods at this unit level, limited to one type of database operation, lets each service decide which repository methods, and how many, it calls.

6.2 Two errors on one endpoint

The running example is a Goodreads-like book-management platform and its API endpoint for creating a book. The payload has two fields: `name`, which is required and is inserted into the database, and an optional `description`. Two different errors can arise on this single endpoint:

- **In the handler.** A business rule says that a book name cannot exceed 500 characters, and a user submits a name of about 700. Validation fails in the controller, which must return a meaningful error to the user.
- **In the repository.** A valid request passes the handler and the service and reaches the repository function `insertNewBook`, which executes an SQL `INSERT` against PostgreSQL. A book with this name already exists in the `books` table, so the database rejects the row with a *unique-constraint violation*.

6.3 Propagating to one middleware

Global error handling works the same way whatever the layer and whatever the type of error: every error is propagated upward to a **global error-handling middleware**. It lives in the middleware layer because there it has access to every incoming request and every outgoing response (Figure 3). In languages with exceptions, such as JavaScript or Python, the layers simply throw and the middleware, the final error handler, catches. In a language like Go, where errors are return values, each layer returns the error, from repository to service and from service to handler, until it reaches the middleware.

Every language allows custom error types, and the layers use them to label what went wrong: the repository raises database errors, the service raises internal errors and the handler raises validation errors. The middleware reads the error, determines its type, performs whatever that type requires and, for a user-facing backend, returns the corresponding HTTP error response.

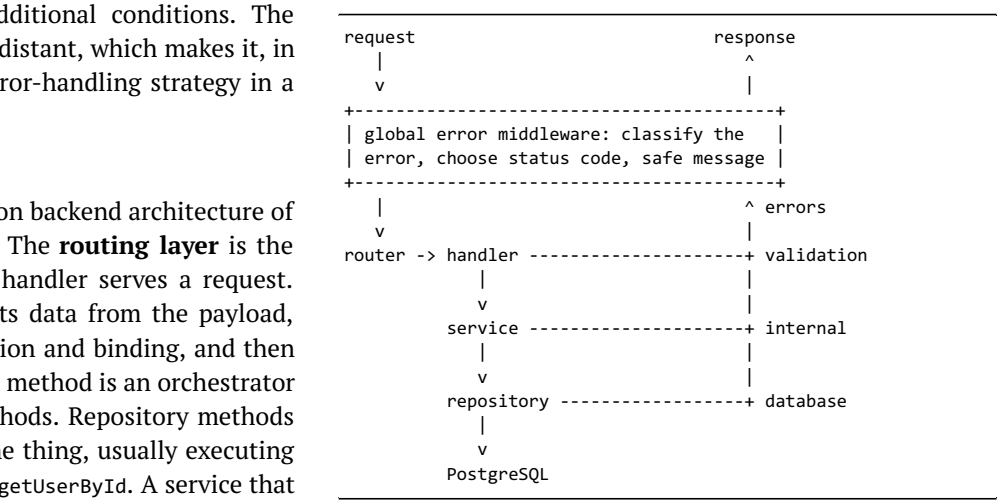


Figure 3: Errors from every layer travel up to one global error-handling middleware (illustrative)

6.4 Mapping errors to responses

Validation error. For the 700-character name, the middleware returns a response with status **400 Bad Request** and attaches all field-level and business-rule errors coming from the handler, so that the user can correct the data.

Unique-constraint violation. The duplicate book name arrives at the middleware as a database error. Database errors are not all treated alike. This one was caused by data the user submitted, so the appropriate answer is again 400: the payload is an incorrect request. An HTTP error body usually has a fixed structure: a `code` field with the HTTP status (400 here), a `message` field, and optionally field-level error data sent as an array of objects. The message can simply say that the book already exists (Figure 4). With PostgreSQL the classification is reliable, because every error carries a five-character SQLSTATE code: 23505 for a unique violation, 23503 for a foreign-key violation, 40P01 for a detected deadlock.

No rows. A second endpoint returns the details of a single book. The frontend page `/books/123` calls the backend route `GET /books/123` with the route parameter 123. The request has no body and the ID is a number, so it passes validation and the service and reaches the repository, which executes `SELECT * FROM books WHERE id = 123`. Suppose that, because of a malformed link or someone trying to trick a user, no book 123 exists. Relational database drivers commonly report this case as a "no rows" error, and the repository simply returns it upward. The middleware sees a database error that is not a unique violation but a no-rows error, on a route that requested a specific resource

by its ID, and deduces that the resource does not exist. HTTP has a dedicated code for a request to a resource that does not exist, **404 Not Found**, so the middleware answers 404 with the message that the book with ID 123 does not exist.

Foreign-key violation. A third endpoint creates a book for an existing author: the POST payload contains the name, the optional description and an `author_id`. The request passes validation, and the repository executes `INSERT INTO books (name, description, author_id) VALUES (...)`. The `author_id` column is a foreign key referring to a row of the `authors` table. The frontend has sent an ID for which no author exists, so the database rejects the query with a foreign-key violation. Since the client referred to a resource that does not exist, the lecturer answers 404 with a message that this author ID does not exist.

Anything else. An error that matches none of the known types falls through to the default branch and becomes **500 Internal Server Error**; Section 7.1 explains what that response may contain.

```
POST /books  {"name": "Dune"}      (name already taken)
HTTP/1.1 400 Bad Request
Content-Type: application/json

{
  "code": 400,
  "message": "Book already exists",
  "errors": [{"field": "name", "message": "already exists"}]
}

GET /books/123                     (no such book)
HTTP/1.1 404 Not Found
Content-Type: application/json

{"code": 404, "message": "Book with ID 123 does not exist"}
```

Figure 4: Error responses produced by the global handler (illustrative)

Error	Raised in	Status	Message
Name over 500 characters	Handler	400	Field errors to correct
Unique violation (name)	Repository	400	Book already exists
No rows for `GET /books/123`	Repository	404	Book 123 does not exist
Foreign-key violation	Repository	404	Author does not exist
Unrecognised error	Any layer	500	Something went wrong

Table 3: Error-to-response mapping in the global handler (the lecture's examples)

A subtle point. The "no rows" error belongs to single-row lookups: Go's `database/sql`, for instance, returns `sql.ErrNoRows` from `QueryRow(...).Scan`. A query that lists rows simply returns an empty result, which is not an error and normally yields 200 with an empty list. Likewise, 404 strictly says that the *target resource* of the URL was not found (RFC 9110); when the missing entity is only referenced in the body, as with `author_id`, many APIs answer 422 or 400 with a field error instead, and many answer a duplicate with 409 Conflict. Each choice is defensible if applied consistently.

6.5 Why centralise

Robustness. Because every error, from whichever layer, passes through one final gateway, no condition can be forgotten in some corner of the code. Without a global handler, the layer where an error occurs is responsible for producing the user-facing message: the repository would have to turn a unique violation into a 400 saying that the book already exists. If one repository method forgets that condition, the database error escapes unhandled. Since most backend set-ups convert any error that is not explicitly identified into a 500 Internal Server Error, meaning "something went wrong, try again later", the user receives that instead of the meaningful message. Keeping all mapping logic in one place closes this gap.

Less redundancy. Without a central point, the handling logic that fits in a single file or function must be spread over all layers. With the three database errors above, every repository method that executes SQL would have to check, directly or through a helper, whether the error is a unique violation, a foreign-key violation or a no-rows error, and handle each case. The repetition multiplies the code and makes it more error-prone.

7 SECURITY IN ERROR HANDLING

7.1 Do not leak internals

Error messages must be chosen with care, because details leaving the backend can compromise the security of the platform or expose users' information to the wrong people. Both are serious, and compromising users is arguably worse, because it damages the company's reputation and the market's trust in it.

The first place to look is the default error-handling path, in the global handler or wherever it lives. A unique-constraint error, like most database errors, contains internal details: table names, index names, constraint names. If the global middleware takes whatever error rises from the repository layer and copies its text into the `message` field of the response, those details reach the client. Advanced or malicious users can use them to plan more targeted attacks, such as more elaborate forms of SQL injection, and to devise ways around the platform's defences, which makes such attacks harder to prevent.

The rule is to control every message that leaves the backend: understand what kind of error occurred, whatever

layer it came from, and generate a message suitable for a user. In the default branch in particular, return 500 with a generic text, "something went wrong" or the "Internal Server Error" default of most libraries, and never attach the message of the underlying error. Reaching the default branch means that the error has already been checked against every known type (validation, uniqueness, business rules) and matched none of them. Its cause is unknown, its text most likely contains internal details, and the user has nothing to gain from it. The full error belongs in the server's logs, within the limits of Section 7.3.

7.2 Authentication messages

The second case is the authentication module. A login endpoint receives an email and a password in the body of a POST request, looks up the user by email and returns an error if none exists; if the user exists, it verifies the password and returns an error if the password is wrong. Authentication is one of the modules attackers target most, so it should follow established security practice. The lecturer points to the OWASP Cheat Sheet Series, which collects concrete advice for many workflows, authentication among them: what not to do and how each mistake compromises the platform or its users.

For error messages the main idea is this: when no user exists for the submitted email, the endpoint must not say "user does not exist", but return the same generic error used for a wrong password, such as "invalid email or password". Distinct messages enable a step-by-step attack on the accounts of legitimate users (Figure 5). First, the attacker tries many email addresses with a single password, and for every unknown address the response says that no user with this email exists. When an address finally belongs to a real account, the response changes to "incorrect password", and the attacker now knows a valid email. In the second step, the attacker tries common passwords, those with a high probability of being right, against that address. The error messages alone have given away half of the credentials.

```
naive messages
  ann@x.io / Summer1 -> 401 "user does not exist"
  bob@x.io / Summer1 -> 401 "user does not exist"
  eve@x.io / Summer1 -> 401 "incorrect password"
  => eve@x.io has an account: now try common passwords

generic message
  ann@x.io / Summer1 -> 401 "invalid email or password"
  eve@x.io / Summer1 -> 401 "invalid email or password"
  => real and unknown accounts look the same
```

Figure 5: Account enumeration through login error messages (illustrative)

A subtle point. Identical messages are necessary but not sufficient. If the endpoint answers at once for an unknown email but spends, say, 100 ms hashing the password for a known one, the response time reveals which accounts exist. The OWASP Authentication Cheat Sheet therefore also asks for consistent response times, for example by verifying the password against a dummy hash when the user is missing, and for the same care on registration and password-reset endpoints.

7.3 Sensitive data in logs

The second security concern is logging. Logs must not contain sensitive information: users' email addresses, passwords, API keys, credit-card numbers and the like. Teams often assume that logs stay on their own servers and infrastructure and can never reach an outside attacker, but that assumption does not hold. A company with considerable traffic produces hundreds of gigabytes of logs per day and has to send them to several external services, for storage, analytics and observability, to manage them efficiently. In many major data breaches it is precisely these logs that leak, and if sensitive fields were logged, they become available to every malicious party who obtains the dump.

When an error occurs, for instance an authentication failure, the log entry must not record sensitive information about the user. It should identify the user by ID rather than by email, and carry a correlation ID that ties together all entries of the same request, which gives enough context for debugging without personal data (Figure 6; see Chapter 16).

```
avoid
{"level": "error", "msg": "login failed",
 "email": "eve@x.io", "password": "Summer1"}

prefer
{"level": "error", "msg": "login failed",
 "reason": "invalid_credentials", "user_id": "u_8412",
 "correlation_id": "7f3c9a1e-52b4-4d0e-9b61-2c8e40f1a6d3"}
```

Figure 6: A structured error log with and without sensitive data (illustrative)

8 CONCLUSION

- Errors are a normal condition of backend work; the goal is to be ready to detect, contain and correct them.
- Logic errors are the most dangerous because nothing crashes; tests and business metrics catch them.
- Database, external-service, validation and configuration errors each need their own defence: robust validation, error mapping, backoff and fallbacks, start-up checks.
- Validate configuration before the server starts, so that a bad release fails while the old version still serves traffic.
- Health checks must prove that components do their job (representative queries, test transactions, test tokens), not merely that the process is alive.

- Monitor performance and business metrics as well as error rates; degradation often precedes failure.
- Retry recoverable errors with exponential backoff without overloading stressed systems; contain unrecoverable ones through graceful degradation.
- Propagate errors deliberately to one global error-handling middleware that maps each type to a status code and a safe message.
- Never expose internal details or the existence of accounts in error messages, and never write sensitive data into logs.

APPENDIX A REVIEW QUESTIONS

1. A nightly job sends 50,000 order-confirmation emails through a provider that allows 100 requests per second. It runs 20 workers, and each worker retries a failed request immediately, up to five times. Twenty minutes in, the provider starts answering 429 and the job's failure count explodes. Explain the sequence of events that produces this pattern, then design a retry policy for the job (delays, limits, and the fate of emails that still fail) and justify why it does not make the overload worse.
2. A release adds an image-generation endpoint that reads `OPENAI_API_KEY` the first time it is called. The key was added to the developer's `.env` file but not to the production parameter store. The platform deploys blue-green and switches traffic once `/health` returns 200. Describe what operators and users observe in the hours after the deployment, then describe how the outcome changes if the application validates its configuration before the server starts, and explain the role of the blue-green switch in each case.
3. A Go repository method for `GET /books/:id` returns the error from `QueryRow(...).Scan` unchanged. The global middleware has three branches: validation errors give 400 with field errors, errors with SQLSTATE 23505 give 400 "Book already exists", and everything else gives 500 with `err.Error()` in the `message` field. Predict the status and message for `GET /books/999` (no such book) and for a `POST /books` whose `author_id` refers to a missing author, state what each response reveals to a client, and change the middleware so that both cases follow the lecture's mapping.
4. An e-commerce backend shows a flat 5xx rate of 0.1 % and normal p95 latency, yet completed checkouts fell by 35 % within an hour of a release that changed how discount codes combine. No alert fired. Using the lecture's taxonomy, identify the most likely class of error and how it could have arisen, list the signals you would examine to confirm it, and redesign the monitoring so that the next incident of this kind raises an alert within minutes.
5. A product catalogue is cached in a single Redis node, which also stores shopping-cart sessions; PostgreSQL is sized for about 10 % of the read traffic. The Redis node becomes unreachable for 20 minutes. Describe what happens to browsing, carts and checkout if the backend has no fallback, then design a degradation plan that keeps payments and order processing working, justifying which features you disable and how you protect PostgreSQL (connect with Chapter 9).
6. A login endpoint returns 401 "user does not exist" for unknown emails and 401 "incorrect password" otherwise; unknown emails are answered in about 5 ms, known ones in about 120 ms because of password hashing. On failure it logs the email and the submitted password, and the logs are shipped to two external analytics services. Describe step by step how an attacker exploits each weakness, and specify the changes to responses, timing and logging that remove them (connect with Chapter 4).
7. An order service calls an invoice-PDF service synchronously over HTTP, without a timeout, inside the request that writes the order. During a traffic peak the PDF service starts answering after 60 seconds, the order service exhausts its workers and database connections, and order creation fails platform-wide. Explain how the failure propagated, classify which errors in the chain are recoverable, and redesign the interaction with error boundaries and controlled propagation so that a slow PDF service cannot stop orders (connect with Chapter 10).
8. A service's health check runs `SELECT 1` every 10 seconds. After a migration dropped an index, the main order query went from 500 ms to 5 s, the connection pool saturated at peak hours, and the health check stayed green throughout. Design a database health check and accompanying metrics that would have flagged this within minutes, discuss how often a representative query can run without loading the database itself, and choose the thresholds you would alert on.

APPENDIX B GLOSSARY

Term	Meaning
Fault tolerance	Continuing to serve, possibly in reduced form, when components fail
Logic error	Code runs without crashing but produces wrong results
Connection pool	Open database connections reused across requests to avoid set-up cost
Unique violation	A write would duplicate a value declared unique
Foreign-key violation	A row references a key that does not exist in the parent table
Deadlock	Transactions wait for each other's locks in a cycle; none can proceed
Rate limiting	A cap on requests per period; excess requests get 429 Too Many Requests
Exponential backoff	Retry policy that doubles the wait after each failed attempt
Graceful degradation	Keeping essential functions in reduced form instead of failing entirely
Health check	Endpoint or probe whose status code reports whether a component works
Structured logging	Logs as machine-parsable records, such as JSON, with metadata fields
Error propagation	Passing an error up the call chain to a level with enough context
Error boundary	Point where propagation stops so a failure cannot spread further
Global error handler	Middleware that maps every error to a status code and a safe message
User enumeration	Learning which accounts exist from differing responses
Correlation ID	Identifier that ties together all log entries of one request

APPENDIX C FURTHER TOPICS

Circuit breaker

A circuit breaker wraps calls to a dependency and counts failures; after a threshold it opens and fails calls immediately for a cool-down period, then lets a trial request through (half-open) to test whether the dependency has recovered. It stops a backend from spending threads, connections and time on a dependency known to be down, and gives that dependency room to recover instead of being hammered by retries. It applies to any synchronous call to an external or internal service and complements backoff and fallbacks. Michael Nygard's *Release It!* popularised the pattern.

Bulkheads

The bulkhead pattern partitions resources such as thread pools, connection pools, queues or whole instances per dependency or per workload, so that exhaustion in one partition cannot starve the others. A backend whose single HTTP client pool is consumed by a slow PDF service can no longer serve payments; with separate pools, only PDF generation suffers. It applies wherever one process calls several dependencies of different reliability, and it is a concrete way to build error boundaries inside a single service.

Timeouts and deadline propagation

Every outbound call needs an explicit timeout, and in a chain of services the remaining time budget of the original request should be passed downstream as a deadline, as gRPC deadlines or Go's context deadlines do. Without it, inner services keep working on requests whose clients have already given up, wasting capacity in exactly the overload that caused the delay. Timeouts should be chosen from measured latency percentiles rather than library defaults, which are often minutes long or infinite. This applies to every network call a backend makes.

Idempotency keys

An idempotency key is a unique value generated by the client and sent with a non-idempotent request such as a payment; the server stores the outcome under that key and returns the stored response if the same key arrives again. It makes retries after timeouts safe, because a repeated request cannot charge a card or create an order twice. Payment providers such as Stripe expose it as an `Idempotency-Key` header. It applies to every POST that a client or a retry policy may resend.

Liveness and readiness probes

Orchestrators such as Kubernetes distinguish a liveness probe, which decides whether a process should be restarted, from a readiness probe, which decides whether it should receive traffic now. Mixing them causes real incidents: a liveness probe that checks the database restarts every replica during a short database blip, turning a dependency problem into a full outage. Deep dependency checks therefore usually belong in readiness probes or in monitoring, while liveness stays shallow. The distinction applies to any containerised backend with automated restarts and load balancing.

Problem Details for HTTP APIs

RFC 9457 defines a standard JSON error body, media type `application/problem+json`, with the members `type`, `title`, `status`, `detail` and `instance`, extensible with custom fields such as a list of field errors. Adopting it gives clients one way to parse errors across all services and saves each team from inventing its own structure. Frameworks such as ASP.NET Core and Spring support it directly. It applies whenever an API designs the error responses of its global handler.

Dead-letter queues

A dead-letter queue receives messages that a consumer failed to process after a configured number of attempts, instead of retrying them forever or dropping them. It keeps a poison message from blocking a queue, preserves the failed work for inspection and manual replay, and provides a clear metric to alert on. RabbitMQ, Amazon SQS and most other brokers support it natively. It applies to every asynchronous workflow used as an error boundary.

Chaos engineering

Chaos engineering deliberately injects failures, such as killed instances, added latency or unavailable dependencies, into a system under controlled conditions, often in production, to verify that its fault-tolerance mechanisms work as designed. Netflix's Chaos Monkey made the practice widely known. It turns the advice to reach error thresholds in advance into a disciplined experiment with a hypothesis, a limited blast radius and a way to stop. It becomes practical once a system has the monitoring needed to observe the experiment's effect.

SLOs and error budgets

A service-level objective sets a target for a user-visible indicator, for example 99.9 % of checkout requests succeeding within 500 ms over 30 days; the error budget is the failure the target permits, 0.1 % here. Alerting on the rate at which the budget is consumed, rather than on raw error counts, focuses attention on failures users actually feel. Budgets also give teams a shared rule for when reliability work takes priority over features. The approach is described in Google's SRE books and suits any backend with well-defined user journeys.

Configuration Management — One-Page Overview

Chapter 13 · every section and sub-section of the paper at a glance

1 Introduction: Configuration as the DNA of a backend

Configuration decides how one codebase behaves in every environment; without a strategy it turns into configuration chaos.

- **1.1 More than passwords and keys** — Besides secrets, configuration governs start-up, connections, per-environment behaviour, logging, metrics and which features are on.
- **1.2 An e-commerce example** — Database details, Stripe keys, a new-checkout flag, pool size, session timeout, maximum order amount; items differ in sensitivity, change rate and environment.
- **1.3 Distributed systems and configuration chaos** — Every integration point needs settings; scattered hard-coded values bring inconsistency, leaks and irreproducible bugs, with high stakes on the backend.

2 Types of configuration

The type of a setting decides where it is stored, how it is protected and who may read it.

- **2.1 Application settings** — Log level (debug vs info), port 8080, pool size and timeouts; a 60 s timeout drops an 80 s image request with 504.
- **2.2 Database configuration** — Host, port, user, password and name form the connection URL; query timeouts belong here too.
- **2.3 External services** — API keys for Mailchimp or Resend, Stripe and Clerk, needed to initialise their clients.
- **2.4 Feature flags** — Enable a new checkout flow dynamically for one segment, e.g. US users but not Indian users, for A/B testing.
- **2.5 Infrastructure, security, performance and business rules** — DevOps settings, JWT and session secrets, `GOMAXPROCS`, centrally kept domain rules.

3 Sources of configuration

Values come from environment variables, files, key-value stores and secret managers, usually combined with a fixed precedence.

- **3.1 Environment variables** — `.env` plus dotenv locally; on deploy the pipeline fetches values from a secret manager and injects them.
- **3.2 Configuration files** — YAML preferred over JSON because it allows comments, TOML newer; Authelia and Apache Answer as examples.
- **3.3 Key-value stores** — Consul and etcd: lightweight, flat, central, used much like environment variables.
- **3.4 Secret managers** — Vault, Parameter Store, Key Vault: central configuration for distributed, autoscaled, multi-cloud deployments.
- **3.5 Hybrid strategies and precedence** — A loader merges Parameter Store, `config.yaml` and the environment by a priority decided in advance, per environment.

4 Configuration per environment

Each environment has its own priorities, so the same code runs with different values.

- **4.1 Priorities** — Development: productivity; test: automated validation; staging: production parity; production: reliability, security, performance.
- **4.2 Same code, different configuration** — Behaviour changes through configuration, never through edits to hard-coded values.
- **4.3 Example: database pool size** — 10 in development, 50 in production for traffic spikes, 2 in staging to save cloud cost.

5 Security and validation

Keep secrets out of code, in an encrypted store with least-privilege access and rotation, and validate everything at start-up.

- **5.1 Never hard-code secrets** — Database URLs and provider keys never belong in the codebase or its history.
- **5.2 Use a secret manager** — Managed services encrypt values at rest and in transit; over-engineering security is the right default.
- **5.3 Access control and rotation** — Front-end, back-end and DevOps each read only what they need; keys and JWT secrets are rotated periodically.
- **5.4 Validate configuration at start-up** — Check every value with Zod or go-playground/validator before serving: the lecture's most important lesson.

Configuration management at a glance — classify every setting by sensitivity, change rate and environment, keep it in the source that fits (environment, file, key-value store or secret manager) under a fixed precedence, restrict and rotate secrets, and validate the whole configuration before the first request is served. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Production-Grade Configuration Management

Chapter 13 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Configuration management is the systematic organisation, storage, access and maintenance of every setting that decides how a backend behaves, from secrets to log levels, feature flags and business rules. The chapter first classifies configuration into application settings, database details, external-service keys, feature flags, and infrastructure, security, performance and business-rule settings. It then compares the sources of configuration (environment variables with dotenv, JSON, YAML and TOML files, key-value stores such as Consul and etcd, and secret managers such as HashiCorp Vault) and shows how a hybrid loader combines them by precedence. Next it explains why development, test, staging and production need different values for the same code, using the database pool size as the example. It closes with security (no hard-coded secrets, managed encryption, least-privilege access, rotation) and with the lecturer's main lesson: validate all configuration at start-up.

Keywords — configuration management, environment variables, dotenv, YAML, feature flags, secret management, HashiCorp Vault, least privilege, configuration validation

1 INTRODUCTION: CONFIGURATION AS THE DNA OF A BACKEND

Configuration management is the systematic approach to organising, storing, accessing and maintaining all the settings of a backend application. The lecturer calls configuration the application's DNA: the same code runs in development, staging and production, and configuration decides how it behaves in each of them.

> **Configuration management.** The discipline of deciding which settings an application has, where each is stored, who may read or change it, how it reaches the running process and how it is checked, so that behaviour can change per environment without changing code.

1.1 More than passwords and keys

Mention configuration and most engineers think of secrets: the database password or secure connection URL, the secret that signs JWTs (see Chapter 4), the API keys of external services such as an email-delivery provider. Reducing configuration to these is, in the lecturer's analogy, like reducing a car to its engine: the engine matters most, but about 90% of the car is left out. Configuration also decides

- how the application starts up and how it connects to external services;
- how it behaves in each environment;
- whether it logs, what it logs and where the logs go, and where performance and business metrics are sent (see Chapter 14);
- which features are enabled in this deployment, which stay disabled, and for which users.

1.2 An e-commerce example

The backend of an e-commerce platform might hold the following configuration (Figure 1):

- **Database connection details**: host, port, username, password and timeouts.
- **Payment-processor API keys**, for example Stripe keys.
- **Feature flags**. A new checkout flow has been built; a flag decides whether the old or the new flow is served, and can enable the new flow for some users while others keep the old one.
- **Performance-tuning parameters**, such as the size of the database connection pool.
- **Security settings**, such as a session timeout of 30 or 60 seconds.
- **Business rules**, such as the maximum order amount per user.

```
# config.yaml of the shop backend
database:
  host: db.internal
  port: 5432
  user: shop
  password: ${DB_PASSWORD}       # secret, injected
  name: shop
  query_timeout: 5s
payments:
  stripe_api_key: ${STRIPE_API_KEY}            # secret
features:
  new_checkout:
    enabled: true
    countries: [US]              # India keeps the old flow
performance:
  db_pool_max: 50
security:
  session_timeout: 60s
business:
  max_order_amount: 5000
```

Figure 1: The lecturer's e-commerce configuration as one YAML file; secrets are placeholders filled in from the environment (illustrative).

These items differ along three axes, and the differences drive every later decision about storage and access (Table 1). **Sensitivity**: some values are secrets whose leak can cause serious damage and financial loss; others are not secret but control how the application behaves while it runs. **Change frequency**: some change often, others once a month or once a quarter. **Environment dependence**: some are identical in development, staging and production; others must differ per environment.

Item	Secret?	Changes	Per environment?
DB host, port, user	no	rarely	differs
DB password	yes	on rotation	differs
Stripe API key	yes	on rotation	test vs live key
New-checkout flag	no	often	may differ
Pool size	no	when tuned	differs
Session timeout	no	rarely	usually same
Max order amount	no	monthly or quarterly	same

Table 1: The e-commerce settings along the lecturer's three axes (illustrative classification).

1.3 Distributed systems and configuration chaos

Modern backends rarely run in isolation. They are parts of distributed systems made of several services, databases, caches such as Redis (see Chapter 9), message queues (see Chapter 10), third-party integrations, authentication providers and email services. Every integration point needs configuration, and not only to connect: the backend must also know how to handle failures of that dependency (see Chapter 12), how to tune its performance and how to stay secure, differently per environment and from values that may come from several sources (Section 3). This is why configuration management is a core part of a modern backend rather than an afterthought.

Configuration chaos. The state a system reaches without a dedicated strategy and pipeline for configuration: hard-coded values scattered through the codebase instead of one central point of control, inconsistent behaviour across environments, security holes from exposed secrets, and debugging that becomes a nightmare because nobody can reproduce an issue or say which setting caused a production break.

The stakes are higher on the backend than on the frontend, because the backend owns the core business logic and the data. A misconfigured frontend may show a feature or dialog it should not, or redirect to the wrong route. A misconfigured backend can expose customer data, process payments incorrectly or take the whole platform down. The same backend may also run on different cloud providers, on-premises servers, containers, serverless functions and edge functions, each with its own way of supplying configuration.

2 TYPES OF CONFIGURATION

Not all configuration is created equal. Its type determines the storage mechanism, the security measures and the access pattern: who may read a value and in which environments it should be available. Table 2 summarises the categories.

2.1 Application settings

The most common category, present in practically every backend: log level, port, connection-pool size and timeout values.

Timeouts bound how long the server waits for a request to complete. Suppose the timeout is 60 seconds and an endpoint generates an AI image and returns it to the frontend as a base64 string, which takes about 80 seconds on average. The work outlives the timeout, the request is dropped and the client receives status 504 (Gateway Timeout). The value must therefore be derived from the real duration of the slowest legitimate request, not left at a default.

A subtle point. By definition (RFC 9110), 504 is sent by a gateway or proxy, such as nginx or a cloud load balancer, whose upstream did not answer in time. If the application server's own timeout fires first, the client sees a closed connection or a framework-specific error (Go's `http.TimeoutHandler`, for instance, answers 503). The effective limit is the smallest timeout along the chain of client, proxy, application server and outbound HTTP client, so these values must be configured together; for work that regularly takes over a minute, a background job with polling (see Chapter 10) is usually the better design.

Log level. In development it is set to `debug`, so that developers see, and can tell apart, every log line while the application runs locally. In production it is raised to `info`, so that the valuable production logs are not cluttered (see Chapter 14).

Port. Locally the backend typically listens on 8080; in production the port depends on the setup, for example a VPS behind a reverse proxy or a Kubernetes deployment.

Connection-pool size. Most backends keep a pool of open database connections and reuse them instead of opening a connection per operation (see Chapter 8). The maximum pool size is a setting that must match the expected load (Section 4.3).

2.2 Database configuration

Everything the application needs to reach its database: host, port, username, password and database name. Combined, they form a connection URL such as `postgres://shop:password@db.internal:5432/shop`, which the running application uses to connect. Because the URL embeds the password, the whole URL is a secret. The category also covers limits such as the query timeout: how long a query may run before it is cancelled.

2.3 External services

Each third-party service brings its own configuration, usually an API key with which its client is initialised: an email provider such as Mailchimp or Resend, a payment processor such as Stripe, an authentication provider such as Clerk. The provider does not matter; each one needs its

key, and often further options, supplied before its client can be created.

2.4 Feature flags

Feature flags enable or disable features of the application dynamically. In the checkout example, the current flow shows the order details, the payment details and a "proceed to payment" button; the new flow calls a different API. The team does not want to release it to all users at once but to run an A/B test on a segment, for instance by location: enabled for users in the US, disabled for users in India. A flag is therefore often a rule evaluated per user rather than a plain boolean, and these rules need the same management as any other configuration.

2.5 Infrastructure, security, performance and business rules

- **Infrastructure configuration**: the DevOps settings of the platform the application runs on.
- **Security configuration**: the JWT secret, the session secret and anything else that affects the application's security.
- **Performance tuning**: runtime parameters beyond the pool size. A Go backend, for example, reads the environment variable `GOMAXPROCS`, which limits how many OS threads execute Go code at once (by default, the number of CPUs).
- **Business rules**: domain rules the application enforces, such as the maximum order amount, kept in central configuration so that they can change without a code change.

Type	Examples	Secret?
Application settings	log level, port, pool size, timeouts	no
Database	host, port, user, password, name, query timeout	password, URL
External services	Mailchimp/Resend, Stripe, Clerk keys	yes
Feature flags	new checkout for US users only	no
Infrastructure	DevOps and platform settings	partly
Security	JWT secret, session secret	yes
Performance tuning	pool size, `GOMAXPROCS`	no
Business rules	maximum order amount	no

Table 2: Types of configuration in a backend.

3 SOURCES OF CONFIGURATION

Where a value is stored depends on its security needs, on how fast it must be read and on the environment.

3.1 Environment variables

Environment variables are the most common store. They are a feature of the operating system, not something the application builds, and every language reads them: `process.env` in Node.js, `os.environ` in Python, `os.Getenv` in Go. Locally, values live in a `.env` file, and a library (*dotenv* is the best known) reads the file at start-up and loads its entries into the environment, which saves the developer from exporting each variable by hand (Figure 2). The convention is most common in Node.js but exists for every backend language.

```
# .env  (developer machine only; listed in .gitignore)
PORT=8080
LOG_LEVEL=debug
DATABASE_URL=postgres://shop:devpass@localhost:5432/shop
DB_POOL_MAX=10
STRIPE_API_KEY=sk_test_...

// server.js
require("dotenv").config();  // .env -> process.env
const port = process.env.PORT;  // "8080", a string

# the same by hand, in a shell
$ export PORT=8080 LOG_LEVEL=debug
$ node server.js
```

Figure 2: A local `.env` file loaded by dotenv, and the manual equivalent with `export` (illustrative).

A subtle point. dotenv does not write into the operating system's environment; it copies the file's pairs into the environment of the current process (`process.env`). Environment variables are per process: a process receives a copy from its parent when it starts and does not see later changes. By default dotenv also leaves already-set variables untouched, so a value injected by the platform wins over a stray `.env` file. Every value arrives as a string: `"false"` is a non-empty, truthy string and a missing number parses to `NaN`, one more reason for Section 5.4.

Outside the developer's machine, container platforms such as Docker and Kubernetes inject environment variables when they deploy the application. With a cloud provider the workflow is as in Figure 3: the deployment is triggered; at one stage the pipeline fetches the values from a secrets-management service (HashiCorp Vault, AWS Systems Manager Parameter Store, Azure Key Vault, Google Secret Manager) and loads them into the environment; the application starts, reads them and behaves accordingly.

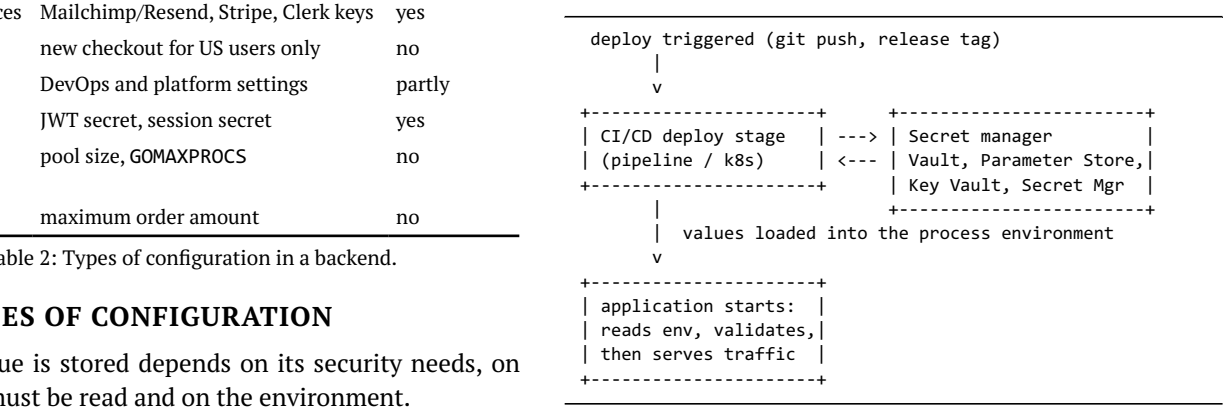


Figure 3: Deployment workflow with environment variables fetched from a secret manager (illustrative).

3.2 Configuration files

Files hold structured, hierarchical configuration. JSON works, but YAML is more popular for configuration because JSON has no comments (RFC 8259 defines none), whereas YAML lets the team record, next to a value, why it is set

that way, which shares knowledge within the team. TOML, a newer format, is also widely used (Figure 4).

```
// JSON: the standard allows no comments
{"server": {"port": 8080}, "log": {"level": "debug"}}

# YAML: comments carry the team's knowledge
server:
  port: 8080       # 80/443 are handled by the ingress
log:
  level: debug     # set to info in production

# TOML: newer, table-based
[server]
port = 8080

[log]
level = "debug"    # set to "info" in production
```

Figure 4: The same settings in JSON, YAML and TOML (illustrative).

Open-source codebases show the pattern. Authelia, an authentication and authorisation server written in Go, keeps its settings in a single `configuration.yml` with sections for the server, the log level (`debug` in the example shown), storage, notifications, identity providers, regulation and sessions; a search for "config" in the repository finds YAML configuration files in many places. Apache Answer (the `incubator-answer` repository), an open-source Q&A platform, ships fixed application settings in YAML: the port the server listens on, the database (SQLite for local environments) and the Swagger UI configuration.

3.3 Key-value stores

Dedicated, cloud-native key-value stores such as Consul and etcd keep configuration in a central service that the application queries over the network. Flat keys and values are lightweight and simpler to use than hierarchical formats; in use they behave much like environment variables, but one store serves every instance.

3.4 Secret managers

Dedicated cloud services, HashiCorp Vault, AWS Parameter Store and Azure Key Vault among them, specialise in configuration and secrets management. Most companies use them because deployments are distributed: Kubernetes, autoscaling, sometimes several cloud providers at the same time. One central tool that already supports all these environments, with dedicated integrations and documentation, makes integration as easy as possible. For an application serving heavy production traffic, an external provider is the sensible choice; its security properties follow in Section 5.2.

3.5 Hybrid strategies and precedence

In practice sources are combined. A configuration module loads values from several places at start-up and constructs the runtime application settings. The team decides the precedence beforehand, for example AWS Parameter Store first, then `config.yaml`, then environment variables, and decides per environment which sources are consulted at all (Figure 5). The rest of the code reads the resulting settings object, never the raw sources.

```
// config/load.ts
export async function loadSettings(env: string) {
  const defaults = { port: 8080, logLevel: "info" };
  const fromEnv = readEnv(process.env);        // lowest
  const fromFile = readYaml(`config.${env}.yaml`);
  const fromStore = env === "production"
    ? await readParameterStore("/shop/prod/")  // highest
    : {};
  // later wins: store > file > env > defaults
  const merged = { ...defaults, ...fromEnv,
                   ...fromFile, ...fromStore };
  return validate(merged);             // Section 5.4
}
```

Figure 5: A hybrid loader with the lecturer's precedence, Parameter Store over `config.yaml` over environment variables (illustrative).

The order itself is a design choice: many frameworks, ASP.NET Core and Spring Boot among them, let environment variables override files, so that an operator can patch a value at deploy time without editing a file.

Source	Examples	Typical use
Environment variables	`.env` + dotenv, container env	per-deployment values, injected secrets
Files	JSON, YAML, TOML	structured, commented settings
Key-value store	Consul, etcd	central values shared by instances
Secret manager	Vault, Parameter Store, Key Vault	secrets at scale, encrypted
Hybrid	loader with precedence	combining the above

Table 3: Sources of configuration.

4 CONFIGURATION PER ENVIRONMENT

4.1 Priorities

Configuration differs per environment because each environment has priorities that the others do not need or should not have.

- **Development** (the developer's machine, `localhost`): developer productivity and debugging, to work fast and catch issues early.
- **Test** (for example an environment that a GitHub Actions job prepares to run unit and integration tests): automated validation and quality assurance.
- **Staging**: a hosted version of the application that works as much like production as possible, so that the team knows how production will behave and can fix and debug issues before release.
- **Production**: reliability, security and performance.

4.2 Same code, different configuration

Configuration is therefore loaded differently, stored differently and set to different values per environment, while the application code stays exactly the same. Behaviour changes because the configuration changes, which is the goal. With hard-coded values every such change would mean editing the code; with centralised configuration the team changes a value and the behaviour follows.

4.3 Example: database pool size

In development a maximum pool size of 10 is enough: developer machines are powerful and nobody hits the limit while building and testing. In production the pool is 50, because the service serves a large user base and must absorb traffic spikes at different times. In staging the pool is 2.

Staging should mirror production as closely as possible, to catch issues early and understand the production system better. A staging environment that matched production exactly would, however, cost as much in cloud resources, which few teams accept, so minimising cloud cost becomes a further staging priority. Since staging is used only by developers and testers, some delay is acceptable and a minimal pool saves money. (The lecturer says "development" at this point but describes staging.) The price is that staging cannot reveal faults that appear only under concurrency, such as pool exhaustion; those need a load test with production-like settings.

Environment	Main priority	Pool max
Development	productivity, debugging	10
Test (CI)	automated validation, QA	—
Staging	production parity at low cost	2
Production	reliability, security, performance	50

Table 4: Environment priorities and the lecturer's pool sizes.

5 SECURITY AND VALIDATION

5.1 Never hard-code secrets

Secrets such as the production database URL or the API keys of Clerk, the email provider or the payment processor must never be hard-coded in the codebase. The lecturer treats this as obvious; the reason is that committed code is copied to every clone, fork, CI runner and backup, and the value stays in Git history after the line is deleted, so a committed secret must be treated as leaked and rotated.

5.2 Use a secret manager

Whenever possible, secrets belong in a cloud secrets-management service: HashiCorp Vault, AWS Parameter Store, Azure Key Vault or Google Secret Manager. For the security of a backend, the lecturer argues, over-engineering is the right default. These services encrypt every value at rest, whatever database or storage they use internally, and protect it in transit when a deployment fetches it through their API; the values are decoded only inside the team's own infrastructure, such as the Kubernetes environment, a GitHub Actions job or the repository's secret store. Both properties are essential, and a managed service provides them without further work.

A subtle point. The lecturer describes the service sending values encrypted and the client decoding them with a private key kept in its infrastructure. With these managed services the usual mechanics differ: in transit the protection is TLS, and the client authenticates with an identity (an IAM role, a Kubernetes service account, a Vault token or a CI secret) and receives the plaintext over that channel; at rest the service encrypts with keys it manages, for Parameter Store a KMS key. What lives in the infrastructure is thus a credential that grants read access, and it deserves the same protection as the secrets it unlocks. Client-side decryption with one's own key is the model of tools such as SOPS or Sealed Secrets.

5.3 Access control and rotation

In a large team, decide deliberately who may read which configuration, following the principle of least privilege (see Chapter 16):

- front-end developers: only what the frontend needs, such as the backend's API URL and the keys of front-end integrations;
- back-end engineers: the databases, the Redis instance, Elasticsearch and similar dependencies;
- the DevOps team alone: credentials for the cloud infrastructure itself, such as access to EC2 instances.

Secrets should also be **rotated** periodically (API keys, JWT secrets and similar values), so that a leaked copy soon becomes useless.

5.4 Validate configuration at start-up

In the lecturer's experience configuration is often not validated at all: dotenv loads the variables and the code reads `process.env.SOME_NAME` wherever it needs it. A missing mandatory variable is then hard to spot; production either breaks or behaves strangely, far from the actual cause. The remedy is to validate the whole configuration when it is loaded, which is at start-up after deployment and before the application serves anything, whatever the source: the environment, AWS Parameter Store or a `config.yaml` file. A proper validation library does the work: Zod for a TypeScript backend (Figure 6), `go-playground/validator` for Go, or whatever the language and framework offer (see Chapter 5). The schema states which values are mandatory, which are optional and which defaults the application code supplies. The lecturer, who learned this the hard way, calls it the single most important point of the lecture: always validate configuration, wherever it comes from.

```
// config.ts (TypeScript + Zod)
import { z } from "zod";

const Schema = z.object({
  NODE_ENV: z.enum(["development", "test",
                    "staging", "production"]),
  PORT: z.coerce.number().int().default(8080),
  LOG_LEVEL: z.enum(["debug", "info", "warn", "error"])
    .default("info"),
  DATABASE_URL: z.string().url(),            // mandatory
  DB_POOL_MAX: z.coerce.number().int().min(1).max(100),
  STRIPE_API_KEY: z.string().startsWith("sk_"),
  REQUEST_TIMEOUT_MS: z.coerce.number().default(60_000),
});

const parsed = Schema.safeParse(process.env);
if (!parsed.success) {
  // lists the failing keys and the reasons
  console.error(parsed.error.flatten().fieldErrors);
  process.exit(1);                 // fail before serving
}
export const config = parsed.data;  // typed and coerced
```

Figure 6: Start-up validation of environment configuration with Zod (illustrative).

In Figure 6, `z.coerce.number()` turns the string `"50"` into a number and rejects `"fifty"`; a missing `DATABASE_URL` or `DB_POOL_MAX` stops start-up with a message naming the key, while `PORT`, `LOG_LEVEL` and the timeout fall back to their defaults.

Fail fast. If validation fails, the process must report which keys are wrong (never the secret values) and exit with a non-zero status before accepting traffic. A crash at start-up is visible at once, and an orchestrator stalls the rollout while old instances keep serving; a missing key discovered at the first payment request is a production incident. Defaults belong only to values that are safe in every environment, never to connection strings or keys.

6 CONCLUSION

- Configuration is the application's DNA: it covers start-up, connections, logging, metrics, feature flags and business rules, not only secrets.
- Classify each setting by sensitivity, change frequency and environment dependence; the class decides storage and access.
- Without a strategy, configuration chaos follows: scattered hard-coded values, inconsistent environments, leaked secrets and irreproducible bugs.
- Environment variables are the universal interface; files add structure and comments; Consul, etcd and secret managers centralise values for distributed deployments.
- Combine sources in one loader with a precedence decided in advance, and let the code read only the resulting settings object.
- Same code, different values: development favours productivity, staging parity at low cost, production reliability, security and performance.
- Timeouts and pool sizes must follow real workloads: a 60 s timeout cannot serve an 80 s request.
- Keep secrets out of code, in an encrypted manager, readable on a least-privilege basis and rotated periodically.
- Validate every value at start-up with a schema library and fail before serving: the lecture's most important lesson.

APPENDIX A REVIEW QUESTIONS

1. An endpoint generates an AI image and returns it as base64; generation takes 80 s on average and up to 105 s. Requests pass a cloud load balancer with a 60 s idle timeout, then nginx with `proxy_read_timeout 90s`, then an application handler with a 120 s timeout, which calls the model provider through an HTTP client with a 100 s timeout. For a 70 s and a 105 s generation, state which component gives up first, what the client receives and what work continues uselessly on the server; then propose a consistent set of timeouts or a different design (see Chapter 10) and justify it.
2. Your loader merges sources with the precedence Parameter Store over `config.yaml` over environment variables. During an incident an engineer rotates the Stripe key and updates `STRIPE_API_KEY` in the Kubernetes Deployment; the pods restart, but production payments still fail with "invalid API key" while staging works. Explain the most likely cause from the precedence rule, list what you would check and in which order, and propose a change that makes each setting's effective source visible without ever logging the secret.
3. A Node.js service contains `const pool = parseInt(process.env.DB_POOL_MAX);`, `const sqlDebug = process.env.SQL_DEBUG ? true : false;` and `const port = process.env.PORT || 8080;`. The production manifest sets `SQL_DEBUG=false`, sets `PORT` to an empty string and misspells the pool variable as `DB_POOL_MAXX`. Predict the value of each constant and its visible effect in production, then give the schema rules (required, optional with default, coercion, enumeration) that would have turned each mistake into a start-up error.
4. The new checkout flow must be enabled for 10% of US users, stay disabled in India, and be switchable off within one minute if the payment error rate rises; the service runs on 40 autoscaled pods. Compare keeping the flag in an environment variable, in a `config.yaml` baked into the image, and in a central key-value store or flag service: for each, what turning the flag off requires, how long until all pods agree, and what inconsistency users can observe in the meantime.
5. Production runs 12 replicas with a database pool of 50 each against a PostgreSQL server whose `max_connections` is 700; during a sale the autoscaler grows the deployment to 16 replicas. Staging runs one replica with a pool of 2 and never showed a problem. Predict what happens at 16 replicas, explain why staging could not reveal it, and propose a per-environment pool-size

strategy that keeps staging cheap but still catches this class of fault (see Chapter 8).

6. A front-end contractor's laptop is stolen; it held a `.env` file, copied from a back-end engineer, containing the production `DATABASE_URL`, the JWT signing secret and an AWS access key. Order the response by urgency, explain what each leaked value lets an attacker do, and redesign who may read which configuration so that the same theft would have exposed only front-end settings. Include how to rotate the JWT secret without logging out every user at once (see Chapter 4).
7. A service treats `REDIS_URL` as optional with the default `redis://localhost:6379`. In a new production deployment the variable is misspelled `REDIS_ULR`; the service starts and passes its health checks, but p95 latency rises from 40 ms to 300 ms and database CPU doubles. Explain the chain of events from the misspelling to the symptoms and why nothing failed loudly, and state which validation and default policies per environment would have stopped the rollout; weigh failing at start-up against availability during a rolling deployment.
8. A company runs three services on Kubernetes in both AWS and GCP, with 25 engineers split into front-end, back-end and DevOps teams. It must manage database credentials, third-party API keys, per-environment pool sizes and log levels, a feature flag for a new checkout, and a maximum order amount that the business changes monthly. Assign each category to a storage source and a precedence order, state who may read it in each environment, and justify where you accept extra operational complexity.

APPENDIX B GLOSSARY

Term	Meaning
Configuration management	Systematic organisation, storage, access and maintenance of all settings of an application
Configuration chaos	Scattered hard-coded values, inconsistent environments, exposed secrets and irreproducible bugs
Application settings	Common runtime settings: log level, port, pool size, timeouts
Connection pool	Set of reusable open database connections with a configurable maximum size
Feature flag	Setting that enables a feature for all users, none, or a chosen segment
A/B test	Serving two variants to different user segments to compare them
Environment variable	Named string in a process's environment, copied from its parent at start
`.env` / dotenv	Local file of variables, and the library that loads it into the process environment
YAML / TOML	Human-readable configuration formats that, unlike JSON, allow comments
Key-value store	Networked central store of flat keys, e.g. Consul or etcd
Secret manager	Service storing secrets encrypted, with access control: Vault, Parameter Store, Key Vault
Precedence	Predefined order deciding which source wins when several define a key
Staging	Hosted, production-like environment for final checks, often scaled down
Least privilege	Each person or service may read only the configuration it needs
Secret rotation	Periodic replacement of keys and secrets to limit the value of a leak
Fail-fast validation	Checking all configuration at start-up and exiting before serving if it is invalid

APPENDIX C FURTHER TOPICS

Dynamic configuration and hot reload

Values read only at start-up need a restart to change, yet some settings, such as log level, feature flags and rate limits, are most useful when they can change while the process runs. Consul and etcd clients can watch keys, and Kubernetes refreshes ConfigMaps mounted as files (but not those injected as environment variables) after a short delay. The backend must then apply a change atomically and re-validate it, because a bad value now reaches every running instance at once. Hot reload suits operational knobs; connection strings and secrets are usually safer as restart-only settings.

Kubernetes ConfigMaps and Secrets

Kubernetes keeps non-sensitive configuration in ConfigMaps and sensitive values in Secrets, both injectable as environment variables or mounted as files. A Secret's data is only base64-encoded, not encrypted; confidentiality depends on enabling encryption at rest for etcd and on RBAC rules that limit who can read Secrets. Mounting secrets as files keeps them out of the environment that child processes inherit and that error reporters often dump. Operat-

ors such as External Secrets can sync values from Vault or a cloud secret manager into the cluster.

Encrypted configuration in Git

Teams that keep all configuration in Git and let a controller apply it (GitOps) still cannot commit plaintext secrets. SOPS encrypts only the values of a YAML or JSON file with a KMS, age or PGP key, and Sealed Secrets encrypts a Kubernetes Secret with a public key whose private half lives only in the cluster. Configuration changes then get review, history and rollback like code. The approach fits teams that want one reviewed source of truth for every environment.

Dynamic secrets and workload identity

Instead of long-lived static keys, Vault's database secrets engine can create a short-lived database user for each lease and revoke it when the lease ends. Cloud platforms let a workload authenticate with its own identity (IAM roles for pods or VMs, GCP Workload Identity, Azure managed identities), so that no access key is stored anywhere. A leaked credential is then useful for minutes rather than months, and rotation becomes automatic. It applies whenever the platform can vouch for the identity of the running service.

Secret scanning and leak response

A secret committed by mistake survives in Git history, forks and CI caches after the line is deleted. Scanners such as gitleaks and TruffleHog, and GitHub secret scanning with push protection, detect key patterns before or after a push. The correct response to a leak is to revoke and rotate first and clean history afterwards, since the value may already have been copied. Pre-commit hooks and CI scans are cheap and belong in every repository that handles credentials.

Feature-flag lifecycle

Pete Hodgson distinguishes release, experiment, ops and permission toggles, which differ in lifetime and in how dynamically they must change. Release flags such as the new-checkout switch should be removed once the rollout ends, because every flag doubles the paths to test and stale flags get reused by mistake; Knight Capital lost more than $400 million in under an hour in 2012 after an old flag was repurposed while one server still ran obsolete code. Flag services (LaunchDarkly, Unleash, providers for the OpenFeature API) add targeting, percentage rollouts, audit logs and kill switches. Each flag needs an owner and an expiry date.

Configuration change management

Configuration changes cause many production outages because they often bypass the review, testing and staged rollout that code receives. Treating configuration as code means versioning it, reviewing changes, validating them in CI with the same schema the service uses, rolling them out to one environment or a fraction of instances first, and keeping a one-step rollback. An audit trail of who changed which key and when turns the debugging nightmare of configuration chaos into a lookup. It applies to every shared setting, from timeouts to flag rules.

ASP.NET Core configuration and the Options pattern

ASP.NET Core builds one `IConfiguration` from layered providers: by default `appsettings.json`, `appsettings.{Environment}.json`, user secrets in Development, environment variables and command-line arguments, each later provider overriding the earlier ones. Hierarchical keys use a colon (`Database:PoolSize`), written with a double underscore in environment variables (`Database__PoolSize`). The Options pattern binds sections to typed classes, and data-annotation validation with `ValidateOnStart` makes the application fail at start-up instead of at first use, which is the practice of Section 5.4 built into the framework.

Logging, Monitoring & Observability — One-Page Overview

Chapter 14 · every section and sub-section of the paper at a glance

1 Introduction: Why a Backend Must Be Observable

A distributed backend cannot be inspected from one terminal, so three practices keep track of it, applied on a spectrum rather than as fixed rules.

- **1.1 Why distributed backends need them** — Many servers, regions and users; teams need tools that track every service and every piece of infrastructure.
- **1.2 A first look at the three practices** — Logs record events with metadata, monitoring tracks state, observability adds traces of each request's path.
- **1.3 From monitoring to observability** — Monitoring says that something is wrong; observability also shows what is wrong.

2 Three Practices, Three Questions

Logs tell what happened, metrics show patterns and trends, traces show how components interacted.

- **2.1 Logging** — A journal of important events with timestamp, user ID and debugging context.
- **2.2 Monitoring** — Health and performance over time, near real time with a 10–15 s delay.
- **2.3 Observability and the three pillars** — Internal state inferred from external outputs: logs, metrics and traces together.

3 From Alert to Root Cause

An alert on a metric leads to the logs behind it, and a log leads to the trace of the failing request.

- **3.1 Alerts** — An error-rate rule posts to Slack through a webhook.
- **3.2 Metrics** — Predefined numbers, current and historical: requests, failures, to-dos created.
- **3.3 From metrics to logs to traces** — One click from the failing metric to a log line to the failing function.
- **3.4 The tool landscape** — Prometheus, Grafana, Loki and Jaeger versus New Relic or Datadog.

4 Logging in Practice

Levels decide which events are written; the format decides who can read them.

- **4.1 Log levels** — Debug, info, warn, error and fatal; production runs at info.
- **4.2 Unstructured and structured logs** — Readable console text in development, JSON for log parsers in production.

5 Instrumenting a Go To-Do API

A real service shows each practice in code, using New Relic as an all-in-one tool.

- **5.1 The application and the tool** — A Go to-do back end; New Relic chosen for simplicity.
- **5.2 Logger set-up** — `getLogLevel` and the log format follow the environment.
- **5.3 Monitoring middleware and instrumentation** — A middleware wraps the app and measures every request.
- **5.4 OpenTelemetry** — An open, language-independent standard and SDKs for instrumentation.
- **5.5 The tracing middleware** — One transaction per request, enriched with attributes and carried in the context.
- **5.6 Instrumenting a service method** — `createTodo` adds attributes, records errors and logs at three levels.

6 Reading the Dashboard

Unauthorized requests are followed from the error metrics to a log record, its trace and the runtime view.

- **6.1 Generating traffic** — Requests without a token produce 401 errors.
- **6.2 Errors and metrics** — Error list, average transaction time, throughput, ~80% error rate.
- **6.3 A log record** — Application, environment, host, level, route and span ID.
- **6.4 Traces, transactions and the runtime** — Per-route transactions and Go runtime data such as GC time and memory.
- **6.5 A spectrum and a shared responsibility** — Developers instrument the code; DevOps engineers collect the data.

Observability at a glance — an alert on a metric leads to the logs behind it and a log leads to the trace of the failing request; log levels, JSON logs, a tracing middleware and instrumented services are what make that path exist. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Logging, Monitoring and Observability: Seeing Inside a Running Backend

Chapter 14 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Modern backends run on many servers and in many regions, so their behaviour can only be understood through deliberate practices: logging, monitoring and observability. The chapter defines the three and the pillars of observability (logs for what happened, metrics for patterns and trends, traces for how components interacted) and follows the debugging path from an alert through metrics and logs to a trace. It then covers log levels from debug to fatal and the choice between readable console logs in development and structured JSON logs in production. A Go to-do API instrumented with New Relic shows the practices in code: an environment-aware logger, monitoring and tracing middleware that put one transaction per request into the context, and a service method that adds attributes, records errors and writes business-event logs. Finally the New Relic dashboard is followed from error metrics to a log record, its trace and the runtime view, and the work is placed on a spectrum shared by developers and DevOps engineers.

Keywords — logging, log levels, structured logging, monitoring, metrics, distributed tracing, observability, instrumentation, OpenTelemetry

1 INTRODUCTION: WHY A BACKEND MUST BE OBSERVABLE

Logging, monitoring and observability are three related practices, each large enough for a lecture of its own. They are not a fixed set of rules. Every company, start-up and individual developer implements them somewhere on a spectrum, and no product or team can claim to follow every good practice; the long list of keywords, products and tools used across the industry is therefore no reason for intimidation. Because these practices only make sense in terms of how they are implemented, this lecture, unlike most of the course, looks at code: a to-do API written in Go and instrumented with New Relic (Sections 5 and 6).

1.1 Why distributed backends need them

Modern backend and full-stack applications run in distributed environments: on many servers, in several regions, for users spread across the world (see the course's introductory videos). When a request fails, no single terminal shows what happened. A team therefore needs practices, tools and methodologies that keep track of what is happening in every service and every piece of infrastructure. The same practices apply to front-end applications, but the lecture limits itself to the backend.

1.2 A first look at the three practices

Logging records the events that matter in the application (ordinary important events, suspicious events and security-related events), each with metadata: the ID of the user who triggered the request, the latency of the request, the method or function that handled it, and whatever else will be needed later to understand the system.

Monitoring keeps track of the state of the application and of its components: the server's CPU and memory, how many requests per second it is processing at the moment, and the state of the database connections, for example how many connections of the pool are open (see Chapter 8).

Observability is the umbrella. In its usual theoretical form it rests on three *pillars* (logs, metrics and traces), and a backend is called observable only when all three are in place. Logs are the records just described; metrics are closely related to monitoring. A *trace* can be imagined as a transaction: it records where a request originated (a front end, a load balancer or the backend itself) and every component it passed through while the backend executed it: handler, validation, service, repository and database layer (see Chapter 6).

1.3 From monitoring to observability

Observability as a movement is only a few years old. A decade ago, catching errors at the infrastructure level depended mostly on monitoring: dashboards and alert workflows, for example in Grafana, that report that the application has a problem. That is where monitoring stops: it informs the team that something is wrong, and nothing more. Observability also raises the alarm, but in addition shows exactly what is wrong, provided that logs, metrics and traces have all been implemented.

2 THREE PRACTICES, THREE QUESTIONS

2.1 Logging: the application's journal

Logging is the practice of recording all important events in the life cycle of the application: its start-up, every request it serves, its shutdown. Typical events are a user logging in, a database query being executed, or an operation failing. A failure is logged with a timestamp, the user ID, the database query and all other context needed to debug it. A log works like a journal or diary that the backend keeps, so that when the time comes the team can learn what happened, when it happened and why.

2.2 Monitoring: health and trends

Monitoring continuously checks the health and performance of the system, tracks patterns over time and presents aggregated data: how the application performed and behaved over a period, up to and including the present. "Real time" is meant loosely. Tools exist that report with a delay of milliseconds or a few seconds, but traditional setups collect data with a delay of roughly 10 to 15 seconds, because sending every value every millisecond or every second would overwhelm the logging, monitoring and observability systems themselves. Freshness is traded for load.

2.3 Observability and the three pillars

> **Observability.** A system is observable when its internal state can be determined by looking at its external outputs. The term comes from control theory; in a backend the outputs are its logs, metrics and traces.

The three practices work together because each produces one concrete kind of evidence (Table 1). Logs say *what exactly happened*. Metrics, the product of monitoring, show *patterns and trends*. Traces, the signal specific to observability, show *how components interacted* while serving a request. Each covers a gap of the others: a metric shows that the error rate rose but not why, a single log line lacks the path of the request, and one trace does not say whether its failure is typical.

Practice	Pillar	Answers	Demo example
Logging	Logs	What exactly happened?	401 on GET /todos, with user, host, route
Monitoring	Metrics	What are the patterns and trends?	error rate ~80%, throughput, mean time
Observability	Traces	How did components interact?	middleware, service, database path

Table 1: The three practices, their pillars and the question each answers.

3 FROM ALERT TO ROOT CAUSE

In a production system with proper logging, monitoring and observability, debugging follows a fixed path from a notification to the failing line of code (Figure 1).

3.1 Alerts

Alert rules are configured on metrics. In the lecturer's example, if the error rate goes above 80%, a webhook (see Chapter 26) posts a message to a Slack channel: something is wrong with the API service and someone should look into it. The alert is only the entry point; it carries no diagnosis.

3.2 Metrics

A metric is a numeric parameter of the system, available both as a current value and as history. Some are generic: how many requests have been processed and how many have failed. Others belong to the domain: in a to-do application, how many to-dos were created and how many creations failed. Metrics are read over windows (the last 30 minutes, the last hour, or longer history) and are decided in advance, when the system is configured: in the code, which counts and measurements are recorded, and in the logging, monitoring and observability tool, which of them are collected and displayed. After the alert, the engineer opens the metrics and confirms the picture: the error rate is indeed above 80%.

> **A subtle point.** The lecture counts as failed any request "whose status code is more than 200". That is too broad: 201 Created, 204 No Content and redirects are successes. Error-rate metrics normally count 5xx responses as the service's own failures; 4xx responses, such as the 401s of the demo, are client errors, and whether they enter the alerting error rate is a deliberate choice. The 80% threshold is only an example; production alerts usually fire at a few percent.

3.3 From metrics to logs to traces

Tools such as Grafana (open source) or the New Relic dashboard (proprietary) link the three pillars. From the error-rate metric the engineer jumps directly to the logs behind it, that is, all the failed requests. From a log line reporting a 500, one click opens its trace: the request started in this function, travelled to that function and then to a third, and failed at this particular point. The engineer learns exactly where things went wrong and can debug it at once, instead of searching through servers; this is the whole benefit of implementing the three practices.

```
ALERT    error rate > 80% on the API service
         -> webhook -> Slack channel
   |
   v
METRICS  requests, failures, latency, throughput
         now and over the last 30 min / 1 h
   |
   v
LOGS     the failed requests behind the metric
         level=error status=500 route=/todos ...
   |
   v
TRACE    middleware -> validation -> service
         -> repository -> database   (failed here)
```

Figure 1: The debugging path from an alert through metrics and logs to the trace of the failing request (illustrative).

3.4 The tool landscape

Two routes exist. The open-source route, used in most enterprises, is the "Grafana stack": Prometheus collects the data and builds the metrics, Grafana is the front end (dashboards and alerts) over all sources, Loki stores logs that Promtail ships to it, and Jaeger stores and displays traces (Table 2). The ELK stack (Elasticsearch, Logstash, Kibana) is a common alternative for log management. The proprietary route buys an all-in-one service such as New Relic or Datadog. Configuring and maintaining the open-source tools is complicated, so a team without the size, experience or time to run them gains more from a proprietary one-stop solution.

Need	Open-source tool	Role
Metrics	Prometheus	collects and stores metric time series
Dashboards, alerts	Grafana	front end over metrics, logs and traces
Logs	Loki + Promtail; ELK	ship, store and query logs
Traces	Jaeger	store and display traces
Instrumentation	OpenTelemetry	vendor-neutral SDKs and collector
All in one	(New Relic, Datadog)	proprietary, hosted

Table 2: The tool landscape named in the lecture. Grafana Labs has since deprecated Promtail in favour of Grafana Alloy.

4 LOGGING IN PRACTICE

4.1 Log levels

Each logged event is assigned a *level*, a feature every logging library should support. The common levels, from least to most severe, are debug, info, warn, error and fatal (Table 3). The logger is configured with a minimum level and discards every event below it; running production at info level is how debug logs are "disabled".

Debug is for development: while troubleshooting, the developer wants as much detail about the behaviour of the system as possible. Debug output is overwhelming, so it is usually disabled in production.

Info records general application operations: business events and successful operations. In a to-do application, the creation of a to-do is logged with `log.info`.

Warn sits between info and error: the operation did not succeed, but the situation is not critical enough to be an error. When a user's authentication fails because of a wrong password, the event is logged as a warning, since it is not the backend's fault.

Error covers failures: validation errors, failed database queries and every other kind of error. It is the most common level and one of the reasons logging exists at all.

Fatal marks a very serious issue: the application shuts down, after which the infrastructure may restart it, depending on its configuration. In Go, a fatal log from the standard `log` package or from zerolog ends with `os.Exit(1)`, so deferred clean-up does not run (see Chapter 15).

Level	Used for	Lecture's example	Production
debug	troubleshooting detail	"to-do created with ID …"	off
info	operations, business events	to-do created	on
warn	failed, but not our fault	wrong password	on
error	failures	database query failed	on
fatal	cannot continue	process shuts down	on

Table 3: Log levels from least to most severe.

A subtle point. The lecture logs validation errors at error level but a wrong password at warn because it "is not our fault". By the same rule a validation failure, where the client sent bad input (see Chapter 5), is a warn or info event. Reserving error for faults of the backend keeps error logs and error-based alerts meaningful.

4.2 Unstructured and structured logs

Logs are written in one of two ways. In development, when the backend runs locally, logs go to the console (the terminal or the editor's output panel) and should be easy for a person to read: plain text, attractively formatted, with colours. Readable logs make issues easy to spot and to fix.

Structured logging writes each event as data instead of prose, and JSON is the most popular format: one object per event, with fields for the level, the status, the message and every other parameter. Read by a person during development, JSON is tiring and errors are easily missed. In production, however, logs are not read line by line; depending on the configuration, they are parsed by a log-management system such as the ELK stack or the Grafana stack with Loki and Promtail. Free text is hard for these tools to parse: parsing fails often, and extracting values such as the user ID or the request ID from a line of text is inefficient. JSON is parsed reliably and every field becomes searchable. Hence the rule: unstructured, readable logs in development and structured JSON logs in production (Figure 2, Table 4).

```
LOG_FORMAT=console, ENV=local
10:42:07 INF connecting to database
10:42:07 INF started background job server
10:42:07 INF starting server addr=:8080

LOG_FORMAT=json, ENV=production (one line per event,
wrapped here)
{"level":"info","service":"todo-api","env":"production",
 "time":"2025-03-14T10:42:07Z",
 "message":"connecting to database"}
```

Figure 2: The same start-up events as console text and as structured JSON (illustrative).

Aspect	Console (unstructured)	JSON (structured)
Reader	a developer	log-management tools
Form	plain text, colours	one object per event
Used in	development	production
Strength	issues easy to spot	fields such as user ID extracted
Weakness	hard to parse reliably	hard to read, issues missed

Table 4: Unstructured and structured logging compared.

5 INSTRUMENTING A GO TO-DO API

5.1 The application and the tool

The demo is the back end of a to-do application written in Go that follows the practices above, with New Relic as a complete solution for its logs, metrics and traces. The lecturer's advice applies to the code shown: the aim is not to understand or memorize it, but to see how each practice becomes concrete. The trade-off of Section 3.4 decided the tool: wiring up Prometheus, Grafana, Promtail and Jaeger

is complicated, and without the experience or the time a proprietary solution makes more sense.

5.2 Logger set-up

A dedicated file creates the logger before the application starts, and two of its settings implement Section 4 (Figure 3). A function `getLogLevel` checks whether the application runs in development mode (locally) or in production mode (deployed on the infrastructure) and returns the debug level for the first and the info level for the second: locally more logs help debugging, in production only informational logs are wanted. The log format defaults to `console`, because the default environment is development.

In the demo the format is switched to JSON and the environment from local to production, and the server is started again: every start-up log now appears as a JSON object, good for production but hard to read. With the development settings restored, the same start-up produces three readable lines: connecting to the database, starting the background job server (see Chapter 10) and starting the HTTP server. The JSON version carries the same information but is meant for other tools.

```
func getLogLevel(env string) zerolog.Level {
    if env == "production" {
        return zerolog.InfoLevel   // debug lines dropped
    }
    return zerolog.DebugLevel      // local: everything
}

func NewLogger(cfg *Config) zerolog.Logger {
    zerolog.SetGlobalLevel(getLogLevel(cfg.Env))
    var out io.Writer = os.Stdout          // JSON lines
    if cfg.LogFormat == "console" {        // default
        out = zerolog.ConsoleWriter{Out: os.Stdout}
    }
    return zerolog.New(out).With().Timestamp().
        Str("service", cfg.ServiceName).Logger()
}
```

Figure 3: Logger set-up in which level and format follow the environment (illustrative Go, zerolog style).

5.3 Monitoring middleware and instrumentation

For monitoring, the router registers a New Relic middleware that wraps the whole application. Its source shows what it does on every incoming request: it *instruments* the request, measuring its route, duration and outcome and reporting them to New Relic, where they become the transaction metrics of Section 6.

> **Instrumentation.** The practice of measuring attributes of the code at run time, such as how long a function took, what it received and whether it failed, and emitting them as logs, metrics and traces. Instrumentation is what makes a system observable.

5.4 OpenTelemetry

The second term heard constantly in this field is *OpenTelemetry*: a recent open standard together with an ecosystem of tools, SDKs, best practices and resources for instrumenting applications properly. It does not depend on the language: the community has built APIs, SDKs and tools for all major languages, Node.js, Go and Python among them. Even with a proprietary back end such as New Relic, an OpenTelemetry collector can be integrated to gain more control over how requests and components are instrumented; the instrumentation then survives a change of vendor.

5.5 The tracing middleware: one transaction per request

A second middleware, called *enhanced tracing*, is the first point of contact in the life cycle of every request. It creates a new transaction with the New Relic API and attaches attributes: the service name (from the configuration), the environment (local or production), the client's IP address and user agent, the request ID, and the user ID, user email and tenant ID: all the information the logs and traces will need later. It then stores the transaction in the request context (see Chapter 6). Every later layer takes the transaction out of the context instead of creating its own, so a single request yields a single trace that starts in the middleware and continues through the validation layer into the service layer (Figures 4 and 5).

```
func EnhancedTracing(app *newrelic.Application,
    cfg *Config) func(http.Handler) http.Handler {
  return func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter,
        r *http.Request) {
      txn := app.StartTransaction(r.Method + " " +
          r.URL.Path)
      defer txn.End()
      txn.AddAttribute("service.name", cfg.ServiceName)
      txn.AddAttribute("environment", cfg.Env)
      txn.AddAttribute("client.ip", r.RemoteAddr)
      txn.AddAttribute("user_agent", r.UserAgent())
      txn.AddAttribute("request.id", requestID(r))
      txn.AddAttribute("user.id", userID(r))
      // ... user email, tenant ID
      ctx := newrelic.NewContext(r.Context(), txn)
      next.ServeHTTP(w, r.WithContext(ctx))
    })
  }
}
```

Figure 4: The enhanced-tracing middleware creates one transaction per request and puts it into the context (illustrative Go).

Standard tracing vocabulary names the parts: a trace is a tree of *spans*, one per unit of work, each with its own ID and timing. New Relic calls the traced request a *transaction* and its timed parts *segments*. The span ID seen in the log record of Section 6.3 is what links a log line to its place in the trace.

```
trace 7f3a9c...   POST /todos                    41 ms
EnhancedTracing (transaction)  [==========================]
  validation                   [==]
  todo.create (service)           [====================]
    repository: CreateTodo          [==================]
      database: INSERT todos         [================]
```

Figure 5: One request, one trace: nested segments from the middleware to the database (illustrative).

5.6 Instrumenting a service method

The `createTodo` service, which creates a new to-do, shows how the three pillars meet in one function (Figure 6):

1. The first statement takes the transaction out of the context. A deferred call ends this function's segment, the to-do create service, when the function returns.

2. Two attributes are added to the transaction: the user ID and the title of the requested to-do.
3. The operation is logged as `create_todo`, with the title.
4. If the request payload contains a priority, it is added as an attribute, so that it appears in the trace, and it is logged.
5. An info log records that the creation of a new to-do is starting; then the parent and child relation of the to-dos is validated.
6. The database operation creates the to-do. On failure the error is logged at error level and recorded on the trace through the transaction, and the attribute `operation=create` shows that this operation ended in an error.
7. On success, attributes record the operation `create` and the new to-do's ID, and a debug log states that the to-do was created with that ID; this line never appears in production logs.
8. Finally a *business-event* log records that a to-do was created, with its metadata: ID, title, category ID and priority.

```
func (s *TodoService) CreateTodo(ctx context.Context,
    userID string, p *CreateTodoPayload) (*Todo, error) {
  txn := newrelic.FromContext(ctx)
  defer txn.StartSegment("todo.create").End()
  txn.AddAttribute("user.id", userID)
  txn.AddAttribute("todo.title", p.Title)

  log := s.log.With().Str("operation", "create_todo").
      Str("title", p.Title).Logger()
  if p.Priority != nil {
    txn.AddAttribute("todo.priority", *p.Priority)
    log = log.With().Str("priority", *p.Priority).Logger()
  }
  log.Info().Msg("creating new todo")
  // ... validate parent/child relation

  todo, err := s.repo.CreateTodo(ctx, userID, p)
  if err != nil {
    log.Error().Err(err).Msg("failed to create todo")
    txn.NoticeError(err)
    txn.AddAttribute("operation", "create")
    return nil, err
  }
  txn.AddAttribute("operation", "create")
  txn.AddAttribute("todo.id", todo.ID)
  log.Debug().Str("todo_id", todo.ID).
      Msg("todo created successfully")  // not in prod
  log.Info().Str("event", "todo_created").
      Str("todo_id", todo.ID).Str("title", todo.Title).
      Str("category_id", todo.CategoryID).
      Str("priority", todo.Priority).Msg("business event")
  return todo, nil
}
```

Figure 6: The createTodo service with trace attributes, error recording and logs at three levels (illustrative Go).

Warning. The context carries the trace. If any layer passes `context.Background()` instead of the request context, `FromContext` finds no transaction; the New Relic Go agent treats a nil transaction as a no-op, so nothing fails, but the segment, its attributes and its errors silently vanish from the trace. The dashboard can only show what the code records.

6 READING THE DASHBOARD

6.1 Generating traffic

For the logs to reach New Relic, the format is set back to JSON and the environment to production. The application page of the New Relic dashboard shows data such as the error rate and the transaction time; with no activity in the previous 30 minutes it is empty. Requests are therefore fired several times at the "get all to-dos" endpoint from the API's OpenAPI interface (see Chapter 25). No token is sent, so every call fails with 401 Unauthorized (see Chapter 4); the demo then checks that these errors are recorded.

6.2 Errors and metrics

After a refresh, the Errors page, filtered to the HTTP error type, lists several Unauthorized errors together with their related logs. The Summary page shows the metrics: average transaction time, throughput and errors, actual numbers that quantify the state of the system. Because the recent traffic consisted mostly of the unauthenticated calls, the error rate averages about 80%.

6.3 A log record

From the errors the engineer reaches the logs, and opening the Unauthorized log shows everything recorded with it (Table 5): enough context to tell which service, environment, machine and route produced the error and, through the span ID, where in which trace it happened.

Field	What it tells
application name	which service wrote the log
environment	local or production
error	Unauthorized
host name, IP	which machine handled the request
level, message	severity and description
method, route	GET /todos
span ID	link to the trace
timestamp	when it happened

Table 5: Fields of the Unauthorized log record in the demo.

6.4 Traces, transactions and the runtime

The log record is connected to its trace; clicking it opens the whole trace for further detail. The Transactions view lists the transactions created in code by the middleware and enriched in the service, with the whole path of each. Selecting `/todos` shows that transaction's own error rate, response time and further data. The Go runtime view describes the process in which the backend runs: garbage-collection time, memory use (about 3 MB in the demo), throughput and average response time.

6.5 A spectrum and a shared responsibility

No system is completely observable, and none logs or monitors 100% of its parameters; the practices are applied on a spectrum, with different tools to record and measure them. Whether the tools are open source (Grafana, Loki, Prometheus, Jaeger) or proprietary and simpler to set up (New Relic, Datadog), they show the state of the service, its

infrastructure and the interactions of its components only after the code has been instrumented as in Section 5. The work is collective: developers instrument at code level, and infrastructure or DevOps engineers set up the collection of metrics, logs and traces (see Chapter 27). No single skill is involved, but the practices are an essential part of any production system.

7 CONCLUSION

- Logging, monitoring and observability are practices applied on a spectrum; no system is fully observable.
- Logs answer what happened, metrics show patterns and trends, traces show how components interacted; observability needs all three.
- Monitoring tells the team that something is wrong; observability also tells it what is wrong.
- Debugging runs from an alert to a metric, to the logs behind it, to the trace of the failing request.
- Levels filter logs: debug in development, info and above in production; warn for client-caused failures, fatal stops the process.
- Readable console logs belong to development, structured JSON logs to production, where tools must extract fields such as user and request IDs.
- One transaction per request, created in middleware and carried in the context, joins all layers into one trace that services enrich with attributes and errors.
- The Grafana stack and all-in-one services such as New Relic are chosen by team capacity; OpenTelemetry keeps instrumentation vendor-neutral.
- The dashboard shows only what the code records: developers instrument, DevOps engineers collect.

APPENDIX A REVIEW QUESTIONS

1. A team's only alert fires when the API's error rate exceeds 80% over ten minutes. Overnight, 3% of `POST /orders` requests fail with 500 for two hours while thousands of GET requests succeed, nobody is paged, and in the morning the on-call engineer has only plain-text logs spread over four servers. Reconstruct why the incident went both unnoticed and unexplained, and design the alert, metrics, log format and traces that would have led from a page to the failing function within minutes.
2. A new deployment of the service whose logger is shown in Figure 3 sets `ENV=Production` (capital P) and leaves `LOG_FORMAT` unset. Predict exactly which log lines of the createTodo service in Figure 6 reach the log-management tool, in what format, and what happens when the team then filters the logs by user ID. Connect your answer with the configuration practices of Chapter 13.
3. A payments service logs every request body at debug level, wrong passwords and failed payload validation at error level, and database time-outs at warn, and runs in production at debug level "to be safe". Its log bill has tripled and the on-call channel receives hundreds of error alerts a day that nobody reads. Reassign the levels event by event, choose the production level, and justify what is lost and what is gained.
4. A request passes a load balancer, is handled by the Go to-do API of Section 5, and enqueues an e-mail job that a separate worker process runs later (Chapter 10). In the dashboard the trace ends at the enqueue call, and the worker's logs cannot be connected to the original request. Explain why the mechanism of Figure 4 stops at that boundary, and design what must travel with the job, weighing the trade-offs.
5. Two organizations need observability: a four-person start-up with one Go service and no operations staff, and a 300-engineer company with 80 services, strict data-residency rules and a large monthly vendor bill. For each, choose between the Grafana stack and an all-in-one service such as New Relic or Datadog, decide whether to instrument with OpenTelemetry, and justify both choices in terms of cost, effort and lock-in.
6. Metrics are collected every 15 seconds and an alert fires when the error rate averaged over five minutes exceeds 10%. The service normally handles 200 requests per second with 0.5% errors; a deploy causes 100% errors for 40 seconds and is then rolled back. Compute whether the alert fires and when, and discuss how the window, the threshold and the collection delay trade detection speed against false alarms.
7. To find slow users, a developer adds the user ID and the request ID as labels on the Prometheus request-duration metric of a service with two million users. Within a day Prometheus runs out of memory and the dashboards time out. Explain the failure, and decide for each piece of request context (user ID, request ID, route, status code, tenant ID) whether it belongs in metrics, in logs or in trace attributes.
8. In the to-do API, the service logs every database error and records it on the transaction (Figure 6), the handler logs it again before responding, and a global error middleware (Chapter 12) logs it a third time and reports it to New Relic. Predict what one failed insert looks like in the logs, in the error count and in the error-rate metric, and redesign the responsibilities of each layer so that every failure is recorded exactly once with full context.

APPENDIX B GLOSSARY

Term	Meaning
Log	Timestamped record of one event with its context (user ID, request ID, route, latency).
Log level	Severity tag (debug, info, warn, error, fatal) used to filter what is written.
Structured log	Log written as machine-parseable fields, usually one JSON object per line.
Log management	System that collects, parses, stores and queries logs, e.g. the ELK stack or Loki.
Monitoring	Continuous checking of a system's health and performance over time.
Metric	Numeric measurement of the system over time, e.g. throughput or error rate.
Alert	Rule on a metric that notifies people, e.g. a Slack message via a webhook.
Observability	Ability to infer a system's internal state from its logs, metrics and traces.
Trace	Record of one request's path through all the components it touched.
Span	One timed unit of work inside a trace; its ID links logs to it.
Transaction, segment	New Relic's names for a traced request and a timed part of it.
Instrumentation	Code that measures functions and emits logs, metrics and traces.
OpenTelemetry	Open, vendor-neutral standard and SDKs for instrumentation and telemetry export.
Throughput	Number of requests handled per unit of time.
Error rate	Share of requests that failed within a time window.
Request context	Per-request object that carries values such as the transaction through all layers.

APPENDIX C FURTHER TOPICS

Metric types and percentiles

Metric systems distinguish counters (values that only grow, such as requests served), gauges (values that go up and down, such as open connections) and histograms (distributions, such as request durations in buckets). Averages hide the slow tail: a mean response time of 50 ms can coexist with 1% of requests taking several seconds. Percentiles computed from histograms, such as p95 and p99, describe what the slowest users experience. They apply whenever latency is measured, alerted on or promised to clients.

The four golden signals, RED and USE

Google's SRE practice recommends four golden signals for any user-facing service: latency, traffic, errors and saturation. The RED method (rate, errors, duration) applies the same idea to request-driven services, and the USE method (utilization, saturation, errors) to resources such as CPU, disks and connection pools. These checklists give every service a minimal, comparable set of metrics instead of dashboards assembled ad hoc, and they are the natural starting point when instrumenting a new service.

SLIs, SLOs and error budgets

A service level indicator (SLI) is a measured quantity such as the fraction of requests served successfully within 300 ms; a service level objective (SLO) is its target, for example 99.9% over 30 days. The gap between the target and 100% is an error budget that the team may spend on releases and experiments. Alerting on how fast the budget is being burned, instead of on fixed thresholds, pages people only for problems that users actually feel and reduces alert fatigue.

Trace context propagation

A request context lives inside one process, so a trace stops at every network call or queue unless its identity travels along. The W3C Trace Context standard defines the `traceparent` HTTP header, which carries the trace ID and the parent span ID to the next service; message queues carry the same data in message headers. OpenTelemetry propagators inject and extract it automatically. Without propagation, a request that crosses services or background workers yields disconnected fragments instead of one trace.

Trace sampling

Recording every span of every request is expensive at high traffic, so tracing systems sample. Head-based sampling decides at the start of a request, for example keeping 10% of traces, which is cheap but may discard the rare failing request. Tail-based sampling, typically done in an OpenTelemetry collector, decides after the trace is complete and can keep all errors and slow requests. The sampling policy determines whether the trace needed during an incident exists at all.

Metric cardinality

In systems such as Prometheus, every distinct combination of label values creates a separate time series. Labels with unbounded values, such as user ID or request ID, multiply the number of series until memory, storage and query time explode, and vendors bill by the same measure. High-cardinality details belong in logs and trace attributes, while metric labels stay bounded, for example route, method and status class.

Sensitive data in logs

Logs are copied to several systems, kept for months and read by many people, so passwords, tokens, card numbers and personal data must never be written to them. Common defences are logging an allow-list of fields instead of whole request bodies, redacting known sensitive keys in the logger, and masking identifiers. Regulations such as the GDPR make personal data in logs a legal issue as well as a security one (see Chapter 16).

Log volume, retention and cost

Log volume grows with traffic, and hosted tools usually charge per gigabyte ingested and stored. Teams control the cost with retention tiers (recent logs searchable, older logs archived cheaply), sampling of high-volume info events,

and turning repetitive events into metrics. Retention periods also follow audit and legal requirements, so they are a design decision rather than a default.

Graceful Shutdown — One-Page Overview

Chapter 15 · every section and sub-section of the paper at a glance

1 Introduction: Why a server must leave politely

A deployment eventually stops the old server, and whatever it is doing at that moment is at risk.

- **1.1 The deployment moment** — Zero-downtime deployment still ends with the old instance stopping, possibly mid-payment: lost transactions or double charges.
- **1.2 Good manners for a backend** — Finish conversations, say goodbye, clean up, then close the door: better user experience, no corrupted data.

2 The process life cycle and Unix signals

A backend runs as a process, and the system asks it to stop through signals instead of pulling the plug.

- **2.1 Every backend is a process** — Born at start, alive while executing, dead at termination; shutdown is the last phase.
- **2.2 Shutdown as a conversation** — Signals are the Unix IPC mechanism; on servers, Unix means Linux in almost every case.
- **2.3 Signal handlers** — Code the application registers to run when a given signal arrives, stopping itself in predefined steps.

3 SIGTERM, SIGINT and SIGKILL

Two polite signals that a process can handle, and one that it cannot.

- **3.1 SIGTERM: a polite request** — Sent by Kubernetes, systemd, PM2: finish requests, clean up, exit.
- **3.2 SIGINT: the interrupt from the keyboard** — Ctrl+C, user-initiated; handle it exactly like SIGTERM (PM2 in fact sends SIGINT).
- **3.3 SIGKILL: pulling the plug** — Cannot be caught or ignored; the fate of a process that does not respect the polite signals.

4 Connection draining

Stop accepting new work, let in-flight work finish within a hard limit, then close.

- **4.1 In-flight requests** — Tens to thousands of requests already in progress when the signal arrives.
- **4.2 The restaurant at closing time** — Stop admitting guests, give seated guests 15–20 minutes, then close.
- **4.3 Draining depends on the architecture** — HTTP servers, databases and WebSocket servers drain differently; the idea is the same.
- **4.4 Choosing the drain timeout** — Commonly 30 s; too short cuts legitimate work, too long slows deployments; must fit the manager's deadline.
- **4.5 Load balancers and service discovery** — Fail health checks and deregister so that no new traffic reaches a stopping instance.

5 Resource cleanup

Release everything acquired while running, in reverse order of acquisition.

- **5.1 What counts as a resource** — File handles, network and database connections, temporary files, caches.
- **5.2 File handles and network connections** — The OS mediates both and limits them per process; exit frees them, but not their side effects.
- **5.3 Database transactions** — Commit or roll back explicitly; the real risks are lost work, held locks and cross-system inconsistency.
- **5.4 Reverse order of acquisition** — Release dependents before what they depend on.

6 A worked example in Go

A Go backend with a database and a Redis-backed job server, stopped with Ctrl+C.

- **6.1 Registering the signal handler** — `signal.NotifyContext` cancels a context on a signal, which triggers the shutdown function.
- **6.2 The shutdown function** — `Server.Shutdown`, closing the database pool, shutting down the Asynq job server.
- **6.3 The demonstration** — Start and stop logs; about one second to shut down with no in-flight requests.
- **6.4 Libraries and understanding** — Frameworks supply the code; the engineer must know the steps and their order.

Graceful shutdown at a glance — A polite signal (SIGTERM or SIGINT) starts a fixed sequence: stop accepting work, drain in-flight requests within a timeout, release resources in reverse order and exit before the process manager falls back to SIGKILL. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Graceful Shutdown: Stopping a Backend Without Losing Work

Chapter 15 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — A backend stopped for a deployment may be in the middle of a payment, and stopping it abruptly can lose the transaction or charge the customer twice. This chapter places graceful shutdown in the process life cycle and explains how Unix signals let the operating system or a process manager ask a backend to stop. It contrasts the catchable signals SIGTERM and SIGINT with SIGKILL, which ends a process without warning. It then describes connection draining, the choice of a drain timeout and coordination with load balancers, followed by resource cleanup in reverse order of acquisition. A Go backend with a database and a Redis-backed job server illustrates the whole sequence and its logs.

Keywords — graceful shutdown, process life cycle, Unix signals, SIGTERM, SIGINT, SIGKILL, connection draining, drain timeout, resource cleanup

1 INTRODUCTION: WHY A SERVER MUST LEAVE POLITELY

1.1 The deployment moment

A server is processing a critical payment when a deployment requires it to restart: someone has pushed new code to production. Zero-downtime deployment keeps the old server serving until a server running the new code is up and ready to receive traffic, but it cannot remove the moment that follows. Once the new server goes online, the old one must stop receiving traffic and shut down, and whatever it is doing at that instant — a customer's purchase on Amazon or Flipkart, for instance — is exposed. The payment may be lost, or a race condition may charge the customer twice. The problem is as old as servers, and its established solution is *graceful shutdown*: stopping the server deliberately rather than abruptly.

1.2 Good manners for a backend

The lecturer's simplification is that graceful shutdown teaches a backend good manners. A host whose guests are still there at nine o'clock does not push them out and slam the door; the host finishes the conversation, says goodbye, tidies up and only then closes the door. A backend asked to stop behaves the same way: it finishes its ongoing conversations with clients, releases what it holds and then exits.

The payoff is twofold. Users get a good experience, because requests already in progress complete instead of failing. Data stays correct, because a payment in progress is neither lost nor duplicated, which also spares the business the double charges and refunds that would follow.

> **Graceful shutdown.** The controlled termination of a process: it stops accepting new work, completes the work already in progress within a time limit, releases the resources it acquired, and then exits.

2 THE PROCESS LIFE CYCLE AND UNIX SIGNALS

2.1 Every backend is a process

A backend is an application, and every application runs inside a *process* on some server: everything an operating system executes runs as a process. Like a living thing, a process has a life cycle. It is born when it starts, lives while it executes and dies when it terminates. Graceful shutdown is the discipline applied to the last phase, and its mechanics follow directly from how an operating system ends a process.

2.2 Shutdown as a conversation

When the system decides that an application must stop, it does not simply kill it. It follows an established protocol that can be pictured as a conversation: the operating system says "it is time for you to stop", and the application answers "give me a few seconds; then I will stop myself, or you may stop me". Programs do not exchange sentences, so the conversation is carried by *signals*, the Unix mechanism for inter-process communication (IPC), through which processes communicate by a fixed protocol that the application does not have to implement itself. A signal is a small predefined notification, identified by a name and a number, that the kernel delivers to a process.

"Unix" covers the Linux distributions (Ubuntu, Arch and the rest) and macOS, whose kernel descends from Unix. For servers it effectively means Linux: when a backend is deployed on a server or with a cloud provider, the operating system is Linux in about 99% of cases by the lecturer's estimate, with Windows Server reserved for specialised use cases.

2.3 Signal handlers

The application takes part in the conversation by registering *signal handlers*: code that runs when a particular signal arrives and carries out the shutdown. By registering a handler the application in effect tells the operating sys-

tem: “when you want me to stop, send me this specific signal, and I will stop myself through predefined steps”.

At the kernel level a handler is not a loop that keeps polling: the process registers a function, and when the signal is delivered the kernel interrupts the process to run it. Runtimes such as Go’s turn the signal into a value sent on a channel, which an ordinary goroutine waits for — the picture of code “waiting in the background” that the lecturer uses.

3 SIGTERM, SIGINT AND SIGKILL

In each name the prefix SIG stands for *signal* and the rest names the request. SIGTERM and SIGKILL are the two major signals; SIGINT is a third that a backend treats like SIGTERM.

3.1 SIGTERM: a polite request

SIGTERM (*terminate*) is the polite way to ask an application to shut down. It is a nudge rather than an order — a tap on the shoulder meaning “excuse me, could you please finish up and leave”. Because it is a request, the application gets a window, typically a few seconds, to complete what it is already doing. For an HTTP backend that means requests: clients (a front end, a web or mobile app, a browser extension) keep sending HTTP requests, so at any random moment a backend may be processing 10 or 12 of them, and a large application 500 or 600 concurrently. On SIGTERM the backend performs three steps:

1. finish the existing requests;
2. clean up its resources;
3. exit.

SIGTERM is the signal used by deployment systems, process managers and orchestration platforms — whatever system manages the process, such as Kubernetes, systemd or the PM2 process manager. It is also what the shell command `kill` sends when no signal is named.

3.2 SIGINT: the interrupt from the keyboard

SIGINT (*interrupt*) is the signal a developer sends by pressing Ctrl+C, the usual way to stop a command-line program instantly. In the lecturer’s demonstration a Go backend runs locally, ready to serve clients; pressing Ctrl+C stops it, and because the application implements graceful shutdown, it logs that a signal has been received and that its type is *interrupt*. Since it needs a key press, SIGINT is called a *user-initiated shutdown* and appears mostly in development, while programs that signal other programs normally use SIGTERM.

In practically every case SIGINT should be handled exactly like SIGTERM. A backend stopped with Ctrl+C on a laptop and the same backend running on an AWS EC2 instance, stopped by PM2 programmatically or by an operator, express the same intention: shut down, and do it cleanly. Whether a human or a program initiates the shutdown does not matter.

> **A subtle point.** “SIGINT for humans, SIGTERM for programs” is a tendency, not a rule. Any process can send SIGINT (`kill -INT <pid>`), and PM2 — the lecturer’s example of a SIGTERM sender — sends SIGINT when it stops or restarts an application, followed by SIGKILL 1.6 s later by default (its `kill_timeout` option). An application that handles only one of the two signals leaves the other with its default action, immediate termination; handling both identically is a requirement, not a convenience.

3.3 SIGKILL: pulling the plug

SIGKILL orders immediate termination, and it can be neither caught nor ignored. The application cannot register a handler for it, so no cleanup code ever runs; nor can it decline the signal and keep running. The kernel removes the process at that moment, and nothing else happens. It is the nuclear option: the equivalent of pulling a computer’s power plug instead of choosing Shut down from the system menu. From a shell it is sent with `kill -9`.

This is why graceful shutdown matters. SIGTERM and SIGINT are the polite signals: they let a process finish its work, clean up and exit. A process that does not respect them — ignores them, or takes too long to act on them — eventually receives SIGKILL from whatever manages it, and then loses even the chance to clean up after itself.

Signal	Typical sender	Catchable	Expected response
SIGTERM (15)	Kubernetes, systemd, `kill`	yes	finish, clean up, exit
SIGINT (2)	Ctrl+C; also PM2	yes	same as SIGTERM
SIGKILL (9)	`kill -9`; managers after their deadline	no	none: the process dies

Table 1: The three shutdown signals.

4 CONNECTION DRAINING

4.1 In-flight requests

An HTTP server processes many requests concurrently, so when it is told to stop it is usually in the middle of several: 10 or 12 on a small service, hundreds or thousands on a large one. These are the *in-flight* (on-the-fly) requests — the requests already being processed when shutdown begins. Finishing them is the first of the two substantive steps of graceful shutdown; cleaning up resources (Section 5) is the second.

4.2 The restaurant at closing time

When a restaurant closes — at 10:30 or 11 at night, or for any other reason — the owners do not switch off the lights and throw the diners out. First, they tell the person at the door to stop admitting new customers, so that nobody arrives who must then be turned away. Second, they announce to the diners already eating that the restaurant is closing and that they have 15 or 20 minutes, ample time to finish the meal, pay the bill and the tip, and leave.

The same procedure for a backend is *connection draining*. On a shutdown signal — SIGTERM from a process manager or SIGINT from a developer's Ctrl+C — the application first stops accepting new connections and requests, because every new arrival makes the shutdown messier and harder to complete. It then lets the requests already in progress finish as quickly as possible, and finally closes the connections (Figure 1).

```
t = 0     SIGTERM or SIGINT arrives
  |
  +-- 1. stop accepting new connections and requests
  |       (fail health checks, deregister)
  |
  +-- 2. drain: in-flight requests run to completion
  |       ..... bounded by the drain timeout
  |
  +-- 3. clean up: commit or roll back, close pools,
  |       files and caches, in reverse order
  |
  +-- 4. exit

t = 30 s  still running? the manager sends SIGKILL
```

Figure 1: The sequence of a graceful shutdown on a timeline (illustrative; 30 s stands for a typical process-manager deadline).

4.3 Draining depends on the architecture

The three steps are always the same — stop accepting, finish the existing work, close — but their technical form depends on the kind of application (Table 2). An HTTP backend stops accepting new HTTP requests from any client and lets in-flight requests complete. A database is also a backend in the broad sense (see Chapter 8): it finishes the queries and transactions it is executing and refuses new ones before closing its connections. A WebSocket server must first notify its clients that it is closing and only then close the sockets, never close them abruptly: a long-lived connection that simply drops looks like a failure to the client, while a notified client can reconnect elsewhere.

Application	Stops accepting	Lets finish	Then
HTTP backend	new HTTP requests	in-flight requests	closes connections
Database	new queries and transactions	running queries and transactions	closes connections
WebSocket server	new connections	current messages; notifies clients	closes sockets

Table 2: Connection draining in three kinds of application.

4.4 Choosing the drain timeout

The hard part of draining is timing. Existing connections deserve enough time to complete, but the server cannot wait as long as they would like, so production systems bound the drain with a timeout — commonly 30 seconds, sometimes 60. It is the maximum the system waits; when it expires, the shutdown continues by force. Once new requests are refused, 30 seconds is usually far more than ordinary requests need. The limit is the backup plan for the exceptions, such as a request stuck in a blocking operation, which must not be allowed to delay the shutdown for as long as it likes.

The value itself is a design trade-off. A timeout that is too short interrupts legitimate operations; one that is too long makes every shutdown sluggish, which slows deployments and the system's responsiveness. The right value follows from the application's typical request duration and its operational requirements, not from a fixed rule: 30 seconds is enough for a conventional backend, whereas WebSocket servers and other more complex architectures need a timeout derived from an understanding of their own traffic.

> **Two clocks.** The application's drain timeout is not the only deadline. The manager that sent the polite signal keeps its own and sends SIGKILL when it expires: Kubernetes after `terminationGracePeriodSeconds` (default 30 s), `docker stop` after 10 s, systemd after `TimeoutStopSec` (default 90 s), PM2 after 1.6 s. The drain timeout plus the cleanup that follows must fit inside that outer deadline, or SIGKILL interrupts the shutdown halfway — the outcome graceful shutdown exists to prevent.

4.5 Load balancers and service discovery

Draining one process is not enough when it sits behind a load balancer: the rest of the infrastructure must also stop sending it traffic. Connection draining therefore has to cooperate with health checks and with service discovery, so that a stopping instance is reported as unhealthy and deregisters itself. Service discovery is the mechanism by which deployed components — the backend, the database, an Elasticsearch instance — find and connect to each other after deployment. An instance that keeps receiving traffic after it has stopped accepting connections produces exactly the client errors that draining was meant to avoid. The lecturer flags this coordination as advanced material and does not go further.

5 RESOURCE CLEANUP

5.1 What counts as a resource

Before leaving a desk for the night, one carries the coffee cup to the sink and tidies the cables. A backend's equivalent is releasing everything it acquired while running: file handles, network connections, database connections, temporary files, caches and any other system resources (Table 3).

Resource	Cleanup at shutdown	If skipped
File handle	flush buffers, close	buffered writes lost
Network connection	close with a protocol goodbye	peers see abrupt errors
DB transaction	commit or roll back	work lost, locks held
DB connection pool	stop new queries, close	queries cut mid-flight
Temporary file	delete	disk fills over restarts
Write-back cache	flush to its store	unsaved entries lost

Table 3: Resources a backend releases at shutdown.

5.2 File handles and network connections

A process cannot access the file system directly. It asks the operating system, which, through a defined protocol, gives it a *handle* to the file (on Unix, a file descriptor). The

handle must be released at some point; a program that keeps acquiring handles without releasing them consumes more and more memory until it runs out.

Network connections are the most common resource to clean up. The operating system mediates all network traffic: every request from the internet reaches the network card, and the operating system, acting as the driver, receives it and passes it to the application. It therefore knows every connection a process holds, and it limits how many file handles and network connections one process may have open simultaneously. A backend that does not give up connections after using them eventually exhausts that limit or runs into memory and performance problems.

A subtle point. The lecturer's argument — unreleased handles eat RAM until the machine runs out — describes a leak in a long-running process, and even there the limit usually hit first is the per-process descriptor limit (`ulimit -n`), which surfaces as "too many open files". At shutdown the argument does not apply: when a process exits, the kernel closes all its descriptors and frees all its memory, whether or not the code did. Cleanup at exit matters for what the kernel cannot do: flushing buffered writes, committing or rolling back transactions, sending protocol-level goodbyes (a WebSocket close frame, a database client's terminate message) and releasing locks, leases or registrations held in other systems.

5.3 Database transactions

Before the backend process ends, every database transaction it is handling must be either committed or rolled back explicitly by the application. The lecturer warns that otherwise the transactions may be left in an inconsistent state, with deadlocks, data corruption and similar problems as the result.

A subtle point. A database does not keep a half-applied transaction when its client disappears: by atomicity, the server rolls the transaction back once the connection closes. The real costs are different. The work in progress is lost; the transaction's locks stay held until the server notices that the connection is gone — which can take long if the client's host vanished without closing its TCP connection — and block other sessions meanwhile; and an operation spanning several systems, such as a card charged by a payment provider whose order row was never committed, is left inconsistent, because no single database can roll back both. The last case is the double charge of Section 1.

5.4 Reverse order of acquisition

Resources are released in the reverse of the order in which they were acquired. If the application established a Redis connection, then a database connection, then further components, shutdown releases the further components first, then the database connection, and Redis last. The reason is dependency: a component acquired later often relies on one acquired earlier, and releasing the earlier one first would leave the later one running without something it needs — for example, a job worker still finishing a task after its database pool has been closed. Reverse order guarantees that nothing is torn down while something that depends on it is still alive.

6 A WORKED EXAMPLE IN GO

The lecturer's demonstration backend is written in Go and has three components: a database, a background job processing server backed by Redis, and the HTTP server. The narrative matters more than the Go syntax.

6.1 Registering the signal handler

The program first registers a handler that waits for signals. In Go this is done through a *context*, Go's cancellation mechanism: `signal.NotifyContext` from the `os/signal` package returns a context that is cancelled when an interrupt arrives from the operating system. When that happens, the program calls its graceful-shutdown function (Figure 2). Listing `syscall.SIGTERM` next to `os.Interrupt` makes production stops take the same path as Ctrl+C (Section 3.2).

```
func main() {
    db := mustOpenDB()      // acquired 1st
    jobs := mustStartJobs() // acquired 2nd (Asynq, Redis)
    srv := &http.Server{Addr: ":8080", Handler: mux}

    ctx, stop := signal.NotifyContext(context.Background(),
        os.Interrupt, syscall.SIGTERM)
    defer stop()

    go func() {
        err := srv.ListenAndServe() // acquired 3rd
        if !errors.Is(err, http.ErrServerClosed) {
            log.Fatal(err)
        }
    }()

    <-ctx.Done() // blocks until SIGINT or SIGTERM
    gracefulShutdown(srv, db, jobs)
}

func gracefulShutdown(srv *http.Server, db *sql.DB,
    jobs *asynq.Server) {
    ctx, cancel := context.WithTimeout(
        context.Background(), 30*time.Second)
    defer cancel()
    srv.Shutdown(ctx) // stop accepting, drain in-flight
    db.Close()        // finish queries, close the pool
    jobs.Shutdown()   // wait for workers, close Redis
    log.Println("server exited properly")
}
```

Figure 2: Start-up and shutdown structure of the demonstration backend in Go (illustrative sketch; the shutdown order follows the lecturer's code).

6.2 The shutdown function

1. **HTTP server.** The HTTP engine is the core of a backend, so it is shut down first. Every HTTP framework or library provides a method for this, which internally stops receiving additional connections, lets the existing ones finish and then returns. In Go's standard library it is `Server.Shutdown`: it closes the listeners, closes idle connections and waits for active ones to become idle, giving up when the context passed to it — the drain timeout of Section 4.4 — expires. It does not wait for hijacked connections such as WebSockets; the applica-

tion must notify and close those itself, for instance through `RegisterOnShutdown`.

2. **Database.** Next the program closes the database: no new queries or transactions are started, the running ones finish, and every connection held is given up. A backend talks to its database over TCP, and an active TCP connection is the only way it can communicate with it. With connection pooling (see Chapter 8) the backend keeps several such connections open and uses one of them per operation. Closing the pool means: stop sending new queries through these connections, let the current queries and transactions finish, then close the connections one by one.
3. **Background job server.** Finally the program shuts down its background job processing server (see Chapter 10), which uses Redis; the shutdown call also closes the Redis connections. After this step the backend has shut down gracefully.

6.3 The demonstration

The lecturer starts the server with `task run` and presses Ctrl+C, sending SIGINT to the backend and starting the shutdown procedure. The start-up log has three lines, in order: connected to the database, starting the background job server, server started. After Ctrl+C the log shows the database connection being closed and the background job server stopping. The job server's lines come from Asynq, the Redis-based Go task-queue library the backend uses: when its shutdown method is called, it logs that it is starting a graceful shutdown, that it is waiting for all workers to finish, that all workers have finished, and that it is exiting. A final application line reports that the server exited properly (Figure 3). With no in-flight requests on a local machine the shutdown was almost instantaneous, yet it still took about one second.

```
$ task run
connected to the database
starting background job server
server started
^C
signal received: interrupt
database connection closed
asynq: Starting graceful shutdown
asynq: Waiting for all workers to finish...
asynq: All workers have finished
asynq: Exiting
server exited properly
```

Figure 3: Log of the demonstration run, start-up and Ctrl+C shutdown (illustrative reconstruction; real Asynq lines also carry a PID, a timestamp and a level).

A subtle point. The start-up log gives the acquisition order database, job server, HTTP server, so the rule of Section 5.4 calls for the shutdown order HTTP server, job server, database. The demonstrated code closes the database before the job server. With idle workers, as in the demonstration, this is harmless; but a worker still finishing a job that writes to the database fails while Asynq is "waiting for all workers to finish", because its pool is already closed — the dependency problem the reverse-order rule exists to prevent.

6.4 Libraries and understanding

Developers rarely write this machinery from scratch. Whatever the language — Node.js, Go, Rust or Python — frameworks provide the shutdown primitives, and code for the whole workflow is widely available to copy. What cannot be copied is the understanding of what happens during a graceful shutdown and why each step exists, which is what lets an engineer adapt that code to a particular system. The complete workflow — starting, running and stopping — is the process life cycle; its controlled final phase is the graceful shutdown procedure, which matters most during deployments, when in-flight requests and workflows would otherwise be corrupted and users would feel it.

7 CONCLUSION

- Zero-downtime deployment still ends with the old instance stopping; graceful shutdown protects the work it is doing at that moment.
- The system asks a process to stop with signals: SIGTERM from process managers and orchestrators, SIGINT from Ctrl+C.
- Both polite signals can be caught and must take the same shutdown path; PM2, for one, sends SIGINT.
- SIGKILL cannot be caught or ignored; it is what a process receives when it does not finish in time.
- Connection draining: stop accepting new connections, let in-flight requests finish, then close — adapted to HTTP servers, databases or WebSockets.
- The drain timeout (commonly 30 s) trades interrupted work against slow deployments and must fit inside the manager's grace period.
- A stopping instance must also fail its health checks and leave load balancers and service discovery.
- Release files, connections and transactions explicitly, in reverse order of acquisition.
- Frameworks supply the primitives; the engineer supplies the order, the timeouts and the dependencies.

APPENDIX A REVIEW QUESTIONS

1. A service on Kubernetes keeps the default termination grace period. On SIGTERM its Go code calls `Server.Shutdown` with a 45-second context, then closes its PostgreSQL pool and finally flushes an in-memory audit-log buffer to disk; one report endpoint can run for 40 seconds. A deployment starts 2 seconds after a report request arrives. Describe what happens to that request, the pool and the audit buffer, and redesign the timeouts.
2. A Node.js API runs under PM2 with default settings and registers `process.on('SIGTERM', shutdown)`, where `shutdown` drains requests that can last up to 5 seconds. When the team stops it locally with `kill <pid>`, the shutdown logs appear; after every `pm2 reload` in production they never do, and some clients see reset connec-

tions. Explain the discrepancy and list every change needed for PM2 restarts to drain those requests.

3. A backend starts in this order: PostgreSQL pool, Redis client, a job worker whose jobs write to PostgreSQL and publish to Redis, HTTP server. Its shutdown function closes Redis, then PostgreSQL, then the HTTP server, and last the worker. A deployment catches it with 3 HTTP requests and 5 jobs in progress. Predict what each of them experiences, and give a correct shutdown order with its justification.
4. A chat service holds 20,000 WebSocket connections per instance; instances are replaced one at a time with a 30-second drain timeout, and clients reconnect immediately when their socket closes. Design the shutdown sequence of one instance, explain what happens to the remaining instances if every socket is closed within the first second, and connect your design with Chapter 22.
5. A checkout handler first charges the card through a payment provider and then commits the order row in PostgreSQL. An instance is killed with SIGKILL after the charge succeeds but before the commit. Describe the state of each system afterwards, what the client sees and probably does next, and judge whether a better graceful-shutdown implementation could have prevented the outcome; if not, propose the design that does, drawing on Chapter 12.
6. Two services share a pipeline that replaces 50 instances one at a time, 20 times a day. Service A has a p99 latency of 300 ms; service B has an export endpoint whose requests last up to 6 minutes. Compute the worst-case time a single deployment can spend waiting on drains with a 30-second and with a 6-minute timeout, and propose a design for B that keeps deployments fast without cutting off exports.
7. After a deployment, PostgreSQL shows several sessions "idle in transaction" holding row locks for minutes, and requests touching those rows time out. The old instances were stopped by a script that sends SIGTERM and then waits without any deadline. Explain which processes hold the locks and why the database has not rolled the transactions back, and identify every part of the shutdown design that would have prevented the incident.
8. An instance behind a load balancer closes its listener the moment it receives SIGTERM. The load balancer probes health every 10 seconds and removes an instance after 3 failed probes, and clients see a burst of connection errors after every deployment. Reconstruct the timeline that produces the errors and redesign the first seconds of the shutdown sequence.

APPENDIX B GLOSSARY

Term	Meaning
Graceful shutdown	Stop new work, finish current work within a limit, release resources, exit
Process	Running instance of a program; started, running, terminated
Signal	Predefined notification the kernel delivers to a process; a Unix IPC mechanism
Signal handler	Function a process registers to run when a given signal arrives
SIGTERM	Signal 15; polite termination request from managers and orchestrators
SIGINT	Signal 2; interrupt sent by Ctrl+C; handled like SIGTERM
SIGKILL	Signal 9; immediate termination; cannot be caught or ignored
Process manager	Tool that starts, supervises and stops processes (systemd, PM2, Kubernetes)
In-flight request	Request already being processed when shutdown begins
Connection draining	Refusing new connections while existing ones finish, then closing them
Drain timeout	Upper bound on the drain before shutdown proceeds by force
Grace period	Time a manager waits after the polite signal before sending SIGKILL
Health check	Probe that decides whether an instance should receive traffic
Service discovery	Mechanism by which deployed components find and reach each other
File descriptor	Kernel handle to an open file or socket; limited per process
Connection pool	Set of open database connections reused across operations

APPENDIX C FURTHER TOPICS

Readiness probes and the endpoint-removal race

In Kubernetes, deleting a pod starts two things at once: the container receives SIGTERM, and the pod is removed from the Service's endpoints, a change that kube-proxy and external load balancers apply only after a short delay. A process that closes its listener the instant SIGTERM arrives can therefore still be sent new connections for a few seconds and refuse them. The usual remedy is a `preStop` hook that sleeps a few seconds, or an application that keeps serving while failing its readiness probe before it starts to drain. It applies wherever the load balancer learns about removals asynchronously.

Draining persistent connections

HTTP/1.1 clients reuse keep-alive connections, so a server that stops accepting new TCP connections may still receive new requests on connections that are already open. Draining therefore also means closing idle connections and marking final responses with `Connection: close`; in HTTP/2 the server sends a GOAWAY frame naming the last stream it will process, so the client retries later requests on another connection (RFC 9113). Go's `Server.Shutdown` handles its own connections this way, but reverse proxies and custom

servers need checking. The issue matters most for clients and proxies that keep long-lived connection pools to the backend.

Rolling and blue-green deployments

The zero-downtime deployment the lecturer mentions is implemented by strategies that bring new instances up before old ones are drained. A rolling update replaces instances a few at a time — in Kubernetes, bounded by a Deployment's `maxSurge` and `maxUnavailable` settings — while a blue-green deployment runs a complete second environment and switches traffic over at once. Both are lossless only if the old instances shut down gracefully, and in both the drain timeout adds directly to the duration of a deployment. See Chapter 27 for deployment pipelines.

PID 1 and signal forwarding in containers

Inside a container the application often runs as process ID 1, which Linux treats specially: a signal for which PID 1 has installed no handler is not delivered to it (SIGKILL from the host excepted), so an application without a SIGTERM handler simply waits out the grace period and is then killed. A related trap is starting the application through a shell (the shell form of Docker's `CMD`), which makes the shell PID 1; many shells do not forward SIGTERM to their child. The remedies are the exec form of `CMD`, `exec` in entrypoint scripts, or a minimal init such as tini (`docker run --init`). Every containerised backend should verify that the signal actually reaches its code.

Idempotency keys

Graceful shutdown narrows the window in which a payment can be lost or duplicated but cannot close it, because SIGKILL, crashes and network failures remain. An idempotency key — a unique identifier the client generates and sends with a non-idempotent request — lets the server recognise a retry and return the stored result of the first attempt instead of executing the operation again; payment APIs such as Stripe's accept it in an `Idempotency-Key` header. It applies to every operation with external side effects that a client may retry after an interrupted request, and it is what actually prevents the double charge described in the introduction.

Draining queue consumers

A worker that consumes from a message queue drains differently from an HTTP server: on SIGTERM it stops fetching new messages, finishes or abandons the current ones, and acknowledges only completed work. Messages that were never acknowledged are delivered again — after the visibility timeout in Amazon SQS, or when the consumer's channel closes in RabbitMQ — so a task interrupted by SIGKILL runs a second time and must tolerate repetition. Asynq, used in the lecture's demonstration, pushes unfinished tasks back to Redis when its shutdown timeout expires. See Chapter 10 for task queues.

WebSocket close codes and reconnection

RFC 6455 defines a close handshake in which either side sends a close frame carrying a status code; code 1001, "going away", tells clients that the server is shutting down. Well-behaved clients then reconnect with exponential backoff and random jitter, because thousands of clients reconnecting at the same instant would overload the remaining instances (a thundering herd). Spreading the close frames over the drain window serves the same purpose on the server side. It applies to WebSockets and to other long-lived connections such as server-sent events; see Chapter 22.

Crash-only design and exit codes

SIGKILL, the kernel's out-of-memory killer and hardware failures bypass every handler, so robust systems are built so that an abrupt stop is always safe: durable state, atomic writes and recovery at start-up — the approach Candea and Fox called crash-only software. Graceful shutdown then becomes an optimisation for the common case rather than the only line of defence. When a process does die from a signal, shells and container platforms report exit status 128 + n, so 137 means SIGKILL and 143 an unhandled SIGTERM; these codes are the first clue when diagnosing shutdowns that overran their deadline.

Backend Security — One-Page Overview

Chapter 16 · every section and sub-section of the paper at a glance

1 Introduction: Thinking Like an Attacker

Security is a habit of questioning one's own assumptions, not a list of techniques.

- **1.1 Four kinds of security** — Browser, network, server and backend; this chapter covers the backend's code.
- **1.2 The attacker's question** — Where did the developer assume clean input, honest users or their own frontend?

2 Injection: When Data Becomes Code

User input that crosses into SQL, a query object or a shell must stay data.

- **2.1 One application, many languages** — Each language has special characters that change meaning across boundaries.
- **2.2 SQL injection, step by step** — `' OR '1'='1' --` makes the condition true: every row returns.
- **2.3 Destructive payloads and least privilege** — Stacked `DROP TABLE`; DML-only credentials.
- **2.4 Parameterized queries** — Statement and values sent apart; the value is never parsed.
- **2.5 NoSQL injection** — `$ne` operators smuggled in request JSON.
- **2.6 Command injection** — `; rm -rf /` in a file name; argument arrays skip the shell.
- **2.7 The general rule** — Never concatenate input into text another system interprets.

3 Authentication I: Storing Passwords

Each storage scheme answers the attack that broke the one before.

- **3.1 Build or buy** — Sessions, OAuth and account linking favour a provider.
- **3.2 Plain text** — Breaches plus password reuse expose users everywhere.
- **3.3 Hashing** — One-way, fixed-length, deterministic.
- **3.4 Rainbow tables and salting** — A random per-user salt defeats precomputed tables.
- **3.5 Brute force and slow hash functions** — Argon2id, bcrypt: from billions of guesses per second to a few.

4 Authentication II: Sessions, Tokens and Rate Limits

Remembering a login safely, and limiting how often anyone may try.

- **4.1 Server-side sessions** — Random ID, stored with metadata, sent in a cookie.
- **4.2 Cookie flags** — `HttpOnly`, `Secure`, `SameSite`.
- **4.3 JSON Web Tokens** — Signed claims; no lookup per request.
- **4.4 The weaknesses of JWTs** — Revocation, readable payload, storage.
- **4.5 Sessions or tokens?** — Prefer sessions; Redis handles scale.
- **4.6 Layered rate limiting** — Per IP, per account, global.

5 Authorization: Checking Access Where It Happens

A logged-in user with a permission may still not own the object requested.

- **5.1 Where the confusion lies** — Checks at the router, none in the repository.
- **5.2 Broken object level authorization** — Scope every query by owner.
- **5.3 403 or 404?** — One owner-scoped query; 404 leaks nothing.
- **5.4 Broken function level authorization** — A hidden admin URL is obscurity; require the role.
- **5.5 Predictable identifiers** — Sequential IDs invite enumeration; UUIDs.
- **5.6 Horizontal and vertical attacks** — Other users' data versus higher privilege.
- **5.7 A prevention framework** — Centralize, default deny, test, audit.

6 Cross-Site Scripting and Request Forgery

Injection into the browser, and requests forged with the victim's cookies.

- **6.1 Why XSS is dangerous** — Attacker script acts as the logged-in user.
- **6.2 Stored XSS** — A script in a saved comment runs for every reader.
- **6.3 Other variants and the root cause** — Reflected, DOM-based: content treated as code.
- **6.4 Content Security Policy** — A header that blocks inline scripts: the last line.
- **6.5 Cross-site request forgery** — `SameSite` and CORS make it minor today.

7 Misconfiguration

Sound code still leaks through secrets, logs and missing headers.

- **7.1 Secrets** — Never in git; rotate after a leak.
- **7.2 Debug output in production** — Stack traces and SQL in production logs.
- **7.3 Security headers** — Frame options against clickjacking; one-line middleware.

8 Boundaries and Layers

Every vulnerability sits at a boundary, and no single defence suffices.

- **8.1 Data crossing boundaries** — Three questions at every crossing.
- **8.2 Defence in depth** — Validation, parameters, authorization, headers, monitoring.
- **8.3 Further study** — PortSwigger Academy, OWASP Top 10, cheat sheets.

Backend security at a glance — every attack exploits an assumption about data crossing a boundary: keep input as data, store passwords with slow salted hashes, check access where it happens, and stack defences so that no single failure is fatal. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Backend Security: Thinking Like an Attacker

Chapter 16 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Every vulnerability a backend suffers from begins with an assumption its developer made about data, users or requests. Injection attacks — SQL, NoSQL and command injection — arise when user input crosses into another language and is executed as code; parameterized queries and argument arrays keep the two apart. Authentication is examined through password storage (hashing, salting, slow hash functions), session cookies and their flags, the trade-offs of JSON Web Tokens and layered rate limiting. Authorization failures are classified as horizontal (broken object level authorization, predictable identifiers) or vertical (broken function level authorization) and countered by checks at the point of access, default deny, automated tests and audit logs. Cross-site scripting, Content Security Policy, cross-site request forgery and common misconfigurations complete the survey, which closes with two organising ideas: boundaries and defence in depth.

Keywords — injection, parameterized queries, password hashing, sessions, JWT, rate limiting, broken access control, XSS, defence in depth

1 INTRODUCTION: THINKING LIKE AN ATTACKER

Security failures are among the most destructive a backend can suffer, for its finances, its business and its resources. The aim of this chapter is not a set of techniques to deploy tomorrow: no application is ever truly secure, because languages, libraries and attacks keep evolving. The aim is a mindset — healthy paranoia about one's own code — and a mental model that holds in Go, Python, Node.js or Rust alike: at every line, ask *what could go wrong here?*

1.1 Four kinds of security

Browser security concerns HTML, cookies, local storage and other client-side components; *network security* the transport — HTTP versus HTTPS, encryption, compression; *server security* the operating system in which the backend process runs. *Backend security*, the subject here, concerns the vulnerabilities created by the application's own code. Each domain could fill a course; the chapter keeps to the vulnerabilities and rules that matter in practical backend work.

1.2 The attacker's question

Rather than list abstract vulnerabilities, the lecturer teaches how an *attacker* — anyone who wishes the application harm — thinks. Attackers do not care about the framework, library or language; they ask one question: **where did the developer make an assumption?** Every vulnerability in this chapter reduces to one: that input from the frontend is clean; that users are who they claim to be; that requests come from the developer's own frontend; that nobody opens the browser's network tab, reads the calls and modifies the parameters. Under deadlines, especially in startups, such assumptions feel reasonable, because developers think in the *happy path*: users fill forms correctly, click the intended buttons and navigate as designed. Attackers poke at every boundary, break every input and try to guess every assumption.

2 INJECTION: WHEN DATA BECOMES CODE

2.1 One application, many languages

A backend speaks several languages, one for each system it talks to. To read or change data it speaks SQL to the database; to serve static assets it speaks HTML, CSS and JavaScript to the browser; to open files or run programs through system calls it speaks the shell's language to the operating system. Each language has its own grammar, its own special characters and its own way of separating commands from data. Users reach the backend through the browser, in the browser's languages. Vulnerabilities appear when input written in one context crosses into another, where some of its characters acquire a special meaning — a delimiter, a command, a comment. This is the root cause of the *injection* family, known for decades and still common: untrusted input is parsed by another interpreter, and part of the data is executed as code (Table 1).

Context	Special syntax	Safe interface
SQL	`' ; -- OR DROP`	parameterized query
MongoDB	keys starting with `$`	typed scalar values
Shell	`; | &` and more	argument array
HTML and JS	`<script>`, event attributes	escaping, sanitizer, CSP

Table 1: The languages a backend speaks and how to keep user input as data

2.2 SQL injection, step by step

Consider a login page with email and password fields. The server receives both, asks the database for the user with that email and checks the password. The danger lies in how the query is built. If the developer concatenates strings — takes the template `SELECT * FROM users WHERE email = '…'` and drops the input between the quotes — whatever the user typed becomes part of the SQL text. On the happy path the user types `alice@gmail.com`, the query finds Alice's row and she logs in.

The attacker instead types `' OR '1'='1' --`. The lecturer runs both queries in a database client against PostgreSQL, on a table holding Alice and Bob (Figure 1): the normal query returns one row, the injected one returns both, although no email was supplied. The trace: the input's first character closes the template's opening quote, so the left side of the condition is `email = ''`, false because no stored email is empty. `OR` makes the condition true if either side is, and the right side compares the string `'1'` with the string `'1'` — always true. The two hyphens open an SQL comment that swallows the template's closing quote, which would otherwise be a syntax error. The `WHERE` clause is now a constant truth, and the query is equivalent to `SELECT * FROM users`. The characters are harmless in themselves; they become malicious only in combination with the template. The demo table had two columns; a production users table holds password hashes, addresses, phone numbers and more.

```
CREATE TABLE users (name text, email text);
INSERT INTO users (name, email) VALUES
  ('Alice', 'alice@gmail.com'), ('Bob', 'bob@gmail.com');

-- happy path, input alice@gmail.com: 1 row
SELECT * FROM users WHERE email = 'alice@gmail.com';

-- attack, input ' OR '1'='1' -- : 2 rows
SELECT * FROM users WHERE email = '' OR '1'='1' --';

-- what the database effectively evaluates
SELECT * FROM users;
```

Figure 1: The lecturer's SQL injection demo on PostgreSQL — one template, a normal and a malicious email

2.3 Destructive payloads and least privilege

The same trick admits a second command. With `'; DROP TABLE users; --`, the quote closes the string, the semicolon ends the statement, `DROP TABLE users` starts a new one and the comment neutralises the rest. The first statement returns nothing (emails are normally `NOT NULL` and never empty); the second deletes the table. In the demo the client reported success with an empty result and the table survived: many modern drivers refuse such stacked statements by default, precisely because SQL injection is so old. (PostgreSQL's extended query protocol accepts one statement per message; most MySQL drivers disable multi-statement queries unless enabled.) An older or careless driver would run the `DROP`.

A second defence is *least privilege*: the database user the application connects with should hold only DML rights (data manipulation: `SELECT`, `INSERT`, `UPDATE`, `DELETE`), never DDL rights (data definition: `CREATE`, `ALTER`, `DROP`), so that even a successful injection cannot alter or remove tables. Skilled attackers go further: `UNION` queries extract other tables such as payment data, database-specific functions read files from the server, and some configurations even allow operating-system commands through the database. This attack surface has kept SQL injection among the most destructive vulnerabilities for decades.

2.4 Parameterized queries

In SQL the single quote delimits strings, the semicolon separates statements, the double dash starts a comment, and keywords such as `OR`, `DROP` and `WHERE` have fixed roles. Concatenation trusts the input to contain none of them — an impossible guarantee, since even a benign user may type a quote. Data meant as data becomes code.

The fix is the *parameterized query*, or *prepared statement* (Figure 2). The statement holds a placeholder, `$1`, instead of the value — a *parameter*, a slot later filled with data unknown when the code was written — and contains no user data, only tables, conditions and operations. The call passes two things to the database: the statement and, separately, the values. The database binds each value into its slot purely as data, so a quote inside it closes nothing and no part of it is read as a command; the attack string of Section 2.2 becomes an odd string that matches no email. Validation should already have rejected it as not an email (see Chapter 5), but if validation is weak the driver still protects the database. Every driver supports parameters and ORMs use them by default; an application is vulnerable only if it deliberately builds raw query strings. Placeholders stand only for values: identifiers such as table and column names or a sort direction cannot be parameters and must be chosen from an allow-list in code.

```
// vulnerable: the input is spliced into the SQL text
q = "SELECT * FROM users WHERE email = '" + email + "'"
db.query(q)

// safe: statement and values travel separately
stmt = "SELECT * FROM users WHERE email = $1"
db.query(stmt, [email])
// "' OR '1'='1' --" is now just text: 0 rows
```

Figure 2: String concatenation versus a parameterized query (illustrative pseudo-code after the lecturer's sketch)

2.5 NoSQL injection

A document database does not remove the problem. MongoDB queries are JSON-like objects — to find a user, an object with an `email` field and the value — and may contain operators such as `$ne` (not equal), `$gt` (greater than) and `$exists`, written as nested objects with keys starting with a dollar sign. If the application passes JSON from the request straight into a query, the attacker controls the structure, not just the value: `{"$ne": ""}` in place of a string matches every user. The defence is the same idea: accept only the expected scalar type per field and never forward user-supplied objects as query structure.

2.6 Command injection

Command injection targets the operating system. Suppose users upload images that the server resizes or compresses with a CLI tool such as FFmpeg, called through an OS function with height and width options and an output file name chosen by the user. If the command is one string interpreted by a shell, the name `out.jpg; rm -rf /` ends the FFmpeg command at the semicolon and starts one that deletes everything from the root directory (Figure 3). Pipes

can feed output into further commands, ampersands can start background processes such as spyware, and every shell — bash, zsh, fish — has its own special characters and escape sequences for a skilled attacker to research.

The fix again separates structure from data. Node.js, Go, Python and other languages offer functions that take the program as one parameter and its arguments as a list; the arguments go directly to the new process without passing through a shell, so none of their characters is interpreted. One risk remains: an argument beginning with a hyphen may still be read as an option by the program (*argument injection*), so user-chosen file names should match a strict pattern or be generated by the server.

```
# vulnerable: one string interpreted by a shell
exec("ffmpeg -i in.jpg -vf scale=220:120 " + name)
#   name = "out.jpg; rm -rf /"
#   the shell runs ffmpeg ... out.jpg, then rm -rf /

# safe: program and argument list, no shell involved
execFile("ffmpeg",
         ["-i", "in.jpg", "-vf", "scale=220:120", name])
#   name reaches ffmpeg as a single argument
```

Figure 3: Command injection through a user-chosen output name, and the argument-array fix (illustrative)

2.7 The general rule

Whenever user input — anything typed or pasted — travels into another system, whether SQL, NoSQL, shell commands, HTML, LDAP or XML, it must remain pure data and never be executed or interpreted. The safe tools separate structure from data: parameterized queries for databases, argument arrays for processes. The mistake to avoid is building sensitive operations with string concatenation or template strings that mix user input with control syntax. The trigger for paranoia: whenever you build a string that another system will interpret and that contains user input, stop and find the parameterized alternative — it almost always exists.

3 AUTHENTICATION I: STORING PASSWORDS

3.1 Build or buy

Authentication verifies identity: it finds the row of the users table that belongs to whoever is making the request (see Chapter 4). If it fails, attackers impersonate users, read their private data, act on their behalf and, where payments are involved, steal their money; account takeovers are reported daily. The lecturer's first advice: if the budget allows, do not implement the whole authentication flow yourself in production — use an authentication provider. Signing a JWT, keeping it in a cookie and verifying it is easy; a production flow is not. It usually needs stateful sessions, so that users can see every device they are logged in on and revoke one or all at once, and the operator can log a user out everywhere — which brings a database, a Redis cache, decisions on where permissions live, timeouts and revocation from both stores. It needs social login (Google, GitHub, Twitter) through OAuth flows spanning backend and frontend, with redirects for a smooth experience. And it needs account linking: a user who signed up with email and password and later presses "Sign in with Google" with the same email must be recognised as the same user. Each costs weeks; cutting corners trades away security or user experience.

Providers such as Clerk solve these problems behind an API key and add role-based permissions and advanced security features. They may have UX issues or vulnerabilities of their own, but authentication is their whole business: a dedicated security team handles edge cases others would miss and reacts quickly to new attacks. Their bills grow with the user base, but by the time they reach, say, $10,000–20,000, the product should have the revenue and engineering bandwidth to migrate to an in-house system. Even with a provider, engineers must understand how authentication works, what can go wrong and how to integrate it securely.

3.2 Plain text

At sign-up a user sends an email and a password. The naive backend stores the password as sent, say `12345`, in a text column and compares it at login. But databases are breached every day, at companies of every size — through a vengeful insider, a breached third-party service and many other routes. Resetting every credential after the breach does not save the users: in the lecturer's figure, more than 70% of users reuse the same email and password on many sites, and few use password managers. The attacker replays the leaked pairs on payment, e-commerce and social sites and takes over accounts, usually before the users have heard of the breach. Even without a breach, plain text lets every developer, database administrator, contractor and freelancer with database access read every password.

3.3 Hashing

A hash function — not invented for passwords, but well suited to them — accepts input of any length and always returns output of a fixed length (if it promises 27 characters, every output has 27); it is deterministic, returning the same output for the same input every time; and it is one-way: recovering the input from the output is infeasible with current computing power. The server therefore stores only the hash — the lecturer's example uses bcrypt, whose call takes the password and a cost factor. At login it hashes the submitted password and compares the result with the stored hash. A breach now leaks hashes, not passwords.

3.4 Rainbow tables and salting

Because hashing is the industry standard and the popular algorithms are known, attackers precompute tables mapping common passwords — `12345`, `password`, `password@123` — to their hashes. After a breach they look up every leaked hash; since some share of users always picks common passwords and the function is deterministic, those hashes are reversed exactly.

Salting removes this sameness. For each user the server generates a random string, the *salt*, with a cryptographically secure pseudo-random number generator (available from libraries such as OpenSSL), stores it in the user's row beside the name, email and hashed password, and hashes the password concatenated with the salt. Two users with the password `12345` now have different hashes, and a rainbow table run against the leaked database finds zero matches. The lecturer notes that bcrypt was long the web's default password hash while Argon2id is today's standard, and recommends Lucia — once an authentication library, now a guide to implementing authentication with industry practices — for further reading.

A subtle point. Modern password hashers manage the salt themselves. bcrypt and Argon2id generate a fresh random salt on every hash and encode it, with the algorithm and cost parameters, in the output string (`$argon2id$v=19$m=…,t=…,p=…$<salt>$<hash>`), so one column suffices and the verify function reads everything it needs from the stored value. Salts are not secrets; their job is uniqueness. Strictly, a plain password-to-hash map is a *lookup table*; a *rainbow table* compresses it into chains of hash and reduction steps, trading computation for storage. Salting defeats both.

3.5 Brute force and slow hash functions

Salting still leaves *brute force*. GPUs, made ubiquitous by LLMs and rentable by the hour from cloud providers, compute billions of SHA-256 hashes per second. With a breached database an attacker brute-forces each user offline: take that user's salt, hash candidate passwords with it, compare with the leaked hash. At billions of attempts per second, every password up to eight characters, every common pattern and every dictionary word with common substitutions falls within days. General-purpose hashes such as MD5 and SHA-256 — fast by design, and used for instance to check the integrity of transferred files — are therefore unsuitable for passwords.

Passwords call for deliberately slow functions built for the purpose: bcrypt, scrypt and Argon2id. A *cost factor* (or *work factor*) sets how long each hash takes — 5 ms or 100 ms, as the system's computing budget and latency allow. At 400 ms per hash, a genuine login takes 400–600 ms, which a user barely notices, while an attacker's rate falls from a billion guesses per second to four or five, so that cracking which took days now takes decades or centuries (Table 2). Argon2id and scrypt are also memory-hard: each guess needs a configurable amount of memory, which limits how many guesses a GPU can run in parallel.

Storage	A breach yields	Broken by
Plain text	every password	nothing; reuse spreads it
Fast hash	hashes	rainbow-table lookup
Salted fast hash	hashes and salts	GPU brute force, billions/s
Salted slow hash	hashes, salts, cost	impractical: a few guesses/s

Table 2: Four ways to store passwords and what a database breach costs

4 AUTHENTICATION II: SESSIONS, TOKENS AND RATE LIMITS

4.1 Server-side sessions

After login, users cannot be asked for their password on every request; the server must remember that they authenticated — the *session*. On a successful login the server does three things (Figure 4). It generates a random *session identifier*, long (the lecturer says 128 to 256) and drawn from a cryptographically secure pseudo-random number generator, because whoever guesses a session ID can hijack that session. It stores the ID — in Redis for fast access, or in the primary database such as PostgreSQL or MySQL — with metadata: the owning user; the creation time, for timeouts and revocation; the expiry, say seven days; the IP address, so that a dashboard can show where the user is signed in; and the `User-Agent` header, which identifies browser and device — Chrome or Firefox, Android or iPhone. Finally it sends the ID to the browser in a cookie. The browser returns the cookie with every later request; the server extracts the ID, looks it up and knows which user is calling. JWTs (Section 4.3) keep nothing on the server but identify the user on each request in the same spirit.

```
POST /login     {"email": "...", "password": "..."}
  hash verified; session row created in Redis/Postgres:
  id=9f2c8e41  user_id=14  expires_at=now+7d
  ip=203.0.113.7  user_agent="Chrome on Android"
HTTP/1.1 200 OK
Set-Cookie: sid=9f2c8e41; HttpOnly; Secure; SameSite=Strict

GET /books      Cookie: sid=9f2c8e41
  sid looked up -> user 14 -> request proceeds
```

Figure 4: Creating and using a server-side session (illustrative; a real ID is far longer)

A subtle point. Session-ID strength is measured in bits of randomness, not characters: the lecturer's "128 to 256" is best read as 128 to 256 bits (32 to 64 hex characters), and OWASP asks for at least 64 bits of entropy. His comparison with the atoms in the universe is also off: $2^{128} \approx 3.4 \times 10^{38}$, far fewer than the roughly 10^{80} atoms of the observable universe (even $2^{256} \approx 1.2 \times 10^{77}$ falls short). The conclusion stands: at a billion guesses per second, hitting any one of a million live sessions in a 128-bit space takes more than 10^{15} years.

4.2 Cookie flags

Three cookie attributes belong on every production session cookie. `HttpOnly` hides the cookie from JavaScript. It matters if the site has an XSS vulnerability (Section 6): an attacker who runs script in a user's browser through the site can read local storage and every cookie lacking this flag — which is why keeping session IDs or JWTs in local storage is discouraged. `Secure` lets the browser send the cookie only over HTTPS. Plain HTTP can be read on public Wi-Fi in cafés, by malicious ISPs, by a router with spyware, or by anyone watching the hop with a tool such as Wireshark; over TLS the observer sees only ciphertext.

`SameSite` decides whether the cookie accompanies cross-site requests. With `Strict` it is sent only for requests originating from the site itself; even following a link to the site from elsewhere does not send it. With `Lax` it is sent for top-level navigations such as a direct link, but not for requests triggered by images or iframes on other sites — the tools of cross-site request forgery (Section 6.5). With `None` it is sent with every request, and browsers then require `Secure`. A session cookie should be `Strict` where possible and never `None`.

4.3 JSON Web Tokens

With sessions the server keeps the session data and the client holds only its ID. With a *JSON Web Token* (JWT) the session data itself goes to the client, which presents it with every request: the client tells the server who the user is. The token's construction prevents a user from simply claiming to be someone else. A JWT has three parts, coloured separately on jwt.io (Figure 5). The *header* describes the token and names the signing algorithm. The *payload* carries *claims*: `sub`, the subject — the user's platform ID, a UUID or any other identifier; `iat`, the issue timestamp, from which the server can judge expiry; and custom claims, which are just key–value pairs such as the user's name and an `admin` flag. The *signature* is computed over header and payload with a secret — a random, cryptographically secure string.

At login, password checking with hashing and salting is unchanged. The server then builds the token (the library adds `iat`), signs it with the secret held in an environment variable and returns it. The client stores it — in memory, sometimes in local storage, or in a cookie — and sends it back in the `Authorization` header. The server calls the library's `verify` function with the secret; a valid signature proves that the payload was not tampered with. Nothing is stored or looked up on the server, which removes the database or Redis round trip and makes horizontal scaling easier. (The standard `exp` claim of RFC 7519 lets libraries reject expired tokens automatically, and the verifier should accept only the algorithm it expects.)

```
header.payload.signature   (each part Base64url-encoded)

header     {"alg": "HS256", "typ": "JWT"}
payload    {"sub": "1234567890", "name": "John Doe",
            "admin": true, "iat": 1516239022}
signature  HMACSHA256(base64url(header) + "." +
                      base64url(payload), secret)
```

Figure 5: The three parts of a JWT, with a decoded sample like the one on jwt.io (illustrative)

4.4 The weaknesses of JWTs

Revocation is hard. Logging out a session means deleting its row; the next request fails. A JWT lives in the client, and the server cannot make the client delete it. If a user reports a compromised account and asks to be logged out of every device, a purely stateless system cannot help. Two workarounds exist. A *blacklist* records revoked tokens in the database or Redis and refuses requests that carry them — reintroducing a lookup. Or *short-lived access tokens with refresh tokens*: after login the server issues an access token valid for 5–10 minutes and a refresh token valid for one to seven days, depending on how paranoid the system is (Figure 6). The access token accompanies every request; once it expires the server answers `401 Unauthorized`, the client presents the refresh token, and the server checks its validity and expiry and issues a new access token and a new refresh token. A stolen access token works for at most five minutes, a stolen refresh token for at most a day. Neither workaround is perfect.

The payload is readable. It is merely Base64-encoded. In the demo, decoding it shows the full JSON — ID, name, admin flag, issue time; changing "John Doe" to "John Do", re-encoding and pasting it back into jwt.io yields "signature verification failed", so a server would reject the token. Integrity is protected, confidentiality is not: the payload must hold nothing that would help an outsider. **Storage is awkward.** Local storage is still the most common choice although any XSS payload can read it; the safe alternative, an `HttpOnly` cookie, brings the design back to cookie-based sessions.

```
POST /login    -> access (5 min) + refresh (1 day)
GET  /books    Bearer <access>   200 OK
     ... five minutes later ...
GET  /books    Bearer <access>   401 Unauthorized
POST /refresh  <refresh> -> new access + refresh
GET  /books    Bearer <new>      200 OK
```

Figure 6: Short-lived access tokens renewed with a refresh token (illustrative)

A subtle point. In the refresh flow the lecturer has the client keep the refresh token in local storage, which contradicts his own storage advice. The refresh token is the long-lived credential and the one an XSS payload should least be able to read: keep it in an `HttpOnly`, `Secure` cookie limited to the refresh endpoint, record it (or its hash) on the server so that it can be revoked, and rotate it on every use — if an already-used refresh token is presented again, treat it as stolen and revoke the whole chain.

4.5 Sessions or tokens?

The lecturer's verdict: unless there are specific scaling requirements — many servers that must each authenticate requests — prefer stateful sessions, and even then keep them in a distributed key–value store such as Redis. Sessions are simpler, revocation is immediate and the architecture suits most SaaS projects; the trade-offs of stateless tokens are not worth it. If JWTs are used: short expirations (minutes to hours, not days), refresh-token flows with server-side storage, and `HttpOnly` cookies rather than local storage. These remove the pain points but still feel like hacks next to sessions.

4.6 Layered rate limiting

Without rate limiting an attacker can send thousands of login attempts per second, cycling through username and password combinations. Either they find a match and take over an account, or the volume crashes the server. Rate

limiting is therefore mandatory — strict on authentication endpoints, more generous on the others, which also deserve a global limit. For authentication the lecturer recommends three layers (Table 3).

A *per-IP limit*, for example 10 login attempts per minute, stops simple automated attacks. But in organisations and universities many users share one outgoing IP address, and attackers rotate addresses through botnets, proxy networks and VPNs. A *per-account limit*, for example five failures in 15 minutes locking the account for 24 hours unless the user contacts support, stops attacks on one account from rotating IPs. It fails against *password spraying*: one password such as `12345` tried once on account A, once on account B and so on, from many IPs, so neither layer triggers. The last resort is a *global limit* on failed logins across the system, say 100 per minute: a spraying campaign crosses it, an alert fires, and the team can block the offending addresses or show a CAPTCHA to everyone to separate genuine users from bots. Together the layers prevent account takeover by brute force and crashes under load. (Hard lockouts can themselves be abused to lock victims out, so many systems prefer temporary, growing delays.)

Layer	Example	Stops	Misses
Per IP	10 attempts/min	simple scripts	botnets, proxies; hits shared IPs
Per account	5 failures/15 min, lock 24 h	IP rotation on one account	spraying across accounts
Global	100 failures/min	spraying, floods (alert)	blunt: affects all users

Table 3: Three layers of login rate limiting

5 AUTHORIZATION: CHECKING ACCESS WHERE IT HAPPENS

5.1 Where the confusion lies

Authentication identifies the user; authorization decides what that user may do (see Chapter 4). Engineers rarely confuse the definitions; vulnerabilities come from confusing the two in implementation. A request passes through layers (see Chapter 6): routing, the entry point; handlers, which parse and validate the body; services; repositories. An authentication middleware such as `requireAuth` sits at the routing layer on every route except login and other public routes; it checks the session cookie (stateful) or the JWT in the `Authorization` header (stateless) and rejects failures with `401`. Coarse authorization sits there too: is the user a member, admin or super admin — or, more granularly, does the user hold read or write permission on the entity books? Once a request has passed, a false sense of security sets in: the user is ours, so every handler, service and repository is assumed safe.

The lecturer's realistic case: a frontend lists books, with a filter that accepts several book IDs. The user enters 5; the frontend calls `GET /books?id=5`; the request passes authentication and the `read:books` check; the repository runs `SELECT * FROM books WHERE id = 5`, and the book is displayed. But book 5 was not created by this user and is not accessible to them. The query needed `AND user_id = ctx.userId`, matching the user ID that the authentication middleware stored in the request context. Read access to books is not access to every book: for every query, the developer must decide which entities this user may reach.

5.2 Broken object level authorization

This is *broken object level authorization* (BOLA): checks at the routing layer, none on individual objects in the repository. With books it looks harmless; with invoices or payment records, a script iterates over every possible ID, downloads every invoice and harvests every user's financial data, or tampers with invoice URLs and account numbers. The fix is to stay mindful of authorization down to the last point of access: not "fetch invoice 5" but "fetch invoice 5 that belongs, or is accessible, to this user", with the context's user ID in the query itself. This applies to every query — reads, updates, deletes and inserts.

5.3 403 or 404?

A subtler mistake comes from optimising the happy path's user experience. Some engineers fetch the invoice by ID, compare `ctx.user` with `invoice.userId` and return `403 Forbidden` on mismatch. The status is technically right, but it confirms that the guessed invoice exists. Looping over IDs such as 5, 17, 18, an attacker enumerates the invoices that exist and prepares the next layer of attack, typically *social engineering* — tricking people into revealing details. The better pattern is one query with both conditions, `WHERE id = 7 AND user_id = ctx.userId`: for someone else's invoice it returns no rows, which maps to `404 Not Found` (Figure 7). The attacker can no longer tell a missing invoice from another user's, and the information that would feed further attacks never leaves the server.

```
// leaks existence: 403 only for invoices that exist
inv = db.one("SELECT * FROM invoices WHERE id = $1", id)
if inv.user_id != ctx.userId: return 403

// owner in the query: missing and foreign look alike
inv = db.one("SELECT * FROM invoices " +
             "WHERE id = $1 AND user_id = $2",
             id, ctx.userId)
if inv == null: return 404
```

Figure 7: Ownership check after the fetch versus inside the query (illustrative)

5.4 Broken function level authorization

Broken function level authorization (BFLA) concerns functions rather than data — sensitive functions meant for administrators. `GET /admin/invoices` lets site admins review every invoice and support clients, so no ownership filter applies; any `WHERE` condition only filters, say, active or pending invoices. In the vulnerable design the admin panel's URL is simply shared with nobody. Members and admins log in through the same flow, so any logged-in user who learns the URL — for instance by watching traffic — can call the API and read every invoice. This is *security through obscurity*: hiding information and relying on as-

sumptions instead of enforcing access, a pattern still found in legacy systems. The fix belongs to the routing layer's authorization step. Members and admins may both hold `read:invoices`; what differs is the role, so a role middleware must require `admin` in addition to the permission check.

5.5 Predictable identifiers

Insecure direct object references (IDOR) and related access-control flaws thrive on sequential database IDs exposed in URLs such as `/invoices/102`: they reveal that 101, 103 and 109 exist and invite enumeration-based attacks. One prevention is UUIDs as primary keys — random, URL-friendly and widely used — though they carry performance costs worth studying (random keys scatter inserts across B-tree indexes; see Chapter 8). Unguessable IDs make enumeration impractical but do not replace the ownership check, because identifiers leak through shared links, logs and browser history.

5.6 Horizontal and vertical attacks

Instead of memorising named vulnerabilities, the lecturer sorts authorization attacks by direction. In a *horizontal* attack user A reaches user B's resources: the attacker's scope widens across users, usually because authorization is missing at the database level. BOLA belongs here, as does IDOR, which the lecturer treats as the same thing. In a *vertical* attack a user already inside the system widens their scope upwards, reaching sensitive functions reserved for admins: BFLA. Horizontal attacks are stopped by the owner condition in the query, vertical ones by the role check at the router.

5.7 A prevention framework

- **Centralize authorization.** One layer that every request passes through holds all authorization logic; scattered checks are applied inconsistently or forgotten, while one place is easy for the whole team to understand, maintain and scale.
- **Default deny.** Whatever the logic does not explicitly allow — a permission, a role, a resource — is denied, so new resources and endpoints are protected by default even when someone forgets to grant access to them.
- **Test authorization specifically.** Beyond the happy path, verify that user A cannot reach user B's resources, that a member cannot call admin functions and that an unauthenticated client reaches nothing protected. Automate these tests and run them in CI/CD on every change; manual checks are never comprehensive, and some later change will silently break a guarantee (see Chapter 23).
- **Audit logs.** Record every access to sensitive resources such as admin endpoints — who, what, when — and every failed authorization check, such as user A requesting B's resource, as a dedicated security event, so that someone probing the system is noticed in time.

6 CROSS-SITE SCRIPTING AND REQUEST FORGERY

6.1 Why XSS is dangerous

Cross-site scripting (XSS) is any attack in which an attacker gets their JavaScript executed in a genuine user's browser in the context of your site. Inside the page such a script can read everything displayed, including sensitive data; call APIs while impersonating the logged-in user, since requests carry the user's cookies and the script can read local storage and any cookie without `HttpOnly`; redirect the user to phishing pages that harvest credentials; and alter the page so that the user believes they are looking at something else entirely.

6.2 Stored XSS

In *stored XSS* the script is saved on the server and served to other users. Consider a blogging platform whose discussion board accepts comments in Markdown — a hash for a heading, hyphens for bullet points. To display them, Markdown is converted to HTML (in the React ecosystem by libraries such as remark and rehype, with plug-ins for customisation and styling), and the HTML — bullets become `<ul>` and `<li>` elements — is injected directly into a DOM node. Popular libraries handle the safety, but in a hand-rolled or legacy system the client may send the converted HTML, or raw HTML, to the server, which stores it and returns it to everyone who views the comments. An attacker who finds a hole in the conversion slips a `<script>` tag into the stored HTML, and it runs as the platform's own JavaScript for every reader (Figure 8): it can send their session cookies and local storage to a server the attacker controls, to view and hijack their sessions, or redirect them to a phishing page. React names its raw-HTML property `dangerouslySetInnerHTML` for this reason: whoever uses it is responsible for preventing XSS. Rendering user markup requires injecting HTML; the flaw is storing it unchecked. On receipt, the server must *sanitize* the markup — look for signs of an exploit, such as a script tag, and strip them before saving.

6.3 Other variants and the root cause

Reflected XSS carries the script in the URL, from which the page renders it back; *DOM-based XSS* arises in client code that writes attacker-controlled data into the page, for instance through `innerHTML`, much as above. All variants share one root cause: user-provided content treated as code rather than data. It is SQL injection again, with the boundary crossed into the browser's HTML and JavaScript instead of the database's SQL. The basic prevention is sanitization in the validation layer before storage. More broadly, whenever the backend handles user data — a search query, an HTML fragment, an image that may be huge — it must be deliberate about how the data is processed, stored or written to files, what access it is given and what resources the operation may consume; in the lec-

turer's estimate this habit alone avoids 99% of vulnerabilities.

> **A subtle point.** Stripping `<script>` tags is not sanitization. Browsers do not even execute script elements inserted through `innerHTML`; real payloads use event-handler attributes (`<img src=x onerror=…>`), `javascript:` URLs or SVG, which do run. User markup should pass an allow-list HTML sanitizer (such as DOMPurify) that keeps only known-safe tags and attributes, and all other user data should be *escaped* for its output context (`<` becomes `<`) so that it renders as text — which template engines and ordinary JSX rendering do by default.

6.4 Content Security Policy

Content Security Policy (CSP) is an HTTP response header in which the server tells the browser exactly which resources to trust and execute and which to block. A policy can allow scripts only from specific origins such as the site's own domains, forbid inline scripts altogether — which blocks the injected script of Section 6.2 — and restrict the domains images may load from, so that other images raise a CSP error. Even if an attacker injects a script, the browser refuses to run it. CSP is nevertheless not the prevention: the vulnerability itself must be fixed by sanitizing user content. CSP is the last line of defence, for the remote edge case that slips past everything else.

```
POST /comments  "Nice!<script>steal()</script>"
  stored as HTML, never sanitized
GET /posts/42   page HTML includes every stored comment
  every reader's browser parses the script and runs it
  steal() acts as the site: reads storage, calls APIs

Content-Security-Policy: script-src 'self'; img-src 'self'
  inline script refused: steal() never runs
```

Figure 8: Stored XSS through a comment, and the CSP header that blocks it (illustrative)

6.5 Cross-site request forgery

The lecturer treats *cross-site request forgery* (CSRF) briefly: with current libraries and frameworks he considers it no longer a major threat, although legacy systems and outdated practices keep it alive. It exploits the cross-site nature of cookies: the browser stores cookies per site and attaches them automatically to every request to that site, even when the request originates from another site. A user logged into `bank.com` holds its session cookie. They then visit `evil.com`, through a phishing link or ordinary browsing, whose page auto-submits a hidden form, or uses an image tag, to send a request to `bank.com`. It is still the user's browser making the request, so it attaches the bank's cookie; the bank sees an authenticated request and performs the destructive action (Figure 9).

Two mechanisms make CSRF minor today. The first is the `SameSite` attribute of Section 4.2: `Strict`, or `Lax` if frontend and backend live on different domains; modern browsers apply `Lax` by default, so unless the cookie is set to `None` it is not sent with the forged request. The second is CORS: not a security mechanism in itself, but the backend sends CORS headers only for its own frontend's origin, and without them the browser blocks the request — a further, weaker line of defence.

```
1  user logs in to bank.com: session cookie for bank.com
2  user opens evil.com (phishing link, ordinary browsing)
3  page auto-submits a hidden form to bank.com:
     POST https://bank.com/transfer  to=evil&amount=1000
4  browser attaches the bank.com cookie automatically
5  bank.com sees a valid session and runs the transfer
```

Figure 9: Anatomy of a CSRF attack without `SameSite` protection (illustrative)

> **A subtle point.** `SameSite` compares *sites* (scheme plus registrable domain), not origins: `app.shop.com` and `api.shop.com` are one site, so even `Strict` cookies flow between them. A frontend and API on genuinely different domains are cross-site; a `Lax` cookie then accompanies only top-level navigations, not the frontend's `fetch` calls, so such a setup needs `SameSite=None; Secure` plus an explicit CSRF defence such as an anti-CSRF token. Lax-by-default is the behaviour of Chromium-based browsers, not of all browsers, so set the attribute explicitly, and never let a `GET` change state. CORS, finally, only decides whether a page may *read* a response: a plain cross-site form `POST` is still sent and executed; only requests that need a preflight (JSON bodies, custom headers) are held back.

7 MISCONFIGURATION

7.1 Secrets

Misconfiguration opens as many holes as faulty code, and secrets management is the most abused area. *Secrets* are API keys, database passwords, encryption keys, JWT signing secrets and other authentication secrets. A secret in source code is committed to git and pushed to GitHub, Bitbucket or GitLab, where everyone with access to the repository can read it — and use it to open the database, call external services, and tamper with user data through the authentication secrets and encryption keys. Secrets belong in environment variables or in an external secret manager such as AWS Systems Manager Parameter Store or HashiCorp Vault (see Chapter 13).

> **Warning.** Deleting a committed secret in a later commit does not remove it: it stays in the repository history for anyone who looks. The moment a secret reaches version control, rotate it — revoke the old value and issue a new one.

7.2 Debug output in production

During local development the log level is usually *debug*: the terminal receives detailed errors with stack traces, which reveal function names, file names and code structure; the exact SQL queries; database pool configuration and transaction details; sometimes sensitive user data printed while debugging. In production the level should be *info*, at which none of these lines is emitted. If production is not explicitly set to a non-debug level, all of it reaches the production logs, and a breach of those logs tells the at-

tacker how the code is structured and exposes sensitive facts about users and the database. Engineers should know the log level of every environment — production, staging and local (see Chapter 14).

7.3 Security headers

Besides CSP, several HTTP response headers close whole classes of problems. `X-Frame-Options`, properly set, stops other sites from embedding yours in an `<iframe>`. Without it, an attacker can show your site inside a frame on a page they control; the frame's content stays isolated from them, but they can overlay or disguise it so that the user's clicks land on your buttons (*clickjacking*), or surround it with their own scripts and inputs to phish credentials. (The CSP directive `frame-ancestors` is the modern equivalent.) Setting every header by hand is unnecessary: practically every web framework — Express in Node.js, Echo and Gin in Go, Django in Python — offers security middleware (for Express, the Helmet package) that applies the industry-standard headers in a one-line change.

8 BOUNDARIES AND LAYERS

8.1 Data crossing boundaries

Every vulnerability in this chapter, and the thousands not covered, concerns the same thing: **boundaries** — between systems, privilege levels, programming languages, markup structures, trust zones. Each one met here occurred at a crossing: in SQL injection untrusted input crossed into the database's query language; in XSS a user's Markdown crossed into HTML; in broken authorization requests crossed the line between what different users may access. Hence three questions to ask repeatedly while writing code: *Where is data crossing a boundary? What assumptions am I making about it? What if those assumptions are wrong?* Asked consistently, they avoid — in the lecturer's estimate — 99% of the vulnerabilities modern backends suffer from.

8.2 Defence in depth

No single defence is perfect, so defences come in layers (Table 4). **Input validation** at the entry point, right after route matching, checks everything with as many conditions as possible, so that data reaching the handler has exactly the expected structure (see Chapter 5). **Parameterized operations** — explicit-parameter APIs for databases and for OS-level work such as file operations — replace commands built by concatenation. **Authorization at the point of access**: BOLA arose because access was checked at the router while data was fetched in the repository, so the check belongs where the access happens, not layers upstream. **Security headers and policies** — CORS, CSP and the rest — are not robust on their own but limit the damage when an attack passes the first lines. **Monitoring and logging** record suspicious activity, ship it to the logging service and raise alerts (see Chapter 14). A weakness in one layer then does not suffice: an attacker must bypass all of them at once. Security is therefore not a technique learned once but a paranoid mindset brought to every line of code — always think about boundaries and assumptions.

Layer	Role	Where
Input validation	only expected shapes pass	Chapter 5
Parameterized operations	input stays data	Section 2
Authorization at access	owner and role checked where data is touched	Section 5
Headers and policies	CSP, CORS, framing rules limit damage	Sections 6–7
Monitoring and logging	probing seen, alerts raised	Chapter 14

Table 4: Layers of defence in a backend

8.3 Further study

The lecturer recommends two free resources. The *PortSwigger Web Security Academy*, from the makers of the penetration-testing tool Burp Suite, explains vulnerabilities with theory and practical labs: SQL injection, XSS, CSRF, clickjacking, server-side request forgery (SSRF), OS command injection, authentication and OAuth flaws, JWT attacks and more. *OWASP*, a security foundation, publishes the OWASP Top 10, revised every few years; in the edition shown, Broken Access Control — covering BOLA and BFLA — tops the list, with Security Misconfiguration and Injection also present, each with its severity and real-world cases. The *OWASP Cheat Sheet Series* treats single topics, such as Authentication (good and bad practices, threats) and Session Management (standards for session IDs and why they must come from a cryptographically secure generator). Coming at security from many domains builds the mindset this chapter is about.

9 CONCLUSION

- Every vulnerability starts with an assumption: clean input, honest users, one's own frontend, an unopened network tab.
- Injection is code–data confusion; parameterized queries and argument arrays keep user input as data in SQL, NoSQL and the shell.
- Store passwords with a slow, salted, purpose-built hash (Argon2id or bcrypt); plain text and fast hashes fall to reuse, rainbow tables and GPUs.
- Prefer server-side sessions in `HttpOnly`, `Secure`, `SameSite` cookies; JWTs trade revocation for statelessness.
- Rate-limit logins in layers — per IP, per account and globally — against brute force, spraying and overload.
- Check authorization at the point of access: scope every query by owner, answer 404 rather than 403, and require roles for admin functions.
- Centralize authorization, deny by default, test it automatically and log every failure.
- XSS is injection into the browser: sanitize user markup and add CSP as a last line; `SameSite` cookies blunt CSRF.

- Keep secrets out of git, debug logs out of production and security headers on — and think in boundaries and layers.

APPENDIX A REVIEW QUESTIONS

1. A reporting endpoint accepts `GET /reports?sort=created_at&dir=desc`. After a security review, a developer rewrites its query as `SELECT * FROM reports WHERE owner_id = $1 ORDER BY $2 $3` and passes the owner ID, the sort column and the direction as parameters. Predict what the database does with this statement, and with `sort` set to `created_at; DROP TABLE reports`. Then design a version that is both safe and actually sorts.
2. A breached users table stores SHA-256(password + salt), with a 16-byte random salt per user, for 2 million users. The attacker's rented GPUs compute 10 billion SHA-256 hashes per second in total. Estimate how long it takes to test a list of 10^9 likely passwords against every user, and repeat the estimate for Argon2id tuned to 400 ms per hash. Your login service has 8 cores and peaks at 150 logins per second: plan the migration to Argon2id, including the cost factor and the fate of existing hashes, without forcing a global password reset.
3. Your API issues 24-hour JWTs that the single-page app keeps in local storage; there are no refresh tokens and no server-side state. At 09:00 a customer reports a stolen laptop and asks to be logged out everywhere immediately. Explain what you can and cannot do at 09:01, then redesign the scheme so that the next such request takes effect within one minute while adding as little per-request load as possible, and state what your design costs per request.
4. In one hour the login endpoint receives 40,000 failed attempts from about 8,000 IP addresses; no IP exceeds 6 attempts, no account receives more than 2, and most attempts use the password `Summer2024!`. The limits are 10 attempts per IP per minute and 5 failures per account per 15 minutes, and no alert fired. Diagnose the attack, explain why each existing control stayed silent, and specify a control that would have fired — including what it would cost genuine users while the attack lasts.
5. A handler reads `inv = repo.findById(id); if (inv.userId != ctx.userId) return 403; return inv`, and the same service exposes `PATCH /invoices/:id`, backed by `UPDATE invoices SET status = $1 WHERE id = $2`. Invoice IDs are sequential integers. List, in the order an attacker with an ordinary account would exploit them, every weakness here; rewrite the two repository functions and their status codes; and name the automated tests that would have caught each weakness in CI.
6. An admin page at an unlisted URL calls `GET /admin/invoices`. Members and admins both hold `read:invoices`, and the frontend hides the admin link from members. Predict the response when a member with a valid session calls the endpoint directly, classify the flaw as horizontal or vertical, and describe how a centralized, default-deny authorization layer would treat this endpoint and a new `/admin/refunds` endpoint added next month without any explicit rule.
7. A comment service removes `<script>` tags with a regular expression before storing HTML, and pages carry `Content-Security-Policy: script-src 'self'`. An attacker posts an image tag whose `onerror` attribute sends `document.cookie` to another domain. Trace what happens in a reader's browser and which layer stops the attack; then state what the attacker obtains if the policy also allowed `'unsafe-inline'` and the session cookie lacked `HttpOnly`, and what should replace the regular expression.
8. The frontend runs on `app.shop.com` and the API on `api.shop.com`, with the session cookie set to `SameSite=Lax`. A page on `evil-deals.com` auto-submits a form `POST` to `api.shop.com/orders`. Predict whether the cookie is sent and whether CORS prevents the order. The company then moves the frontend to `shopapp.io` but keeps the API: predict what breaks for genuine users, and choose cookie attributes and a CSRF defence for the new layout.

APPENDIX B GLOSSARY

Term	Meaning
Injection	Untrusted input parsed as code by another interpreter (SQL, shell, browser)
Parameterized query	Statement with placeholders; values sent separately and bound as data
Salt	Random per-user value hashed with the password so equal passwords hash differently
Rainbow table	Precomputed structure mapping common passwords to their hashes
Work factor	Tunable cost that makes each password hash deliberately slow
Password spraying	Trying one common password once against many accounts
Session identifier	Random secret in a cookie, keyed to session data stored on the server
Cookie flags	`HttpOnly`: hidden from JS; `Secure`: HTTPS only; `SameSite`: limits cross-site sending
JWT	Signed token carrying readable claims such as `sub` and `iat`
Refresh token	Long-lived credential exchanged for new short-lived access tokens
BOLA / IDOR	Reaching another user's object because ownership is not checked
BFLA	Reaching privileged functions because the role is not checked
Default deny	Anything not explicitly allowed by the authorization layer is refused
XSS	Attacker JavaScript running in a user's browser in the site's context
CSP	Response header listing the sources from which the browser may run scripts and load resources
CSRF	Forged cross-site request that carries the victim's cookies

APPENDIX C FURTHER TOPICS

Server-side request forgery (SSRF)

SSRF arises when the backend fetches a URL that the user controls — an image import, a link preview, a webhook target — and the attacker points it at addresses only the server can reach, such as internal admin services or the cloud metadata endpoint `169.254.169.254`, which may hand out credentials. The server's network position turns a harmless-looking feature into a door past the firewall. Defences are allow-lists of schemes and hosts, resolving the host and rejecting private and link-local ranges (again after every redirect), and egress rules that limit what the application server may reach. It applies wherever a request parameter decides what the server downloads, including webhooks (see Chapter 26).

Session fixation and ID rotation

In session fixation an attacker plants a session ID they know in the victim's browser before login; if the server keeps that ID after authentication, the attacker shares the authenticated session. The defence is to issue a fresh session ID at login and at every privilege change, invalidating the old one, alongside idle and absolute timeouts. It matters for every cookie-based session system, and framework defaults should be checked rather than assumed.

Multi-factor authentication and passkeys

A second factor — a one-time code from an authenticator app or a hardware key — means that a password leaked or reused elsewhere no longer suffices for account takeover. Passkeys, built on WebAuthn, replace the password with a public-key credential bound to the site's origin, which resists phishing and leaves nothing reusable in the server's database. For a backend engineer this changes the login flow, account recovery and the threat model of credential stuffing. It applies first to admin accounts and to any account that can move money.

HTTP Strict Transport Security

The `Strict-Transport-Security` header tells the browser to use only HTTPS for the domain for a stated period, so even a typed `http://` address never produces a plain-text request that an attacker on the network could intercept or downgrade. It complements the `Secure` cookie flag, which protects only cookies, and domains can be added to browsers' preload lists to protect the very first visit. It applies once the whole site, including subdomains when `includeSubDomains` is set, is served over HTTPS (RFC 6797).

JWT verification pitfalls

Well-known JWT attacks exploit lax verification: accepting tokens whose header says `"alg": "none"`, algorithm confusion in which an RSA public key is used as an HMAC secret, weak HMAC secrets brute-forced offline from a single captured token, and missing checks of `exp`, `iss` and `aud`. The defence is to pin the expected algorithm and key per issuer, validate the registered claims, use long random secrets and rely on maintained libraries. It applies whenever a backend verifies tokens, including tokens issued by an external authentication provider.

Mass assignment

Mass assignment — in OWASP's API list, broken object property level authorization — occurs when request JSON is bound directly onto a model, letting a user set fields such as `role`, `isAdmin`, `balance` or `ownerId`; the reading side of the same flaw returns whole objects and leaks fields such as password hashes. The defence is explicit input and output models (DTOs) that list exactly which properties may be written and returned. It applies to every ORM and framework with automatic model binding.

Path traversal

When a user-supplied file name is joined to a base directory, sequences such as `../` can escape it and read or overwrite arbitrary files, including configuration that holds secrets. The defence is to generate file names on the server, or to canonicalize the final path and verify that it still lies under the base directory; object storage keys avoid the file system altogether (see Chapter 20). It applies to uploads, downloads, report exports and any feature that picks files by name.

Vulnerable dependencies and the supply chain

Most of a backend's code comes from third-party packages, and a known vulnerability in a transitive dependency, a typosquatted package name or a compromised maintainer account is as exploitable as a bug in one's own code; OWASP lists vulnerable and outdated components among its top risks. Lockfiles, automated dependency scanning in CI, a small dependency footprint and prompt patching are the defences. It applies continuously, not only at release time.

User enumeration through responses and timing

Login, sign-up and password-reset endpoints that answer differently for unknown and known accounts — "no such user" versus "wrong password", or a fast reply when no hash is computed — reveal which emails are registered, the same leak as a 403 on someone else's invoice. Generic messages, identical status codes and a dummy hash computation for unknown users remove the signal, and secrets such as tokens should be compared in constant time. It applies to every endpoint that takes an identifier from an unauthenticated client.

Scaling & Performance, Part 1 — One-Page Overview

Chapter 17 · every section and sub-section of the paper at a glance

1 Introduction: What "Fast" Means

Performance is judged by latency, the time a user waits from an action to its result.

- **1.1 The journey of a request** — Browser, server, database, email API and back again.
- **1.2 Latency as the users' metric** — When users say "slow", they mean high latency.

2 Latency, Percentiles and Throughput

Latency varies per request, so it is described by percentiles and read together with throughput.

- **2.1 A distribution, not a number** — Caches, CDNs and concurrent load make every request differ.
- **2.2 Why averages mislead** — 99% at 50 ms plus 1% at 5 s still averages 100 ms.
- **2.3 Percentiles** — P50, P90, P99: the share of requests at or below a value.
- **2.4 Why the tail matters** — The slowest requests carry the heaviest work and the paying users.
- **2.5 Throughput** — Requests per second; can the system survive Black Friday?

3 Utilization: Why Latency Explodes Near Capacity

Queueing keeps latency flat at low utilization and makes it soar near 100%, so systems need headroom.

- **3.1 The ice-cream shop** — Same 2-minute service, much longer wait: the queue.
- **3.2 Utilization and the latency curve** — Latency bends sharply upwards as utilization nears 100%.
- **3.3 The highway** — 50% smooth, 80% slower, 90% erratic, 100% standstill.
- **3.4 Headroom and bursty traffic** — Run at 60–80%; bursts overshoot any average.

4 Finding the Bottleneck: Measure, Do Not Guess

Timing logs, profilers and distributed tracing show where the time goes before any fix is chosen.

- **4.1 The bottleneck** — Stock remedies (cache, upgrade, more servers) often miss it.
- **4.2 A week spent on the wrong fix** — Redis for a 10 ms query; a synchronous log write cost 500 ms.
- **4.3 Profiling and flame graphs** — Sampled call stacks; wide frames are where time goes.
- **4.4 I/O-bound work and distributed tracing** — A trace shows 2 ms in code, 800 ms in the query.

5 Database Performance

N+1 queries, missing or excessive indexes and connection setup are the usual database costs.

- **5.1 The usual suspect** — Heavy work, but not guilty by default.
- **5.2 The N+1 query problem** — One query per item: 1,000 × 5 ms = 5 s.
- **5.3 N+1 in the server and the ORM** — Batch or eager-load; read the SQL the ORM emits.
- **5.4 Indexes and the full table scan** — A B-tree catalogue instead of reading every row.
- **5.5 The price of an index** — Space and slower writes; index the obvious, then measure.
- **5.6 Composite and covering indexes** — Column order matters; index-only reads.
- **5.7 Finding the missing index with EXPLAIN ANALYZE** — Find the Seq Scan, add the index, confirm.
- **5.8 Connection cost and pooling** — Handshake, TLS, memory and limits; a pool reuses connections.
- **5.9 Internal and external pools** — Per-replica pools overrun the limit; PgBouncer caps it.

6 Caching for Performance

A cache hides expensive work but raises questions of invalidation, placement, pattern and hit ratio.

- **6.1 The idea** — 800 ms through the database, 50 ms from Redis.
- **6.2 Cache invalidation** — TTL needs a good guess; event-based needs discipline.
- **6.3 Local, distributed and multi-level caches** — Local maps diverge; Redis costs a round trip.
- **6.4 Caching patterns** — Cache-aside, write-through, write-behind.
- **6.5 The cache hit ratio** — Driven by TTL, cache size and access patterns.

7 Vertical and Horizontal Scaling

A bigger machine is simple but bounded; more machines remove the ceiling, in theory.

- **7.1 The capacity question** — Traffic outgrows the server; two ways to add capacity.
- **7.2 Vertical scaling** — More cores, RAM, disk and network; the code stays the same.
- **7.3 The limits of vertical scaling** — Hardware ceiling, single point of failure, one location.
- **7.4 Horizontal scaling** — More copies: capacity, redundancy, regions — in theory.

Scaling and performance at a glance — measure latency as percentiles next to throughput, keep utilization below the knee of the curve, find the bottleneck by measurement, remove the database's usual costs and cache what is expensive, and only then choose between a bigger server and more servers. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Scaling and Performance, Part 1: From Latency to Vertical and Horizontal Scaling

Chapter 17 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Latency, the time a user waits between an action and its result, is a distribution rather than a number, so it is described by percentiles such as P50, P90 and P99 and read together with throughput. Utilization explains why latency stays low under light load and climbs steeply near full capacity, which is why production systems keep headroom for bursty traffic. Bottlenecks must be located by measurement, with timing logs, profilers and flame graphs, and distributed tracing, before any fix is chosen. On the database side the chapter covers N+1 queries, the benefits and costs of indexes, composite and covering indexes, EXPLAIN ANALYZE, and connection pooling with internal pools and PgBouncer. It then treats caching through invalidation, cache placement, caching patterns and the hit ratio, and closes with vertical and horizontal scaling and the limits of a single machine.

Keywords — latency percentiles, throughput, utilization, profiling and tracing, N+1 query, database indexes, connection pooling, caching, vertical and horizontal scaling

1 INTRODUCTION: WHAT "FAST" MEANS

Scaling and performance are among the most used words in backend engineering, and they span a wide territory: in the browser they concern rendering and interactivity, in infrastructure they concern machines and networks. This chapter stays in the server context. Its goal is an intuition rather than a catalogue of techniques — how systems behave under load, where bottlenecks hide, and how to think clearly when a loaded system seems to collapse — and that intuition applies to any system, not only to web servers.

1.1 The journey of a request

A user presses a button in a web application. The browser sends a request over the internet to the server behind the application. The server processes it: most likely it talks to the database, to save something or to read information about the user, and if needed it calls other APIs — for instance an email service such as Resend to send the user an email. Then it sends a response, usually JSON, back over the internet; the browser parses it, runs its JavaScript and renders, say, a few cards. The whole time between the press and the cards appearing is the *latency* of the interaction (Figure 1).

```
t = 0  user presses the button
       browser --request over the internet--> server
                 server --SQL query--> database
                 server --POST /emails--> email API
       browser <--JSON response-------------- server
t = L  browser parses the JSON, renders the cards

latency L: everything between the press and the cards
```

Figure 1: The journey of one request and the latency the user perceives (illustrative)

1.2 Latency as the users' metric

Latency is the first and most important performance concept because it is a measurable quantity: performance needs a unit that can be stated in numbers, not figurative words such as "speed". It is also what users perceive. When someone says an application is slow — whether or not they know the word — they are talking about latency.

2 LATENCY, PERCENTILES AND THROUGHPUT

2.1 A distribution, not a number

Latency is not a single figure such as "500 ms"; it varies from request to request. One request completes in 50 ms, another in 200 ms, because real traffic crossing the internet is never as predictable as a controlled measurement. The fast request may have been answered by a CDN or by an in-memory cache such as Redis that already held the result — a store much faster than the database and possibly much closer to the user — while the slow one travelled all the way to the server and ran a database query. Or no cache was involved: the fast request reached an idle server, and the slow one arrived while the server was already processing about 50 other requests concurrently. With real traffic, any of these can be the cause.

2.2 Why averages mislead

The obvious summary is the mean: 50 ms and 200 ms average 125 ms. The mean is a useful statistic in many contexts, but for performance it is misleading — practically useless, in the lecturer's view — because what matters is the variation. Suppose 1,000 requests have a mean latency of 100 ms, while 99% of them finished in under 50 ms and the remaining 1% took about 5 seconds (0.99×50 ms + $0.01 \times 5{,}000$ ms is roughly 100 ms). The mean describes no real request, and the 1% are real users with a terrible experience. Scaled to 1 million requests a day, 10,000 requests wait 5 seconds: 10,000 times a user presses a button and watches a loader for 5 seconds before any data appear. The mean still reports a healthy system, so it never reveals that disappointment, nor the limits of the system that cause it. Performance is therefore discussed in *percentiles*, the basic vocabulary of every conversation about measuring performance and scaling.

2.3 Percentiles

Percentile latency. The p-th percentile latency, written Pp, is the value at or below which p% of the measured requests fall. P50 is the median; P90, P95 and P99 describe the slow tail.

Three values recur in every discussion of latency, optimization and scaling (Table 1). A P50 of 400 ms means half the requests are served within 400 ms. A P90 of 900 ms means 10% of requests take 900 ms or more. A P99 of 2 seconds means 1% take 2 seconds or more — equivalently, 99% are faster than 2 seconds. A quick reading rule is to subtract the percentile from 100: P99 concerns the slowest 1%, P90 the slowest 10%. With requests sorted by latency the curve is flat for most of its length and rises steeply at the right end: most users feel little delay, a few feel the maximum.

Percentile	Example	Reading
P50 (median)	400 ms	half the requests finish within 400 ms
P90	900 ms	the slowest 10% take 900 ms or more
P99	2 s	the slowest 1% take 2 s or more

Table 1: The lecturer's example percentiles and how to read them

A subtle point. The lecturer reads a P90 of 900 ms as "10% of users have a 900 ms delay". Strictly, the slowest 10% of *requests* take at least 900 ms, and P90 says nothing about how much longer; P99 and the maximum show that. Counting requests rather than users also matters: a page that triggers 20 backend calls, each with a P99 of 1 s, meets at least one slow call on $1 - 0.99^{20}$, about 18%, of page loads (Dean and Barroso, 2013).

2.4 Why the tail matters

In practice backend engineers watch P95 and P99 most closely, and not only out of care for unhappy users. The slowest requests are where the heaviest work happens: the most complex business logic, the most complex database queries, and synchronous calls to external services such as sending emails or calling webhooks. Those requests usually come from the most valuable customers. A user who pays or completes a purchase triggers more complex queries and more business logic than one who only browses, so the slow tail is disproportionately made of paying users.

2.5 Throughput

Latency measures how long one request takes. *Throughput* measures how many requests the system processes in a period of time, usually in requests per second (RPS) or per minute; it is the system's capacity. The two must be read together, because latency depends on load: at 10 requests per second a system may answer in a comfortable 150 ms, and at 1,000 requests per second the same system may take 2 seconds.

Together they answer the practical questions of capacity. Can the platform — an e-commerce store, a learning-management system, a site selling courses — survive the peak of Black Friday, when traffic grows in one day to hundreds, thousands or millions of requests? Can it absorb an email campaign, a mention on a podcast or a feature on a popular website or blog? How many concurrent users and how many requests per second can it support before it needs more resources? The relationship between the two quantities is not intuitive: as throughput rises, latency first grows slowly and then, beyond a certain point, abruptly.

3 UTILIZATION: WHY LATENCY EXPLODES NEAR CAPACITY

3.1 The ice-cream shop

On a Sunday evening you walk into an ice-cream shop and find it empty. You ask for an ice cream, pay and receive it at once: nobody else was there, so the worker behind the counter was barely busy. This is the case of low utilization and low latency. On a Tuesday at lunchtime the same shop has a long queue, and you are somewhere in it. The worker is exactly as fast as on Sunday — each ice cream still takes about 2 minutes — yet your waiting time has grown dramatically, because you must wait until everyone ahead of you has been served. The speed of the work has not changed; the time you perceive has.

Servers behave the same way with their CPU and memory. Under low usage a request arrives, the processor picks it up and the response goes out almost immediately. As requests multiply, each must wait for those ahead of it to be processed, and together they form a queue. The higher the load, the longer the queue and the longer each request waits.

3.2 Utilization and the latency curve

Utilization is formally the percentage of a system's capacity that is in use, actively working, at a given moment. At 0% the system is idle and uses none of its resources; at 100% it is exhausted, can take on no more work and is on the verge of collapse. The intuitive expectation is that latency rises linearly with utilization. It does not: latency rises slowly at first, and as utilization approaches 100% the curve bends sharply upwards — the lecturer calls the growth exponential (Figure 2). This counter-intuitive relationship between usage and latency is the core idea of the lecture's first part, and it produces the throughput behaviour described in Section 2.5.

```
util.   response time (S = time to serve one request)
  25%     1.3 S  #
  50%       2 S  ##
  75%       4 S  ####
  80%       5 S  #####
  90%      10 S  ##########
  95%      20 S  ####################
  99%     100 S  ########################################  ...
```

Figure 2: Latency against utilization (illustrative: mean response time in the simplest single-server queue, M/M/1, which is S / (1 − utilization))

A subtle point. The curve is not literally exponential. In the simplest queueing model — random arrivals at a single server (M/M/1) — the mean response time is S / (1 − ρ), where S is the time to serve one request and ρ the utilization. It doubles between 50% and 75% utilization and grows without bound as ρ approaches 1, which near full load is steeper than any exponential.

3.3 The highway

A national highway offers a second, simplified picture (Table 2). At 50% of capacity every car has room to overtake and arrives at the estimated time. At 80% slowdowns appear: overtaking is no longer simple, at least not safely, and drivers think twice and plan before changing lanes. At 90% the flow becomes unpredictable. When the cars happen to be evenly spaced it is still smooth, but impatient drivers who will not give way form jams, so the flow depends on people's mood and on uncontrolled events; whenever someone slows down, changes lane or merges, a wave runs backwards through the cars behind. At 100% every spot is taken and nobody moves. Such analogies are simplifications, and in computing the behaviour feels less intuitive, but the relationship between utilization and latency is the same.

Load	On the highway	On a server
50%	room to overtake; on time	requests start at once
80%	lane changes need planning	queues begin to form
90%	smooth or jammed; braking ripples back	latency spikes, high variance
100%	every spot taken; nothing moves	requests stall; collapse

Table 2: Utilization on a highway and on a server

3.4 Headroom and bursty traffic

The conclusion every backend engineer reaches at some point is that a system cannot run at 100% utilization and be expected to work well, or at all. It needs reserve capacity that can absorb incoming requests. Real production systems typically run at 60–80% utilization, keeping at least about 20% as a buffer for traffic peaks.

The buffer is also needed because traffic is not linear. It is not a metronome delivering a fixed number of requests every second; it comes in *bursts*, a term that recurs in every discussion of performance and scaling. For a minute the server is flooded with requests, the next minute they are gone, and half an hour later another thousand arrive. Even when average traffic is well below capacity — 50% or even 40% — a burst can push the system past 100% instantly, and without headroom it fails. Utilization, its non-linear effect on latency and the burstiness of traffic form the mental model on which the vocabulary of vertical and horizontal scaling rests.

4 FINDING THE BOTTLENECK: MEASURE, DO NOT GUESS

4.1 The bottleneck

A slow system is slow because something specific in it causes the slowness; locating that part is called finding the *bottleneck*. Obvious as it sounds, the step is often skipped in favour of standard remedies. Latency appears, so caching is added, because every resource says caching cures latency. The database looks guilty, so PostgreSQL 16 is upgraded to 18, or the packages are updated. The server looks like the bottleneck, so more servers are added — horizontal scaling (Section 7.4). Sometimes these ready-made fixes help. More often days or weeks go into solving a problem that does not exist, the real bottleneck is ignored, and the system stays slow because its cause is still there.

4.2 A week spent on the wrong fix

An endpoint `GET /products/:id` returns one product and looks slow. (The lecturer first framed it as a `PATCH` that updates a product, then switched to `GET`, since caching an update makes no sense.) The database is the first suspect — "the database is always the problem" — so the team adds Redis to the stack, places a caching function in front of the query, spends a week on it and deploys it to production. The endpoint is still slow.

Only then does the team measure instead of assuming. It adds timing measurements to the code, at least around this one function: when the request arrives, when the database query runs, when the cache is called. The query turns out to take about 10 ms and the new Redis lookup about 5 ms. The measurements also expose a logging function in the service code that writes each record to a remote logging service such as Elasticsearch. Remote logging is normal and often necessary; the problem is that the call is synchronous. Instead of running concurrently or in parallel, it blocks the request until the remote service has accepted the record, and only then does the handler resume — about 500 ms per request (Figure 3; concurrency and parallelism are the subject of Chapter 19). The database was never the problem. Caching was added by rule rather than by measurement, and the real culprit went unnoticed for a week.

```
GET /products/5    timings logged inside the handler
  cache lookup (Redis)                  5 ms
  database query                       10 ms
  remote log write (synchronous)      500 ms
  ------------------------------------------
  the response waits for the log write to finish
```

Figure 3: What the timing measurements revealed for the slow product endpoint (illustrative layout, lecturer's numbers)

Many other non-obvious parts of a request's workflow can cause latency, yet the blame usually goes to the database, the indexing strategy or the caching strategy, because they are the obvious suspects. An API whose investigation starts at its database queries may in fact be slow in a JSON or XML serialization step; or the code may be fast while a

huge response payload makes network transfer the dominant cost. Never guess which component adds the milliseconds: always measure. Correct measurement is the basis of all performance, scaling and optimization work.

4.3 Profiling and flame graphs

Profiling measures where a program actually spends its time. A profiler attaches to the running program and, while it processes real requests, records samples of what happens in every corner of the system: which functions execute, when they start and when they return, from the beginning of a request to its end. The raw output is overwhelming — thousands of functions, each responsible for a share of the total time — so it is usually viewed as a *flame graph*. A flame graph draws call stacks, a call stack being the chain of functions currently executing with their inputs and outputs. Functions that consume more time are drawn wider, so they attract attention, and functions called by another function are drawn above it.

The first look at a profile is usually a surprise, because the bottleneck in most real programs is not where it was expected. The suspected CPU-heavy business logic is cheap, and the time goes to JSON serialization; the database queries are fine, and the real cost is an external API request made inside a loop. Such things appear only when the program is measured end to end.

A subtle point. The lecturer describes a flame graph as call stacks over time. In a classic flame graph the horizontal axis is not time: identical stacks are merged and sorted alphabetically, and a frame's width is its share of the samples; a left-to-right timeline is a different view, usually called a flame chart. Ordinary CPU profilers also sample only code running on the CPU, so time spent waiting for a socket or a disk is invisible to them — the reason they miss I/O. Serialization, by contrast, is CPU work, which is why it shows up so clearly in profiles.

4.4 I/O-bound work and distributed tracing

In the lecturer's experience profilers cope poorly with I/O-bound tasks — database queries, file processing, external API calls — whose time consists of issuing input or output and waiting for the result. Profilers are most useful for CPU-bound tasks, where the program spends processor cycles on loops and calculations. Yet most performance problems in backend servers are I/O problems, at the database, at an external service or elsewhere. Unless the application does image processing or machine-learning computation with heavy matrix manipulation, the culprit in a typical SaaS application is almost always I/O.

The tool for I/O is *distributed tracing*, part of observability (see Chapter 14). A trace follows one request through the system and records when it entered each component, when it started a database query, when it entered and left a function, when it called an external API, and how long each step took. Once tracing is set up, a request to `/products/5` may show only 2 ms in the API's business logic and about 800 ms in the database query (Figure 4), and the optimization effort can go straight to that query. Tracing is the quick way to see at which level and in which component the time is spent.

```
span                          0 ms                 802 ms
GET /products/5               [=========================]
  business logic (handler)    [=]                    2 ms
  SQL query: SELECT ...         [======================]
                                                     800 ms
```

Figure 4: A distributed trace of the product request (illustrative layout, lecturer's numbers)

5 DATABASE PERFORMANCE

5.1 The usual suspect

Databases are the bottleneck in most backend applications and among the first things examined during optimization, for good reasons. They do the heaviest work: they keep all data durably on disk so it is available whenever the system needs it; they serve inserts, updates and deletes; they guarantee that failures and errors do not lose data; they keep data consistent under concurrent reads and writes, with locking and other complex mechanisms; and they run complex queries over millions or billions of rows. All of that takes time, so the database cannot be blamed blindly for every problem. Three common, rather mundane issues account for most database performance problems: N+1 queries, indexing and connection cost.

5.2 The N+1 query problem

The pattern is easiest to see from the frontend. A React developer builds the home page of a blog that shows 20 posts, each with its author's name, but the list API does not return author details. The developer calls the list API once and then, in a loop, calls an author API once per post: 20 author calls plus the list, 21 calls for one page. This access pattern is the *N+1 query* problem: one query fetches N items, then N more queries fetch a detail for each item. For 100 posts it costs 101 calls, for 1,000 posts 1,001; the number of calls grows linearly with the number of items.

Two problems follow. First, the number of network calls should not be tied linearly to the number of items displayed; any engineer senses that a thousand requests to display a thousand items is wrong. Second, every query carries fixed overhead. The request and response cross the network between the application server and the database server; before that a TCP connection must be set up, unless a pool already holds one (Section 5.8); and the database must parse the query, plan its execution, execute it and return the results. Even if one query takes only 5 ms, which is extremely fast, 1,000 queries take 5,000 ms: users watch a loader for 5 seconds, not an acceptable experience by 2025 standards.

The fix needs no special machinery; it is batch retrieval. Collect all author IDs into one array or list — whatever structure the language offers — and fetch all the authors with one request. The page then needs two requests, one for the posts and one for their authors, whether it shows 20, 100 or 1,000 posts (Figure 5).

```
// N+1: one query for the list, then one per post
const posts = await db.select().from(postsTbl).limit(20);
for (const post of posts) {
  post.author = await db.select().from(users)
    .where(eq(users.id, post.authorId));     // 20 more
}

// batched: two queries for 20, 100 or 1,000 posts
const ids = [...new Set(posts.map(p => p.authorId))];
const authors = await db.select().from(users)
  .where(inArray(users.id, ids));

-- or a single SQL query with a join
SELECT p.id, p.title, u.name AS author
FROM posts p JOIN users u ON u.id = p.author_id
LIMIT 20;
```

Figure 5: The N+1 pattern and its batched fix (illustrative, Drizzle-style TypeScript and SQL)

5.3 N+1 in the server and the ORM

The frontend version only makes the pattern intuitive. The server stores no data itself; every post and every author it returns is fetched from the database. Relative to the database, the backend is the frontend, and that is where the N+1 problem traditionally arises: a `fetchPosts` function selected the posts with one query and then, in a `for post in posts` loop, selected each author from `users` by the post's `author_id`, one query per post.

ORMs made the problem common because they express data access through the types and constructs of the programming language. After `posts = await db.select()...` the code holds an ordinary array, and `for (const post of posts) await fetchAuthor(...)` looks like ordinary TypeScript. Behind the scenes each call becomes a raw SQL query, but because the code looks familiar nobody thinks about what it executes. That is why modern ORMs can print the SQL they generate: during development one can check whether the query pattern is the intended one, change it, or write raw SQL instead. Every mainstream ORM offers primitives for fetching related data in bulk (Table 3), and with raw SQL the answer is a join, inner or left. The rule is to tell the ORM to load all the required related data in bulk before the loop runs, so that two queries, or one query with a join, replace a thousand.

Stack	Bulk or eager loading
Django	`select_related` (foreign keys), `prefetch_related` (many-to-many)
Ruby on Rails	`includes`
TypeScript ORMs (Prisma, Drizzle)	joins (`leftJoin`), relation loading (`include`, `with`)
Raw SQL	`JOIN` (inner or left), `WHERE id IN (...)`
EF Core (.NET)	`Include`, `ThenInclude`

Table 3: Primitives that replace a query-per-item loop (the EF Core row is added for .NET readers)

5.4 Indexes and the full table scan

The most common source of database performance problems is indexes — more precisely, missing indexes. The method is to learn the application's data access patterns by observing it and to add indexes to the tables accordingly.

The classic analogy is a library with about a million books across shelves, sections, rooms and floors, but no catalogue recording where each book is. A reader asks for all books by John Green. The librarian's only option is to walk through the entire library, examine every book and hand over a box of the matching ones — perhaps three days of work. When the same reader, or another, asks for John Green again, the three-day adventure starts over, because the books went back on the shelves in random order. Examining every item to see whether it matches is a *sequential scan*, or *full table scan*: the database reads every row, tests it, and collects the matches one by one. It does not take three days, but on a very large table it typically takes about 1,000 ms to 2 seconds, still slow by modern standards.

The librarian's fix is a catalogue sorted alphabetically by author that records the location of each author's books; with it the books are collected in two or three minutes, depending on the size of the library. In a database the catalogue is an *index*, usually a B-tree (other kinds, such as hash indexes, exist). A B-tree index keeps a sorted copy of the indexed column's values, each with a pointer to the rows that hold it. With `books` indexed on `author_id`, a query for all books by author 3 follows the pointers straight to those rows: the database knows which rows to fetch instead of scanning every one. The lecturer's figures for a million-row table are about 4 seconds without the index and roughly 40–100 ms with it.

5.5 The price of an index

Indexes are not free and not a universal cure. The first cost is space: the sorted values and pointers are stored somewhere, and the index grows with the table. The second and more serious cost falls on writes. An index must stay synchronized with its table so that queries read correct data, so every `INSERT`, `UPDATE` or `DELETE` must also update each of the table's indexes. A team that indexes every column to "improve performance" will see write latency rise sharply, because each write now updates all those structures one after another.

The practical strategy (see also Chapter 8) has two stages. When a migration creates a table, add the one or two indexes that are obviously needed. The primary key needs nothing — PostgreSQL indexes it automatically, so lookups by `id` already use an index — but a foreign key such as `books.author_id` must be indexed by hand, and since "all books by this author" is so common it belongs in the table definition before any API is written. For the non-obvious cases — columns that may or may not appear in many queries or in joins, which also use indexes — wait and observe the running application. Logs and traces show which endpoints are hit most, and tracing may show, for example, that a frequent query is slow because the `isbn` column has no index; only then is `isbn` indexed. Because every index has overhead, be careful rather than generous with them.

5.6 Composite and covering indexes

A *composite index* spans several columns. If queries often filter on both `user_id` and `created_at`, one index on `(user_id, created_at)` serves them better than two single-column indexes. Column order matters: with `user_id` first, the index serves queries on both columns and also queries on `user_id` alone, its leading column, but not queries that filter only on `created_at`.

A *covering index* holds every column a query needs, so the database can answer from the index without returning to the table. The lecturer's example is a `departments` table of about 100 columns that the internal sales team often changes as clients require, while the web application displays only each department's name and needs its ID. A covering index on `name` that also holds the ID lets those queries be served entirely from the index. More columns can be added, but each one enlarges the index, so whether it pays off must be analysed case by case (Figure 6).

```
-- composite: serves (user_id) and (user_id, created_at),
-- but not a filter on created_at alone
CREATE INDEX idx_orders_user_created
    ON orders (user_id, created_at);
SELECT * FROM orders
 WHERE user_id = 42 AND created_at >= '2025-01-01';

-- covering: answered from the index, table untouched
CREATE INDEX idx_departments_name
    ON departments (name) INCLUDE (id);
SELECT id, name FROM departments ORDER BY name;
```

Figure 6: A composite and a covering index in PostgreSQL syntax (illustrative)

5.7 Finding the missing index with EXPLAIN ANALYZE

Distributed tracing identifies the slow query — say a complex `SELECT` with several joins and a `GROUP BY` — but not the column that needs an index. For that most databases offer `EXPLAIN ANALYZE`. Placed before a query, it makes the database report how it actually obtained the data: which table it started from, which tables it joined, which index it used for each fragment, the whole plan from start to finish. Look for tables read with a sequential (full table) scan instead of an index; if such a table holds thousands or millions of rows, that scan is a probable cause of the latency. Add an index on the relevant column, run `EXPLAIN ANALYZE` again and check that the plan now shows an index scan where the sequential scan was, and that the query is much faster (Figure 7). Tracing to find the exact query, then `EXPLAIN ANALYZE` to find the exact column, is the measured route to most database performance problems.

```
EXPLAIN ANALYZE
SELECT * FROM books WHERE author_id = 3;

-- before: all 1,000,000 rows are read and tested
Seq Scan on books  (actual rows=37 loops=1)
  Filter: (author_id = 3)
  Rows Removed by Filter: 999963

-- after adding an index on books (author_id)
Index Scan using idx_books_author on books
  (actual rows=37 loops=1)
  Index Cond: (author_id = 3)
```

Figure 7: Reading a plan before and after adding an index (illustrative, abridged PostgreSQL output)

> **Warning.** `EXPLAIN ANALYZE` really executes the statement. On an `INSERT`, `UPDATE` or `DELETE` it changes data, so wrap it in `BEGIN` … `ROLLBACK`, or use plain `EXPLAIN`, which shows the estimated plan without running the query.

5.8 Connection cost and pooling

Scaling a server brings the realization that database connections are not as cheap as they seem. Locally they cost so little that nobody thinks about them; at scale they become expensive. Opening a connection runs a sequence of steps: a TCP connection is established with the three-way handshake (SYN, SYN-ACK, ACK); the client authenticates if required and negotiates encryption — keys, ciphers and the rest of the TLS machinery; a session state for data access is set up; and the database allocates memory for the connection, on the order of a few megabytes. Each step costs time and resources, every time a connection is opened.

An application that opens a connection for each database operation and closes it right afterwards pays this setup latency again and again, on every HTTP request and every query. The second problem is the database's connection limit. A typical PostgreSQL deployment allows a few hundred connections — the lecturer cites 400–600, depending on the memory and CPU configured; the default `max_connections` is 100. On a peak day or during a sale, a server handling thousands or tens of thousands of HTTP requests per second or per minute easily exhausts that limit, and further requests fail.

A *connection pool* solves both problems and is a central concept in performance and scaling. Instead of talking to the database directly, the server talks to an intermediary that maintains a set of open connections, idle and ready. When the server needs the database it borrows a connection, runs its query and returns the connection. The pool does not close it but, depending on its configuration, keeps it open for a while so that the next query reuses it instead of creating a new one.

5.9 Internal and external pools

An *internal* pool is provided at the application level by practically every modern database driver: the driver keeps the pool inside the server process, and the code borrows and returns connections through it. It belongs to one server instance, and a scaled web application never has just

one: horizontal scaling adds copies of the server as traffic grows, each with its own pool. Suppose each server keeps a pool of up to 150 connections and, during a peak, the Kubernetes autoscaler adds two replicas. Three replicas can now open up to 450 connections; if the database handles only 300 and traffic keeps growing until the replicas hold 400 between them, the database fails because its connection limit is exhausted.

Hence *external* poolers. For PostgreSQL the popular one is PgBouncer, a lightweight, open-source connection pooler. In the same scenario the replicas no longer keep pools of their own: one PgBouncer maintains a pool of, say, 250–300 connections to the database, and every replica reaches it over the network. Because a single pool stands between all replicas and the database, a traffic surge can no longer exhaust the database's connection limit unnoticed (Figure 8). In production an external pooler such as PgBouncer is generally preferred to internal pooling, especially when the application's traffic surges are known.

```
internal pools: every replica brings its own
  app-1 [pool 150] ---+
  app-2 [pool 150] ---+--> PostgreSQL (limit 300)
  app-3 [pool 150] ---+     up to 450 requested: fails

external pooler: one shared pool for all replicas
  app-1 ---+
  app-2 ---+--> PgBouncer [pool 250] --> PostgreSQL
  app-3 ---+    extra clients wait here (limit 300)
```

Figure 8: Per-replica internal pools versus one external pooler (illustrative, lecturer's numbers)

6 CACHING FOR PERFORMANCE

6.1 The idea

When the queries are optimized, the relevant indexes exist and pooling is set up, yet the database is still the bottleneck, the obvious next step is caching. Its mechanics are covered in Chapter 9; here it matters as an integral part of system performance. The idea is simple: store the result of an expensive operation — here a complex database query — in Redis or another caching solution. The next request first checks whether the cache already holds the result and, if so, returns it directly. A path that takes about 800 ms through the database takes about 50 ms from the cache: one change cuts the perceived latency sixteen-fold.

6.2 Cache invalidation

The most troublesome part of caching is *cache invalidation*. A well-known saying, usually attributed to Phil Karlton, holds that programming has only two hard problems: naming things — variables, functions, classes — and cache invalidation. It is no exaggeration. With multi-level caching, invalidation touches every layer from the servers to the CDN, the clients' browsers and everything between, such as reverse proxies. When the underlying data change — when the result of that complex SQL query is no longer what the database would return — stale copies must stop being served, and the cache must be invalidated and resynchronized with the database. Two methods are commonly used.

Time-based expiry sets in advance how long an entry stays valid, its time to live (TTL), say 5 or 10 minutes. When it expires, the next request goes to the original source instead of the cache. The difficulty is choosing the TTL: the best value depends on users' access patterns and on how often the API is called, and both differ between endpoints and between services.

Event-based invalidation ties invalidation to changes in the source data. If user profiles are cached, then whenever a user updates a name, an email or a setting, the same handler that updates the database also deletes the profile's cache entry. The next request for the profile finds no entry, reads the profile from the database and returns it. No guessing is involved: the code knows exactly when to invalidate. The catch is that, without automatic expiry, every code path that changes the profile must remember to invalidate; forgetting a single case risks showing users stale data. Both methods have advantages and drawbacks.

6.3 Local, distributed and multi-level caches

A *local* cache is a data structure such as a dictionary or map inside the server process, holding the cached values in the server's own memory. With 10 server replicas there are 10 such maps, and they drift apart: the same key can hold different values on different replicas. A *distributed* cache is an external service — usually Redis or Memcached, or the newer Valkey — that all replicas share, so their views cannot disagree. Its drawback is that every read becomes a network round trip. The lecturer puts an in-memory read at about 2–3 ms, depending on payload size and serialization format, and a network read, even inside an internal network, at 50 ms or more.

> **A subtle point.** Both figures are pessimistic. A lookup in an in-process map takes nanoseconds to microseconds, plus deserialization if values are stored serialized. A Redis call within one data centre typically costs a fraction of a millisecond to about a millisecond; 50 ms is closer to a cross-continent round trip. The argument for a local first level rests on that gap of orders of magnitude, not on the absolute numbers.

Many systems therefore use *multi-level caching*: a small local cache for the hottest data and, upstream of the servers, a distributed cache such as Redis. Which data count as hottest depends on the caching algorithm — whether it favours the most recently used or the most frequently used values, for instance — but they are the data in highest demand. A lookup checks the local cache first, then the distributed cache, then the database; after a hit in the distributed cache, the algorithm's configuration decides whether to keep a local copy, depending on how hot the item is.

6.4 Caching patterns

A caching pattern decides at which moment data are written to the cache. The choice depends on the access patterns of the application, its users and the specific endpoint (Table 4).

Cache-aside, or lazy loading, is the most common and most intuitive pattern, often implemented without its name being known. For `/users/2`, the request first checks the caching layer for this user's profile. On a hit, the result is returned at once. On a miss, the program runs the complex query against the database, stores the result in the cache and returns it. When the user is updated, the code explicitly deletes or invalidates the entry, so the next request misses, loads fresh data from the database, and users never receive stale data (Figure 9).

```
GET /users/:id                        // cache-aside read
  key = "user:" + id
  profile = cache.get(key)
  if profile: return profile          // hit: ~50 ms path
  profile = db.runProfileQuery(id)    // miss: ~800 ms path
  cache.set(key, profile, ttl=300)    // 5-minute TTL
  return profile

PATCH /users/:id                      // invalidate on write
  db.updateUser(id, changes)
  cache.delete("user:" + id)          // next GET reloads
```

Figure 9: Cache-aside with event-based invalidation (illustrative pseudocode)

Write-through updates the cache instead of invalidating it: each write goes to the database and the cache together. The entry is therefore always present and never misses. The price is that every write takes a few milliseconds longer, because it must succeed in both the database and the caching layer before a success response is returned.

Write-behind (write-back) removes that cost. An update — a new user name, say — goes only to the caching layer, which is extremely fast; the success response is returned immediately and the database is updated asynchronously. The risk is that if the database write fails or the database is unavailable, cache and database are left inconsistent, and a write already confirmed to the user exists only in the cache.

Pattern	On a write	Gain	Cost
Cache-aside	update the DB, delete the entry	simple; caches only what is read	first read after a write misses
Write-through	update DB and cache together	the entry never misses	slower writes
Write-behind	update cache now, DB asynchronously	fastest writes	DB can lag or lose the write

Table 4: The three caching patterns compared

6.5 The cache hit ratio

The *cache hit ratio* is the percentage of requests that find what they need in the cache; the rest must go to the original source. A ratio of 90% means only 10% of requests reach the database, which is very good. A ratio of 20% means that, although a caching layer exists, something in its algorithm or pattern must be found and fixed. Three factors drive the ratio. The first is the TTL: a long TTL keeps entries longer and so makes future hits more likely, but raises the risk of staleness, and a TTL of 7 days defeats the purpose of setting one, leaving the team fighting stale data instead. The second is cache size: more memory holds more data and serves more requests. The third is the data access pattern. Without knowing which endpoints are called most and at what times — without understanding how users behave — no caching algorithm, pattern or strategy will reach a good ratio such as 70–80%. The ratio depends directly on how well the platform's users and their access patterns are understood.

7 VERTICAL AND HORIZONTAL SCALING

7.1 The capacity question

An application runs on servers. As traffic and the user base grow, the servers' resources become insufficient and more capacity is needed, again and again. The textbook answers to how to obtain it are the two fundamental approaches to scaling: vertical and horizontal.

7.2 Vertical scaling

Vertical scaling, or scaling up, makes the existing server more powerful or replaces it with a larger one — it increases the server's "height". The upgrades are straightforward: more CPU cores (2, 4, 8 and so on); more main memory, the RAM, from 2 GB to 4, 8, 16, 32 GB or whatever the server needs; more storage, from 30 GB to 300 GB to 1 TB, on faster media such as SSDs and NVMe SSDs; and a modern network card, for example 10 Gbps, for more internet traffic.

Its appeal is simplicity. The architecture of the system and of the server does not change; there is no need to think about statelessness or similar concerns in the code; the machine is upgraded and the code stays as it is. Capacity scales intuitively: a server with twice the processors handles roughly twice the requests, and one with twice the RAM caches twice as much data locally — going from 16 GB to 32 GB doubles local caching capacity. Economically, the lecturer notes, one server of double size usually costs less than two servers of half its size, and it avoids the operational cost of running several machines: security, backups and maintenance for each, load-balancer configuration and the handling of distributed state. Two qualifications apply: in public clouds, prices within an instance family usually grow roughly linearly with size, so most of the saving is operational; and doubling the cores doubles throughput only while the work is CPU-bound and parallel rather than waiting on the database.

7.3 The limits of vertical scaling

If vertical scaling is so simple, why does not every company rely on it? Because it has hard limits. The first is a ceiling set by hardware: however much money is spent, there is always a most powerful machine. Cloud providers such as GCP or DigitalOcean offer very large instances, but eventually the largest one on offer is reached — 32 cores in the lecturer's example — and if traffic keeps growing there is nowhere further to go.

The second is a single point of failure. A server with 32 cores and 96 GB of RAM is a powerful beast, but servers fail every day for many reasons, a dependency problem or anything else. If the single server is down for 2 hours, the

whole platform is unavailable for 2 hours. Backup servers and automatic failover reduce the risk, but one server, however powerful, remains a single point of failure.

The third is the lack of geographic distribution. If the server is in the USA and 40% of the users are in India, those 40% experience higher latency than the other 60%, and with a single machine nothing can be done about the delay they meet every day, however important they are to the business. So although vertical scaling is intuitive and needs no extra programming or operational complexity, its drawbacks have moved most of the industry to horizontal scaling.

7.4 Horizontal scaling

Horizontal scaling, or scaling out, adds more copies of the same server instead of replacing it with a more powerful one, and all copies work together to serve the growing traffic: several machines of medium size and capacity instead of one powerful machine. The lecturer deliberately calls its advantages *theoretical*. The first is arithmetic: if one server handles 1,000 requests per second, five servers handle 5,000; before a big campaign more copies are added, and more again, with no hard limit on how many servers can work together. The second is redundancy: if one server fails, the users' traffic is spread evenly across the remaining copies, which take over its share, and the service keeps running. The third answers the geographic weakness of vertical scaling: since servers are simply added to the stack, they can be added in other regions, close to the users (Table 5). The lecturer promises to return to why these advantages are only theoretical (see Chapter 18); this chapter has already shown two of the costs, per-replica connection pools (Section 5.9) and diverging local caches (Section 6.3).

Aspect	Vertical (scale up)	Horizontal (scale out)
Method	bigger CPU, RAM, disk, network	more copies of the server
Code	unchanged	must suit many replicas
Ceiling	largest machine on offer	none, in theory
Failure	single point of failure	survivors absorb the traffic
Geography	one location	servers near the users
Operations	one machine to run	load balancing, backups, security per machine

Table 5: Vertical and horizontal scaling as presented in part 1

8 CONCLUSION

- Latency is what users feel, and it is a distribution: describe it with P50, P90 and P99, never with the mean alone.
- The slow tail (P95, P99) holds the heaviest logic and often the most valuable users.
- Read throughput together with latency; as utilization nears 100%, queueing makes latency climb steeply, so run at 60–80% and keep headroom for bursts.
- Locate the bottleneck before fixing anything: timing logs, profilers and flame graphs for CPU work, distributed tracing for I/O.
- Replace N+1 loops with batched or eager-loaded queries, and read the SQL the ORM generates.
- Index what real queries need, mind column order in composite indexes and confirm with `EXPLAIN ANALYZE`; every index slows writes.
- Reuse database connections through a pool; with many replicas, cap the total with an external pooler such as PgBouncer.
- Cache expensive results and choose invalidation, placement and pattern deliberately; the hit ratio shows whether it works.
- Vertical scaling is simple but bounded and fragile; horizontal scaling removes the ceiling in theory, at costs taken up in Chapter 18.

APPENDIX A REVIEW QUESTIONS

1. A checkout API serves 2 million requests a day. Its dashboard shows a mean latency of 110 ms, a P50 of 45 ms and a P99 of 3.8 s, and checkout requests are 3% of the traffic. A manager wants to report only the mean because it is "well under the 200 ms target". Reconstruct what the latency distribution most likely looks like, argue which requests populate the tail and why they matter commercially, and specify what you would report and alert on instead.
2. A service averages 55% CPU utilization, but several times an hour traffic arrives in one-minute bursts at 1.5 times the average rate, and P99 latency already spikes during those bursts. To save money, a colleague proposes removing a third of the instances, which would raise average utilization to about 82%. Using the relationship between utilization and latency, predict P99 during bursts and between them after the change, and propose a capacity target you could defend to the finance team.
3. After a release, the P95 latency of `GET /orders/:id` rose from 120 ms to 900 ms. The release added a call to an external audit-log service and two computed fields to the response; database dashboards show unchanged query times, and the team lead wants to put Redis in front of the orders query this week. Lay out how you would locate the bottleneck, what a CPU profiler and a distributed trace would each show for the plausible causes, and why the proposed cache is a gamble.
4. A handler loads 50 posts with one query, then for each post awaits one query for its author and another for its comment count; each query costs 4 ms including the network round trip, with connections taken from a pool. Compute the number of queries and the database time for 50 and for 500 posts. Restructure the handler so that the number of queries no longer depends on the number of posts, state how many remain, and explain

how your figures would change if every query opened its own connection.

5. A PostgreSQL table `events(id, user_id, type, created_at, payload)` holds 50 million rows and receives 3,000 inserts per second. The hot queries are the 20 newest events of one user, all `login` events of one user, and a nightly report of all events in the last hour. Design the indexes, including column order, justify each against its write cost, and describe how you would use `EXPLAIN ANALYZE` to prove that each query uses the index you intended.
6. An API runs on Kubernetes with 3 to 12 replicas; each replica's driver keeps a pool of up to 40 connections, and PostgreSQL allows 300 connections. Predict at which replica count and under what load connection errors appear and what users experience. Redesign the setup with an external pooler, give concrete pool sizes, and explain where requests wait during a surge in your design and what that does to latency.
7. A profile endpoint receives 5,000 reads and 20 writes per second across 8 replicas, and profiles are changed by two handlers: the user's settings page and a new admin tool. Choose where to cache (local, distributed or both), which caching pattern and which invalidation method to use, and justify each choice. Then predict what users will see if the admin tool's developer forgets to invalidate, and how your TTL choice bounds the damage.
8. A startup runs on one 16-core, 64 GB server in the USA; 40% of its users are in India, the server was down for two hours last month, and Black Friday is expected to bring four times normal traffic. The CTO wants to move to a 64-core machine. Compare this plan with running six smaller replicas in terms of capacity, failure, latency for Indian users and required code changes, and explain what happens to per-replica connection pools and local caches in the second plan.

APPENDIX B GLOSSARY

Term	Meaning
Latency	Time from a request (a user's action) to its response (the result on screen)
Percentile (P50, P90, P99)	Latency at or below which that share of requests falls
Throughput	Requests a system completes per unit of time, e.g. per second
Utilization	Share of a system's capacity in active use
Bursty traffic	Arrivals that come in clusters rather than at a steady rate
Bottleneck	The component that limits performance or adds most latency
Profiler	Tool that samples a running program to show where time goes
Flame graph	Picture of sampled call stacks; width shows share of time
Distributed tracing	Record of each step of one request across components, with timings
N+1 query problem	One query for a list, then one more query per item
Full table scan	Reading every row to find matches; also called sequential scan
Composite index	One index over several columns, usable from its leading column
Covering index	Index holding every column a query needs, so the table is not read
Connection pool	Set of open connections that requests borrow and return
Cache-aside	Read the cache, load from the database on a miss, delete on write
Cache hit ratio	Share of requests answered by the cache

APPENDIX C FURTHER TOPICS

Little's Law

Little's Law states that the average number of requests inside a system equals the arrival rate multiplied by the average time each request spends there ($L = \lambda W$). It turns throughput and latency into concurrency: 2,000 requests per second at 50 ms each means about 100 requests in flight, which sizes thread pools, connection pools and replica counts. The law holds for any stable system whatever the arrival pattern, so it is the first sanity check in capacity planning and in reading load-test results.

Coordinated omission and HDR histograms

Many load generators send the next request only after the previous one returns, so when the server stalls they stop sending and silently omit the requests that would have waited — a bias Gil Tene named coordinated omission. The resulting percentiles look far better than what real users, who do not wait politely, experience. Generators that issue requests at a constant rate, such as wrk2, and recorders based on HDR Histogram, which keeps percentiles accurate across many orders of magnitude, avoid the bias. It matters whenever P99 figures from a load test are used to decide capacity.

Service level objectives and error budgets

A service level objective sets a target for a measured indicator, for example “99% of checkout requests complete within 300 ms over 28 days”. It turns percentiles into an engineering contract, and the remaining allowance — the error budget — decides when a team must pause feature work and invest in reliability. SLOs apply as soon as users depend on a service, and they give alerting a principled threshold instead of an arbitrary one. Google's *Site Reliability Engineering* book describes the practice.

Hedged requests

When tail latency comes from a few temporarily slow replicas, a client can send a second copy of a request to another replica once the first has been outstanding longer than, say, the P95 latency, and use whichever answer arrives first. Dean and Barroso report large reductions in tail latency for only a few percent of extra load. The technique suits idempotent reads against replicated services such as caches, search shards or storage, and is unsafe for non-idempotent writes.

Cache stampede and request coalescing

When a popular cache entry expires or is invalidated, many concurrent requests miss at the same moment and all run the same expensive query, which can overload the very database the cache was protecting. Remedies include request coalescing (one request recomputes while the others wait for its result), a short lock around recomputation, randomized TTLs so that entries do not expire together, and refreshing hot keys early in the background. It matters for any hot key under heavy read traffic, especially after deployments or cache restarts.

PgBouncer pooling modes

PgBouncer can return a server connection to its pool when the client disconnects (session mode), after each transaction (transaction mode) or after each statement (statement mode). Transaction mode lets a few hundred database connections serve thousands of application clients, which is what makes an external pooler effective under autoscaling. In exchange, session-level state — `SET` commands, advisory locks, `LISTEN` and, in older versions, protocol-level prepared statements — cannot be relied on across transactions, so drivers and ORMs must be configured for it.

Partial and expression indexes

PostgreSQL can index only the rows that satisfy a condition, as in `CREATE INDEX ON orders (created_at) WHERE status = 'pending'`, or the result of an expression, as in `CREATE INDEX ON users (lower(email))`. A partial index is smaller and cheaper to maintain on writes because rows outside the condition are skipped, and an expression index lets queries that filter on a computed value use an index at all. Both apply when queries consistently target a small subset of rows or a transformed column, as in queue-like tables and case-insensitive lookups.

Amdahl's Law and the Universal Scalability Law

Amdahl's Law bounds the speed-up of a workload by its serial fraction: if 5% of the work cannot run in parallel, no number of processors yields more than a 20-fold speed-up. Gunther's Universal Scalability Law adds a coherency term for the cost of keeping nodes consistent, and predicts that throughput can even fall beyond some number of servers. Both explain why “twice the servers, twice the throughput” is only a first approximation, for vertical and horizontal scaling alike, and both can be fitted to load-test data to predict where adding capacity stops paying.

Scaling & Performance, Part 2 — One-Page Overview

Chapter 18 · every section and sub-section of the paper at a glance

1 Introduction: Statelessness, the Key to Horizontal Scaling

Horizontal scaling works only if no instance holds data the others cannot reach.

- **1.1 Stateful and stateless instances** — Any instance must give the same result, even after one is removed.
- **1.2 Sessions and uploaded files** — In-memory sessions cause 401s elsewhere; files on local disk go missing.
- **1.3 Externalize every piece of state** — Redis for sessions, S3 or R2 for files, a shared database server.

2 Load Balancers

The load balancer receives all traffic and picks an instance for each request by an algorithm.

- **2.1 The decision point** — A middleman that forwards requests and returns responses.
- **2.2 Round robin and weighted round robin** — Rotation suits uniform requests; weights reflect unequal servers.
- **2.3 Least connections and other algorithms** — Open connections reveal expensive work; also response time, resources.
- **2.4 Health checks** — Periodic test requests exclude a dead instance and readmit it on recovery.

3 Scaling the Database

The stateful database is scaled by replication and sharding, best bought from a managed provider.

- **3.1 Why the database resists scaling** — Its data must stay consistent across instances.
- **3.2 Read replicas** — Replicas serve the 70–90% of queries that are reads; the primary takes writes.
- **3.3 Replication lag and consistency** — A read right after a write may be stale; three mitigations.
- **3.4 Sharding** — A table split across instances by a shard key such as order date.
- **3.5 Distributed and managed databases** — PlanetScale, Neon, CockroachDB, YugabyteDB; do not run your own.

4 CDNs and Edge Computing

Serving content and light logic near users removes most of the physical latency.

- **4.1 The speed-of-light floor** — Tokyo to Virginia costs about 100 ms before any processing.
- **4.2 Edge locations and their two wins** — Latency falls to 2–3 ms and the origin carries less load.
- **4.3 What to cache and how to purge** — Static bundles, media, some API responses; purge by tag.
- **4.4 Protection against DDoS attacks** — A large CDN absorbs and filters attack traffic.
- **4.5 Edge computing** — Code on edge nodes: early 401s, localization by browser language.
- **4.6 Why not run everything at the edge** — Small machines and restricted runtimes such as V8 isolates.

5 Asynchronous Processing

Work the user need not wait for leaves the request path through a queue and workers.

- **5.1 Perceived latency** — Adopted from the start, for operations without synchronous behaviour.
- **5.2 The invitation example** — Responding before the email call cuts 400 ms to 100 ms.
- **5.3 Producers, queues and workers** — Redis queues or RabbitMQ, BullMQ; workers scale on their own.
- **5.4 Media processing and account deletion** — Encoding and thumbnails; an 8 s deletion becomes a 100 ms reply.

6 Microservices

Microservices scale large teams more than machines and bring distributed-system costs.

- **6.1 The monolith** — One deployable unit: simple to develop, test and deploy.
- **6.2 What microservices scale** — Independent deploys, per-service scaling, per-service stacks.
- **6.3 The costs** — Network calls, cross-service debugging, data consistency.
- **6.4 When they are worth it** — Large teams with distinct scaling or technology needs.

7 Serverless Computing

Event-triggered functions remove capacity planning but bring cold starts and limits.

- **7.1 Servers and capacity planning** — Under-provisioning crashes; over-provisioning wastes money.
- **7.2 Autoscaling and its limits** — Slow to start, capped, reactive, never free at zero traffic.
- **7.3 Functions, events and pay-per-use** — An API gateway maps routes to functions; pay per invocation.
- **7.4 Cold starts** — Warm-up pings, Firecracker micro-VMs, V8 isolates.
- **7.5 Execution limits, statelessness and fit** — 15-minute cap, no pools or sockets; best for irregular jobs.

8 Rules of Thumb for Scaling

Measure first, keep it simple and scale for the problems you actually have.

- **8.1 Start with the problem: measure first** — Find the real bottleneck before picking a solution.
- **8.2 Prefer simple solutions** — Complexity is a cost that needs justification.
- **8.3 Scale for the problems you have** — Build for current load plus headroom.
- **8.4 Measure from day one** — Observability is the exception to simplicity; performance is a mindset.

Scaling, part 2 at a glance — Make instances stateless so a load balancer can spread traffic over them, scale the stateful database with replicas and shards, push content and light logic to the edge, move slow work to queues, and adopt microservices or serverless only when measurement justifies their costs. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Scaling and Performance, Part 2: From Stateless Servers to Serverless

Chapter 18 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Horizontal scaling, introduced in Chapter 17, works only when application instances are stateless, so sessions, files and data move to shared stores. A load balancer then distributes requests by round robin, least connections or their weighted variants, while health checks keep traffic away from failed instances. The database, the stateful remainder, is scaled with read replicas, whose replication lag causes stale reads, and with sharding by a carefully chosen key, in practice through managed or distributed databases. CDNs and edge computing serve content and light logic near users, beating the roughly 100 ms speed-of-light floor and absorbing DDoS traffic. Asynchronous processing, microservices and serverless computing are examined with their trade-offs, and the chapter closes with the lecturer's rules: measure first, prefer simple solutions, scale for current problems and build observability from day one.

Keywords — statelessness, load balancing, read replicas, sharding, CDN, edge computing, asynchronous processing, microservices, serverless

1 INTRODUCTION: STATELESSNESS, THE KEY TO HORIZONTAL SCALING

Chapter 17 covered latency and percentiles, utilization, profiling, database performance, caching, and vertical versus horizontal scaling, and ended by calling the advantages of horizontal scaling "theoretical". They become real only when the stack is built for them, and the property that makes this possible is *statelessness*.

1.1 Stateful and stateless instances

Horizontal scaling runs several instances of the same code — instances A, B, C and D — and spreads client requests across them. An instance is *stateful* when it holds data exclusive to itself: something it remembers about a client or user that the other instances cannot see. If D holds data that A and C cannot reach, the outcome depends on where a request lands, and the stack shows errors that look random. This is why horizontal scaling, unlike vertical scaling (adding cores and memory in the infrastructure dashboard), must be planned from the ground up: it affects the code, the infrastructure and the whole stack.

> **Statelessness.** A horizontally scaled service is stateless when it does not matter which instance receives a request: the result is the same, and it stays the same if an instance is removed (A, C and D behave exactly as A, B, C and D did). Any data that must persist lives outside the instances, in a store that every instance can reach.

1.2 Sessions and uploaded files

Session-based authentication (see Chapter 4) shows the failure. The user submits email and password; instance A verifies them, creates a session, sets the session ID in a cookie and keeps the session record in an in-memory structure — in effect an array inside its own process. The next request, carrying the cookie, is routed to instance B. B cannot see A's memory, finds no session and answers `401 Unauthorized`: a user who has just logged in is told, intermittently, to log in again. The fix is to write the session to a shared in-memory store such as Redis; B then looks up the session ID there, finds the record A wrote, and serves the request.

File uploads fail the same way. If A saves an uploaded file on its own SSD, a later request that lands on C finds nothing and fails. The file belongs in object storage reachable by every instance — Amazon S3, Cloudflare R2 or a self-hosted MinIO (see Chapters 20 and 21).

```
STATEFUL: state inside one instance
  POST /login  -> A   A keeps session in its memory
  GET  /me     -> B   B has no session  -> 401
  POST /upload -> A   file written to A's local disk
  GET  /file   -> C   file not found    -> error

STATELESS: state outside all instances
  POST /login  -> A   A writes session  -> Redis
  GET  /me     -> B   B reads session   <- Redis -> 200
  POST /upload -> A   A stores object   -> S3 / R2
  GET  /file   -> C   C reads object    <- S3 / R2
```

Figure 1: Where state lives decides whether routing matters (illustrative).

1.3 Externalize every piece of state

The pattern repeats for every kind of state, so the rule is general: once horizontal scaling is chosen, no information, file or data may be kept inside a particular instance; everything is centralized. Sessions and in-memory caches go to Redis, files to S3 or other blob storage, and the database must be a server reached over the network — PostgreSQL, Amazon RDS or similar — rather than an SQLite file on the instance's disk. With state externalized, capacity grows roughly linearly with the number of identical servers placed behind a load balancer, limited only by what one can afford.

2 LOAD BALANCERS

2.1 The decision point

With several instances, something must decide which instance receives each request, and the question is also

where in the stack that decision is made. The load balancer (LB) is that place, and a horizontally scaled architecture starts by setting one up. All traffic from the internet goes to the load balancer instead of the servers; it chooses an instance, forwards the request, receives the response and returns it to the client over the same HTTP connection. Forwarding is simple middleman work; the interesting part is the *load-balancing algorithm* that makes the choice.

2.2 Round robin and weighted round robin

Round robin sends requests in rotating order: the first to A, the second to B, the third to C, the fourth to A again, and so on. With a trivial algorithm it carries a lot of traffic, provided two conditions hold: requests cost roughly the same (similar database or queue work), and the servers have equal capacity, for example 4 GB of RAM and two CPU cores each.

The lecturer breaks the first condition with two request types. Type 1 is a cheap read — fetching a user profile with a `SELECT` — taking about 200–300 ms depending on where database and users are. Type 2 is expensive: it calls an external server (Elasticsearch, an email provider), then writes to a large table carrying several indexes, which every insert or update must maintain (see Chapter 8); the whole call takes about 2 s. Round robin routes blindly, knowing nothing about the cost of what it forwards. When expensive requests arrive in step with the rotation — every third request being type 2, say — they all land on one instance, which may eventually crash under the skew.

Weighted round robin addresses the second condition. If A has 8 GB of RAM and four cores while B and C have 4 GB and two cores, A gets twice the weight: of a burst of eight requests, two go to A, one to B, one to C, two to A again, and so on, so A always receives twice as many. It still cannot tell a cheap request from an expensive one.

2.3 Least connections and other algorithms

Least connections sends each request to the server that currently holds the fewest active connections. In the HTTP request–response cycle the client waits for its response, and while the server makes its external call or performs its write, the HTTP connection (and the TCP connection beneath it) stays active. Expensive requests therefore hold connections longer, making the count a live signal of load. In the lecturer's walk-through (Figure 2) all counts are zero at start-up, so the first request — a 2 s type 2 — may go to any server, say A. Requests 2 and 3, both cheap, go to B and C, which have no connections. By the time request 4 arrives, B or C has finished its 200 ms request and dropped back to zero while A is still busy, so request 4 goes to B or C and A is spared. *Weighted least connections* applies the same rule with capacity weights, so an 8 GB, four-core A receives about twice the share of B and C.

```
t = 0      req1 (type 2, ~2 s)    -> A   A:1  B:0  C:0
t = 10 ms  req2 (type 1, ~200 ms) -> B   A:1  B:1  C:0
t = 20 ms  req3 (type 1, ~200 ms) -> C   A:1  B:1  C:1
t = 250 ms B and C have finished         A:1  B:0  C:0
t = 260 ms req4 (type 1)          -> B   A:1  B:1  C:0
           A is still busy with req1: it gets nothing new
```

Figure 2: Least connections with one expensive and three cheap requests; the counts are active connections after each step (illustrative timings).

A subtle point. The lecture notes that HTTP/2 and HTTP/3 multiplex requests but assumes an HTTP/1.1 connection closes once the response is sent. In fact HTTP/1.1 connections are persistent by default (keep-alive). Load balancers in HTTP mode therefore count their own active connections to each backend or, as in Envoy's "least request" policy, outstanding requests; the idea — route to the instance with the least unfinished work — is unchanged.

Other algorithms use other signals. *Least response time* favours instances currently answering fastest, so a struggling, slow instance receives fewer requests. *Resource-based* algorithms read each instance's CPU and memory usage and favour the least loaded. Each is a piece of logic that picks the instance for the next request (Table 1).

Algorithm	Chooses by	Suits
Round robin	Fixed rotation A, B, C, A …	Uniform requests, equal servers
Weighted round robin	Rotation in proportion to weights	Unequal capacity
Least connections	Fewest active connections	Requests of mixed cost
Weighted least connections	Connections relative to weight	Mixed cost, unequal servers
Least response time	Fastest recent responses	Instances that slow down
Resource-based	Lowest CPU and memory usage	Uneven resource use

Table 1: Load-balancing algorithms named in the lecture.

2.4 Health checks

Suppose instance A crashes, overloaded by round robin or brought down by an infrastructure misconfiguration. An algorithm that keeps selecting A sends requests to a dead machine: those users get `502` or `503` errors while others do not, and under round robin a user served by B a moment ago may hit A next — a frustrating experience. The remedy is the *health check*. Alongside user traffic, the load balancer sends its own test request — a cheap `GET` endpoint — to every instance, say every second, and expects a success status, by convention `200`. The endpoint does no heavy work, so the probes add no real load. When A fails a probe it goes on an exclusion list and gets no user requests; B and C take the traffic under the configured algorithm. Probes to A continue — otherwise the balancer could never learn that A has recovered — and the first successful response takes A off the list. In practice both transitions usually require several consecutive results (for example three failures, two successes) so that one slow probe does not eject a healthy instance, and a dead process shows up as a

refused connection or a timeout rather than an HTTP status.

3 SCALING THE DATABASE

3.1 Why the database resists scaling

Once state is externalized, the application tier scales by adding servers, but the state has not disappeared: it now lives in the database, the stateful part of the architecture. A database cannot be duplicated like an application server. Each instance holds a large amount of data in local files, and that data must be consistent — any instance asked the same question must return the same answer — and coordinating this is what makes database scaling tricky. Meanwhile every new application instance talks to the same database, itself a server with, say, 4 GB of RAM and four cores, which eventually runs out of resources. The industry's tried-and-tested answers are replication and sharding.

3.2 Read replicas

In a read-replica architecture one instance is the *primary* (also called master or parent) and the others are *replicas* (secondary, child or slave). Replicas hold all of the primary's data but serve only reads, `SELECT` queries; every write goes to the primary. Replicas are usually spread geographically: with the primary in the US and replicas in India, China and Japan, and application instances deployed in those countries as well, local users are served by nearby backends reading nearby replicas. The two benefits are less load on the primary and lower latency. The load effect is large because most applications, SaaS products above all, are read-heavy: 90% reads is typical, and even a conservative 70% means the primary, which used to handle all queries, now handles only the 30% that are `INSERT`, `UPDATE` and `DELETE`.

3.3 Replication lag and consistency

The price is consistency. PostgreSQL, MySQL and other modern databases replicate natively, carrying changes from the primary to the replicas, but no mechanism beats physics: data between the US and India travels through undersea optical fibre, and physical distance dominates latency. If the trip takes 200 ms, that is the *replication lag*.

> **Replication lag.** The delay between a write committing on the primary and the same change becoming visible on a replica. A read issued inside that window can return stale data even though the write has already been acknowledged to the client.

The lecturer's scenario (Figure 3): a user in India changes the name on a profile page from "A" to "AB" and clicks Save. The write goes to the primary in the US, which succeeds, and the backend returns `200`. The moment the client — a single-page application or a server-rendered page — sees success, it fires a `GET` to refresh the form. This is a read, so it goes to the Indian replica a few milliseconds later, before replication has finished, and returns the old name. The interface has just confirmed the change and now shows the old value: confusing for a profile, potentially damaging for payments or invoices.

```
India: user, backend, replica        US: primary

t = 0       PUT /profile {"name": "AB"} -> primary
            primary commits; the client gets 200
t + 10 ms   GET /profile -> replica (India)
            replica returns {"name": "A"}    stale
t + 200 ms  the change reaches the replica
            GET /profile returns {"name": "AB"}
```

Figure 3: A read-your-writes violation caused by 200 ms of replication lag (illustrative timings).

Many solutions have been proposed and implemented; which one fits depends on the trade-off one accepts (Table 2). The backend can route intelligently: since the users table was just written, the reads that follow are sent to the primary instead of a replica. It can track the replication lag (about 200–250 ms on average here) and hold reads until replication completes, so the response carries current data. Or the frontend can avoid fetching instantly, with a planned delay of, say, 300 ms before the next `GET`. Read replicas are a distributed system, and every distributed design trades something — most often consistency, that is, whether every instance has up-to-date information. Managed providers such as Amazon RDS and Google Cloud SQL create replicas in chosen regions with a few configuration flags or toggles and ship tested mechanisms for these problems, so a backend engineer rarely solves them at the infrastructure level, but should know the trade-offs exist.

Technique	How it works	Cost
Route to primary	Reads after a write to the same data go to the primary	More primary load; routing logic
Wait for replication	Track the lag; hold reads until the replica has caught up	Slower reads
Planned client delay	Frontend re-fetches after about 300 ms	Breaks when the lag grows

Table 2: Mitigations for stale reads from replicas proposed in the lecture.

3.4 Sharding

Sharding, which the lecture also calls partitioning, divides one table across several physical database instances. Consider an e-commerce application like Amazon or Flipkart receiving thousands to millions of orders a day, whose `orders` table grows to billions of rows. A user's orders page runs `SELECT * FROM orders WHERE user_id = 5`; even with an index on `user_id`, a query over billions or hundreds of billions of rows takes long. That is the first problem, query latency. The second is that more traffic needs more resources, that is, more database instances.

Sharding addresses both. Picture the table as ten rows, each standing for a billion: rows 1–5 go to one database instance and rows 6–10 to another. The difficult part is choosing the *shard key*, the criterion that decides which data lives on which instance. In the lecture's example it is the order date: the first instance holds January to June, the second July to December. Two physical instances serve

more requests per second, solving the second problem, and each searches half the rows, reducing query latency. With more volume the split can be finer, one shard per month. A routing layer — in the backend or a dedicated database router — decides before each query which shard holds the data, sends the query there and returns the result (Figure 4).

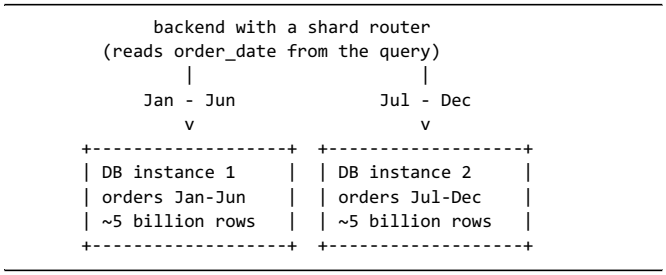


Figure 4: The orders table sharded by order date across two database instances (illustrative).

A subtle point. The shard key must match the dominant query. With orders sharded by date, `WHERE user_id = 5` carries no date, so the router cannot pick a shard: it must query every shard and merge the results. A date key also sends all current writes to the newest shard, which becomes a hot spot. For a "my orders" page, sharding by `user_id` (often hashed) keeps each user's orders together. Strictly, "partitioning" usually names splitting a table inside one server (as in PostgreSQL declarative partitioning), "sharding" spreading the pieces over separate servers.

3.5 Distributed and managed databases

Replication and sharding — read replicas and shards — are the two terms to know; creating and querying them involves deep technical work beyond this lecture. The current trend (the lecture dates from December 2025) is distributed databases that handle sharding, replication and distributed transactions automatically: PlanetScale, built on Vitess for MySQL; Neon, a serverless PostgreSQL whose storage layer is written in Rust; CockroachDB; and YugabyteDB. Without deep expertise in database administration and scaling, one should not run one's own database infrastructure: replication, sharding and backups — keeping users' data safe in several places — are each hard. Realistically, one picks a provider (AWS, Neon, PlanetScale) on features, pricing and team preference, creates a database, receives a URL and credentials and plugs them into the backend. The concepts still matter, because the provider's console asks how often to take backups, in which regions to place read replicas and how to plan sharding. The provider does the work; the engineer must understand the terms well enough to express the configuration.

4 CDNS AND EDGE COMPUTING

4.1 The speed-of-light floor

The caching discussed so far (see Chapters 9 and 17) operates inside a data centre. A further layer works at global scale and is organized by geography: the content delivery network (CDN). The case for it starts with a physical limit. Light in optical fibre travels at about 200,000 km/s, and all intercontinental traffic runs through undersea fibre at that speed. A request from Tokyo to a server in North Virginia (AWS region `us-east-1`, home to many commercial applications) and its response travel about 20,000 km, roughly 100 ms. No technology available today can beat that minimum.

The 100 ms covers only transport. Routing inside the server is fast, mostly regular-expression matching of the path to a handler (see Chapter 2). The handler then deserializes the HTTP body into the language's data structures — a JavaScript object in Node.js, a struct in Go — at a cost that grows with payload size (see Chapter 3). The service layer runs business logic, usually against the database: even with server and database in the same region, a medium to complex query takes about 50–100 ms. An external API call can add another 200 ms. Together the 100 ms floor becomes roughly 500–800 ms (Table 3). Database optimization and in-memory caches such as Redis shrink the middle terms, but the last cause of latency remains the geographic distance between user and server, which is the problem CDNs address.

Stage	Cost for a Tokyo user, origin in us-east-1
Network round trip, about 20,000 km	about 100 ms, a physical floor
Routing to the handler	negligible
Deserializing the body	grows with payload size
Database query, same region	50–100 ms
External API call	about 200 ms
Total, as estimated in the lecture	500–800 ms

Table 3: Where the time goes for a distant user.

4.2 Edge locations and their two wins

A CDN places servers, called *edge locations* or *points of presence* (PoPs), in the users' regions — Tokyo, Mumbai, Singapore — and caches content on them. A Tokyo user's request then travels 100–200 km instead of 20,000 km, and latency falls from about 100 ms to 2–3 ms: barely perceptible to a human, but large in performance terms. The second win is load. Requests answered at the edge never reach the origin in the US, which in the lecturer's rough figure receives about half the traffic it otherwise would; it uses fewer resources and needs horizontal scaling less often.

4.3 What to cache and how to purge

Choosing what to place on the CDN is a decision of its own (the lecturer refers to his dedicated CDN video). The obvious candidates are *static content*, which rarely changes: JavaScript and CSS bundles, HTML files, images, videos and fonts. A single-page application such as a React app is built into a bundle of JavaScript, CSS and HTML at deployment; cached on the CDN, it is served from the edge instead of the origin fetching it from an S3 bucket for each request. This is the standard way to deploy SPAs and static

sites such as blogs. Some *API responses* can be cached too, for example a product catalogue that changes rarely.

Cached content must also be removable — *purged*, in CDN terms. Cloudflare, one of the largest CDN providers, lets the application attach tags to cached content and purge by tag. If every cached blog post of a user carries a tag containing the user ID, then when the user publishes a new post the application purges that tag; the CDN deletes those entries, and the next requests fetch fresh data from the origin and cache it again. With such conditions defined, serving stale content is not a concern.

4.4 Protection against DDoS attacks

The third benefit, prominent in Cloudflare's own pitch, is protection against distributed denial-of-service (DDoS) attacks. An attacker controls many machines through malware — say 20,000 bots worldwide — and a script makes them all send traffic to one server. Its resources run out: with hard limits it crashes; with autoscaling it keeps launching instances to absorb the traffic, and the bill may reach $50,000 for a single day. The damage is either an outage or a financial loss, and such attacks are in the news constantly. With the CDN in front of the servers, all traffic passes through it: cached content is answered at the edge, the rest is forwarded. Cloudflare detects attacks from sudden surges using methods refined over years and responds, for example by presenting CAPTCHAs to suspicious users; and because its network is so large, even terabytes or petabytes of attack traffic are spread across its nodes and cannot seriously harm the origin.

4.5 Edge computing

CDN nodes are called *edge nodes* because they sit at the edge of the network: CDNs are placed strategically, usually in collaboration with internet service providers (ISPs), at the first point a user's request touches, which is why they respond so fast. A traditional CDN does no processing: the user asks for a file by name and the node returns it. *Edge computing*, a similar-sounding term that causes confusion, means running code on these nodes, so the request is processed at the edge before the response is produced. Edge nodes are, and will remain, far more numerous than primary data centres such as the big AWS or GCP regions, which require heavy financial, operational and geographic investment, whereas edge nodes ride on ISP infrastructure (even Cloudflare and AWS build theirs together with ISPs). The processing takes as long as it would at the origin, but the round trip is short, so the response is genuinely faster.

Authentication is the classic use case. With stateful authentication, the Tokyo user's cookie travels to the US, where the server looks up the session ID in the database or Redis and either lets the request through or returns `401 Unauthorized`. Spending a 100 ms round trip and origin resources only to send a rejection is waste. Running the same session check on the edge node answers invalid requests in 2–3 ms, and only authorized requests reach the origin, which then handles legitimate traffic only (Figure 5). Localization is another case: an edge node knows which region it serves and can read the request headers, so if a Tokyo user's browser language is Japanese, it returns the Japanese version of a page directly instead of the English default. Location-based configuration, customization and user preferences generally suit the edge.

```
Session check at the origin
  Tokyo --- ~100 ms round trip ---> origin (us-east-1)
                                    invalid -> 401
Session check at the edge
  Tokyo --- 2-3 ms ---> edge node (Tokyo)
                        invalid -> 401 at once
                        valid   -> forwarded to origin
```

Figure 5: Rejecting unauthenticated requests at the edge (illustrative).

4.6 Why not run everything at the edge

Two constraints prevent it. The first is resources. A request from India passes through the user's ISP (Airtel, ACT and so on), several routing hops, undersea cables and a foreign ISP before reaching the server. The ISPs' primary job is routing; CDN functionality is something they host at some of these points in collaboration with Cloudflare or AWS, on modest hardware — the lecturer's figure is about 1 GB of RAM and one CPU core instead of 8 or 16 GB (a Cloudflare Worker, for instance, is limited to 128 MB of memory). The second is the runtime. Edge computing is designed for very low-latency work: Cloudflare Workers, the best-known platform, runs code in V8 isolates (the JavaScript engine from Chrome), with no file system and only the networking the platform exposes, not arbitrary TCP. Primary servers on robust data-centre infrastructure therefore remain necessary; the edge takes the strategic slice of logic that benefits from proximity — authentication, customization, validation, choosing which server a request should go to — and works alongside them.

5 ASYNCHRONOUS PROCESSING

5.1 Perceived latency

Asynchronous processing reduces the latency users perceive on the frontend. Unlike horizontal scaling, adopted at a threshold (10,000 or 50,000 users, when vertical scaling stops making sense), it is usually built in from the start because its benefits are immediate. A normal API call has synchronous behaviour: when a user changes their name from A to B, the server runs `UPDATE users SET name = 'B' ... RETURNING id` and may return `200` only after the database confirms, because a refreshed page must show the new name. Some operations have no such requirement: the user does not need to see their effect the moment the response arrives.

5.2 The invitation example

SaaS products such as Jira or Notion let users invite teammates to a workspace. The user types `user1@gmail.com` and clicks Invite; browser JavaScript sends an HTTP request, which passes routing, the handler, validation and authentication. The service layer checks in the database that the person is not already a member, then inserts a row into an

`invites` (or team-members) table with the email and a status of `pending`, later accepted or rejected. With the database in the same region this takes about 50–80 ms, at most 100 ms. Then the invitation email must be sent through an external provider — Mailchimp, SendGrid, Resend — by an API call carrying the address and a template ID or ready-made HTML. The provider's location and infrastructure are outside the backend's control; assume 200–300 ms. Done synchronously, the user watches a spinner for about 400 ms before the success toast and the refreshed list showing the pending invite. If the response is sent right after the database work, the interaction takes 100 ms (Figure 6). 400 ms is not slow in itself, but other calls may add up, or the email provider may be down; the goal is to cut the latency the user perceives.

```
Synchronous
  validate, DB check, insert invite      ~100 ms
  call the email provider                ~300 ms
  200 OK reaches the user                at ~400 ms

Asynchronous
  validate, DB check, insert invite      ~100 ms
  push job send_invite to the queue      ~1 ms
  200 OK reaches the user                at ~100 ms
  worker calls the email provider        later, unseen
```

Figure 6: Taking the email call off the request path (lecture's figures; the enqueue time is illustrative).

5.3 Producers, queues and workers

Sending the email becomes a *task* or *job* (see Chapter 10). Before responding, the server — the *producer* — pushes the job into a queue, typically a Redis-based queue or RabbitMQ (Kafka and other event-processing systems are alternatives). *Consumers*, also called *workers*, detect the job, take it from the queue and run its handler: call SendGrid or Resend with the address. A worker can run from the same codebase as the server, or as a separate codebase and deployment so that workers can be scaled horizontally on their own under heavy traffic, an infrastructure choice teams make as they grow. The user does not need the email sent instantly: the invitee will receive it and eventually accept or reject the invitation. Emails and notifications are the standard examples; nobody expects a notification within a millisecond of the action.

The tooling is simple: a Redis instance, usually managed (Upstash, for example), and a queue library. In Node.js, BullMQ is a well-known background-job library on top of Redis that provides error handling, rate limiting and the other features expected of a production-grade queue.

> **Acknowledge only what is durable.** Answering `200` before the work is done is safe only if the job cannot be lost: respond after the job is persisted in the queue, make the handler safe to run twice, and let failed jobs retry or move to a dead-letter queue (see Chapter 10).

5.4 Media processing and account deletion

Uploading a video to YouTube keeps the tab open only while bytes are read from the local file system and uploaded. Afterwards the tab can be closed: YouTube pushes jobs into a queue — generate thumbnails, encode HD versions, generate subtitles — run in parallel or in sequence and taking perhaps 10 or 20 minutes depending on load, and users expect it to take time. Image uploads and resizing belong to the same category.

Account deletion is a less obvious case. Deleting an account in a to-do application means deleting the user's todos, categories, profile data (picture, bio), calendar schedules and so on — rows in many tables — and finally the row in `users`, last because the others reference it by foreign key. The naive `DELETE` handler runs all of these in one transaction (all or nothing; see Chapter 8), then responds `200` and logs the user out. For a user active for ten years, with a million rows in each table, finding and deleting the data is slow: at roughly half a second per table, eight tables take about 4 s (the transcript's "5 milliseconds" per table cannot produce that total), and with business logic and request overhead the round trip reaches about 8 s — an unacceptable spinner, unless the dialog asks the user to wait 10 or 20 seconds without closing the tab, which cannot be expected. Asynchronously, the server only checks that the user exists and is authenticated (50–80 ms), pushes a `delete_user` job with the user's ID and responds; the browser logs the user out after about 100 ms. A worker then runs the deletions one by one, and whether they take 5, 10 or 30 seconds no longer matters, since nobody waits.

The first step is identifying tasks whose result the user does not expect immediately: emails, notifications, user-data deletion, video and image uploads. Offloading them markedly improves perceived latency, which makes this one of the easiest and most recommended performance measures.

6 MICROSERVICES

6.1 The monolith

Anyone asking how to scale a backend will eventually be told to split it into independent services, *microservices*, and many companies already work that way, so it is important to know what they are and when they apply. Implementing them is a large subject with its own technologies; the focus here is their role in scaling. The starting point is the *monolith*: a backend that is a single deployable unit. Authentication, order processing, notifications, payments and webhooks are modules — different files — in one codebase that call each other and run together in one process (or several identical processes when scaled horizontally). Monoliths are simple to develop, test and deploy: all code is in one GitHub repository, a refactoring touches several modules and ships as one deployment, and tests run in one process. A monolith can also be scaled horizontally whenever needed — so why consider anything else?

6.2 What microservices scale

Microservices mainly scale the *team* — the humans behind the machines — rather than machine performance. The prerequisite is a large organization: subjective, but in the

order of more than 100–200 developers on one application. At that size the monolith shows three problems.

- **Deployment dependency.** The payments team finishes a critical change and wants to ship it, but the main branch — the one deployed to production — also holds notification-team changes that are not broken but not ready for users. Feature flags and a disciplined branch life cycle (only deploy-ready code on `main`) help, and in a startup of 10–30 developers everybody knows what everybody is doing, so the problem rarely appears. With 500 or 1,000 developers and rapid development in some modules, one unit of deployment makes teams block each other.
- **Uniform scaling.** In-app notifications need little — some database inserts, perhaps WebSocket messages — while order processing and payments need much CPU and memory, and failures there are unaffordable. Scaling a monolith, vertically or horizontally, scales everything; separate deployable units can each be scaled to their own current or expected traffic.
- **One technology stack.** A blogging platform such as Medium, Hashnode or DEV needs Markdown cleaning, Markdown rendering and website parsing, for which the Node.js or Python ecosystems offer ready libraries — a specific npm package, say. It also resizes and manipulates uploaded images, CPU-bound work where Go or Rust takes about 50 ms against about 500 ms in Node.js or Python. A single deployable unit has one language; as services, the Markdown part runs on Node.js and the image part on Rust, each deployed separately.

6.3 The costs

- **The network.** What was a function call inside one process becomes a network call, gRPC or HTTP, between physically separate services. Latency appears, depending on how services are organized, and so do failures: every call needs failure handling, retries and timeouts (see Chapter 12).
- **Debugging.** Microservices are a distributed system. One request may pass the load balancer, the routing layer, then the order, payment and notification services; understanding it means reading the logs of four services at once. This calls for distributed tracing tools and services designed so a single request can be followed across them (see Chapter 14).
- **Data consistency.** In mature architectures each service owns its database, which brings back the consistency problem of Section 3: even with replication, some lag always exists between separate database instances.

Concern	Monolith	Microservices
Deployment	One unit; teams wait for each other	Each service ships alone
Scaling	The whole application	Per service
Technology	One stack	A stack per service
Calls between parts	Function calls	Network calls: latency, failures
Debugging	One process, one log	Distributed tracing
Data	One database	Database per service; consistency

Table 4: Monolith and microservices compared on the lecture's criteria.

6.4 When they are worth it

A large team is mandatory before considering microservices: large organizations naturally have clear boundaries, and so do services, so the human organization and the technical one align. The further criteria are services with genuinely different scaling needs and parts of the backend that require different technologies. Because the complexities usually outweigh the advantages, the move is a major decision; without clear answers about team size, specific technology requirements and deployment velocity, it is typically not worth it.

7 SERVERLESS COMPUTING

7.1 Servers and capacity planning

Serverless computing, popularized by Vercel, Netlify and Cloudflare, has become a common way to scale backends (the lecturer has a separate playlist on its origins, on how Vercel works and on serverless databases). Before it, an application ran on a server: a VM — an AWS EC2 instance or similar — with an operating system such as Ubuntu. The engineer provisions it, installs the needed software (nginx, Docker), configures it to fetch, build and run the code, and manages it for its whole life: broken configuration, DNS and the rest. The model has worked for decades and is predictable — two or four cores, 4, 16 or 32 GB of RAM, 30 GB of disk, 1 TB of bandwidth per month — and the application processes requests as they arrive.

Its first challenge is *capacity planning*: before serving anyone, one must decide how many servers and how much RAM, CPU and disk each, which means predicting traffic and user behaviour. Under-provision — 4 GB of RAM, then a spike from an online campaign or a blog post — and the server crashes or becomes unresponsive, with 3–4 s responses and error toasts; new users leave before becoming paying customers, and paying users start looking for alternatives, a financial and reputational loss. Over-provision — 32 GB where 8 GB would have done — and a week might cost $5,000 instead of about $500 while only 20% of the capacity is used. Traffic cannot be predicted, so both directions lose money.

7.2 Autoscaling and its limits

Autoscaling is the long-standing answer. Start with two 4 GB servers; as users arrive and both reach a threshold such as 70% memory usage, the autoscaler launches a third in-

stance, traffic spreads over three servers and each drops to about 40%. Autoscaling remains standard practice, but it has four drawbacks:

- **Time.** A new instance must boot an operating system (usually a Linux distribution such as Ubuntu), fetch or build the code, start the process and expose it to the internal network and load balancer: seconds to minutes depending on OS, language and build. A burst arriving within 3–4 seconds outruns it.
- **Limits.** Every configuration sets a minimum and a maximum number of instances. A maximum too low (say 10) brings back under-provisioning, with slowness and errors beyond it. One too high (say 1,000) lets an attack, or genuine Black Friday traffic, push the fleet to 500–600 instances and the bill to about $100,000 in a day.
- **Reactiveness.** Scaling begins only after the threshold is crossed, so by the time the new instance runs, the existing ones are already overloaded; the autoscaler cannot anticipate traffic, and if the window is short, crashes and errors still occur.
- **Always-on cost.** There is always a minimum — two instances, or one machine with 4 GB, two cores and 30 GB of disk — paid for whether traffic is 500 requests per second or zero.

For steady, predictable traffic, which describes most SaaS applications, the always-on *serverful* model makes sense: even a single request finds a server ready. Serverless was invented for workloads without that predictability.

7.3 Functions, events and pay-per-use

Serverless does not mean there are no servers: code still needs a machine, a VM, an operating system and configuration. It means the developer does not deal with them — not with the machine, its OS, or whether it has 4 or 8 GB of RAM. The developer supplies only two things: *functions* (the code) and the *events* that trigger them. For HTTP, an *API gateway* sits in front, receives requests and maps each route — the event — to the function to run (Figure 7).

The computing model differs. A traditional server is always present, listening and paid for. On a serverless platform such as AWS Lambda or Cloudflare Workers, deployment provisions no machine; the code is simply stored. When the first request or event arrives, the provider starts an instance — machine and capacity unknown to the developer — runs the code and returns the response. Depending on the provider's configuration, the instance then lives a few more seconds (3, 5 or 30) or returns to a pool, so the next request can reuse it instead of starting a new one. The pricing model differs too: instead of paying 24/7, one pays only for the CPU and memory time of each invocation, so the monthly bill is the sum of the milliseconds actually spent executing code.

```
client -> API gateway: GET /orders/42
          route /orders/:id  ->  function getOrder
                |
                +-- warm instance in pool? -> run code
                |
                +-- none -> start instance (cold start)
                            -> load runtime -> run code
          response; the instance idles a few seconds,
          then is reused or shut down
billing: only the CPU and memory time of invocations
```

Figure 7: Request path of an HTTP-triggered serverless function (illustrative).

7.4 Cold starts

The catch, and the main issue of serverless from the beginning, is the *cold start*: since no machine is pre-assigned, the time needed to start an instance after a request arrives is added to that request. Two mitigations exist. The first keeps instances warm with automated pings: one's own infrastructure sends a request every few seconds so that an instance is ready when a real user arrives. It cannot be overdone, because permanently warm instances are paid for 24/7, which defeats the purpose of serverless. The second, more effective, is runtimes built for minimal cold start. The two main causes of cold starts are booting an operating system or VM, and starting the language runtime (Java, Rust, Go, Node.js, Python). For the first, AWS built Firecracker, a KVM-based micro-VM technology used for Lambda that has become an industry standard for serverless. Cloudflare Workers instead run code in V8 isolates, lightweight VM-like isolation inside the V8 JavaScript runtime, which Cloudflare says start in about 0–1 ms. For the second, the lecturer argues that interpreted languages such as JavaScript and Python start faster in serverless environments than Java with its compilation and JVM. Combining isolates with JavaScript, Workers reach cold starts of about 5 ms. Cold starts never vanish entirely: a traditional server is always online, whereas serverless starts instances on demand by design.

> **A subtle point.** The lecture links slow cold starts to languages with a compilation phase. What costs time at startup is runtime initialization — starting the JVM, loading classes and dependencies — not compilation as such. Ahead-of-time compiled Go and Rust binaries start almost instantly and have among the shortest cold starts, and JavaScript is itself JIT-compiled by V8. Package size and the function's initialization code matter as much as the language.

7.5 Execution limits, statelessness and fit

The second problem is limits: providers cap how long an invocation may run — an AWS Lambda function at most 15 minutes — so a long operation, say 30 minutes, fails partway. The third is statelessness. Traditional setups assume a stateful process: open TCP connections held to the database, WebSocket connections held with browsers, and similar long-lived behaviour. A serverless instance lives only seconds and cannot keep data in its operating system, because the machines running the same code are constantly

replaced as traffic changes. Serverless architecture is therefore stateless, and database connections and WebSockets must be reimagined.

It is easier to say when not to use serverless: latency-sensitive, user-facing applications such as banking or payments; long-running work such as long HTTP calls or WebSocket connections; and applications needing many database connections, which also require a serverless database once the backend goes serverless. Good fits are specific, irregular operations in a workflow. Video processing and image resizing — unless they are the application's core business — run on demand instead of on heavy infrastructure paid 24/7; event-driven operations, triggered by a new queue message, a file upload or a database change, fit as well. In the lecturer's view the industry overhypes serverless: a powerful tool for specific cases, not a universal replacement for servers (Table 5).

Aspect	Traditional server	Serverless
You manage	Machine, OS, application	Functions and events
Capacity	Planned or autoscaled	Per request, by the provider
Cost	24/7, even at zero traffic	Execution time only
Start-up	Always running	Cold starts
Duration	Unlimited	Capped (Lambda: 15 min)
State	Pools, WebSockets possible	Stateless
Fits	Steady, latency-sensitive load	Irregular, event-driven jobs

Table 5: Traditional servers and serverless functions compared.

8 RULES OF THUMB FOR SCALING

The lecture closes with rules that apply to everything in both parts: latency and throughput, database optimization and caching, vertical and horizontal scaling, distributed systems, infrastructure components such as API gateways and load balancers, and microservices.

8.1 Start with the problem: measure first

Every technique in these two lectures is a solution, and solutions should follow a known problem. Is the system actually slow, and how slow? Which component is the bottleneck — and if it is the database, is that because a particular index is missing or because the data needs sharding? Answering requires measurement: observability through logs, metrics and traces (see Chapter 14), plus performance and load testing. Self-hosted Prometheus and Grafana, or a managed service such as New Relic, trace every request and show where the time goes. Applying solutions before knowing the specific answer is *premature optimization*. The worst outcome is fixing the wrong bottleneck, because it gives the impression the problem is solved while the real one grows. Only with the numbers in hand — average latency, error rates — should one reach for caching, aggressive indexing and the other techniques.

8.2 Prefer simple solutions

Simple solutions are easier to understand, debug and operate. Microservices are not simple, and neither is running Kubernetes for horizontal scaling; such tools are for when no other choice is left. Table 6 lists the lecturer's pairs. Complexity always has a cost: every component added is another that can fail, must be monitored, and must be understood and operated. This does not mean always choosing the simplest option — sometimes complexity is necessary — but it needs justification and should be accepted only when simplicity is genuinely insufficient.

Start with	Before reaching for
One large server (vertical scaling)	Horizontal scaling from day one
Proper indexes for the queries	A Redis cache in front of every query
A monolith	Microservices

Table 6: The simpler option first, as recommended in the lecture.

8.3 Scale for the problems you have

There is no need to build for a million users on the first day; most platforms never reach a million. Build for the current scale with a reasonable buffer for growth, and with logs, metrics and traces in place the bottlenecks reveal themselves as the system grows. Each application has its own characteristics, so generic advice, including engineering blogs from Netflix, Facebook or Google, may not apply; learning from one's own measurements is more solid than copying a big company's solution.

8.4 Measure from day one

Observability is the exception to the preference for simplicity. It is tempting to start with plain logs and postpone metrics and traces until 50,000 or 100,000 (one lakh) users, but in the lecturer's experience production-grade observability from day one always pays off: no random errors or crashes to puzzle over, no guessing where the bottleneck is, and enough warning to change the system or infrastructure before problems reach users. Finally, as with security (see Chapter 16), performance and scaling are a mindset built by experience: build systems, watch them struggle, optimize, see what helps, and after much trial and error the system scales for the traffic it actually receives rather than a generic number. The job is not to predict every problem but to build systems that handle problems gracefully when they occur, and to develop the skill to measure, diagnose and resolve them quickly.

9 CONCLUSION

- Horizontal scaling works only with stateless instances: sessions in Redis, files in object storage, data in a shared database.
- The load balancer decides where each request goes: round robin for uniform traffic, weights for unequal servers, least connections for mixed request costs, health checks to skip dead instances.

- The database is the stateful bottleneck; read replicas absorb the 70–90% of queries that are reads, at the price of replication lag and stale reads.
- Sharding splits a table across instances to cut query latency and per-instance load; the shard key must match the dominant queries.
- Use managed or distributed databases, but understand replication, sharding and backups well enough to configure them.
- CDNs cut the roughly 100 ms speed-of-light floor to a few milliseconds, offload the origin and absorb DDoS traffic; edge computing adds light logic such as session checks, within tight limits.
- Queues and workers take emails, media processing and bulk deletions off the request path, cutting perceived latency.
- Microservices scale organizations more than machines; serverless trades always-on cost for cold starts, time limits and statelessness.
- Measure first, prefer simple solutions, scale for current problems and build observability from day one.

APPENDIX A REVIEW QUESTIONS

1. A team runs three instances of a Node.js API behind a load balancer. Sessions live in a `Map` inside each process, avatars are written to `./uploads` on local disk, and a nightly report reads an SQLite file stored next to the code. After a fourth instance is added, users are logged out at random, avatar images break, and report totals differ from night to night. Explain each symptom from the routing behaviour, and design the changes that make the instances interchangeable, including what happens to users who are logged in while you migrate.
2. Three identical instances serve 90% profile reads (about 250 ms) and 10% order writes that call an external API and update a heavily indexed table (about 2 s). Compare round robin and least connections for this mix, including when the expensive requests arrive in bursts. Then instance C's database pool is exhausted: it answers every request with `500` within 5 ms, while its `/health` endpoint, which touches nothing, still returns `200`. Predict what least connections does with C, and redesign the health check so that a brief outage of the shared database does not eject every instance at once.
3. Your primary database is in `us-east-1` and a read replica in Mumbai lags by about 200 ms. A user in Pune updates their billing address, and the single-page app immediately requests a PDF invoice, built from a replica read, which is emailed to their accounting department. Describe exactly what the invoice can contain and why. Then compare routing to the primary after a write, waiting for replication and a client-side delay for this flow, in latency and primary load, and choose one.
4. An `orders` table of 12 billion rows is sharded into twelve shards by order month. State which shards are touched by (a) a "my orders" page, `WHERE user_id = 5`, (b) a dashboard of the last seven days, and (c) inserting today's orders, and describe the load on each shard during a Black Friday sale. Propose a shard key for an application dominated by per-user queries and explain which of the three workloads becomes harder under it.
5. An e-commerce origin runs in `us-east-1` and 40% of its users are in Japan. Pages need the React bundle, a product-catalogue API that changes a few times a day, per-user cart data, and a session check that rejects about 30% of requests (bots and expired sessions). Decide what to serve from the CDN cache, what to run at the edge and what must stay at the origin; explain how a price change reaches Japanese users quickly, and estimate with the lecture's figures what a Tokyo user waits for each part.
6. Account deletion was moved to a queue: the API checks the user, enqueues `delete_user(42)` and returns `200`, and a worker deletes rows from eight tables. In production a worker is killed after deleting the to-dos and categories but before the profile and the `users` row; meanwhile the user signs up again with the same email. Describe the resulting state and what the user experiences, and state which properties the job and its enqueueing need (see Chapter 10). Would you run the eight deletions in one transaction inside the worker? Justify it.
7. A product has three workloads: a payments API at a steady 80 requests per second with a 150 ms p99 target and pooled PostgreSQL connections; nightly resizing of about 20,000 uploaded images within an hour; and a monthly data export that runs for 40 minutes. For each, choose autoscaled servers or serverless functions, weighing cold starts, the 15-minute execution limit, connection handling, the always-on minimum and over-provisioning cost, and say how you would set the autoscaling minimum and maximum where servers win.
8. A startup with 12 developers and 3,000 daily users plans to split its monolith into eight microservices on Kubernetes because "the app feels slow"; it has only plain logs. Using the lecture's rules, explain what you would do first, which measurements would decide the next step, which simpler fixes would probably come before any split, and which concrete signals — team size, deployment conflicts, scaling profiles or technology needs — would justify extracting a first service later.

APPENDIX B GLOSSARY

Term	Meaning
Statelessness	No instance holds data the others cannot reach; any instance can serve any request
Load balancer	Entry point that forwards each request to an instance chosen by an algorithm
Least connections	Algorithm that picks the instance with the fewest active connections
Health check	Periodic test request whose failure takes an instance out of rotation
Read replica	Copy of the primary database that serves reads only
Replication lag	Delay before a write on the primary is visible on a replica
Sharding	Splitting a table's rows across separate database instances
Shard key	Column or rule that decides which shard holds a row
CDN	Network of edge locations (points of presence) that cache content near users
Cache purge	Removing cached CDN content, for example by tag, so it is fetched again
Edge computing	Running code on CDN edge nodes close to the user
Producer, consumer	Code that enqueues a job; worker that takes and executes it
Monolith	Application deployed as one unit containing all modules
Microservice	Independently deployable service owning one area of functionality
Autoscaling	Adding or removing instances when metrics cross thresholds
Cold start	Delay to start a serverless instance before it can handle a request

APPENDIX C FURTHER TOPICS

Consistent hashing

Consistent hashing places both keys and nodes on a hash ring, so adding or removing one of n nodes remaps only about 1/n of the keys, whereas plain `hash(key) mod n` remaps almost all of them. It underlies data placement in many distributed databases and caches, and load balancers use it when a key must keep reaching the same backend. A backend engineer meets it when resharding data, growing a cache cluster or choosing a hash-based shard key. Virtual nodes are usually added to keep the load even.

Session affinity (sticky sessions)

Sticky sessions make the load balancer send every request of one client to the same instance, usually via a cookie it sets, so in-memory sessions keep working without a shared store. It is a quick fix for a stateful application but weakens horizontal scaling: load becomes uneven, and when an instance dies or is scaled in, its users lose their state. Affinity remains legitimate for WebSocket connections and for instance-local caches, as an optimization rather than the home of essential state.

Synchronous replication and failover

With asynchronous replication the primary acknowledges a write before any replica has it, which causes the lag described in this chapter; with synchronous replication it waits for at least one replica, trading write latency for no data loss when a replica must take over. On primary failure a replica is promoted, clients are redirected and the old primary must be prevented from accepting writes (split brain). Managed databases automate failover, but the engineer chooses the mode and must know how many seconds of writes a failover can lose. PostgreSQL exposes the choice through `synchronous_commit` and `synchronous_standby_names`.

HTTP caching headers for CDNs

CDNs decide what to cache and for how long from standard HTTP headers (RFC 9111): `Cache-Control` with `public`, `max-age` and `s-maxage` for shared caches, plus validators such as `ETag`. The `stale-while-revalidate` extension (RFC 5861) lets a cache serve a slightly stale copy while fetching a fresh one, hiding origin latency. Content-hashed file names such as `app.3f9c1.js` allow year-long lifetimes for static bundles with no purge on deploy. Correct headers, including `Vary` and `private`, are what keep personalized responses from being shared between users.

Transactional outbox

When a handler writes to the database and also enqueues a job, either step can fail alone: the row is saved but the job lost, or the job runs for a row that was rolled back. The outbox pattern stores the job as a row in an outbox table inside the same transaction as the business change, and a relay process publishes those rows to the queue, retrying until it succeeds. Delivery becomes at-least-once, so consumers must be idempotent. It applies whenever asynchronous work such as invitations, payments or deletions must never be silently dropped.

Saga pattern

With a database per microservice, a business operation spanning several services cannot use one ACID transaction. A saga splits it into local transactions, coordinated by events or by an orchestrator, and defines a compensating action for each step, such as refunding a payment when the shipment cannot be created. The result is eventual rather than immediate consistency, with intermediate states visible to users and other services. Sagas are the standard answer to the data-consistency cost of microservices named in the lecture.

Load shedding and backpressure

Because autoscaling reacts after the fact, a service must survive the minutes before new capacity arrives. Load shedding rejects excess requests early, typically with `503` or `429` and a `Retry-After` header, so that accepted requests still finish within their deadlines instead of all of them slowing until the instance falls over. Backpressure carries

the same signal upstream, for example by bounding queue lengths so producers slow down. Deciding which requests to shed first, and which (payments, health checks) last, is part of capacity design.

Connection pooling for serverless and large fleets

Every function instance or server that opens its own database connections can exhaust the database's connection limit within seconds of a spike, and PostgreSQL spends a whole process on each connection. A pooler or proxy between the code and the database — PgBouncer in transaction mode, Amazon RDS Proxy, or the HTTP-based drivers of serverless databases — multiplexes many short-lived clients onto few real connections. It is a prerequisite for moving a database-heavy backend to functions and helps any horizontally scaled fleet. Transaction-mode pooling restricts session-level state such as `SET` variables and advisory locks, which the code must respect.

Concurrency & Parallelism — One-Page Overview

Chapter 19 · every section and sub-section of the paper at a glance

1 Introduction: the cost of waiting

A request spends most of its life waiting for I/O, and a server that waits idly wastes almost all of its CPU.

- **1.1 Where the time of a request goes** — One query waits 1–2 ms locally, 20–30 ms across zones, 90–100 ms across regions.
- **1.2 The cost of an idle CPU** — A 100 ms wait forgoes 300 million instructions; a typical call idles about 95% of the time.

2 Concurrency and parallelism

Parallelism executes several things at the same instant; concurrency interleaves several things, even on one core.

- **2.1 Parallelism** — Simultaneous instructions; needs as many cores.
- **2.2 Concurrency** — Start, pause and resume tasks: a matter of program structure.
- **2.3 Two requests, one core and two** — A and B interleave around their database waits; with two cores they overlap.

3 I/O-bound and CPU-bound work

Waiting-dominated work needs concurrency; computation-dominated work needs parallel cores.

- **3.1 I/O-bound work** — Database, files, logs, network; concurrency is mandatory.
- **3.2 CPU-bound work** — Parsing is light; images, crypto, video and ML are heavy.
- **3.3 A backend needs both** — But it is overwhelmingly I/O-bound.

4 Threads: the operating system's model

OS threads give concurrency and parallelism but cost memory, creation time and context switches.

- **4.1 What a thread is** — An OS execution unit with a stack and an instruction pointer.
- **4.2 Scheduler and preemption** — Time slices; a thread is interrupted when its slice ends.
- **4.3 Blocking operations** — A thread waiting on I/O is marked blocked, later runnable.
- **4.4 Shared memory** — Processes are isolated; threads of one process share the heap.
- **4.5 The cost of threads** — Stacks, kernel creation calls, 1–10 µs context switches.

5 The event loop

One thread multiplexes thousands of I/O waits through OS readiness notification, provided nothing blocks it.

- **5.1 One thread that never blocks** — Tasks hand over control at I/O and are resumed later.
- **5.2 epoll and kqueue** — The OS reports which watched sockets are ready.
- **5.3 Strength and trade-off** — No switches, no stacks; but 100 ms of CPU stalls everyone.
- **5.4 Async keywords as yield points** — `await` marks where a function gives up the CPU.

6 Two requests, two models

The same two database-bound requests, handled by blocking threads and by callbacks on a loop.

- **6.1 The thread model** — The thread blocks on `read`; the OS runs another until the reply.
- **6.2 The event-loop model** — Register a callback, return to the loop, run it when the socket is readable.
- **6.3 From callbacks to async/await** — Nested callbacks became flat code; what follows `await` is the callback.

7 Goroutines: virtual threads in Go

Go runs cheap goroutines on a few OS threads through its own runtime scheduler.

- **7.1 Virtual threads** — Goroutines are scheduled by the Go runtime, not the OS.
- **7.2 A goroutine per request** — `net/http` starts one with the `go` keyword.
- **7.3 The runtime scheduler** — `GOMAXPROCS` OS threads carry many goroutines.
- **7.4 A Go handler** — Blocking-style code; the goroutine parks during the query.

8 Inside async/await: a state machine

The runtime turns an async function into a state machine whose transitions are its awaits.

- **8.1 The function** — Two awaited queries, then a return.
- **8.2 The state-machine rewrite** — A `switch` on a state variable, advanced by callbacks.
- **8.3 Two consequences** — `await` needs `async`; a blocked loop freezes every machine.

9 Shared state and race conditions

Interleaved access to shared state loses updates, with threads and with async code alike.

- **9.1 The lost update** — Read, add, write interleaved: two increments give 1.
- **9.2 Races without threads** — A check before `await` is stale after it: balance −100.
- **9.3 Locks and channels** — Mutual exclusion, or a single owner fed by messages.

10 Choosing a concurrency model

I/O-bound services favour async or goroutines; CPU-bound work favours parallel threads.

- **10.1 The three models compared** — Scheduler, cost, switching, style, cores.
- **10.2 Matching model to workload** — The lecturer's closing recommendations.

Concurrency and parallelism at a glance — a request mostly waits for I/O, so a server must keep working while it waits: threads, event loops and goroutines are three ways to do so, parallel cores pay off only for CPU-bound work, and shared state is the price of all of them. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Concurrency and Parallelism: I/O-Bound versus CPU-Bound Work

Chapter 19 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — A typical API call spends most of its time waiting for databases and external services, so a server that handles one request at a time leaves its CPU idle about 95% of the time. The chapter distinguishes concurrency, which interleaves tasks even on one core, from parallelism, which executes them at the same instant on several cores, and relates the two to I/O-bound and CPU-bound work. It then examines the two basic mechanisms: operating-system threads, with their scheduling, shared memory and costs, and the single-threaded event loop built on epoll and kqueue. Two database-bound requests are traced through the thread model, the callback-based event loop and Go's goroutines, and async/await is explained as a state machine. The chapter closes with the race conditions caused by shared state, including in single-threaded async code, their classic remedies, and how to match a model to a workload.

Keywords — concurrency, parallelism, I/O-bound, CPU-bound, threads, context switching, event loop, goroutines, race conditions

1 INTRODUCTION: THE COST OF WAITING

In this course a backend application means an HTTP server. If such a server could process only one request at a time, then while it served one user, the thousand others sending requests at the same moment would either wait or receive an error saying that the server is busy; no production system with thousands of users can work that way. Every backend must therefore handle several things at once, and every language and runtime (Node.js, Python, Rust, Go, Java) has its own way of expressing that requirement: threads, `async/await`, goroutines, virtual threads. Most engineers learn these keywords from documentation and can go far without knowing what they do mechanically. This chapter builds that mechanical model from the operating system up to application code. It offers no recipe to copy; its value is a clear picture of how a backend processes and multitasks its requests, which helps in structuring an application and in long-term architectural decisions.

1.1 Where the time of a request goes

Take a single client: a user whose browser sends a request to the server. The request passes the routing, handler, service and repository layers (see Chapter 6), and processing it essentially means talking to the database: send a query, wait while the database executes it, and send the response back. That is the shape of a typical API call. The wait depends on where the database runs. For a very simple query it is about 1–2 ms when the server runs on localhost, 20–30 ms in production when the database sits in another availability zone, and 90–100 ms when it is in a distant region (Table 1). The central question is what the server does during those 2, 30 or 100 milliseconds.

Database location	Wait per simple query	Instructions a core could run
Same machine (localhost)	1–2 ms	3–6 million
Another availability zone	20–30 ms	60–90 million
A distant region	90–100 ms	270–300 million

Table 1: Time a server waits for one simple query, by database location (lecturer's figures), and the instructions a core executing about 3 billion per second could have run meanwhile.

1.2 The cost of an idle CPU

If the server handles one request at a time and executes synchronously, line by line with no concurrency, it does nothing during the wait: its CPU is *idle* while the network packets from the database are on their way. A modern CPU core executes roughly 3 billion instructions per second, i.e. 3 million per millisecond. During a 100 ms wait it could have executed 300 million instructions; it executed none.

A mid-to-complex API call multiplies the waste. It typically makes three to five database queries and may also call external services: an email provider, or Redis to read items from the cache or insert them (see Chapter 9). Count five network operations in total at an average of 50 ms each: the call spends about 250 ms waiting for responses, against perhaps 10 ms of actual CPU work. The server's hardware, CPU and memory, is unused about 95% of the time. Such waiting is called waiting for **I/O** (input/output), a category that covers every operation on the network, on files, and on standard input and output, such as keyboard input or output to a screen. In a typical backend, waiting for the database, for external APIs (mail, cache) and for the file system (temporary files, uploads, downloads) takes more than 70% of the time. This is the problem concurrency solves: it lets the CPU and memory do other work while network calls are pending.

2 CONCURRENCY AND PARALLELISM

2.1 Parallelism

A program is parallel when it executes several instructions at the same instant. This requires hardware support. A core executes one instruction at a time, so executing two instructions at the same moment needs at least two cores, and four instructions four cores. (Pipelining and simultaneous multithreading, in which one core exposes two hardware threads, refine this picture without changing it.) Parallelism is therefore intuitive, and its degree is bounded by the hardware.

2.2 Concurrency

Concurrency also means doing several things at the same time, but it can be achieved on a single core, because it is mainly a matter of how the program is *structured*. The program is built so that tasks can be started, paused and resumed at any time: one task starts, steps aside while something it needs is being processed elsewhere, another task runs, and the first resumes later. Several tasks are thus in progress although, at any given instant, the single core executes one instruction. The structure may come from the operating system, from the runtime of the language, or from the program itself.

> **Concurrency and parallelism.** Parallelism is *doing* several things at once: instructions execute at the same instant on different cores. Concurrency is *dealing with* several things at once: tasks are interleaved so that all make progress, even if only one executes at any moment (Rob Pike's formulation).

2.3 Two requests on one core and on two cores

Figure 1a places two requests, A and B, arriving at the same moment at a server with a single core. A takes the core first and spends 5 ms on CPU work: routing, validation, JSON deserialization. Meanwhile B waits. A then queries the database, a wait of about 40 ms, and no longer needs the CPU; at that moment the operating-system scheduler or the language runtime, whichever manages the tasks, gives the core to B. B has more processing to do (heavy validation, say, or a long recursive loop) and uses the core for 15 ms; then it too queries the database and waits until about 50 ms. A's reply arrives first, but the core is not handed to A the instant its data is there: scheduling and priorities decide, and A gets the core back at 50 ms, filters the returned values and responds to its client. B's reply has arrived as well, and B finishes the same way.

While A held the core, B made no progress at all; B used the CPU only once A was suspended on I/O. Yet even with one core no CPU time was lost to waiting. A real server juggles far more than two requests: it writes log messages to standard output (also I/O), sends telemetry to external APIs and runs background tasks, hundreds of things at once. The core's time is divided among all of them; every task that starts waiting on I/O gives up the core, which moves on to another. From the client's point of view both requests are in progress throughout, since neither has finished or failed, but in any narrow time window only one of them is being processed by the CPU. That is concurrency. With two cores (Figure 1b), B gets the second core as soon as it arrives, and in the same window both requests execute: that is parallelism.

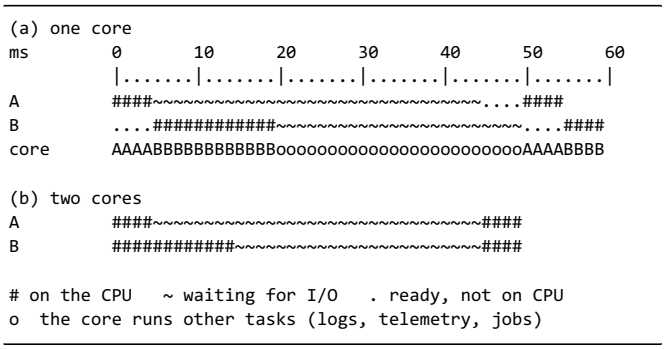


Figure 1: Requests A and B on one core (a) and on two cores (b); one character is 1.25 ms (illustrative, after the lecturer's numbers).

3 I/O-BOUND AND CPU-BOUND WORK

3.1 I/O-bound work

The CPU is used only when the program performs actual computation, executing instructions on a core. Everything else in the life cycle of a request, such as interacting with the database, the file system, the log or standard input and output, consists of sending a request to some device or service and waiting for its response. Tasks that spend their time waiting for an external resource rather than computing are called **I/O-bound**, and most of what happens in a typical API call is I/O-bound. For such tasks concurrency is mandatory. A server that processes one request at a time wastes about 95% of its resources, whereas the aim is to handle as many simultaneous requests as possible and, meanwhile, keep the CPU busy with the requests that do need computing power. The number of cores changes little: with one core or with eight, the bottleneck of a backend application is I/O.

3.2 CPU-bound work

Work in which the program actually computes, such as validating input, traversing JSON bytes, checking, comparing or adding values, is **CPU-bound**. In ordinary request handling it is light and goes unnoticed: CPUs are fast, and it usually finishes within one or two milliseconds. Some operations, however, consume a great many cycles: image processing, with its many matrix multiplications; cryptography, for example verifying a JWT in the authentication middleware; video encoding; machine-learning workloads. For CPU-bound tasks parallelism is what helps. The goal is to finish the instructions as quickly as possible, and spreading them over several cores is always faster than concurrency on one core, which executes one instruction at a time.

A subtle point. Verifying a JWT is cheap: an HMAC-SHA256 check takes microseconds, an RSA or ECDSA signature check well under a millisecond. The genuinely expensive cryptography on an authentication path is password hashing with bcrypt, scrypt or Argon2, deliberately tuned to cost tens to hundreds of milliseconds per hash (see Chapter 4); that, not token verification, is what stalls a server under login load.

3.3 A backend needs both

A backend system needs concurrency for its I/O-bound workloads: many database connections, calls to external APIs, file-system access, logging, background tasks. It needs parallelism to complete heavy computations as fast as possible. Most backend applications, however, are overwhelmingly I/O-bound unless they do specialised work such as video encoding or encryption (Table 2).

Aspect	I/O-bound	CPU-bound
Time goes to	waiting for network, disk, devices	executing instructions
Backend examples	queries, cache, email API, logs, files	parsing, images, crypto, video, ML
More cores	barely help	help, up to the core count
Needs	concurrency	parallelism

Table 2: I/O-bound and CPU-bound work.

4 THREADS: THE OPERATING SYSTEM'S MODEL

Computers do several things at once through two basic mechanisms: threads and event loops. Every concurrency primitive of a mainstream language is built on one of them, even when application code never touches a thread directly: goroutines in Go, `async` in JavaScript, TypeScript and Node.js, virtual threads in Java.

4.1 What a thread is

A thread is a facility of the operating system, independent of any programming language: an independent unit of execution that runs on the CPU. When a thread is created, the OS allocates, among other things, two essentials. The **stack** keeps track of function calls, in order: when `main` calls `getUsers`, the new call is pushed on top, and so on. It also holds local variables: when `main` declares `a = 3`, the memory for `a` lives on the stack. The **instruction pointer** records exactly where in the code the thread is, so that execution can return to that point after a context switch.

4.2 The OS scheduler and preemption

When several threads exist, a component called the *scheduler* decides which thread runs, which one is paused or blocked, and which one resumes. The defining property of threads, and what distinguishes them from the other models in this chapter, is that this scheduler is controlled by the operating system (some language runtimes, as Section 7 shows, add a scheduler of their own on top to make things lighter). The scheduler gives each thread a *time slice* of CPU processing, measured in milliseconds or less. With a 2 ms slice it pauses thread 1 after 2 ms, saves its state, including the last instruction it was executing, switches to thread 2, and after another 2 ms to thread 3. Because threads are interrupted when their slice ends, whether or not they have finished and without any say in the matter, this is called **preemptive scheduling**. Scheduling algorithms are a large subject; the lecturer recommends *Operating Systems: Three Easy Pieces* for an intuition of how threads and scheduling really work, which in turn makes the concurrency abstractions of programming languages easier to understand.

4.3 Blocking operations

A *blocking* operation is one during which the CPU cannot do anything useful for the thread: any input/output operation, any task involving a device. Logging and network calls (the network card, packets, TCP, HTTP) are examples; a database query is itself a network call, but important enough to be treated as a category of its own. The CPU can only compute and process data, so it is useless while the thread waits. A thread that encounters a blocking operation informs the operating system, which marks it *blocked*, i.e. not eligible for the CPU, and switches to another thread. When the I/O completes, for example when the database replies, the thread becomes *runnable* again. It does not receive the CPU the instant the response arrives; it is placed in a state from which the scheduler picks it up at the next opportunity.

4.4 Processes, threads and shared memory

Every program runs as a *process* of the operating system. Two processes, even started from the same code, are isolated: a thread of process 1 cannot see the memory of process 2, which matters for security and protects against memory corruption. Two threads of the *same* process, by contrast, share memory: the heap, global variables, anything dynamically allocated. If thread 1 allocates an object on the heap (a struct in Go, an object in JavaScript), thread 2 can reach that object through its address. Threads can therefore communicate simply by reading and writing the same memory location, whether a variable, a map or any other data structure. Because they work through pointers, nothing is copied or serialised, which makes the communication fast; the same property makes it dangerous (Section 9). With several cores, threads also run in parallel: on two cores, threads 1 and 2 can both execute during the same window, say from 0 to 50 ms; on four cores, four threads. This can greatly speed up CPU-intensive work.

4.5 The cost of threads

Threads carry three kinds of overhead.

- **Memory.** Every thread gets its own stack. On Linux the default stack size is about 8 MB; most of it is virtual memory, backed by physical memory only when actually needed, but each thread still costs real memory. Suppose a framework created a new OS thread for every HTTP request (frameworks do not spawn threads without limit; they reuse threads from a bounded pool).

A traffic spike of 10,000 requests would mean 10,000 native threads; at 500 KB to 1 MB each, that is roughly 5–10 GB just for request handling, on a server that needs memory for hundreds of other tasks. It runs out of memory, and the program, or the whole server, crashes.

- **Creation.** Creating a thread requires a system call into the kernel, which then sets up the stack, allocates the data structures that manage the thread internally (instruction pointer, saved state) and adds the thread to the scheduler, which will pause, block and resume it through its phases. This takes from a few microseconds to a few milliseconds: negligible for a human, but a significant source of latency at the scale of function calls.
- **Context switching**, usually the largest cost. For every switch between threads the scheduler must save the CPU registers of the current thread, update its bookkeeping of thread states, select the next thread, and restore that thread's saved registers and state. On modern hardware this takes about 1–10 µs, depending on whether the data are still in the CPU cache. With four cores and about a hundred threads the scheduler switches constantly; with thousands of threads the switching time adds up to milliseconds. It is pure housekeeping that must happen before the actual work, and the CPU time it consumes is lost to the program.

These costs are why the thread-per-request model has always struggled with high concurrency, a limitation known since the late 1990s as the C10K problem: serving ten thousand concurrent connections.

5 THE EVENT LOOP

5.1 One thread that never blocks

The second mechanism does not rely on multiplying threads. An event loop usually runs a single thread, which does many things at once through waiting, callbacks and the OS support described below. Its first rule is that the loop must never be blocked. When request A needs a database query, it does not block waiting for the answer. Through the language's primitives, usually the `async` syntax, it tells the loop: "I want the result of this query; when the response arrives, resume my function; until then I give up the CPU so that you can run other things." The task is then set aside in a queue, and the loop keeps track of every task that is waiting for I/O before it can use the CPU again. A real event loop has several queues with their own priorities and algorithms; a single queue is a sufficient model here.

5.2 OS support: epoll and kqueue

A language cannot implement this on its own: the operating system must be able to watch many I/O sources at once and report which of them are ready. Linux provides `epoll`, macOS and the BSDs provide `kqueue`, and Windows offers I/O completion ports (IOCP). With them a single loop monitors thousands of connections. In each iteration it checks which I/O operations have completed, puts the corresponding tasks back in the queue according to their priority, executes their callbacks, giving them CPU time again, and returns to the top of the loop (Figure 2).

```
while (true) {
  // 1. run every task whose input is ready (CPU work)
  while (readyQueue.length > 0) {
    const task = readyQueue.shift();
    task();             // runs until it finishes or awaits
  }
  // 2. ask the OS which watched sockets became readable
  const events = epoll_wait(watchedSockets, timeout);
  // 3. queue the callback registered for each of them
  for (const ev of events) readyQueue.push(callbackOf(ev));
}
```

Figure 2: The skeleton of an event loop in JavaScript-like pseudo-code (illustrative).

A subtle point. "Single-threaded" describes where JavaScript callbacks run, not the whole process. Readiness interfaces such as `epoll` work for sockets but not for regular files, so Node.js's libuv performs file-system calls, DNS lookups through `getaddrinfo` and some CPU-heavy crypto and zlib functions on a small worker pool, four threads by default; the loop receives only their completions.

5.3 Strength and trade-off

For CPU-bound problems, more threads (up to the number of cores) make a program faster. For I/O-bound problems the number of threads does not matter; more threads only mean more context switches, and eventually the program slows down. For concurrent, I/O-bound workloads the event loop is therefore more efficient than thread-based concurrency: with one thread there are no context switches between threads, and no megabytes or gigabytes of per-thread stacks, so the memory, latency and switching problems of Section 4.5 largely disappear.

The trade-off is that the loop must never be blocked. A piece of CPU-intensive code that takes, say, 100 ms stops the whole loop for 100 ms: it can do nothing but wait for that task to finish. The callbacks executed after I/O or CPU processing must therefore be fast. For most SaaS backends this holds: tasks of 100 ms of pure computation are rare and appear mainly with animations, image or video rendering and ML workloads, while the latency of a typical SaaS request is due to I/O.

Never block the event loop. Any synchronous step on the loop thread, be it a long computation, the parsing of a huge JSON body or a synchronous library call such as `fs.readFileSync`, delays every other request, timer and callback in the process by its full duration. A single 100 ms computation adds 100 ms to the latency of everything queued behind it.

5.4 Async keywords as yield points

Because of this constraint, languages built on an event loop, JavaScript foremost, provide callbacks, promises and `async`/`await`. All of them are syntactic sugar: convenience features that give the programmer the right mental model

of how the program runs and keep blocking operations out of the loop. They let a function relinquish control at specific points, around I/O. Writing `await` before a database query or an API call says: "I give up my share of the CPU now; when this network operation completes, return the CPU to me so that I can finish this function."

6 TWO REQUESTS, TWO MODELS

In every model a request goes through the same steps and produces the same result; the models differ in how a task that waits for the database is paused and resumed, and in how much that costs. The scenario has two requests, A and B, each querying the database.

6.1 The thread model

Request A arrives. The server parses the JSON body or the query parameters, which is CPU work because bytes are processed and loaded into memory, then routes and validates. The handler must return all users, so it issues `SELECT * FROM users`. This happens in phases. The handler establishes a TCP connection to the database, or obtains one from the connection pool (see Chapter 8); it sends the query as bytes; then it calls `read` on the socket to wait for the response. That `read` is a blocking I/O operation, one the CPU can do nothing about because it depends on something external. In a thread-based model, thread 1 is now blocked, and the task tells the OS scheduler to switch to another thread.

Meanwhile request B has arrived and runs on thread 2 in the same way: receive it, route it, parse the body into a structure or object, validate and transform it, then query `SELECT * FROM orders` over a TCP connection and call `read` on the socket: another blocking operation. The scheduler looks for a thread 3, a new one if the server creates a thread per request or a free one if it has a pool, and starts the same work there. If no thread is available, everything waits until one of the outstanding database responses comes back.

When the network card delivers the database's packets, the OS scheduler, which handles all pausing and resuming of threads, wakes thread 1: the response is back, and the rest of the logic is CPU work. The handler reads the bytes returned by the database, parses them and loads them into a native data structure of the language: objects or an array of objects in JavaScript, a dictionary in Python, a struct in Go. It then serialises that structure to JSON for the HTTP response (see Chapter 3), sends the response, and the thread that executed request A returns to the thread pool.

In code (Figure 3, Python, with no `async`), the handler reads like a sequence of lines: the query, then the response. Since everything happens within micro- or milliseconds, the code appears to run line by line. The block and the switches between threads are invisible, but they happen behind the scenes.

```
def handle_request(request):
    user_id = request.query["id"]          # CPU: parse
    # this OS thread blocks here until the database
    # replies; meanwhile the scheduler runs other threads
    row = db.query("SELECT * FROM users WHERE id = ?",
                   (user_id,))
    return json_response(row)              # CPU: serialize
```

Figure 3: A handler in the thread model (Python, no async); the blocking call looks like an ordinary line (illustrative).

6.2 The event-loop model

Request A is parsed in the same way, and its query is sent over a TCP connection in the same way. The difference lies in the next step. Instead of blocking, the handler *registers a callback*: it calls something like `db.query(sql, params, callback)`, passing a piece of logic to be executed once the result of the query comes back. This is how it hands control over: the operation is now waiting for I/O, so the CPU may run other tasks meanwhile. Then the handler returns. "Returns" does not mean that the HTTP response is sent; it means that control goes back to the loop, which switches to another task. Request B, which arrived while A was being parsed, is taken next: parsed, its own query sent, its own callback registered, and control returns again.

The loop, conceptually an infinite loop, hence the name, checks on every iteration whether the I/O of A or of B has completed, using the OS facilities of Section 5.2, which make these checks efficient. Between checks it does other work, such as writing logs to standard output or sending emails. After a few iterations `epoll` reports that B's database socket has become *readable*. The callback registered for B now runs on the loop: it loads the rows into the language's data structures according to the model, serialises them to JSON and returns the result to the client. Then the response for A arrives, and A's callback runs and answers the browser. Completion order follows the I/O, not the order of arrival.

The contrast with threads lies in how execution resumes. A thread resumes from the instruction where it was paused, using the operating system's heavyweight native structures: stack, saved registers, instruction pointer. An event-loop task resumes through its callback.

6.3 From callbacks to async/await

In JavaScript before promises and `async/await`, a handler receives the request and a function for sending the response (an abstraction over the HTTP exchange). It calls the database module, which holds the connection and the TCP handling, with the query, the user ID taken from the request, and, as third argument, an anonymous function (Figure 4a). Once the loop receives the database's reply, that function is called with an error, if any, as its first parameter and the result as its second, and it sends the user back to the client. A callback, in short, is logic handed to the event loop in advance, to be executed when the I/O completes.

The problem appears when a handler needs further I/O after the first: another query (the orders, say) or an API call

must be written inside the first callback, with its own callback. Five or six I/O-bound operations produce functions nested inside functions (Figure 4b), the shape known as *callback hell*. `async/await` was introduced to make such code readable (promises arrived with ES2015, `async/await` with ES2017). It is syntactic sugar over callbacks: once a function writes `await`, everything that follows, up to the end of *that function* rather than of the program, can be thought of as a callback passed to the I/O operation (Figure 4c). Several I/O operations then need no nesting. The idea is unchanged: at `await` the function hands control to the event loop, which does other things until the database responds and then continues from that point, all on a single thread, without OS context switches or costly stack allocations. This is why the event loop is more efficient than the thread model for I/O-bound operations; for CPU-bound operations threads remain faster.

```
// (a) callbacks: the driver calls the function later
function getUser(req, res) {
  const sql = "SELECT * FROM users WHERE id = ?";
  db.query(sql, [req.params.id], function (err, user) {
    if (err) return res.status(500).send(err);
    res.json(user);      // runs when the reply arrives
  });                    // getUser itself returns at once
}

// (b) several I/O steps nest ("callback hell")
db.query(q1, a, (err, user) => {
  db.query(q2, b, (err, orders) => {
    cache.set(key, orders, (err) => {
      mail.send(msg, (err) => {
        res.json({ user, orders });
      });
    });
  });
});

// (c) async/await: same flow, no nesting
async function getUser(req, res) {
  const sql = "SELECT * FROM users WHERE id = ?";
  const user = await db.query(sql, [req.params.id]);
  res.json(user);      // rest of function = callback
}
```

Figure 4: The same handler with a callback (a), the nesting that several I/O steps produce (b), and async/await (c) (illustrative).

7 GOROUTINES: VIRTUAL THREADS IN GO

7.1 Virtual threads and goroutines

A third model is neither plain threads nor an event loop. In the strict sense a *thread* is an entity managed by the operating system, not by the language runtime, and its creation, memory and switching overheads make it heavyweight. Go instead uses what can be described as *virtual threads*, units managed by the language runtime. Go's documentation never uses that term; it calls them **goroutines**. A Go program starts as one goroutine, the main goroutine, which can create others while it runs: the keyword `go` before a function call runs that function in a new goroutine. Java offers the same idea, under the name virtual threads, in its recent versions.

7.2 A goroutine per request

Section 4.5 showed that a thread per request exhausts memory at 10,000 or 20,000 requests. Go's standard library nevertheless creates a new goroutine for every incoming request. In `net/http` the function `Serve` listens for and accepts new connections and, at the end of its loop, starts a goroutine with `go c.serve(...)`, which runs the handler configured by the application (Figure 5a). (Strictly, it starts one goroutine per accepted connection; requests on a keep-alive connection are served by that goroutine in turn, and HTTP/2 runs each stream's handler in a goroutine of its own.)

Requests A and B therefore get goroutines G1 and G2. Each does the familiar work: parse the request, validate and transform it, deserialize it into a struct; then open the TCP connection, convert the SQL query into bytes, send them and wait for the network, a blocking I/O operation. At that moment the *Go runtime scheduler*, not the operating system's, pauses the goroutine. Just as the OS scheduler pauses a thread, switches context and runs the next thread, the Go scheduler pauses a goroutine and runs another: G2, G3, or any of a thousand, since every request has its own and which one comes next does not matter.

```
// (a) net/http, Server.Serve (simplified)
for {
    rw, err := l.Accept()     // wait for a new connection
    if err != nil { ... }
    c := srv.newConn(rw)
    go c.serve(connCtx)       // a goroutine per connection
}

// (b) an application handler: blocking-style code
func getUser(w http.ResponseWriter, r *http.Request) {
    id := r.URL.Query().Get("id")
    rows, err := db.Query(
        "SELECT * FROM users WHERE id = $1", id)
    // the goroutine is parked until the rows arrive
    ...
}
```

Figure 5: The accept loop of Go's HTTP server, simplified from the standard library (a), and a handler (b, illustrative).

7.3 The runtime scheduler on top of OS threads

The lower layer is the thread model of Section 4: OS threads, kept few for good reasons, namely their creation, context-switching and stack-memory overheads. Their number is usually related to the number of cores, since a core runs one thread at a time, but it need not equal it: a single core can host four threads, of which only one runs at any moment. On top of this layer comes the Go runtime with its own scheduler, built into the language and unrelated to the operating system. This layer is why Go handles both CPU-bound and I/O-bound work efficiently without an event-loop architecture.

A Go program has a setting, `GOMAXPROCS`, that defines how many OS threads it uses; by default it equals the number of available cores. With four cores the program is limited to four threads, M1 to M4 (Figure 6). Each thread can be pictured as a queue: whenever a goroutine is created, the scheduler inserts it into one of the queues according to

its algorithm, so a single OS thread handles many goroutines. Pausing, resuming, context switching and memory management still happen, but at goroutine level, managed by the runtime, and they are much lighter: switching from one goroutine to another is essentially redirecting a pointer, and a goroutine starts with a stack of a few kilobytes that grows on demand instead of an OS thread's megabytes. Thousands or even millions of goroutines can therefore exist at once. Memory runs out eventually, but only after about a hundred times as many goroutines as the OS threads one could create, which is why Go's HTTP package can afford one per request. When G1 blocks on I/O, for instance a database operation, the runtime pauses it and M1 starts executing G2.

```
goroutines   G1 G2 G3      G4 G5       G6 G7      G8 ...
                 |           |           |          |
Go scheduler [queue 1]   [queue 2]   [queue 3]   [queue 4]
                 |           |           |          |
OS threads       M1          M2          M3         M4
                 |           |           |          |
CPU cores     core 1      core 2      core 3     core 4

GOMAXPROCS = 4 (default: number of cores). When G1
waits on a query, M1 parks it and runs G2 from its queue.
```

Figure 6: Goroutines multiplexed by the Go runtime scheduler onto GOMAXPROCS operating-system threads (illustrative).

A subtle point. In Go's G-M-P scheduler the run queues belong not to OS threads (M) but to logical processors (P): `GOMAXPROCS` sets the number of Ps, i.e. how many threads execute Go code at the same time, while the runtime may create additional threads for goroutines stuck in blocking system calls. On sockets Go does not block a thread at all: its network poller registers the socket with `epoll` or `kqueue` and parks the goroutine. Internally, Go is an event loop hidden behind blocking-style code.

7.4 A Go handler

A handler (Figure 5b) receives the request and a response writer and runs `db.Query` with `SELECT * FROM users WHERE id = $1` and the user ID as second argument. The moment the query is sent, the goroutine executing this function pauses, and the Go scheduler picks another goroutine: another request, a logging operation, whatever is ready. When the response arrives, the paused goroutine runs again and sends the response to the client. The virtual-thread model is thus the thread model with one more layer of abstraction: the Go scheduler instead of the OS scheduler, goroutines instead of OS threads. That layer is what makes it light.

8 INSIDE ASYNC/AWAIT: A STATE MACHINE

How the `async` and `await` keywords work differs between Python, JavaScript, Rust and other languages, and is a frequent source of confusion in the event-loop model. A useful technical picture, shown here in JavaScript, is that the runtime turns an `async` function into a state machine that interacts with the event loop.

8.1 The function

`fetchUserData(userId)` (Figure 7a) first awaits `db.getUser(userId)`, a blocking operation in the sense of this chapter. Writing `await` in front of it defers processing to the event loop until the call completes, and everything after that line belongs to a callback executed only once the response is in. That code issues the second database call, again returns control to the loop while waiting, and the remainder, which returns the user and the orders, forms another callback.

8.2 The state-machine rewrite

Figure 7b has the same signature. Called with user ID 123, it initialises a variable `state = 0` and two variables for the user and the orders, then defines an inner function `step` and returns it; functions are first-class values in JavaScript and can be returned from other functions. Each call of `step` executes, through a `switch`, the code of the current state. On the first call the state is 0: `step` sets `state = 1` and starts the I/O operation `db.getUser(userId)`. Its `then` clause is the callback part: it assigns the result to `user` and calls `step` again. Between the start of the query and that callback the function is not running; the event loop is free to run other functions and tasks. On the next call, state 1 sets `state = 2` and starts `db.getOrders`, whose callback assigns `orders`; after the loop delivers that result, state 2 returns the user and the orders. Each `await` in the source becomes one transition between states.

```
// (a) as written
async function fetchUserData(userId) {
  const user = await db.getUser(userId);
  const orders = await db.getOrders(userId);
  return { user, orders };
}

// (b) roughly what the runtime makes of it
function fetchUserData(userId) {
  let state = 0, user, orders;
  function step() {
    switch (state) {
    case 0:
      state = 1;
      db.getUser(userId)
        .then(u => { user = u; step(); });
      return;                    // the event loop is free
    case 1:
      state = 2;
      db.getOrders(userId)
        .then(o => { orders = o; step(); });
      return;                    // the event loop is free
    case 2:
      return { user, orders };
    }
  }
  return step;
}
```

Figure 7: An async function (a) and an equivalent state machine (b); each await becomes a state transition (illustrative).

8.3 Two consequences

- **`await` is allowed only inside `async` functions.** Marking a function `async` is what makes the runtime transform it

into a state machine, and `await` marks the transitions; the two work only together.

- **Blocking the event loop is fatal.** Transitions are driven by the loop running callbacks. If the loop is blocked, no state machine can move from state 0 to 1 or from 1 to 2, and every pending function in the process stalls.

9 SHARED STATE AND RACE CONDITIONS

Concurrency brings its own set of problems, and nearly all of them, whatever their category, have one cause: shared state, i.e. shared memory.

9.1 The lost update

Two threads update a shared variable `counter`, initially 0, at the same time, each incrementing it by one. An increment takes three operations: read the current value of the counter into a CPU register; add one to the register; write the register back to the variable. Table 3 shows one interleaving over six time units. At 1 ms thread A reads 0; at 2 ms, before A has incremented anything, thread B also reads 0. At 3 and 4 ms both registers become 1; at 5 ms A writes 1, and at 6 ms B writes 1. Each thread added one, so the expected value is 2; because the operations interleaved, the final value is 1, and one increment has vanished completely. This is the **lost update** problem, an instance of a **race condition**: the result depends on the timing of concurrent operations on shared state.

Time	Thread A	Thread B	counter
1 ms	reads 0 into register	—	0
2 ms	—	reads 0 into register	0
3 ms	register 0 + 1 = 1	—	0
4 ms	—	register 0 + 1 = 1	0
5 ms	writes 1	—	1
6 ms	—	writes 1	1

Table 3: The lost update: two increments of a shared counter interleave, and the final value is 1 instead of 2.

9.2 Races without threads

A single-threaded runtime such as JavaScript's with `async`/`await` does not make race conditions disappear. In Figure 8 a global `balance` starts at 100. The function `withdraw(amount)` checks whether `balance >= amount`; if so, it awaits `processWithdrawal(amount)`, an asynchronous function that may run a database query, and then subtracts the amount from the global balance. Calling `withdraw(100)` twice can produce the interleaving of Table 4. The first call checks 100 ⩾ 100, enters, starts the asynchronous function and, at `await`, hands control back to the event loop. The second call now runs: its check also passes, because the balance is still 100, and it too yields at `await`. When the first call resumes it sets the balance to 100 − 100 = 0. When the second resumes, its database call having completed, it executes the deduction on a balance that is already 0: 0 − 100. The balance ends at −100, an invalid value. Had the second check run after the first deduction, it would have failed; it ran while the first call was suspended.

```
let balance = 100;

async function withdraw(amount) {
  if (balance >= amount) {            // 1. check
    await processWithdrawal(amount);  // 2. yield to loop
    balance -= amount;                // 3. act on old check
  }
}

withdraw(100);
withdraw(100);   // both pass the check: balance ends at -100
```

Figure 8: A check-then-act sequence across an await in single-threaded JavaScript (illustrative).

Step	First call	Second call	balance
1	100 ⩾ 100: enters	—	100
2	awaits, yields	—	100
3	—	100 ⩾ 100: enters	100
4	—	awaits, yields	100
5	resumes: 100 − 100	—	0
6	—	resumes: 0 − 100	−100

Table 4: Two overlapping calls of `withdraw(100)` on a balance of 100.

> **Every await is a yield point.** Between an `await` and the line after it, any other task may run and change shared state. A condition checked before an `await` must be treated as stale after it: check-then-act sequences that span an `await` need a lock or an atomic operation, exactly as with threads.

9.3 Locks and channels

Race-prevention mechanisms are treated in depth in operating-systems books; the best known are locks, or **mutexes**. In Python's `threading` library (Figure 9a), the increment function acquires a lock before the read-modify-write. The block inside is a *critical section* under *mutual exclusion*: while one thread holds the lock, only that thread can be inside, and every other thread trying to execute the same function waits until the holder has finished and released it. This prevents the race of Table 3, though only to a certain extent: only code that takes the lock is protected. For the withdrawal of Figure 8 the check and the deduction must likewise become one indivisible step, for example under an async lock or as a single conditional `UPDATE` in the database (see Chapter 8).

Go offers a different primitive, **channels**. Instead of many goroutines, threads or virtual threads updating a global or shared variable, they pass messages to each other, and only one particular goroutine updates the variable (Figure 9b). Many other techniques exist for the various problems of concurrency.

```
# (a) Python: a mutex around the read-modify-write
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    with lock:          # one thread at a time in here
        counter += 1    # read, add, write: now indivisible
```

```
// (b) Go: one goroutine owns the counter
inc := make(chan int)
go func() {
    counter := 0
    for d := range inc {
        counter += d    // the only writer
    }
}()
inc <- 1              // any goroutine sends a message
```

Figure 9: Two remedies for the lost update: a lock in Python (a) and a channel in Go (b) (illustrative).

10 CHOOSING A CONCURRENCY MODEL

10.1 The three models compared

Threads and goroutines both let code be written in a blocking, sequential style and both use several cores; they differ in who schedules them and in what each unit costs. The event loop avoids a thread per task altogether, at the price of a single thread that must never be blocked (Table 5).

Aspect	OS threads	Event loop	Goroutines
Scheduled by	OS, preemptively	the loop; tasks yield at `await`	Go runtime
Cost per task	MB-scale stack, kernel call	a callback or state object	KB-scale stack
Switch	kernel context switch, 1–10 µs	call the next callback	runtime switch, about a pointer change
Code style	blocking, sequential	callbacks, `async`/`await`	blocking, sequential
Uses many cores	yes	no, one loop per process	yes, up to `GOMAXPROCS`
Strongest at	CPU-bound work	many I/O waits	both

Table 5: The three concurrency models compared.

10.2 Matching the model to the workload

For I/O-bound workloads that need a high degree of concurrency, such as web servers, API gateways and services that make many calls to other services, `async`/`await` on an event loop, goroutines on Go's runtime, and the virtual threads of modern Java are far more efficient than the classic thread model. For CPU-bound workloads, meaning number crunching, string manipulation, image or video processing, moving bytes in and out of memory, any work done by the CPU itself, threads running in parallel are more efficient than an event-loop architecture.

Concurrency keeps a program productive by not wasting CPU cycles while it waits for input and output, in the broad sense of any work not done by the CPU: the network card sending and receiving packets, standard input and output, calls to external APIs or databases. Parallelism lets a program use several cores to do several things at the same instant. Most backend work leans towards concurrency, not parallelism; some systems need both, or more parallelism than concurrency. Understanding the difference between I/O-bound and CPU-bound work, the one concept this lecture is built around, is what allows choosing the right tool.

11 CONCLUSION

- A typical request waits far longer than it computes: about 250 ms of I/O against 10 ms of CPU leaves the hardware idle some 95% of the time unless the server is concurrent.
- Parallelism executes several instructions at the same instant and needs several cores; concurrency interleaves tasks by pausing and resuming them and works on a single core.
- I/O-bound work needs concurrency; CPU-bound work such as image processing, heavy cryptography, video or ML benefits from parallelism; most backends are I/O-bound.
- OS threads are scheduled preemptively by the kernel and share their process's memory, but cost stack memory, creation time and 1–10 µs per context switch, which limits thread-per-request designs.
- An event loop multiplexes thousands of I/O waits on one thread through `epoll` or `kqueue`; its one rule is never to block the loop.
- Callbacks, promises and `async`/`await` are syntactic sugar for yielding at I/O; an async function behaves like a state machine whose transitions are its awaits.
- Go runs cheap goroutines, one per request, on `GOMAXPROCS` OS threads through its own runtime scheduler, combining blocking-style code with efficient I/O and parallelism.
- Shared mutable state causes race conditions such as the lost update, with threads and with single-threaded async code alike; locks and channels are the classic remedies.
- Choose by workload: async, goroutines or virtual threads for highly concurrent I/O; parallel threads for CPU-bound computation.

APPENDIX A REVIEW QUESTIONS

1. A single Node.js process on a 4-core VM serves an endpoint that runs four sequential database queries of 30 ms each and spends about 6 ms of CPU on parsing, validation and JSON serialisation. At 150 requests per second, estimate how many requests are in flight at once and how busy the event-loop thread is. Predict what happens to latency when traffic doubles, and decide whether the endpoint should then be called I/O-bound or CPU-bound.
2. After a team adds password hashing with bcrypt's synchronous API (about 250 ms of CPU per hash) to the login route of an Express service, the p99 latency of every route, including a `/health` route that does no work, climbs to several seconds during morning login peaks;

one core sits at 100% while three are idle. Explain the mechanism in terms of the event loop, explain why enlarging the database connection pool would not help, and propose two fixes with their trade-offs.

3. A Java service on 4 cores uses a servlet container with a pool of 200 platform threads; each request waits 150 ms on a downstream HTTP API and needs about 1 ms of CPU. Traffic grows from 800 to 2,000 requests per second. Predict the throughput ceiling, where the excess requests wait and which metric reveals it, then compare raising the pool to 2,000 threads with moving to virtual threads in memory, context switching and code changes.
4. In JavaScript, `count` starts at 0 and the async function `inc()` executes `const c = count;`, then `await sleep(10);`, then `count = c + 1;`. The program runs `await Promise.all([inc(), inc(), inc()])` and prints `count`. Predict the printed value and justify it with a timeline like Table 3; then predict the value when the `await` line is deleted, and explain why a single-threaded runtime can still need a lock.
5. A service has a thumbnail endpoint that spends 200 ms of pure CPU per request and a `/status` endpoint that returns a constant. Fifty thumbnail requests arrive together, followed by one `/status` call. For a single Node.js process and for a Go service with `GOMAXPROCS` = 4 on 4 cores, estimate when the last thumbnail completes and what the `/status` caller experiences, and justify both from the scheduling models of this chapter.
6. A photo-sharing backend written in Node.js on an 8-core machine must accept uploads, store originals in object storage and produce three resized versions costing about 300 ms of CPU each. Design how the work is split between the event loop, the remaining cores and possibly other machines so that upload latency stays low during bursts. Justify each placement in I/O-bound versus CPU-bound terms and relate the design to the task queues of Chapter 10.
7. A C# service counts page views in a static integer that request handlers on many thread-pool threads increment; under load the total is noticeably lower than the access logs show. Wrapping the increment in a lock makes the numbers match, until the service is scaled to three replicas behind a load balancer. Explain both failures and propose a design that stays correct for any number of replicas, drawing on Chapters 8 and 9.
8. Three fixes are proposed for the `withdraw` function of Figure 8: (a) subtract the amount before the `await` and add it back if `processWithdrawal` throws; (b) hold an async mutex across the check, the `await` and the deduction; (c) drop the in-memory balance and run one conditional `UPDATE` in the database. Compare their correctness when two calls overlap, their behaviour if the process crashes mid-operation and their throughput, and state which of them survive running several instances of the service.

APPENDIX B GLOSSARY

Term	Meaning
Concurrency	Structuring a program so that several tasks are in progress at once, interleaved by pausing and resuming them
Parallelism	Executing several instructions at the same instant on several cores
I/O-bound	Work whose duration is dominated by waiting for network, disk or devices
CPU-bound	Work whose duration is dominated by computation on the processor
Process	A running program with its own memory, isolated from other processes
Thread	OS-scheduled unit of execution with its own stack and instruction pointer, sharing its process's memory
Preemptive scheduling	The OS interrupts a thread when its time slice ends, finished or not
Context switch	Saving one thread's registers and state and restoring another's; pure overhead
Blocking operation	A call during which the thread cannot proceed until I/O completes
Event loop	One thread that repeatedly runs ready callbacks and asks the OS which I/O has completed
epoll / kqueue	Linux and BSD/macOS interfaces that report which of many sockets are ready
Callback	A function handed over in advance, run when an awaited operation completes
Goroutine	Go's lightweight unit of execution, scheduled by the Go runtime
GOMAXPROCS	Go setting for how many OS threads execute Go code at once; defaults to the core count
Race condition	A result that depends on the timing of concurrent operations on shared state
Mutex	A lock that admits one thread at a time into a critical section

APPENDIX C FURTHER TOPICS

Multi-threaded async runtimes

.NET's `async/await`, Rust's Tokio and Kotlin's coroutines combine the two basic models: tasks yield at `await` as on an event loop, but their continuations run on a pool of OS threads, so many tasks are both concurrent and parallel. ASP.NET Core, for instance, serves requests on the .NET thread pool, with I/O completion ports or `epoll` underneath. For a backend engineer this means that blocking a pool thread (with `.Result`, `.Wait()` or `Thread.Sleep`) can starve the pool much as blocking code stalls Node's loop, and that state shared across awaits can be touched by two threads at the same instant. It applies whenever async code is written in C#, Rust or Kotlin.

Offloading CPU-bound work

An event-loop service keeps heavy computation off the loop by handing it to other threads or processes: `worker_threads` in Node.js, a process pool in Python, or a separate worker service fed by a queue. The loop stays responsive while the computation uses other cores, at the cost of copying or serialising data between workers and of sizing the pool. The technique applies as soon as a single CPU-

bound step takes more than a few milliseconds, as with image resizing, report generation or password hashing.

The global interpreter lock

CPython's global interpreter lock (GIL) lets only one thread at a time execute Python bytecode. Python threads therefore give concurrency for I/O-bound work, because the lock is released while a thread waits, but no parallelism for pure-Python CPU-bound code; multiple processes, or native extensions that release the GIL, are the usual way to use several cores. PEP 703 introduced an optional free-threaded build of CPython starting with Python 3.13. The GIL decides whether threads, `asyncio` or multiprocessing suit a given Python workload.

Atomic operations and compare-and-swap

Processors offer indivisible instructions such as atomic add and compare-and-swap (CAS), which perform a read-modify-write as one step. Languages expose them as `sync/atomic` in Go, `Interlocked` in .NET and `java.util.concurrent.atomic` in Java. An atomic increment fixes the lost update without a lock and is cheaper under low contention, and CAS loops are the basis of lock-free data structures. Atomics suit single variables such as counters, flags and reference counts, but not invariants spanning several variables, which still need a lock or a transaction.

Deadlocks

A deadlock occurs when two or more threads each hold a lock that another needs, so all of them wait forever. It requires four conditions together, the Coffman conditions: mutual exclusion, hold-and-wait, no preemption and circular wait. The standard prevention is to acquire locks in one global order, keep critical sections short and free of I/O, or put timeouts on acquisition. Databases face the same problem between transactions and resolve it by detecting the cycle and aborting one transaction, which the application must be ready to retry.

Concurrency control in the database

When the shared state lives in a database, races between requests are prevented with the database's own tools: transaction isolation levels, pessimistic row locks (`SELECT ... FOR UPDATE`), optimistic concurrency with a version column, or a single conditional `UPDATE` whose `WHERE` clause re-checks the condition. Unlike an in-process mutex, these work across threads, processes and replicas. They are the right tool for balances, stock levels, seat bookings and any invariant that several instances of a service can touch.

Bounded concurrency and backpressure

Concurrency is cheap inside a process, but the resources behind it are not: an unbounded fan-out of async calls or goroutines can exhaust a database connection pool, a partner's rate limit or the process's memory. Semaphores, bounded worker pools and queues with a maximum length cap the work in flight, and a full queue pushes back on callers, for example with HTTP 429 or 503. Any service that fans out calls or accepts unbounded traffic needs such limits, precisely because cheap tasks make overload easy to create.

Cancellation and timeouts

When a client disconnects or a deadline passes, the work started on its behalf should stop instead of holding connections and CPU. Go propagates cancellation and deadlines through `context.Context`, .NET through `CancellationToken` and JavaScript through `AbortController`; I/O calls accept the signal and abort. Without it, abandoned requests pile up behind a slow dependency and amplify an outage (see Chapter 12). It applies to every outbound network or database call.

Amdahl's law

Amdahl's law bounds the speed-up that parallelism can deliver: if a fraction p of a job can be parallelised, n cores yield at most $1 / ((1 - p) + p/n)$. With $p = 0.9$, eight cores give about 4.7× and infinitely many at most 10×. The law shows a backend engineer when adding cores or threads to a CPU-bound job stops paying off, and why the serial part, such as a lock every task needs, must be shrunk first.

Object Storage, Part 1 — One-Page Overview

Chapter 20 · every section and sub-section of the paper at a glance

1 Introduction: Why Files Need Their Own Storage

Saving uploads to the local disk and their paths to the database collapses once files grow or servers multiply.

- **1.1 The naive design** — Form data in, file to ext4, path into the database.
- **1.2 The 4 GB video** — Whole body read into 4 GB of RAM: OOM kill, restart, files gone, 404s across instances.
- **1.3 Files are not application data** — Different structure, access model and cost profile: they need object storage.

2 Six Problems with Files on the Server's Disk

Six independent failures motivate a separate storage system.

- **2.1 Ephemeral container disks** — Each restart or deploy starts a fresh container; a persistent volume fixes only this.
- **2.2 Horizontal scaling** — Round robin reaches the instance holding the file with probability 1/*N*.
- **2.3 Fixed disk size** — Disks grow vertically, with downtime and a ceiling; uploads grow fastest.
- **2.4 Single-disk durability** — Availability can wait, durability cannot; replication is a field of its own.
- **2.5 Serving downloads** — 50 MB at 2 Mbit/s holds a worker for minutes: I/O-bound work for a CDN.
- **2.6 No transaction across disk and database** — Orphan files or dangling paths, whichever write goes first.

3 Block, File and Object Storage

Three interfaces that trade richness against scale.

- **3.1 Block storage** — Numbered 512 B / 4 KB blocks, microsecond latency, one machine (SSD, EBS).
- **3.2 File storage and POSIX semantics** — Seek, atomic rename, locks — coordination that does not cross networks.
- **3.3 Object storage: the minimal interface** — PUT, GET, DELETE, LIST; immutable objects, no rename, millisecond latency.
- **3.4 What the restrictions buy** — No coordination, no tree, any node serves any request; immutability buys scale.

4 Objects, Buckets and Keys

Flat keys in globally named buckets; folders are a listing trick and key layout is a design decision.

- **4.1 Anatomy of an object** — Key, value, system metadata (size, ETag …) and user metadata.
- **4.2 Buckets** — Named containers, unique across the platform.
- **4.3 Folders are an illusion** — Prefix + delimiter → `CommonPrefixes`; moving a folder copies every object.
- **4.4 Key design** — Never the user's filename; randomness first; owner in the key.

5 Inside an Object Store

A simple data plane beside a hard, strongly consistent metadata plane.

- **5.1 The data plane and the metadata plane** — Huge bytes and simple operations versus tiny records at a huge rate.
- **5.2 Replication versus erasure coding** — Reed–Solomon 14 + 6: 1.43× storage, six failures survived.
- **5.3 Eleven nines, and why durability is not backup** — One object in 10 million per 10,000 years; versioning still needed.
- **5.4 The metadata index** — Range-partitioned sorted index: hot prefixes, slow LIST, no rename.
- **5.5 Strong read-after-write consistency** — S3 since December 2020; retry-after-upload loops are obsolete.

6 Conditional Writes

Since 2024 S3 can refuse a write, which makes it a coordination primitive.

- **6.1 Silent last-writer-wins** — For almost 20 years lost updates were undetectable.
- **6.2 If-None-Match: create only if absent** — `If-None-Match: *` returns 412 if the key exists.
- **6.3 If-Match: compare-and-swap with ETags** — A stale writer gets 412 instead of erasing another update.
- **6.4 A coordination primitive** — Locks, leader election, write-ahead logs on S3.
- **6.5 S3-compatible is not a guarantee** — Check each provider's documentation.

7 Uploading Through the Backend

Proxying suits small files only if it streams, and it still meets infrastructure limits.

- **7.1 The proxy architecture** — Sees every byte; plain middleware authorization; no CORS.
- **7.2 The buffering trap** — 20 × 100 MB uploads exhaust 2 GB; invisible in local development.
- **7.3 Streaming and a body-size cap** — Chunks go straight to the SDK: flat memory for a 500 MB file.
- **7.4 Limits that streaming cannot remove** — 60 s balancer timeouts, Nginx 413, serverless payload caps.

8 Presigned URLs

The browser uploads straight to a private bucket with short-lived, signed permission.

- **8.1 Direct upload to a private bucket** — One operation, one key, a few minutes, no credentials.
- **8.2 Anatomy of a presigned URL** — Query parameters plus an HMAC over the request description.
- **8.3 Demo: expiry and tampering** — 200 at first; 403 after expiry or one changed character.
- **8.4 The missing size limit** — A profile-photo URL accepted 300 MB.
- **8.5 Presigned POST with a policy** — Size range, content type and key prefix enforced by the store.

9 Keeping the Database and the Bucket in Sync

A two-phase upload and a cleanup job reconcile two sources of truth.

- **9.1 The two-phase upload** — Pending row and key first; completion verified with HEAD.
- **9.2 Abandoned uploads and reconciliation** — Lifecycle rule and background job after 24 hours.
- **9.3 CORS for direct uploads** — Allow the origin and PUT/POST; expose `ETag`.

Object storage at a glance — local disks fail on ephemerality, scaling, size, durability, delivery and consistency; object storage trades mutability for unlimited, coordination-free scale, and production uploads go from the browser straight to a private bucket through a presigned POST policy inside a two-phase workflow. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Object Storage, Part 1: From Local Disks to Presigned Uploads

Chapter 20 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Saving uploaded files on the server's local disk and their paths in the database fails in production for six reasons: ephemeral container disks, horizontal scaling, fixed disk size, single-disk durability, the cost of serving downloads, and the absence of a transaction between disk and database. A comparison of block, file and object storage shows that object storage gives up in-place modification, hierarchy and rename in return for coordination-free, effectively unlimited scale and direct HTTP access. The chapter then describes objects, buckets, the folder illusion created by prefixes, and three rules for key design. It looks inside an object store — data and metadata planes, erasure coding, eleven nines, the sorted metadata index, strong consistency — and presents conditional writes as a coordination primitive. Finally it compares uploads through the backend with presigned URLs and presigned POST policies, and closes with the two-phase upload that keeps the database and the bucket consistent.

Keywords — object storage, Amazon S3, erasure coding, strong consistency, conditional writes, ETag, presigned URL, POST policy, two-phase upload

1 INTRODUCTION: WHY FILES NEED THEIR OWN STORAGE

A backend that already has authentication, a database, services and repositories receives a simple, practical requirement: users must be able to upload files — in the basic scenario, to update their profile photo. Large files (multipart and resumable uploads), upload sizes, range requests and video seeking, and the cost profile of object storage are treated in part 2 (see Chapter 21).

1.1 The naive design

Without knowing how production systems or textbooks do it, intuition produces three steps. The browser sends the file as *form data* (`multipart/form-data`, the encoding browsers use to transfer files). The handler extracts the file from the request and saves it in the server's local file system — typically ext4 or a similar file system, since most servers run Linux — because a file seems to belong in a file system, and the server is a physical machine with its own memory, processor and storage. Finally, it records where the file was saved by writing the path to the database, because the next request will be for the file itself: the browser must fetch it to display the profile photo. On that request the backend reads the path from the user's row and sends the file back. The design is transparent and works in a demo. It is also wrong, and it can break as early as the first time the user asks for the file.

1.2 The 4 GB video

The first and most obvious problem is size. Profile photos are small: roughly 300 KB to 10 MB from a phone such as an iPhone, perhaps 15–20 MB from a high-end mirrorless camera. Such files fit easily in the service's memory and cause no traffic spikes. Then a new requirement brings video — posts in a social-network feed, or videos sent between users in a messaging feature — and files now range from several megabytes to several gigabytes.

Keeping the same scheme, a user uploads a 4 GB video to publish in the feed. A typical startup runs instances with about 4 GB of memory each, and the framework, whichever one is used, kindly reads the entire file into memory before handing it to the handler. The instance runs out of memory and the container is terminated instantly. With the automatic restarts of a production setup (Chapters 17 and 18), the container comes back — and every file saved on its local file system is gone, avatars and videos alike, because a container's file system is *ephemeral*: not permanent, not durable, alive only until the container restarts. The backend was also scaled to three instances last month to cope with load. Files uploaded to the other two instances survived, but they exist only on those instances; the restarted instance cannot reach them, and a large share of profile-photo requests now return 404 Not Found.

1.3 Files are not application data

Both failures follow from one false assumption: that a file is just more of the application's data. In about 95% of cases a server deals with JSON — small, structured records. Files have a completely different structure, a different access model and a different cost profile, and, as the failures show, they need a different storage system: *object storage*. Its production upload workflow is, in one phrase, presigned URLs and the processes built around them (Section 8). Object storage is also an area of backend work where cost has to be weighed constantly.

2 SIX PROBLEMS WITH FILES ON THE SERVER'S DISK

2.1 Ephemeral container disks

Everything a container saves — in main memory or on its disk — exists only until the next restart. That is what *ephemeral* means, and it applies however the backend is deployed: with Docker and Kubernetes, on a platform-as-a-service such as Railway or Render, or on serverless plat-

forms such as Vercel and Netlify (see Chapter 27). When a container exits or restarts, a new one takes its place: the same image, but a brand-new container, which is one reason application containers are called *stateless*. Deployments do this on purpose: CI/CD builds a new image with the latest code, the old container is destroyed, and a fresh one is started from the new image. The image contains the code, not the users' files, so every deployment deletes every file kept in the container's file system.

Readers who know the container ecosystem will think of a *persistent volume*: storage attached to a container that outlives it. Persistent volumes exist to counter the ephemerality of containers, not to hold user files, although they can be used that way, and using one is a legitimate fix for this first problem. It does nothing for the next two.

2.2 Horizontal scaling

Three instances run behind a load balancer, a normal setup (Chapter 17). A user uploads a file; the balancer routes the request to instance 1, which writes the file to its local disk. Two minutes later the user comes back to the page and the browser requests the photo. The request passes through the load balancer, the first point of contact with the infrastructure, which distributes requests with an algorithm — here simple *round robin*, cycling through the instances in a fixed order. Other requests have advanced the rotation in the meantime, so this one lands on instance 3, which knows nothing about the file and returns 404. The more instances are added for load or traffic, the worse it gets: the probability of landing on the instance that holds the file is about $1/N$ for N instances. Local files violate the most important property of a horizontally scalable server, statelessness: a server must never hold state that the other servers do not have.

A subtle point. If a request reaches the right instance with probability $1/N$, it fails with probability $(N - 1)/N$: two requests in three with three instances, not "every third request" as the opening scenario puts it. Files that lived on the restarted instance fail on every request.

2.3 Fixed disk size

Instances scale horizontally; a disk scales only vertically. A server cannot keep acquiring new disks, so growth means enlarging the existing one, which on most clouds involves detaching the disk, increasing its size and attaching a disk with the data copied over — usually with downtime. Every provider also has a maximum disk size. Meanwhile, user uploads are the fastest-growing data in almost any application, because many users upload files at every moment of the day: 1,000 users each uploading 100 MB a month add 100 GB every month, forever, since people delete far fewer files than they upload.

2.4 Single-disk durability

The server has one disk, and disks fail in real life for many reasons — power failures, hardware faults; even laptop disks die. This is normal and can happen at any moment. If the disk holding all user files dies, the files are gone, because it held the only copy.

Availability and durability. *Availability*: can the data be accessed right now? *Durability*: does the data still exist anywhere in the world, that is, does at least one copy survive? An availability problem can be solved, in the worst case, by waiting: once the server is repaired, users get their data. A durability problem cannot be waited out, because the file no longer exists.

The industry's general answer to durability is *replication*, a grand word for keeping copies of the data elsewhere, so that several copies exist at every moment. Copies in turn require checksums and background processes that detect corrupted data and repair it. Replication, checksums and recovery processes form a separate, large engineering field — not something to build for a to-do-list application.

2.5 Serving downloads

Once a file is stored, the server must also send it back on request, and that is work too. A 50 MB file sent to a phone on a 2 Mbit/s mobile connection (megabits, not megabytes) is 400 Mbit, or 200 seconds — more than three minutes. For that whole time one worker of the server — a goroutine in Go, an open stream on the Node.js event loop, whatever the language or concurrency primitive — does nothing but copy bytes slowly over a poor connection to a phone thousands of miles away. Servers can do this, and constantly do, but it is a poor use of their resources: the server becomes a very expensive, very slow content delivery network, limited to perhaps 100 simultaneous connections. Content delivery networks (CDNs) exist to solve exactly this problem. As discussed in Chapter 19, a server's work is either *CPU-bound* — computation that runs on the processor's cores — or *I/O-bound*: data is sent to, or awaited from, another component such as a database, an external client or API, or the file system. Long I/O-bound work of this kind is what a backend should avoid as far as possible.

2.6 No transaction across disk and database

The file system and the database are two completely separate systems, and neither controls the other. Write the file to disk and then insert its path as a row: if the insert fails, the file stays on disk with no row pointing to it. Users cannot reach it; it is an *orphan* that, at best, costs extra money for storage. Reverse the order — insert the row as soon as the file arrives, then write the file — and a failed write leaves a row with an invalid path: every request for the profile photo follows the path to nothing and, depending on the error handling, returns 500 or 404. Without a transaction spanning both systems there is no way to guarantee that both operations succeed or both fail. Object storage does not remove this problem and is not a direct solution to it, but it makes the problem easier to solve with a specific pattern (Section 9). Table 1 summarises the six problems.

Problem	Cause	Symptom	Object storage
1 Ephemeral disk	fresh container per restart or deploy	files vanish	solves
2 Horizontal scaling	file on one instance only	404 on the others	solves
3 Fixed disk size	disks scale vertically	downtime, hard ceiling	solves
4 Single disk	one copy of each file	permanent loss	solves (5.2)
5 Serving downloads	slow clients hold workers	wasted server capacity	offloads to HTTP clients
6 No transaction	two independent systems	orphans, dangling paths	eases (Section 9)

Table 1: Why files on the server's local disk fail in production

3 BLOCK, FILE AND OBJECT STORAGE

A backend can use essentially three kinds of storage, each with its own set of trade-offs; Table 2 compares them.

3.1 Block storage

Block storage provides an array of fixed-size blocks, usually 512 bytes or 4 KB each, and every block has a number. That is almost the entire interface: at any time one can read or write, say, block number 9,000,000, and the device does it. There is no concept of files, names or folders. An SSD (solid-state drive) is exactly this, and so is what services such as Amazon EBS (Elastic Block Store) offer; file systems such as ext4 on Linux or NTFS on Windows are built on such blocks. Block storage is the lowest-level interface and usually the fastest, with typical latencies measured in microseconds. That is why databases such as Postgres and MySQL run on block storage: they still use a file system, but not the way users do — they manage their own data layout and keep their path to the block device as direct as possible to preserve those microseconds. The trade-offs: the interface is so basic and primitive that an application cannot be built directly on it, and — a critical constraint — a block device can be attached to only one machine at a time, so the same storage cannot be shared by several machines.

3.2 File storage and POSIX semantics

File storage — a file system — is a layer of abstraction over block storage. On top of blocks it builds a hierarchy of directories (folders), files and paths, with access permissions, inodes and the rest of the file-system machinery. In return it offers a very rich interface: open and close a file; jump to byte 9,000,000 or 5,000,000; overwrite exactly four bytes in the middle of a file; rename a folder instantly and atomically; lock a file; append to it; update it — every operation performed daily through programs such as Finder or a file manager. These operations are *POSIX semantics*. POSIX is a standard, primarily for operating systems, that defines the operations a typical OS must offer; an OS that supports most of it is called POSIX-compliant, and the operations above form its file-system part.

The convenience has a price: the operations need coordination. Rename, performed without a second thought, must be completely atomic — it either completes or does not happen, with no intermediate or damaged state — because many applications and system components depend on that property and would fail at once without it. Partial writes must be seen consistently. When a file is edited in VS Code or Sublime Text, the editor process, which holds the file handle, writes each edit to a temporary file in the location the OS provides for temporary files; only on save is the new version put in place of the old one and the temporary file removed. A reader at any moment gets either the original version or the complete new one, never a partial state mid-write. Likewise, two processes writing to the same file — very common with log files, where many programs write at once — need some form of locking. This locking and coordination does not scale across network boundaries, data centres or continents. Network file systems such as NFS exist, with their own set of trade-offs.

3.3 Object storage: the minimal interface

Object storage starts from a different question. Instead of offering the richest interface, it asks: what is the minimum interface needed to scale to unlimited capacity — exabytes, "planetary scale", further than any file system, at least as far as coordination is concerned? The answer is four operations: put (write, upload) an object by name, get an object by name, delete an object by name, and list the names. That is essentially the whole interface.

The trade-offs follow. What is stored is called an *object*, not a file: technically it is not quite a file, and calling it one raises expectations it does not meet. An object cannot be changed in place — it cannot be updated or appended to. To change one byte in the middle of a 1 GB object, the client must download the whole 1 GB, apply the change and upload the whole object again under the same key: objects are immutable and can only be replaced. There are no real directories or folders; providers create an illusion of folders with *prefixes* (Section 4.3). There is no rename: renaming means copying the entire object under the new name and deleting the old one, so a rename costs as much as the object's full size. There is no file locking. And latency is not microseconds but at least several milliseconds, often tens of milliseconds, because every operation is an HTTP request that crosses the network to a machine somewhere in a data centre.

> **A subtle point.** The lecture also lists "no partial reading" among the restrictions. Object stores do support partial *reads*: a GET with an HTTP `Range` header returns any byte range of an object, which is how video seeking works (see Chapter 21). What is missing is partial *writes* — no seeking to an offset to overwrite, no append.

3.4 What the restrictions buy

With such restrictions, why use object storage at all? First, because nothing is modified in place, there is nothing to coordinate, and a whole class of distributed-systems problems disappears. Second, there is no hierarchy at all — no folders, no directory structure — so there is no tree to

maintain, lock or rebalance as ordinary file systems must. Third, every operation is an independent HTTP request, so any of, say, 10,000 servers around the world can handle any request, and the system scales horizontally without limit. Capacity is effectively infinite: nothing is provisioned or resized, objects simply keep being uploaded. Well-known services such as Amazon S3 add a durability guarantee of "eleven nines" (Section 5.3). Finally, every object is directly reachable over HTTP, so a browser, a video player or a mobile app — almost every computing device today speaks HTTP — can interact with object storage directly, without an intermediate server.

The same set of trade-offs appears throughout distributed systems — in caching, in stateless servers, in immutable deployments. Almost whenever the goal is unlimited scale, the price is giving up *mutability*, the ability to modify something in place. Object storage is one of the purest and best-known examples of this trade-off.

	Block	**File**	**Object**
Unit	numbered block, 512 B or 4 KB	file in a directory tree	object under a key
Interface	read/write block *n*	POSIX: seek, rename, lock	PUT, GET, DELETE, LIST
Change in place	yes	yes, any byte	no, replace whole
Rename	—	atomic	copy + delete
Latency	microseconds	microseconds (local)	milliseconds to tens of ms
Sharing	one machine	local; NFS over network	any HTTP client
Capacity	fixed volume	fixed volume	virtually unlimited
Examples	SSD, Amazon EBS	ext4, NTFS, NFS	Amazon S3, Cloudflare R2

Table 2: The three kinds of storage available to a backend

4 OBJECTS, BUCKETS AND KEYS

4.1 Anatomy of an object

An object consists of four elements. The **key** is a string that completely and uniquely identifies the object. The **value** is simply an array of bytes, a binary blob. **System metadata** is maintained by the store: size, last-modified time, content type and *ETag*. **User metadata** is a set of arbitrary key–value pairs that the application adds for its own purposes: to record who uploaded an object, for example, add a field `uploaded-by` holding the user's ID from the application's database. It is stored with the object, at the object-storage level (in S3, as `x-amz-meta-*` headers).

4.2 Buckets

Objects live inside a *bucket*. A bucket is not a folder but a named container one level above objects. Bucket name plus object key identify an object globally, because the bucket name itself must be globally unique: on a cloud platform such as AWS, a bucket's name must be unique across the entire platform, not just within one account.

4.3 Folders are an illusion

The biggest trick of object storage is that there are no folders and no directories; the concept does not exist. In the AWS console or the Cloudflare dashboard one sees a folder `uploads`; double-clicking it shows a folder `2026`, and inside it an image. The hierarchy is an invention, created so that the interface feels intuitive to people used to hierarchical files. Behind the scenes there is a single object whose key is the whole string `uploads/2026/cat.png`. The slashes are ordinary characters, part of the name with no special meaning: a hyphen, a hash or any other character would do. The "folder" is not a separate object; there is no parent and no nesting — just one bucket containing millions, billions or trillions of key strings.

The lecturer demonstrates this with the AWS CLI (Figure 1). One object is uploaded with the key above. The management console now shows two folders, which it has invented. A raw listing shows what actually exists: one key, one string. When the listing is requested with the prefix `uploads/` and the delimiter `/`, the "folder" appears in a field called `CommonPrefixes` — and that array is the entire folder feature. The server finds all keys in the bucket that begin with the prefix; for each it examines the rest of the key, and if the rest contains the delimiter, cuts it off after the delimiter and reports the result as one common prefix. Folders are a string-grouping algorithm.

```
$ aws s3api put-object --bucket demo-bucket \
      --key uploads/2026/cat.png --body cat.png

$ aws s3api list-objects-v2 --bucket demo-bucket \
      --query "Contents[].Key"
[ "uploads/2026/cat.png" ]   <- one key, no folders

$ aws s3api list-objects-v2 --bucket demo-bucket \
      --prefix uploads/ --delimiter /
"CommonPrefixes": [ { "Prefix": "uploads/2026/" } ]
```

Figure 1: The folder illusion with the AWS CLI: one key, two console "folders", one common prefix (illustrative)

Two consequences follow. Moving a "folder" inside a bucket takes a long time and costs money, because there is no folder to move: there are, say, 40,000 individual objects, each of which must be copied under a new key and the old one deleted. And an empty folder cannot exist, since folders have no identity in object storage. Tools imitate empty folders by creating a zero-byte object whose key ends with a slash; because of the cut-off algorithm, the listing then shows a folder — technically a trick played on the system with an empty object.

4.4 Key design

Because the key is the object's entire identity, choosing it is one of the most consequential decisions a backend engineer makes with object storage. The lecturer gives three rules (Figure 2).

Rule 1: never use the user's filename as the key. There are three separate reasons. *Collisions*: if two users upload `resume.pdf`, the second silently replaces the first.

The upload operation in object storage is called PUT for a reason: as in HTTP and REST (Chapters 1 and 7), PUT replaces the whole resource at that address, whereas POST creates a new record. A PUT to an existing key overwrites whatever is there — no error, no warning that something was replaced. *Path traversal*: a filename can contain dots and other special characters, and depending on what is done with the name afterwards, it can escape the prefix — the "folder" — the application intended to write into.*Arbitrary content*: a filename can contain emoji, characters that switch the text direction to right-to-left, null bytes — almost anything a random user can send — and any of it can become an error somewhere in the backend. The solution: generate the key yourself, as an opaque random identifier, and keep the original filename in the database, optionally also in the object's user metadata. The original name is used only when that user asks to download the file.

Rule 2: put anything random at the beginning of the key, not at the end. Many engineers write keys as `uploads/<timestamp>/<id>`. The object store partitions its index by key prefix — it is a huge sorted index (Section 5.4). If every key written during the next hour starts with the same prefix because it starts with the timestamp, every upload of that hour lands in the same index partition, the one people call a *hot partition*; when traffic grows, clients receive `503 SlowDown` errors. The fix is *high entropy* — strong randomness — at the very start of the key: a UUID, a generated user identifier, or the first few characters of a hash. Instead of `date/user/file`, use a layout such as `tenant/user/uuid`. AWS has greatly improved automatic partitioning in recent years, and a small application will probably never hit the problem, but doing it right costs nothing on day one, unlike changing strategy at significant scale. (S3 documents at least 3,500 writes and 5,500 reads per second per partitioned prefix.)

Rule 3: let the key carry its owner. This is the lecturer's personal advice. With keys of the form `<tenant-id>/<user-id>/<object-id>`, an authorization check in middleware is a string comparison, done before any request to S3, and an access policy can restrict a set of credentials to a single prefix. With random UUIDs in a flat namespace, almost every authorization check has to go through the database. A naming decision saves a network request.

```
avoid   resume.pdf                     the user's filename
avoid   uploads/2026-11-01T14/8812.png hot prefix per hour
prefer  t_42/u_7/f47ac10b.png          owner first, random id
        original name "resume.pdf" -> database row, and
        optionally x-amz-meta-original-name
```

Figure 2: Key layouts to avoid and to prefer (illustrative)

5 INSIDE AN OBJECT STORE

5.1 The data plane and the metadata plane

What happens when a PUT request carries a 100 MB file? The examples are from AWS S3, the best-known object store. The request first reaches one of thousands — or millions — of stateless front-end nodes run by S3. That node authenticates the request by verifying its signature (Section 8.2). From there the system splits the work into two completely separate problems, one of the fundamental architectural ideas of object storage (Figure 3). The *data plane* must accept 100 MB of bytes and store them so that they survive — so that they exist and do not disappear. The *metadata plane* must remember that the key `uploads/2026/cat.png` in this bucket points to these particular pieces of data in these particular places. The two have opposite characteristics. The data plane handles enormous volumes but very simple operations. The metadata plane handles a tiny amount of data by comparison, but a vast number of operations, and it must be *strongly consistent* and extremely fast.

```
PUT /uploads/2026/cat.png        (100 MB, signed)
  |
  v
front-end node: one of thousands, stateless;
verifies the signature, then splits the work
  |                              |
  v                              v
DATA PLANE                     METADATA PLANE
store 100 MB so it survives    remember (bucket, key)
14 data + 6 parity fragments     -> fragment locations,
on 20 disks, racks, power        size, ETag, content type
domains
huge volume, simple ops        tiny records, huge op rate,
                               strongly consistent, fast
```

Figure 3: A PUT request split between the data plane and the metadata plane (illustrative)

Strong consistency means that once one user has created or uploaded an object, any other user who reads it is guaranteed to get it, even a second later. *Eventual consistency* means there is some delay — a gap between the latest write and a read that reflects it — before others can see the new data. Of the two planes, the metadata plane is the harder engineering problem. Storing exabytes on disks has been a solved problem for decades; a strongly consistent, globally distributed, sorted index over hundreds of trillions of keys, where one lookup must take only a few milliseconds, is extremely hard. At AWS scale, countless production systems depend on precisely this property of object storage: how strong its consistency is.

5.2 Replication versus erasure coding

Durability simply means whether the data exists somewhere in the world or has disappeared for good. The naive way to make data durable is replication: keep three copies on three machines, ideally in three different *failure domains*, so that no single failure can affect all three at once. The architecture is simple and it works: if one machine dies, its data is copied again from another. The problem is cost. Storing one petabyte of customer data requires buying three petabytes of disk — 200% overhead — and at the scale of billions of customers that is billions of dollars spent on replication alone.

Large providers therefore use *erasure coding*. The object is split into, say, 14 fragments, and a mathematical transformation — the classical Reed–Solomon algorithm — computes six additional *parity* fragments, for 20 in total.

The 20 fragments are scattered over 20 different disks in different racks and different sections of the data centre, in separate power domains so that one power failure cannot take out all of them — ideally in different buildings. The scheme's remarkable property is that *any* 14 of the 20 fragments are enough to rebuild the original object completely, no matter which 14 survive. The numbers: 20 fragments for 14 fragments of real data is about 1.43 times the raw size, against 3 times for triple replication — less than half the price — while the system survives six simultaneous failures instead of two (Table 3). The price is processor time: computing the parity, plus the extra reads during reconstruction. Disks cost more than computation, so the exchange is clearly profitable.

Scheme	Raw storage	Failures survived	Extra cost
Triple replication	3.0× (200% overhead)	2 copies	disks
Reed–Solomon 14 + 6	20/14 ≈ 1.43×	any 6 fragments	CPU for parity; repair reads

Table 3: Replication versus erasure coding for durability

5.3 Eleven nines, and why durability is not backup

Amazon S3's famous promise is eleven nines of durability, 99.999999999%. Roughly, if you store 10 million objects, you can statistically expect to lose one of them once every 10,000 years. The figure sounds absurd, but that is what it means, and it is calculated from the fragment mathematics above. Earlier the lecture equated durability with having a backup copy for simplicity; strictly, durability is not backup.

> **Durability is not backup.** Eleven nines protect against destroyed or failed disks, not against your own actions. A DELETE on the wrong prefix — a system error or a programmer's mistake — is executed reliably, with eleven-nines confidence, on all data under that prefix, and durability is no ground for getting it back. If an application bug overwrites objects, the store will faithfully keep exactly the corrupted versions. For data that matters, enable *versioning*, so that an overwrite or delete creates a new version instead of destroying the old one, and replicate to another bucket, preferably in another cloud account, so that compromised credentials cannot reach both copies.

5.4 The metadata index

Conceptually, the metadata plane is a huge sorted key–value index. It can be pictured as a distributed B-tree — the balanced tree that databases use heavily (Chapter 8) — mapping bucket plus key to the locations of the shards, together with size, ETag, content type and the other system metadata. The index is partitioned by key range, which is the origin of the hot-partition problem of Section 4.4. Three properties follow from this structure.

- **LIST is slow.** Listing a bucket, even with pagination of 1,000 keys, is a range scan over a distributed sorted index, fundamentally different from and more expensive than a direct lookup of one key. Hence one of the most important rules: never use LIST on the request path. The database must be the index of which files exist; calling the list-objects API on every request to answer "which files does this user have?" does not scale and causes serious latency problems.
- **No atomic rename.** Renaming would mean changing the key in the index while the data stays immutable. That sounds simple, but the key determines the partition, so a renamed entry would have to move between partitions.
- **Instant metadata.** The store can report an object's size immediately, without touching the disk, because the size is in the index.

5.5 Strong read-after-write consistency

S3 used to be eventually consistent: a GET issued immediately after an upload could return 404, because the read was answered by an index that had not yet caught up with the write. This behaviour caused trouble for many systems and industries. Since December 2020 S3 provides *strong read-after-write consistency*: once an upload has returned 200, a read issued the very next second sees the object. Old blog posts and Stack Overflow answers still recommend retry loops or delays after an upload to avoid the 404; for S3 that pattern is now obsolete, since strong consistency is native.

6 CONDITIONAL WRITES

6.1 Silent last-writer-wins

Conditional writes are the lecturer's favourite feature of object storage and one reason why a write-ahead log on top of S3 is so widely discussed today: many distributed systems are built on S3 precisely because of it. The feature is fairly new. For almost 20 years object stores had no comparison and no *compare-and-swap*: every upload simply overwrote the existing data. If two processes wrote to the same key at the same time, one invisibly won and the other's write was lost. That alone would not be a problem if the loss could be detected, but it could not: there was no way to find out which of the two processes had actually succeeded, and no way to state a condition such as "write only if the object does not exist yet". This changed in 2024, when S3 added conditional writes.

6.2 If-None-Match: create only if absent

An upload (PUT) request may carry the header `If-None-Match: *`, meaning: create this object only if the key does not exist yet. If it does exist, the request fails with status 412 Precondition Failed instead of silently overwriting the existing object. In the lecturer's AWS CLI demonstration (Figure 4), the first command returns 200 and an ETag; repeating exactly the same command returns 412, because the previous command created the object and the header says to upload only when the key is absent.

```
$ aws s3api put-object --bucket demo --key locks/leader \
      --body me.json --if-none-match "*"
{ "ETag": "5d41402abc4b2a76b9719d911017c592" }     # 200 OK

$ aws s3api put-object --bucket demo --key locks/leader \
      --body me.json --if-none-match "*"
An error occurred (PreconditionFailed) when calling the
PutObject operation: At least one of the pre-conditions you
specified did not hold                              # 412
```

Figure 4: Conditional create with If-None-Match: the repeated request gets 412 (illustrative)

6.3 If-Match: compare-and-swap with ETags

The second header, `If-Match`, takes an ETag: replace the object only if its current content still matches what was read. An ETag is a fingerprint of an object's content, a kind of identifier of its current version. Two clients, A and B, illustrate it (Figure 5). Client A reads an object; S3 returns it with its ETag. A modifies the data locally and sends a PUT with `If-Match` set to that original ETag. If the ETag still matches the one S3 holds, the write succeeds and S3 generates a new ETag. But suppose that while A was working locally, client B updated the same object and B's request reached S3 first. S3 has already replaced the ETag because of B's write, so A's PUT fails with 412, "precondition failed". Without the condition, whichever write arrived last would have been kept and the other client's update silently lost; with it, B's update is protected and A learns that it worked on a stale copy.

```
t1  A: GET report.json          -> body, ETag "e1"
t2  B: GET report.json          -> body, ETag "e1"
t3  B: PUT  If-Match: "e1"      -> 200, new ETag "e2"
t4  A: PUT  If-Match: "e1"      -> 412 Precondition Failed
t5  A: GET again ("e2"), redo the edit, PUT If-Match: "e2"
without the header, t4 returns 200 and B's update is lost
```

Figure 5: Compare-and-swap with If-Match: client A's stale write is refused (illustrative)

6.4 A coordination primitive

The 412 is the important part: it is the building block of what distributed systems call a distributed lock. With these two headers, object storage gained a primitive on which coordination can be built. That is why people now build distributed systems, leader election and even databases on top of object stores, including databases that keep their *write-ahead log* there: a write-ahead log requires that exactly one writer wins each append, and conditional writes guarantee it. Combined with strong consistency (Section 5.5), they make object storage well suited to storing such logs. For an ordinary application the direct value is simpler: no upload by any client or user can ever accidentally destroy an existing object, which by its nature also protects against the key-collision mistake of Section 4.4.

6.5 S3-compatible is not a guarantee

Many object-storage providers describe themselves as *S3-compatible*, meaning they speak the same protocol as AWS S3. Cloudflare R2, for example, is S3-compatible: the AWS SDK works against it. But compatibility does not always include this critical feature. The lecturer's example is Backblaze, a well-known provider that is S3-compatible but, in his account, does not support conditional writes. Before relying on conditional writes, check the provider's documentation, whatever it claims about S3 compatibility.

7 UPLOADING THROUGH THE BACKEND

A file can travel from the user's laptop into the bucket along two architectures: through the backend, or directly from the browser (Section 8).

7.1 The proxy architecture

The obvious option: the browser sends the file to the API, and the backend receives it and uploads it to the bucket. The backend sees every byte, so it can inspect the content properly, scan it, compute a hash if needed, and reject it for any reason before it reaches object storage. Authorization is the ordinary middleware layer (Chapter 6) — nothing complicated — and no CORS configuration is needed. For small files — profile photos, CSV files, documents of a few megabytes — it can be appropriate, and it is sometimes used; for small files there is no objective reason to call it wrong.

7.2 The buffering trap

The hidden problem is buffering. Calling something like `ParseMultipartForm` in Go, or the standard body parser of a Node.js web framework, reads the whole file into the server's RAM. A 100 MB upload costs 100 MB of RAM; 20 users uploading 100 MB files at the same moment exhaust a 2 GB memory limit instantly. That is the problem from the start of the chapter: the container hits its memory limit and, depending on how much memory it was given, is killed with an out-of-memory (OOM) error — a very familiar error for containers — and must restart. The danger of the pattern is that it is invisible during local development: tests use one or a few small files, the connection is fast (a 100 Mbit/s network), and the development machine is powerful, with 16 GB of RAM or more, so nothing causes trouble.

> **A subtle point.** Go's `ParseMultipartForm(maxMemory)` keeps at most `maxMemory` bytes of file parts in RAM (32 MB when triggered by `FormFile`) and writes the rest to temporary files. The whole body is still buffered before the handler runs — partly on the container's ephemeral disk, which has its own limit — so the conclusion is unchanged: stream.

7.3 Streaming and a body-size cap

The correct implementation of this pattern streams: the file is never held in memory as a whole. The handler reads the request body in parts and writes each part to the storage client — the Amazon S3 SDK or whatever is in use — as soon as it arrives, so memory holds only one small buffer at a time, a few kilobytes, whether the file is 1 MB or 1 GB. In Go this means using the multipart reader directly: take the file part as an `io.Reader` and pass that reader to the uploader, instead of parsing the whole form (Figure 6).

```
const maxBody = 10 << 20 // 10 MB

func upload(w http.ResponseWriter, r *http.Request) {
    r.Body = http.MaxBytesReader(w, r.Body, maxBody)
    mr, err := r.MultipartReader() // not ParseMultipartForm
    if err != nil {
        http.Error(w, "bad form", http.StatusBadRequest)
        return
    }
    part, err := mr.NextPart() // the file field, a stream
    if err != nil { /* 400 */ return }
    in := &s3.PutObjectInput{
        Bucket: aws.String("avatars"),
        Key:    aws.String(newKey(r)), // ours, not filename
        Body:   part,                  // read chunk by chunk
    }
    // uploader := manager.NewUploader(client): part-sized
    // buffers, so memory does not grow with the file
    _, err = uploader.Upload(r.Context(), in)
    if err != nil { /* 502 */ return }
    w.WriteHeader(http.StatusCreated)
}
```

Figure 6: Streaming a multipart upload to S3 in Go with a body-size cap (illustrative; AWS SDK for Go v2)

Cap the body before reading it. Almost every language offers a safety mechanism that limits the request body size before anything is read from it — `http.MaxBytesReader` in Go, the size-limit options of Node.js body parsers. Set it on every upload route, so that a client sending a 4 GB file is cut off at the limit instead of being allowed to try; it closes this whole class of failures.

The lecturer's demonstration runs two servers that each receive the same 500 MB file twice, while memory is watched in the `btop` process monitor. In the buffered version, which reads the entire file into memory before sending it to storage, the memory column rises as the request body arrives and ends about 500 MB higher than at the start. The streaming version receives the same file from the same client in the same time — 6.5 seconds — but holds only one chunk at a time: its memory line stays almost flat. That is the whole difference between holding a file in memory and passing it through.

7.4 Limits that streaming cannot remove

Even with streaming, the architecture has limits that cannot be bypassed, because the bottleneck is now network bandwidth. A load balancer has an idle timeout, typically 60 seconds by default; an upload over a slow mobile connection will most likely hit it unless the timeout was raised. A reverse proxy such as Nginx or Caddy has a request-body size limit — Nginx is notorious for its 413 error, "request entity too large", produced in exactly this scenario. Serverless functions and API gateways impose hard payload limits of a few megabytes that cannot be changed at all. The proxy architecture therefore works in some scenarios — with luck, for small files or traditional tasks — but it is not recommended and is not a production-grade architecture.

8 PRESIGNED URLS

8.1 Direct upload to a private bucket

The correct architecture for all the problems above is to let the browser talk to the bucket directly, instead of the server acting as an intermediary with delegated authority: the data never passes through or touches the server. The difficulty is that in most production systems the bucket is private, as it should be — opening a bucket to the public is a serious security risk — and the server's AWS credentials can obviously not be handed to the browser. The browser needs permission to perform exactly one operation on one key, for the next five minutes, without receiving any credentials. The *presigned URL* answers this very elegantly.

8.2 Anatomy of a presigned URL

A presigned URL is an ordinary URL with a set of query parameters (Figure 7): the signing algorithm, the access-key identifier, a timestamp, the validity period in seconds, the list of signed headers, and the signature — an HMAC computed with the secret key over a string that describes the request: HTTP method, bucket, key, expiry and almost everything else. The browser sends its PUT to this URL. The storage service — Amazon S3 here — rebuilds the same string from the incoming request, computes the HMAC with its own copy of the secret, and compares the two. If they match, the request is authorized. If the browser changes even one character of the key or the method, or uses the URL after it has expired, the signatures no longer match and the request is rejected. The credentials never leave the server, yet the browser can interact with the bucket without any security risk. (In AWS Signature Version 4 the HMAC key is itself derived from the secret, the date, the region and the service, so the secret never appears in the URL.)

```
PUT https://avatars.s3.eu-west-1.amazonaws.com/t_42/u_7/
    f47ac10b.png
  ?X-Amz-Algorithm=AWS4-HMAC-SHA256
  &X-Amz-Credential=AKIAIOSFODNN7EXAMPLE%2F20261101%2F
      eu-west-1%2Fs3%2Faws4_request
  &X-Amz-Date=20261101T101500Z
  &X-Amz-Expires=60
  &X-Amz-SignedHeaders=host
  &X-Amz-Signature=5d3b...a91c     (64 hex digits)

signature = HMAC-SHA256(key derived from the secret,
    string describing: method, bucket, key, date,
    expiry, signed headers, ...)
```

Figure 7: Anatomy of a presigned PUT URL, Signature Version 4 (illustrative; line-wrapped)

8.3 Demo: expiry and tampering

In the demonstration, a front-end page first asks the server for a signed URL. The response contains the algorithm, the credential, the date, a validity of 60 seconds, the signed headers and, at the end, the signature. The upload then appears in the browser's network panel as a PUT sent directly to the bucket's domain, not to the server, carrying no credentials; the status is 200, a successful save. A button simulates waiting 60 seconds and resends exactly the same re-

quest: 403 Forbidden, because the URL has expired. Another button takes a URL that is still valid but changes one character of the key: the signature no longer matches, and the bucket rejects the request with 403.

8.4 The missing size limit

The architecture still hides a flaw. The server creates a presigned URL for a user who says they want to upload a profile photo. What stops that user from uploading a 5 GB file to the URL? Nothing: the signature covers the method, bucket and key, but nothing limits the request body size. In round one of the lecturer's demo, the page requests a presigned URL for a profile photo and, instead of an image of a few kilobytes, uploads a 300 MB file. The network panel shows the PUT going straight to the S3 bucket; S3 accepts and stores the file and returns 200. The bucket now holds 314 million bytes uploaded through a URL signed for a profile photo, which is clearly wrong.

8.5 Presigned POST with a policy

The solution is a *presigned POST* with a *policy*. Instead of signing a URL, the server signs a small JSON policy document and gives the client a URL together with a set of form fields. The policy contains conditions. The important one is `content-length-range`, a hard minimum and maximum size that the object-storage service itself enforces; the server does not need to intervene, because the responsibility is delegated to the store. The policy can also require that the content type start with something like `image/`, and that the key start with a given prefix, so that the client cannot write outside its own "folder" and bypass the security restrictions. If the client violates any condition, the object store rejects the whole request — with an "entity too large" error for the size — and the data is never stored.

The lecturer's server sets three conditions (Figure 8): a size between 1 KB and 5 MB, a content type starting with `image/`, and a key starting with the tenant and user prefix, so that the client cannot leave its boundaries. On the client (Figure 9), two details matter: the file field must be added *last* to the form data, and the `Content-Type` header must not be set in the `fetch` request, so that the browser sets the multipart boundary itself.

```
import { createPresignedPost }
  from "@aws-sdk/s3-presigned-post";

const prefix = `${tenantId}/${userId}/`;  // owner in the key
const { url, fields } = await createPresignedPost(s3, {
  Bucket: "avatars",
  Key: prefix + crypto.randomUUID(),
  Conditions: [
    ["content-length-range", 1024, 5 * 1024 * 1024],
    ["starts-with", "$Content-Type", "image/"],
    ["starts-with", "$key", prefix],
  ],
  Fields: { "Content-Type": contentType },
  Expires: 300, // seconds
});
// the SDK builds the JSON policy, signs it, and returns
// url + fields: sent to the browser with the upload id
```

Figure 8: Server side: a presigned POST whose policy limits size, content type and key prefix (illustrative; AWS SDK for JavaScript v3)

```
const form = new FormData();
for (const [k, v] of Object.entries(fields)) {
  form.append(k, v);          // key, policy, signature ...
}
form.append("file", file);    // the file field goes LAST
await fetch(url, { method: "POST", body: form });
// no Content-Type header: the browser must set the
// multipart boundary itself
```

Figure 9: Client side: posting the policy fields and the file directly to the bucket (illustrative)

Round two of the demo repeats the attack in a second tab. The same 300 MB file, uploaded this time with a policy-based URL, gets 400 "entity too large", and nothing is stored in the bucket. An image of about 415 KB under the same policy passes and is stored, because it satisfies the size condition. A switch that tries to upload into another user's prefix gets 403: S3 rejects the request and states the exact reason. Table 4 compares the upload architectures.

Architecture	Bytes through server	Size enforced by	Verdict
Proxy, buffered	yes, whole file in RAM	server, if capped	OOM risk
Proxy, streaming	yes, chunk by chunk	server body cap	small files; proxy limits
Presigned PUT	no	nothing	unbounded size
Presigned POST + policy	no	the object store	production choice

Table 4: Upload architectures compared

9 KEEPING THE DATABASE AND THE BUCKET IN SYNC

9.1 The two-phase upload

One question from the start of the chapter remains. If the browser uploads straight to storage, the server never sees the upload happen — so how does the database learn about it? Later, when a GET request asks for the profile photo or video, the backend must find the right row and column, read the object's key, fetch the object and return it; without the exact key it cannot. The solution is a two-phase process (Figure 10).

In the first phase, the client asks the backend for permission to upload. The backend authorizes and validates the request and generates the object's key in advance — the name that will become the object's identity. It writes a row to the database with status `pending`, storing the key together with the expected conditions and limits, and responds with a presigned POST. So before any data is transmitted, the database already holds a record of the client's intent, with the object's identifier. In the second phase the browser uploads. When the upload finishes, it calls a completion endpoint on the backend with the *upload ID* obtained in the first phase — not the key. The server then sends a `HEAD` request (HeadObject) to the object store. HEAD is a metadata-only request that transfers no data; it checks that the object really exists, that its size falls within the expected range and that its content type is correct.

When all conditions hold, the row's status changes from `pending` to, say, `completed` or `ready`.

```
1  browser -> API   POST /uploads {kind, size, type}
2  API              authorize; key = t_42/u_7/<uuid>
3  API -> DB        INSERT row: key, status 'pending',
                    expected max size, content type
4  API -> browser   { uploadId, url, fields }
5  browser -> S3    POST url: fields first, file last
6  browser -> API   POST /uploads/{uploadId}/complete
7  API -> S3        HEAD key: exists? size? type?
8  API -> DB        UPDATE row: status 'ready'
9  cleanup job      'pending' after 24 h -> 'expired'
```

Figure 10: The two-phase upload (illustrative)

9.2 Abandoned uploads and reconciliation

The remaining gap needs a cleanup workflow. Some percentage of uploads will be abandoned: connection errors, a browser going offline, a closed tab — anything can happen on the client side, which the backend does not control, and on a slow or mobile network there are hundreds of reasons why a file never finishes uploading, or finishes but the completion endpoint is never called. The usual remedy has two parts: a *lifecycle rule* on the bucket that deletes everything under the `pending` prefix after 24 hours, and a small background job (Chapter 10) that scans pending records older than 24 hours and marks them expired. The job matters because it prevents the database and object storage from drifting apart: they are two different sources of truth. The database is the source of truth for which files exist; the bucket is the source of truth for the bytes of the files themselves. Whenever the two disagree, a background job has to reconcile them.

> **A subtle point.** A lifecycle rule sees keys, tags and ages, not database statuses. Expiring "everything under `pending/`" is safe only if uploads land under a `pending/` prefix and the completion step moves each verified object to its final key (a server-side copy plus delete, Section 4.3), or if the rule filters on an object tag that completion changes. If objects are written to their final keys from the start, a prefix rule would delete completed uploads as well, and the background job must delete the objects of expired rows itself.

9.3 CORS for direct uploads

Whenever a browser is involved, CORS issues follow. A browser uploading directly to S3, Cloudflare R2, Backblaze B2 or any other object store makes a cross-origin request, which triggers CORS. For the request to succeed, the bucket needs a CORS configuration (Figure 11) that explicitly allows the frontend's origin — the frontend's domain — allows the PUT or POST method, and exposes the `ETag` header, which multipart uploads will need (see Chapter 21). If an upload works with curl in a terminal or in Postman but fails in the browser, the cause is almost certainly CORS — those tools do not enforce it, browsers do — and the bucket's CORS policy is the first thing to check.

```
[
  {
    "AllowedOrigins": ["https://app.example.com"],
    "AllowedMethods": ["PUT", "POST"],
    "AllowedHeaders": ["*"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3000
  }
]
```

Figure 11: Bucket CORS configuration for direct browser uploads (illustrative)

10 CONCLUSION

- Files on the server's disk fail for six independent reasons: ephemerality, horizontal scaling, fixed disk size, single-copy durability, slow delivery, and no transaction with the database.
- Block, file and object storage trade interface richness for scale; object storage keeps only PUT, GET, DELETE and LIST and gives up in-place modification, rename and locking.
- Folders are a listing trick over flat keys: moving a "folder" copies and deletes every object under it.
- Generate keys yourself, put entropy first, and encode the owner so that authorization is a string check.
- Erasure coding (14 + 6 fragments, 1.43× storage) delivers durability; eleven nines still do not replace versioning and cross-account replication.
- The metadata index makes LIST a range scan: the database, not the bucket listing, is the index of files.
- Conditional writes (`If-None-Match`, `If-Match`) turn S3 into a coordination primitive, but not every S3-compatible store has them.
- Proxy uploads must stream and cap the body; production uploads use presigned POST policies that let the store enforce size, type and key prefix.
- A two-phase upload with HEAD verification, lifecycle rules and a cleanup job keeps database and bucket in agreement.

APPENDIX A REVIEW QUESTIONS

1. A photo-sharing API runs four replicas on Kubernetes behind a round-robin load balancer; uploads are saved under `/var/data/uploads` inside the container and the path is stored in Postgres. After Friday's deployment, every photo uploaded before Friday is missing, and about three in four requests for photos uploaded since then return 404. Explain both symptoms quantitatively, and state which of them a per-pod persistent volume would fix and which it would leave in place.
2. A video service proxies uploads through its Node.js backend to S3. Pods have a 1 GB memory limit, the upload route uses an in-memory multipart parser, files are up to 50 MB, and at peak 30 users upload at once; pods are OOM-killed although every local test passed. Explain why local testing missed the failure, redesign the route, and name two infrastructure limits that would

still break uploads from users on a 1 Mbit/s uplink after your fix.

3. A chat application stores attachments under `uploads/2026-11-01T14/<user-id>/<original-filename>` and peaks at 8,000 uploads per second. Identify every weakness of this key scheme, predict the errors and data loss you would observe under load, and propose a replacement scheme; explain how your scheme also changes the cost of an authorization check in middleware.
4. A customer with 40,000 documents under `acme/contracts/` asks for a "rename folder" button, and the current UI builds each page's sidebar by calling ListObjectsV2 with a delimiter. Describe exactly what work each feature causes inside the object store and why; then redesign both so that page loads involve no listing and a rename does not grow with the number of documents.
5. A storage team must hold 2 PB of user data and compares three-way replication with a Reed–Solomon layout of 10 data plus 4 parity fragments. Compute the raw capacity each needs and the number of simultaneous disk failures each survives, explain what the erasure-coded layout costs instead, and then answer a product owner who argues that with eleven nines of durability the company can stop keeping backups.
6. Two workers try to become leader by running `put-object --key jobs/leader --if-none-match "*"` at the same moment. Later the leader renews its lease by reading the object and writing it back with `--if-match` and the ETag it read, while a paused former leader wakes up and does the same with the ETag it read an hour ago. Predict the status code of each request, explain why this works as a lock, and say what changes if the bucket moves to an S3-compatible provider without conditional-write support.
7. An avatar endpoint returns a presigned PUT URL valid for ten minutes for the key `t_9/u_31/a1b2.png`. One user uploads a 3 GB video with it, another edits the URL to write to `t_9/u_32/a1b2.png`, and a third replays it after eleven minutes. Predict the response to each request and what ends up in the bucket, then redesign the endpoint so that size, content type and key are all constrained, stating which component enforces each condition.
8. A month after launching direct uploads with a two-phase workflow, the `uploads` table holds 120,000 rows still marked `pending` and older than a day, while the bucket stores 900 GB more than the sum of the sizes of `ready` rows. List plausible client- and server-side causes of each discrepancy, and design the reconciliation — background job, lifecycle rules and HEAD checks — including how your key layout interacts with the lifecycle rule.

APPENDIX B GLOSSARY

Term	Meaning
Object storage	Store of immutable objects under keys, accessed over HTTP with PUT, GET, DELETE, LIST
Object	Key, value (bytes), system metadata and user metadata
Bucket	Named container of objects; name unique across the platform
Common prefix	Key prefix up to a delimiter, returned by LIST to simulate a folder
Ephemeral storage	Storage that disappears when its container restarts
Block storage	Numbered fixed-size blocks; an SSD, Amazon EBS
POSIX semantics	Standard file operations: seek, overwrite, atomic rename, locks, append
Durability	Whether data still exists anywhere; availability is whether it is reachable now
Erasure coding	*k* data plus *m* parity fragments; any *k* rebuild the object
Hot partition	Index partition overloaded by writes that share a key prefix
Strong consistency	A read after a successful write always sees that write
ETag	Fingerprint of an object's current content version
Conditional write	PUT with `If-None-Match` or `If-Match`; 412 if the condition fails
Presigned URL	URL with a time-limited HMAC signature for one operation on one key
POST policy	Signed JSON conditions (size range, type, key prefix) enforced by the store
Lifecycle rule	Bucket rule that expires or moves objects by prefix, tag or age

APPENDIX C FURTHER TOPICS

Event notifications

Object stores can emit an event whenever an object is created or deleted; S3 Event Notifications deliver to SQS, SNS, Lambda or EventBridge. A backend can mark an upload ready from the event instead of relying only on the browser's completion call, which closes the gap left by clients that upload and disappear. Delivery is at least once and not strictly ordered, so handlers must be idempotent. It applies whenever direct uploads feed further processing.

End-to-end checksums

An upload can carry a checksum of its content — `Content-MD5` or one of S3's additional checksum algorithms such as CRC32C or SHA-256 — and the store rejects the object if the received bytes do not match. This catches corruption in transit or in client code, which durability guarantees do not cover, since they protect bytes only after they arrive. Storing the checksum in the database also enables deduplication and later integrity audits. It applies to documents, backups and anything whose exact bytes matter.

Serving user content safely

Files uploaded by one user are untrusted when served to others. An HTML or SVG file served from the application's own origin can run script there (stored XSS), so user con-

tent is usually served from a separate domain with an explicit `Content-Type`, `X-Content-Type-Options: nosniff`, and `Content-Disposition: attachment` where the file should be downloaded rather than rendered. The `Content-Disposition` filename parameter is also where the original filename kept in the database is returned to the user. It applies to every system that lets users download what others uploaded.

Asynchronous processing of uploads

With direct uploads the server never sees the bytes, so checks that need the content — malware scanning, verifying that an "image" really decodes as one, re-encoding images to strip metadata such as EXIF GPS coordinates, thumbnails, video transcoding — move to a background job after the upload (Chapter 10). The upload row then passes through further states such as processing, ready or rejected. Re-encoding also neutralises files crafted to exploit image parsers. It applies whenever uploaded content is shown to other users or processed further.

Encryption at rest and key management

Since January 2023 S3 encrypts every new object at rest by default with S3-managed keys (SSE-S3). SSE-KMS uses keys held in AWS Key Management Service, so reading the data also requires permission on the key and every use is logged; client-side encryption keeps keys away from the provider entirely. The choice matters for compliance regimes and for limiting the damage of leaked storage credentials. It applies to personal data, documents and anything regulated.

Object Lock and write-once retention

S3 Object Lock makes object versions undeletable for a retention period (governance or compliance mode) or indefinitely under a legal hold, and it requires versioning. In compliance mode not even an administrator with full permissions can delete a locked version, which protects backups against ransomware and compromised credentials better than versioning alone. It is also the standard way to meet write-once-read-many (WORM) retention rules. It applies to audit logs, financial records and backup copies.

Bucket policies and Block Public Access

Access to buckets is governed by IAM policies on identities and resource policies on the bucket, which can restrict actions to key prefixes — the mechanism behind owner-based keys. S3 Block Public Access overrides any policy or ACL that would make data public and has been enabled by default for new buckets since April 2023. Misconfigured public buckets are a recurring cause of data leaks, so least-privilege policies per service and per prefix matter. It applies to every bucket from the day it is created.

Local S3-compatible storage for development and tests

MinIO and LocalStack run an S3-compatible API locally or in a container, so presigned flows, cleanup jobs and key layouts can be exercised in integration tests (Chapter 23) without cloud costs or credentials. Compatibility is not complete — conditional writes, POST policies, CORS and error codes may behave differently — so critical behaviour should also be verified against the production provider. It applies to any backend whose tests touch object storage.

Object Storage, Part 2 — One-Page Overview

Chapter 21 · every section and sub-section of the paper at a glance

1 Introduction: Why a Single PUT Stops Working

One request carrying a whole large file fails for four independent reasons.

- **1.1 The hard size limit** — The object-store API caps a single PUT at 5 GB.
- **1.2 All-or-nothing transfers** — A drop at 4.8 of 5 GB restarts the upload from byte zero.
- **1.3 The throughput of one TCP connection** — About window / RTT; one loss collapses the window; parallel streams each get their own.
- **1.4 Progress, pause and resume** — One request: coarse progress, no pause, no resume.

2 The Multipart Upload Protocol

Open a staging area, send independent parts, commit with a metadata operation.

- **2.1 Create: an upload ID and a staging area** — The key returns 404 until the upload is completed.
- **2.2 Upload part: independent requests** — Parallel, any order, retried singly; keep every part's ETag.
- **2.3 Complete: a metadata operation** — Part numbers and ETags in order; same speed for 50 GB.

3 Part Size and Integrity

Size parts from the file size, keep them cheap to retry, and do not trust the multipart ETag.

- **3.1 The 10,000-part ceiling** — 5 MB parts cap the file at 50 GB; part 10,001 fails.
- **3.2 Choosing the part size dynamically** — Part size = max(default 16–32 MB, size / 10,000).
- **3.3 The part is the retry unit** — A failed part is resent whole; 16–64 MB fits almost every case.
- **3.4 The ETag is not a checksum** — Multipart ETag = MD5 of part MD5s + `-N`; use checksums.

4 Abandoned Uploads and the Lifecycle Rule

Incomplete uploads are invisible yet billed until a lifecycle rule aborts them.

- **4.1 The invisible leak** — Demo: 184 MB visible in the console, 460 MB of hidden parts.
- **4.2 The abort-incomplete lifecycle rule** — Abort after 7 days, on every bucket, from day one.

5 Browser Uploads: Multipart with Presigned URLs

The backend plans and verifies; the browser sends the parts straight to the bucket.

- **5.1 Initiation** — Auth, part size, create upload, pending row, presigned part URLs.
- **5.2 Parallel transfer and completion** — 4–6 parallel PUTs; complete, HEAD check, `file.uploaded` event.
- **5.3 Four practical rules** — Presign in batches, cap concurrency, lazy `slice()`, expose ETag in CORS.
- **5.4 Resuming after a refresh** — `ListParts` finds stored parts; only missing ones are re-signed.
- **5.5 The demo: a 2 GB upload** — 16 MB parts, 125 PUTs to the bucket, five at a time.

6 Serving Downloads

Bytes leave from the bucket or a CDN, never through the backend.

- **6.1 Never relay downloads through the backend** — Relays hold workers, pay traffic and lose to a CDN.
- **6.2 Pattern 1: public content behind a CDN** — Private bucket, CDN-only read, long cache, content-addressed keys.
- **6.3 Pattern 2: presigned GET** — Unique URLs defeat the CDN cache; align expiry to the hour.
- **6.4 Pattern 3: CDN-signed URLs and signed cookies** — Checks at the edge; cookies suit video segments.

7 Range Requests

HTTP lets a client fetch any byte range of an object.

- **7.1 The HTTP mechanism** — `Range`; `206` with `Content-Range`; `416`.
- **7.2 Seeking, resuming and parallel downloads** — Jump to 30:00, resume at byte 400,000,000, six ranges.
- **7.3 The demo** — MinIO: 206 for 100 bytes, PNG signature from 8 bytes, resumed curl.

8 Segmented Streaming and Adaptive Bitrate

Video quality adapts per segment while storage sees only objects.

- **8.1 The limit of a single file** — One file cannot suit every connection.
- **8.2 HLS, DASH and adaptive bitrate** — Transcode, segment, manifest; the player picks each segment's quality.
- **8.3 What storage sees, and build versus buy** — Buy (Mux, Bunny) unless video is the product.

9 Pricing and the Complete Rule Set

Storage, operations and egress drive the cost; both parts' rules in one table.

- **9.1 Three things you pay for** — GB-month, requests (writes about 10× reads), egress; R2 has free egress.
- **9.2 The rules of both parts** — From key design to range reads and video.

Object storage, part 2, at a glance — Large files travel as independently retried parts that a metadata-only call assembles, sent straight from the browser through presigned URLs; files leave the bucket through a CDN or signed URLs, never through the backend, and range requests and segments make seeking, resuming and adaptive video possible. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Object Storage, Part 2: Large Files, Downloads and Streaming

Chapter 21 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Part 2 of object storage starts where a single PUT request fails for large files: the 5 GB limit, all-or-nothing transfers, the throughput of one TCP connection and the lack of progress or resume. It presents multipart upload (create, upload part, complete), shows how to size parts against the 10,000-part limit, why the multipart ETag is not a checksum, and why every bucket needs a rule that aborts incomplete uploads. Multipart upload is then combined with presigned URLs into a resumable browser upload architecture. For downloads, relaying through the backend is ruled out and three delivery patterns are compared: public content behind a CDN, presigned GET URLs, and CDN-signed URLs or cookies. Finally the paper covers HTTP range requests, segmented adaptive-bitrate streaming and the three components of object-storage pricing.

Keywords — multipart upload, part size, ETag, lifecycle rule, presigned URL, CDN, range request, adaptive bitrate, egress

1 INTRODUCTION: WHY A SINGLE PUT STOPS WORKING

Chapter 20 built the small-file path: objects and keys, the metadata and data layers, conditional writes with ETags, uploads through the backend, presigned URLs and the database record that keeps bucket and database in sync. That path holds for files of a few hundred megabytes. Beyond that size, one PUT request carrying the whole file breaks down for four independent reasons.

1.1 The hard size limit

A single PUT request may carry at most 5 GB. The limit is imposed neither by an HTTP specification nor by the backend but by the object-store API itself. Products that accept large videos, CAD files, database dumps, long wedding recordings or aerial photography routinely receive files above it, so for them a different upload mechanism is an architectural requirement, not an optimization.

1.2 All-or-nothing transfers

A single PUT either completes or fails as a whole. Suppose 4.8 GB of a 5 GB file have been sent when the Wi-Fi disappears for a moment, for example while the connection switches from the primary to the backup line. The request is cancelled; there is no resume, and the next attempt starts again from byte zero. On a typical consumer connection, mostly mobile data rather than 100–200 Mbit/s professional Wi-Fi, such a transfer takes 20–30 minutes, and a single short outage anywhere in that window throws all of it away. Short outages are common, especially on mobile networks, so the larger the file, the more likely the upload never finishes.

1.3 The throughput of one TCP connection

HTTP runs over TCP, and a single TCP connection cannot fill a fast link. Its throughput is bounded roughly by the window size divided by the round-trip time (RTT): with an effective window of 256 KiB and an RTT of 70 ms, one connection moves about 30 Mbit/s however fast the link is (illustrative numbers). The connection is also extremely sensitive to loss: one lost packet collapses the congestion window, which must then grow back gradually. This is why a user with a 200 or 500 Mbit/s connection still sees a single-stream transfer stuck around 30 Mbit/s.

The fix lies at the network level: more connections. Parallel streams each get their own congestion window, so a stream that slows down after a loss does not slow the others. Download managers such as Internet Download Manager or Free Download Manager rely on exactly this. They do not change the physical network speed; they open eight connections for the same file and download its pieces in parallel.

1.4 Progress, pause and resume

The fourth reason is user experience. With one request the client can display only coarse progress, and there is no way to pause the transfer or to resume it after a network error, a closed tab or a page refresh.

Problem	Single PUT	Multipart upload
Size	at most 5 GB	up to 10,000 parts
Failure	restart from byte zero	retry the failed part
Throughput	one TCP window	parallel parts, own windows
Experience	coarse progress, no pause	per-part progress, resume

Table 1: Why a single PUT fails for large files, and how multipart upload answers each problem.

2 THE MULTIPART UPLOAD PROTOCOL

Multipart upload answers all four problems with a protocol of three calls.

> **Definition.** A **multipart upload** sends one object as independently numbered parts into a staging area identified by an upload ID; a final call turns the parts into a single object at the key, and until then nothing exists at that key.

2.1 Create: an upload ID and a staging area

The client first calls *create multipart upload* (in S3, `CreateMultipartUpload`) with the bucket, the key and the con-

tent type, and receives an **upload ID**. The upload ID names a temporary staging area. The analogy is Git's staging area: changes prepared but not yet committed, which can still be discarded. The parts uploaded later accumulate there. Nothing exists behind the key yet: a GET for the key returns `404 Not Found`, because the object is created only at the very end, when the upload is committed.

2.2 Upload part: independent requests

Each fragment is sent with *upload part*: the upload ID, a part number and the bytes. Every such call is a completely independent HTTP request, with three consequences. Parts can be sent in parallel. They can arrive in any order (with seven parts, part 7 may land before part 2), because the part number, not the arrival order, fixes the position of the bytes in the final object. And a failed part is retried on its own, without affecting the other parts or requests. Every successful part returns an ETag; this is the second use of ETags, after the conditional writes of Chapter 20. The client must store each pair of part number and ETag, because the final call needs all of them.

2.3 Complete: a metadata operation

The client finishes with *complete multipart upload*, sending the upload ID and the complete list of part numbers with their ETags, in ascending order; the store rejects the call if a listed ETag does not match the stored part. The store then combines the parts into one object. Remarkably, this call takes the same short time whatever the number of parts and whatever the final size: even for a 50 GB object it is fast, because the store does not physically concatenate 50 GB of data. Completion is a metadata operation that records which parts, in which order, make up the object. It is a direct consequence of the separation between the metadata layer and the data layer (see Chapter 20). A fourth call, *abort multipart upload*, discards the staging area and its parts (Section 4).

```
# 1  Create: open a staging area; nothing exists at the key
POST /media/u42/7f3a.mp4?uploads
  -> 200  UploadId = "VXBsb2FkSUQ"
GET  /media/u42/7f3a.mp4              -> 404 Not Found

# 2  Upload parts: independent requests, any order
PUT  /media/u42/7f3a.mp4?partNumber=7&uploadId=VXBs...
  -> 200  ETag: "b54357fa..."         (keep it)
PUT  /media/u42/7f3a.mp4?partNumber=2&uploadId=VXBs...
  -> 200  ETag: "9c3e01d2..."         (keep it)

# 3  Complete: every part number + ETag, in order
POST /media/u42/7f3a.mp4?uploadId=VXBs...
  [ {1,"a1f0..."}, {2,"9c3e..."}, ..., {7,"b543..."} ]
  -> 200  object now visible at the key
```

Figure 1: The three calls of a multipart upload in S3's REST form (illustrative).

3 PART SIZE AND INTEGRITY

3.1 The 10,000-part ceiling

One multipart upload may consist of at most 10,000 parts, so the part size fixes the largest file the upload can carry. With 5 MB parts (the smallest size S3 accepts for every part except the last) the ceiling is 5 MB × 10,000 = 50 GB. If a 60 GB file is uploaded with 5 MB parts, nothing goes wrong until part 10,000 has been sent; part 10,001 is rejected and the upload fails after 50 GB have already been transferred, a very expensive way to discover the limit.

3.2 Choosing the part size dynamically

The correct approach is to derive the part size from the file size instead of fixing it. Dividing the file size by 10,000 gives the smallest part size that keeps the upload within the limit. That value alone is wrong for small files, however: there is no sense in cutting a 200 MB file into 10,000 parts of 20 KB. The rule therefore takes two values, a default chosen for the application's typical files and the computed minimum, and uses whichever is larger. The lecturer uses a floor of 16 or 32 MB, which covers files up to 160 or 320 GB before the computed value ever takes over, so most products never need to think further about the formula.

```
const long MiB      = 1024 * 1024;
const long MaxParts = 10_000;
const long Floor    = 16 * MiB;   // default for typical files

static long PartSize(long fileSize)
{
    // smallest part size that still fits in 10,000 parts
    long needed = (fileSize + MaxParts - 1) / MaxParts;
    long size   = Math.Max(Floor, needed);
    return (size + MiB - 1) / MiB * MiB;   // whole MiB
}

// file        part size   parts
// 200 MiB     16 MiB         13
// 60 GiB      16 MiB      3,840
// 400 GiB     41 MiB      9,991
```

Figure 2: Dynamic part size with a 16 MiB floor (illustrative C#).

A subtle point. The lecture phrases the rule as "divide the file size by 10,000 to get the number of pieces" and "take the larger number of parts". Both quantities are part *sizes*: size / 10,000, rounded up, is the smallest part size that fits, and the rule is part size = max(default, size / 10,000). The larger part size gives the smaller part count. On S3 a part must also be at least 5 MB (except the last) and at most 5 GB.

3.3 The part is the retry unit

Very large parts would avoid the ceiling altogether, but the part is the unit of retry: when a part fails, all of its bytes are sent again in a new request. On a mobile connection a failed 500 MB part costs another 500 MB, whereas a failed 16–32 MB part costs almost nothing, so retries stop being a concern. Parts of 16 to 64 MB solve almost every case.

3.4 The ETag is not a checksum

For a regular PUT, the ETag of an object is the MD5 hash of its content. Many developers therefore build an integrity check that looks perfectly logical: hash the file locally, upload it, and compare the hash with the returned ETag; if they match, the file arrived intact. This works for ordinary uploads. For a multipart upload the ETag is no longer the MD5 of what was uploaded: it is a hash computed over the MD5 hashes of the individual parts, followed by a hyphen and the number of parts, so a multipart ETag ends in

something like `-42`. A comparison with a local MD5 of the whole file always fails. (Even for a single PUT the ETag is not an MD5 when the object is encrypted with SSE-KMS or a customer-provided key.)

True end-to-end integrity comes from a separate feature: checksums. Modern SDKs, such as the AWS S3 SDKs, compute a CRC32C or SHA-256 checksum for each part, and the full checksum of the object can be requested for verification by the service. The lecturer's own approach is simpler: compute a SHA-256 of the file on the client, send it as user metadata, and let a background job verify the stored object against it after the upload. The same hash gives deduplication for free: two uploads with the same SHA-256 are the same content, so the second can point at the object that already exists.

```
single PUT     ETag "5d41402abc4b2a76b9719d911017c592"
               = MD5(file bytes)        (unencrypted object)

multipart      ETag "e2fc714c4727ee9395f324cd2e7f331f-42"
               = MD5(concat(MD5(p1), ..., MD5(p42)))
                 + "-" + number of parts

local check    MD5(file) == ETag    single PUT: equal
                                    multipart:  never equal

own hash       x-amz-meta-sha256: 9f86d081884c7d65...
               set by the client, verified by a job
```

Figure 3: Single-PUT and multipart ETags, and a client-side SHA-256 in user metadata (illustrative).

4 ABANDONED UPLOADS AND THE LIFECYCLE RULE

4.1 The invisible leak

Consider a multipart upload interrupted midway: 300 parts of about 32 MB have been uploaded, roughly 10 GB waiting in the staging area, when the user closes the tab. Neither complete nor abort is ever called. The 10 GB stay stored and are billed every month, indefinitely. They are also invisible: listing the bucket does not return them, because the object they belong to does not exist yet, and the console does not show them either.

The lecturer demonstrated this on the bucket prepared for the video. The console shows one object of 184 MB, apparently all the data being billed. The AWS CLI command that lists multipart uploads started but never completed (`aws s3api list-multipart-uploads`) returns three uploads, initiated a few days earlier, whose parts already total about 460 MB. None of them appears in any object listing or in the console, yet all of it is paid for: the hidden data is more than twice the visible data.

4.2 The abort-incomplete lifecycle rule

The remedy is a bucket lifecycle rule that aborts every incomplete multipart upload a fixed number of days after it was initiated, instead of letting parts accumulate over days, weeks and months. Seven days is a good default: long enough for a user to come back and resume, short enough that abandoned parts never pile up. The rule costs nothing and closes the leak permanently. It belongs in the configuration of every production bucket, set before the first function or line of upload code is written. It is a safety net: when the backend knows an upload is abandoned, for example because the user pressed Cancel, it should still call abort at once.

```
$ aws s3api list-multipart-uploads --bucket demo-media
  3 uploads, initiated days ago, parts total ~460 MB
  (console and object listing: 1 object, 184 MB)

$ aws s3api put-bucket-lifecycle-configuration \
    --bucket demo-media \
    --lifecycle-configuration file://lifecycle.json

lifecycle.json:
{ "Rules": [ {
    "ID": "abort-incomplete-multipart-uploads",
    "Status": "Enabled",
    "Filter": { "Prefix": "" },
    "AbortIncompleteMultipartUpload":
        { "DaysAfterInitiation": 7 }
} ] }
```

Figure 4: Finding hidden uploads, and the rule that aborts them after seven days (illustrative; the counts are those of the lecture's demo).

> **Warning.** Parts of an incomplete multipart upload are billed like stored data but appear neither in object listings nor in the console's object count; only `list-multipart-uploads` or storage analytics reveal them. Without an abort rule, every tab closed during a large upload becomes a permanent monthly cost.

5 BROWSER UPLOADS: MULTIPART WITH PRESIGNED URLS

Production systems that accept large files from web browsers combine multipart upload with the presigned URLs of Chapter 20: the backend keeps control over who uploads what, while the bytes travel directly from the browser to the bucket.

5.1 Initiation

The browser calls the backend, for example `POST /uploads/init`, with the file name, size and type. The backend authenticates and authorizes the request (see Chapter 4), computes the part size as the larger of the default and the dynamic value (Section 3.2), and calls create multipart upload. It records the upload in the database with the status *pending*, the intent-first record of Chapter 20, and presigns one URL per part, each bound to the upload ID and its part number. The response carries the upload ID, the part size and the part URLs.

5.2 Parallel transfer and completion

The browser now talks to the bucket directly. It runs n parallel PUT requests, one per part, with n set to the optimal number of simultaneous connections, typically four to six. When all parts have succeeded, it calls the backend again, for example `POST /uploads/complete`, with the upload ID and an array holding the number and ETag of every part. The backend calls complete multipart upload on the bucket and then, as in Chapter 20, sends a HEAD request for the new object to check that its content type, size and number

of parts match what the browser declared. The database row moves from *pending* to *completed*. In a queue architecture the backend also publishes an event such as `file.uploaded` (see Chapter 10); its consumers can run an audit or push a confirmation to the user interface over a WebSocket (see Chapter 22).

```
Browser -> API     POST /uploads/init {name, size, type}
  API              authenticate, authorize
  API              partSize = max(default, size / 10,000)
  API -> bucket    CreateMultipartUpload -> uploadId
  API -> DB        INSERT uploads(..., status='pending')
  API              presign UploadPart URLs (first batch)
API -> Browser     {uploadId, partSize, urls[]}

Browser -> bucket  PUT parts, 4-6 in flight -> ETags

Browser -> API     POST /uploads/complete
                   {uploadId, parts:[{partNumber, etag}]}
  API -> bucket    CompleteMultipartUpload
  API -> bucket    HEAD key: type, size, part count
  API -> DB        UPDATE uploads SET status='completed'
  API -> queue     publish file.uploaded
  queue            audit; WebSocket notice to the UI
```

Figure 5: Multipart upload from the browser with presigned part URLs (illustrative).

5.3 Four practical rules

- **Presign in batches.** A file that needs 10,000 parts must not be answered with 10,000 presigned URLs: the response to the initial request would grow to several megabytes of JSON. Sign about a hundred at a time and hand out the next batch as the browser finishes the previous ones.
- **Keep concurrency at four to six.** This is the middle ground; beyond it, the connections compete for the same bandwidth with each other and with other devices and processes on the network. The figure also matches the roughly six HTTP/1.1 connections a browser opens per host.
- **Read parts lazily.** The browser does not load the whole file. The JavaScript `File` object provides `slice()`, which returns a lazy `Blob`: its bytes come into memory only when the request sends them. Memory use is therefore about part size × concurrency (80 MB for 16 MB parts with five in flight), not the size of the file.
- **Expose the ETag in CORS.** The bucket's CORS configuration must expose the `ETag` response header, a point raised at the end of Chapter 20; otherwise the browser's JavaScript cannot read the ETag the bucket returns, and the upload cannot be completed.

```
// browser: upload part n (1-based), bytes read lazily
const start = (n - 1) * partSize;
const blob  = file.slice(start, start + partSize);
const res   = await fetch(urls[n - 1],
                          { method: "PUT", body: blob });
parts.push({ partNumber: n, etag: res.headers.get("ETag") });

// bucket CORS configuration (S3 JSON form)
[ { "AllowedOrigins": ["https://app.example.com"],
    "AllowedMethods": ["PUT"],
    "AllowedHeaders": ["*"],
    "ExposeHeaders":  ["ETag"] } ]
```

Figure 6: Reading one part with `slice()`, and the CORS rule that makes its ETag readable (illustrative).

> **Warning.** JavaScript may read only the CORS-safelisted response headers (`Cache-Control`, `Content-Language`, `Content-Length`, `Content-Type`, `Expires`, `Last-Modified`, `Pragma`) unless the response names others in `Access-Control-Expose-Headers`. `ETag` is not among them: without `ExposeHeaders` in the bucket's CORS rule, every part upload succeeds with status 200, yet `res.headers.get("ETag")` returns `null`.

5.4 Resuming after a refresh

Once all this is in place, large uploads become resumable. When the page is refreshed, the client asks the backend whether an upload is already in progress for this file (matched, for instance, by user, file name and size). The backend finds the incomplete row in the database and calls *list parts* (`ListParts`) with its upload ID to learn which parts already sit in the staging area. It returns fresh presigned URLs only for the missing parts, those that failed or were cut off by the refresh or the closed tab, and the client continues from that point, say from part 341, instead of starting from scratch. Because list parts also returns the ETags of the stored parts, the completion list can be rebuilt even though the refreshed page has lost the ETags it held in memory.

5.5 The demo: a 2 GB upload

The demo uses the upload page of Chapter 20, this time with a 2 GB file. Unlike the small-file case, the server signs no single URL. It starts a multipart upload and returns a plan: 16 MB parts, 125 parts in total, and five of them sent to the bucket at a time, the concurrency limit chosen on the browser side. The page draws a grid with one square per part: grey squares wait, blinking blue squares are in flight (five at any moment), and green squares are done. The progress bar is computed from the counts of completed, running and pending parts, so it advances smoothly instead of jumping. The browser's network panel shows about 125 separate PUT requests, all sent to the bucket's own domain and none to the backend. The speed and remaining-time estimates are not supplied by the server but computed in the browser from the completed and pending parts; since these numbers are stable, the display does not jump either. When the last part completes, the browser sends the completion request, with the list of parts and their ETags, to the API, not to S3.

6 SERVING DOWNLOADS

Browsers and applications retrieve files far more often than they upload them, so downloads, not uploads, account for most of the data an application moves.

6.1 Never relay downloads through the backend

The rule stated for uploads in Chapter 20 matters even more here: do not relay or proxy downloads through the application. A relaying handler calls get object on the store and copies the bytes into its response as they arrive. Each download then occupies one of the server's workers, whether a goroutine, an event-loop slot or a thread (see Chapter 19), for the whole transfer. Production also pays for bandwidth, not only for CPU cycles and memory: AWS and practically every cloud or platform-as-a-service provider charge for the traffic. On top of that, relayed downloads are slow compared with the right solution, a caching CDN, which takes the traffic off the server entirely and serves it from nodes in nearly every region and country, whereas the backend runs in one region or two or three. Every byte a user receives from the bucket should therefore come from the bucket directly or, where latency matters, from the CDN in front of it, never through the server. Three patterns achieve this.

6.2 Pattern 1: public content behind a CDN

When the content is genuinely public, such as images for websites and apps, CSS bundles or product photos, and neither security nor privacy is a concern, the bucket can be made public. Better still: keep the bucket private, put a CDN in front of it and allow only the CDN to read from the bucket. Add a very long cache lifetime (for example `Cache-Control: public, max-age=31536000, immutable`) and **content-addressable keys**, keys derived from a hash of the content whatever the file is, such as `products/9c3e01d2b4f7.webp`. When the content changes, the key and therefore the whole URL change, so no cache ever has to be invalidated (see Chapter 9). This simple, widely used pattern is the cheapest and fastest option available.

6.3 Pattern 2: presigned GET

For private files, the presigned GET mirrors the presigned PUT of Chapter 20, except that a GET request is signed. The browser asks the backend for a file; the backend checks that this user may access this specific object and, if so, returns a presigned GET URL valid for as long as the application needs, say 5 to 15 minutes.

The pattern has a catch. A presigned URL contains a timestamp and a signature, so every request for the same file receives a different URL. To a CDN each of them is a separate resource, as if it were a different object, and the cache hit ratio, the share of requests answered from the cache, drops to zero. The CDN was placed in front of the bucket solely to cache, yet it caches nothing, and the full outbound traffic (egress) is still paid. There are two remedies. The cheap one changes the expiry: instead of "valid for the next 15 minutes", the URL is signed as "valid until the start of the next hour", so every user who requests the file within that hour receives the same URL and the CDN can cache it. A little precision in the expiry is traded for a much higher hit ratio. The proper remedy, once the system reaches a certain scale, is pattern 3.

> **A subtle point.** A SigV4 presigned URL carries both its signing time (`X-Amz-Date`) and its lifetime (`X-Amz-Expires`), and the signature covers both. An hour-aligned expiry therefore yields identical URLs only if the backend also signs with the clock truncated to the start of the window. Removing the signature from the CDN's cache key instead would be worse: cached bytes would then be served to requests whose signature was never checked.

6.4 Pattern 3: CDN-signed URLs and signed cookies

At scale, the right tool is the CDN's own signed URLs or signed cookies. The backend still decides who may read what, but the check runs at the edge, the small servers distributed around the world at the network's nodes, and the CDN controls caching entirely without involving the backend. Signed cookies are especially convenient for video: one playback session downloads hundreds or thousands of segment files, and routing each of them through the backend would mean signing hundreds or thousands of URLs per viewing. One cookie whose policy covers the video's path hands that load to the CDN.

Pattern	Authorization	CDN caching	Use for
Public via CDN	none; only the CDN reads	full, long-lived	assets, product images
Presigned GET	backend, per request	none, unless hour-aligned	private files, modest scale
CDN-signed URL or cookie	backend issues, edge checks	full	private files at scale, video

Table 2: Three ways to serve files without the backend in the data path.

7 RANGE REQUESTS

> **Definition.** A **range request** is an HTTP GET with a `Range` header naming the bytes wanted; the server answers `206 Partial Content` with those bytes and a `Content-Range` header, or `416 Range Not Satisfiable` if the range is invalid.

7.1 The HTTP mechanism

A range request lets the client say: do not send the whole thing, send bytes 1,000,000 to 3,000,000. The server responds with status 206, partial content in HTTP semantics, and a `Content-Range` header that states exactly which bytes are in the body and the total size of the resource. If the range sent by the client is invalid or meaningless, the answer is 416. A server signals its support with `Accept-Ranges: bytes`. Byte positions are zero-based and inclusive, so `bytes=0-99` is the first 100 bytes.

All of this is HTTP semantics (see Chapter 1), not an invention of object storage. Chapter 20 named the HTTP interface as one of the reasons object storage is so convenient and powerful, HTTP being implemented on almost

every computing device, and object stores support range reads natively. Combining the two, clients that can ask for part of a resource and a store that can send part of an object, opens up the uses below.

```
GET /media/u42/talk.mp4 HTTP/1.1
Host: cdn.example.com
Range: bytes=1000000-2999999

HTTP/1.1 206 Partial Content
Accept-Ranges: bytes
Content-Range: bytes 1000000-2999999/734003200
Content-Length: 2000000

GET /media/u42/talk.mp4 HTTP/1.1
Range: bytes=900000000-

HTTP/1.1 416 Range Not Satisfiable
Content-Range: bytes */734003200
```

Figure 7: A range answered with 206, and a range beyond the end answered with 416 (illustrative).

7.2 Seeking, resuming and parallel downloads

- **Seeking in video.** When the user drags the slider from zero to the 30-minute mark, the browser does not load the first 30 minutes. It consults the file's index, computes roughly which byte offset corresponds to 30:00, which takes micro- to milliseconds, and issues a range request starting at that byte. The request reaches the CDN, which sends the bytes from that point, and playback starts there at once.
- **Resumable downloads.** A download manager that has received 400,000,000 bytes when the Wi-Fi drops does not start over: once the connection is back, its retry logic requests `Range: bytes=400000000-`, the exact point where the transfer stopped.
- **Parallel downloads.** The download-manager speed-up of Section 1.3 builds on the same header: six connections open to one object, each requesting a different range.

7.3 The demo

The demo runs against a local MinIO object store in Docker; the specific store is irrelevant, the point is the interaction between client and store. A request for the first 100 bytes of an object returns status 206. More usefully, a request for the first eight bytes, piped into the hex-dump tool `xxd`, shows a PNG header, so the file type is known without downloading the file. Such techniques look simple, but when client and server performance is being optimized they save resources by cutting latency, traffic and processing cost. Finally a large download is started, interrupted and restarted: curl requests exactly the missing range, MinIO serves only that range, and the download continues instead of starting over.

```
$ curl -s -D - -o /dev/null -r 0-99 \
      http://localhost:9000/demo/photo.png
HTTP/1.1 206 Partial Content
Accept-Ranges: bytes
Content-Length: 100
Content-Range: bytes 0-99/48213

$ curl -s -r 0-7 http://localhost:9000/demo/photo.png | xxd
00000000: 8950 4e47 0d0a 1a0a                      .PNG....

$ curl -o big.bin http://localhost:9000/demo/big.bin
^C                                      (interrupted)
$ curl -C - -o big.bin http://localhost:9000/demo/big.bin
** Resuming transfer from byte position 400000000
```

Figure 8: Range requests against a local MinIO store: a 206 response, the PNG signature from eight bytes, and a resumed download (illustrative reconstruction of the demo).

8 SEGMENTED STREAMING AND ADAPTIVE BITRATE

8.1 The limit of a single file

Progressive download through range requests works well up to a point, but it has one limitation that ranges cannot fix: there is only one version of the file. Streaming services such as Netflix, YouTube, Amazon Prime Video and Hotstar have complex architectures, but they share one idea: they do not serve the same file to every user. They determine the quality of each user's connection and, depending on it, send a small low-quality stream or a 4K one.

8.2 HLS, DASH and adaptive bitrate

This is done with segmented streaming in formats such as HLS or DASH. First the original upload is transcoded, converted between video formats and encodings, into several qualities. Each quality is then cut into segments a few seconds long. Finally a manifest file records the result: essentially a list of the qualities and of the segments that make up each. On the client, the player fetches the manifest first and then loads the segments one by one. For each segment it measures how long the download took, and from the time of the last segment it decides whether to request the next one at a higher quality or to switch to a lower one. This is **adaptive bitrate** streaming.

```
[bucket after transcoding: plain objects]
videos/7f3a/master.m3u8
videos/7f3a/360p/index.m3u8
videos/7f3a/360p/seg_00001.ts ... seg_01200.ts
videos/7f3a/1080p/index.m3u8
videos/7f3a/1080p/seg_00001.ts ... seg_01200.ts

[master.m3u8: one entry per quality]
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=800000,RESOLUTION=640x360
360p/index.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080
1080p/index.m3u8

[1080p/index.m3u8: the segments, 6 s each]
#EXTM3U
#EXT-X-TARGETDURATION:6
#EXTINF:6.0,
seg_00001.ts
#EXTINF:6.0,
seg_00002.ts
```

Figure 9: A two-hour video stored for HLS: segments and manifests are ordinary objects (illustrative).

8.3 What storage sees, and build versus buy

Nothing changes on the object-storage side: the same files are stored, the segments are simply objects, and so is the manifest that lists them and their relationships. Object storage plus a CDN therefore provides almost everything needed to deliver a video platform. The hard engineering lies elsewhere, in the transcoding pipeline: the background process that runs every time someone uploads a video, with its queues and workers (see Chapter 10). Delivery is already solved by the large object-storage and CDN providers. Hence the recommendation: if video is a small feature of the product, choose a ready-made video service such as Mux or Bunny (both used by the lecturer) and let it handle transcoding and the related engineering; if video is 90% of the product, build your own pipeline to optimize costs.

9 PRICING AND THE COMPLETE RULE SET

9.1 Three things you pay for

The pricing model of the chosen store shapes the architecture. It has three components:

- **Storage**, billed in gigabytes per month.
- **Operations**, billed per request. Writes (PUTs and other modifications) cost about ten times as much as reads such as GET. On S3, LIST is billed at the write rate, while DELETE is free.
- **Egress**, or outbound traffic: all data that leaves the provider. Every file users download from the store counts.

Egress is why Cloudflare R2 is so popular: it charges nothing for outbound traffic, however much data is read, a major advantage for large projects. Egress also explains the CDN patterns of Section 6; on AWS, transfer from S3 to CloudFront is not billed. Part size feeds into the operations line as well: 5 MB parts turn a 50 GB upload into 10,000 billed PUT requests, 64 MB parts into about 800. Concrete prices change every few years and belong to the providers' pricing pages rather than to an architecture discussion.

9.2 The rules of both parts

The lecture closes with a summary of both parts. Object storage exists because local file systems do not scale. It scales by giving up mutability (no in-place edits, renames, folders or locks) and gets in return virtually unlimited capacity, the “eleven nines” of durability that AWS quotes, and a complete HTTP interface. Table 3 collects the resulting rules.

Area	Rule	Lecture
Model	Key, bytes, metadata in a flat bucket; folders are prefix grouping	24
Keys	Generate keys yourself; high entropy first; encode the owner	24
Metadata	Set at write time; changing it rewrites the object	24
Durability	Erasure coding is not backup; set up versioning, replication	24
Listing	Never list the bucket for user queries; the database is the index	24
Small uploads	Through the server only streamed, with a body-size limit	24
Direct uploads	Presigned POST with content-length-range, not open PUT	24
Consistency	Intent in the database first; HEAD check; background cleanup	24
Large uploads	Multipart, dynamic part size; never trust the multipart ETag	25
Cleanup	Abort-incomplete rule on every bucket from day one	25
Downloads	Never relay; presigned URLs, private bucket behind a CDN	25
Ranges, video	Ranges for seek, resume, parallel; segments and ABR for video	25

Table 3: The rules of object storage collected in the lecture's closing summary.

10 CONCLUSION

- A single PUT fails for large files four ways: the 5 GB cap, all-or-nothing transfer, one TCP window, and no progress, pause or resume.
- Multipart upload creates a staging area, accepts independent parts in any order, and completes with a metadata operation that is fast at any size.
- Part size = max(default 16–32 MB, file size / 10,000); the part is the retry unit, and 16–64 MB fits almost every case.
- A multipart ETag is an MD5 of part MD5s plus “-N”; verify integrity with checksums or your own SHA-256 in user metadata.
- Give every bucket a lifecycle rule that aborts incomplete multipart uploads after seven days, from day one.
- Browser uploads: init with a pending row, batched presigned part URLs, 4–6 parallel PUTs, complete plus HEAD check, ETag exposed in CORS, resume via list parts.

- Never relay downloads: public content via a CDN with content-addressed keys, private files via presigned GET or CDN-signed URLs and cookies.
- Range requests (206, 416) give seeking, resumable and parallel downloads; segmented ABR streaming adapts video quality, and storage sees only objects.
- Cost is storage, operations (writes about ten times reads) and egress; egress often decides the design.

APPENDIX A REVIEW QUESTIONS

1. A field-survey app uploads drone footage of 3 to 80 GB from tablets on LTE links that drop for a few seconds several times an hour. Choose a part-size rule and a concurrency level, compute the resulting part counts for 3 GB and 80 GB, and justify the choice against the 10,000-part limit, the cost of a retry and the number of billed requests. State what the app must persist locally so that an upload survives an app restart.
2. Over six months the storage bill of a video-upload product grew by 40%, while the console's object count and total size stayed nearly flat. The upload page has a Cancel button that only stops the JavaScript loop sending parts. Explain the most likely cause, the commands you would run to confirm it, and the two changes, one in bucket configuration and one in code, that stop the growth.
3. A client uploads a 1 GiB file as 64 parts of 16 MiB, computes the MD5 of the whole file locally and compares it with the ETag returned after completion. Predict the outcome and the form of the ETag, and explain why the same check passed in tests that used a 3 MB file. Design an integrity check that holds in both cases and also lets the system notice that two users uploaded the same file.
4. Part uploads to presigned URLs show status 200 in the browser's network panel, but `res.headers.get("ETag")` returns `null` in JavaScript and every completion fails, while the same flow scripted with curl works. Explain the mechanism behind the difference, name the exact configuration change, and predict what the store does if the client sends the completion call with empty ETags.
5. A user starts a 40 GB upload with 32 MiB parts, closes the laptop after 700 parts and returns nine days later to the same page; the bucket aborts incomplete uploads after seven days. Walk through what the resume request finds in the database and in the bucket, what the user should see, and how the upload states and the cleanup job should be designed so that the database never refers to a staging area that no longer exists.
6. A photo-sharing service puts a CDN in front of its private bucket and returns presigned GET URLs valid for 10 minutes. The CDN reports a cache hit ratio of 0.3% and the egress bill did not change. Explain why, then choose a delivery pattern for (a) profile photos viewed by thousands of users and (b) private medical documents, and compare what an attacker could do with a leaked URL under each choice.
7. An object is exactly 4,000,000 bytes long. Predict the status code and the `Content-Range` header for the requests `Range: bytes=0-99`, `Range: bytes=3999900-` and `Range: bytes=5000000-5000100`. Then explain how a player seeks to minute 30 of a two-hour MP4 served this way, and why the same mechanism cannot lower the quality for a viewer whose bandwidth drops mid-film.
8. A download endpoint streams objects through the API (get object, then copy into the response). At peak, 400 users at once download 2 GB files over 20 Mbit/s links. Estimate how long each transfer holds a worker, and explain the effect on a thread-pool server and on an event-loop server (see Chapter 19) and on the traffic bill. Redesign the endpoint so that it keeps per-file authorization but serves no bytes itself.

APPENDIX B GLOSSARY

Term	Meaning
Multipart upload	Upload of one object as numbered parts, committed by a final call
Upload ID	Identifier of a multipart upload's staging area
Part	One independently uploaded fragment; the unit of retry
Multipart ETag	MD5 over the part MD5s plus “-N” (N parts); not a content hash
Checksum	CRC32C or SHA-256 per part or object, verified by the store
Incomplete upload	Multipart upload never completed or aborted; billed, not listed
Lifecycle rule	Bucket rule acting by age, e.g. abort incomplete uploads after 7 days
ListParts	Call returning the stored parts and ETags of an upload ID
Congestion window	TCP's cap on unacknowledged data; limits one connection's rate
Edge	CDN servers at network nodes close to users
CDN signed URL / cookie	Access grant checked by the CDN at the edge
Content-addressable key	Key derived from a hash of the object's content
Range request	GET with `Range`; answered 206 with `Content-Range`, or 416
Segmented streaming	Video as short segments in several qualities plus a manifest (HLS, DASH)
Adaptive bitrate	Player choosing each segment's quality from measured download time
Egress	Data leaving the provider, billed per gigabyte

APPENDIX C FURTHER TOPICS

Resumable upload protocols

tus is an open protocol for resumable uploads over plain HTTP: the client creates an upload resource, sends bytes with PATCH requests that carry an `Upload-Offset` header, and after an interruption asks the server for the current offset with HEAD. The IETF HTTP working group is standardizing a similar mechanism as “Resumable Uploads for

HTTP". It matters when uploads go to your own servers or to a store without an S3-style multipart API, and it gives resumability without per-part bookkeeping in the client.

Transfer acceleration

S3 Transfer Acceleration lets clients upload to the nearest CloudFront edge location, from which the data travels to the bucket's region over the AWS backbone instead of the public internet. It shortens the long, lossy path that limits single-connection throughput, so it helps most for users far from the bucket's region. It is enabled per bucket, uses a separate endpoint and is billed per gigabyte on top of normal transfer, so it is worth it only where measurements show a gain.

Server-side copy of large objects

Objects cannot be renamed or moved, so "moving" one means copying it to a new key and deleting the old one. A single copy request handles objects up to 5 GB; larger objects are copied with a multipart upload whose parts are byte ranges of the source (`UploadPartCopy`), and the bytes never leave the provider. This is how a backend reorganizes keys, changes the metadata of large objects or migrates them between buckets without downloading and re-uploading.

Storage classes and lifecycle transitions

Object stores offer cheaper storage classes for rarely read data, such as infrequent-access tiers and archive tiers like S3 Glacier, at the price of retrieval fees, minimum storage durations and, for archives, retrieval delays. Lifecycle rules, the same mechanism that aborts incomplete uploads, can move objects between classes by age and delete them on expiry. For backups, old media and logs this often cuts storage cost several-fold, but access patterns must be known first, because retrieval fees can exceed the savings.

Tiered caching and origin shield

Each CDN edge keeps its own cache, so a popular object may be fetched from the origin once per edge location. Tiered caching (an origin shield in CloudFront, tiered cache in Cloudflare) adds a regional cache layer that the edges consult before the origin. It raises the overall hit ratio and cuts requests and egress at the bucket, which matters for large catalogues viewed from many regions.

CMAF and low-latency streaming

The Common Media Application Format (CMAF, ISO/IEC 23000-19) defines fragmented-MP4 segments that both HLS and DASH manifests can reference, so a platform stores one set of segments instead of one per format. Its chunked encoding also underlies low-latency HLS and low-latency DASH, which bring live-stream delay down from tens of seconds to a few. For a backend it decides how the transcoding pipeline writes its output and how much segment storage dual-format delivery needs.

DRM and content protection

Signed URLs and cookies control who may fetch segments, not what happens to the bytes afterwards. Digital rights management systems (Widevine, FairPlay, PlayReady) encrypt the segments, commonly with the Common Encryption scheme, and the player obtains decryption keys from a license server that applies the business rules. Licensed premium content usually requires DRM by contract; user-generated or internal video rarely does.

Upload scanning and validation

A completed upload is untrusted input. A typical pipeline consumes the `file.uploaded` event, checks the real file type from the leading bytes (the magic numbers seen in the PNG demo) rather than the declared content type, scans for malware, and only then moves the object out of a quarantine prefix or marks it available. This protects other users and downstream processors such as image libraries and PDF renderers; see Chapter 16 for the wider security context.

Bucket event notifications

Object stores can emit events when objects are created or deleted; S3 Event Notifications deliver them to queues, functions or EventBridge, with a dedicated event type for completed multipart uploads. They let the backend learn about a new object even when a client uploads directly with a presigned PUT or POST and never calls the completion endpoint, and they are a natural trigger for scanning, thumbnails or transcoding. Delivery can repeat an event, so consumers must be idempotent (see Chapter 10).

Real-Time Backends — One-Page Overview

Chapter 22 · every section and sub-section of the paper at a glance

1 Introduction: When the Server Has News

Real-time features need the server to speak first, which request-response does not allow.

- **1.1 The live-board requirement** — Two users on one board; a moved task must appear for the other live.
- **1.2 The client always speaks first** — The server can only answer; techniques are judged by delay and by cost per event versus per user.

2 Polling

Asking on a timer works for small tools, but its delay and cost scale badly.

- **2.1 Asking on a timer** — Every 3 s: anything new on board 412? Demo counters show most requests wasted.
- **2.2 Latency is half the interval** — 3 s polling means 1.5 s average delay; polling faster is the obvious fix.
- **2.3 Cost grows with users, not events** — 10,000 users: 3,333 req/s at 3 s, 10,000 at 1 s, each a full request with a database query.
- **2.4 Batteries and the real goal** — Polls wake the phone radio; goal: server-initiated delivery, cost per event.

3 Long Polling

The server holds the request until news arrives, but every response leaves a gap.

- **3.1 Letting the server take its time** — Request held up to a minute, answered on change, re-issued at once.
- **3.2 The gap between responses** — RFC 6202: about one transit on average, over three in the worst case.

4 Server-Sent Events

One HTTP response kept open carries a stream of events from server to client.

- **4.1 A response that never ends** — GET, `200`, `text/event-stream`, never closed; plain HTTP.
- **4.2 The event-stream format** — Four fields: `id`, `event`, `data`, `retry`.
- **4.3 Reconnection built into the browser** — Automatic reconnect that sends `Last-Event-ID`.
- **4.4 Not a toy: who uses SSE** — Uber driver push, LinkedIn messaging, LLM token streaming.
- **4.5 One direction only** — Client actions need separate POST requests.

5 WebSockets

An HTTP request upgraded into a two-way stream of compact frames.

- **5.1 The opening handshake** — GET with `Upgrade` and a 16-byte key; `101 Switching Protocols`.
- **5.2 What the key proves** — SHA-1 of key plus a fixed GUID: proof of understanding, not security.
- **5.3 Frames** — FIN, opcode, MASK, 7-bit length; a 2-byte header for small payloads.
- **5.4 Masking against cache poisoning** — A random per-frame XOR key stops forged requests reaching proxies.
- **5.5 Ping, pong and dead connections** — TCP drops are silent; no pong within 30 s means dead.

6 Connections per Machine

One Linux machine meets descriptors, ports and memory in turn.

- **6.1 The experiment** — 16 cores, 61 GB, a Go server that only holds sockets.
- **6.2 File descriptors** — `ulimit -n` 1024 stops at 1,017; clients just time out.
- **6.3 Why Go reported a million** — The Go runtime raises its own limit at startup; Python does not.
- **6.4 Ephemeral ports and the four-tuple** — 28,190 client ports; three client addresses reach 75,000.
- **6.5 Memory per connection** — About 9.6 KB of heap each; a million need about 10 GB.

7 Real-Time Across Many Instances

A connection is state in one process, so instances must share events.

- **7.1 A connection is state** — Users on different instances do not see each other's changes.
- **7.2 Sticky sessions solve a different problem** — Same client to same server, not two clients.
- **7.3 Publish-subscribe between instances** — Publish to the board topic; each instance delivers locally.
- **7.4 At most once: messages lost in deploys** — Redis Pub/Sub keeps nothing; Streams or Kafka for at least once.
- **7.5 Catching up after a disconnect** — Sequence numbers; the client reports the last one received.

8 Fan-Out and Reconnection Storms

Large audiences and mass reconnects are the remaining scale problems.

- **8.1 Fan-out: events times subscribers** — Discord: 0.9 to 2.1 s per event for 30,000 online users.
- **8.2 Reconnection storms** — 50,000 clients loading state at once; Slack's edge cache.
- **8.3 Where the topic continues** — Presence, collaborative editing, CRDTs.

Real-time backends at a glance — the server gains a voice step by step: polling, long polling, SSE and WebSockets; then one machine's limits (descriptors, ports, memory) and many machines joined by pub/sub, with catch-up, fan-out and storm control. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Real-Time Backends: From Polling to WebSockets at Scale

Chapter 22 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — A live board needs the server to tell a client about a change the client did not ask for, which the request-response model cannot express. Polling achieves it with an average delay of half the interval and a cost that grows with users rather than events, and long polling leaves a gap after every response. Server-Sent Events keep one HTTP response open with browser-managed reconnection but flow one way, while WebSockets upgrade an HTTP request into a two-way stream of compact, masked frames with ping/pong heartbeats. A capacity experiment on one Linux machine meets three limits in turn: file descriptors, the client's ephemeral ports and about 9.6 KB of memory per connection. Finally, persistent connections are treated as state spread over many instances and joined by publish-subscribe, with delivery guarantees, catch-up by sequence numbers, fan-out and reconnection storms.

Keywords — real-time, polling, long polling, Server-Sent Events, WebSocket, file descriptors, ephemeral ports, publish-subscribe, fan-out

1 INTRODUCTION: WHEN THE SERVER HAS NEWS

Every interaction in the backend built so far has the same shape: the browser asks and the server answers. Authentication, handlers, services, repositories, caching, background jobs and object storage all live inside that cycle. A common product requirement does not fit it.

1.1 The live-board requirement

A task-management app shows a board. Two people have the same board open, and one of them drags a task from the *To do* column to *In progress*. The other person, possibly on another continent, must see the card move live: not on the next page refresh, not after navigating away and back, but at the moment it happens.

1.2 The client always speaks first

Thought through without knowledge of the available tools, the requirement runs into one limitation at once. In the model used so far (see Chapter 1) the client initiates every exchange and the server responds. The server has no mechanism to speak first: it can answer what the browser asked, or hold a connection open for a long time, but it cannot open the conversation. News the browser did not request, such as "a task moved on board 412", therefore has no path from server to client. Each technique below is a way around this constraint, and each is judged by two measures: how quickly a change reaches the other user, and whether the resources spent grow with the number of *events* or with the number of *users*.

2 POLLING

2.1 Asking on a timer

If the server cannot announce a change, the client must ask for it repeatedly. Every three seconds the browser sends: "Has anything changed in the last three seconds on board 412, the board I am viewing?" Most answers are "no". Occasionally, after someone has moved a task, the answer is "yes", and the server either returns the specific change or merely reports that something changed, after which the client reloads the whole board and redraws the screen.

The lecture's demo runs two browsers, Ava's and Leo's, side by side, both polling every 3 s and each showing two counters: requests sent and "no change" answers. The counters climb together almost all the time. Only when Ava moves a task does Leo's next poll find a change, so his no-change counter does not advance for that one request. Every other request is wasted.

Polling works. For an internal tool with a small number of users, polling every few seconds is a perfectly reasonable architecture. An end-user product, however, must scale, so the question becomes what breaks, and at which level.

2.2 Latency is half the interval

A change happens at a random moment between two polls and waits for the next one, so on average it waits half the interval. With a 3-second interval the average delay before the other user sees a moved, updated or deleted task is 1.5 s, whatever the server's speed: it is pure arithmetic. The obvious remedy is to poll more often, for instance every second, so that the delay is no longer noticeable.

2.3 Cost grows with users, not events

Suppose the product succeeds and 10,000 people have a board open at the same time. At one poll every 3 s that is 3,333 requests per second, arriving continuously. Each is a full HTTP request: it passes the load balancer, the authentication middleware validates the token, the logging middleware writes a log line, the service layer queries the database, and the result is serialized back to JSON (see Chapter 6). That amounts to roughly 3,000 database queries per second, almost all of which answer "nothing changed". Shortening the interval to 1 s raises the load to 10,000 requests per second: the user experience becomes three

times better and the monthly cloud bill three times worse (Table 1).

Interval	Average delay	Requests/s	DB queries/s
3 s	1.5 s	3,333	about 3,333
1 s	0.5 s	10,000	about 10,000

Table 1: Polling cost with 10,000 open boards and one database query per poll

The deeper fault is what the cost is proportional to. A board on which nothing happens all day consumes exactly the same resources as a board that changes every second, because the system pays per request and per connected user, not per change. The value lies in the events themselves; requests are only the means of carrying them, and in polling a very poor means.

2.4 Batteries and the real goal

On mobile phones every poll wakes the radio module, one of the most energy-hungry operations a smartphone performs. A polling loop therefore drains the user's battery and mobile data even when nothing on the screen changes. Two goals follow for every better design: the server must be able to initiate delivery, and resources must be spent on actual events rather than on the number of users, browsers or requests.

3 LONG POLLING

3.1 Letting the server take its time

The first widely used workaround keeps HTTP's rule that the client asks and the server answers, but lets the server take its time. In *long polling* the browser sends a request and the server does not respond immediately; it holds the request open, sometimes for a minute, and waits. When a change occurs, the server writes a response describing it and the request completes. The browser receives it and immediately sends the next request, starting another cycle (Figure 1).

```
browser                          server
   | GET /boards/412/poll  ----->  |  held up to ~60 s
   |                               |  task 7 moved
   | <----  200 {"task": 7}        |  request completes
   | parse, render, re-request     |  gap: no request open
   |                               |  task 9 moved
   | GET /boards/412/poll  ----->  |  answered at once
   | <----  200 {"task": 9}        |  only if buffered
```

Figure 1: Long polling and the gap between a response and the next request (illustrative)

3.2 The gap between responses

Long polling is documented, together with its drawbacks, in RFC 6202. Its central weakness is the window after each response: the browser must receive and process the response and then open a new request, and during that window the server has no connection to the client. A change that occurs in the gap is lost unless the server buffers it. According to the RFC, the average latency is close to one network transit, but the worst case exceeds three (response, new request, response). Every message also carries a complete set of HTTP headers, hundreds of bytes each time. Long polling served real-time needs for years, but it was eventually superseded.

A subtle point. Long polling has not disappeared: it survives as the last-resort transport of real-time libraries. ASP.NET Core SignalR, for example, tries WebSockets first, then Server-Sent Events, then long polling, so that clients behind proxies that block the first two still work.

4 SERVER-SENT EVENTS

4.1 A response that never ends

The next idea removes the end of the long-poll response. The browser sends an ordinary HTTP GET; the server answers with status 200 and the header `Content-Type: text/event-stream`, and then never closes the response. Whenever it has something to say, it writes a small piece of text into the same open response. The connection lasts as long as the tab is open. It closes when the tab or browser closes or the client's network fails, and the client's retry mechanism then reopens it; while the application is open, the channel is permanent. This is not a separate protocol: it is HTTP throughout, using HTTP headers and an HTTP response body.

Server-Sent Events (SSE). A one-way stream of events from server to client, carried in a single long-lived HTTP response of media type `text/event-stream` and consumed in the browser through the `EventSource` interface of the HTML standard.

4.2 The event-stream format

The demo uses curl as the client against a local server. The request declares in advance that it accepts `text/event-stream` and targets `/api/events`; from then on events arrive as they occur (Figure 2). An event is a few `field: value` lines ended by an empty line, and the specification defines only four fields:

- `event`: the event type, such as `task.moved`, `task.created` or `task.deleted`;
- `data`: the payload, which the interface can render immediately, for example by adding or moving a card;
- `id`: an identifier the client remembers, so that after a dropped connection, a refreshed tab or any failure it can say "this is the last event I received; send me everything after it" and catch up without delay;
- `retry`: how long, in milliseconds, the browser should wait before reconnecting.

```
$ curl -N -H "Accept: text/event-stream" \
       http://localhost:3000/api/events

retry: 3000

id: 41
event: task.moved
data: {"board":412,"task":7,"to":"in_progress"}

id: 42
event: task.created
data: {"board":412,"task":9,"title":"Fix login"}
```

Figure 2: An SSE stream read with curl; each event ends with a blank line (illustrative values)

4.3 Reconnection built into the browser

Because SSE is part of the HTML standard, reconnection needs no application code. When the connection is lost, the browser waits and reconnects by itself, and on reconnecting it sends the request header `Last-Event-ID` with the `id` of the last event it received. The server's share of the work is to keep recent events and resume the stream after that identifier.

4.4 Not a toy: who uses SSE

Since WebSockets are the best-known real-time technology, SSE is sometimes regarded as an inferior or toy version of the "real" architecture. Production systems contradict this. Uber's push-notification platform, which tells a driver about a trip, is built on SSE, and LinkedIn's instant messaging, with typing indicators and read receipts, uses SSE as well; in both cases plain HTTP removed compatibility concerns across devices and network environments, because it simply worked everywhere. Large language model APIs also stream their tokens to every client, whether CLI or browser, as Server-Sent Events: text appearing word by word is the `data` field of Figure 2 arriving event after event. SSE has its own purpose; the design question is where it fits and where something more powerful is needed.

4.5 One direction only

The name states the limitation: the events are sent by the server. The server can talk all day, but the browser cannot send anything back over the same connection. When a user drags a task, the browser must notify the server with a separate POST request. For many features and products this split is entirely acceptable. The live board, however, is taken here as needing two-way communication over a single connection.

5 WEBSOCKETS

5.1 The opening handshake

The demo points a command-line WebSocket client at a server on port 7821 that offers an echo endpoint at `/echo`. The client's log shows a TCP connection to port 7821, then "Connected to WebSocket", and among the received headers `Upgrade: websocket`, `Connection: Upgrade` and `Sec-WebSocket-Accept`. This exchange is the WebSocket *handshake*. Afterwards the client sends a message (a task moved on board 412), the server answers instantly, then sends something of its own, and so on: both directions over one connection.

A WebSocket connection does not start as a WebSocket. It starts as an ordinary HTTP GET, and that is its strength: it passes through everything that already understands HTTP, which is practically every device. The request carries `Upgrade: websocket` and `Sec-WebSocket-Key`, 16 random bytes encoded in base64. The server answers with status `101 Switching Protocols`, switching from HTTP to the WebSocket protocol, and returns `Sec-WebSocket-Accept` (Figure 3).

```
GET /echo HTTP/1.1
Host: localhost:7821
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=

Accept = base64( SHA-1( Key +
         "258EAFA5-E914-47DA-95CA-C5AB0DC85B11" ) )
```

Figure 3: The opening handshake and the accept computation; key and accept are the worked example of RFC 6455 (illustrative)

5.2 What the key proves

To compute the accept value, the server appends a fixed string, a constant written in the specification, to the client's key, takes the SHA-1 hash of the result and encodes it in base64. This looks like a security measure but is not one, since anyone can compute it. It proves only that the party answering understood that a WebSocket was requested, and is not, for example, a cache replaying an old 200 response. Once the exchange completes, the HTTP part is over. The TCP connection stays open, but there are no more headers, methods or status codes: it is now a two-way stream of *frames*, and either side may write whenever it wants.

5.3 Frames

RFC 6455 defines the frame as a bit layout (Figure 4). The first bit, FIN, marks the last fragment of a message. Three reserved bits follow, then a four-bit opcode giving the frame type: 1 text, 2 binary, 8 close, 9 ping, 10 pong, and 0 for a continuation of the previous message. The second byte holds the MASK bit and a seven-bit payload length, an efficient encoding: a payload shorter than 126 bytes stores its length directly and the whole header costs two bytes; the value 126 means the real length is in the next two bytes, and 127 that it is in the next eight. A short message therefore travels with a two-byte header (six bytes from a client, which adds the masking key), against the hundreds of bytes of headers on every long-polling request.

```
byte 0   FIN (1 bit) | RSV1-3 (3 bits) | opcode (4 bits)
byte 1   MASK (1 bit) | payload length (7 bits)
           0-125  the length itself    header = 2 bytes
           126    next 2 bytes hold the length
           127    next 8 bytes hold the length
then     masking key, 4 bytes (client-to-server only)
then     payload (byte i XOR key[i mod 4] if masked)

opcode   0 continuation   1 text    2 binary
         8 close          9 ping   10 pong
```

Figure 4: The WebSocket frame header described in RFC 6455 (illustrative layout)

5.4 Masking against cache poisoning

Every frame the browser sends to the server is *masked*: each payload byte is XORed with one byte of a four-byte key, the key repeating cyclically. The key travels in clear, in the four bytes before the payload of every frame, so masking is not encryption; anyone reading the traffic can undo it with one line of code. It exists because, while the protocol was being designed, researchers ran an experiment against real caching proxies deployed on the Internet and poisoned them. The attack works as follows. A malicious page opens a connection to a server the attacker controls, performs something resembling an HTTP upgrade, and then sends bytes carefully crafted to look like a GET request for a popular file, such as an analytics script loaded by thousands of sites. A proxy in the middle that does not understand the new protocol sees an ordinary request and response and caches the attacker's response. From then on, everyone behind that proxy who requests the real file receives the attacker's version.

Masking defeats this. The browser XORs each frame with a fresh random key that the page's JavaScript cannot predict, so the attacker chooses what to send but not how it looks on the wire, and cannot place a fake HTTP request in the stream. That is why the specification demands a new, unpredictable key for every frame. Masking protects the network infrastructure between the parties, not the data; frames from server to client are not masked.

5.5 Ping, pong and dead connections

Opcodes 9 and 10 implement a *heartbeat*: either side may send a ping, and the other must answer with a pong. It is needed because TCP reports nothing when a connection silently dies, for instance when the user steps into an elevator or the phone switches from Wi-Fi to mobile data. No error and no close arrive; the server keeps holding the socket and believes the client is connected, potentially for a very long time. From the socket's point of view an idle connection and a dead one look exactly the same, because in both nothing happens. The only way to tell them apart is to send something and see whether an answer comes back. A common production rule treats a ping that receives no pong within 30 seconds as a broken connection, which the side that sent it then closes, freeing the resources it held. (TCP keepalive could also detect a dead peer, but on Linux its default idle time is two hours, which is why the heartbeat lives in the application protocol.)

Technique	How it works	Main weakness
Polling	client asks on a timer	delay = interval / 2; cost per user
Long polling	server delays its answer	gap after each response; full headers
SSE	one HTTP response kept open	server-to-client only
WebSocket	HTTP upgrade to two-way frames	stateful connections to scale

Table 2: The four real-time techniques compared

6 CONNECTIONS PER MACHINE

6.1 The experiment

With persistent two-way connections in place, the next question is how many of them one machine can hold. The test machine is an ordinary Linux server with 16 cores and 61 GB of free memory. It runs a WebSocket server written in a few lines of Go that only accepts connections and holds them; there is no application logic, so only the cost of connections is measured. Table 3 summarizes the three runs.

6.2 File descriptors

On Unix-like systems such as Linux and macOS everything is a file, and a network socket is no exception: the process works with it through a file descriptor, with kernel abstractions on top. The number of connections a process can hold is therefore bounded by the number of files it may keep open. `ulimit -n` reports 1024. Started under that limit and offered 2,000 connections, the server stops at 1,017. The difference is expected: standard input, output and error, the listening socket and the epoll instance the runtime uses to watch the sockets account for about seven descriptors.

The two sides see different symptoms. The client received no "connection refused" and no error, only timeouts while waiting for the handshake: the kernel still completes the TCP handshake and queues the connection, but the process can never accept it, so the WebSocket handshake gets no reply. The server's log shows the real cause, "too many open files", repeated with exponential backoff as its accept loop retries. The server has not crashed. It is fully functional, keeps serving its existing connections and answers the health checks configured for it; it simply accepts no new connections.

> **Warning.** Descriptor exhaustion is a quiet failure: the process is alive, existing users are served, and new users merely time out. Alert on accept errors such as `EMFILE` ("too many open files") and on the count of open descriptors, not only on process health.

6.3 Why Go reported a million

On the same machine and in the same terminal, a Go program asked for its descriptor limit reports 1,048,575, while Python reports 1024. The Go runtime raises its own limit at startup. A comment in the Go standard library explains

that some systems set an artificially low limit for compatibility with code that uses `select`, whose maximum descriptor is hard-coded by the size of its descriptor set; Go does not use `select`, so the Go team decided that the runtime should raise the limit for itself and leave older software as it is. The lecturer reads the code as setting the limit deliberately one below the maximum, so that a later change by someone else can be detected, which is why the value is 1,048,575 rather than 1,048,576 (2^20). The famous limit of about a thousand descriptors is therefore real, but it depends on the runtime: a Node or Python server meets it unless the limit is raised, while a Go server has far more room.

A subtle point. The lecture lists Java among the affected languages, but on Linux the HotSpot JVM (option `MaxFDLimit`, on by default) and the .NET runtime also raise their soft limit to the hard limit at startup; Node and Python do not. What bounds every runtime is the *hard* limit, set by `ulimit -Hn`, systemd's `LimitNOFILE` or the container runtime.

6.4 Ephemeral ports and the four-tuple

With the descriptor limit removed, the next run stops at 28,190 connections with a different error: "cannot assign requested address". Asked which ports it may use for outgoing connections (`/proc/sys/net/ipv4/ip_local_port_range`), the kernel answers 32,768 to 60,999, that is 28,232 ports. Forty-two of them were already taken by other services on the shared machine, which leaves exactly 28,190. The limit could have been calculated before the test. Neither memory nor descriptors ran out; port numbers did, and on the client, not on the server.

This refutes a common belief that a server can hold at most about 65,000 connections because there are only 65,536 ports. A TCP connection is identified by four values, source address, source port, destination address and destination port, and must be unique only in the combination. The server's address and port never change, so every connection to it must differ in the other two. The load-generating client had a single address, which left exactly one variable, its own source port, and only 28,232 of those. The remedy is therefore not a more powerful server. Given three source addresses and asked for 25,000 connections from each, the client opened 75,000 connections with zero errors, and the server held 75,009 file descriptors: the 75,000 sockets plus the same handful as before. Load tests of real-time systems consequently need several client machines or addresses to generate realistic load.

Run	Limit hit	Connections	Symptom
fd limit 1024	file descriptors	1,017	client timeouts; "too many open files"
Go, ~1 M fds, one client IP	client ports	28,190	"cannot assign requested address"
three client IPs × 25,000	none	75,000	zero errors; 722 MB heap

Table 3: The single-machine capacity experiment (16 cores, 61 GB RAM, Go server)

6.5 Memory per connection

The remaining limit is memory. Idle, the server used about 7.5 MB; holding 75,000 connections, 722 MB. Subtracting and dividing gives roughly 9.5 KB per connection of the process's own heap. This is not the full cost to the machine, because the kernel keeps its own structures for every socket and they do not appear in the process's resident memory. Three measurements gave 9,551, 9,752 and 9,881 bytes, about 9.6 KB, so a million connections need just under 10 GB of heap before the kernel takes its share: entirely achievable on a 61 GB machine. A million idle connections are mostly a question of RAM.

Most of those 9.6 KB are not the socket. They are the goroutine started to supervise each connection and the buffers allocated for it. For this reason, systems that hold very large numbers of connections do not dedicate a goroutine to every connection; they watch all sockets from one place with `epoll`.

7 REAL-TIME ACROSS MANY INSTANCES

7.1 A connection is state

Production never runs on one machine. Autoscaling and the overall architecture produce several instances; deployments remove containers and start new ones; machines fail. In each case users' connections break and must be re-established. The demo places the same board behind a load balancer with two server instances. Browser A connects and is routed to instance 1; browser B is routed to instance 2. Both hold live WebSockets. A task moved in browser A does not appear in browser B.

The reason is where the connection lives. Instance 1 has the socket in memory, together with its goroutine, and knows that this socket watches board 412. Instance 2 knows nothing about browser A and cannot, because they are separate processes, possibly in different countries. Most of this course has worked to make the backend stateless so that any instance can serve any request (see Chapters 17 and 18). A persistent connection reverses that: it is state, it lives in the memory of one particular process, and it cannot be moved or shared. Once an instance accepts a WebSocket, it is the only one in the world that can talk to that user.

7.2 Sticky sessions solve a different problem

A frequently proposed fix is *sticky sessions*: the load balancer always routes the same client to the same instance, including on reconnection. With stickiness, browser A keeps landing on instance 1, but browser B is still on instance 2 and still knows nothing about A's change. Stickiness keeps one client with one server; the problem here is two clients on two servers at the same time. The instances themselves must communicate.

7.3 Publish-subscribe between instances

The standard solution is *publish-subscribe*. When a change on board 412 reaches an instance, that instance does not try to deliver it; it publishes the event to a topic. A broker in between (Redis, NATS, Kafka or any other pub/sub system) passes the event to every instance subscribed to that topic. Each of them looks through its own connections, finds those watching the board and writes the event to them (Figure 5). The instance that receives a change and the instances that deliver it can now be different machines. Repeated with pub/sub, the demo shows Ava moving a task on one instance and Leo, connected to the other, seeing it instantly.

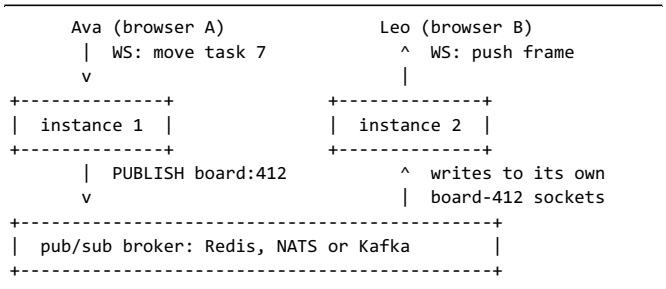


Figure 5: Two instances joined by a pub/sub broker; instance 2 subscribed to board:412 when its first viewer of that board connected (illustrative)

7.4 At most once: messages lost in deploys

Redis Pub/Sub is fire-and-forget. If an instance is not subscribed at the exact moment a message is published, the message disappears for good: it is not queued, postponed or retried. Choosing Redis Pub/Sub is therefore an architectural decision with a known delivery model, *at most once*. Where *at least once* is required, the broker must store messages and track acknowledgements of their receipt, as Redis Streams, Kafka or a message queue do (see Chapter 10).

Warning. Deployments turn message loss into a routine event. While an instance is shut down, its container stopped and a replacement started, every message for the users served by that instance is lost, and their boards show those changes only after the next page refresh.

7.5 Catching up after a disconnect

Even with a durable broker, a user who is not connected misses messages, and disconnection is common: the connection drops, the phone moves from Wi-Fi to mobile data, someone closes the laptop lid, the load balancer times out an idle connection. A production real-time system therefore needs a *catch-up strategy*, not only a delivery strategy. Its form is already familiar from SSE: number every message, let the client report the last number it actually received, and after any connection failure send everything after it. Uber's platform does exactly this (Figure 6). The client connects saying "my sequence is 0", and the server sends messages with increasing sequence numbers. If the connection drops, the client reconnects with "I received up to 2", and the server resumes at 3, the point where the client lost the stream; it is the same mechanism as the `Last-Event-ID` header.

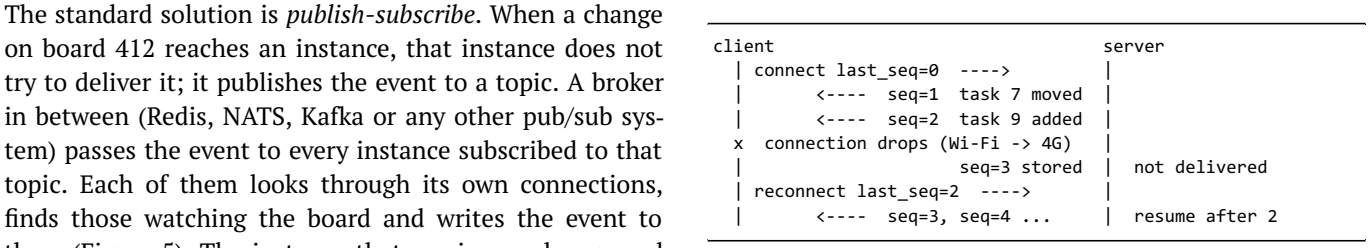


Figure 6: Catch-up with sequence numbers, as in the lecture's Uber example (illustrative)

8 FAN-OUT AND RECONNECTION STORMS

8.1 Fan-out: events times subscribers

Pub/sub solves the problem of many instances, not the problem of volume. If 30,000 people have the same board open, one person moving one task means 30,000 writes. The real work of a real-time server is the number of events multiplied by the number of subscribers. Discord, whose core product is messaging, wrote about communities with 30,000 users online at once: publishing a single event to one of them took between 900 ms and 2.1 s. Their solution was to stop fanning out from one place and to distribute the work across the machines that actually hold the recipients, each delivering to its own share.

Fan-out. The delivery of one event to every subscriber interested in it. Its cost is events multiplied by subscribers, so one change in a very large room can cost as much as thousands of changes in small ones.

8.2 Reconnection storms

Consider an instance holding 50,000 connections that goes down. All its clients start reconnecting at the same moment, and on connecting each wants its initial state: which boards, which tasks, who is online now. The result is 50,000 expensive requests within about a second, aimed at the core of the system. Slack described exactly this scenario. Their fix was to make connecting cheap: a cache at the network edge holds the information a client needs to start, so the load of a reconnection storm is absorbed there and never reaches the original data source.

8.3 Where the topic continues

Several large areas build on these foundations, each opening onto its own technologies: *presence*, showing which users are online right now, and *collaborative editing*, where two people change the same thing at the same time and something must decide which change takes priority, as with the simultaneous cursors in Notion or Google Docs. Collaborative editing in particular requires algorithms such as CRDTs and an understanding of how they behave over the network.

9 CONCLUSION

- Request-response does not let the server speak first; every real-time technique works around that.

- Polling suits small internal tools, but its delay is half the interval and its cost grows with users, not events.
- Long polling holds a request until there is news, yet every response leaves a gap (RFC 6202).
- SSE keeps one HTTP response open, with four fields and browser-managed reconnection via `Last-Event-ID`; it is production-grade but one-way.
- WebSockets upgrade an HTTP GET with `101`; small frames carry two-byte headers, client frames are masked to protect proxies, and ping/pong detects dead peers.
- One machine is bounded in turn by file descriptors, the client's ephemeral ports (the four-tuple) and memory, about 9.6 KB of heap per idle Go connection.
- A persistent connection is state in one process; sticky sessions do not help, pub/sub between instances does.
- Redis Pub/Sub is at most once; durable brokers plus sequence numbers let clients catch up after drops and deploys.
- Fan-out (events times subscribers) and reconnection storms must be designed for: spread delivery and make connecting cheap.

APPENDIX A REVIEW QUESTIONS

1. An analytics dashboard has 25,000 sessions open during business hours and polls `GET /api/metrics/changes` every 5 s; each poll validates a JWT, writes an access-log line and runs one indexed query. Product now asks for an average update delay below 300 ms. Compute the request rate polling would need to meet that target, explain why the resulting bill would not follow how often the metrics actually change, and argue which technique from this chapter you would adopt instead, given that the dashboard never sends data back.
2. A notification service uses long polling: each request is held for up to 60 s and answered with the first new notification. In a busy chat, users report that when several messages arrive within a few hundred milliseconds, some appear only after a page reload, although the server logs show every message was produced. Explain the mechanism that loses them, compare the delay of the second message in a burst with that of the first, and state what the server would have to keep to fix the problem without leaving long polling.
3. An SSE endpoint sends `retry: 10000` and gives every event an `id`, but the server keeps no event history. A user's train enters a tunnel for 20 s, during which three tasks move on the board. Describe step by step what the browser does from the moment the connection drops until events flow again, which header it sends, what the user's board shows afterwards, and what the server must add so the board is correct without a full reload.
4. A server receives the client frame `81 85 37 fa 21 3d 7f 9f 4d 51 58` (hexadecimal). Decode FIN, opcode, MASK, payload length, masking key and payload text, showing the XOR step. Then explain what a server should do with an unmasked frame from a browser client, and why a browser that reused one masking key for a whole connection would reopen the attack that masking was designed to stop.
5. A Python asyncio service on Linux must hold 400,000 mostly idle WebSocket connections per node; `ulimit -n` shows 1024 and you measured 9.7 KB of process memory per connection. List, in the order you would hit them, the limits between zero and 400,000 connections, what each failure looks like on the client and on the server, and the setting or design change that lifts it. Then plan the load test from machines whose local port range is 32768 to 60999: how many source addresses do you need, and why does a bigger server not help?
6. A team runs three instances of a WebSocket service behind a load balancer with sticky sessions; each instance broadcasts board updates to the sockets in its own memory. Users report that "about a third of the time" a colleague's change appears live and otherwise only after a refresh. Explain the one-third figure, why stronger stickiness cannot help, and design the replacement: what is published, to which topic, by whom, and what every instance does on receipt.
7. Instances exchange board events through Redis Pub/Sub, and a rolling deployment replaces one of ten instances every 40 s. During deploys, tickets report boards missing moves until refresh. Explain exactly which messages are lost and why, then design a catch-up scheme with sequence numbers: whether numbers are assigned per board or per user, where messages are retained and for how long, what the client sends on reconnection, and how moving to Redis Streams or Kafka changes the delivery guarantee.
8. An instance holding 50,000 WebSocket connections crashes, and all clients reconnect within about one second, each requesting its initial state with a query the database can serve at 5,000 per second. Estimate how long the database stays saturated and what users experience meanwhile. Then combine the edge-cache approach of this chapter with the caching techniques of Chapter 9 and client-side measures to keep the storm away from the database, and state what each measure costs in staleness.

APPENDIX B GLOSSARY

Term	Meaning
Polling	Client asks for changes on a timer; average delay is half the interval
Long polling	Server holds a request until there is news; client then re-requests at once
Server-Sent Events	One-way event stream in a long-lived `text/event-stream` response
Last-Event-ID	Header the browser sends on SSE reconnect with the last event id received
WebSocket	Two-way, frame-based protocol on one TCP connection, begun by an HTTP upgrade
Sec-WebSocket-Accept	base64 of SHA-1(key + fixed GUID); proves the upgrade was understood
Masking	XOR of client payloads with a random per-frame 4-byte key; protects proxies
Heartbeat	Ping/pong exchange that tells an idle connection from a dead one
File descriptor	Per-process handle to an open file or socket, capped by `ulimit -n`
Ephemeral port	Source port the kernel picks for an outgoing connection
Four-tuple	Source address and port, destination address and port: a connection's identity
Sticky session	Load-balancer routing that sends a client to the same instance each time
Publish-subscribe	Publishers send to topics; a broker delivers to every subscriber
At most / at least once	Delivery that may lose messages / may duplicate them but acknowledges
Fan-out	Delivery of one event to all its subscribers; cost is events times subscribers
Reconnection storm	Mass reconnection after an instance fails, each client loading its state

APPENDIX C FURTHER TOPICS

CRDTs and operational transformation

Conflict-free replicated data types are data structures whose replicas can be updated independently and concurrently and always merge to the same state without coordination; operational transformation, used by Google Docs, instead transforms concurrent operations against each other through a central server. They matter whenever two users edit the same object at once and last-write-wins would silently discard work. Libraries such as Yjs and Automerge implement CRDTs, and the WebSocket layer of this chapter merely transports their updates. They apply to collaborative editors, whiteboards and offline-first applications that synchronize later.

Presence

Presence tracks which users are online and often what they are viewing or typing. It is usually built from heartbeats and time-to-live entries, for example a Redis key per user that expires unless refreshed, combined with pub/sub notifications when the state changes. The difficulty is volume: presence changes constantly and fans out to everyone who can see the user, so systems batch, throttle or scope the updates, for example to the members of the open board. It applies to chat, collaborative tools and multiplayer applications.

Reconnection backoff with jitter and connection draining

Clients that reconnect immediately, or at a fixed interval, after a mass disconnection synchronize into waves that hit the servers together. Exponential backoff with random jitter spreads reconnections over time, and servers can drain connections gradually during deploys, closing them in batches with the WebSocket close code 1001 (going away) instead of all at once (see Chapter 15). Combined with cheap reconnection, this keeps storms within capacity. It applies to every client of a persistent-connection service.

Backpressure and slow consumers

A server writing to thousands of sockets will meet clients that read more slowly than events are produced, such as a phone on a weak network or a suspended tab. Without limits, each such connection's outbound buffer grows until memory runs out, undoing the per-connection budget of Section 6. Production servers bound every connection's send queue and, when it fills, coalesce messages (for instance into a single "reload the board" event) or disconnect the client, which then catches up by sequence number. This is essential wherever fan-out is large or traffic is bursty.

Authenticating long-lived connections

Neither the browser WebSocket API nor `EventSource` can set custom headers such as `Authorization`, so tokens travel in a cookie, in a query parameter (which may end up in logs) or in the first message after the handshake. Because cookies are sent automatically, the server must check the `Origin` header of the handshake to prevent cross-site WebSocket hijacking. A connection may also outlive the token that opened it, so the server must re-validate or close connections when tokens expire or permissions are revoked (see Chapter 4). This applies to every authenticated real-time endpoint.

Proxies, load balancers and idle timeouts

Every hop between client and server must support long-lived and upgraded connections. A reverse proxy such as nginx forwards WebSockets only when configured to pass the `Upgrade` and `Connection` headers over HTTP/1.1, closes connections idle longer than its read timeout (60 s by default), and must not buffer SSE responses; cloud load balancers have idle timeouts of their own. Heartbeats must therefore be more frequent than the shortest idle timeout on the path. This applies whenever a real-time endpoint sits behind a proxy, CDN or load balancer.

HTTP/2 and the browser connection limit

Over HTTP/1.1 browsers allow only about six simultaneous connections per origin, and each open SSE stream occupies one, so a user with several tabs can exhaust the limit and stall ordinary requests. HTTP/2 and HTTP/3 multiplex

many streams over one connection and remove this ceiling for SSE. WebSockets can also run over a single HTTP/2 stream through the extended CONNECT method of RFC 8441. This matters when choosing SSE for multi-tab applications and when configuring proxies for real-time traffic.

Event-loop servers and the C10K problem

The C10K problem, posed by Dan Kegel in 1999, asked how one server could handle ten thousand simultaneous connections. The answer was to stop dedicating a thread to each connection and use readiness notification, epoll on Linux and kqueue on BSD and macOS, so that one thread watches many sockets. Node.js, nginx, Netty and the socket layer of .NET on Linux are built on this model, and the lecture's remark that very large deployments avoid one goroutine per connection is the same idea. It matters when per-connection memory, not CPU, is the binding limit.

SignalR and managed scale-out

Frameworks package the mechanics of this chapter. ASP.NET Core SignalR negotiates the transport, adds hubs and groups for addressing users and boards, and scales out across instances with a Redis backplane, which uses Redis Pub/Sub underneath and inherits its at-most-once behaviour, or with the managed Azure SignalR Service, which holds the client connections itself so that application servers stay nearly stateless. Socket.IO plays the same role in Node.js through its adapters. They apply when a team wants real-time features without implementing frames, heartbeats and fan-out itself.

Testing for Backend Engineers — One-Page Overview

Chapter 23 · every section and sub-section of the paper at a glance

1 Introduction: a test is a second program

Hurried tests breed false confidence; good tests exist above all to catch regressions.

- **1.1 What a test is** — A program that runs yours on fixed inputs and checks claims.
- **1.2 Scope** — Tests a backend engineer writes and runs: about 80% of the total.
- **1.3 The example codebase** — The Go task board; bus and store are swappable interfaces.

2 Anatomy of a test

A runner generates a main, builds and runs it; assertions and CI turn tests into a regression net.

- **2.1 A test is a program** — An `if` that fails on a wrong result is already a test.
- **2.2 The test runner from the inside** — `go test` writes `_testmain.go`, builds `board.test`, runs it.
- **2.3 Assertions** — Checks that fail the test; messages must say what broke.
- **2.4 Regressions and continuous integration** — The suite runs on every pull request.

3 Kinds of tests and an enforceable model

Scope names are vague; classifying tests by what they may touch is checkable.

- **3.1 Unit tests and the undefined unit** — Cost comes from leaving the process, not from logic.
- **3.2 Integration and end-to-end tests** — Parts together versus a full user journey.
- **3.3 The testing pyramid** — Cheap fast tests at the base, slow costly ones on top.
- **3.4 Small, medium and large** — One process, one machine, anything.
- **3.5 Purposes are not scopes** — Regression, functional, performance, security.

4 Test doubles

Stand-ins for real dependencies, and the two classic mistakes made with them.

- **4.1 The obstacle: real dependencies** — Database, payments and email block a handler test.
- **4.2 Five kinds of test double** — Dummy, stub, spy, mock, fake.
- **4.3 State versus interaction testing** — Assert what the system is, not how it got there.
- **4.4 Don't mock what you don't own** — Own a wrapper; check it against the sandbox.

5 Designing code that can be tested

Inject dependencies and keep logic pure inside a thin layer of I/O.

- **5.1 Dependency injection** — Pass dependencies in instead of creating them.
- **5.2 Functional core, imperative shell** — Pure core is easy to test; the shell is thin.

6 Databases, networks and time

Real dependencies made cheap and deterministic keep tests faithful and hermetic.

- **6.1 Why in-memory database fakes mislead** — Data bugs live in the layer the fake replaces.
- **6.2 Transaction per test and template databases** — Roll back, or copy a migrated template.
- **6.3 Testcontainers** — Postgres 18: ~7 s once, nine tests in ~3 s.
- **6.4 Networks** — `httptest` real local server; MSW in Node.js.
- **6.5 Time** — Wait for a signal, not a sleep; inject the clock.
- **6.6 Hermetic tests** — The test starts and stops all it needs.

7 Test-driven development

A red-green-refactor workflow, strongest when the output is clear and the method is not.

- **7.1 Red, green, refactor** — Failing test, minimal code, clean-up.
- **7.2 What each stage buys** — Caller-side design, proof the test can fail.
- **7.3 A worked example** — Scheduled may not jump to done; `canMove`.
- **7.4 The debate** — Beck, Cooper's unit-as-behaviour, DHH.
- **7.5 When TDD fits** — Parsers, pricing rules, state machines.

8 Flaky tests

Tests that pass and fail on the same code: their cost, causes and handling.

- **8.1 Why a flaky test is worse than none** — It teaches the team to rerun, not fix.
- **8.2 Where flakiness comes from** — Async waits, concurrency, test order.
- **8.3 Test order dependency** — Delete or quarantine; never just rerun.

9 Measuring tests: coverage and mutation testing

Coverage shows what never ran; mutation testing shows what the tests really check.

- **9.1 How coverage is measured** — A counter at the start of each basic block.
- **9.2 Line coverage and branch coverage** — One `if`: 100% versus 50%.
- **9.3 What a coverage number says** — Weak link to effectiveness at fixed suite size.
- **9.4 Mutation testing** — 100% coverage, yet a score of only 52.63%.

10 Measuring code: complexity and static tools

Path counts, readability scores and static checks are signals, not rules.

- **10.1 Cyclomatic complexity** — 1 + decision points; McCabe's limit of 10.
- **10.2 Cognitive complexity** — Flat switch cheap, nesting expensive.
- **10.3 Both metrics on the task board** — Hub `Run`: 8 versus 19.
- **10.4 Linters and formatters** — Patterns and layout, not correctness.
- **10.5 Type checkers and static analysis** — Type misuse; values traced statically.

Testing at a glance — a test is a second program that keeps fixed bugs fixed; the lecture moves from runners and test sizes through test doubles, testable design and hermetic real dependencies to TDD, flaky tests, coverage, mutation testing and code metrics. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Testing for Backend Engineers: Mocks, TDD and Coverage

Chapter 23 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — A test is a second program that runs the application and checks claims about what happened; its main value is catching regressions before they reach users. The chapter opens a Go test runner to show what it generates, classifies tests by scope and, more usefully, by what they may touch, and catalogues the five kinds of test double with the rules of state over interaction and of not mocking what one does not own. Dependency injection and a functional core inside an imperative shell make code testable, while per-test transactions, template databases, Testcontainers, loopback HTTP servers and injected clocks keep tests real yet hermetic. Test-driven development is presented as a red-green-refactor workflow with a worked example and its controversies, followed by the causes and handling of flaky tests. The chapter ends with measurement: line and branch coverage and the research on their meaning, mutation testing, cyclomatic and cognitive complexity, and static tools.

Keywords — test runner, test doubles, dependency injection, hermetic tests, Testcontainers, test-driven development, flaky tests, code coverage, mutation testing

1 INTRODUCTION: A TEST IS A SECOND PROGRAM

Twenty-five lectures into the course, not a single test had been written, and deliberately so. A test written in a hurry, an incomplete test, or a test that checks the wrong thing is more harmful than no test at all, because it creates a *false sense of confidence*: the suite is green, so the team believes a behaviour is protected when nothing actually guards it. Adding a little testing to each earlier lecture, where the concepts themselves mattered, would have produced exactly such tests, so testing is treated here as a subject of its own and justified step by step.

1.1 What a test is

> **Test.** A second program that runs your program, or a part of it, on inputs fixed in advance and checks claims about what happened; when a claim does not hold, it reports a failure.

There are several motives for writing tests. In *test-driven development* (TDD, Section 7) the test is written first and serves as the goal the code must reach; tests also act as guidance and as documentation of how code is meant to be used. The most important motive, however, is detecting **regressions**: logic written months ago must keep behaving as it did when it was first checked by hand, while new features are added around it. Every kind of test, metric and tool in this chapter is a variation on the definition above. Testing is also an industry of its own, with people and positions dedicated entirely to it and its sub-types.

1.2 Scope

The chapter covers the testing that a backend engineer writes and runs personally, which the lecturer estimates at about 80% of the testing in a typical team. Corporate manual QA processes, test management systems, mobile testing and browser automation for frontends are out of scope. The examples are in Go, but no Go is required: the theory of testing is small, and the hard part is thinking in tests in code, so code is shown wherever it lets the idea transfer to another language.

1.3 The example codebase

The system under test is the real-time task board of Chapter 22, a server of about eleven Go files (Figure 1). `cmd/board` builds the executable, and its `main` is the only place that knows how to assemble the parts. `internal/board` is the domain (boards, columns, tasks); `internal/hub` holds the rooms, in which each subscriber keeps its own counter. `internal/bus` is an interface with two methods and two implementations: one keeps everything inside a single process and is used for tests, the other uses Redis in the real application. This is one of the main uses of interfaces: one API, interchangeable implementations behind it. `internal/store` follows the same pattern, an interface of three methods with Postgres behind it, and `internal/http/api` holds the entire route table, so every exposed endpoint can be read in one file. At the start, the codebase contains no tests.

```
cmd/board/          main: the only place that wires it all
internal/board/     domain: boards, columns, tasks
internal/hub/       rooms; a counter per subscriber
internal/bus/       interface with two methods:
                      in-process implementation (tests)
                      Redis implementation (production)
internal/store/     interface with three methods: Postgres
internal/http/api/  the whole route table in one file
```

Figure 1: Layout of the task-board server as described in the lecture (illustrative)

2 ANATOMY OF A TEST

2.1 A test is a program

Stripped of all tooling, a test is code that calls existing code with predefined input and inspects the output. Without any library or framework, one can write an ordinary function in the backend's own language that calls part of the backend and contains a condition such as "if the result is not 20, fail" — by returning an error, throwing an ex-

ception or panicking, according to the language. That function is already a test. As backends and real applications grew more complex, writing one's own conditions, runner and assertion logic for every program stopped scaling, and testing libraries exist for the reason any library exists: to solve the problem once. Every language ships a built-in mechanism (Go has the `testing` package and `go test`), and further libraries and frameworks make it more convenient.

2.2 The test runner from the inside

What a language or framework provides first is a **test runner**: a program that finds all test files in the codebase (in Go, files ending in `_test.go`), runs them and reports how many passed and how many failed. The first test of the board creates a board, moves a task from one column to another and checks that it landed where it was sent (Figure 2). A deliberate mistake makes it fail first; the output gives the file, the line, and the expected value next to the received one — the task was "in progress", the test expected "done". With the mistake corrected, the test passes instantly.

```
func TestMoveTask(t *testing.T) {
    b := board.New()
    task := b.Add("write tests", board.Todo)
    b.Move(task.ID, board.InProgress) // planted mistake
    if got := b.ColumnOf(task.ID); got != board.Done {
        t.Errorf("column = %q, want %q", got, board.Done)
    }
}

$ go test ./internal/board
--- FAIL: TestMoveTask (0.00s)
    board_test.go:7: column = "in_progress", want "done"
FAIL
```

Figure 2: The first test and its deliberate failure: file, line, received and expected values (illustrative)

Running it with `-work` makes the tool keep its temporary work directory instead of deleting it, and print its path. Inside is a file nobody in the project wrote, `_testmain.go`, generated by the tool (Figure 3). It declares `package main`, holds a list `tests` with every test function found in the package, and ends with a `main` function that calls `testing.MainStart` and then `os.Exit`. Running again with `-x` prints every command the tool executes; the last ones build a binary named `board.test` and run it. So `go test` writes a main function, compiles it with the tests into a program, and executes that program. This is what a runner is in any language: it supplies the entry point of an executable that runs the test functions and collects their results. Interpreted languages skip the compile step, but their runners play the same role.

```
$ go test -work ./internal/board
WORK=/tmp/go-build2215384047
ok      example.com/board/internal/board    0.004s

# $WORK/b001/_testmain.go, generated (simplified)
package main

import (
    "os"
    "testing"

    _test "example.com/board/internal/board"
)

var tests = []testing.InternalTest{
    {"TestMoveTask", _test.TestMoveTask},
}

func main() {
    m := testing.MainStart(deps, tests, nil, nil, nil)
    os.Exit(m.Run())
}

$ go test -x ./internal/board       # last commands
.../link -o $WORK/b001/board.test ...
$WORK/b001/board.test -test.timeout=10m0s
```

Figure 3: What `go test` does under the hood: a generated main, compiled into `board.test` and executed (illustrative, simplified)

2.3 Assertions

The part of a test that "complains" is the **assertion**: a check that fails the test, or stops it, when its condition does not hold. Most languages use an assertion library offering functions such as `assertEquals`, `assertNotEquals` or numeric checks. Go's standard library has no assert function: the check is an ordinary `if`, and when the condition is false the test calls `t.Errorf` with a message the author writes (`t.Errorf` marks the test failed and continues; `t.Fatalf` also stops it). Go thus forces a message on every check; in other languages the author must make sure each assertion carries a meaningful one, so that whoever sees the failure knows what to check, what broke, and has enough information to debug it.

2.4 Regressions and continuous integration

A regression is a bug that was fixed in the past and comes back, typically because someone on the team changed a feature, an interface or some behaviour — an entirely normal event in team work. Tests catch regressions immediately when they run in continuous integration: with GitHub Actions, for instance, a workflow runs the entire suite on every pull request before it is merged into the main or development branch, and a single failure shows that the new code reintroduces a bug that was already fixed. The fix happens before users complain about broken features. This is the most useful property of a test suite: a deterministic set of tests covering every condition and every error seen in the past lets a team add features quickly without fear of breaking existing ones, because fixed bugs stay fixed (CI pipelines are treated in Chapter 27).

3 KINDS OF TESTS AND AN ENFORCEABLE MODEL

3.1 Unit tests and the undefined unit

A **unit test** tests one piece of code in isolation, regardless of the code around it. What counts as a unit — a function, a struct, a package, a whole service — has no universal answer; one learns to recognise it by writing many tests. A function that performs its own logic and calls two or three helpers can be tested either by letting it call the real helpers or by passing doubles in their place, and both count as unit tests.

The lecturer tests the same behaviour, moving a task, in four ways (Table 1). In the first, the board receives, instead of the real bus, a small object that only records what it was asked to publish and does nothing else (a spy, in the terms of Section 4.2). The second uses the real in-process bus and the real hub, so the move actually reaches a subscriber. The third starts a real HTTP server on a real port, sends a real `POST` and reads the real response. The fourth runs the same check against a real Postgres database. The first three report 0.0 s; the fourth takes 10.5 s, mostly because the Postgres container has to start. The cost of a test therefore does not come from how much logic it covers — the third test crossed the whole HTTP stack and still reported zero — but from whether it goes beyond its own process.

Variant	What is real	Time
1. Recording double	board logic only	0.0 s
2. Bus and hub	in-process delivery to a subscriber	0.0 s
3. HTTP server	port, `POST`, response	0.0 s
4. Postgres	database in a container	10.5 s

Table 1: One behaviour, moving a task, tested four ways (lecture demo)

3.2 Integration and end-to-end tests

An **integration test** checks two or more parts working together. Usually one part is the application's own logic and the others are external, something the team did not write, such as a database. An **end-to-end (E2E) test** drives the entire system as a real user would, along every step of the user's path. Since the first thing a user does is authenticate, an E2E test begins by logging in, keeps the resulting token or session, and uses it for everything that follows — opening the dashboard, calling an API, creating a task — so that it traces the user's journey through the platform. An integration test need not do this: the authentication middleware is the team's own code (see Chapter 4), so it can be bypassed and the test can create a task directly. Integration tests do not follow every path a user takes.

3.3 The testing pyramid

The well-known *testing pyramid* has unit tests as its broad base, integration tests in the middle and E2E tests at the top. Its subject is cost: the wide lower levels are extremely cheap and fast, and each level up is slower and more expensive, where "expensive" means CPU, memory or the developers' time rather than only money. For many years the pyramid has framed discussions, and controversies, about how many tests to write and which parts to test.

3.4 Small, medium and large: what a test may touch

The lecturer prefers a practical model: instead of asking which part of the system a test covers, ask what the test is **allowed to access** (Table 2). A *small* test runs in a single process; it uses no network, no database and no file system, and contains no artificial pauses. A *medium* test may use anything on one machine — a database in a local container, Redis in another — but never reaches the internet or leaves the machine. A *large* test may do anything. (Google classifies its tests the same way; see Winters et al.)

Size	May use	May not use
Small	one process, memory	network, database, file system, sleeps
Medium	one machine: local containers (Postgres, Redis)	the internet, other machines
Large	anything	—

Table 2: Test sizes defined by what a test may touch

The model wins because it can be enforced. Nobody can prove that a test is a "unit" test, since there is no agreed definition and every engineer has an opinion; but the machine running the test can check whether it opened a socket, which turns a subjective argument into a rule that can be applied. It also addresses **flaky tests**, tests that sometimes pass and sometimes fail depending on the environment. A test kept inside its process or machine can be made as deterministic as possible, because much of the instability comes from the internet: external services, servers with their own clocks, and every complication they bring.

> **A subtle point.** The lecturer says this model can eliminate all flakiness. It removes the network-related sources, but not the most common ones: waiting wrongly for asynchronous work, races between concurrent code and dependence on test order all occur inside a single process (Section 8.2). Small tests are more deterministic, not automatically deterministic.

3.5 Purposes are not scopes

Unit, integration and E2E answer one question: how much of the system does the test cover? Other names answer other questions. *Regression tests* target a specific list of bugs that have already happened and must not recur. *Functional tests* check a feature against its specification; the lecturer links them to TDD, where the specification is written first as a test, so that when the work is complete a ready-made check says whether it is finished. *Performance tests* measure how fast the program runs and how much memory and CPU time it consumes. *Security tests* check whether the application bypasses a layer of protection — a sandboxed container, the file system, cookies — and whether it respects privacy. These are variations, not separate types: a regression test can be a unit test, a security test that checks for privacy violations can be an E2E test, and

functional testing is, in the lecturer's view, a variation of integration testing. The terms are worth knowing because the industry uses them, but the lecturer reasons only about what a test interacts with, and the rest of the chapter does the same.

4 TEST DOUBLES

4.1 The obstacle: real dependencies

Why do so many people write no tests at all? On the task board, when someone moves a task, the handler asks the board to apply the change and then publishes an event to the event bus, so that every other connected browser learns of the move (the propagation path of Chapter 22). In a real backend the same handler would also write to Postgres, perhaps call a payment service to verify funds, and send an email through yet another external service. The question is how to test such a workflow without a real database, a real payment provider and real emails to users. The answer is to give the code something that looks and acts like the external service: a **test double**. The name comes from film, where a stunt double performs the risky stunts while the audience believes it is watching the actor.

4.2 Five kinds of test double

"Test double" names a category of five kinds (Table 3), each with a different job, although almost everyone calls all of them "mocks". A **dummy** is a value passed only because the signature requires it and never actually used: a service constructor that demands a logger from `main` receives one that does nothing, because logging is irrelevant to the test. A **stub** returns the same prepared answer however often and whatever it is asked: a query for user 7 returns an object shaped like the stored row, with similar fields, so the test can proceed without real data. A **spy** answers like a stub but also records what is done to it: asked for user 7 it returns user 7, and if the code renames the user from X to Y, the spy records that attempt. A **mock** is configured with expectations in advance — for example, that `Save` is called exactly once with a given argument — and fails the test if they are not met. A **fake** is a working implementation unfit for production, existing only for tests and matching the real one's inputs and outputs: instead of saving a user in Postgres, a fake user store keeps it in an in-memory map or set, so from the caller's point of view the user is saved and can be read back.

Double	Behaviour	Example from the lecture
Dummy	fills a parameter; never used	no-op logger for a constructor
Stub	same canned answer every time	user 7 for any lookup
Spy	stub that records calls	records the rename from X to Y
Mock	preset expectations; fails if unmet	`Save` called exactly once
Fake	working, not production-grade	in-memory map instead of Postgres

Table 3: The five kinds of test double

4.3 State testing versus interaction testing

A test of the move handler can end in two ways (Figure 4). **State testing** asks the system what is true now: is the task in the done column, did the sequence number increase by one? **Interaction testing**, also called behaviour testing, asks the double what it saw: was `Publish` called, how often, with a task event? Suppose someone changes the handler to publish one merged event instead of two. Nothing changes on the outside; every browser still sees the same board. The state test does not notice, because the final state is identical; the interaction test fails, because the number of calls changed. A test that breaks on such harmless changes is a **change-detector test** (Google's testing blog has an article on them): it checks the program's step-by-step behaviour instead of the result it achieves. The rule is to assert what the system is, not how it got there, and to check interactions only when the interaction is itself the requirement — for example, that an email to a user is sent exactly once.

```
// State check: ask the board what is true now
if got := b.ColumnOf(id); got != board.Done {
    t.Errorf("column = %q, want %q", got, board.Done)
}
if got := b.Seq(); got != before+1 {
    t.Errorf("seq = %d, want %d", got, before+1)
}

// Interaction check: ask the double what it saw
if n := len(spy.Published); n != 2 {
    t.Errorf("Publish called %d times, want 2", n)
}
// Merge the two events into one: the board is the
// same, the state check stays green, the interaction
// check fails -- a change detector.
```

Figure 4: State check versus interaction check for the move handler (illustrative)

4.4 Don't mock what you don't own

A second common mistake involves third-party services. A service charges cards through a payment provider; nobody wants to debit a real card several times a day, so the team writes a mock of the provider's client: when `Debit` is called with given arguments, it returns a success object with fields such as data, a success message and the new balance. The tests pass, the code is deployed, and production breaks. Everything the test exercised had been written by the team: the shape of the success object, the field names, the behaviour on retry, the errors and their format. The rule is **don't mock what you don't own**, and two practices replace it (Figure 5). First, wrap the third-party system in a small interface — a wrapper function, struct or class, depending on the language — that speaks only the application's own types: its own success objects, error objects and error messages. The wrapper is used in the production workflow and in tests alike, and because the team owns that interface, it may replace it with doubles. Second, since the full charge flow can be checked by hand once but not on every run, use the sandbox APIs most payment providers offer for a few small tests whose only job is to verify that the request, response, success and error structures as-

sumed by the wrapper match the real platform. These tests are fast and involve no real payment details.

```
// Our types: the only payment types the app sees
type Charge struct {
    ID      string
    Balance int64 // cents
}

var ErrDeclined = errors.New("payments: declined")

type Payments interface {
    Debit(ctx context.Context, card string,
        cents int64) (Charge, error)
}

// production  vendorPayments wraps the vendor SDK
//             and translates its replies and errors
// unit tests  a stub or fake of Payments (we own it)
// contract    a few tests run vendorPayments against
//             the vendor's sandbox API
```

Figure 5: Owning the payment interface: a wrapper expressed in the application's own types (illustrative)

Warning. A mock of a client you do not own returns whatever you told it to return. A green suite built on it proves only that your code agrees with your own assumptions about the provider, and those assumptions are exactly what fails in production.

5 DESIGNING CODE THAT CAN BE TESTED

5.1 Dependency injection

The move handler was hard to test because it created its dependencies "out of thin air": it constructed the database client and the Redis client itself and opened the connections itself. A function that builds what it depends on leaves no way to replace those dependencies from outside. The remedy is to pass them in, as parameters or constructor arguments: **dependency injection** (Figure 6; the handler, service and repository layers are those of Chapter 6). Many dependency-injection frameworks exist and almost all do the same thing; as a backend grows they make the wiring more convenient, but the technique needs no framework. In the board codebase, `main` is the single place where the real implementations are assembled, and tests assemble in-process or test implementations instead.

```
// Before: the handler builds its own dependencies
func (h *Handler) Move(w http.ResponseWriter,
    r *http.Request) {
    db, _ := sql.Open("pgx", dsn)
    rdb := redis.NewClient(opts)
    // ... nothing here can be replaced from outside
}

// After: dependencies are injected
type Handler struct {
    board *board.Board
    store store.Store // Postgres, or a tx in tests
    bus   bus.Bus     // Redis, or in-process in tests
}

func NewHandler(b *board.Board, s store.Store,
    bs bus.Bus) *Handler {
    return &Handler{board: b, store: s, bus: bs}
}
```

Figure 6: Dependency injection in the move handler (illustrative)

5.2 Functional core, imperative shell

The mental model that goes with injection is to move every decision, calculation and business rule — the main body of the logic — into separate *pure* functions that take values and return values, and to keep all communication with the outside world (the internet, external APIs, providers, databases) in a very thin layer around them. The inner part is the **functional core**, the layer around it the **imperative shell** (a name due to Gary Bernhardt). The core is easy to test because nothing in it needs to be replaced: everything there belongs to the application. The shell is hard to test, but it is so thin that it holds almost nothing worth testing. Code structured this way is easier to test and, as a result, more likely to be tested; Section 7.3 extracts a business rule into exactly such a function.

6 DATABASES, NETWORKS AND TIME

6.1 Why in-memory database fakes mislead

Once dependencies can be injected, the question is which of them to replace with test implementations, and when that is right. The commonest case is the database. Most people instinctively reason that the database is slow, so they wrap the Postgres layer with something that performs every operation in memory with structs and maps; the tests lose their external dependency and run in seconds. In Node.js this is often done by swapping the data layer in memory, in Go by writing fake repositories, and in Java with H2, an in-memory database. The gain is speed, ease of maintenance and efficiency; the price is **fidelity**. The bugs that live at the data level are a wrong column name, a missing index that slows a query, a unique constraint that was forgotten, a transaction that does not behave as expected (see Chapter 8), or a JSON operator that exists in Postgres but not in the fake, or the reverse. Every one of them lives in Postgres, not in the application logic, and Postgres is precisely the layer the in-memory fake replaced. The answer is to test against a real database and to make that convenient enough that nobody is tempted to swap it out.

6.2 Transaction per test and template databases

Two techniques make a real database practical. With a **transaction per test**, each test opens a transaction, runs everything inside it and rolls back instead of committing: the database is left intact, the next test starts from a clean slate, no test corrupts another's data, and it is very fast. Its one requirement is that the code under test accepts a transaction handle instead of a database handle — dependency injection again, since functions must receive their dependencies rather than create them. (A rolled-back test never exercises what happens only at commit, such as deferred constraints, and code that opens its own transactions needs savepoints or the next technique.) With a **template database**, the suite uses Postgres's ability to create a database as a copy of an existing one (Figure 7): all migrations run once on a template, and each test creates its own database from it in a few milliseconds. The latency is prac-

tically unnoticeable, and because every test has its own database, tests can run in parallel. Postgres copies a template only while no other session is connected to it.

6.3 Testcontainers

To obtain a real Postgres instance in tests, the well-known **Testcontainers** library, with SDKs for almost every language and modules for Postgres and many other databases, starts a real container from the test code and returns the address at which the database can be reached. In the demo, Testcontainers starts a real Postgres 18 and waits until it accepts connections. The timestamps show the container starting at second 58 of one minute and ready at second 5 of the next: about 7 seconds, paid once for the whole package rather than per test. Each of the nine tests then creates its own database by copying the template and runs in milliseconds. The whole package takes 10.5 seconds including start-up: about 7 seconds to set up a real database and about 3 seconds for all the tests.

```
-- once per package, after the container is ready
CREATE DATABASE board_template;
-- ... run every migration against board_template ...

-- once per test: a few milliseconds
CREATE DATABASE test_042 TEMPLATE board_template;

Demo timeline, one package, nine tests:
  ..:58     Testcontainers starts postgres:18
  ..:05     ready to accept connections (~7 s, once)
  then      9 tests, each on its own copy (ms each)
  total     10.5 s = ~7 s container + ~3 s tests
```

Figure 7: Template databases and the Testcontainers demo timeline (SQL illustrative; timings from the lecture)

6.4 Networks

When the application calls someone else's API, the first method is the one of Section 4.4: wrap the external service in an interface of one's own and replace that in tests. A second method applies when the HTTP exchange itself is under test: one's own client code, the headers it sets and removes, and the fact that the client really talks HTTP to a server instead of calling functions in-process. Go's `net/http/httptest` package starts a real HTTP server on a real local port during the test, and the client is pointed at it, so each request travels over TCP exactly as in production; Node.js and Python have well-supported libraries for the same purpose. In Node.js, **MSW** (Mock Service Worker) intercepts requests at the network level, so the client need not be replaced at all. The same setup lets one test one's own handlers from the other side, as a client would call them.

6.5 Time

Time causes a great many problems. A test calls a function that starts asynchronous work in the background; since the test must wait for it, it sleeps — say 100 ms — and only then checks the result. The test passes on the laptop and in code review, then fails in CI, for instance on GitHub Actions, because on a heavily loaded machine the work took 110 ms instead of 100. The logic was correct; the clock was the problem. The obvious fix, a longer sleep, is no fix: the tests become slower, and on a busier day the value must be raised again. There are two correct solutions (Figure 8). The first is to **wait for a condition, not a duration**: wait for a specific signal that the work has completed — an event on a channel in Go, a callback in Node.js — so that nothing is guessed. The second is to **inject the clock**: many languages and libraries let code take a clock instance instead of reading real time, which gives the test full control over time and timeouts.

```
// Flaky: guesses how long the background work takes
svc.MoveAsync(id, board.Done)
time.Sleep(100 * time.Millisecond) // 110 ms on busy CI
assertDone(t, b, id)

// Deterministic: waits for the completion signal
svc.MoveAsync(id, board.Done)
select {
case <-svc.Moved: // a signal, not a guess
    assertDone(t, b, id)
case <-time.After(5 * time.Second): // upper bound
    t.Fatal("move never completed")
}

// Injected clock: the test decides what "now" is
clk := fakeclock.New(start)
s := session.New(clk, 30*time.Minute)
clk.Advance(31 * time.Minute) // no real waiting
if !s.Expired() {
    t.Error("session should expire after 30 min")
}
```

Figure 8: Replacing a sleep with a signal, and injecting a clock (illustrative)

6.6 Hermetic tests

> **Hermetic test.** A test that carries its own environment: it depends on no manually configured database or hand-started service, starts everything it needs, and stops what it started when it finishes. "Hermetic" means sealed or insulated.

Hermeticity is what lets the same tests run on any machine — a local computer, a server reached over SSH, a GitHub Actions runner. Testcontainers is a way to make the database hermetic, and clock injection a way to make time hermetic.

7 TEST-DRIVEN DEVELOPMENT

7.1 Red, green, refactor

Test-driven development is the practice of writing the test, the specification, before the code. It is a workflow, not a type of test: it uses ordinary unit, integration and E2E tests and differs only in when they are written — before the application logic instead of after it. The cycle has three stages, named by colour. **Red**: write a test for something that does not exist yet and run it; it must fail. **Green**: write the minimum code that makes the test pass; at this stage bad code that ignores best practices is expected and allowed, because the only goal is a passing test. **Refactor**: with the test green, clean up and restructure, then run the test again; if it still passes, the result is clean code with tested behaviour.

7.2 What each stage buys

Red is first a **design** step. Writing the test before the logic forces the author to imagine using the feature before it exists — to see it as an API, how it will be called, not how it works inside. If the test becomes hard to write, ugly or clumsy, the imagined interface has a flaw, and this is discovered in seconds or minutes instead of at the end, when the function must be reworked and given new parameters. Red is also a **verification** step. A test written after the code by the person who wrote the code will most likely pass on the first try and teach nothing: there is no way to tell whether it passes because the code is correct or because it does not check what it is supposed to check. Seeing it red first, then green, shows that the test can fail; this is one of the properties Kent Beck lists for a good test (his "Test Desiderata"). The third stage is the one most people skip, and it cannot be skipped: the licence to write bad code in the green stage exists only because the refactor stage follows.

7.3 A worked example

The requirement: a task may not move directly from "scheduled" to "done"; it must pass through "in progress", otherwise there is no point in keeping an in-progress status. Nothing in the code enforces this yet (Figure 9). *Red*: the test takes a task in "scheduled", moves it to "done" and expects an error. It fails with "expected an error, got nil", because the move succeeded — the rule does not exist. *Green*: one `if` statement inside `Move`, and the test passes. *Refactor*: the `if` is a business rule sitting in the middle of a method that also locks a mutex, increments the sequence number and clears the history buffer. It is extracted into a function `canMove` that takes the column the task is in and the column it is going to, and nothing else. The whole package is run again and stays green: the code was improved after the green stage and the tests still pass, which completes the cycle. The extracted function belongs to the functional core of Section 5.2.

```
// RED: a test for a rule that does not exist yet
func TestScheduledCannotSkipToDone(t *testing.T) {
    b := board.New()
    task := b.Add("ship it", board.Scheduled)
    if err := b.Move(task.ID, board.Done); err == nil {
        t.Fatal("expected an error, got nil")
    }
}
// go test: FAIL, expected an error, got nil

// GREEN: one if inside Move  ->  ok

// REFACTOR: the rule leaves Move, which also locks
// the mutex, bumps the sequence number and clears
// the history buffer
func canMove(from, to Column) bool {
    return !(from == Scheduled && to == Done)
}
// whole package re-run  ->  ok
```

Figure 9: The TDD cycle for the scheduled-to-done rule (illustrative)

7.4 The debate

TDD is surrounded by controversy. In 2023 Kent Beck, who wrote the book on TDD, published a short article, "Canon TDD", on what the cycle really means. Ian Cooper's talk "TDD, Where Did It All Go Wrong" argues that people read the word "unit" as "class": they wrote one test class per production class and mocked everything else, every collaborator and external integration, so that any refactoring broke hundreds of tests, and many developers abandoned TDD because of it. For Cooper a unit is a *behaviour*: tests exercise the public interface of a module, so the whole class or module can be refactored without changing that interface while the tests stay unchanged and green. This is the state-over-interaction rule of Section 4.3: a test per class with everything mocked checks the module's implementation rather than the state it produces. In 2014 David Heinemeier Hansson (DHH), creator of Ruby on Rails, published the much-discussed "TDD is dead. Long live testing.", arguing against the test-first culture and against designing systems around mocking, in favour of going straight to testing with real components. It led to a series of public discussions between DHH, Kent Beck and Martin Fowler.

7.5 When TDD fits

The lecturer's position: TDD is a very good tool when you know what the answer should look like but not yet how to build it. A parser is the typical case: a JSON parser takes a string and should return a JSON object, so inputs and outputs are easy to define, and stating the expectations is the easy part compared with the implementation. Pricing rules in business logic and state machines are similar; TDD keeps attention on what the logic must produce. It is a poor tool when the shape of the result is not yet known — while exploring the problem space, trying approaches and weighing what feels right before settling on one. Backend features often look like this, with many trade-offs and system-design aspects to consider, so TDD has real but limited uses.

8 FLAKY TESTS

8.1 Why a flaky test is worse than none

A **flaky test** passes and fails on the same code: nothing changes, yet the result does. Such a test is more harmful than a missing one. A missing test is a known gap. A flaky test changes how the team works: code is pushed, the test fails in CI, someone says "that test is a bit flaky, just rerun it", it passes, and everyone moves on. That becomes a habit, and on the day the test fails because of a real error in the system, the first reaction is to rerun it instead of looking for a fix.

> **Warning.** Every "just rerun it" trains the team to treat a red build as noise. An automatic-rerun policy hides precisely the real failures the suite exists to report.

8.2 Where flakiness comes from

A 2014 study by Luo and colleagues took 201 real fixes of flaky tests from open-source projects and classified them by cause; three causes dominate. **Async wait**: the test starts something and then waits for it the wrong way. **Con-**

currency: two things in the test race each other. **Test order dependency**: the test passes only because another test ran earlier and left some state behind. The first two have already been addressed. Async waits are the sleep problem of Section 6.5, solved by waiting for a condition or signal instead of an amount of time. For concurrency, Go provides `testing/synctest`, and other languages and frameworks have their own primitives for testing concurrent code deterministically (see Chapter 19).

8.3 Test order dependency

Test B passes only when it runs after test A and fails when run alone, usually because A created a database row, set a global variable or wrote a file that B relies on. As long as the suite always runs in the same order, B always passes; as soon as someone adds a test in between, or enables parallelism to speed the suite up, it fails. (Randomising the order, for example with `go test -shuffle=on`, exposes such dependencies early.) For a test known to be flaky there are three options. *Delete* it: perfectly acceptable, since it was not really helping and caused more problems than it solved; deleting tests brings more benefit than one might expect. *Quarantine* it: move it from the main suite into a separate one that still runs, recording the owner's name, the date and the reason, so that the instability is documented. *Rerun* it automatically: bad practice. The lecturer prefers quarantine, because the next time the test fails the team knows who wrote it and why, and can contact that person to find the right fix.

9 MEASURING TESTS: COVERAGE AND MUTATION TESTING

9.1 How coverage is measured

The first metric almost everyone reaches for is **code coverage**: the percentage of the code executed while the tests ran. When coverage is requested, the compiler or tool rewrites the source before building: it splits each function into *basic blocks*, straight runs of code without branches, and inserts a counter at the start of each block (`go test -cover` works this way). It then runs the tests and reads the counters; a block whose counter is zero never ran. The limitation follows directly: coverage records whether code ran, not whether it did any work or whether anything checked the result. A test that calls every function of the application and asserts nothing still yields 100% coverage.

9.2 Line coverage and branch coverage

Coverage is not one number either. **Line coverage** asks whether each line was executed; **branch coverage** asks whether each branch of each decision was taken. The same code gives different answers. Take a function with a single `if` and no `else`: a test in which the condition is true executes every line, so line coverage is 100%, but the false path is never taken, so branch coverage is 50%.

9.3 What a coverage number says

The target most teams agree on is 80%. In 2014 Laura Inozemtseva and Reid Holmes presented a study at the International Conference on Software Engineering: they took large real-world Java programs, generated thousands of test suites of different sizes, and measured each suite's coverage and how many faults it detected. The key is **controlling for suite size**. Correlating coverage with faults found shows a very strong relationship, but mainly because larger suites both cover more code and find more faults. Holding size fixed — asking whether, of two suites of equal size, the one with higher coverage finds more faults — they found only a low to moderate correlation, varying from system to system. A fixed coverage value used as a quality target therefore does not produce an effective suite.

Coverage should not be ignored, though, and three lessons go with it. Guideline values exist — 60% acceptable, 75% commendable, 90% exemplary — but no single number suits every product. What the tests do not cover matters more than what they cover: the value of a coverage report lies in its red lines, not in its percentage. And a mandatory 100% makes most engineers write low-value tests to reach the number and stop as soon as they reach it. The more useful direction is mutation testing.

> **A subtle point.** The lecture merges two sources. Inozemtseva and Holmes measured effectiveness with mutation testing — the share of seeded faults (mutants) a suite detects — not with real bugs. The 60/75/90 guideline, the advice to read the uncovered lines and the warning against a mandated 100% come from Google's 2020 testing-blog post "Code Coverage Best Practices".

9.4 Mutation testing

Mutation testing tests the tests. A tool deliberately breaks the code and checks whether the tests notice; the proportion of breakages detected measures the quality of the tests. The tool makes small changes throughout the source — "greater than" becomes "greater than or equal", a plus becomes a minus, a return value becomes zero — and each damaged version is a **mutant**. The suite is run against every mutant. If a test fails, the mutant is *killed*, which is good: the tests detected the damage. If all tests still pass, the mutant *survives*, meaning the tests do not actually check that behaviour. The **mutation score** is the percentage of mutants killed.

In the demo, a module takes a list and returns one page of it, and someone has written a test whose assertions amount to "it returns something": the result is defined, it is an array, the call does not throw, and its length is greater than zero (Figure 10). Every line of the function runs, so line coverage is 100%, and judged by coverage alone the work is done. The mutation tester Stryker, run on the same code, creates 19 mutants, runs the whole suite against each one in turn, and reports a score of 52.63%: nine of the nineteen mutants survived. The second survivor sits in the

loop condition, where `i <= offset + limit` became `i < offset + limit`. That changes how many elements the function returns, yet every test still passed. Validating the tests themselves is the whole value of mutation testing. Because the suite runs once per mutant, it is expensive, and teams usually run it on changed code or on a schedule rather than on every commit.

```
// the test that gave 100% line coverage
it("returns a page", () => {
  const page = paginate(list, 0, 10);
  expect(page).toBeDefined();
  expect(Array.isArray(page)).toBe(true);
  expect(() => paginate(list, 0, 10)).not.toThrow();
  expect(page.length).toBeGreaterThan(0);
});

// Stryker: 19 mutants, 10 killed, 9 survived
// mutation score = 10 / 19 = 52.63 %
// survivor #2, in the loop condition:
//   original   i <= offset + limit
//   mutant     i <  offset + limit   (all tests pass)
```

Figure 10: A coverage-complete but weak test and its Stryker result (test illustrative; numbers from the lecture)

10 MEASURING CODE: COMPLEXITY AND STATIC TOOLS

10.1 Cyclomatic complexity

Cyclomatic complexity, despite its name, simply counts the independent paths through a piece of code. It comes from a 1976 paper by Thomas McCabe and can be computed by hand: draw the function as a graph in which each straight run of instructions is a node and each transition between runs is an edge; the complexity is *edges − nodes + 2*. A shortcut always gives the same answer: one plus the number of decision points — one for the function itself, plus one for each `if`, each loop, each `case` and each `&&` or `||` in a condition. A function without branches scores 1; a function with one `if` scores 2. The common advice that functions should stay at or below 10 is what McCabe himself wrote in the original paper — a reasonable limit, not a magic number — and he found one case where the limit seemed unreasonable: a function consisting of a large switch with many independent cases. A switch with 20 cases scores 21, yet remains perfectly readable.

10.2 Cognitive complexity

Because such a readable switch fails the "below 10" advice, SonarSource developed a second metric, **cognitive complexity**. It is not a purely mechanical count of paths; it is designed to measure how hard the code is to understand. A flat switch is easy to read, so it adds almost nothing (a single increment for the whole switch, however many cases it has). Nesting, in contrast, raises the score at every level: an `if` inside a loop that is itself inside another `if` is far harder to keep in mind than three `if` statements in a row. A long flat switch therefore scores high on cyclomatic and low on cognitive complexity, while three levels of nesting score moderately on cyclomatic and high on cognitive complexity (Table 4).

Code shape	Cyclomatic	Cognitive
flat switch, 20 cases	21	1
three `if`s in sequence	4	3
`if` in a loop in an `if`	4	6
hub `Run` (demo)	8 (third)	19 (first)

Table 4: Cyclomatic versus cognitive complexity (first three rows illustrative, counted with McCabe's and SonarSource's rules; last row from the lecture)

10.3 Both metrics on the task board

Two tools, one per metric, are run on a package of the board. By cyclomatic complexity, which counts paths, the hub's `Run` method scores 8 and ranks third, behind the two stream handlers. By cognitive complexity, which grows with nesting, the same method scores 19 and tops the list together with the WebSocket handler. Reading the code confirms what lies behind the numbers: there are eight ways through the function, and the deep nesting makes it hard to follow. Neither tool should be trusted blindly or treated as a hard rule; the numbers point at places worth a look, and the team decides what to do. Such metrics are a notification system, not the basis for decisions.

10.4 Linters and formatters

Some tools find problems without running the program at all. A **linter** analyses source code and warns whenever it finds a pattern it considers wrong. It does not check syntax, which is the job of the compiler or runtime, and it does not verify that the logic is correct, for which there is no universal, reliable method short of executing the program against expected results. It only checks patterns: an `if` on a variable that is always true because of how it was defined above, or an error returned by a call that is never checked and should be handled. A **formatter**, the most universally known of these tools, rewrites source code into a standard layout, usually agreed by the whole team, and nothing else; it never changes what the program does, only how readable it is.

10.5 Type checkers and static analysis

Languages such as Go, TypeScript and Rust have **type checkers**, usually integrated into the editor (for example VS Code), which warn when types are used incorrectly: arithmetic on a variable declared somewhere above as a string is reported as an error. **Static analysis** in general means checking code without running it; it is a more advanced linter that starts from some point in the program, tracks values through it and checks conditions along the way (Table 5). Code quality itself is subjective, and the lecturer found no objective metric or tool for it: ESLint rules in the TypeScript ecosystem and Go's own linters help to some extent, but beyond them there is no deterministic way to decide whether code is good.

Tool	Checks	Example
Linter	suspicious patterns	always-true condition; unchecked error
Formatter	layout only	`gofmt`, Prettier
Type checker	use of types	arithmetic on a string
Static analysis	values traced through code	conditions checked along paths

Table 5: Tools that find problems without running the code

11 CONCLUSION

- A test is a second program that runs yours and checks claims; its main job is to catch regressions in CI before merge.
- A hurried, incomplete or flaky test is worse than none: it breeds false confidence or teaches the team to ignore red.
- Classify tests by what they may touch — one process, one machine, anything — because cost comes from leaving the process, not from the logic covered.
- Know the five doubles; assert state rather than interactions, and never mock a client you do not own: wrap it and check the wrapper against the sandbox.
- Dependency injection and a functional core inside a thin imperative shell make code testable.
- Test against a real database made cheap — transaction per test, template databases, Testcontainers — and replace sleeps with signals and injected clocks, so tests stay hermetic.
- TDD is a red-green-refactor workflow, strongest when the expected output is clear; a "unit" is a behaviour, not a class.
- Coverage shows what never ran, not what is checked; mutation testing measures whether tests detect breakage.
- Complexity metrics, linters, formatters and type checkers are signals for the team to judge, not rules.

APPENDIX A REVIEW QUESTIONS

1. A team's 600-test suite shares one long-lived Postgres database that was configured by hand on the CI runner. The suite takes 14 minutes, passes in its default order, and fails at random as soon as parallel execution is enabled. Container start-up costs about 7 s and copying a migrated template database about 20 ms. Redesign the suite so that it runs in parallel on any machine, estimate the new set-up overhead, and explain which failures your design removes, which it cannot remove, and why an in-memory fake repository would not be an acceptable shortcut.
2. A test publishes a board event, sleeps 100 ms and then asserts that a subscriber received it. It passed on three laptops and in review, but fails in CI about once in fifteen runs with "expected 1 event, got 0". A colleague proposes raising the sleep to 500 ms and enabling automatic retries. Diagnose the failure in terms of the known causes of flakiness, argue why the proposal makes the suite worse in two distinct ways, and give a replacement design that contains no sleep at all.
3. The move handler calls `bus.Publish` twice per move, once with "task moved" and once with "board updated". Test A asserts that the task ends in the done column and that the sequence number rose by one; test B asserts on a spy that `Publish` was called exactly twice with those event kinds. A colleague merges both events into one payload that every browser renders identically. Predict the outcome of each test, name the kind of test that misbehaves, and describe a product requirement under which an interaction assertion like B's would nevertheless be justified.
4. A checkout service's tests mock the payment vendor's client so that `Debit` returns `{"data": …, "success_message": "ok", "new_balance": 1200}`. After deployment every charge is recorded as failed although customers are billed, because the vendor actually reports success in a differently named status field and answers retried requests with HTTP 409. Explain why the suite was green, then design the replacement: the interfaces involved, which tests are small, medium or large, and what runs against the vendor's sandbox and how often. Relate the retry behaviour to the retry handling of Chapter 12.
5. A function begins with `if amount > limit { return ErrOverLimit }`, followed by two lines that apply the charge, and its only test calls it with amount 50 and limit 100 and asserts that no error is returned. State its line coverage and branch coverage. Decide which of the mutants "`>` becomes `>=`", "`>` becomes `<`" and "return `nil` instead of `ErrOverLimit`" survive, and design the smallest set of extra test cases that kills all three, justifying each boundary value.
6. Function F is a flat `switch` over 14 event kinds with one line per case. Function G loops over subscribers; inside the loop an `if` tests `sub.Active && sub.Room == room`, and inside that an `if` checks whether the send buffer is full. Compute the cyclomatic complexity of both with the decision-point shortcut, rank them by cognitive complexity with a justification, and decide which one a reviewer enforcing "cyclomatic at most 10" would flag and which one actually deserves refactoring, proposing the refactoring.
7. Two tickets arrive: (a) a late-fee rule — 2% per started week overdue, capped at 20%, waived when the task is closed within 24 hours of its deadline; (b) deciding how the real-time board should distribute rooms across several hub instances (see Chapter 22). Argue for which ticket TDD pays off. For that ticket, give the first red test, the minimal green change and what the refactor stage should extract; then explain, in Ian Cooper's terms, how a team that writes one fully mocked test class per production class would suffer at the next refactoring.

8. Chapter 15 requires that on SIGTERM the server stops accepting new requests, lets in-flight requests finish within a 10-second budget and then closes its database pool. Design a hermetic, sleep-free test of this behaviour that runs in under one second in CI: state what is real and what is replaced (server, database, clock, in-flight request), how the test learns that each phase has been reached, which size the test is, and which cause of flakiness a naive version built on `time.Sleep` would exhibit.

APPENDIX B GLOSSARY

Term	Meaning
Test runner	Program that discovers tests, runs them and reports results; in Go, a generated main compiled into a test binary
Assertion	Check that fails, or stops, a test when its condition does not hold
Small / medium / large test	Test classified by what it may touch: one process, one machine, or anything
Test double	Any stand-in for a real dependency during a test
Stub	Double that returns the same canned answer to every call
Spy	Stub that also records how it was called
Mock	Double with preset expectations that fails the test when they are not met
Fake	Working but non-production implementation, such as an in-memory store
Change-detector test	Test that breaks on harmless refactorings because it asserts steps, not outcomes
Functional core, imperative shell	Pure logic wrapped in a thin layer that performs all I/O
Hermetic test	Test that starts and stops everything it depends on
Flaky test	Test that both passes and fails on the same code
Branch coverage	Share of decision outcomes (true and false paths) taken by the tests
Mutation score	Share of deliberately injected faults (mutants) the suite detects
Cyclomatic complexity	Number of independent paths: edges − nodes + 2, or 1 + decision points
Cognitive complexity	SonarSource measure of understandability that penalises nesting

APPENDIX C FURTHER TOPICS

Table-driven and parameterised tests

A table-driven test lists cases as data — name, input, expected output — and runs one test body over all of them, in Go with `t.Run` subtests, in JUnit with parameterised tests, in pytest with `parametrize`. Adding a boundary case costs one line, and each failure is reported under the case's name. It is the natural way to test pure functions of the functional core, validators and parsers.

Property-based testing

Instead of hand-picked examples, a property-based test states a property that must hold for all inputs — `decode(encode(x)) == x`, or "a page never holds more than `limit` items" — and a library such as QuickCheck, Hypothesis or fast-check generates hundreds of random inputs and shrinks any failure to a minimal counterexample. It finds the cases the author did not think of. It fits the same problems TDD fits: parsers, serialisers, pricing rules and state machines.

Fuzzing

A fuzzer mutates inputs, guided by coverage feedback, to search for crashes, panics and hangs. Go has native fuzzing since version 1.18 (`go test -fuzz`), and failing inputs are saved under `testdata` so they become permanent regression tests. It matters most for code that handles untrusted input: request parsers, decoders and validators (see Chapters 5 and 16).

Consumer-driven contract testing

Between a team's own services, each consumer records the requests it sends and the responses it relies on as a contract (Pact is the best-known tool), and the provider verifies every contract in its own CI. A breaking API change is caught before deployment without a shared end-to-end environment. It complements the sandbox checks for third-party APIs and machine-readable specifications such as OpenAPI (Chapter 25).

Golden-file and snapshot testing

The output of a test is compared with a stored file — Go's `testdata/*.golden` with an `-update` flag, Jest snapshots — instead of being asserted field by field. This suits large outputs such as rendered JSON responses, generated SQL or API documents. Snapshots updated without review quickly become change-detector tests, so diffs must be read like code.

Race detectors

Go's `-race` flag and ThreadSanitizer for C and C++ instrument memory accesses and report data races observed while the tests run. Concurrency is one of the main causes of flaky tests, and a race may corrupt state only rarely, so detection beats waiting for a failure. The instrumented run is several times slower and uses more memory, so teams typically run it in CI rather than on every local test run.

Load and soak testing

Performance testing, only named in the lecture, is done with load generators such as k6, Gatling, Locust or wrk, which send traffic at a target rate and report latency percentiles and error rates; soak tests hold the load for hours to expose leaks and slow degradation. They validate the scaling work of Chapters 17 and 18 and the service's latency objectives. They are large tests and need a production-like environment.

Fault injection and chaos engineering

Fault injection deliberately introduces failures — added latency, dropped connections, killed instances — to verify that timeouts, retries and circuit breakers (Chapter 12) behave as designed. In tests it can be done with a proxy such as Toxiproxy placed between the service and its dependency; at system scale, chaos engineering does it in con-

trolled production experiments. Error paths are rarely exercised by ordinary tests, which is why they so often show up red in coverage reports.

The Twelve-Factor App — One-Page Overview

Chapter 24 · every section and sub-section of the paper at a glance

1 Introduction: The Twelve-Factor Contract

Env config, port binding, stdout logs and stateless, disposable processes all come from one 2011 document.

- **1.1 A contract between application and platform** — What must hold so any machine, platform or new teammate can run, scale and update the app.
- **1.2 Scope** — Network services in any language; not desktop or mobile, not DRY/KISS, not Kubernetes.

2 Origins: Heroku, Snowflakes and Erosion

Heroku's view of thousands of deployments became rules against hand-built, drifting servers.

- **2.1 Heroku and platform as a service** — Adam Wiggins published the factors at 12factor.net in 2011.
- **2.2 Deployment before the cloud** — Manual installs, FTP copies, SSH edits: unreproducible snowflakes.
- **2.3 Software erosion** — Apps decay without code changes: OS updates, shared libraries, departed admins.
- **2.4 The 2024 open-source revision** — A public `next` branch restates each factor as principles.

3 Codebase, Dependencies and Config (I–III)

Deploys share one codebase and differ only in configuration.

- **3.1 Factor I: one codebase, many deploys** — Production at commit 97, staging at 99, laptop at 100.
- **3.2 Monorepos and copied code** — Copying code violates the factor; a monorepo does not.
- **3.3 Factor II: declaring dependencies** — Manifests and lock files make setup one command, years later.
- **3.4 Factor II: isolating dependencies** — System `requests` v1, ImageMagick 6 vs 7; Docker freezes the OS.
- **3.5 Factor III: what counts as config** — Whatever varies per deploy; the open-source-it-now test.
- **3.6 Why environment variables** — Hard to commit, language-agnostic; Tasker's 38 `TASKER_` values.
- **3.7 Granular values, not named environments** — No `staging.rb`: a new deploy is new values.
- **3.8 Secrets** — The environment leaks; secrets managers as teams grow.

4 Backing Services, Releases, Processes (IV–VI)

Attach services by URL, ship immutable releases, keep processes stateless.

- **4.1 Factor IV: backing services** — Local or managed, the code sees only a resource handle.
- **4.2 A prophecy fulfilled** — The database off the app server: odd in 2011, the norm today.
- **4.3 Factor V: build, release, run** — Build = code + deps; release = build + config; run starts it.
- **4.4 Immutable releases, instant rollback** — Release 102 fails; re-run 101 in seconds.
- **4.5 No path back from production** — No hot patches; immutability is now explicit.
- **4.6 Factor VI: stateless processes** — Share nothing; disk and memory only as a one-transaction cache.
- **4.7 Sticky sessions** — A workaround that defeats scaling; sessions belong in Redis or Postgres.
- **4.8 When the feature is state** — WebSockets as the exception; the kill-any-process test.

5 Port Binding, Concurrency, Disposability (VII–IX)

Self-contained processes, scaled by type, that start fast and stop safely.

- **5.1 Factor VII: port binding** — No Apache or Tomcat host: the app embeds its HTTP server.
- **5.2 Serverless and the revised factor** — Lambda binds no port; the revision asks for a defined interface.
- **5.3 Factor VIII: the process model** — Web, worker and scheduler types scale independently.
- **5.4 Process formation** — 10 web + 2 workers at lunch: 6 GB instead of 10 GB.
- **5.5 Threads inside, no daemons** — The process is the unit of scale; no PID files.
- **5.6 Factor IX: disposability** — Five deploys of ten instances: 50 stops a week.
- **5.7 Fast startup** — A 60 s boot turns a 10-instance rollout into 10 minutes.
- **5.8 Graceful shutdown and re-entrancy** — Finish requests; requeue transactional or idempotent jobs.
- **5.9 Sudden death** — OOM kills give no warning; crash-only design.

6 Parity, Logs and Admin Processes (X–XII)

Close the environment gaps, emit logs as a stream, run admin work from the same release.

- **6.1 Factor X: three gaps** — Time, personnel and tools.
- **6.2 The tools gap** — `LIKE` differs between SQLite and Postgres, silently.
- **6.3 Managed services and parity** — DynamoDB has no exact local twin; accept a justified gap.
- **6.4 Factor XI: logs as event streams** — Unbuffered stdout; no files or rotation in the app.
- **6.5 Who routes the stream** — Terminal, collector or merged cluster stream; observability.
- **6.6 Factor XII: one-off admin processes** — Migrations and fixes from the same code and config.

7 The Factors in Practice

A running backend shows the factors, and each is judged for 2026.

- **7.1 Demo: Tasker under a Procfile** — Boot order, stdout logs, a one-off job, a graceful Ctrl+C.
- **7.2 How the factors aged** — Most hold; secrets, serverless and parity were revised.

The Twelve-Factor App at a glance — twelve rules from Heroku (2011) that let any platform build, configure, run, scale and replace a service mechanically; the 2024 revision keeps them and updates secrets, serverless, parity and observability. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

The Twelve-Factor App: A Contract Between Service and Platform

Chapter 24 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — This chapter traces conventions adopted throughout the course to their source, the Twelve-Factor App, published by Heroku in 2011 as a contract between a service and the platform that runs it. It first describes the hand-built snowflake servers and the software erosion the factors were written against, and the 2024 open-source revision of the document. Factors I to III cover one codebase with many deploys, declared and isolated dependencies, and configuration in granular environment variables, including the case against environment variables for secrets. Factors IV to VI cover backing services as attached resources, immutable build, release and run stages with instant rollback, and stateless processes without sticky sessions. Factors VII to XII cover port binding and its serverless revision, process formation, fast startup and graceful shutdown, dev/prod parity, logs as unbuffered streams and one-off admin processes, followed by a demonstration and an assessment of how each factor has aged.

Keywords — twelve-factor app, Heroku, environment variables, backing services, build release run, stateless processes, process formation, disposability, dev/prod parity

1 INTRODUCTION: THE TWELVE-FACTOR CONTRACT

Throughout this course a number of decisions were adopted without justification: configuration lives in environment variables, a server binds to a port, logs go to standard output, processes keep no state, and a process can be stopped without losing anything. These decisions share one origin, the *Twelve-Factor App*: twelve rules written in 2011 by the people who ran Heroku, one of the first cloud platforms. Each factor below is traced to the problem it solved, to the place where the course already applied it, and to the way the 2024 community revision restates it.

1.1 A contract between application and platform

In two sentences, a twelve-factor application is a contract between the backend and the platform that runs it. The contract states what must be true of the application so that any machine, any platform or any new team member can run it, scale it and update it without surprises. Heroku was the first platform to expect and require these rules; Docker adopted them, Kubernetes takes them into account, and Render, Railway, Fly.io, Google Cloud Run, AWS App Runner, Vercel and Netlify all assume them to some degree. Several rules have become such ordinary practice that engineers follow them without remembering the source: when a platform says that configuration is injected through the environment, or that logs should be written to standard output so that it can collect them, it is restating a factor. An application that already follows the factors deploys on any of these platforms with little friction, because it already satisfies what each of them demands. Giving the contract a name and a form turns the architecture built piece by piece in earlier chapters into one mental model.

Twelve-factor app. A methodology of twelve rules for software delivered as a service, published by Adam Wiggins of Heroku at 12factor.net in 2011. It specifies the interface between an application and its execution platform (codebase, dependencies, configuration, backing services, build pipeline, processes, ports, scaling, lifecycle, environment parity, logs and administrative tasks) so that the application can be deployed, scaled and replaced mechanically.

1.2 Scope

The document concerns programs that run as a service: web applications, APIs and anything else deployed somewhere and reached over the network. The programming language, the database and the message queue do not matter. Desktop software and mobile applications fall outside it. The factors mostly describe the interface between application and platform; they are not a list of everything that makes software good. Principles such as DRY (*don't repeat yourself*) or KISS (*keep it simple*) are not among them, and the factors say nothing specific about Kubernetes or microservice architecture. This chapter stays on that boundary and does not follow the factors into their many offshoots.

2 ORIGINS: HEROKU, SNOWFLAKES AND EROSION

2.1 Heroku and platform as a service

When the document was written in 2011, Heroku had existed for about three years and was among the best-known *platform-as-a-service* (PaaS) providers. On a PaaS the developer hands over code and the platform does everything else: it compiles the code, prepares the runtime environment, applies configuration, connects the database and handles networking until the application is reachable by anyone who enters its domain name. Running such a business put the Heroku team in an unusually good position to notice patterns: they saw hundreds or thousands of applic-

ations in many languages and frameworks being deployed, scaled, broken in production and degraded over time. Adam Wiggins, one of Heroku's co-founders, condensed the recurring patterns into twelve factors and published them at `12factor.net`. That breadth of experience is why the document is still discussed a decade and a half later.

2.2 Deployment before the cloud

The document answers the deployment practice of its time. AWS existed but was not yet the standard, and cloud hosting was not generally accepted; most teams still ran servers they could physically touch. Putting an application on the internet meant either renting a server from a VPS provider, with far fewer features than today's offerings, or keeping a physical machine in the office. Someone installed the operating system by hand, for example Ubuntu or another Linux distribution, then the runtime of the service's language (Java, Go or Python), then the database and auxiliary libraries. The application was copied onto the machine, often over FTP, and configuration files were edited directly on the server over SSH.

Every server built this way was called a *snowflake*: unique, because it was the product of one engineer's manual steps, and therefore impossible to reproduce. Another engineer repeating the same steps, even from written instructions, was likely to do something differently. If the machine failed, the work started again from scratch, from memory or from the notes. Onboarding a new developer, that is, setting up the codebase and getting a running copy on a laptop, took whole days, and even then the laptop copy did not match production; this is where the phrase "it works on my machine" became popular. Deployments were scheduled events for which the team suspended its other work.

2.3 Software erosion

The document names the consequence of building and operating applications this way *software erosion*: an application slowly stops working over time although nobody changed its code. The causes lie outside the code. The operating system receives updates when the Linux distribution is upgraded. Two applications hosted on the same server share a library, and an update made for one starts to affect or break the other. The person who installed and maintained the infrastructure leaves the company without having documented every step and every change, which, when changes are made by hand, nobody can track completely. Taken together, the twelve factors are the defence of the application and its infrastructure against erosion.

> **Software erosion.** The gradual failure of an application whose code has not changed, caused by drift in its environment: operating-system updates, libraries shared with other applications, and undocumented manual changes. Hand-built snowflake servers are its typical breeding ground.

2.4 The 2024 open-source revision

The document then stayed frozen for about thirteen years; its last update, in 2017, was minor. Meanwhile containers, Kubernetes, serverless computing and the ubiquitous use of managed cloud infrastructure became mainstream. In November 2024 Heroku, now owned by Salesforce, opened the document: it moved the text to a public repository under a neutral organisation, wrote a governance document, and invited maintainers from other companies, including AWS and Google Cloud, to review it and bring it up to modern standards. Heroku's Vish Abrams announced the move on stage at QCon. The review takes place publicly on a branch named `next`. It keeps all twelve names and slogans but rewrites each factor as a numbered list of principles, separating the timeless principles of 2011 from those that apply to today's applications. At the time of the lecture the live site still shows the classic 2011 text, and the rewrite remains an unmerged working draft. Each factor is therefore presented below in its 2011 form, then with what the revision changes and with a judgement of how well it has aged.

3 CODEBASE, DEPENDENCIES AND CONFIG (I–III)

3.1 Factor I: one codebase, many deploys

The first factor reads: *one codebase tracked in revision control, many deploys.* Its first half is obvious today, when development without Git and a service such as GitHub, GitLab or Bitbucket is unimaginable. In 2011 it had to be written down: many production servers of the time were backed by a folder on someone's computer or by an archive named in the style of `final_v2_really_final.zip`, sent to colleagues by email. Making version control a rule turned it into a standard.

The second half is less obvious. A *deploy* is one running instance of the application. Production is a deploy and staging is a deploy; advanced workflows create a separate deploy for each pull request, although most teams still work with production and staging. All deploys share one codebase, and what makes them different is never code. Production may run an older commit than staging, but every commit lives in the same repository. Production talks to a real database while a local run talks to Postgres in a Docker container, and the code running in both places is the same; the difference is *configuration*. Code is what every deploy has in common, configuration is what makes each deploy unique.

The lecturer's example: the repository is at commit 100. Production, deployed last Friday, three days ago, runs commit 97. Staging, visible only to the internal team, runs commit 99, because tomorrow's release is being tested there. A developer's laptop runs commit 100 plus two uncommitted changes. That is four deploys and one codebase (Figure 1). On release day staging moves to 100, then production moves to 100; nothing about the deploys changes except

the commit each one runs. The rule of thumb that follows: if an environment cannot be described as a commit of the one codebase plus its configuration, the setup is wrong, and it will cause problems later.

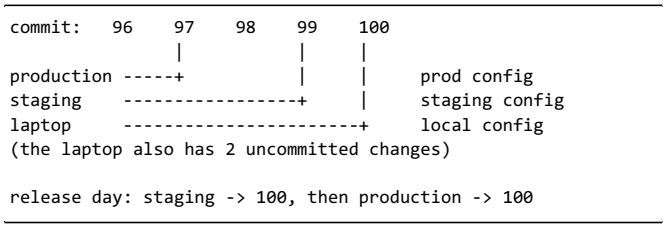


Figure 1: One codebase, several deploys at different commits, each with its own configuration (illustrative, after the lecturer's example)

3.2 Monorepos and copied code

The factor sets two limits. If a system consists of several codebases, it is not one application but a distributed system made of applications, and each of them must follow the twelve factors on its own. If two applications share code by copying it, rather than by depending on one common source, that is a violation: shared code belongs in a library consumed through the dependency manager, which is the subject of the second factor.

A recurring argument in 2026 concerns monorepositories, now a popular way for companies to keep many applications in one repository; some read the factor as declaring monorepos an anti-pattern. The lecturer disagrees. The document defines a codebase as a repository, or a set of repositories that share a root commit, and requires a one-to-one correspondence between codebase and application. A monorepository holding five applications is therefore five codebases that happen to live in one repository, and each application owes the twelve factors separately. What the factor forbids is the reverse: one application living in two places, that is, code shared by copying.

3.3 Factor II: declaring dependencies

The second factor is *explicitly declare and isolate dependencies*, and both verbs matter. Declaration means that the repository contains a file that lists, completely and exactly, every library the application depends on: `package.json` for Node.js, `go.mod` for Go, `requirements.txt` or `pyproject.toml` for Python depending on the tooling, a `Gemfile` for Ruby. Without this *manifest*, a new laptop or a new team member has to dig through the whole codebase, read every import statement and guess which versions it needs, the versions may conflict with each other, and setup easily turns into two or three days of continuous work. With a manifest that pins correct versions, preparing a machine is one command that downloads and installs every package and starts the application. The 2011 text cites the then-dominant Ruby on Rails pair: the `Gemfile` together with `bundle install`.

The idea has become more relevant since. Besides manifests, ecosystems now keep *lock files*, such as `package-lock.json` or `go.sum`, that fix exact versions rather than version ranges. A manifest often names only an approximate list of the packages needed; a lock file records the exact version of every library and of the libraries those libraries depend on. An application run months or years later from its manifest and lock file installs the same versions.

> **A subtle point.** Go divides the roles differently from npm. Go modules use minimal version selection, so `go.mod` by itself already determines the exact version of every dependency, including transitive ones; `go.sum` holds cryptographic checksums that verify the downloaded modules have not changed. In Go, reproducibility comes from `go.mod`, and `go.sum` adds integrity.

3.4 Factor II: isolating dependencies

Isolation is the half that is easily forgotten. Suppose a Python application declares `requests`, the popular HTTP library, at version 2 in its `requirements.txt`, but runs against the system Python, where another tool has already installed `requests` version 1. The import statement finds the system copy first, so despite the manifest the application runs something other than what was declared. Declaration and isolation fail separately: declaration without isolation is useless, because the application ignores the manifest and uses whatever the machine provides; isolation without declaration means days or weeks spent trying to reproduce a working set of versions, perhaps without success. Isolation demands that nothing leaks in from beyond the application's boundary. The 2011 text names `bundle exec` for Ruby and `virtualenv` for Python: tools through which the application sees only its declared dependencies.

The same text extends the rule to system tools, which should be *vendored* into the application rather than assumed to exist. Consider an image-resizing backend that shells out to the ImageMagick binary installed on the system. On a developer's macOS laptop Homebrew installed ImageMagick 7, whose main binary is called `magick`; on the Ubuntu server APT installed ImageMagick 6, whose binary is called `convert` and has a different set of default parameters. The same code snippet runs the shell command on both machines and performs different operations, because these are two different programs, and a major-version gap brings radical changes in behaviour.

Containers changed the reach of this factor more than any other technology. A Docker image freezes not only the libraries but the operating system, the system tools and the runtime. It is the strongest form of declaration and isolation available, which is why practitioners say that Docker solved this factor in practice.

3.5 Factor III: what counts as config

The third factor, *store config in the environment*, is one of the most cited and most debated. Configuration is everything that is likely to differ between deploys (Table 1). The database URL points to `localhost` and a Postgres container on a laptop, and to Amazon RDS inside a VPC in production. An email provider can be used through a sandbox API in development, which sends only from and to test

addresses. Object storage is a locally running MinIO instance in development and real S3 in production; Redis is a container locally and a real, managed instance in production. Across all of these, only configuration changes.

Resource	Laptop	Production
Database URL	localhost, Postgres in Docker	Amazon RDS in a VPC
Email	provider's sandbox API	live sending API
Object storage	local MinIO	AWS S3
Redis	container	managed instance

Table 1: The lecturer's examples of configuration that differs between a local and a production deploy

Whether a value belongs in code or in configuration is decided by the document's test: *could the codebase be made open source right now, at this moment, without compromising any credentials?* The question is not whether the company would publish its internal code but whether it could, even for a minute, if challenged. If the answer is no, some configuration lives inside the code where it should not.

3.6 Why environment variables

Once configuration lives outside the code, the factor says where: in environment variables. It gives three reasons. They are easy to change between deploys without touching files. They cannot be committed to the repository by accident, which is a daily cause of credential leaks. And they are an operating-system standard that depends on no language or framework, so the same mechanism serves Go, Node.js, Rust, Python and a plain bash script. The alternatives of 2011 were per-framework formats, such as Java properties files or the Rails `config/database.yml`; most of them ended up in version control sooner or later, so credentials leaked constantly.

The course's task-management backend, Tasker, follows the factor (see Chapter 13). Its `config` package declares all configuration as one Go structure, with a block per concern: server, database, authentication, Redis, email integration, AWS, monitoring and cron. There are about 38 values in total, all loaded from environment variables carrying a `TASKER_` prefix, such as `TASKER_SERVER_PORT` and `TASKER_DATABASE_HOST` (Figure 2). The project history shows the configuration changing many times as integrations, timeouts and hosts were added, while the code that reads it barely changed. That is exactly what the principle requires: configuration varies with the deploy and over time, the code does not, and things that change at different speeds should not live in the same place.

```
# environment of one deploy
TASKER_SERVER_PORT=8080
TASKER_DATABASE_HOST=localhost
TASKER_DATABASE_PORT=5432
TASKER_REDIS_HOST=localhost

// config package: one struct, one block per concern
type Config struct {
    Server     ServerConfig
    Database   DatabaseConfig
    Auth       AuthConfig
    Redis      RedisConfig
    Email      EmailConfig
    AWS        AWSConfig
    Monitoring MonitoringConfig
    Cron       CronConfig
}
```

Figure 2: Tasker-style configuration loaded from prefixed environment variables (illustrative; the real structure holds about 38 values)

3.7 Granular values, not named environments

The 2011 text warns against grouping configuration into named environments, such as `config/environments/staging.rb` and `production.rb` next to files for development and test. The scheme does not scale as environments multiply. When someone needs a new deploy that resembles staging but uses another database, for a pull-request branch for example, another environment must be added to the code, and every new deploy adds more. Configuration should instead be granular: each value is a separate environment variable, none depends on the name of a mode, and a new deploy is created by setting values, never by selecting a mode. The revised text keeps this: environment variables are granular controls and are not to be grouped into environments.

3.8 Secrets

Environment variables suit a value like the database hostname. Whether they suit secrets, such as the database password, is the contested part. In 2017 Diogo Mónica, then responsible for security at Docker, argued in a widely cited article that they do not, because the environment leaks. Every child process inherits the whole environment, so any shell command the application launches receives all its secrets. Crash reporters and debug pages commonly print the whole environment, since that is what debugging output does. On Linux the environment of a process can be read through `/proc/<pid>/environ` by anyone with access to it, and various tools write the environment into their logs. His alternative is a dedicated secrets-management service from which the application reads its secrets at startup and keeps them only in memory.

The verdict combines both views. The basic rule, configuration strictly separated from code, is unchanged since 2011. Environment variables remain the correct default mechanism, above all for everything that is not secret. For credentials they need not be the default. For a solo developer or a very small team, environment variables injected by the platform's secrets feature are acceptable, and far better than credentials in the repository. As the product, the company and the number of employees grow, a secrets

manager for all credentials becomes the obvious choice: HashiCorp Vault, or Parameter Store on AWS. Secrets are fetched at startup, held in memory, rotated periodically and audited, along with the other security practices of Chapter 16.

> **Warning.** A secret placed in an environment variable is visible to every child process, to crash reporters and debug pages that dump the environment, and to anyone who can read `/proc/<pid>/environ`. Once more than a handful of people or tools can reach the running process, the environment is a channel for configuration, not a store for secrets.

4 BACKING SERVICES, RELEASES, PROCESSES (IV–VI)

4.1 Factor IV: backing services

A *backing service* is any service the application uses over the network to do its work: the Postgres database, Redis, message queues, background-job services, SMTP for sending email, object storage such as S3 (see Chapter 20), third-party APIs such as payment providers. If the program reaches something over the network through some protocol, it is a backing service. The fourth factor, *treat backing services as attached resources*, says that the code must not distinguish a service the team runs itself from one provided by another company. To the program every backing service is an external resource reached through a *resource handle*: an address together with the credentials needed to connect, which by the third factor lives in configuration, not in code. Whether the database runs in a local container or on AWS RDS, the code contains no special logic for either case and stays the same whatever service and environment it runs against; only the handle changes.

The 2011 document offers a test: it must be possible to replace a local Postgres URL with any managed service, such as Neon, PlanetScale, Supabase or Amazon RDS, without changing a single line of code. Only the resource handle in the configuration changes; if that holds, the application follows the factor. A Postgres handle has five parts (Figure 3): the scheme, which selects the Postgres wire protocol; the credentials, a username and a password; the host; the port, by default 5432; and the database name. The one string holds everything the application needs to connect to the resource and work with it.

```
postgres://tasker:s3cret@db.internal:5432/tasker
|          |      |      |           |    |
scheme     user   pass   host        port database

laptop:  postgres://dev:dev@localhost:5432/tasker
managed: postgres://app:***@x.rds.amazonaws.com:5432/tasker
```

Figure 3: The five parts of a Postgres resource handle; swapping the handle swaps the database without code changes (illustrative values)

4.2 A prophecy fulfilled

In 2011 this factor looked eccentric. Heroku's engineers held that the database is someone else's computer and the application merely talks to it through a URL. That was true even then, but it was not the standard: for most teams the server and its database ran on one machine, and the idea that the data lives somewhere outside the server was unfamiliar. A decade and a half later it is one of the most common patterns in any codebase, and running the application server and the database on one instance is considered an anti-pattern as soon as scaling is involved (see Chapters 17 and 18). Databases run on RDS, Neon, PlanetScale or similar services, Redis runs as a managed instance, objects go to S3, and email goes through services such as Resend instead of a self-hosted SMTP server. The factor passed the test of time; it was in effect a prophecy, given how the world was arranged when it was written.

4.3 Factor V: build, release, run

The fifth factor, *strictly separate build and run stages*, describes what a CI/CD pipeline does (Figure 4). In the *build* stage a runner, the program that builds or deploys the application, fetches the code of a specific commit through Git. Deployments are driven by commits in version control, not by FTP or by copying files over SSH. The runner downloads the dependencies listed in the manifest (Factor II), compiles the code if the language is compiled, and produces one executable package, which the document calls the *build*. For a compiled language such as Go the build is a single binary; for an interpreted one, such as a Node.js application, it is the whole application with the `node_modules` it needs; in the container world it is an image. The key property is that a build contains only the code and its dependencies, never configuration.

The *release* stage takes the build from the previous stage and combines it with the configuration of one deploy: staging, production, a test environment, a pull-request preview. The result, an exact build plus an exact configuration ready to execute, is a *release*. The build combined with the production configuration is the production release; the same build combined with the staging configuration is the staging release. The *run* stage takes a release and starts processes from it.

```
commit 97 (Git)
    |
    v
  BUILD    fetch code, install deps, compile
    |      -> build: one binary or image, no config
    v
  RELEASE  build + config of one deploy
    |      -> release 100: numbered, never modified
    v
  RUN      start processes from the release

build 97 + staging config     -> staging release
build 97 + production config  -> production release
```

Figure 4: The three stages of Factor V (illustrative)

4.4 Immutable releases, instant rollback

The strict separation brings two benefits. The first is rollback. A release is just a build plus a configuration, and builds are immutable, so returning to an earlier release costs nothing and is instantaneous. The document requires releases to form an *append-only ledger*: they are never changed or updated, only added to. Every release carries a

unique identifier, a timestamp or an incrementing number, and cannot be modified once it has been created.

The lecturer's scenario follows Table 2. In the afternoon the team builds commit 97 and, with the current configuration, creates release 100. Two hours later someone changes the database password; the configuration has changed, so release 101 is created with the same build and the new configuration. Two hours after that, newly merged pull requests produce release 102, which changes the build and keeps the configuration. As soon as 102 is live, the dashboards fill with errors and users complain in the team's Slack channel. Instead of debugging production while customers shout, the team runs release 101 again, the release that worked before the new code arrived; because builds are immutable, it is active within seconds. Once production is stable, the bad release is debugged without haste, however long that takes. What made a seconds-long incident response possible is immutability alone: release 101 means exactly what it meant two hours earlier, the same build with the same configuration. Every current platform uses this pattern, whether as Heroku releases, Docker image tags or Kubernetes deployments.

Release	Build	Config	Event
100	commit 97	A	afternoon: built, deployed
101	commit 97	B	+2 h: DB password changed
102	newer commit	B	+4 h: PRs merged; errors
101 again	commit 97	B	rollback: stable in seconds

Table 2: The lecturer's release ledger; rolling back means running release 101 again

4.5 No path back from production

The second benefit: if code enters the pipeline only at the build stage, nobody can edit code on a production server, even deliberately, because there is no path from a running machine back to the build. In the world of snowflake servers, logging in over SSH, installing a "hot" patch and restarting the server was common practice, and it is how the running environment and the code repository ended up in two different states. Strict separation makes that situation impossible. The revised edition changes one thing relative to 2011: immutability, implied in the original, becomes an explicit and mandatory part of the principle. Every release is a unique, unchangeable snapshot.

4.6 Factor VI: stateless processes

The sixth factor reads: *execute the app as one or more stateless processes*. The application runs as separate processes that remember nothing between two requests. This statelessness has appeared in almost every lecture of this course, because it is what lets a backend scale. The processes share nothing: no common memory, no common disk. Anything that must persist goes into a stateful backing service, attached as in Factor IV. A service must be designed on the assumption that any request may be handled by any process at any moment, and that any process may disappear, or be destroyed, at any moment.

The document allows one exception: memory and the local disk may serve as a brief, single-transaction cache. Downloading a file, processing it and writing the result to the database, with the file held in a temporary directory during processing, is fine: once the transaction is complete nothing expects the file to remain, and in a temporary directory it disappears anyway. The pattern becomes a problem the moment a future request or another process depends on the file still being there.

4.7 Sticky sessions

The document singles out *sticky sessions*: the load balancer in front of the API remembers which instance each user reached before and sends every later request from that user to the same instance, so that internal state held there keeps working. The need arises as follows. An application stores a user's login session in process memory, which works on one instance. The service is then scaled to three instances behind a load balancer for the load (see Chapter 17). A user logs in on instance 1; the next request is routed to instance 2, which knows nothing about the user, and the user is logged out at random (Figure 5). Sticky sessions are the workaround people invented: force the balancer to remember where each user went and send them there every time.

The document is, as might be expected, firmly against this: sticky sessions are a violation and should never be used, whatever the situation. Binding users to an instance at the balancer level cancels the purpose of having three instances. If one instance is stopped or fails under heavy load, all its users are logged out on their next request; if a new instance is added to share the load, it receives none of the existing users. Either way users must start over and log in again. The fix is to recognise session data as state, and state lives in a backing service: Redis, Postgres or another store chosen according to the latency requirements (see Chapters 4 and 9).

```
Session kept in process memory (3 instances):
  login     -> LB -> inst 1   session stored in inst 1
  next req  -> LB -> inst 2   unknown user: logged out
  sticky    -> LB pins to 1   inst 1 dies: logged out
                              new inst 4 gets no users

Session kept in a backing service:
  any req   -> LB -> inst 1/2/3 -> Redis or Postgres
  instances may be added or killed; sessions survive
```

Figure 5: Why in-memory sessions lead to sticky sessions, and the stateless alternative (illustrative)

4.8 When the feature is state

Chapter 22 met a case where the factor cannot hold literally. A WebSocket connection is state held in process memory: the instance holding the socket is the only one in the world that can talk to the user connected through it. The course solved the cross-instance problem with publish/subscribe: every event is published, and whichever instance holds the user's connection subscribes and tracks the changes. The connection itself, however, still breaks statelessness. The lecturer's position is that statelessness

is the default, applied everywhere, and that it may be broken when the feature itself is state, as with real-time connections. The basic rule of the factor: **if a process or instance cannot be terminated at any moment without losing user data, there is state where it should not be.**

5 PORT BINDING, CONCURRENCY, DISPOSABILITY (VII–IX)

5.1 Factor VII: port binding

The seventh factor, *export services via port binding*, looks self-evident now, but in 2011 it was not. PHP then held roughly the position React holds today, and a PHP application did not run on its own: Apache ran, and the PHP application was a module loaded inside it. A Java application did not run on its own either: a server called Tomcat ran, and the application was a file placed into a Tomcat folder. Web servers such as Apache and Tomcat were the programs; applications were guests inside them. The web server's version and configuration were therefore part of the application's execution environment, that is, part of the physical server, which leads straight back into erosion.

The factor demands that the application be completely self-contained: it includes its own HTTP server as a library, obtained through the dependency manifest of Factor II, and exports itself as a service by binding to a port and waiting for connections there. In 2026 this is the obvious model. Every backend binds to a port, 8080 being common for Go and 4000 or any free port for Node.js, and databases likewise bind to their default ports as services. During development the service is reached at `localhost:8080`; in production a routing layer, a reverse proxy, maps the public hostname to the port the process is bound to (see the course's introductory videos). A convenient consequence is that any application that exports a port can become a backing service for another. Locally the database, the cache and the email service all sit on ports on localhost; in production the same code receives other URLs, which point to services bound to ports on machines in the cloud. One model serves everywhere.

5.2 Serverless and the revised factor

What has aged is the assumption behind the factor. AWS Lambda functions bind to no port: the platform calls a function that the code exports. The 2011 document expected long-running, port-bound servers that work around the clock to remain the template forever. They are still one of the most used templates, but serverless architecture created a separate category in which there are no long-lived servers bound to ports. The revision therefore keeps "Port binding" as the title but generalises the rule: the application must be self-contained and export its service through a defined interface. The principle then covers both a long-running server and a serverless function that the platform starts.

5.3 Factor VIII: the process model

The eighth factor, titled *Concurrency* with the slogan *scale out via the process model*, concerns what applications had to do to absorb more load and more users at peak traffic. In 2011 each runtime had its own mechanism. Java scaled as one huge process: the JVM reserved a large block of memory at startup and multiplexed work inside its own execution environment over threads. PHP was the opposite: Apache, on which PHP ran, spawned small child processes per request. The factor asks to treat the process as a first-class object and to design services so that they scale as individual processes from the very beginning, not as an afterthought. The model is inspired by Unix service daemons and implies process types, each kind of work getting its matching type. HTTP requests are handled by a *web* process. Heavy background tasks such as video encoding or image resizing are handled by a *worker* process (see Chapter 10). Scheduled logic, such as an email sent to every user at midnight, gets a scheduler process of its own. Each type scales independently, by running more copies of it.

5.4 Process formation

The lecturer's example is a food-delivery service in which every process takes about 500 MB of memory. People order far more at lunchtime than at, say, 10:00 or 11:00. At lunch the user load needs 10 web processes, while the queue needs only 2 workers, because background work is light at that hour compared with the number of requests. If web and worker logic were one big process, scaling it to 10 copies for lunch would also produce 10 workers: the web half, 5 GB, does its job, while the worker half, another 5 GB, scaled along with it and mostly has no work. Run as separate process types, the lunch formation is 10 web and 2 workers, 6 GB instead of 10 GB. At night the queues and workers do their main work and requests fall, since few people order food: web stays at, say, 5 while workers scale to 20 (Table 3).

Formation	web	worker	Memory
One process type, lunch	10	10 (2 needed)	10 GB
Separate types, lunch	10	2	6 GB
Separate types, night	5	20	12.5 GB

Table 3: The lecturer's food-delivery formation at 500 MB per process (night memory computed from his counts)

The set of process types and the number of each, such as `web=3`, is called the *process formation*, a term still in use. A Procfile declares the types; a Kubernetes deployment specification states the count, written today in YAML as `replicas: 3` (Figure 6).

```
# Procfile: one line per process type
web:       ./tasker serve
worker:    ./tasker worker
scheduler: ./tasker scheduler

# formation: lunch, then night
web=10 worker=2
web=5  worker=20

# Kubernetes: the count per type, in YAML
kind: Deployment
metadata:
  name: tasker-worker
spec:
  replicas: 2
```

Figure 6: Process types in a Procfile, and their counts as a formation or as Kubernetes replicas (illustrative)

5.5 Threads inside, no daemons

Two remarks complete the factor. First, the process model does not forbid threads. Inside a process any multiplexing is allowed, whether threads in the JVM, goroutines in Go or the event loop in Node.js (see Chapter 19). For scaling, however, the unit must be the process, not threads, goroutines or an event loop, because one machine has limits and vertical scaling runs out sooner or later. A share-nothing process can be scaled across many machines without further thought.

Second, a twelve-factor process never runs as a daemon and never writes a PID file. Managing processes and their lifecycle is not the application's job but the platform's: the operating system's when the application runs directly on a host, Kubernetes' or a process manager's otherwise. The application runs in the foreground, does its work and lets the platform supervise it. Almost the whole internet now runs on Kubernetes or similar orchestrators, whose logic is that the operator declares a configuration and the platform maintains it, stopping, starting and restarting containers and performing every lifecycle operation. The factor has therefore not aged; it is integrated so deeply into daily work that nobody needs to think about it, unless an application is deployed directly on an operating system with systemd, where it still applies in full.

5.6 Factor IX: disposability

The ninth factor, *maximize robustness with fast startup and graceful shutdown*, requires processes to be *disposable*: interchangeable, startable or stoppable at any moment, with nothing harmful or destructive happening in either case. Chapter 15 covered graceful shutdown and the full process lifecycle in depth; here the question is why disposability matters. Processes are started and stopped by the platform all the time. Horizontal scaling launches new processes at arbitrary moments. A deployment terminates old processes and replaces them with new ones from a new container image. The orchestrator moves, deletes and restarts processes across machines whenever it chooses. Deploying five times a week to 10 instances means 50 stops and 50 starts from deployments alone, before any scaling. Termination is therefore nothing unusual: it is part of the development and release cycle, not an incident. That is why processes must be interchangeable, and why both ends of their life must be fast.

5.7 Fast startup

If startup is slow, each of those events is slow. A process that needs 60 seconds to load and start turns a rolling deployment of 10 instances into 10 minutes, and makes an autoscaler one minute late for every traffic spike. At 2 seconds both effects are almost imperceptible. The factor therefore asks that startup take only a few seconds.

5.8 Graceful shutdown and re-entrancy

Careless shutdown loses whatever the process was doing, including background processing in progress. Suppose a shutdown request arrives while a web process is handling five HTTP requests. Unless the process follows the lifecycle from Chapter 15, that is, stop accepting new work, finish the existing work, then stop the process, all active requests fail with sudden errors during every deployment. On the stop signal the web process stops taking new requests, completes those already in progress and only then exits, so that existing users see no errors. This is what the course's Go application does when its server shuts down.

For a worker, graceful means returning the current job to the queue instead of dropping it, because the time to stop has come. Returning it assumes the next run of the job is safe, a property the document calls *re-entrancy*: a job that was interrupted by shutdown and forced back into the queue must not leave "dirty" state behind or perform operations that cause problems when it is executed again. Re-entrancy is achieved either by wrapping the processing in a transaction or by making the operation idempotent: however many times the code runs, the result is the same, so a job processed twice by mistake is still safe because it detects that its work was already done and skips it (see Chapter 10).

> **Re-entrancy and idempotency.** A job is re-entrant when it can be interrupted at any point and executed again from the start without leaving or creating inconsistent state. The two standard means are a transaction, so that an interrupted run leaves no partial effect, and idempotency, so that repeated runs produce the same result as a single run.

5.9 Sudden death

The factor also requires robustness against sudden stops. SIGTERM, the signal a platform, runtime or orchestrator sends before stopping a process, is the polite case, the one the process is prepared for: it receives a grace period, about 30 seconds in the lecturer's example, to finish its current work, clean up and exit, which is why the exit is called graceful. The grace period is set by the platform, by default 30 seconds in Kubernetes and 10 seconds for `docker stop`. A hardware failure gives no warning. If a process tries to use too much memory and its container is killed with an out-of-memory error, the container simply disappears, and the process with it. Systems must therefore be designed so that any process can stop in the middle of any instruction.

The document points to *crash-only design*: a system that survives crashes needs no separate shutdown path, because the crash path is the shutdown path.

6 PARITY, LOGS AND ADMIN PROCESSES (X–XII)

6.1 Factor X: three gaps

The tenth factor, *keep development, staging, and production as similar as possible*, is known in the industry as dev/prod parity, where parity means how alike the environments are. The document names three gaps between development and production (Table 4). The *time gap*: code written today reaches production only on Friday or next week, and a deployment combines many people's code, so a long gap ships many changes at once. The *personnel gap*: the person who writes the code is often not the person who deploys it or fixes its errors, an operations specialist. The *tools gap*: development happens on a MacBook or on Windows with a completely different stack, Docker containers or native installs, while production runs on Linux connected to external servers and services. The whole architecture differs radically, and the difference causes many errors. The document's table contrasts a traditional application with a twelve-factor one: deploy every few hours instead of every week or two; let the developers who wrote the code deploy it, and read its logs when something fails; make the environments as similar as possible. As the gaps are eliminated, continuous deployment becomes smooth.

Gap	Traditional app	Twelve-factor app
Time between deploys	weeks	hours
Code authors vs deployers	different people	same people
Dev vs production	divergent	as similar as possible

Table 4: The three gaps, after the comparison table of the 2011 document

6.2 The tools gap

The time and personnel gaps have largely closed since; the tools gap still exists. A common example is a lightweight database such as SQLite in development instead of a full Postgres. ORMs make this easy: Drizzle or Prisma in Node.js, GORM in Go, SQLAlchemy in Python. Apart from specific cases in which raw SQL is written and the ORM serves only as a driver, the ORM abstracts the database completely: the code follows the ORM's own standards and patterns, and the ORM takes care of whether Postgres, SQLite or MySQL sits behind it. The convenience hides real differences. In Postgres the `LIKE` operator is case-sensitive and `ILIKE` is its case-insensitive variant; in SQLite `LIKE` is case-insensitive for ASCII characters by default (Figure 7). A search feature built locally on SQLite through an ORM and released to production will sooner or later break on assumptions nobody knew had been made. The danger is that there is no error, no warning and no log entry: the behaviour simply changes.

```
-- rows: 'Dune', 'dune messiah', 'Children of Dune'
SELECT title FROM books WHERE title LIKE '%dune%';

SQLite, LIKE:    Dune, dune messiah, Children of Dune
Postgres, LIKE:  dune messiah
Postgres, ILIKE: Dune, dune messiah, Children of Dune
```

Figure 7: The same query through the same ORM returns different rows on SQLite and Postgres, without any error (illustrative)

6.3 Managed services and parity

The document's advice is that every backing service, whether database, cache, queue or anything else, should be the same in development, testing and production. Today the classic excuse is gone, since one command starts the production database in a local container; in 2011 that was far more troublesome. Yet in 2026 the advice cannot be absolute either. Docker and Docker Compose first made local and production environments nearly identical by running the same container in both, but teams then moved to managed services. An application that uses AWS DynamoDB in production cannot run DynamoDB itself locally. Emulators try to reproduce as many of its features as possible; the popular LocalStack, for example, runs many AWS services locally. But DynamoDB is proprietary, everything known about its behaviour comes from the documentation, and it cannot be reproduced exactly on a laptop. That leaves a gap with no clear-cut rule. Nobody can demand that managed services be avoided for the sake of parity, since they bring large advantages in scaling and lower costs from not managing one's own infrastructure. If the benefits of a managed service justify the compromise of two different environments, local and production, that path is the right one.

A subtle point. AWS does publish DynamoDB Local, an official downloadable version for development and testing, so a DynamoDB-compatible endpoint can run on a laptop. It is a separate implementation, not the service: throughput limits and other operational behaviour differ. The lecturer's conclusion stands; the parity gap narrows but does not close, and some tests must still run against the real service.

6.4 Factor XI: logs as event streams

The word log first evokes a log file, because of how logs behaved historically. The eleventh factor, *treat logs as event streams*, says that the file never was the log; it is only one output format. A log is a stream of events ordered by time, which keeps flowing as long as the application runs and has no beginning and no end. Seen as a stream rather than as a format (standard output, a file or an external collection service), the principle follows: a twelve-factor application does not concern itself with routing or storing its stream. It only ensures that the stream exists, and it does not manage log-file paths, rotation or anything similar. Each process writes every event to standard output, the output visible in the terminal right after the program starts, always and unbuffered. The execution environment, whatever platform or orchestrator the application runs on, is responsible for capturing the stream and deciding where

it goes: to a file, or to an external service for analysis and storage.

6.5 Who routes the stream

The division of labour has two reasons. First, logging stops being a concern when the application is deployed, because routing logs is no longer the application's responsibility; the code does not change when the application moves to a new environment, which becomes a matter of configuration. In 2011 standard output typically went to a system such as syslog; today production logs from standard output flow to services such as Loki or Datadog, and the pattern is unchanged. Second, a stream can be combined with other parts of the system in ways a file cannot (Figure 8). On a laptop the stream appears in the terminal and is read in real time. In a container the runtime captures the stream, labels it with the process it came from and passes it to a collector process that stores, analyses or otherwise handles it. In production the orchestrator or cloud platform takes the streams of all instances and merges them into one stream ordered by time; nobody needs to know which instance emitted a line, only when each event happened. The burden of understanding, analysing and searching logs across many instances disappears, and one template, writing to standard output, yields a different architecture in each environment.

```
every process: events -> stdout, unbuffered, no log files

laptop      stdout -> terminal, read in real time
container   runtime captures stdout, tags source process
            -> collector -> storage or analysis
production  platform merges stdout of all instances
            -> one time-ordered stream -> Loki, Datadog
```

Figure 8: One rule, write to standard output, and three routing architectures chosen by the environment (illustrative)

Practice has since grown into *observability*, made up of logs, metrics and tracing (see Chapter 14). The 2011 text mentions only logs because metrics and tracing were not yet in common use. They are additions to the factor, which does not limit them.

6.6 Factor XII: one-off admin processes

A *one-off process* is a task run only once, next to the long-running application process, that then exits. An HTTP backend normally accepts requests, processes them and returns responses, but other things must happen too: a database migration has to run, or a script has to correct certain data in production. These run exactly once, in parallel with the working application. The twelfth factor, *run admin/management tasks as one-off processes*, says where they run: in an environment identical to the main process, with the same codebase at the same commit, so that one deployment covers both, and with the same configuration and the same dependency-isolation model as the main application. The strictness answers earlier practice. To correct a record, engineers connected over SSH to production servers and ran an `UPDATE`, a `DELETE` or another query, with no code review, no control and no record or log of what was done in production. Production then diverged from what the code documented. The best-known example of the factor is database migrations, shipped with the same code and run as a one-off process before the application starts, after which they exit.

7 THE FACTORS IN PRACTICE

7.1 Demo: Tasker under a Procfile

The lecture closes with the course's task-management backend running on a Linux machine reached over SSH. The application is managed through a *Procfile*, the format Heroku invented for declaring process types such as web, worker and scheduler; here it declares a web process, and Overmind, a Procfile-based process manager, runs the system. The boot sequence shows the schema check, then the background-job server, then the HTTP server binding to its designated port, in the order the factors describe, and the process writes its own log stream to standard output. In a second window a one-off admin process for auto-archiving is started; it runs from the same code, the same configuration and the same environment, and exits when done. Finally Ctrl+C stops the application: the server closes its connection pool, the job server waits for all its workers to finish, and the process reports a graceful shutdown, as Factors VIII, IX and XII require (Figure 9).

```
# window 1
$ overmind start             # reads the Procfile
web | schema check passed                  (XII)
web | background job server started        (VIII)
web | HTTP server listening on its port    (VII)
web | request logs on stdout ...           (XI)

# window 2: one-off admin process
$ ./tasker archive           # same code, config, env
archived old tasks; exited                 (XII)

# window 1: Ctrl+C
web | closing database connection pool     (IX)
web | waiting for workers to finish jobs   (IX)
web | graceful shutdown complete           (IX)
```

Figure 9: The Tasker demonstration in outline; commands and output lines are paraphrased (illustrative)

7.2 How the factors aged

Most factors have aged well, and several are so absorbed into platforms that they are no longer noticed. The revisions concern secrets (III), immutability made explicit (V), serverless functions (VII), managed services that cannot run locally (X) and observability beyond logs (XI). Table 5 gathers the verdicts and the lectures in which the course applied each factor.

Factor	Lectures	Verdict in 2026
I Codebase	31	timeless; monorepos compatible
II Dependencies	31	stronger: lock files, containers
III Config	17, 20	env default; secrets to a manager
IV Backing services	11, 12, 24	prophecy fulfilled
V Build, release, run	31	immutability now explicit
VI Processes	7, 21, 26	default; real-time the exception
VII Port binding	2	generalised for serverless
VIII Concurrency	14, 23	absorbed by orchestrators
IX Disposability	19	essential; crash-only design
X Dev/prod parity	27	tools gap remains
XI Logs	18	extended by metrics, tracing
XII Admin processes	11	migrations as one-off runs

Table 5: The twelve factors, the lectures that applied them, and how each has aged

8 CONCLUSION

- The twelve factors (Heroku, 2011) are a contract between a service and its platform, assumed by every modern PaaS and orchestrator.
- They were written against hand-built snowflake servers and the software erosion they cause.
- One codebase per application, many deploys that differ only in configuration; copying code violates the rule, monorepos do not.
- Dependencies are declared in a manifest and lock file and isolated; container images freeze even the OS and system tools.
- Configuration lives outside the code in granular environment variables; secrets move to a secrets manager as the team grows.
- Backing services are attached through resource handles; immutable builds and an append-only release ledger make rollback take seconds.
- Processes are stateless, bind their own port, scale by type, start in seconds, shut down gracefully and survive sudden death.
- Keep environments alike, but accept the gap a managed service brings when its benefits justify it; write logs unbuffered to stdout; run admin tasks as one-off processes.
- The 2024 open-source revision keeps the names and slogans and updates the factors for containers, serverless and observability.

APPENDIX A REVIEW QUESTIONS

1. A team deploys by logging into each of four servers over SSH, running `git pull`, editing `.env` in place and restarting; urgent fixes are sometimes patched directly on one server. On Friday a deploy breaks checkout, and nobody can say which commit and which configuration were live on each server an hour earlier. Identify the factors violated and explain why the rollback takes an hour instead of seconds, then design a build, release and run pipeline with a release ledger, stating exactly what a rollback consists of and how a password change made during the incident is recorded.
2. A Node.js thumbnail service declares `"sharp": "^0.32.0"` in `package.json`, does not commit its lock file, builds `FROM node:latest` with `npm install`, and shells out to whatever `convert` binary the base image happens to contain. Six months later a rebuild of the unchanged commit fails in production, while the old image still works. List every source of non-reproducibility in this setup, argue which of them could explain the failure, and specify the changes that restore Factors II and V.
3. An application keeps `config/environments/staging.rb` and `production.rb`, each holding a database URL, feature switches and a payment-provider API key. The team wants a preview deploy for every pull request, each with its own database; a first attempt moved every value into environment variables, after which the API key appeared in a crash report sent to a third-party error tracker. Explain why the original scheme does not scale and how the key leaked, then design configuration and secret handling for a three-person startup and again for a 200-person company, justifying the difference.
4. An API keeps login sessions in process memory. After it is scaled from one to four instances behind a round-robin load balancer, roughly three quarters of users are logged out on their second request; enabling sticky sessions stops the complaints, but a week later one instance is killed for exceeding its memory and its users are all logged out, while a fifth instance added for a sale stays almost idle. Explain each symptom, including the three-quarters figure, name the factor violated, and choose a session store given a 2 ms budget for session lookup and a requirement that sessions survive instance restarts.
5. A worker job charges a customer's card through a payment API and then marks the order as paid in Postgres. During a deploy the worker receives SIGTERM after the charge succeeds but before the update commits, returns the job to the queue as Factor IX recommends, and another worker runs it again. Predict the outcome, explain why the job is not re-entrant, and redesign it so that requeueing is safe, stating which part a database transaction alone cannot protect and what must take its place.
6. Each instance of a service runs pending database migrations in its startup code and then warms an in-memory cache from the database, so startup takes 75 seconds; the service runs 12 instances updated one at a time, and traffic spikes reach their peak within 40 seconds. Compute how long a deploy takes and how the autoscaler fares during a spike, explain what can go wrong when several instances start against the same pending migration, and redesign startup and migrations according to Factors VI, IX and XII, giving the order of steps in the pipeline.

7. A containerised service writes its logs to `/var/log/app/app.log` through an in-process rotation library with a 64 KB write buffer. On Kubernetes, logs from pods that were killed for memory are missing their last minutes, logs vanish when pods are rescheduled, and engineers debug by opening shells into ten pods and searching files. Explain each symptom, state what changes in the code and what moves to the platform under Factor XI, and describe how a single request that touched three instances should then be traced through the merged stream (see Chapter 14).
8. A team develops on SQLite through an ORM, runs Postgres in production and stores an activity feed in DynamoDB. A search for `dune` returns three books on every laptop and one in production, with no error or log entry, and a DynamoDB query that passes against an emulator is throttled in production. Explain both discrepancies, then design development and test environments that close the relational gap completely and narrow the DynamoDB gap, weighing cost against fidelity and deciding which tests must run against the real managed service (see Chapter 23).

APPENDIX B GLOSSARY

Term	Meaning
Twelve-factor app	Twelve rules (Heroku, 2011) forming the contract between a network service and its platform
Software erosion	Gradual failure of unchanged code caused by drift in its environment, typical of hand-built snowflake servers
Deploy	One running instance of the app: production, staging, a PR preview, a laptop
Manifest	File declaring all dependencies: `package.json`, `go.mod`, `requirements.txt`, `Gemfile`
Lock file	File fixing the exact version of every direct and transitive dependency
Config	Everything that varies between deploys; kept out of code, in granular environment variables
Backing service	Any service used over the network: database, cache, queue, SMTP, object storage, third-party API
Resource handle	URL plus credentials that locate an attached backing service
Build	Executable artefact (binary, bundle, image) of code and dependencies, without config
Release	A build combined with one deploy's config; uniquely numbered and immutable
Sticky session	Load-balancer pinning of a user to one instance; forbidden by Factor VI
Process formation	The process types of an app and the number of running copies of each
Disposability	Ability of a process to start fast and be stopped at any moment without harm
Re-entrancy	Property of a job that can be interrupted and run again without inconsistent state
Dev/prod parity	Keeping development, staging and production close in time, people and tools
One-off process	Admin task, such as a migration or data fix, run once from the same release as the app

APPENDIX C FURTHER TOPICS

Liveness, readiness and startup probes

Orchestrators such as Kubernetes call health endpoints or commands to decide whether a container should receive traffic (readiness), must be restarted (liveness) or is still booting (startup). They are the platform's half of disposability: a rolling deploy routes requests to a new process only after it reports ready, and a deadlocked process that never crashes still gets restarted. The endpoints need care, since a readiness check that pings the database can take every instance out of rotation during a brief database outage. They apply to any service run by an orchestrator or behind a health-checking load balancer.

Expand and contract migrations

Expand and contract, also called parallel change, splits a breaking schema change into compatible steps: add the new column or table, deploy code that writes both and reads the new one, backfill, and only later remove the old structure. During a rolling deploy old and new releases run side by side against one database, and an instant rollback is safe only if the previous release still understands the schema. Factor XII settles where migrations run; this technique settles what they may change at each step. It applies to every zero-downtime deployment of a service with a relational database.

Blue-green and canary deployments

In a blue-green deployment a complete second environment runs the new release and traffic switches to it at once, with the old environment kept ready for rollback. A canary deployment sends a small share of traffic to the new release and widens it while error rates and latency stay healthy. Both build on immutable releases and disposable processes and shrink the damage a bad release can do before anyone notices. They pay off when a failed deploy costs more than the extra infrastructure and traffic management they require.

Feature flags

Feature flags are runtime switches that enable a code path for some or all users without a new deploy. They separate deploying code from releasing a feature, support trunk-based development, and let a misbehaving feature be switched off in seconds. Unlike the configuration of Factor III, flags often vary per user and change while the process runs, so they are usually served by a flag service rather than by environment variables. Stale flags must be removed, or they turn into hidden branches nobody tests.

Workload identity and short-lived credentials

Instead of distributing static passwords and API keys, a cloud platform can give each workload an identity, such as an IAM role for a container or a Kubernetes service account federated through OIDC, and issue short-lived credentials on demand. Access to databases and cloud APIs then needs no long-lived secret at all, neither in environ-

ment variables nor in a secrets manager. This removes the leak paths discussed under Factor III and makes rotation automatic. It applies whenever the platform supports it, which today includes all major clouds.

Reproducible builds and supply-chain provenance

Tags such as `node:latest` are mutable pointers; pinning base images by content digest, committing lock files and producing a software bill of materials (SBOM) make a build traceable and repeatable. Frameworks such as SLSA define levels of build provenance: signed statements of which source and which pipeline produced an artefact. They extend Factors II and V from "the same build runs everywhere" to "the contents of the build can be proven". They matter for security audits, for responding to a vulnerable dependency and in regulated industries.

Infrastructure as code and GitOps

Infrastructure as code (Terraform, OpenTofu, Pulumi, CloudFormation) describes servers, networks, databases and queues in version-controlled files that a tool applies. GitOps goes further: a controller such as Argo CD or Flux continuously reconciles a cluster with the state declared in a Git repository. Together they apply Factor I and the argument against snowflake servers to the platform itself, so that environments can be reviewed, reproduced and rolled back like code. They become necessary as soon as several environments or several engineers share the infrastructure.

Beyond the Twelve-Factor App

In 2016 Kevin Hoffman proposed fifteen factors for cloud-native applications, revising the original twelve and adding API first, telemetry, and authentication and authorization. The additions cover what the 2011 text left out: contracts between services designed before their implementation, metrics and traces next to logs, and security as a property of every service. Comparing the two lists shows which factors stayed stable and which grew with microservices. It is useful background for judging the 2024 community revision.

Container resource limits and runtime awareness

A container's CPU and memory are limited through cgroups, but language runtimes have not always read those limits: Go before version 1.25 sized `GOMAXPROCS` from the host's CPU count, and older JVMs sized their heaps from host memory. The results are CPU throttling and out-of-memory kills, exactly the sudden deaths Factor IX asks processes to survive. Setting resource requests and limits deliberately, and configuring the runtime to match, makes scaling by process count predictable. It applies to every containerised backend.

OpenAPI — One-Page Overview

Chapter 25 · every section and sub-section of the paper at a glance

1 Introduction: one API, three sources of truth

Endpoint knowledge copied into handler, frontend and prose drifts; OpenAPI gives remote calls a signature.

- **1.1 Where endpoint knowledge lives** — For `POST /api/move`, handler, frontend and a document each hold the fields and status codes.
- **1.2 How the copies drift** — Renaming `to` updates the code but not the document; the next integration gets a 400.
- **1.3 Signatures stop at the network** — Local signature changes fail at compile time; HTTP calls fail only at runtime.
- **1.4 OpenAPI** — A programmatic JSON or YAML description of every endpoint's inputs, outputs and behaviour.

2 Origins, purpose and vocabulary

From Wordnik's self-describing JSON in 2010 to a Linux Foundation standard.

- **2.1 Wordnik** — Customers wanted SDKs and docs in many languages; programs read one machine-readable file.
- **2.2 What the file provides** — Interactive docs, generated clients and server code, automatic request validation.
- **2.3 The official definition** — Language-agnostic, for humans and computers, no source code or traffic inspection needed.
- **2.4 OAS, OAD, OAI, Swagger** — The rules, your file, the organisation, and the old name now used for tools.
- **2.5 JSON or YAML** — Equivalent formats; YAML is preferred because it allows comments.
- **2.6 History and versions** — Swagger 2.0 = OpenAPI 2.0 (renamed 2016); 3.2.0 is current, tools favour 3.1.

3 Anatomy of an OpenAPI description

Top-level fields, paths, operations, inputs, responses and schemas, shown on the task board.

- **3.1 The minimal document** — `openapi`, `info`, `paths`; specification version versus API version.
- **3.2 Servers** — Base URLs for local, staging and production; paths stay relative.
- **3.3 Paths** — Keys start with a slash; one entry per method; `{id}` marks a dynamic segment.
- **3.4 Operations** — Summary, Markdown description, tags and a unique, function-style `operationId`.
- **3.5 Parameters** — `in`: path, query, header or cookie; path parameters are always required.
- **3.6 Request bodies** — `content` maps media types to schemas; `actor` defaults to guest.
- **3.7 Responses** — Quoted status codes with description and body; the move's 200, 400, 404 and 502.
- **3.8 Schema objects** — Types, lengths, patterns, enums, ranges, items and properties nest freely.

4 Code first or design first

Generate the file from code, or write the contract first and build both sides from it.

- **4.1 Two orders of work** — Code first makes the file a byproduct; design first makes it the source of truth.
- **4.2 Contract first for new projects** — Frontend, backend and parallel agents start from agreed rules.
- **4.3 Zod and ts-rest** — Schemas and a contract in TypeScript generate `openapi.json` without comments.
- **4.4 FastAPI** — Handler annotations produce and serve the description.

5 A design-first pipeline for the task board

One file drives linting, mocking, clients, server interfaces, documentation and tests.

- **5.1 One file, six consumers** — A small new project; everything conforms to one `openapi.yaml`.
- **5.2 Linting with Spectral** — Flags a missing contact, missing descriptions and operation IDs.
- **5.3 Mocking with Prism** — Answers with the file's examples before the backend exists.
- **5.4 A typed client** — openapi-typescript and openapi-fetch; a wrong field name fails compilation.
- **5.5 Server interfaces** — oapi-codegen emits a Go interface the handlers must implement.
- **5.6 Documentation** — Swagger UI, ReDoc and Scalar render the same file.
- **5.7 Testing the server** — Schemathesis sends valid and invalid requests and checks every response.

6 OpenAPI as an industry standard

Gateways, published platform descriptions and AI agents.

- **6.1 Clients, gateways, platforms** — GitHub's description has about 344,000 lines, Stripe's about 171,000.
- **6.2 AI agents** — Named as a use case; the recording breaks off at this point.

OpenAPI at a glance — An HTTP call has no compile-time signature, so endpoint knowledge copied into code and prose drifts; an OpenAPI description restores that signature as one machine-readable file from which documentation, clients, server interfaces, mocks and tests are all derived. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

OpenAPI: A Contract Across the Network Boundary

Chapter 25 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Knowledge of an HTTP endpoint usually lives in three copies (the handler, the frontend call and a prose document) and drifts, because unlike a local function call a network call has no signature a compiler can check. The chapter introduces OpenAPI, a language-agnostic JSON or YAML description of every endpoint, traces its origin from Wordnik's Swagger to the OpenAPI Initiative, and separates the specification, the description, the organisation and the Swagger tools. It then dissects a description of the task-board API: top-level fields, servers, paths, operations, parameters, request bodies, responses and schema objects. Code-first and design-first workflows are compared, with Zod, ts-rest and FastAPI as code-first examples. A design-first pipeline follows in which one file feeds Spectral, Prism, openapi-typescript, oapi-codegen, three documentation renderers and Schemathesis, and the chapter closes with OpenAPI's role across the industry.

Keywords — OpenAPI, Swagger, API contract, JSON Schema, design first, code generation, mock server, contract testing

1 INTRODUCTION: ONE API, THREE SOURCES OF TRUTH

An HTTP API is a contract: a client must know which routes exist, what each accepts and what each can answer. The question behind this lecture is where that knowledge physically lives, and what happens when it lives in more than one place.

1.1 Where endpoint knowledge lives

The task-board project of the real-time lecture (see Chapter 22) has about six routes. One of them, `POST /api/move`, accepts a JSON body with three fields, `taskId`, `to` (the target column) and `actor`, and answers with a status code that depends on what happened in the service and repository layers: 200 on success; 400 or 404 for client errors such as an invalid body or a missing task; 500 or 502 when something fails on the server side. The React frontend knows all of this: it calls `/api/move`, sends the three fields and interprets 404 as "the task does not exist".

That knowledge therefore exists in three copies (Figure 1). The **backend handler** holds it because it is the processing point: it reads and checks the body. The **frontend** holds it because it is the point of call, a `fetch` or Axios call or whatever library is used. And if the engineer who wrote or integrated the API keeps a **document** of its requirements and expectations, the knowledge lives there as well.

1.2 How the copies drift

Three copies of one truth must be synchronised by hand. Suppose a colleague renames `to` to `column` because it reads more naturally. They must update the handler, and they must update the frontend, otherwise the backend's validation rejects every request. The document, however, enforces no behaviour; it is only reference material, so it is the copy that gets forgotten. Some sprints later an engineer who has never touched this endpoint integrates it into another page. They consult the document rather than the handler, which is the real source of truth, send `to`, and receive a validation error. The defect is not the rename but the existence of several sources of truth for one endpoint.

```
KNOWLEDGE OF POST /api/move LIVES IN THREE PLACES

1 backend handler   reads taskId, to, actor; validates
2 frontend call     fetch/axios sends the same fields
3 API document      prose kept by an engineer

A colleague renames "to" -> "column":
  handler    updated    (it enforces the field)
  frontend   updated    (or every request fails)
  document   FORGOTTEN  (it enforces nothing)

Sprints later a new engineer follows the document:
  POST /api/move {"taskId": "t7", "to": "done"}
  -> 400 validation error, found only at runtime
```

Figure 1: Three copies of the knowledge of `POST /api/move`, and how a rename breaks the forgotten one (illustrative).

1.3 Signatures stop at the network boundary

This class of bug appears only where a call leaves the codebase. Inside a Go backend a function has a signature that fixes its argument types and its return value. If someone renames a field of a struct the function takes, or changes a type from `string` to `int`, every call site fails at *compile time*, so the break is found and fixed during development. A remote call across the HTTP boundary has no signature: a renamed field or a changed type is discovered only at *runtime*, by a failing request.

The aim is to give remote calls the same protection. That means capturing the "signature" of every endpoint (the available routes, the path and query parameters, the request-body fields down to the type of each field, and every kind of response) in one central document. That document is not prose or Markdown but a *programmatic* description that tools can read and enforce across the network boundary. The specification that defines it is OpenAPI.

1.4 OpenAPI

OpenAPI description. A single file, in JSON or YAML, that lists every endpoint of an API and, for each one, describes its inputs, its outputs and its behaviour in a form that both people and programs can read

2 ORIGINS, PURPOSE AND VOCABULARY

2.1 Wordnik and the self-describing server

In 2010 the startup Wordnik had an API and about six engineers. Its customers wanted client SDKs in several languages (Python, PHP, JavaScript), each with matching documentation. Tony Tam, who started what became the OpenAPI Specification, summed up the motivation in one sentence: "we often write code faster than we have time to document it".

The idea was that a server *describes itself* in a JSON file and that separate programs consume that file for different purposes. One renders it as clear, attractive, useful documentation; another generates client libraries for PHP, Python, JavaScript, Go, Rust or any other language. This works because the file is only a description of the backend and imposes no programming language: each generator builds the client from its own language's primitives. The one requirement is a guarantee that the file truly reflects the backend. The description is JSON or YAML rather than text precisely so that any program can parse, search, transform and render it; prose offers no comparable programmatic access.

2.2 What one machine-readable file provides

- **Documentation.** Either read-only, so that an engineer can understand the API and integrate it in any language, or interactive, as in Swagger UI, Scalar and many other tools: the reader fills in a sample request body, adds a token, sends the call and inspects the response.
- **Client libraries.** A generator reads the file and emits ready-to-call functions, so nobody hand-codes or hand-documents each condition and property of each endpoint.
- **Server code.** On the server side the file generates interfaces and boilerplate that would otherwise be written, maintained and synchronised by hand.
- **Request validation.** Modern libraries validate path parameters, query parameters and request bodies against the file automatically. Because the file already holds the server's complete requirements, the validation code of Chapter 5 need not be written a second time.

The lecturer calls this the single most productive addition to a codebase, because it helps backend, frontend and infrastructure work and every engineer involved in one way or another.

2.3 The official definition

The specification defines itself as a standard, language-agnostic interface to HTTP APIs that allows both humans and computers to discover and understand the capabilities of a service without access to source code, documentation or inspection of network traffic. Each clause carries weight. *Language-agnostic*: the description does not care whether the server is written in Go, Node.js, Python, Java or PHP, because it records behaviour, requirements and output, not implementation. *Humans and computers*: the same file yields interactive documentation for people and drives code generation and validation in programs without human intervention. *No source code or traffic inspection*: nobody has to read the handler, trust possibly outdated prose or open the browser's network tab to learn how the API responds and how it reports validation errors.

2.4 OAS, OAD, OAI and Swagger

Four names are routinely confused (Table 1). The specification is the rulebook; the description is the physical file that must obey every rule; the initiative is the organisation that publishes and evolves the rules; Swagger was the original name of the specification and is now the name of a family of tools. A "Swagger file" therefore means the description, `openapi.json` or `openapi.yaml`, whereas "Swagger UI" means the tool that renders that file graphically and lets one test the API.

Name	Stands for	What it is
OAS	OpenAPI Specification	The rules: which fields exist and what they mean
OAD	OpenAPI Description	Your file, `openapi.json` or `openapi.yaml`
OAI	OpenAPI Initiative	Body that publishes and maintains the OAS; part of the Linux Foundation
Swagger	Former name	The spec's old name; now tools such as Swagger UI

Table 1: Four names that are often confused.

2.5 JSON or YAML

A description is a single JSON object, which may be written in JSON or in YAML. Both formats have the same feature set and tools generally accept either, so the choice depends only on tooling. Most teams choose YAML: it is a superset of JSON and, unlike JSON, allows comments (Figure 2).

```
JSON form:
{
  "openapi": "3.1.1",
  "info": {"title": "Task Board API", "version": "1.0.0"},
  "paths": {}
}

YAML form of the same object:
openapi: 3.1.1          # version of the specification
info:
  title: Task Board API
  version: 1.0.0        # version of this API
paths: {}               # no endpoints yet
```

Figure 2: The smallest valid description, as JSON and as the equivalent YAML with comments (illustrative).

2.6 History and versions

Year	Event
2010	Wordnik describes its API in JSON and builds a page that renders a UI from it; a team designer names it Swagger
2011	Swagger 1.0 released as open source
2014	Swagger 2.0; broad adoption begins
2015	SmartBear buys Swagger and donates the specification to the Linux Foundation; founding members include Google, Microsoft, IBM and PayPal
Jan 2016	Renamed OpenAPI with no change of content
Today	3.2.0 is current; most tools still target 3.1

Table 2: From Swagger to OpenAPI.

Because the 2016 rename changed no content, OpenAPI 2.0 and Swagger 2.0 are the same specification. At the time of recording the current version is 3.2.0, yet the lecturer still writes 3.1: the tools that render documentation, generate client libraries or produce type definitions for the frontend mostly support only 3.1, and a description is useful only to the extent that its tools can read it.

3 ANATOMY OF AN OPENAPI DESCRIPTION

3.1 The minimal document and its two versions

The smallest valid description has three fields (Figure 2). `openapi` states which version of the specification the file conforms to; this matters because many versions have appeared over the years and tools still target specific ones rather than all having moved to 3.1. `info` carries the API's title and version. `paths` lists the endpoints and may start empty. Two different versions thus appear in one file: `openapi: 3.1.1` is the version of the *specification*, while `info.version` is the version of *your API*, the internal versioning discussed with routing (see Chapter 2). `openapi` and `info` are required, and at least one of `paths`, `components` or `webhooks` must be present (Table 3).

Field	Required	Purpose
`openapi`	yes	Specification version, e.g. 3.1.1
`info`	yes	Title and the API's own version
`paths`	one of three	Endpoints and their operations
`components`	one of three	Reusable definitions
`webhooks`	one of three	Requests the API sends out (Chapter 26)
`servers`	no	Where the API is hosted
`security`	no	Authentication for all endpoints
`tags`	no	Groups of endpoints

Table 3: Top-level fields of a description.

3.2 Servers

`servers` is a list, because the same backend can be hosted in several places: a developer's machine, staging, production. For the locally running task board the entry is `http://localhost:8080`. Since the base URL is written once here, every path below is written relative to it and never repeats the host.

3.3 Paths and path templating

`paths` is a map whose keys are endpoint paths, each beginning with a slash. The value under a key is a *path item*: everything that can be done at that address, with one entry per HTTP method. The task board has `get` under `/api/board` and `post` under `/api/move`; any method may appear: `get`, `post`, `put`, `patch`, `delete`, and where needed `head` and `options`.

A path may contain a dynamic part. HTTP semantics place the identifier of a specific entity in the path rather than in the query string (see Chapter 2), so fetching one task means `/api/tasks/1`, `/api/tasks/2` or `/api/tasks/<uuid>`. The identifier cannot be documented in advance, because only the client knows it, so OpenAPI marks such a segment with curly braces: `/api/tasks/{id}` (Figure 3).

3.4 Operations

Each method under a path is an *operation*. It has a `summary` that fits on one line and a longer `description` that supports Markdown, so a table or other formatting can be added where it helps. `tags` let documentation tools group operations, for example every task-related endpoint in one section. `operationId` gives the operation a unique name, and best practice is to write it as a function name (`getBoard`, `moveTask`), because code generators turn each operation into a real function or method named after it (Section 5.5). Clients then call that function instead of implementing the endpoint's details themselves, which is why the name must have function-name format and be unique across the entire file. An operation further holds its `parameters`, `requestBody` and `responses`.

```
servers:
  - url: http://localhost:8080
    description: local development
paths:
  /api/board:
    get:
      operationId: getBoard
      summary: Return every column with its tasks
      tags: [board]
  /api/tasks/{id}:             # {id}: dynamic segment
    get:
      operationId: getTask
      parameters:
        - name: id
          in: path
          required: true       # always true for path
          schema: {type: string}
  /api/poll:
    get:
      operationId: pollEvents
      parameters:
        - name: since
          in: query
          required: false
          description: last sequence number received
          schema: {type: integer, minimum: 0}
  /api/events:
    get:
      operationId: streamEvents
      parameters:
        - name: Last-Event-ID
          in: header           # sent by the browser
          schema: {type: string}
```

Figure 3: Servers, paths, a templated path and parameters in three locations for the task board (illustrative).

3.5 Parameters

An operation has two kinds of input. *Parameters* are named values whose location is given by the `in` field: `path`, `query`, `header` or `cookie` (Table 4). Whatever the location, they are all simply parameters. Each has a `name`, its `in`, a `required` flag stating whether it must be present, and a `schema` describing its value, which validators then enforce. The task board's polling endpoint takes `since`, an integer query parameter holding the last sequence number the client received. Its server-sent-events endpoint receives the same value in the `Last-Event-ID` header, which the browser itself sends when it re-establishes the stream (see Chapter 22).

Path parameters are always required. An optional path segment would make the URL meaningless: `/api/tasks/{id}` with and without the id are two different URLs, that is, two different routes, each needing its own path item.

`in`	Task-board example	Required
path	`id` in `/api/tasks/{id}`	always
query	`since` on the polling endpoint	optional or required
header	`Last-Event-ID` on the stream	optional or required
cookie	a session cookie	optional or required

Table 4: Parameter locations.

3.6 Request bodies and media types

The second kind of input is the `requestBody` carried by `post`, `put` and `patch`. It is described by `content`, a map from *media type* (the value that travels in the `Content-Type` header) to the schema of the body. Three media types cover most APIs: `application/json` for sending and receiving JSON; `multipart/form-data` for file uploads; and `application/x-www-form-urlencoded` for classic HTML forms, very common before single-page applications and still usable today. Each media type listed needs its own schema.

The `moveTask` operation accepts `application/json`, JSON serialised as a string for the network (see Chapter 3), with three fields (Figure 4). `taskId` and `to` are listed as required, so the file states that the backend rejects a body without them. `actor` is optional with `default: guest`: if the client omits it, the backend records the move as made by `guest`. A reader learns this behaviour without opening the backend code, which is the point of the file. Strictly, `default` is an annotation: it documents what the server does, and the handler (or a validator configured to fill defaults) must still do it.

```
  /api/move:
    post:
      operationId: moveTask
      summary: Move a task to another column
      description: |
        Saves the move, then publishes an event.
        **502** means saved but **not** published.
      tags: [tasks]
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [taskId, to]
              properties:
                taskId: {type: string}
                to:     {type: string}
                actor:  {type: string, default: guest}
      responses:
        "200":
          description: Moved; returns the event
          content:
            application/json:
              schema: {$ref: "#/components/schemas/Event"}
        "400": {description: Invalid body}
        "404": {description: No task with this id}
        "502": {description: Saved, but not published}
```

Figure 4: The `moveTask` operation with its request body and four documented responses; error bodies are omitted (illustrative).

3.7 Responses

`responses` maps status codes to outputs. The code tells the client how to interpret the answer: 2xx success, 3xx redirection, 4xx client error, 5xx server error (see Chapter 1). Every response must have a `description` and usually has `content`, a map from media type to schema of the same form as the request body but describing the response body; `$ref` in Figure 4 points to a reusable schema kept under `components`.

`moveTask` documents four outcomes. With 200 the client receives the resulting event in the event schema. With 400 it receives an error body instead of an event: the request was invalid. With 404 no task has the given id, again with an error body. With 502 the move itself succeeded, but publishing the event to the message bus failed. The last case shows why responses deserve documentation: a client

that read every 5xx as "nothing was saved" would retry a move that is already persisted, whereas the description tells it otherwise.

> **Quote the status codes.** Write response keys as strings, `"200"`, not `200`. JSON object keys are always strings, but in YAML an unquoted `200` is an integer; the specification requires the quotation marks so that JSON and YAML descriptions stay equivalent, and tools may otherwise misread the key

3.8 Schema objects

The schema object is the part of the file clients use most, because it documents every data type and field. Its `type` is one of `string`, `number`, `integer`, `boolean`, `array`, `object` or `null` (Table 5). An object has `properties` whose keys are strings and whose values are again schemas of any type. A string can be bounded with `minLength` and `maxLength`, constrained by a regular-expression `pattern`, or restricted by `enum` to a fixed set of values, for example a `status` that may only be `active`, `inactive` or `deleted`. Numbers take `minimum` and `maximum`. An array declares `items`, itself a schema of any type, including another array or object. Because schemas nest, combining these keywords describes almost any data structure. In OpenAPI 3.1 the schema object is JSON Schema (draft 2020-12), so the keywords behave exactly as JSON Schema defines them.

`type`	Typical keywords	Example
string	minLength, maxLength, pattern, enum	status: active, inactive, deleted
number, integer	minimum, maximum	`since` ⩾ 0
boolean	none	a done flag
array	items (a schema)	the tasks of a column
object	properties, required	the move request body
null	none	an absent value

Table 5: Schema types and their main keywords.

A further part of the file describes authentication: which kind of token or credential the client sends. Schemes are declared once and applied to every operation through the top-level `security` field, or to individual operations. The lecturer leaves the remaining properties aside, because in practice they are rarely written by hand (Section 4.3).

4 CODE FIRST OR DESIGN FIRST

4.1 Two orders of work

If the description is the contract on which both server and client base their behaviour, which comes first: the file or the code? In **code first**, the handlers, services and logic are implemented completely, and the description is then generated from the code, through annotations or comments directly above the handlers or from the types themselves; most ecosystems have libraries for this. The file becomes a byproduct of the code. In **design first**, also called **contract first**, the description is written before implementation begins; the backend is built next, and client code, mocks and tests are derived from the same file. The contract is the single source of truth (Table 6).

Aspect	Code first	Design first
Order	Implement, then generate the file	Write the file, then implement
File comes from	Annotations, comments or types	Hand-written or a contract library
Source of truth	The code	The contract
Client work	Waits for the server	Starts in parallel
Typical fit	Existing servers	New projects

Table 6: Code first and design first compared.

4.2 Contract first for new projects

The Swagger team began code first because they already had a server and wanted the description derived from it; later they concluded, as a 2013 report states, that contract first was the more logical order. The lecturer's experience matches: code first is what one does when a server already exists and a description is added on top of it, for interactive testing, client generation or another reason. For almost every new project he chooses contract first, because it makes many workflows more efficient. Once the rules that server and client will follow are agreed, frontend and backend development start in parallel; the integration does not wait for a finished backend.

The argument is stronger with coding agents. One first works with an agent to settle how each endpoint behaves: what it accepts and what it returns. When the contract is ready, two agents run in parallel, one on the server and one on the frontend, and neither blocks the other.

4.3 Code first with Zod and ts-rest

The Tasker application from the lecturer's playlist on professional Go development shows a code-first setup that does not rely on comments (Figure 5). **Zod** schemas define the OpenAPI schemas as structured TypeScript objects. A **ts-rest** contract associates these schemas with paths, methods and status codes. A generator function takes the contract and produces `openapi.json`, which the Go server then serves for interactive documentation and other purposes. Nobody writes `openapi.json` by hand.

This is why the lecture does not tour every feature of the specification. With libraries like ts-rest an engineer must express only three essentials: the endpoint path; its inputs (query and path parameters, headers, request body); and each response's schema and status code. No OpenAPI-specific pattern has to be memorised; it is ordinary JavaScript, written as a JSON Schema object or, more briefly, with Zod. The lecturer prefers this to the comment-based variant of code first, in which comments above the handlers are parsed into the file: code is easier to maintain than comments and causes fewer problems in many scenarios.

```
// schemas.ts: Zod objects instead of hand-written JSON
const Task = z.object({
  id: z.string(),
  title: z.string().min(1).max(200),
  status: z.enum(["todo", "in_progress", "done"]),
});

// contract.ts: ts-rest binds schemas to paths/methods
export const contract = c.router({
  getTask: {
    method: "GET",
    path: "/api/tasks/:id",
    responses: { 200: Task, 404: ErrorBody },
  },
});

// generate.ts: the contract becomes openapi.json
const doc = generateOpenApi(contract,
  { info: { title: "Tasker", version: "1.0.0" } });
writeFileSync("openapi.json", JSON.stringify(doc));
```

Figure 5: Code first without comments: Zod schemas, a ts-rest contract and a generator (illustrative; `c` comes from `initContract()`, `generateOpenApi` from `@ts-rest/open-api`).

A subtle point. The lecture files ts-rest under code first, yet the contract is not derived from the Go handlers: it is written separately, and the Go server merely serves the generated file. In workflow terms it is a contract expressed in TypeScript instead of YAML, and the lecturer himself later names ts-rest as one way to produce the design-first file (Section 5.1). What separates the approaches is whether the description is derived from the implementation or precedes it, not whether it is written in code. With a Go server, nothing then checks the handlers against the TypeScript contract unless a later stage such as Schemathesis (Section 5.7) does

4.4 Code first in FastAPI

Python's FastAPI integrates OpenAPI natively. The framework parses the handler code itself, through its type annotations, builds the OpenAPI JSON in the background and serves it, so the description follows the whole handler code rather than separate comments. By default it is served at `/openapi.json`, with interactive pages at `/docs` and `/redoc`.

5 A DESIGN-FIRST PIPELINE FOR THE TASK BOARD

5.1 One file, six consumers

The task-board application of Chapter 22 has no description yet. It is small and new, with few endpoints, which is exactly the situation in which design first costs least and is worth making a habit. The pipeline has five stages plus a verification step, and each one reads the same file (Figure 6). The file may be written by hand in YAML or JSON, or generated from a ts-rest contract; either way one `openapi.yaml` or `openapi.json` is the single source of truth, and the mock server, the client code and the server implementation must all adhere to it.

```
hand-written YAML or JSON    ts-rest + Zod + generator
           |                              |
           +--------------+---------------+
                          |
        openapi.yaml   (the single source of truth)
                          |
stage  job      tool                  produces
1      lint     Spectral              rule violations
2      mock     Prism                 a fake server
3      client   openapi-typescript    TS types and
                + openapi-fetch       typed requests
4      server   oapi-codegen          Go interfaces
5      docs     Swagger UI, ReDoc,    interactive
                Scalar                reference
6      verify   Schemathesis          mismatches of the
                                      running server
```

Figure 6: The design-first pipeline; every stage reads the same file (illustrative).

5.2 Stage 1: linting with Spectral

Just as ESLint and the Go and Python linters check code, an OpenAPI linter reads the description and reports problems such as a missing description, an operation without an `operationId`, or a response without a schema. The lecture uses **Spectral**, which ships with the OpenAPI rules built in, so one run tells whether the file meets the specification. In the demo, Spectral reports three problems: the `info` object lacks contact information, and two operations lack a description. After the fixes, a second run passes cleanly. Each rule carries a severity; style rules such as the missing contact are typically warnings, and the team's ruleset decides which severities fail a CI run.

5.3 Stage 2: a mock server with Prism

A mock server reads the specification, written before any backend code exists, and starts a fake server that answers every request the file documents, using the examples written in the file; its answers are therefore only as realistic as those examples. It is available before the implementation. If frontend and backend are separate teams, the frontend team builds against the mock while the backend team writes the handlers from start to finish. The lecture uses **Prism**: with nothing but the file, a request for the board returns the example board from the description, and the move operation accepts exactly the request schema the file defines.

5.4 Stage 3: a typed client

For a JavaScript client the lecture pairs **openapi-typescript**, which converts the description into TypeScript types, with **openapi-fetch**, which performs the API requests using those types (Figure 7). The generated types know every path, every endpoint, every field, its type and its constraints. A misspelled field name or a value of the wrong type is a compiler error: the protection that Section 1.3 found missing at the network boundary, now restored on the client side.

```
$ npx openapi-typescript openapi.yaml -o src/api.d.ts

import createClient from "openapi-fetch";
import type { paths } from "./api";   // generated types

const api = createClient<paths>({
  baseUrl: "http://localhost:8080",
});

const { data, error } = await api.POST("/api/move", {
  body: { taskId: "t7", column: "done" },
});
// tsc: 'column' does not exist in the body type,
//      and the required property 'to' is missing
```

Figure 7: A client typed by openapi-typescript and called through openapi-fetch; a field that is not in the contract fails to compile (illustrative).

A subtle point. The lecture says the frontend uses the generated types "at runtime". TypeScript types are erased by the compiler, so their protection is entirely at compile time: they check the code the frontend writes, not the data the server actually returns. If the server drifts from the file, the client still compiles; catching that drift is the job of Section 5.7 or of a runtime schema validator

5.5 Stage 4: server interfaces with oapi-codegen

On the Go side, **oapi-codegen** converts the file into Go types and a server interface, and whatever handler code is written must satisfy that interface (Figure 8); other languages get their own equivalent structures. Note that the optional `actor` becomes a pointer, so "absent" and "empty" remain distinguishable. The behaviour documented in the contract is thereby turned into a type the compiler checks, which is what "design first" means: the handlers conform to the contract, not the other way round. If the contract file later changes and the handlers do not, compilation fails. In the demo a field is added to the file, the code is regenerated, and the build breaks at the handler level, because the implementation no longer conforms to the contract.

```
$ oapi-codegen -generate types,chi-server -package api \
      openapi.yaml > api/api.gen.go

// api.gen.go: generated, never edited by hand
type MoveTaskJSONBody struct {
    TaskId string  `json:"taskId"`
    To     string  `json:"to"`
    Actor  *string `json:"actor,omitempty"`
}

type ServerInterface interface {
    // (GET /api/board)
    GetBoard(w http.ResponseWriter, r *http.Request)
    // (POST /api/move)
    MoveTask(w http.ResponseWriter, r *http.Request)
    // (GET /api/poll)
    PollEvents(w http.ResponseWriter, r *http.Request,
        params PollEventsParams)
}

// handlers.go: hand-written; fails to compile if
// *Server stops satisfying the generated interface
var _ api.ServerInterface = (*Server)(nil)
```

Figure 8: oapi-codegen turns the contract into Go types and an interface that the handlers must implement (illustrative).

A subtle point. The compiler catches only contract changes that alter what the handler code touches: a new operation adds a method to the interface, a new or renamed parameter changes a method signature, and a renamed or retyped field breaks the code that uses it. A newly added response field that the handler never sets still compiles, and Go silently sends its zero value or omits it. Generated interfaces fix the shape of the server; Section 5.7 checks its behaviour

5.6 Stage 5: documentation

The same file is rendered for people in three ways. **Swagger UI** is the traditional interactive API reference, with an optional "Try it out" button that sends real requests. **ReDoc** renders the same file with a different user interface. **Scalar** also turns the file into an interactive environment for testing the API; it is the lecturer's personal choice for its modern, clean interface.

5.7 Verification: testing the server with Schemathesis

None of the earlier stages shows that the running server actually matches the file. The first step towards that is to test the server against the file. **Schemathesis** reads the description, generates requests from its schemas, both valid and deliberately invalid, sends them to the real server, and checks every response against the contract, flagging each place where documentation and implementation disagree. In the demo it derives requests from each schema and validates each response against the file. This is property-based testing driven by the contract (see Chapter 23), and it closes the loop of the pipeline: written once, the file is linted, mocked, turned into client and server code, rendered as documentation and finally used to test the server.

6 OPENAPI AS AN INDUSTRY STANDARD

6.1 Clients, gateways and published descriptions

The file serves more than one project's pipeline. It can be used to create an API client, and API gateways use it to validate requests before they reach a service. Major platforms publish their descriptions so that anyone integrating with them can use the file as documentation and build on it: GitHub's description has about 344,000 lines and Stripe's about 171,000, natural sizes for large platforms that have accumulated endpoints over many years. This is what is meant by calling OpenAPI *the* standard for APIs: the technology industry has settled on it as the way to describe backend systems.

6.2 AI agents

The lecturer names one more relevant use: giving an AI agent a tool to call. The recording breaks off in the middle of that sentence and continues directly with the closing summary. The connection is direct, however: an operation already holds what a tool definition needs (a unique name in `operationId`, a description, and a JSON Schema for its in-

puts), which is why agent platforms can expose OpenAPI operations as callable tools.

7 CONCLUSION

- Knowledge of an endpoint copied into the handler, the frontend and a prose document drifts; the forgotten copy breaks the next integration, and only at runtime.
- Local calls get compile-time checks from signatures; OpenAPI gives remote HTTP calls an equivalent, machine-readable signature.
- The description (OAD) is a JSON or YAML file that follows the specification (OAS) maintained by the OAI; "Swagger" is the old name and now a family of tools.
- `openapi` and `info` are required, plus one of `paths`, `components` or `webhooks`; the specification version differs from the API version.
- Operations carry a summary, a description, tags, a unique function-style `operationId`, parameters (path ones always required), a body per media type, and responses keyed by quoted status codes.
- Nested schema objects (types, lengths, patterns, enums, ranges, items, properties) describe data down to each field.
- Code first derives the file from code; design first writes the contract first, so frontend, backend and agents can work in parallel.
- In a design-first pipeline one file feeds Spectral, Prism, openapi-typescript, oapi-codegen, Swagger UI, ReDoc, Scalar and Schemathesis.
- Generated types catch shape errors at compile time; only testing the running server against the contract proves its behaviour.

APPENDIX A REVIEW QUESTIONS

1. A web app and a mobile app both call `POST /api/orders`. Two sprints ago the backend renamed `qty` to `quantity` in `openapi.yaml`, regenerated its Go server interface and the web client, and shipped; since then only users of older mobile builds receive 400 responses. Explain why every compiler involved was satisfied, which stages of a design-first pipeline could have stopped the release, and what contract-change policy you would adopt, connecting your answer with API versioning in Chapters 2 and 7.
2. A description declares the path `/api/tasks/{taskId}`, but its only parameter is `{name: id, in: path, required: false}`, and its success response key is an unquoted `200` in YAML. Predict how a linter, a mock server and a Go server generator would each treat this operation, and argue which of the defects could still reach production unnoticed if the team skipped the linting stage.
3. You inherit an Express backend with 60 undocumented endpoints and must ship a typed React client within two weeks; next quarter a new billing service will be built by separate frontend and backend teams, partly with coding agents. Choose code first or design first for each case, justify the choice from these constraints, and sketch the toolchain and the CI steps for both.
4. `POST /api/move` returns 502 when the move was persisted but the event could not be published to the message bus, and the frontend's HTTP wrapper retries every 5xx up to three times with exponential backoff. Trace what the user, the database and other connected clients observe during one such failure, and redesign the contract (status code, response body, idempotency) so that a generated client handles the case correctly without special-case code (see Chapters 12 and 22).
5. A team adds an optional integer `priority` with `default: 3` to the task response schema and regenerates both the oapi-codegen server and the openapi-typescript client; both builds pass. In production every task shows priority 0 in the mobile app and no priority at all in the web app. Explain the chain of events from the generated Go struct to the two clients, why no compiler complained, and which check would have caught the discrepancy before release.
6. A public API serves 5,000 requests per second and its schemas change weekly. Compare validating requests against the OpenAPI file at the API gateway, in validation middleware inside each service, and with hand-written validation in the handlers (see Chapter 5), with respect to latency, consistency with the contract, quality of error messages, and rules a schema cannot express, such as "the target column must exist".
7. A frontend team built screens for three weeks against a Prism mock; at integration half of them break, because the real server wraps the board as `{"data": {"columns": [...]}}` while the example in the file was `{"columns": [...]}`. Determine which artefact was wrong, how the process let both teams believe they were compliant, and design CI checks that would catch this class of mismatch on the day it is introduced.
8. Two coding agents work in parallel from one contract, one on the Go server and one on the React client. Midway the server agent finds that `GET /api/board` must be paginated because a board can hold 20,000 tasks. Specify the procedure for changing the contract (who edits the file, what is regenerated, what each pipeline stage reports to each agent) and how you would keep the client agent's existing work compiling during the transition.

APPENDIX B GLOSSARY

Term	Meaning
OpenAPI Specification (OAS)	Rules defining which fields a description may contain and what they mean
OpenAPI Description (OAD)	A concrete file following the OAS, usually `openapi.json` or `openapi.yaml`
OpenAPI Initiative (OAI)	Linux Foundation project that publishes and maintains the OAS
Swagger	Original name of the specification; today a family of tools such as Swagger UI
Path item	Everything that can be done at one path, one entry per HTTP method
Path templating	Curly braces marking a dynamic segment, as in `/api/tasks/{id}`
Operation	One method on one path, with summary, parameters, body and responses
`operationId`	Unique, function-style name of an operation, used by code generators
Parameter	Named input located in the path, query, a header or a cookie
Request body	Payload of post, put or patch, described per media type under `content`
Media type	The `Content-Type` value, e.g. `application/json`
Schema object	Description of a value's type and constraints; JSON Schema in OpenAPI 3.1
Code first	Generating the description from implemented code, annotations or types
Design first	Writing the description (the contract) before server and client; also contract first
Mock server	Fake server built from the description that answers with its examples
Contract testing	Sending generated requests to a real server and checking its responses against the description

APPENDIX C FURTHER TOPICS

Components and $ref

The `components` object holds reusable definitions (schemas, parameters, responses, request bodies, security schemes) that operations reference with `$ref`, a JSON Pointer such as `#/components/schemas/Task`. Reuse keeps one definition per entity, so a change to `Task` reaches every operation that returns it, and generators emit one named type instead of many anonymous ones. References may also point into other files, letting a large description be split across a repository and bundled for tools that need a single file. Any description with more than a handful of endpoints should define its entities here rather than inline.

OpenAPI 3.0 versus 3.1

Version 3.0 used an extended subset of an older JSON Schema draft with OpenAPI-specific keywords such as `nullable: true`; version 3.1 aligned the schema object with JSON Schema 2020-12, so a nullable string is written `type: [string, "null"]` and `exclusiveMinimum` takes a number instead of a boolean. The difference matters because many tools support only one of the two, and a file valid for one may be misread by a tool built for the other. Before choosing a version, check what the linter, generators and documentation renderers actually support.

Polymorphism: oneOf, anyOf, allOf and discriminator

Real payloads often come in variants: an event may be a move, a rename or a deletion, each with different fields. JSON Schema expresses this with `oneOf` (exactly one alternative matches), `anyOf` (at least one) and `allOf` (all, used for composition), and OpenAPI adds a `discriminator` naming the property, such as `type`, that identifies the variant. Generators turn these into union or tagged types, though support varies between tools and is a frequent source of awkward generated code. They apply whenever one endpoint accepts or returns several shapes.

Security schemes

Authentication is declared under `components.securitySchemes`, with types such as `http` (bearer or basic), `apiKey` (in a header, query or cookie), `oauth2` with its flows and scopes, and `openIdConnect`. The top-level `security` field applies schemes to all operations, and an operation-level `security` overrides it. Documentation tools use the declarations to offer an authorise dialog and generated clients to attach credentials, which keeps the authentication contract of Chapter 4 next to the endpoints it protects.

Detecting breaking changes

Because the description is data, two versions of it can be compared mechanically: tools such as oasdiff report removed endpoints, renamed or newly required fields and narrowed types as breaking changes. Running such a diff in CI against the last released description turns the rename incident of Section 1.2 into a failed pull request. It becomes essential once external or mobile clients exist that cannot upgrade in step with the server, and it combines naturally with API versioning (see Chapter 7).

Runtime validation middleware

Instead of generating code, a server can load the description at startup and validate every request, and optionally every response, against it, for example with the `openapi3-filter` package of the Go library kin-openapi or with express-openapi-validator in Node.js. This enforces the contract on the data that actually crosses the wire, including from clients that ignore the generated SDK, and yields consistent 400 errors without hand-written checks. Response validation is usually enabled in development and tests, where its cost is acceptable and a mismatch reveals drift early.

Webhooks and callbacks

An API also sends requests. Callbacks describe requests the server makes to a URL that the client supplied in an earlier call, and the top-level `webhooks` field, new in 3.1, describes out-of-band requests such as event notifications. Documenting them gives receivers the same schemas, generated types and validation as ordinary endpoints. They

apply to any backend that notifies other systems, the subject of Chapter 26.

AsyncAPI

OpenAPI describes request-response HTTP APIs and is a poor fit for a WebSocket channel or a topic on a message broker. AsyncAPI is a sister specification, also hosted by the Linux Foundation, that describes channels, messages and their schemas for event-driven systems over Kafka, MQTT, AMQP or WebSockets. A backend like the task board, which publishes events to a message bus and streams them to browsers, needs both specifications to describe its full contract.

Consumer-driven contract testing

Schemathesis checks a server against its description; consumer-driven contract testing, popularised by Pact, records what each client actually uses and verifies the provider against those recorded expectations. The two answer different questions: whether the server honours what it documents, and whether a change breaks any real consumer. Consumer-driven contracts suit organisations with many internal services and teams; schema-based testing suits public APIs whose consumers are unknown.

OpenAPI as tool definitions for AI agents

An agent platform needs, for each tool, a name, a description and a JSON Schema of its arguments, which is what an OpenAPI operation already contains, and some platforms import a description directly to give an agent access to an API. The quality of summaries and descriptions then matters more than ever, because the model chooses operations and fills parameters from them. Deciding which operations and scopes to expose is a security decision (see Chapter 16).

Webhooks — One-Page Overview

Chapter 26 · every section and sub-section of the paper at a glance

1 Introduction: Server-to-Server Events

A payment confirmed in Stripe's system must reach our server reliably; polling is slow and costly.

- **1.1 The payment problem** — The browser cannot receive the signal: unreliable network, closed tabs, tampering.
- **1.2 Polling and its costs** — Average delay T/2, wasted requests, 10,000 req/s per provider, rate limits.

2 Webhooks: Concept and Vocabulary

A webhook reverses the direction: the provider POSTs events to a URL we register.

- **2.1 Reversing the direction** — A plain HTTP POST to our endpoint; a callback between APIs.
- **2.2 Origin** — Jeff Lindsay, 2007: a Unix pipe for the web; PubSubHubbub two years later.
- **2.3 Vocabulary** — Provider, consumer, endpoint, event, delivery, subscription.

3 Receiving Webhooks in Practice

A GitHub demo shows the request, the success contract, registration, handshakes and tunnels.

- **3.1 A minimal receiver** — `POST /hooks/github`: read, verify, record, respond 202.
- **3.2 Anatomy of a delivery** — Event, delivery-ID and signature headers plus a JSON body.
- **3.3 What counts as success** — A 2xx within the deadline: GitHub 10 s, Shopify 5 s, Slack 3 s.
- **3.4 Registration and per-provider endpoints** — URL, secret, event types; one endpoint per provider.
- **3.5 Handshakes** — Token echo, ping/pong, confirmation URL; they prevent DDoS by proxy.
- **3.6 Local development and tunnels** — Routers block inbound traffic; use ngrok or Cloudflare Tunnel.

4 Verifying the Sender

A public endpoint must prove that each request came from the provider, recently.

- **4.1 The threat** — Guessable URLs, forged payments; HTTPS does not identify the sender.
- **4.2 Five ways to authenticate the provider** — URL token, IP allowlist, mTLS, HMAC, public keys.
- **4.3 HMAC signatures** — HMAC-SHA256 of the body with a shared secret, recomputed and compared.
- **4.4 What exactly is signed** — Body, `t.body`, `id.t.body` or URL plus fields.
- **4.5 Verify raw bytes before parsing** — Re-encoded JSON breaks the HMAC: read, verify, parse.
- **4.6 Replay attacks and timestamps** — A signed timestamp and a five-minute tolerance.
- **4.7 Constant-time comparison** — `hmac.Equal`, `timingSafeEqual`, `compare_digest`.

5 Delivery Semantics: Duplicates and Ordering

Delivery is at least once and unordered; receivers must be idempotent and order-tolerant.

- **5.1 At least once, never exactly once** — A lost 200 triggers a retry of the same event.
- **5.2 Delivery IDs** — IDs stable across retries, stored under a unique constraint.
- **5.3 One transaction for the ID and the work** — Both commit or neither: idempotence.
- **5.4 Ordering** — Delete before create; fetch state, versions, buffers or paid ordering.

6 Retries, Deadlines and Load

Retries plus traffic spikes punish slow handlers; acknowledge fast, work in the background.

- **6.1 Provider retry schedules** — Stripe up to 3 days, Svix 5 s to 10 h, GitHub none.
- **6.2 Deadlines, retries and traffic spikes** — Missed deadlines turn into extra load.
- **6.3 A thin receiver and a background worker** — Verify, store, respond 200; a queue worker does the rest.

7 Sending Webhooks: Becoming a Provider

As a provider we need an outbox, a signing dispatcher with backoff, and SSRF protection.

- **7.1 A webhook for the task board** — `task.moved`; users give a URL, we give a secret.
- **7.2 The naive implementation** — Coupled latency, lost events, false events.
- **7.3 The transactional outbox** — Task update and event row in one transaction.
- **7.4 The dispatcher** — Check, sign, 10 s timeout, record, retry, disable after 10 failures.
- **7.5 Exponential backoff and jitter** — Growing, randomized intervals; `Retry-After`.
- **7.6 Server-side request forgery** — Resolve, reject private ranges, never follow redirects.
- **7.7 The demo** — 500, then 200; loopback and private hosts refused on every send.

8 When to Use Webhooks

Webhooks are notifications; mirroring provider data should follow the ordered event log.

- **8.1 Webhooks are notifications** — Chat messages, CI/CD builds, plan unlocks.
- **8.2 Misuse: synchronizing provider data** — Copies built from raw webhooks inherit every edge case.
- **8.3 The event-log approach** — Poll the log with a cursor, or use the webhook only as a trigger.

Webhooks at a glance — The provider calls us with a signed HTTP POST that we verify on raw bytes, record idempotently and acknowledge fast; as a provider we deliver through an outbox with backoff and SSRF checks. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

Webhooks: How the Server Calls You

Chapter 26 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — Webhooks let one backend notify another of an event by sending an HTTP POST to a URL the receiver registered, replacing slow and costly polling. The chapter starts from a payment confirmed by Stripe, defines providers, endpoints, events and deliveries, and dissects a real GitHub delivery together with deadlines, handshakes and local tunnels. It then treats sender verification: HMAC signatures over the raw body, signed timestamps against replays and constant-time comparison. Because delivery is at least once and unordered, receivers must deduplicate by delivery ID inside the transaction of the work, tolerate reordering, and acknowledge quickly while a background worker does the work. On the sending side, a transactional outbox, a signing dispatcher with exponential backoff and jitter, and SSRF checks make delivery reliable and safe. The chapter closes by separating notifications, the proper use of webhooks, from data synchronization, which is better driven by the provider's ordered event log.

Keywords — webhooks, HMAC signatures, replay attacks, idempotency, at-least-once delivery, event ordering, transactional outbox, exponential backoff, SSRF

1 INTRODUCTION: SERVER-TO-SERVER EVENTS

All previous lectures assumed that the client initiates communication: it sends a request and the server responds, never the other way round. The real-time patterns of Chapter 22 relaxed this only partly; the server could push data, but only over a connection the client had opened first. This lecture covers communication between two services, mainly two backends, with no browser and no human in between: the service where an event happens must inform the service that needs to know about it.

1.1 The payment problem

The task-management application gets a pricing page with a paid plan. Purchases go through a payment provider such as Stripe, so the decisive event, a successful transaction, happens in Stripe's system, not in ours. The plan may be unlocked only after a signal from Stripe confirms that the funds were sent and received without problems on their side.

The browser in the user's window cannot be the system that receives this signal. **Reliability:** the browser depends on the user's network, which can fail at any moment; the user may close the tab; the machine may crash under load. In each case the signal is lost. **Security:** anything on the client can be modified, and a payment confirmation is far too critical to be trusted there. The signal must reach our server, which is the single source of truth for such events.

1.2 Polling and its costs

The first, brute-force idea is the one Chapter 22 started with: after sending the payment, our server polls the provider's status API, say once per second. While the answer is "pending" it keeps asking. When Stripe decides, for example that the payment failed or succeeded, the same API returns the outcome; the server updates the database and tells the frontend that the plan is active. The approach has three costs.

- **Latency.** With polling interval T the average delay is T/2: half a second when polling every second.
- **Waste.** Every request answered with "pending" consumes resources on both sides and carries no information.
- **Load and rate limits.** If a provider has 10,000 merchants polling every second, it receives 10,000 requests per second for this alone. Providers protect their infrastructure with aggressive rate limits, so one-second polling is not even permitted; at an allowed interval of 60 seconds the average delay becomes 30 seconds.

2 WEBHOOKS: CONCEPT AND VOCABULARY

2.1 Reversing the direction

The solution is to reverse the direction of communication. Instead of asking the provider every second, we rely on it to notify us as soon as the event occurs. This mechanism is called a *webhook*, and in itself it is nothing special: an ordinary HTTP POST request (Chapter 1) that the provider sends to us. The target is not arbitrary. We open an endpoint, give its URL to the provider, and the provider posts event data to it. The pattern is a callback at the level of APIs: normally we call a service's API when we need something; here the service calls our API to report an event.

> **Webhook.** A user-defined HTTP callback: a POST request that a provider sends to a URL registered by the consumer whenever a subscribed event occurs, with the event data in its body

2.2 Origin

In 2007 Jeff Lindsay argued that web architecture needed a primitive like the Unix pipe: a way for one service to pass its output to another when an event occurs, just as a shell redirects the standard output of one program into another.

He called these user-defined callbacks, carried by HTTP POST requests, *webhooks*. Two years later Google engineers built a protocol on the same idea for blog feeds, PubSubHubbub.

2.3 Vocabulary

Each provider documents webhooks somewhat differently, and implementations vary. Table 1 fixes the terms used in this chapter.

Term	Meaning
Provider (sender)	The system where the event occurs, e.g. Stripe
Consumer (receiver)	Us: the service that receives the event
Endpoint	A URL we expose for the provider's POST requests
Event	Anything defined in advance: a payment succeeded, a task moved to another board
Delivery	One attempt to send one event to one endpoint
Subscription	Our registration: this URL, these event types

Table 1: Webhook vocabulary

The difference between an event and a delivery is central: one event can have several deliveries, so a receiver must be designed from the start for the same event arriving more than once (Section 5).

3 RECEIVING WEBHOOKS IN PRACTICE

3.1 A minimal receiver

The demo receiver is a small Go program of three packages. `main` configures a single route, `POST /hooks/github`. The `inbox` package holds the handler: it reads the entire request body, verifies the signature with the `verify` package, stores the delivery ID and responds. Read, verify, record, respond is the typical structure of a webhook handler (Figure 2 in Section 4).

The receiver runs on a Linux machine without a public address, so a *tunnel* exposes it: a program that gives a public HTTPS address to a local port that is not reachable from the internet (ngrok, Cloudflare Tunnel and similar services). A random tunnel hostname forwards to port 8081. In the GitHub repository settings a webhook is added with that URL, content type JSON, a secret key, and only the push event. GitHub immediately sends a `ping` event to the new hook, which shows up in the receiver's log. After a commit is pushed, the receiver logs one POST request: the event name, the delivery ID and the signature in the headers, and the 202 the handler returned. GitHub keeps its own copy of every attempt: under *Recent Deliveries* it shows the same headers, the payload and our response.

3.2 Anatomy of a delivery

The received request (Figure 1) is a POST to the registered URL with: `Content-Type: application/json`, since JSON was selected; `X-GitHub-Event`, the event type; `X-GitHub-Delivery`, a unique delivery ID; `X-Hub-Signature-256`, the signature whose verification Section 4 explains; and a body with the full event data as JSON.

```
POST /hooks/github HTTP/1.1
Host: random-name.trycloudflare.com
Content-Type: application/json
X-GitHub-Event: push
X-GitHub-Delivery: 72d3162e-cc78-11e3-81ab-4c9367dc0958
X-Hub-Signature-256: sha256=7d38cdd689735b008b3c702edd92...

{"ref": "refs/heads/main",
 "repository": {"full_name": "user/taskboard"},
 "commits": [ ... ]}

HTTP/1.1 202 Accepted
```

Figure 1: A GitHub push delivery and the receiver's answer (illustrative values)

3.3 What counts as success

The provider judges a delivery like any HTTP call: by its status code. Any 2xx response is success; whether 200 or 202 rarely matters. Any other code, 3xx or 4xx included, is a failure. The second condition is time: the response must arrive within the provider's deadline, which is 10 seconds at GitHub, 5 seconds at Shopify, and 3 seconds at Slack and at Microsoft Graph (the Outlook API). A response after the deadline counts as a failure even if the handler completed its work. Deadlines differ, so each provider's documentation must be checked; every failure feeds the provider's retry mechanism (Section 6).

3.4 Registration and per-provider endpoints

A registration form typically asks for three things: the endpoint URL, a secret key, and the list of event types. A real system usually integrates several providers, say GitHub and Stripe, and each should get its own endpoint: `/hooks/github` and `/hooks/stripe`, never a generic shared `/hooks/callback`. Every provider has its own headers, its own signature scheme and its own retry behavior, and its handler must encode all of them; separate endpoints keep the handlers and their logic separate per provider.

3.5 Handshakes

Some providers add a verification step before the first event: they send a request to the newly registered URL and wait for a specific answer, and only after receiving it do they start delivering the subscribed events (Table 2).

Provider	Handshake request	Expected answer
Microsoft Graph	a validation token	the token, echoed as plain text
Discord	a ping message	a pong message
Amazon SNS	a subscription confirmation with a URL	visit the URL to confirm

Table 2: Registration handshakes of three providers

The reason is abuse. A webhook system sends events to whatever URL was registered. Without a handshake, an attacker could register a victim's address with a large provider such as AWS, Stripe or GitHub, subscribe to every event type, and turn the provider's delivery traffic into a denial-of-service attack on the victim. The handshake proves that whoever controls the URL actually wants the events.

3.6 Local development and tunnels

A webhook is by nature an inbound request: it comes from the internet into your system. A laptop behind a consumer home router cannot receive it, because such routers do not forward unsolicited requests from the internet to devices, and `localhost` means nothing to a server on the internet, which works with domain names and public IP addresses. To test a webhook feature during development, run a tunnel such as ngrok or Cloudflare Tunnel: it provides a temporary public HTTPS address, which is registered with the provider, and forwards every request arriving there over a secured connection to the local port.

4 VERIFYING THE SENDER

4.1 The threat

The endpoint is a public URL that accepts a POST and, in the payment example, unlocks a paid plan. Such URLs are easy to guess, since most follow a standard form: `/webhooks`, `/hooks`, `/callback`. Anyone who finds it can post a JSON body stating that the payment for a given user ID succeeded, and exploit the system. HTTPS does not solve this: it prevents others from intercepting or reading the bytes in transit, but it does not establish who sent them. The receiver needs a guaranteed way to confirm that a request really comes from its provider.

4.2 Five ways to authenticate the provider

Method	Mechanism	Assessment
1. Secret in the URL	a token in the URL, like basic authentication	weakest: URLs end up in logs
2. IP allowlist	accept only the provider's published IPs	safer; incomplete alone
3. Mutual TLS	the provider presents a client certificate	strong; more setup
4. HMAC signature	a MAC of the body with a shared secret	most common (about 65 %)
5. Public-key signature	provider signs with a private key, we verify with its public key	no shared secret; e.g. Discord

Table 3: Ways to authenticate webhook requests

The first method can work, but URLs are written into many systems: local log files and logging services such as CloudWatch or Elasticsearch. A leak of any of them reveals the token, and anyone holding it can send forged requests. For the second, providers such as GitHub, Stripe and the payment provider Polar publish the IP addresses from which all their events originate; the firewall or cloud network configuration admits only those and blocks everyone else. This is safer than a token but not a complete solution on its own. With mutual TLS, the provider authenticates itself with a client certificate during the TLS handshake, and the receiver accepts only connections that present it. The fourth method is used by almost every platform, about 65 % of providers by the lecturer's estimate: a signature computed from the request body and a secret known only to the provider and the receiver. The fifth uses asymmetric keys: the provider signs with its private key and receivers verify with the public key; Discord works this way. The rest of the lecture uses the fourth method.

4.3 HMAC signatures

The signature mechanism is HMAC, a hash-based message authentication code. A cryptographic hash function (see Chapters 1 and 16) maps input of any length to a fixed-length fingerprint, and changing a single byte of the input changes the entire fingerprint. HMAC mixes a secret key into this computation, so only holders of the key can produce the right value. The provider computes HMAC-SHA256 over the request body with the shared secret and puts the result in a header. The receiver repeats the computation over the received body with the same secret and compares both values. If they match, the request comes from the provider and not from a third party trying to exploit the system, and the body was not altered.

```
func Verify(secret, body []byte, header string) bool {
    mac := hmac.New(sha256.New, secret)
    mac.Write(body)
    want := "sha256=" + hex.EncodeToString(mac.Sum(nil))
    return hmac.Equal([]byte(want), []byte(header))
}

func (h *Inbox) ServeHTTP(w http.ResponseWriter,
    r *http.Request) {
    body, err := io.ReadAll(r.Body)   // 1. raw bytes
    sig := r.Header.Get("X-Hub-Signature-256")
    if err != nil || !Verify(h.secret, body, sig) { // 2.
        w.WriteHeader(http.StatusUnauthorized) // 401
        return
    }
    id := r.Header.Get("X-GitHub-Delivery")
    h.store.Save(id, body)         // 3. dedup + enqueue
    w.WriteHeader(http.StatusAccepted)       // 4. 202
}
```

Figure 2: A GitHub receiver in Go: read, verify, record, respond (illustrative)

4.4 What exactly is signed

Providers differ in what they feed into the HMAC, so each provider's documentation is the authority. Stripe's Go SDK shows a typical verifier: the signature header is parsed into `t`, a Unix timestamp, and one or more `v1` signatures, each compared with the locally computed value using `hmac.Equal`. Table 4 lists the variants from the lecture.

Provider	Signed content	Algorithm and header
GitHub	body only	HMAC-SHA256, `X-Hub-Signature-256`
Stripe	timestamp `.` body	HMAC-SHA256, `Stripe-Signature` (`t=`, `v1=`)
Standard Webhooks	ID `.` timestamp `.` body	HMAC-SHA256, `webhook-signature`
Twilio	full URL + form fields, sorted	HMAC-SHA1, `X-Twilio-Signature`

Table 4: What different providers sign

Each addition closes a gap. Stripe signs the timestamp to protect against replay attacks (Section 4.6). The Standard Webhooks specification also signs the delivery ID, so the ID is protected too. Twilio signs the full endpoint URL with

all form fields in sorted order, so a signature is valid only for the exact URL it was sent to.

4.5 Verify raw bytes before parsing

The HMAC is computed over exact bytes, which creates a trap in frameworks that parse bodies automatically. In Node.js with Express, `express.json()` hands the handler a parsed object. If the handler encodes that object back into a string and computes the HMAC over it, the bytes can differ from those the provider signed: a provider that wrote JSON without spaces and an encoder that adds a space after each colon produce different strings, hence different HMACs, and the delivery fails verification. The rule for every webhook handler: read the body as bytes, verify those bytes, and only then parse.

The demo shows three outcomes. With the correct secret, the receiver verifies the raw bytes and returns 202. After the secret is changed on the GitHub side and the delivery is redelivered, the computed value no longer matches the header; the receiver returns 401 Unauthorized, and GitHub records the 401. With the secret fixed again but the handler changed to parse the JSON and re-encode it before verifying, the check fails once more, although sender and secret are both correct.

> **Warning.** A body parser installed globally destroys the signed bytes before the handler sees them. Give the webhook route its own raw-body handling (in Express, `express.raw({ type: 'application/json' })` on that route instead of `express.json()`), verify, and only then parse the JSON yourself.

4.6 Replay attacks and timestamps

A valid signature proves who created a request, not when. Suppose an attacker, through a man-in-the-middle position, captures one complete delivery from the provider to your service: all headers and the full body. An hour or two later the attacker sends exactly the same request, possibly a thousand times. The signature is genuine, so every copy passes verification and reaches the business logic. This is a *replay attack*. The defense is a timestamp inside the signed string. Stripe puts the time of the attempt into the signed content, so it cannot be changed without breaking the signature, and its library rejects anything older than five minutes.

> **A subtle point.** A timestamp limits replays to the tolerance window; it does not eliminate them. Within those five minutes a captured request still verifies, and so does every copy of it. The second defense is the delivery-ID deduplication of Section 5: a replayed request carries an ID that was already processed and changes nothing. Signing the ID, as Standard Webhooks does, stops an attacker from altering it.

4.7 Constant-time comparison

The computed and received signatures must not be compared with the language's ordinary string equality. Ordinary comparison stops at the first byte that differs, so its execution time reveals how many leading bytes were correct. By measuring response times over thousands or millions of requests, an attacker can reconstruct a valid signature byte by byte. Every language therefore provides a constant-time comparison whose duration does not depend on where, or whether, the inputs differ: `hmac.Equal` in Go, `crypto.timingSafeEqual` in Node.js, `hmac.compare_digest` in Python. Its timing gives no clue about where a mismatch occurred.

5 DELIVERY SEMANTICS: DUPLICATES AND ORDERING

5.1 At least once, never exactly once

Assume the signature is valid. The handler does its work, writes to the database and responds 200, but the response is lost on the way: the connection drops or a load balancer restarts at that moment, events that are routine in backend services and cloud infrastructure. The provider never receives the 200, so from its point of view the delivery failed, and it retries with the same payload, perhaps seconds or minutes later, in an order that cannot be predicted. Provider documentation therefore promises *at-least-once* delivery: an event that reaches your backend will reach it at least once and will not be lost, but it may arrive more than once. Exactly once cannot be promised, because the acknowledgement itself can be lost.

5.2 Delivery IDs

The receiver must be safe to call any number of times with the same event, while its side effects happen only once. The tool is the event's unique identifier, which is identical across all retries of that event:

- GitHub: the `X-GitHub-Delivery` header;
- Stripe: the event `id` inside the body;
- Standard Webhooks: the `webhook-id` header.

The receiver keeps a table of the webhooks it has processed, keyed by this ID under a unique constraint, so that two rows with the same delivery ID can never exist. A retried event then fails to insert and is acknowledged without repeating the work.

5.3 One transaction for the ID and the work

The unique table alone is not enough: inserting the delivery ID and performing the side effects, such as inserting or updating rows, must happen in the same database transaction. Without that, both orders fail.

- **Work first, then ID.** The server crashes between the two steps, for example the container is restarted after an out-of-memory error. The work is done, the ID is not saved; the retry finds no record and the work runs a second time.
- **ID first, then work.** The insert or update of the actual work fails. The retry finds the ID, concludes that the event was handled and skips it, although the work never happened and should be attempted again.

Inside one transaction (Figure 3) either both actions happen or neither does, however often the event arrives.

```
BEGIN;
INSERT INTO processed_webhooks (delivery_id)
VALUES ('evt_1NxY...')
ON CONFLICT (delivery_id) DO NOTHING;
-- 0 rows inserted: duplicate -> ROLLBACK, respond 200
UPDATE subscriptions SET plan = 'pro'
 WHERE user_id = 42;
COMMIT;   -- ID and side effect are stored together
```

Figure 3: Deduplication and the side effect in one transaction (illustrative PostgreSQL)

Idempotence. The property that performing an operation several times has the same effect as performing it once; for a webhook receiver, any number of deliveries of one event produce its side effects exactly once

5.4 Ordering

Retries also break ordering. In 2017 an engineer working on Stripe's API described the case: the provider generates two events for one object, *created* and then *deleted*. Delivery of *created* fails once, perhaps because the receiver was being deployed, and is retried five minutes later; *deleted* is sent a second later and succeeds. The receiver sees the deletion before the creation, and a handler that applies events literally ends up with a row for an object that no longer exists. Stripe's documentation states it plainly: events are not always delivered in the order in which they were generated. Timestamps do not help either: they have a resolution of one second, so two events generated less than a second apart carry the same timestamp regardless of their order.

Four remedies exist.

1. **Treat the event as a signal.** Do not use the payload's data. On receipt, load the current state of the object from its source, the provider's API, and decide based on that. Order no longer matters because the current state is the source of truth; the cost is one API call per event.
2. **Version the object.** Store a version or `updated_at` field with the local copy and apply an event only if it is newer than what is stored, checking and writing in one transaction.
3. **Buffer.** Hold the events of an object until all earlier events have arrived. This is hard to make predictable: the receiver cannot know how long to wait for a missing event.
4. **Buy ordering.** Some providers offer strict, guaranteed ordering as a feature, for an extra charge.

6 RETRIES, DEADLINES AND LOAD

6.1 Provider retry schedules

When an endpoint returns an error or does not answer in time, almost every provider retries, but each on its own schedule, which must be read in its documentation before integrating (Table 5).

Provider	Retry behavior	Notes
Stripe	exponential backoff for up to 3 days	then disables the endpoint and sends an email
Svix	at once, 5 s, 5 min, 30 min, 2 h, 5 h, 10 h, …	webhooks as a service
Shopify, Slack, Polar	own schedules	see their documentation
GitHub	no automatic retries	manual redelivery or an API script

Table 5: Retry behavior of webhook providers

Svix is a popular webhooks-as-a-service platform. Sending webhooks properly is not a small function: it requires significant engineering for the edge cases, security issues and retries described in this lecture, so many companies delegate it to Svix and use its built-in interface or its API. GitHub is the opposite extreme: it makes zero retry attempts. If the server was unavailable when GitHub tried to deliver, that delivery is lost unless someone clicks the redeliver button, or a script uses the GitHub API to list the failed deliveries and request their redelivery.

6.2 Deadlines, retries and traffic spikes

Deadlines and retries combine into a load problem. The receiver has 10, 5 or 3 seconds to acknowledge each attempt; every attempt it fails to acknowledge comes back as a retry, on top of the traffic generated by its own users. Webhook traffic is also spiky by nature: a payment provider renews all subscriptions on the first of the month; a marketing tool sends emails to half a million people and then sends a delivery receipt for each recipient. If the handler does all the work inside the request (verify, update two or three tables as side effects, call other APIs, then respond), it will most likely miss the deadline under such a spike. Every miss produces a retry, retries add load, and more deadlines are missed.

6.3 A thin receiver and a background worker

The robust design makes the receiver as lightweight as possible, with a short fixed list of steps: read the body, verify the signature, save the delivery to a database table, respond 200 immediately, and stop. The actual processing runs in a background worker fed by a queue (see Chapter 10), which takes the stored records and performs the work asynchronously (Figure 4).

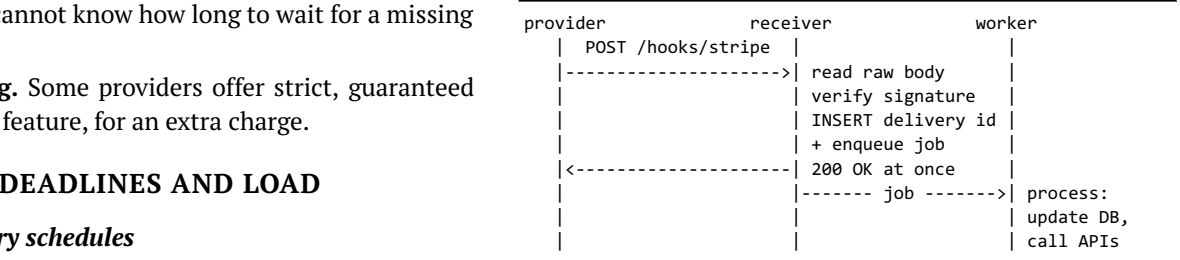


Figure 4: The receiver acknowledges within milliseconds; a worker does the work (illustrative)

Once the receiver has answered 200, the provider will never retry that event, so retrying failed processing becomes the worker's responsibility. This is why the delivery is per-

sisted in the database, in the same transaction as the deduplication record, before the acknowledgement is sent.

7 SENDING WEBHOOKS: BECOMING A PROVIDER

7.1 A webhook for the task board

The roles reverse when our own product offers webhooks. In the task board, moving a task between boards produces a `task.moved` event carrying a sequence number, the task ID, who moved it and when. A user wants a signal whenever a task reaches the "completed" status, to post it to a chat, process it programmatically or run their own logic. They give us a URL, and we, now the provider, give them a secret key, as Stripe and GitHub did for us.

7.2 The naive implementation

The obvious code calls `http.Post` to the user's URL inside the move handler, right after saving the move to the database. Three problems follow.

- **Coupled latency.** If the user's endpoint is slow or down, the move handler absorbs the delay: it waits for that response before answering our own frontend. A third party's unoptimized server now sets our latency.
- **Lost events.** The database update succeeds, then the container crashes or the server restarts before the POST is sent. The move happened; the webhook never does.
- **False events.** Reversing the order, POST first and database second, the POST succeeds but the update fails or the backend crashes. The user was told about a move that does not exist in our database.

7.3 The transactional outbox

The solution is the *outbox*: an ordinary table in the same database that holds the tasks themselves. The move handler opens a transaction, updates the task, inserts a row with the event data into the outbox, and commits (Figure 5). Since both writes belong to one transaction, either both happen or neither does. A separate process, a background worker, reads the outbox and performs the delivery attempts asynchronously. This is the write-ahead-log idea from Chapter 8 applied to events: first record the intention in a reliable place that survives failures, then let another process carry it out. The move handler no longer depends on the user's endpoint and answers the frontend as quickly as before.

```
-- move handler: one transaction
BEGIN;
UPDATE tasks SET status = 'completed', seq = seq + 1
 WHERE id = 17;
INSERT INTO outbox (event_type, payload)
VALUES ('task.moved',
  '{"seq": 9, "task_id": 17, "moved_by": "u42",
    "moved_at": "2026-11-01T10:15:03Z"}');
COMMIT;   -- task change and event: both or neither
-- a dispatcher reads the outbox later, in its own process
```

Figure 5: Writing the event to the outbox with the business change (illustrative)

7.4 The dispatcher

The demo implementation uses three tables: `endpoints`, holding each subscriber's URL and secret; `outbox`; and `deliveries`, with one row per attempt. The dispatcher takes outbox rows one at a time and, for each delivery:

1. checks the destination URL against private addresses before anything else (Section 7.6);
2. signs the ID, timestamp and body in the Standard Webhooks format with the endpoint's secret (Figure 6);
3. sends the POST with a 10-second timeout. There is no hard rule; the value depends on the service, its constraints and its commitments, but 10 seconds is the common standard;
4. records the attempt in `deliveries`: the status code, the first 2 KB of the response and the time;
5. on a 2xx marks the event delivered and stops; otherwise computes the next attempt time with exponential backoff and jitter, honoring a `Retry-After` value sent by the receiver;
6. after the last attempt marks the event *exhausted*; after 10 consecutive failures disables the endpoint and emails its owner.

Each endpoint has its own queue, so a subscriber whose URL is broken or slow cannot delay deliveries to all other subscribers.

```
// sign "id.timestamp.body" (Standard Webhooks format)
ts := strconv.FormatInt(time.Now().Unix(), 10)
mac := hmac.New(sha256.New, ep.Secret)
mac.Write([]byte(ev.ID + "." + ts + "." + string(ev.Body)))
sig := base64.StdEncoding.EncodeToString(mac.Sum(nil))

req, _ := http.NewRequest("POST", ep.URL,
    bytes.NewReader(ev.Body))
req.Header.Set("webhook-id", ev.ID)
req.Header.Set("webhook-timestamp", ts)
req.Header.Set("webhook-signature", "v1,"+sig)

client := &http.Client{
    Timeout:   10 * time.Second,
    Transport: safeTransport, // dials verified IPs only
    CheckRedirect: func(*http.Request,
        []*http.Request) error {
        return http.ErrUseLastResponse // never follow
    },
}
```

Figure 6: The dispatcher signs and sends one delivery (illustrative Go)

7.5 Exponential backoff and jitter

Exponential backoff means that the interval between attempts grows instead of staying fixed, as it would with a retry every 5 minutes: for example 5 seconds, then 5 minutes, then 15 minutes, then 1 hour. *Jitter* adds a random amount to each computed delay: where backoff says 5 minutes, the attempt happens after, say, 5 minutes and 42 seconds. When thousands of deliveries to different users fail together, jitter keeps their retries out of the same window: one fires at 5 minutes and 1 second, another at 5 minutes and 35 seconds, instead of all at exactly 5 minutes. A receiver in trouble may also send `Retry-After`, meaning "we have service issues right now; instead of your

schedule, try again in at least one hour" (or six), and the dispatcher respects it.

7.6 Server-side request forgery

A provider faces a threat of its own. It calls whatever URL a customer enters in the registration form, and it calls it from inside its own network. An attacker can register `http://169.254.169.254/...`, the metadata service of most cloud providers, which hands out credentials, or the hostname of an internal administration service. The webhook dispatcher dutifully sends its POST there, and the attacker makes the provider's servers issue requests to internal targets on their behalf: *server-side request forgery* (SSRF).

Checking the URL once at registration is not enough. The attacker can point the domain at a public address while it is being checked and switch it to a private address after registration, or a public address can simply redirect to a private one. The correct defense: resolve the name yourself, check the resulting address against a fixed set of private ranges in code, and never follow redirects. Typical blocked ranges are loopback (127.0.0.0/8, ::1), the private networks of RFC 1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), link-local addresses including the metadata service (169.254.0.0/16), and IPv6 unique-local addresses (fc00::/7). HTTP clients follow 301, 302 and similar responses by default, as the protocol suggests; a webhook client must not. This is also why a provider treats only a 2xx as success and any 3xx as a failure. Where possible, webhooks should leave from a separate network through an *egress proxy*; Stripe publishes an open-source one, Smokescreen.

> **Warning.** Check the address the connection actually uses. If the code resolves and checks a hostname but the HTTP client then resolves it again, a DNS server controlled by the attacker can answer the check with a public address and the connection with a private one. Dial the verified IP (the `safeTransport` of Figure 6), or let an egress proxy make both decisions.

7.7 The demo

Moving a task immediately creates a row in the outbox, before anything has been sent. The subscriber's endpoint is configured to answer 500, so the first attempt fails; the `deliveries` table already shows when the next attempt is due, and successive attempts show growing delays from backoff and jitter. Once the endpoint is fixed, the next attempt receives 200, the event is marked delivered, and no further retries happen. The SSRF guard is tested twice: registering the task board's own admin route on the loopback address is refused with a stated reason, and so is a hostname that resolves to a private address. Because the check runs before every send and not only at registration, a URL that pointed to a public resource when it was registered and was changed later is still blocked.

8 WHEN TO USE WEBHOOKS

8.1 Webhooks are notifications

A webhook is a notification: something happened, here is its data, and here is where it happened if you want to look at it. Used as notifications, webhooks are the right tool: when an event occurs, post it to a chat channel; start a CI/CD pipeline or a build in GitHub Actions; unlock a paid plan for the user.

8.2 Misuse: synchronizing provider data

Because of how they work, webhooks are also used for something they handle badly: as a synchronization mechanism that keeps a copy of a provider's data in one's own database. The lecturer's example is Clerk, a widely used authentication and authorization provider. Authentication involves many flows that every user needs: passwords, login via Google, GitHub or other OAuth providers (see Chapter 4), email flows for registration, login and password reset, viewing one's sessions, removing a device from the session history. Rather than every company reinventing them, Clerk provides SDKs for frontend and backend that reduce the logic to a few calls to its API. Consequently all data about users, organizations, roles and permissions lives in Clerk's database.

Calling Clerk's API for every permission check, user lookup or user list adds network latency compared with a query to one's own database in the same region as the server; the lecturer quotes 70 to 600 ms instead of 50 to 60 ms. The natural response is to keep a local copy of the user, organization and role tables and keep it in sync with Clerk. Clerk offers webhooks for the creation, update and deletion of users, organizations and roles, so teams create an endpoint per entity and populate their tables from the incoming events. Keeping a copy is legitimate; the problem is maintaining it from raw webhooks, which inherits every edge case of this lecture: deadlines, retries, duplicates and, above all, arbitrary ordering. All of them must be handled correctly for the copy to stay up to date.

8.3 The event-log approach

The robust alternative still uses webhooks, but differently. Providers of this kind keep an event log, available through an API and in their dashboard. The log is correctly ordered, because entries are created in the provider's own infrastructure together with the change they describe; ordering problems arise only in distributed scenarios, where separate network calls cannot share one transaction. The log holds all events since the beginning and is read through an endpoint such as `/events`. Two strategies use it.

- **Periodic pull.** If the copy may be stale for a while, say five minutes, a background job runs every five minutes, reads the new events from the log and inserts, updates or deletes local rows accordingly. It stores the ID of the last processed event, so each run continues from there instead of starting from the beginning and repeating useless work.

- **Webhook as a trigger.** Subscribe to a webhook that fires whenever the provider adds an entry to its event log. Suppose a user changes their name through the Clerk SDK in the frontend, so our backend never sees the request. Clerk updates the name and appends a log entry in the same transaction, then sends the webhook. On receipt, a background job reads all events after the stored last ID, finds the name-change event and updates the local row (Figure 7). The copy is synchronized within milliseconds or seconds, and webhooks are no longer abused: the webhook carries no state that could arrive out of order.

```
on webhook "event log changed":     # payload ignored
    verify signature; enqueue sync job; respond 200

sync job:
    cursor = SELECT last_event_id FROM sync_state
    for e in GET /events?after={cursor}:   # ordered
        BEGIN
        apply e to users / organizations / roles
        UPDATE sync_state SET last_event_id = e.id
        COMMIT
```

Figure 7: A webhook used only as a trigger to read the provider's event log (illustrative)

A subtle point. Not every provider exposes a complete, ordered event log through its API, and those that do may limit it: Stripe's event list, for instance, reaches back only 30 days. Where no log exists, the signal-and-fetch remedy of Section 5.4 (reload the current object from the provider's API) gives the same independence from ordering; where the log is time-limited, a long outage requires a full reconciliation.

9 CONCLUSION

- A webhook is a plain HTTP POST from a provider to a URL we registered; it replaces polling, which costs T/2 latency, wasted requests and rate-limited load.
- A delivery succeeds only with a 2xx inside the provider's deadline (GitHub 10 s, Shopify 5 s, Slack 3 s); give every provider its own endpoint.
- Authenticate every request: HMAC over the raw bytes before parsing, a signed timestamp against replays, and constant-time comparison.
- Delivery is at least once: deduplicate by delivery ID under a unique constraint, in the same transaction as the side effects.
- Delivery is unordered: treat events as signals and fetch current state, or apply only newer versions.
- Keep the receiver thin (verify, store, respond) and do the work in a queue worker, or missed deadlines turn into retry storms.
- As a provider, write events to an outbox in the business transaction and deliver them from a dispatcher with signing, timeouts, backoff with jitter, `Retry-After` and per-endpoint queues.
- Guard the dispatcher against SSRF on every send: resolve, reject private ranges, never follow redirects.
- Use webhooks for notifications; to mirror provider data, read the provider's ordered event log with a cursor and use the webhook only as a trigger.

APPENDIX A REVIEW QUESTIONS

1. A store receives Shopify order webhooks, which must be answered within 5 seconds. The handler verifies the signature, writes order and inventory rows and calls a shipping API, taking 1.5 s at the median and 7 s at the 99th percentile. During a flash sale 30,000 orders arrive in ten minutes, the database CPU reaches 100 %, and the rate of incoming webhook requests keeps rising after the sale has ended. Explain this behavior step by step, redesign the receiving path, and state which failure responsibilities move from Shopify to your system and how you discharge them.
2. A Node.js receiver behind `app.use(express.json())` recomputes the HMAC over `t + "." + JSON.stringify(req.body)`, compares it with the header using `===`, accepts timestamps up to 24 hours old, and remembers processed event IDs in an in-memory `Set` on each of its three pods. The provider sends bodies such as `{"id": "evt_9", "type": "payment.succeeded"}`, with a space after every colon. Predict what happens to genuine deliveries, to a captured delivery replayed 40 times an hour later, and to an attacker probing signatures over millions of requests, and specify the change each case requires.
3. Customers occasionally receive a purchased credit pack twice. The logs show the same Stripe event ID processed twice about six minutes apart, and each time the pod was restarted with reason OOMKilled shortly after the first processing began. The handler runs `UPDATE accounts SET credits = credits + 500` and then `INSERT INTO processed_events(id)`, each in autocommit mode. Reconstruct the sequence that produced the duplicate, explain why swapping the two statements would trade this bug for a different one, and give a corrected design.
4. Your service mirrors a provider's `subscription.updated` events and applies a payload only if its `created` timestamp, in whole seconds, is greater than or equal to the stored one. Event A (plan = pro) and event B (plan = free) are both created at 10:00:00, in that order; A's first delivery times out and is retried at 10:05, while B is delivered at 10:00:01. Predict the final stored plan and explain why the guard failed, then compare fetching the current object from the API with a provider-supplied version number for 2 million events per day under an API rate limit of 100 requests per second.
5. Your task-board dispatcher runs on a cloud VM. A customer registers `https://hooks.attacker.example/x`, which resolved to 203.0.113.10 when it was validated; weeks later its DNS record has a TTL of 0 and alternates between 203.0.113.10 and 169.254.169.254. Before each send the dispatcher calls `isPublic(host)`, which resolves the name and checks the address, and then calls

`http.Post(url, ...)` with Go's default client. Predict what the attacker can achieve and through which two independent paths, and design a delivery path in which the check and the connection cannot disagree.

6. An application uses an external identity provider and keeps a local copy of 50,000 users and their roles for permission checks. A revoked role must take effect within 10 seconds, while profile names may be up to an hour stale; the provider offers per-entity webhooks and an ordered `/events` API. Compare syncing from raw entity webhooks, polling the event log every five minutes, and a webhook-triggered event-log reader, including how each recovers after your service has been down for two hours, and recommend one design.
7. A hosting outage makes 3,000 of your subscribers' endpoints fail from 09:00 to 09:20 while 120,000 deliveries to them are pending. Your dispatcher retries every failed delivery exactly 5 minutes later, serves all endpoints from one shared queue, and disables an endpoint after 10 consecutive failed attempts. Describe what healthy subscribers, the recovering endpoints and the dispatcher experience during and after the outage, estimate how many endpoints end up disabled, and propose a revised policy with the effect of each change.
8. A CI service starts a build for every GitHub push webhook. During deploys, which restart its three instances one at a time behind a load balancer, a few pushes per deploy never trigger a build; GitHub's recent deliveries show some as connection errors and others as timeouts after 10 seconds. Explain plausible causes in terms of GitHub's delivery semantics and graceful shutdown (Chapter 15), and design a mechanism that finds and replays the missed deliveries without ever starting the same build twice.

APPENDIX B GLOSSARY

Term	Meaning
Webhook	HTTP POST a provider sends to a registered URL when an event occurs
Endpoint	The URL a consumer exposes to receive webhook requests
Delivery	One attempt to send one event to one endpoint
Subscription	Registration of an endpoint URL and the event types it wants
Handshake	Provider's check that a newly registered URL really wants events
Tunnel	Service giving a local port a temporary public HTTPS address
HMAC	Keyed hash proving that a message was produced by a holder of the secret
Replay attack	Resending a captured, validly signed request
Constant-time comparison	Comparison whose duration does not reveal where inputs differ
At-least-once delivery	Guarantee that an event arrives, possibly several times
Delivery ID	Identifier of an event that stays the same across retries
Idempotence	Repeating an operation has the same effect as doing it once
Transactional outbox	Table written with the business change, delivered later by a worker
Exponential backoff	Retry intervals that grow with every failed attempt
Jitter	Random delay added to spread retries over time
SSRF	Tricking a server into requesting internal addresses for an attacker

APPENDIX C FURTHER TOPICS

Webhook secret rotation

Shared secrets leak through logs, repositories and former employees, so they must be replaceable without losing events. Signature headers that carry several signatures (Stripe's repeated `v1` entries, Standard Webhooks' space-separated list) exist for this: during a grace period the provider signs with both the old and the new secret, and the receiver accepts either. A backend engineer should store secrets per endpoint, support two active secrets at once, and plan rotation as a routine operation rather than an emergency.

Asymmetric signatures with Ed25519

With public-key signatures the provider signs with a private key and receivers verify with a published public key, so no secret is shared and a compromised receiver cannot forge events. Discord signs interaction requests with Ed25519 in the `X-Signature-Ed25519` and `X-Signature-Timestamp` headers, and the Standard Webhooks specification defines an asymmetric variant as well. The scheme suits providers with many receivers and receivers that cannot keep secrets safely; verification is slightly costlier but still cheap.

Change data capture as an outbox relay

Instead of a worker polling the outbox table, a change-data-capture tool such as Debezium reads the database's transaction log and publishes each new outbox row to a broker such as Kafka. This removes polling load and delay and preserves commit order. It applies when outbox volume is high or several consumers need the events; the cost is running and monitoring the CDC pipeline.

Circuit breakers per endpoint

A circuit breaker stops calling a dependency after repeated failures and probes it occasionally until it recovers (see Chapter 12). In a webhook dispatcher, a breaker per subscriber endpoint prevents a dead endpoint from consuming worker time, connections and timeouts on every scheduled retry. It complements backoff: backoff spaces the attempts of one event, while the breaker pauses all deliveries to one endpoint at once.

Dead-letter queues and event replay

Events that exhaust their retries, on either side, should not vanish: they go to a dead-letter queue or a failed state from which they can be inspected and replayed once the cause is fixed. Providers expose this as a redeliver button or API; receivers need the same for jobs their workers could not process. Replay is safe only because the receiver is idempotent, so the two designs belong together.

Versioning webhook payloads

Webhook payloads are a public API, and changing their shape breaks every subscriber at once. Providers therefore version payloads, for example by pinning each endpoint to an API version, as Stripe does, and let subscribers upgrade deliberately. A provider should treat event schemas with the same compatibility rules as REST responses (see Chapter 7), and a receiver should record the payload version it was built against.

Idempotency keys for outgoing calls

A worker that processes a webhook often calls other APIs, such as charging a card or sending an email, and a retried job may repeat those calls. APIs that accept an idempotency key, such as Stripe's `Idempotency-Key` header, return the original result for a repeated key instead of acting twice. Deriving the key from the delivery ID carries the receiver's idempotence through to external side effects that a local database transaction cannot cover.

WebSub

WebSub, a W3C Recommendation since 2018, is the standardized successor of PubSubHubbub. It defines a hub that fans out content to subscribers, a verification-of-intent handshake in which the subscriber echoes a challenge, and HMAC signatures in an `X-Hub-Signature` header, the origin of GitHub's header name. It is a useful reference design for anyone building a general publish-subscribe webhook system.

DevOps for Backend Engineers — One-Page Overview

Chapter 27 · every section and sub-section of the paper at a glance

1 Introduction: Why DevOps Exists

Deployment made simple, cheap and boring ends the dev-versus-ops conflict.

- **1.1 Two teams, one conflict** — Speed versus stability; mutual blame; slow releases.
- **1.2 The DevOps movement** — Automation, Git for everything, one-step deploys, feature flags, shared observability.

2 Measuring Delivery: The DORA Metrics

Four numbers; the fastest teams are also the most stable.

- **2.1 Four metrics** — Lead time, deployment frequency, change failure rate, recovery time.
- **2.2 Speed and stability together** — Small deployments expose the faulty change at once.

3 Continuous Integration and Branching

Merge daily into one branch, verified by automated tests.

- **3.1 CI is a practice** — The server only checks; people must merge daily.
- **3.2 Branching models** — Git Flow for installed software; trunk-based plus flags for backends.

4 Builds, Pipelines and Versioning

A reproducible artifact moves through staged jobs to the cloud.

- **4.1 Reproducible builds** — Pinned toolchain, committed lock files, commit hash stamped in.
- **4.2 The deployment pipeline** — Cheap, fast stages first; production last.
- **4.3 GitHub Actions** — Workflow, event, job, runner, step.
- **4.4 Secrets versus OIDC** — Short-lived signed tokens instead of stored cloud keys.
- **4.5 Versioning deployments** — Semver for libraries, commit hashes for services.

5 Containers from the Kernel Up

An ordinary process, isolated, limited and given a layered file system.

- **5.1 Shipping the environment** — Runtime and libraries travel with the code.
- **5.2 Not a small virtual machine** — No own kernel; special treatment by the host kernel.
- **5.3 Namespaces** — Eight kinds; `clone`, `unshare`, `setns`.
- **5.4 Containers from scratch** — A short Go program with three clone flags.
- **5.5 Control groups** — Folders of limits; `memory.max` and the OOM killer.
- **5.6 The layered file system** — OverlayFS: shared read-only layers, thin writable top.

6 Images, Registries and Runtimes

Content-addressed layers, shipped by registries, started by runc.

- **6.1 The image format** — Layer blobs, config, manifest, SHA-256 digests.
- **6.2 Dockerfiles and caching** — Dependencies before source; multi-stage builds.
- **6.3 Registries** — Push and pull only missing layers.
- **6.4 From docker run to runc** — dockerd, containerd, runc, shim; OCI and CRI.
- **6.5 Good images and buildpacks** — Small, one process, non-root, pinned, scanned.

7 Running Services on One or a Few Machines

systemd and a TLS proxy on one machine; Kamal on several.

- **7.1 One machine** — Unit file, journal, reverse proxy behind DNS and firewall.
- **7.2 Automatic TLS** — ACME challenges, 90-day Let's Encrypt certificates.
- **7.3 Several machines with Kamal** — Start new, health-check, switch proxy, stop old.
- **7.4 Immutable infrastructure** — Servers are replaced, never modified.

8 Orchestration with Kubernetes

Controllers reconcile a declared desired state across machines.

- **8.1 Desired state and controllers** — Declare what, not how; control loops close the gap.
- **8.2 Control plane and nodes** — API server, etcd, scheduler, kubelet, kube-proxy.
- **8.3 Pods, Deployments and Services** — Ephemeral pods, labels, stable names, ingress.
- **8.4 Configuration and resources** — ConfigMaps, Secrets, requests and limits.
- **8.5 Rolling updates** — maxSurge, maxUnavailable, readiness, rollback.
- **8.6 Probes** — Readiness, liveness, startup; the restart-wave trap.
- **8.7 Scaling and adoption** — Change a number or use the HPA; start small.

9 Delivery Strategies, GitOps and Environments

How new versions replace old ones, and Git as source of truth.

- **9.1 Delivery versus deployment** — Technical property versus business decision.
- **9.2 Recreate and rolling update** — Downtime gap versus two versions at once.
- **9.3 Blue-green and canary** — Instant rollback at double cost; metric-guided rollout.
- **9.4 GitOps** — In-cluster agent applies Git; deploy by pull request.
- **9.5 Environments** — Same image, other config; previews and promotion.

10 Infrastructure as Code

Infrastructure declared in versioned files and created by tools.

- **10.1 The one-hour rebuild test** — Hand-built infrastructure cannot be reproduced.
- **10.2 Three tool families** — Terraform/OpenTofu, Ansible, Packer.
- **10.3 Terraform state** — Remote, locked, encrypted, never in Git.
- **10.4 Drift** — Console changes are reverted by the next apply.

11 Operating the Service: SLOs, Error Budgets and On-Call

Reliability measured from the user's side and spent like a budget.

- **11.1 SLI, SLO and SLA** — Measurement, target over a window, contract.
- **11.2 Nines and error budgets** — 99.9% is 43 minutes a month; empty budget, frozen deploys.
- **11.3 On-call and paging** — Weekly rotation answers the alerts.

DevOps at a glance — a commit merged daily is built reproducibly, packed into a container image, rolled out without downtime by an orchestrator or GitOps agent on infrastructure declared as code, and judged by user-facing SLOs. **Appendices** in the full paper: A — review questions, B — glossary, C — further topics.

DevOps for Backend Engineers: From Commit to Production

Chapter 27 · Backend Engineering Fundamentals

Prepared for personal study

Abstract — DevOps is introduced as the answer to the conflict between development and operations, and the four DORA metrics show that frequent, small deployments improve speed and stability together. Continuous integration, trunk-based development, reproducible builds and a GitHub Actions pipeline that reaches the cloud through OIDC carry a change to a versioned artifact. Containers are then built up from Linux namespaces, control groups and the layered OverlayFS file system, followed by the OCI image format, Dockerfile caching, registries and the runtime chain. Services run on one machine with systemd, a reverse proxy and ACME certificates, on several with Kamal, and on a cluster with Kubernetes, with rolling updates, probes and autoscaling. Deployment strategies, GitOps, environments and infrastructure as code with Terraform follow, and the chapter closes with SLIs, SLOs, SLAs, error budgets and on-call duty.

Keywords — DevOps, DORA metrics, continuous integration, containers, namespaces and cgroups, Kubernetes, GitOps, infrastructure as code, error budgets

1 INTRODUCTION: WHY DEVOPS EXISTS

Everything built in this course so far has run on one machine and answered on `localhost`, out of reach of users. This chapter follows a change from the developer's commit to a service that runs on the internet, stays up while its code changes every day, and grows from one machine to a fleet. That path is what is called *DevOps*, a combination of *development* and *operations*.

1.1 Two teams, one conflict

In medium and large companies the work was traditionally split between two teams. The development team — backend and frontend engineers, testers, everyone who writes code — is measured by how fast it builds features and gets them into production. The operations team sets up the servers, monitors them for failures and alerts, and is measured by how rarely production fails. Since every deployment may disturb the infrastructure and add chaos, operations wants as few deployments as possible, while developers want as many as possible. The conflict surfaces in every incident — developers blame the machines, operations blames the code — and releases slip to once a week or once a fortnight, because a release only succeeds when everyone is aligned.

1.2 The DevOps movement

The fixes that companies adopted and documented grew into the DevOps movement, a set of ideas for optimising development and deployment:

- **Automated infrastructure**: servers are created and configured by code, not by hand each time.
- **One version-control system**: the infrastructure configuration lives in Git next to the code.
- **One step to build, one to deploy**: a single command each, not a multi-stage procedure.
- **Feature flags**: functions are switched on or off from a central place, such as a database or a distributed cache, without redeploying the application.
- **Shared observability**: logs, alerts and metrics such as server CPU and memory are visible to developers too, so they see how their code behaves in production (see Chapter 14).

The common pattern is to make deployment **simple, cheap and boring**: one action — a merge into a given branch or a button — that starts the whole deployment in seconds instead of two or three hours of checking servers and settings. When deploying is that cheap, nobody hesitates to do it several times a day.

> **DevOps.** The practices and tools that join development and operations — automated, version-controlled infrastructure, one-step builds and deployments, feature flags and shared observability — so that shipping a change becomes small, frequent and routine.

2 MEASURING DELIVERY: THE DORA METRICS

2.1 Four metrics

The DORA group (DevOps Research and Assessment) has surveyed tens of thousands of teams for about a decade and reduced delivery performance to four numbers. Two measure speed. **Change lead time** runs from a commit to its running in production: a change merged into `main` (or whichever branch goes to production) at 9:00 and live for users at 11:00 has a lead time of two hours. **Deployment frequency** is how often code actually reaches production. Two measure stability. The **change failure rate** is the share of deployments that cause a failure in production — 4 out of 100 in a month give 4% — and the **recovery time** is how long it takes to recover from a failed deployment. Table 1 gives the 2024 figures for the best group, about 20% of teams worldwide, and for the weakest.

Metric	Kind	Best group	Weakest group
Change lead time	speed	under one day	—
Deployment frequency	speed	on demand	every 1–6 months
Change failure rate	stability	about 5%	about 40%
Recovery time	stability	under one hour	about a week

Table 1: The four DORA metrics and the 2024 figures quoted in the lecture.

2.2 Speed and stability together

The group that deploys most is also the most stable — the opposite of the operations instinct that every deployment adds chaos. Speed and stability are not a trade-off; with good practices a team gets both. The reason lies in what a deployment contains. A team that deploys monthly ships a month of work from 10–20 people, perhaps 200 commits at once. When production breaks — and it will, since no software is free of errors — finding the responsible commit can take most of a day, and the fix becomes another large deployment. A team that deploys each commit separately ships one change by one person, written an hour or a day earlier: the faulty change is obvious, and its author still remembers it and fixes it quickly, whereas after a month the details are forgotten. The rule: deploy **as often as possible, in changes as small as possible**.

3 CONTINUOUS INTEGRATION AND BRANCHING

3.1 Continuous integration is a practice

Continuous integration (CI) means that every developer merges their work into one shared branch at least once a day, and every merge is verified by an automated build that runs all the tests of the application. "CI" usually refers to the server that runs the tests on each push, but CI is the practice; the server is a tool that supports it. It can check whether a merge passes the tests; it cannot make people merge.

The daily rule exists because divergence grows with time. Two engineers who spend three weeks on a feature branch find, when they finally merge into `main` or `develop`, that it is so far behind that resolving the conflicts takes hours and risks new errors. With daily merges, each change is evaluated only against what is already in the shared branch: the divergence is one day of changes from, say, ten engineers, everyone remembers what they changed, and the conflicts that still arise are solved in minutes.

3.2 Branching models

The best-known model, still dominant, has a `develop` branch, release branches, hotfix branches and a `master` (now usually `main`) branch whose every change is final and ready for release. Ten years after publishing it, its author noted that continuously updated web applications should use something simpler. The model was designed for software whose versions are installed by customers — desktop or mobile applications that must support version 2 while version 3 is developed. A backend has only one version in production, the latest. The more efficient model is **trunk-based development**: one `main` branch, short-lived feature branches that live a day or two and are deleted after merging, and every deployment made from `main`. A feature that takes two or three weeks is merged in pieces every day or two and deployed behind a feature flag, so it is in production while users cannot reach it.

4 BUILDS, PIPELINES AND VERSIONING

4.1 Reproducible builds

A *build* turns source code into something that can run: it compiles the code, fetches every dependency and packs the result into an *artifact*, a binary or executable. As in the twelve-factor discussion (see Chapter 24), it must not depend on what happens to be installed on the machine: it declares its compiler version and the exact versions of its dependencies, so the same source with the same build configuration ideally yields a byte-identical result however often and wherever it is built. The lock file is the input that makes this possible. For a Go service the build is `go build` with a pinned Go version and a committed (not git-ignored) `go.sum`, which fixes the exact dependencies while the pinned toolchain fixes compiler and runtime. For Node.js it is `npm ci` with the committed `package-lock.json`; it installs exactly the locked versions and fails if they disagree with `package.json`. The artifact is usually stamped with the hash of the commit on `main` it was built from, so a running server can show whether it contains the latest commit.

4.2 The deployment pipeline

Builds and CI tests together form the *deployment pipeline*: a sequence of stages through which the source passes, the last one putting it into production. Stages run from fast and cheap to slow and expensive. The first compiles the code and runs the unit tests in a few minutes; if it fails, the pipeline stops at once and the developer who pushed is notified. Slower stages follow — integration tests against real databases, deployment to a staging environment, perhaps manual verification — and finally deployment to production.

4.3 GitHub Actions

GitHub Actions, one of the most used pipeline tools, rests on five words. A **workflow** is a YAML file committed under `.github/workflows/` that defines when the pipeline starts and what it runs. An **event** triggers it: a push, a pull request, a schedule or a manual start. A **job** is a set of steps executed on one machine; jobs run in parallel or wait for each other, as configured. The **runner** is that machine: a clean virtual machine that GitHub starts for the job and discards when it ends, successfully or not (or a self-hosted runner in one's own infrastructure). A **step** is one command or reusable action inside a job. Because a runner is always new, everything must be installed from scratch or restored from a cache, which is why every job starts with a checkout.

The lecturer's workflow, triggered by a push, has three jobs (Figure 1). The test job runs first on a fresh Linux runner: checkout, set up Go, run the tests. The build job waits for it, compiles the binary and uploads it as an artifact, shown on the run page with its size and hash. The image job waits for both and builds the container image (Section 6). With `needs:`, a job starts only if its dependencies succeeded unless a condition such as `if: always()` says otherwise; an artifact should never come from a commit whose tests failed.

```
# .github/workflows/ci.yml
on:
  push: { branches: [main] }
permissions:
  contents: read
  id-token: write        # lets jobs request an OIDC token
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with: { go-version: "1.23.4" }
      - run: go test ./...
  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with: { go-version: "1.23.4" }
      - run: >
          go build -o app
          -ldflags "-X main.commit=${{ github.sha }}" .
      - uses: actions/upload-artifact@v4
        with: { name: app, path: app }
  image:
    needs: [test, build]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/ci
          aws-region: eu-central-1
      - run: |               # $REG: the registry address
          docker build -t "$REG/api:${{ github.sha }}" .
          docker push "$REG/api:${{ github.sha }}"
```

Figure 1: A three-job workflow like the lecturer's; the binary and the image carry the commit hash, and the image job obtains cloud credentials through OIDC (illustrative).

4.4 Cloud credentials: secrets versus OIDC

At some stage the artifact must reach the cloud: an image is built and deployed to a machine or a cluster. The pipeline runs in GitHub's infrastructure and the cloud belongs to another company, so the pipeline needs credentials. For years the answer was *repository secrets*: keys saved per repository, referenced from workflow files and never displayed in logs. Their weakness is that every job and every action in the repository can read them, and a leaked key opens the infrastructure from anywhere until it is rotated.

The better mechanism is **OpenID Connect** (OIDC, Figure 2). When a job starts, GitHub's identity provider issues a short-lived token, a signed JSON Web Token stating that it belongs to a job of this repository, this branch and this workflow run. On the cloud side, say AWS, a trust relationship is configured in advance: AWS accepts tokens signed by GitHub only if their claims name this repository and the `main` branch. The job presents the token, AWS verifies the signature, repository and branch, and returns temporary credentials that expire shortly after the job and are useless anywhere else. No permanent cloud key is stored in the repository.

```
runner job              GitHub OIDC            AWS STS
  | request token  ----------> |
  | <---------- signed JWT     |
  |   iss  token.actions.githubusercontent.com
  |   sub  repo:example/api:ref:refs/heads/main
  |   aud  sts.amazonaws.com        (short-lived)
  |
  | AssumeRoleWithWebIdentity(role ARN, JWT) ---------->
  |        AWS: 1. signature valid (GitHub's public keys)?
  |             2. trust policy: sub == repo + main branch?
  | <---- temporary credentials (short-lived, one job)
```

Figure 2: A pipeline job exchanging a GitHub OIDC token for temporary AWS credentials (illustrative).

4.5 Versioning deployments

Library versions exist for their users and follow *semantic versioning*, `MAJOR.MINOR.PATCH`: a bug fix that changes no behaviour is a patch, a new function that breaks no existing behaviour or interface is a minor release, and a change that breaks users is a major release. A deployed service is different: its users are HTTP clients that do not care which deployment is running (route versioning such as `/v1` is a separate matter, see Chapter 7). The version is for the engineers, so services skip semantic versioning and label a deployment with the hash of the last commit on `main`, sometimes with a build identifier. The artifact carries this hash, the container image is tagged with it, and the backend returns it from an info or health endpoint, so after a deployment anyone can check that the latest code is what runs in production.

5 CONTAINERS FROM THE KERNEL UP

5.1 Shipping the environment

A Go binary that works on a laptop can fail when copied to a server with an older system library, a different certificate store or no time-zone database. A Node.js service written for Node 22 fails on a server with Node 18 if it uses a feature that Node 18 lacks. The solution is to ship the environment with the code — not a whole operating system, but the parts the process needs: runtime, libraries, files and their exact versions — packed into one unit that behaves the same on a laptop, a data-centre server, a rented virtual machine or a VPS. That unit is a *container*: a process with its own file system that sees only the small part of the machine it depends on.

5.2 A container is not a small virtual machine

A virtual machine (VM) is an environment created by software that partitions the hardware: it has virtual disks and its own operating-system kernel. A host with two VMs runs two kernels and two operating systems, and the host's virtualisation software (the hypervisor) isolates them. The

isolation is very strong, but VMs are heavy: each contains a full OS, needs gigabytes of RAM and its own disk, and boots slowly.

A container has no kernel of its own. The kernel, simplified, is the heart of the operating system, and a container is an ordinary process run by the host's kernel; on a Linux host, `ps` lists container processes among all the others. The isolation is special treatment by the host kernel, which gives the process an illusion of its own machine: its own process numbers, hostname, root file system and network interfaces, and limits on memory and CPU time. Three Linux kernel features create the illusion: **namespaces**, **control groups** and a **layered file system**. They are Linux features, which is why Docker on macOS and Windows runs containers inside a small Linux VM.

Aspect	Virtual machine	Container
Kernel	its own guest kernel	the host's kernel
Isolated by	hypervisor	namespaces, cgroups
Isolation	very strong	weaker (shared kernel)
Weight	full OS, GBs of RAM	runtime and libraries
Start	boots an OS	starts a process

Table 2: Virtual machines versus containers.

Container. An ordinary process run by the host's kernel, which gives it its own view of the machine through namespaces, limits its resources through control groups and gives it a root file system assembled from image layers.

5.3 Namespaces

A *namespace* wraps a global resource of the machine so that a group of processes sees its own copy. The operating system offers processes with IDs, hostnames, mount points and networking, and each has its own kind of namespace. A process in a new **PID** namespace sees only its own processes — itself as PID 1 and whatever it starts — and cannot see or reach anything outside. A **UTS** namespace gives it its own hostname, a **mount** namespace its own table of mount points, and a **network** namespace its own interfaces, ports and routing table, which is why two containers can both listen on port 80. Current Linux has eight kinds (Table 3).

Namespace	Gives the process its own …
PID	process numbers (it is PID 1)
Mount	mount table and root file system
Network	interfaces, ports, routing table
UTS	hostname
IPC	System V IPC objects, message queues
User	user and group IDs (root inside only)
Cgroup	view of the cgroup hierarchy
Time	boot-time and monotonic clocks

Table 3: The eight kinds of Linux namespaces.

Only the operating system can create and change namespaces, so programs ask for them through three system calls: `clone` creates a new process and, given the right flags, places it in new namespaces; `unshare` moves the calling process into new namespaces; `setns` joins an existing one. Every container runtime behind Docker and Kubernetes does all its namespace work with these three calls.

5.4 Containers from scratch

A container can be built by a very short program, from Liz Rice's talk "Containers from scratch" (about 76 lines of Go, Figure 3). The program runs itself twice. The parent calls `clone` with three flags — new UTS, new PID and new mount namespace — asking the kernel to run the same binary, with the argument `child`, as a child process. The child is PID 1 in its namespace. It sets its hostname to `container`, visible only in its UTS namespace; thanks to its mount namespace it changes its root to a folder holding an Ubuntu file system, which becomes `/`; it mounts a fresh `/proc` so that `ps` reads the new PID namespace; and it runs the requested command, a shell. Inside that shell, `hostname` answers `container`, `ps` lists only a few processes with the program itself as PID 1, and every file is an Ubuntu file. On the host, `pstree` shows the same shell as an ordinary process under the program, with its real host PID. That is all a container is.

```
func main() {
    switch os.Args[1] {
    case "run":   parent()    // ./demo run /bin/bash
    case "child": child()
    }
}

func parent() {
    cmd := exec.Command("/proc/self/exe",
        append([]string{"child"}, os.Args[2:]...)...)
    cmd.SysProcAttr = &syscall.SysProcAttr{
        Cloneflags: syscall.CLONE_NEWUTS |
            syscall.CLONE_NEWPID | syscall.CLONE_NEWNS,
    }
    must(cmd.Run())
}

func child() {             // PID 1 in the new namespaces
    must(syscall.Sethostname([]byte("container")))
    must(syscall.Chroot("/var/lib/demo/ubuntu-rootfs"))
    must(os.Chdir("/"))
    must(syscall.Mount("proc", "proc", "proc", 0, ""))
    cmd := exec.Command(os.Args[2], os.Args[3:]...)
    must(cmd.Run())        // the shell
}
```

Figure 3: The core of "Containers from scratch", condensed; I/O wiring and the error helper are omitted (illustrative).

5.5 Control groups

Namespaces control what a process can see, not how much it can consume: a container that sees only its own processes can still use all of the host's memory. *Control groups* (cgroups) limit consumption. A cgroup is simply a folder: the kernel exposes the hierarchy as a file system at `/sys/fs/cgroup`, and creating a folder there creates a group containing ordinary files such as `memory.max`, `cpu.max` and `pids.max`. A value written into `memory.max` limits the memory of every process in the group; a process ID written into `cgroup.procs` moves that process into it (Figure 4). When Kubernetes gives a container a 512 MB memory limit, that number ends up in `memory.max`, written by the con-

tainer runtime on the kubelet's behalf. When the container exceeds it, the kernel's out-of-memory (OOM) killer terminates the process and the container shows the familiar status `OOMKilled`. In the lecturer's demo, a program that keeps allocating memory inside a group grows to the limit and is killed there, and the kernel log states exactly why. A process that exceeds its CPU quota is throttled instead of killed.

```
$ mkdir /sys/fs/cgroup/demo           # a folder = a group
$ ls /sys/fs/cgroup/demo
cgroup.procs  cpu.max  memory.current  memory.max  pids.max
$ echo 512M > /sys/fs/cgroup/demo/memory.max
$ echo "50000 100000" > /sys/fs/cgroup/demo/cpu.max
$ echo 64 > /sys/fs/cgroup/demo/pids.max
$ echo $$ > /sys/fs/cgroup/demo/cgroup.procs
$ ./hog                         # allocates memory in a loop
Killed
$ dmesg | tail -n 1
Memory cgroup out of memory: Killed process 4242 (hog) ...
```

Figure 4: A cgroup made by hand (as root): 512 MB of memory, half a CPU (50 ms per 100 ms), 64 processes; the memory hog dies at the limit (illustrative).

5.6 The layered file system

The child program changed its root into a folder with about 100 MB of Ubuntu files. Copying 100 MB at every container start would be wasteful; container file systems are instead built from *layers*. A layer is a tar archive of changes, like a zip of a change set: files added, changed and deleted. An image is a stack of layers, and at run time the kernel assembles them with **OverlayFS**, which takes several read-only lower directories and one writable upper directory and presents a single merged view. A read finds the file in the highest layer that contains it; a write first copies the file into the upper layer and changes it there. The lower layers never change, so ten containers started from one image share every layer and each adds only a thin writable layer of its own. A file written inside a container lives only in that writable layer; this is why the container file system is called disposable, and data that must survive belongs elsewhere — a database, a volume or object storage (see Chapters 20 and 24).

A subtle point. The demo shows a written file vanishing once the container is stopped. Strictly, `docker stop` followed by `docker start` keeps the writable layer; it is deleted when the container is removed (`docker rm`, or automatically with `--rm`). The conclusion stands: every deployment starts new containers from the image, so nothing written to the container file system survives a release.

6 IMAGES, REGISTRIES AND RUNTIMES

6.1 The image format

A *blob*, roughly, is unstructured binary data: unlike a text file, a PDF or a video cannot be read or edited character by character, because its bytes make sense only as a whole. A container image has three parts. The **layers** are blobs identified by the SHA-256 hash of their bytes. The **config** is a JSON document listing the layers in order together with the run settings — command, environment, working directory, ports — that a Dockerfile describes. The **manifest** is a JSON document that points to the config and to each layer with its size and hash, and the image's identifier, its *digest*, is the hash of the manifest (Figure 5).

The hashes make sharing possible. Two images with a common base layer refer to the same hash, so a registry stores it once and a machine downloads it once. Changing a line in a Dockerfile changes only that layer and the ones after it; all earlier layers keep their hashes. Tags such as `ubuntu:24.04` or `myapp:v2.3.1` are names pointing to a digest: an image is a Git commit for a file system. In the lecturer's demo an image inspector shows a base layer, a layer adding certificates and a layer adding the 11 MB binary; selecting a layer shows the files it changed.

```
container A        container B
[ upper, rw ]      [ upper, rw ]      thin, per container
--------- merged view by OverlayFS ---------
layer 3  COPY api  (11 MB binary)     read-only,
layer 2  ca-certificates              shared by all
layer 1  base image                   containers

manifest -> config  sha256:9f2c...
              (command, env, workdir, ports, layer order)
         -> layer   sha256:a1b2...  (size, digest)
         -> layer   sha256:c3d4...
         -> layer   sha256:e5f6...
digest = sha256(manifest);  tag api:3f9c2ab -> digest
```

Figure 5: Layers merged by OverlayFS at run time, and the manifest that names them by digest (illustrative).

6.2 Dockerfiles, caching and multi-stage builds

A Dockerfile is the recipe for these layers: every instruction that changes the file system creates one layer. `FROM` chooses the base image, `COPY` adds files, `RUN` executes commands. The builder caches each layer keyed on the instruction and its inputs, and the cache is used from the top down to the first instruction whose inputs changed; everything below it is rebuilt. Order therefore matters. If the Dockerfile copies the whole source tree and then installs dependencies, every code change invalidates the copy and every build reinstalls all dependencies. If it first copies only the dependency manifest — `package.json`, `go.mod`, `Cargo.toml`, `requirements.txt` — installs the dependencies and only then copies the source, a code change rebuilds only the last layers. Code changes a hundred times a day; dependencies change rarely.

For compiled languages such as Go or Rust a *multi-stage build* goes further (Figure 6). The first stage holds the compiler and produces the binary; the second starts from a very small base image and copies in only the binary, since everything the program needs at run time is compiled into it. The compiler and build-only dependencies never reach the final image, which shrinks considerably.

```
# stage 1: build with the full Go toolchain
FROM golang:1.23.4 AS build
WORKDIR /src
ARG COMMIT
COPY go.mod go.sum ./
RUN go mod download     # cached while go.sum is unchanged
COPY . .                # code changes invalidate from here
RUN CGO_ENABLED=0 go build -trimpath \
      -ldflags "-X main.commit=${COMMIT}" -o /out/api .

# stage 2: tiny runtime image, no compiler, no shell
FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/api /api
USER nonroot
EXPOSE 8080
ENTRYPOINT ["/api"]
```

Figure 6: A cache-friendly multi-stage Dockerfile for a Go service: pinned base, dependencies before source, non-root user (illustrative).

6.3 Registries

A *registry* stores images and serves them to machines. The pipeline also pushes the artifact there after the build stage and pulls it before the deployment stage, because the two may run on different machines: an artifact kept only on the build machine is out of reach of the deploying one. A registry is an HTTP server with two kinds of objects: blobs, stored and fetched by digest, and manifests, fetched by tag or digest. A push uploads each layer the registry lacks — a lookup by hash tells whether it exists — then the config, then the manifest. A pull fetches the manifest, reads the layer digests and downloads only the layers missing locally. That is why a new version after a small code change is pulled in seconds: the base image and dependency layers are already on the machine, and only the last blob travels. Docker Hub is one registry; GitHub, AWS, GCP and Azure run their own, and a registry can even run in a container.

6.4 From docker run to runc

The `docker` command makes no system calls itself; it is only a client. It talks over a socket to `dockerd`, a background daemon, which hands the work to `containerd`, a smaller daemon that manages the life cycle of images and containers: pulling layers, setting up overlay mounts, supervising running containers. `containerd` invokes `runc`, which reads a JSON description of the container, makes the `clone` and `unshare` calls with the flags above, writes the cgroup files, changes the root directory and starts the process. Then `runc` exits, leaving a small *shim* as the parent of the container process.

The split exists because of a standard, the **Open Container Initiative** (OCI), which defines the image format, how a runtime is told to run an image, and how a registry serves images. An image built with Docker therefore runs in a cluster with no Docker at all: Kubernetes talks to `containerd` directly through the Container Runtime Interface (CRI), and `containerd` calls `runc`. The C code in `runc` creates namespaces in three stages — the parent clones a child, the child reads the flags, then it unshares into the new namespaces — with the same system calls as the small test program.

6.5 Good images and buildpacks

- **Small**: fewer files mean less to update and less to download on every deployment.
- **One process**: only the service — no init system, no cron daemon, nothing else.
- **Not root**: the process runs as an unprivileged user inside the container, so a security bug cannot escalate to root on the host kernel.
- **Pinned**: the base image has a specific version, not `latest`, so a build is reproducible tomorrow, next month or next year.
- **Scanned**: base images and their packages accumulate known vulnerabilities, published weekly; a scanner compares the image's package list with them and shows what to fix, update or remove — all the more important after recent supply-chain incidents.

Hosting platforms (PaaS) such as Render or Railway take a commit from GitHub and put the application on the internet without a Dockerfile. They use *buildpacks* or newer tools such as Nixpacks, which detect the language from `package.json`, `go.mod`, `requirements.txt` or `Cargo.toml` and build an image with a workflow already optimised for that runtime, leaving the developer to focus on the code.

7 RUNNING SERVICES ON ONE OR A FEW MACHINES

7.1 One machine

Running an image on a Linux machine means three things: a process that the operating system starts at boot and restarts whenever it stops; a reverse proxy in front of it on port 80, or 443 for HTTPS, that terminates TLS and forwards requests to the service's port; and logs stored where they can be read. The first is the job of **systemd**, the init system of every popular Linux distribution. A *unit file* (Figure 7) states what to run, as which user, and that it must be restarted on failure; for a container, the command is a `docker run`. Everything the process prints to standard output goes to the journal, read with `journalctl` and the unit name, and if the process exits, systemd starts it again after a delay. The reverse proxy is Nginx or Caddy. A request travels from the domain, whose DNS record at the registrar points to the machine's IP address, through a firewall that admits ports 80 and 443, to the reverse proxy and on to the process.

```
# /etc/systemd/system/api.service
[Unit]
Description=API container
After=docker.service
Requires=docker.service

[Service]
User=deploy
ExecStart=/usr/bin/docker run --rm --name api \
    -p 127.0.0.1:8080:8080 ghcr.io/example/api:3f9c2ab
ExecStop=/usr/bin/docker stop api
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

# /etc/caddy/Caddyfile: Caddy obtains the certificate itself
api.example.com {
    reverse_proxy 127.0.0.1:8080
}

$ sudo systemctl enable --now api
$ journalctl -u api -f
```

Figure 7: One machine: a systemd unit that runs and restarts the container, and Caddy as the TLS reverse proxy (illustrative).

7.2 Automatic TLS with ACME

To serve HTTPS instead of unprotected HTTP, the proxy needs a certificate. Getting one used to take many manual steps; today no human is involved, because the process became a protocol, **ACME**, and **Let's Encrypt** is a free certificate authority that runs it at large scale:

1. The ACME client on the server generates a key and tells the authority it wants a certificate for a name.
2. The authority sets a challenge: serve a random token at a given URL on port 80, or publish it as a DNS record.
3. The client completes it, and the authority fetches the token from several places on the internet, which proves that the requester controls the domain.
4. The client sends a signing request; the authority signs it and returns a certificate valid for 90 days.

The short validity costs nothing because clients renew automatically every 60 days, and a stolen key stays useful for only a few weeks. Caddy has the client built in: given a domain, it obtains, installs and renews the certificate itself. In the lecturer's demo a DNS record points to the machine, the Caddy configuration names the domain and the upstream port, and Caddy's log shows the certificate request, the challenge answered on port 80 and the issued certificate; a request from outside then shows the issuer and the expiry date.

7.3 Several machines with Kamal

As users grow, the next step is horizontal scaling — more servers rather than more RAM, storage or CPU in one (see Chapters 17–18). With three machines, deploying by hand becomes slow: build the image, run it on every machine, start the new version and stop the old one without a second of downtime, so that no user gets a 500 error. **Kamal** automates this from a local machine over SSH. It connects to each host, installs Docker if missing, logs into the registry, builds the image and pushes it. It then pulls the new image on every host and starts a new container while the old one still runs, and waits until the new version is healthy by polling an unauthenticated endpoint such as `/health` or `/status`. Only then does it tell the reverse proxy to route traffic to the new container (Kamal ships its own small proxy for this), and finally it stops the old containers.

7.4 Snowflakes and immutable infrastructure

All of this assumes the machine is valuable: it must be maintained, updated and protected, and one or two people are responsible for configuring and repairing it. That is the *snowflake* server of Chapter 24: it grows more complex and unique, only those people can maintain it, and their knowledge leaves with them. Hence the rules: never configure servers by hand, always use an automated, code-driven approach; and with three or four instances, never SSH into one to change it — change the code and redeploy the whole stack, replacing the old copies, or that server starts to differ from the others. This is **immutable infrastructure**: a server is never modified after launch, only replaced by a new version. Containers made the practice cheap, because the unit being replaced is an image, not a physical server with its operating system and configuration. Everything about clusters rests on this: servers are cheap, replaceable and never modified after launch.

8 ORCHESTRATION WITH KUBERNETES

8.1 Desired state and controllers

Cheap servers bring a new problem. With 40 containers on 10 machines: which container runs where? When a machine fails, who notices and starts replacements elsewhere? When traffic doubles, who launches more copies? When a new image is released, who replaces the 40 old containers with 40 new ones in the right order, without users noticing? A program that answers these questions is an *orchestrator*, and the best known is **Kubernetes**, which Google open-sourced on the basis of years of running its own infrastructure on similar internal systems, so the problems a critical system meets in the real world had long been met and fixed.

Kubernetes is not told what to do but what is wanted — the final state — and works out the steps itself. A document says, for example, that three copies of an image should run, listening on a given port, with a given memory limit. It is submitted to the **API server**, which stores it, and programs called **controllers** constantly watch the store. A controller compares what was requested with what exists: three copies wanted, two running, so it starts one; later a machine breaks, the count drops to two again, and it starts another. This comparison loop is the core of how Kubernetes works.

8.2 Control plane and nodes

The **control plane** consists of the API server — the interface to the cluster and the only component that talks to the store — and the store itself, **etcd**, a well-known distributed key-value database; the **scheduler**, a controller that

chooses a machine for each new pod; and the **controller manager**, which runs the built-in control loops. On every machine (*node*) runs **kubelet**, the node agent, which asks the API server what should run there and starts it through the container runtime, and **kube-proxy**, which implements Service networking.

8.3 Pods, Deployments and Services

The smallest object Kubernetes runs is a **pod**, not a container: one or more containers that run together on one machine and share a network namespace and volumes. Most pods hold one container; the extra level exists because a process sometimes needs a helper, such as a log shipper reading the same files or a proxy sharing `localhost`. The pod is the unit of scheduling: it gets its own IP address — so two pods can both listen on port 8080 — and it is destroyed when its time comes. Pods are never created by hand but declared in configuration files, which say what is wanted, not how. When a pod's machine fails, the pod is gone for good; a controller creates a new one elsewhere. Pods are grouped and found through **labels**, key-value pairs such as `app=api` or `version=v2`.

Three objects turn pods into a service (Figure 8). A **ReplicaSet** is a controller that keeps *n* pods of a template running. A **Deployment** sits one level above and manages ReplicaSets: when the template changes — a new image, say — it creates a new ReplicaSet and moves the replica count from the old one to it. Engineers write Deployments, never ReplicaSets. A **Service** is a fixed name, an address for a group of pods: pods are ephemeral by design and their IP addresses keep changing, while a Service has a stable virtual IP and DNS name and forwards to the pods matching its selector; other services in the cluster call it by name. Traffic from the internet enters through an **Ingress**, or its newer replacement the **Gateway API**, which maps host names and paths to Services, while a controller provisions the actual load balancer. The full path is domain, load balancer, Service, pods; only the pods change at each deployment.

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: api }
spec:
  replicas: 3
  selector: { matchLabels: { app: api } }
  strategy:
    rollingUpdate: { maxSurge: 25%, maxUnavailable: 25% }
  template:
    metadata: { labels: { app: api, version: v2 } }
    spec:
      containers:
        - name: api
          image: ghcr.io/example/api:3f9c2ab
          ports: [{ containerPort: 8080 }]
          envFrom:
            - configMapRef: { name: api-config }
            - secretRef: { name: api-secrets }
          resources:
            requests: { cpu: 250m, memory: 256Mi }
            limits:   { cpu: 500m, memory: 512Mi }
          readinessProbe:
            httpGet: { path: /health, port: 8080 }
          livenessProbe:
            httpGet: { path: /livez, port: 8080 }
---
apiVersion: v1
kind: Service
metadata: { name: api }
spec:
  selector: { app: api }
  ports: [{ port: 80, targetPort: 8080 }]
```

Figure 8: A Deployment and its Service, with configuration, resources and probes (illustrative).

8.4 Configuration, Secrets and resources

Configuration lives in a **ConfigMap**, key-value pairs mounted into containers as environment variables or files; it is not confidential, and exposure does no harm. Credentials and API keys live in a **Secret**, structured like a ConfigMap but provided only to pods that explicitly ask for it (see Chapter 13).

> **Warning.** A Kubernetes Secret is by default only base64-encoded in etcd, not encrypted. It becomes confidential only with encryption at rest for etcd and access rules (RBAC) that limit who may read it; many teams keep secrets in an external store and sync them in.

Each container also declares **resources**. The *request* is what the machine reserves for the container (the scheduler places pods by requests); the *limit* is the ceiling it may not exceed. These are the cgroups of Section 5.5: a 512 MB memory limit is written to `memory.max` and the CPU limit, a quota, to `cpu.max`. A container that exceeds its memory limit is killed by the kernel, the pod reports `OOMKilled`, and kubelet restarts it.

8.5 Rolling updates

When a new image is set, the Deployment controller creates a new ReplicaSet and shifts replicas to it in steps bounded by two numbers: `maxSurge`, how many pods above the desired count may exist during the change, and `maxUnavailable`, how many below it are allowed. Both default to 25%. With three replicas (25% of three rounds up to one extra pod and down to zero missing ones), the controller scales the new set up by one and waits for that pod to be

ready, scales the old set down by one, and repeats until the new set has three and the old one zero (Figure 9).

Two safety mechanisms are built in. The controller waits for *readiness*: a new pod gets no traffic until it reports ready, so a broken image can only stall the rollout, never take the service down. And the old ReplicaSet is scaled to zero but not deleted, so rolling back is easy. In the lecturer's demo three pods run the old version; after the image changes, one new pod appears and waits for readiness, one old pod terminates, and so on, one by one. Both versions are live for about 30 seconds and no request fails: zero downtime.

```
replicas 3; maxSurge 25% -> 1; maxUnavailable 25% -> 0
old v1  new v2  Deployment controller
  3       0     template changed: create a new ReplicaSet
  3       1     surge: start one new pod, wait for readiness
  2       1     ready: scale the old ReplicaSet down by one
  2       2     start another new pod, wait for readiness
  1       2     scale the old set down
  1       3     start the third new pod, wait
  0       3     done; old ReplicaSet kept with 0 replicas
```

Figure 9: A rolling update of three replicas with the default limits (illustrative).

8.6 Probes

Readiness is exposed by the service's own code, usually an endpoint such as `/health` or `/status` whose 200 response means the server works. Kubelet checks it with a *probe*, a periodic check that runs a command, requests an HTTP path or opens a TCP port. There are three probes (Table 4): a failing **readiness** probe removes the pod from the Service's endpoints and a passing one adds it back, with no restart; after several consecutive **liveness** failures kubelet kills and restarts the container; a **startup** probe holds the other two back until a slow-starting process is up.

Probe	Question	Effect of failure
Readiness	Can it serve now?	out of Service endpoints; no restart
Liveness	Is it alive, not hung?	container killed and restarted
Startup	Has it finished starting?	other probes wait until it passes

Table 4: The three Kubernetes probes.

Warning. A liveness probe that checks the database turns a slow database into an outage. When the database slows down, the probe times out and kubelet restarts perfectly healthy processes; all pods start a wave of restarts, and the reconnecting pods slow the database further. Liveness should check only whether the process is hung; readiness checks whether it can serve requests right now.

8.7 Scaling, and whether to adopt Kubernetes

Inside an orchestrator, scaling is a change of one number: setting the Deployment's replicas from 3 to 10 in the file makes the ReplicaSet controller start seven more pods, and nothing else is needed. The **Horizontal Pod Autoscaler** changes the number automatically: it tracks average CPU load across the pods, adds pods when the load exceeds the target and removes them when it falls, within a minimum and maximum fixed in advance.

Whether a team needs Kubernetes at all is a frequent question. The shared view of many specialists is to start small and grow the infrastructure with the users, so that Kubernetes becomes a natural step rather than the default: first one machine with systemd, Docker Compose or a hosting platform; with significant traffic, several machines with a tool like Kamal; a cluster only when the scale demands it.

9 DELIVERY STRATEGIES, GITOPS AND ENVIRONMENTS

9.1 Continuous delivery and continuous deployment

Continuous delivery means the software is always deployable: every merged commit passes the full pipeline, and anyone can deploy the latest code to production at any moment with one button, without fearing that something breaks. **Continuous deployment** means the pipeline presses that button itself: every change that passes all checks goes to production automatically, several times a day. Continuous delivery is a technical property; continuous deployment is mainly a product and business decision. A team with continuous delivery may still release weekly because customers want a weekly changelog or compliance rules require checks. The reverse is impossible: continuous deployment requires continuous delivery first.

9.2 Recreate and rolling update

The simplest way to replace a version, **recreate**, stops all old copies and then starts the new ones. The gap between the two, even a few seconds, is downtime: neither version runs, and a request from the frontend fails with a blank page, an empty state or other behaviour users notice. The Kubernetes default is therefore the **rolling update**, replacing one copy at a time after its readiness check. Despite its zero-downtime promise it has the catch seen in the demo: both versions run at once, and a request may land on either. A rolling update suits only changes for which old and new versions serving side by side is acceptable.

9.3 Blue-green, canary and progressive delivery

Blue-green deployment guarantees zero downtime and only one version serving at any moment. Two complete production environments exist; the colours are arbitrary but conventional. Blue is current and receives all traffic. The new version is deployed to green, which has no user traffic, and tested there; when the team is satisfied, the router is switched and all traffic goes to green. Blue still runs the previous version, so if problems appear five or ten minutes later, traffic switches back instantly. The price is cost: the infrastructure is doubled while both run.

Canary deployment, a kind of A/B strategy, releases gradually. With ten pods, the new version goes to one, which receives 10% of the traffic while nine serve the rest. Error rate, latency and other metrics of the new pod are compared with the old ones; if they look normal, coverage

grows to 20%, 50%, 75% and finally 100%, and if they worsen, traffic is routed back. **Progressive delivery** automates the canary: a controller widens coverage on a schedule, compares the same metrics a person would at every step, and rolls back by itself when a check fails.

Strategy	Downtime	Versions serving	Cost and rollback
Recreate	yes, a gap	one	cheap; redeploy old
Rolling update	none	both during rollout	cheap; old set kept
Blue-green	none	one	double; instant switch
Canary	none	both, new on a share	metric-driven; route back

Table 5: Deployment strategies compared.

9.4 GitOps

So far the pipeline deploys to the cluster itself. Many large teams instead use **GitOps**, best defined by its properties since it is a practice rather than a tool. The desired state of the whole system — Deployments, Services, ConfigMaps — is described declaratively in files that state the final state, not the steps. The files live in Git with any provider (GitHub, GitLab, Bitbucket), so every change is versioned, reviewed and revertible. An agent inside the cluster pulls them and applies them continuously; a change made directly in the cluster without the files is detected and reverted, which makes Git the single source of truth. Each deployment is a pull request that changes the container image hash, and merging it deploys. The pipeline holds no cluster credentials, because it never talks to the cluster: it pushes the image and opens the pull request. The best-known agents, Argo CD and Flux, are themselves controllers. The lecturer presents GitOps as the balanced middle between simple setups (systemd on one machine, Kamal on a few) and a cluster whose Services and Deployments people configure and repair by hand.

9.5 Environments

Environments are where the pipeline's stages live — development, testing, staging, production — and, as in the twelve-factor discussion (see Chapter 24), each runs the same code and the same image with different configuration. A **preview environment** is a temporary environment created for each pull request, so the author and the reviewer can test the change live instead of only reading the code; it is destroyed when the pull request is merged or closed. **Promotion** moves the built artifact from one environment to the next, usually through an approval step in which a person decides whether the code goes forward.

10 INFRASTRUCTURE AS CODE

10.1 The one-hour rebuild test

Deployments and clusters run on machines, and so far someone configured those by hand — the cluster, the EC2 hosts, the load balancer, DNS records, security groups — through the AWS console or CLI. The test: given a new cloud account and one hour, could the infrastructure be rebuilt identical to the existing one, with every setting and edge case? Perhaps, if its builder documented every step on every screen — and even then hardly within an hour without mistakes. Without the document it is much harder; without the person it is practically impossible. **Infrastructure as code** (IaC) answers it: machines, networks, access rights and configuration are written in files under version control, and a tool reads them and creates exactly that infrastructure in an AWS, Azure or Google Cloud account — the same infrastructure however many times it runs.

10.2 Three tool families

Provisioning, the family a backend engineer uses most, creates machines, networks and cloud services. The most popular tool is **Terraform**: resources are declared in blocks, `terraform plan` shows everything that will be created or changed, and once the plan looks right it is approved and applied. Terraform calls the cloud's API internally, so the engineer needs Terraform's syntax, not every cloud API. After Terraform's licence changes, **OpenTofu** is a fully open-source fork that does almost the same and belongs to no corporation. **Configuration management** installs and configures software on machines from files or scripts instead of by hand; the common tool is **Ansible**. **Image baking** builds a machine image, like a container image, with everything pre-installed, so a new machine is ready from boot; the best-known tool is **Packer**, with **cloud-init** running a script at first boot.

Family	What it does	Typical tools
Provisioning	creates machines, networks, services	Terraform, OpenTofu
Configuration	installs and configures software	Ansible
Image baking	pre-built machine images	Packer, cloud-init

Table 6: The three families of infrastructure-as-code tools.

10.3 Terraform state

When Terraform creates a machine from a block, it must remember that this block corresponds to that machine, identified by the provider's resource ID. It keeps the mapping, with every attribute of the resource, in a **state file**. On the next run it reads the state, asks the cloud for the current state of those resources, compares them with the configuration, and shows the differences in the plan. Three consequences follow. If the state file is lost, Terraform loses its memory of the infrastructure, concludes that none exists and plans to create everything again. If two people run Terraform at once with two copies of the state, they overwrite each other; the state therefore lives in central remote storage with locking, never on laptops. And the state holds every attribute the cloud returns, secrets included, so it must never reach Git and must be stored protected and encrypted.

10.4 Drift

Drift is the gap between the code and reality. An urgent problem arises, and instead of changing the Terraform files and re-applying, someone edits a security group directly in the AWS console or CLI. The files and state still describe the old configuration, so on the next `plan` Terraform sees a change that exists in the cloud but not in the code and, the code and state being its only source of truth, proposes to remove it; applying the plan undoes every console change. In the lecturer's demo, a plan for two machines and a security group first reports nothing to change. Port 22 is then opened to the whole internet through the CLI; the next plan shows one change, closing the port, and applying it closes the port again (Figure 10).

```
resource "aws_security_group" "api" {
  name = "api"
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

$ terraform plan
No changes. Your infrastructure matches the configuration.

$ aws ec2 authorize-security-group-ingress \
    --group-id sg-0abc123 --protocol tcp --port 22 \
    --cidr 0.0.0.0/0          # made outside Terraform

$ terraform plan
  # aws_security_group.api will be updated in-place
  ~ resource "aws_security_group" "api" {
      ~ ingress = [
          - { from_port = 22, to_port = 22, ... },
            # (1 unchanged element hidden)
        ]
    }
Plan: 0 to add, 1 to change, 0 to destroy.
```

Figure 10: Drift: a port opened outside Terraform is closed again by the next apply (illustrative, security group only).

11 OPERATING THE SERVICE: SLOS, ERROR BUDGETS AND ON-CALL

11.1 SLI, SLO and SLA

Once the service runs, the question is how to know, as a number, what works and what does not. Continuous deployment, cheap servers and infrastructure as code are internal practices; whether servers are configured by hand or automatically does not matter to users, who care only about results. A **service level indicator** (SLI) measures a property users care about: the share of requests that succeed, or the share served faster than 300 ms. A **service level objective** (SLO) is a target for an SLI over a period, such as 99% successful requests over the last 30 days; an SLO always has a time window. A **service level agreement** (SLA) is a contract with customers built on SLIs and SLOs, with consequences: if 99.9% of requests over the last 30 days did not succeed, the provider refunds part of the fee, extends the plan for some months or pays compensation.

11.2 Nines and error budgets

Availability of 99.9% over 30 days allows only about 43 minutes of failure in the month; 99.99% allows about 52 minutes in a whole year, roughly 4.4 minutes a month (Table 7). Every added nine promises the customer ten times more.

Availability	Downtime per 30 days	Downtime per year
99%	7.2 hours	3.65 days
99.9%	43.2 minutes	8.8 hours
99.99%	4.3 minutes	52.6 minutes
99.999%	26 seconds	5.3 minutes

Table 7: Allowed downtime per availability target (the 99% and 99.999% rows are computed, not from the lecture).

A 99.9% target permits 0.1% failure, and teams turn that tolerance into an **error budget**, tracked actively like any other budget and spent deliberately on deployments, maintenance windows, expected failures, experiments and risky migrations. It is no licence for careless code; it only acknowledges a few short windows in which unavoidable failures may occur. While budget remains, teams deploy as soon as possible, many times a day. Once it is exhausted — say on the 23rd, after yesterday's outage used up the 43 minutes — deployments freeze until the budget recovers at the end of the window, and all effort goes into reliability.

Error budget. The unreliability an SLO permits over its window — for 99.9% over 30 days, about 43 minutes or 0.1% of requests — tracked as a resource that funds deployments and experiments and, once exhausted, freezes releases.

11.3 On-call and paging

Because the indicators are quantified, they can be tracked, and when one falls someone must look at it, find the cause and fix it. That person is **on call** under an *on-call rotation*: in a team of three engineers, one engineer per week is responsible for responding to every *page*. The term comes from pagers, devices that notified people of emergencies, schedules and meetings before mobile phones were common. Nobody carries one today, but the word stayed: paid incident-management and paging services put an app on the engineer's phone or laptop that raises the alert when an indicator falls. The transcript ends at this point, in the middle of the description of paging.

12 CONCLUSION

- DevOps makes deployment simple, cheap and boring; DORA data show that frequent, small deployments improve speed and stability together.
- Continuous integration means merging into one shared branch at least daily; backends use trunk-based development, with feature flags for long features.
- Builds are reproducible (pinned toolchain, committed lock files), artifacts and images carry the commit hash, and pipelines reach the cloud with short-lived OIDC credentials.

- A container is an ordinary Linux process: namespaces decide what it sees, cgroups how much it uses, OverlayFS layers which files it has.
- Images are content-addressed layer stacks; Dockerfile order and multi-stage builds decide build time and size; good images are small, single-process, non-root, pinned and scanned.
- One machine needs systemd, a reverse proxy with automatic ACME certificates, and logs; servers are immutable and replaced, never patched by hand.
- Kubernetes reconciles declared state: Deployments, Services, requests and limits, rolling updates gated by readiness — and liveness probes must never depend on the database.
- Recreate, rolling, blue-green and canary trade downtime, version mixing and cost; GitOps and Terraform make Git the source of truth for cluster and infrastructure.
- SLIs, SLOs and error budgets measure reliability from the user's side and decide when to ship and when to freeze; the on-call engineer answers the pages.

APPENDIX A REVIEW QUESTIONS

1. A team of twelve deploys every three weeks. The last release bundled about 180 commits and broke checkout for six hours while engineers hunted for the faulty change; management now proposes monthly releases with a two-day manual QA phase. Predict how each DORA metric would move under this proposal and under daily trunk-based deployments, and specify the practices the team would need to make the second option safe for a feature that takes three weeks to build.
2. A Node.js service is built from `FROM node:latest`, `COPY . .`, `RUN npm install`, `CMD ["node", "server.js"]`. Builds take six minutes on every commit, the image is 1.1 GB, the process runs as root, and two builds of the same commit a month apart behave differently. Rewrite the Dockerfile, state which layers are rebuilt after a one-line change to `server.js` before and after the rewrite, and name the image property each change restores.
3. At 14:02 database latency rises from 20 ms to 4 s. By 14:05 all 12 API pods show climbing restart counts, the database CPU is at 100%, and even endpoints that never touch the database return errors. Each liveness probe calls `GET /health`, which runs `SELECT 1` with a 1 s timeout and three allowed failures. Reconstruct the chain of events and redesign the probes so the same slowdown degrades the service instead of taking it down.
4. A Deployment has 4 replicas with `maxSurge` and `maxUnavailable` at 25%. A new image is rolled out whose readiness probe never succeeds because a required environment variable is missing. Describe the pod counts of both ReplicaSets and what users experience over the next ten minutes, how the team recovers, and how the outcome would differ under the recreate strategy.
5. A release renames a JSON response field and the database column behind it. The service runs 20 pods, its SLO is 99.9% successful requests per 30 days, and 70% of this window's error budget is already spent. Compare rolling update, blue-green and canary for this release, explain how two versions serving at once affects the handling of the field and the column, and justify your choice.
6. During an incident an engineer opens port 22 to `0.0.0.0/0` on a security group through the AWS console. The next day a colleague runs `terraform apply` from a laptop for an unrelated change, using a state file copied a month ago. Predict what the plan shows and what happens to the port and to other resources, then design a state-handling and change process that prevents both problems.
7. A workflow keeps a long-lived AWS access key as a repository secret and uses a third-party linting action in one job; that action's maintainer account is compromised. Explain what the attacker can do and for how long, redesign the setup with OIDC including the trust-policy conditions, and compare the result with a GitOps setup in which the pipeline holds no cluster credentials.
8. A service handles 50 million requests per 30 days against a 99.9% success SLO. On the 20th, a bad deployment fails 5% of requests for 90 minutes at 20,000 requests per minute. Compute the share of the error budget the incident consumed, decide what the team should do for the rest of the window, and estimate what a canary at 10% of traffic with automatic rollback after five minutes would have cost instead.

APPENDIX B GLOSSARY

Term	Meaning
DORA metrics	Lead time, deployment frequency, change failure rate, recovery time
Continuous integration	Merging into a shared branch at least daily, verified by tests
Trunk-based development	One main branch, short-lived branches, deploys from main
Feature flag	Runtime switch that enables or hides a function without deploying
OIDC token	Short-lived signed JWT proving a job's repository and branch
Namespace	Kernel feature giving processes their own view of a global resource
Cgroup	Kernel folder of resource limits for a group of processes
OverlayFS	Merges read-only image layers with one writable layer
Image digest	SHA-256 hash of an image manifest; tags point to it
OCI	Standards for image format, runtime and registry distribution
Immutable infrastructure	Servers are replaced, never modified after launch
Reconciliation loop	Controller logic driving actual state towards desired state
Readiness probe	Check deciding whether a pod receives traffic
GitOps	Declarative state in Git applied continuously by an in-cluster agent
Drift	Difference between infrastructure code and real infrastructure
Error budget	Failure an SLO permits; spent on change, frozen when exhausted

APPENDIX C FURTHER TOPICS

Expand–contract schema changes

Expand–contract (parallel change) splits a breaking change to a schema or API into backward-compatible steps: add the new column or field, deploy code that writes both and reads the new one, backfill, and remove the old one only in a later release. Rolling updates and canaries run two versions side by side, and a rollback runs old code against the new schema, so every step must work with both. It applies to every rename, type change or column removal in a continuously deployed service.

Graceful termination of pods

When Kubernetes stops a pod it sends SIGTERM, removes the pod from Service endpoints in parallel, and sends SIGKILL after `terminationGracePeriodSeconds` (30 s by default). The backend must stop taking new work, finish in-flight requests and close connections in that window, often after a short `preStop` delay so load balancers stop routing first. Without this, every rolling update drops requests despite readiness checks; Chapter 15 covers the application side.

Burn-rate alerting on SLOs

Instead of alerting on raw thresholds, teams alert on how fast the error budget is being spent: a burn rate of 1 uses the budget exactly over the window, while a burn rate of 14.4 uses 2% of a 30-day budget in one hour. Combining a short and a long window pages quickly for severe incidents and stays quiet for brief blips. It is the standard way to connect SLOs to the on-call rotation.

Blameless postmortems and incident roles

After an incident, a postmortem records the timeline, impact, causes and follow-up actions without blaming individuals, so people report problems honestly. In larger incidents, explicit roles — an incident commander, a communications lead, responders — keep the work coordinated. These practices turn pages into lasting reliability improvements instead of repeated firefighting.

Software supply-chain security

Beyond scanning, teams generate a software bill of materials (SBOM) for each image, sign images (for example with Sigstore's cosign) and record build provenance following the SLSA framework, then admit only signed images into the cluster. This defends against tampered dependencies, compromised build systems and images pushed from outside the pipeline. It matters most for services with sensitive data or compliance duties.

Container hardening beyond a non-root user

The kernel offers further restrictions: dropping Linux capabilities, seccomp profiles that block unneeded system calls, a read-only root file system, and user namespaces that map root inside the container to an unprivileged user outside. In Kubernetes they are set in the pod's `securityContext` and enforced cluster-wide with Pod Security Standards. Because containers share the host kernel, each restriction narrows what an exploited service can do.

Cluster autoscaling

The Horizontal Pod Autoscaler adds pods, but pods need nodes with free capacity for their requests. A cluster autoscaler (the Kubernetes Cluster Autoscaler, or Karpenter on AWS) adds machines when pods cannot be scheduled and removes underused ones. Without it a traffic spike yields pending pods instead of capacity, and with it accurate resource requests drive both scheduling and cost.

Packaging manifests with Helm and Kustomize

Real services need the same Deployment, Service and ConfigMap with different values per environment. Helm packages manifests as versioned charts with templated values; Kustomize applies patches (overlays) to a common base without templating. GitOps agents such as Argo CD and Flux use both to render each environment's desired state from one source.

References

The sources cited by each chapter, in chapter order

Chapter 1 · The HTTP Protocol

1. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
2. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Caching*, RFC 9111, IETF, 2022.
3. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP/1.1*, RFC 9112, IETF, 2022.
4. M. Thomson and C. Benfield (Eds.), *HTTP/2*, RFC 9113, IETF, 2022.
5. M. Bishop (Ed.), *HTTP/3*, RFC 9114, IETF, 2022.
6. WHATWG, *Fetch Standard* (CORS protocol), Living Standard, https://fetch.spec.whatwg.org/.
7. L. Masinter, *Returning Values from Forms: multipart/form-data*, RFC 7578, IETF, 2015.
8. E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF, 2018.

Chapter 2 · Routing

1. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
2. T. Berners-Lee, R. Fielding and L. Masinter, *Uniform Resource Identifier (URI): Generic Syntax*, RFC 3986, IETF, 2005.
3. J. Gregorio, R. Fielding, M. Hadley, M. Nottingham and D. Orchard, *URI Template*, RFC 6570, IETF, 2012.
4. E. Wilde, *The Sunset HTTP Header Field*, RFC 8594, IETF, 2019.
5. R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, doctoral dissertation, University of California, Irvine, 2000.
6. WHATWG, *URL Standard*, Living Standard, https://url.spec.whatwg.org/.
7. Microsoft, *Routing in ASP.NET Core*, ASP.NET Core documentation, Microsoft Learn.

Chapter 3 · Serialization & Deserialization

1. T. Bray (Ed.), *The JavaScript Object Notation (JSON) Data Interchange Format*, RFC 8259, IETF, 2017.
2. Ecma International, *ECMA-404: The JSON Data Interchange Syntax*, 2nd edition, 2017.
3. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
4. ISO/IEC 7498-1:1994, *Information technology — Open Systems Interconnection — Basic Reference Model: The Basic Model*, ISO/IEC, 1994.
5. T. Bray, J. Paoli, C. M. Sperberg-McQueen, E. Maler and F. Yergeau (Eds.), *Extensible Markup Language (XML) 1.0 (Fifth Edition)*, W3C Recommendation, 2008.
6. O. Ben-Kiki, C. Evans and I. döt Net, *YAML Ain't Markup Language (YAML) Version 1.2*, yaml.org, 2009.
7. Google, *Protocol Buffers Documentation*, protobuf.dev.
8. M. Kleppmann, *Designing Data-Intensive Applications*, Chapter 4: Encoding and Evolution, O'Reilly Media, 2017.

Chapter 4 · Authentication & Authorization

1. W. Diffie and M. E. Hellman, *New Directions in Cryptography*, IEEE Transactions on Information Theory 22(6), 644–654, 1976.
2. R. Morris and K. Thompson, *Password Security: A Case History*, Communications of the ACM 22(11), 594–597, 1979.
3. A. Barth, *HTTP State Management Mechanism*, RFC 6265, IETF, 2011.
4. D. Hardt (Ed.), *The OAuth 2.0 Authorization Framework*, RFC 6749, IETF, 2012.
5. M. Jones, J. Bradley and N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.
6. N. Sakimura, J. Bradley, M. Jones, B. de Medeiros and C. Mortimore, *OpenID Connect Core 1.0*, OpenID Foundation, 2014.
7. T. Lodderstedt, J. Bradley, A. Labunets and D. Fett, *Best Current Practice for OAuth 2.0 Security*, RFC 9700, IETF, 2025.
8. OWASP Foundation, *Authentication Cheat Sheet* and *Password Storage Cheat Sheet*, OWASP Cheat Sheet Series, cheatsheetseries.owasp.org.

Chapter 5 · Validation & Transformation

1. OWASP Foundation, *Input Validation Cheat Sheet*, OWASP Cheat Sheet Series, cheatsheetseries.owasp.org.
2. A. Wright, H. Andrews, B. Hutton and G. Dennis, *JSON Schema: A Media Type for Describing JSON Documents*, Draft 2020-12, json-schema.org, 2020.
3. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
4. M. Nottingham, E. Wilde and S. Dalal, *Problem Details for HTTP APIs*, RFC 9457, IETF, 2023.
5. ITU-T, *Recommendation E.164: The international public telecommunication numbering plan*, International Telecommunication Union, 2010.
6. G. Klyne and C. Newman, *Date and Time on the Internet: Timestamps*, RFC 3339, IETF, 2002.
7. Unicode Consortium, *Unicode Normalization Forms*, Unicode Standard Annex #15.
8. A. King, *Parse, don't validate*, essay, lexi-lambda.github.io, 2019.

Chapter 6 · Handlers, Services, Repositories & Middleware

1. R. Fielding, M. Nottingham and J. Reschke (eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
2. M. Nottingham and R. Fielding, *Additional HTTP Status Codes*, RFC 6585, IETF, 2012.
3. WHATWG, *Fetch Standard* (the CORS protocol), fetch.spec.whatwg.org.
4. M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002 (Service Layer, Repository, Unit of Work).
5. The Go Authors, *Package context*, Go standard library documentation, pkg.go.dev/context.
6. Microsoft, *ASP.NET Core Middleware*, ASP.NET Core documentation, learn.microsoft.com.
7. OpenJS Foundation, *Using middleware* and *Writing middleware for use in Express apps*, Express documentation, expressjs.com.

Chapter 7 · REST API Design

1. R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, doctoral dissertation, University of California, Irvine, 2000 (Chapter 5: Representational State Transfer).

2. R. T. Fielding and R. N. Taylor, "Principled Design of the Modern Web Architecture", *ACM Transactions on Internet Technology*, 2(2), 2002, pp. 115–150.
3. R. Fielding, M. Nottingham and J. Reschke (eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
4. L. Dusseault and J. Snell, *PATCH Method for HTTP*, RFC 5789, IETF, 2010.
5. T. Berners-Lee, R. Fielding and L. Masinter, *Uniform Resource Identifier (URI): Generic Syntax*, RFC 3986, IETF, 2005.
6. T. Bray (ed.), *The JavaScript Object Notation (JSON) Data Interchange Format*, RFC 8259, IETF, 2017.
7. M. Nottingham, E. Wilde and S. Dalal, *Problem Details for HTTP APIs*, RFC 9457, IETF, 2023.
8. OpenAPI Initiative, *OpenAPI Specification*, version 3.1.0, 2021, spec.openapis.org.

Chapter 8 · Databases with PostgreSQL

1. E. F. Codd, "A Relational Model of Data for Large Shared Data Banks", *Communications of the ACM*, 13(6), 377–387, 1970.
2. R. Bayer and E. McCreight, "Organization and Maintenance of Large Ordered Indexes", *Acta Informatica*, 1(3), 173–189, 1972.
3. The PostgreSQL Global Development Group, *PostgreSQL Documentation*: Data Types; Data Definition; Indexes; Triggers; PL/pgSQL, postgresql.org/docs.
4. ISO/IEC 9075-2:2023, *Information technology — Database languages SQL — Part 2: Foundation (SQL/Foundation)*, ISO, 2023.
5. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
6. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
7. OWASP Foundation, *SQL Injection Prevention Cheat Sheet*, OWASP Cheat Sheet Series.
8. A. Macneil et al., *dbmate: a lightweight, framework-agnostic database migration tool*, github.com/amacneil/dbmate.

Chapter 9 · Caching

1. P. Mockapetris, *Domain Names — Concepts and Facilities*, RFC 1034, IETF, 1987.
2. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Caching*, RFC 9111, IETF, 2022.
3. M. Nottingham and R. Fielding, *Additional HTTP Status Codes*, RFC 6585, IETF, 2012.
4. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2017.
5. E. Nygren, R. K. Sitaraman and J. Sun, *The Akamai Network: A Platform for High-Performance Internet Applications*, ACM SIGOPS Operating Systems Review, 44(3), 2010.
6. R. Nishtala et al., *Scaling Memcache at Facebook*, 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2013.
7. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
8. Redis Ltd., *Redis Documentation* (key eviction, persistence), redis.io.

Chapter 10 · Task Queues & Background Jobs

1. G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*, Addison-Wesley, 2003.
2. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
3. Celery Project, *Celery: Distributed Task Queue*, official documentation, docs.celeryq.dev.
4. Taskforce.sh, *BullMQ Documentation*, docs.bullmq.io.
5. Amazon Web Services, *Amazon Simple Queue Service Developer Guide*, AWS documentation.
6. RabbitMQ, *Consumer Acknowledgements and Publisher Confirms*, RabbitMQ documentation.
7. Redis, *Redis Pub/Sub*, Redis documentation.
8. M. Brooker, *Exponential Backoff and Jitter*, AWS Architecture Blog, 2015.

Chapter 11 · Full-Text Search with Elasticsearch

1. C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.
2. S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
3. J. Zobel and A. Moffat, "Inverted Files for Text Search Engines," *ACM Computing Surveys*, vol. 38, no. 2, 2006.
4. M. McCandless, E. Hatcher and O. Gospodnetić, *Lucene in Action*, 2nd ed., Manning Publications, 2010.
5. C. Gormley and Z. Tong, *Elasticsearch: The Definitive Guide*, O'Reilly Media, 2015.
6. Elastic, *Elasticsearch Reference* (mapping, Query DSL, similarity), official documentation, elastic.co.
7. The PostgreSQL Global Development Group, *PostgreSQL Documentation*, Chapter 12: Full Text Search, and Appendix F: pg_trgm, postgresql.org.

Chapter 12 · Error Handling & Fault Tolerance

1. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
2. M. Nottingham and R. Fielding, *Additional HTTP Status Codes*, RFC 6585, IETF, 2012.
3. M. Nottingham, E. Wilde and S. Dalal, *Problem Details for HTTP APIs*, RFC 9457, IETF, 2023.
4. M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed., Pragmatic Bookshelf, 2018.
5. B. Beyer, C. Jones, J. Petoff and N. R. Murphy (Eds.), *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, 2016.
6. M. Brooker, *Exponential Backoff and Jitter*, AWS Architecture Blog, 2015.
7. OWASP Foundation, *Authentication Cheat Sheet* and *Logging Cheat Sheet*, OWASP Cheat Sheet Series.
8. The PostgreSQL Global Development Group, *PostgreSQL Documentation*, Appendix A: PostgreSQL Error Codes.

Chapter 13 · Configuration Management

1. A. Wiggins, *The Twelve-Factor App*, https://12factor.net, 2011.
2. J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
3. P. Hodgson, *Feature Toggles (aka Feature Flags)*, martinfowler.com, 2017.
4. T. Bray (Ed.), *The JavaScript Object Notation (JSON) Data Interchange Format*, RFC 8259, IETF, 2017.
5. O. Ben-Kiki, C. Evans and I. döt Net, *YAML Ain't Markup Language (YAML) Version 1.2*, 2009.
6. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.

7. E. Barker, *Recommendation for Key Management: Part 1 – General*, NIST SP 800-57 Part 1 Rev. 5, NIST, 2020.
8. OWASP Foundation, *Secrets Management Cheat Sheet*, OWASP Cheat Sheet Series.

Chapter 14 · Logging, Monitoring & Observability

1. B. Beyer, C. Jones, J. Petoff and N. R. Murphy (Eds.),*Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, 2016.
2. C. Majors, L. Fong-Jones and G. Miranda, *Observability Engineering: Achieving Production Excellence*, O'Reilly Media, 2022.
3. B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan and C. Shanbhag, *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*, Google Technical Report dapper-2010-1, 2010.
4. OpenTelemetry Authors, *OpenTelemetry Specification*, Cloud Native Computing Foundation, https://opentelemetry.io/docs/specs/otel/.
5. W3C, *Trace Context*, W3C Recommendation, 2021.
6. R. Gerhards, *The Syslog Protocol*, RFC 5424, IETF, 2009.
7. A. Wiggins, *The Twelve-Factor App*, factor XI: Logs, https://12-factor.net, 2011.

Chapter 15 · Graceful Shutdown

1. IEEE and The Open Group, *The Open Group Base Specifications Issue 7* (POSIX.1), IEEE Std 1003.1-2017, 2018.
2. M. Kerrisk, *The Linux Programming Interface*, No Starch Press, 2010 (Chapters 20–22, Signals).
3. The Kubernetes Authors, *Pod Lifecycle* (Termination of Pods), Kubernetes Documentation, kubernetes.io.
4. The Go Authors, *Package net/http* (Server.Shutdown) and *Package os/signal* (NotifyContext), Go standard library documentation, pkg.go.dev.
5. A. Wiggins, *The Twelve-Factor App*, factor IX: Disposability, 12-factor.net, 2011.
6. I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455, IETF, 2011.
7. G. Candea and A. Fox, "Crash-Only Software", in *Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, USENIX, 2003.

Chapter 16 · Backend Security

1. M. Jones, J. Bradley and N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.
2. A. Barth, *HTTP State Management Mechanism*, RFC 6265, IETF, 2011.
3. A. Biryukov, D. Dinu, D. Khovratovich and S. Josefsson, *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*, RFC 9106, IETF, 2021.
4. N. Provos and D. Mazières, "A Future-Adaptable Password Scheme", *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*, 1999.
5. M. West (Ed.), *Content Security Policy Level 3*, W3C Working Draft.
6. OWASP Foundation, *OWASP Top 10:2021*, 2021.
7. OWASP Foundation, *OWASP API Security Top 10 – 2023*, 2023.
8. OWASP Foundation, *OWASP Cheat Sheet Series* (Authentication, Password Storage, Session Management), cheatsheetseries.owasp.org.

Chapter 17 · Scaling & Performance, Part 1

1. J. Dean and L. A. Barroso, "The Tail at Scale", *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.
2. B. Gregg, "The Flame Graph", *Communications of the ACM*, vol. 59, no. 6, 2016.
3. B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed., Addison-Wesley, 2020.
4. L. Kleinrock, *Queueing Systems, Volume 1: Theory*, Wiley, 1975.
5. B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan and C. Shanbhag, "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure", Google Technical Report dapper-2010-1, 2010.
6. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
7. The PostgreSQL Global Development Group, *PostgreSQL Documentation*: "Indexes" and "Using EXPLAIN", postgresql.org.
8. PgBouncer project, *PgBouncer Documentation*, pgbouncer.org.

Chapter 18 · Scaling & Performance, Part 2

1. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
2. B. Beyer, C. Jones, J. Petoff and N. R. Murphy (Eds.),*Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, 2016.
3. S. Newman, *Building Microservices*, 2nd ed., O'Reilly Media, 2021.
4. A. Agache et al., "Firecracker: Lightweight Virtualization for Serverless Applications," in *Proc. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020.
5. E. Nygren, R. K. Sitaraman and J. Sun, "The Akamai Network: A Platform for High-Performance Internet Applications," *ACM SIGOPS Operating Systems Review*, 44(3), 2010.
6. E. Jonas et al., *Cloud Programming Simplified: A Berkeley View on Serverless Computing*, Technical Report UCB/EECS-2019-3, University of California, Berkeley, 2019.
7. D. E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys*, 6(4), 1974.
8. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Caching*, RFC 9111, IETF, 2022.

Chapter 19 · Concurrency & Parallelism

1. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books, 2018.
2. M. Kerrisk, *The Linux Programming Interface*, No Starch Press, 2010.
3. R. Pike, *Concurrency Is Not Parallelism*, talk at Heroku Waza, San Francisco, 2012.
4. D. Kegel, *The C10K Problem*, kegel.com/c10k.html, 1999.
5. The Go Authors, *Effective Go*, go.dev/doc/effective_go.
6. OpenJS Foundation, *Don't Block the Event Loop (or the Worker Pool)*, Node.js documentation, nodejs.org.
7. Ecma International, *ECMAScript 2017 Language Specification*, ECMA-262, 8th edition, 2017.
8. R. Pressler and A. Bateman, *JEP 444: Virtual Threads*, OpenJDK, 2023.

Chapter 20 · Object Storage, Part 1

1. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
2. L. Masinter, *Returning Values from Forms: multipart/form-data*, RFC 7578, IETF, 2015.
3. H. Krawczyk, M. Bellare and R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, IETF, 1997.
4. I. S. Reed and G. Solomon, "Polynomial Codes over Certain Finite Fields", *Journal of the Society for Industrial and Applied Mathematics*, 8(2), pp. 300–304, 1960.

5. C. Huang, H. Simitci, Y. Xu, A. Ogus, B. Calder, P. Gopalan, J. Li and S. Yekhanin, "Erasure Coding in Windows Azure Storage", *USENIX Annual Technical Conference*, 2012.
6. Amazon Web Services, *Amazon Simple Storage Service User Guide*, AWS documentation.
7. IEEE and The Open Group, *IEEE Std 1003.1-2017 (POSIX.1-2017): The Open Group Base Specifications Issue 7*, 2018.
8. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.

Chapter 21 · Object Storage, Part 2

1. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
2. M. Allman, V. Paxson and E. Blanton, *TCP Congestion Control*, RFC 5681, IETF, 2009.
3. M. Mathis, J. Semke, J. Mahdavi and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM Computer Communication Review*, 27(3), 1997.
4. R. Pantos (Ed.) and W. May, *HTTP Live Streaming*, RFC 8216, IETF, 2017.
5. ISO/IEC 23009-1:2022, *Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats*, ISO, 2022.
6. Amazon Web Services, *Amazon Simple Storage Service User Guide* (multipart upload, lifecycle configuration, checking object integrity, CORS), AWS documentation.
7. Amazon Web Services, *Amazon CloudFront Developer Guide* (serving private content with signed URLs and signed cookies), AWS documentation.

Chapter 22 · Real-Time Backends

1. I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455, IETF, 2011.
2. S. Loreto, P. Saint-Andre, S. Salsano and G. Wilkins, *Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP*, RFC 6202, IETF, 2011.
3. WHATWG, *HTML Living Standard*, section "Server-sent events", https://html.spec.whatwg.org/.
4. L.-S. Huang, E. Y. Chen, A. Barth, E. Rescorla and C. Jackson, "Talking to Yourself for Fun and Profit", *Web 2.0 Security and Privacy Workshop (W2SP)*, 2011.
5. W. Eddy (Ed.), *Transmission Control Protocol (TCP)*, RFC 9293, IETF, 2022.
6. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
7. Redis documentation, *Redis Pub/Sub*, https://redis.io/docs/.
8. D. Kegel, *The C10K Problem*, http://www.kegel.com/c10k.html.

Chapter 23 · Testing for Backend Engineers

1. K. Beck, *Test-Driven Development: By Example*, Addison-Wesley, 2003.
2. G. Meszaros, *xUnit Test Patterns: Refactoring Test Code*, Addison-Wesley, 2007.
3. S. Freeman and N. Pryce, *Growing Object-Oriented Software, Guided by Tests*, Addison-Wesley, 2009.
4. T. Winters, T. Manshreck and H. Wright (Eds.), *Software Engineering at Google: Lessons Learned from Programming Over Time*, O'Reilly, 2020.
5. Q. Luo, F. Hariri, L. Eloussi and D. Marinov, "An Empirical Analysis of Flaky Tests", in *Proc. 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2014.
6. L. Inozemtseva and R. Holmes, "Coverage Is Not Strongly Correlated with Test Suite Effectiveness", in *Proc. 36th International Conference on Software Engineering (ICSE)*, 2014.
7. T. J. McCabe, "A Complexity Measure", *IEEE Transactions on Software Engineering*, SE-2(4), pp. 308–320, 1976.
8. G. A. Campbell, "Cognitive Complexity: An Overview and Evaluation", in *Proc. 2018 International Conference on Technical Debt (TechDebt)*, ACM, 2018.

Chapter 24 · The Twelve-Factor App

1. A. Wiggins, *The Twelve-Factor App*, 12factor.net, 2011.
2. K. Hoffman, *Beyond the Twelve-Factor App*, O'Reilly Media, 2016.
3. G. Candea and A. Fox, "Crash-Only Software," in *Proc. 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, USENIX, 2003.
4. J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
5. B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, 2016.
6. The PostgreSQL Global Development Group, *PostgreSQL Documentation*, Section 9.7: Pattern Matching.

Chapter 25 · OpenAPI

1. OpenAPI Initiative, *OpenAPI Specification*, Version 3.1.1, The Linux Foundation, 2024.
2. A. Wright, H. Andrews, B. Hutton and G. Dennis, *JSON Schema: A Media Type for Describing JSON Documents* (draft 2020-12), Internet-Draft draft-bhutton-json-schema-00, IETF, 2020.
3. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
4. T. Bray (Ed.), *The JavaScript Object Notation (JSON) Data Interchange Format*, RFC 8259, IETF, 2017.
5. O. Ben-Kiki, C. Evans and I. döt Net, *YAML Ain't Markup Language (YAML) Version 1.2*, yaml.org, 2009.
6. A. Lauret, *The Design of Web APIs*, Manning, 2019.

Chapter 26 · Webhooks

1. H. Krawczyk, M. Bellare and R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, IETF, 1997.
2. R. Fielding, M. Nottingham and J. Reschke (Eds.), *HTTP Semantics*, RFC 9110, IETF, 2022.
3. Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot and E. Lear, *Address Allocation for Private Internets*, RFC 1918, IETF, 1996.
4. J. Genestoux and A. Parecki (Eds.), *WebSub*, W3C Recommendation, 2018.
5. Standard Webhooks, *Standard Webhooks Specification*, standardwebhooks.com, 2023.
6. GitHub, *Validating webhook deliveries*, GitHub Docs.
7. OWASP, *Server-Side Request Forgery Prevention Cheat Sheet*, OWASP Cheat Sheet Series.
8. M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.

Chapter 27 · DevOps for Backend Engineers

1. N. Forsgren, J. Humble and G. Kim, *Accelerate: The Science of Lean Software and DevOps*, IT Revolution Press, 2018.
2. J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
3. DORA (Google Cloud), *Accelerate State of DevOps Report 2024*, 2024.

4. B. Beyer, C. Jones, J. Petoff and N. R. Murphy (Eds.),*Site Reliability Engineering*: *How Google Runs Production Systems*, O'Reilly Media, 2016.
5. B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, "Borg, Omega, and Kubernetes", *Communications of the ACM*, 59(5), 2016.
6. Open Container Initiative, *OCI Image Format Specification*, v1.1, The Linux Foundation, 2024.
7. R. Barnes, J. Hoffman-Andrews, D. McCarney and J. Kasten,*Automatic Certificate Management Environment (ACME)*, RFC 8555, IETF, 2019.
8. K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2nd ed., O'Reilly Media, 2020.